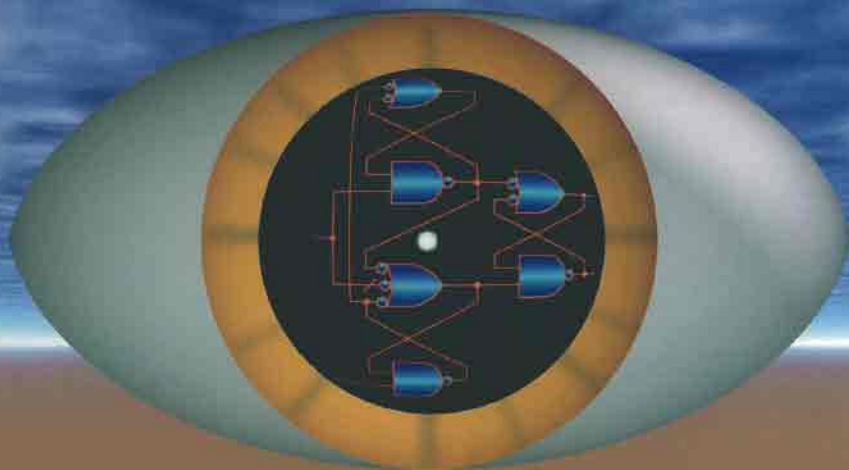


SECOND EDITION

REVISED  
EDITION

# ENGINEERING DIGITAL DESIGN



RICHARD F. TINDER



# **Engineering Digital Design**

*Second Edition, Revised*

*This page intentionally left blank*

---

# Engineering Digital Design

*Second Edition, Revised*

---

*by*

**RICHARD F. TINDER**

*School of Electrical Engineering and Computer Science  
Washington State University  
Pullman, Washington*



**ACADEMIC PRESS**

A Harcourt Science and Technology Company

SAN DIEGO/SAN FRANCISCO/NEW YORK/BOSTON/LONDON/SYDNEY/TOKYO



This book is printed on acid-free paper. 

Copyright ©2000, Elsevier Science (USA).

All Rights Reserved.

No part of this publication may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopy, recording, or any information storage and retrieval system, without permission in writing from the publisher.

Requests for permission to make copies of any part of the work should be mailed to:  
Permissions Department, Academic Press, 6277 Sea Harbor Drive,  
Orlando, Florida 32887-6777

Academic Press

*An imprint of Elsevier Science*

525 B Street, Suite 1900, San Diego, California 92101-4495, USA

<http://www.academicpress.com>

Academic Press

84 Theobalds Road, London WC1X 8RR, UK

<http://www.academicpress.com>

Library of Congress Catalog Card Number: 99-066780

International Standard Book Number: 0-12-691295-5

PRINTED IN THE UNITED STATES OF AMERICA

02 03 04 05 06 07 MV 9 8 7 6 5 4 3 2

***Disclaimer:***

***This eBook does not include the ancillary media that was packaged with the original printed version of the book.***

***This book is lovingly dedicated to my partner in life, Gloria***

*This page intentionally left blank*

*For the sake of persons of different types, scientific truth should be presented in different forms, and should be regarded as equally scientific, whether it appears in the robust form and the vivid coloring of a physical illustration, or in the tenuity and paleness of a symbolic expression.*

**James Clerk Maxwell**

**Address to the Mathematics and Physical Section,  
British Association of Sciences, 1870**

*This page intentionally left blank*

---

# Contents

---

|                                                           |            |
|-----------------------------------------------------------|------------|
| <b>Preface</b>                                            | <b>xix</b> |
| <b>1. Introductory Remarks and Glossary</b>               | <b>1</b>   |
| 1.1 What Is So Special about Digital Systems?             | 1          |
| 1.2 The Year 2000 and Beyond?                             | 3          |
| 1.3 A Word of Warning                                     | 5          |
| 1.4 Glossary of Terms, Expressions, and Abbreviations     | 5          |
| <b>2. Number Systems, Binary Arithmetic, and Codes</b>    | <b>31</b>  |
| 2.1 Introduction                                          | 31         |
| 2.2 Positional and Polynomial Representations             | 32         |
| 2.3 Unsigned Binary Number System                         | 33         |
| 2.4 Unsigned Binary Coded Decimal, Hexadecimal, and Octal | 34         |
| 2.4.1 The BCD Representation                              | 34         |
| 2.4.2 The Hexadecimal and Octal Systems                   | 36         |
| 2.5 Conversion between Number Systems                     | 37         |
| 2.5.1 Conversion of Integers                              | 38         |
| 2.5.2 Conversion of Fractions                             | 40         |
| 2.6 Signed Binary Numbers                                 | 43         |
| 2.6.1 Signed-Magnitude Representation                     | 44         |
| 2.6.2 Radix Complement Representation                     | 45         |
| 2.6.3 Diminished Radix Complement Representation          | 48         |
| 2.7 Excess (Offset) Representations                       | 49         |
| 2.8 Floating-Point Number Systems                         | 49         |
| 2.9 Binary Arithmetic                                     | 52         |
| 2.9.1 Direct Addition and Subtraction of Binary Numbers   | 52         |
| 2.9.2 Two's Complement Subtraction                        | 53         |
| 2.9.3 One's Complement Subtraction                        | 54         |
| 2.9.4 Binary Multiplication                               | 55         |
| 2.9.5 Binary Division                                     | 58         |
| 2.9.6 BCD Addition and Subtraction                        | 62         |
| 2.9.7 Floating-Point Arithmetic                           | 64         |
| 2.9.8 Perspective on Arithmetic Codes                     | 67         |
| 2.10 Other Codes                                          | 68         |
| 2.10.1 The Decimal Codes                                  | 68         |
| 2.10.2 Error Detection Codes                              | 69         |
| 2.10.3 Unit Distance Codes                                | 70         |
| 2.10.4 Character Codes                                    | 70         |
| Further Reading                                           | 72         |
| Problems                                                  | 72         |
|                                                           | <b>ix</b>  |

|           |                                                                                          |            |
|-----------|------------------------------------------------------------------------------------------|------------|
| <b>3.</b> | <b>Background for Digital Design</b>                                                     | <b>79</b>  |
| 3.1       | Introduction                                                                             | 79         |
| 3.2       | Binary State Terminology and Mixed Logic Notation                                        | 79         |
| 3.2.1     | Binary State Terminology                                                                 | 79         |
| 3.3       | Introduction to CMOS Terminology and Symbology                                           | 82         |
| 3.4       | Logic Level Conversion: The Inverter                                                     | 83         |
| 3.5       | Transmission Gates and Tri-State Drivers                                                 | 84         |
| 3.6       | AND and OR Operators and Their Mixed-Logic Circuit Symbology                             | 87         |
| 3.6.1     | Logic Circuit Symbology for AND and OR                                                   | 87         |
| 3.6.2     | NAND Gate Realization of Logic AND and OR                                                | 88         |
| 3.6.3     | NOR Gate Realization of Logic AND and OR                                                 | 89         |
| 3.6.4     | NAND and NOR Gate Realization of Logic Level Conversion                                  | 90         |
| 3.6.5     | The AND and OR Gates and Their Realization of Logic AND and OR                           | 92         |
| 3.6.6     | Summary of Logic Circuit Symbols for the AND and OR Functions and Logic Level Conversion | 94         |
| 3.7       | Logic Level Incompatibility: Complementation                                             | 95         |
| 3.8       | Reading and Construction of Mixed-Logic Circuits                                         | 97         |
| 3.9       | XOR and EQV Operators and Their Mixed-Logic Circuit Symbology                            | 98         |
| 3.9.1     | The XOR and EQV Functions of the XOR Gate                                                | 100        |
| 3.9.2     | The XOR and EQV Functions of the EQV Gate                                                | 100        |
| 3.9.3     | Multiple Gate Realizations of the XOR and EQV Functions                                  | 101        |
| 3.9.4     | The Effect of Active Low Inputs to the XOR and EQV Circuit Symbols                       | 102        |
| 3.9.5     | Summary of Conjugate Logic Circuit Symbols for XOR and EQV Gates                         | 103        |
| 3.9.6     | Controlled Logic Level Conversion                                                        | 103        |
| 3.9.7     | Construction and Waveform Analysis of Logic Circuits Containing XOR-Type Functions       | 104        |
| 3.10      | Laws of Boolean Algebra                                                                  | 105        |
| 3.10.1    | NOT, AND, and OR Laws                                                                    | 106        |
| 3.10.2    | The Concept of Duality                                                                   | 107        |
| 3.10.3    | Associative, Commutative, Distributive, Absorptive, and Consensus Laws                   | 108        |
| 3.10.4    | DeMorgan's Laws                                                                          | 110        |
| 3.11      | Laws of XOR Algebra                                                                      | 111        |
| 3.11.1    | Two Useful Corollaries                                                                   | 114        |
| 3.11.2    | Summary of Useful Identities                                                             | 115        |
| 3.12      | Worked Examples                                                                          | 116        |
|           | Further Reading                                                                          | 120        |
|           | Problems                                                                                 | 121        |
| <b>4.</b> | <b>Logic Function Representation and Minimization</b>                                    | <b>131</b> |
| 4.1       | Introduction                                                                             | 131        |
| 4.2       | SOP and POS Forms                                                                        | 131        |
| 4.2.1     | The SOP Representation                                                                   | 131        |
| 4.2.2     | The POS Representation                                                                   | 134        |
| 4.3       | Introduction to Logic Function Graphics                                                  | 137        |
| 4.3.1     | First-Order K-maps                                                                       | 138        |
| 4.3.2     | Second-Order K-maps                                                                      | 138        |
| 4.3.3     | Third-Order K-maps                                                                       | 140        |
| 4.3.4     | Fourth-Order K-maps                                                                      | 143        |

|           |                                                                                               |            |
|-----------|-----------------------------------------------------------------------------------------------|------------|
| 4.4       | Karnaugh Map Function Minimization                                                            | 144        |
| 4.4.1     | Examples of Function Minimization                                                             | 146        |
| 4.4.2     | Prime Implicants                                                                              | 148        |
| 4.4.3     | Incompletely Specified Functions: Don't Cares                                                 | 150        |
| 4.5       | Multiple Output Optimization                                                                  | 152        |
| 4.6       | Entered Variable K-map Minimization                                                           | 158        |
| 4.6.1     | Incompletely Specified Functions                                                              | 162        |
| 4.7       | Function Reduction of Five or More Variables                                                  | 165        |
| 4.8       | Minimization Algorithms and Application                                                       | 169        |
| 4.8.1     | The Quine–McCluskey Algorithm                                                                 | 169        |
| 4.8.2     | Cube Representation and Function Reduction                                                    | 173        |
| 4.8.3     | Qualitative Description of the Espresso Algorithm                                             | 173        |
| 4.9       | Factorization, Resubstitution, and Decomposition Methods                                      | 174        |
| 4.9.1     | Factorization                                                                                 | 175        |
| 4.9.2     | Resubstitution Method                                                                         | 176        |
| 4.9.3     | Decomposition by Using Shannon's Expansion Theorem                                            | 177        |
| 4.10      | Design Area vs Performance                                                                    | 180        |
| 4.11      | Perspective on Logic Minimization and Optimization                                            | 181        |
| 4.12      | Worked EV K-map Examples                                                                      | 181        |
|           | Further Reading                                                                               | 188        |
|           | Problems                                                                                      | 189        |
| <b>5.</b> | <b>Function Minimization by Using K-map XOR Patterns and Reed–Muller Transformation Forms</b> | <b>197</b> |
| 5.1       | Introduction                                                                                  | 197        |
| 5.2       | XOR-Type Patterns and Extraction of Gate-Minimum Cover from EV K-maps                         | 198        |
| 5.2.1     | Extraction Procedure and Examples                                                             | 200        |
| 5.3       | Algebraic Verification of Optimal XOR Function Extraction from K-maps                         | 204        |
| 5.4       | K-map Plotting and Entered Variable XOR Patterns                                              | 205        |
| 5.5       | The SOP-to-EXSOP Reed–Muller Transformation                                                   | 207        |
| 5.6       | The POS-to-EQPOS Reed–Muller Transformation                                                   | 208        |
| 5.7       | Examples of Minimum Function Extraction                                                       | 209        |
| 5.8       | Heuristics for CRMT Minimization                                                              | 217        |
| 5.9       | Incompletely Specified Functions                                                              | 218        |
| 5.10      | Multiple Output Functions with Don't Cares                                                    | 222        |
| 5.11      | K-map Subfunction Partitioning for Combined CRMT and Two-Level Minimization                   | 225        |
| 5.12      | Perspective on the CRMT and CRMT/Two-Level Minimization Methods                               | 229        |
|           | Further Reading                                                                               | 229        |
|           | Problems                                                                                      | 230        |
| <b>6.</b> | <b>Nonarithmetic Combinational Logic Devices</b>                                              | <b>237</b> |
| 6.1       | Introduction and Background                                                                   | 237        |
| 6.1.1     | The Building Blocks                                                                           | 237        |
| 6.1.2     | Classification of Chips                                                                       | 238        |
| 6.1.3     | Performance Characteristics and Other Practical Matters                                       | 238        |
| 6.1.4     | Part Numbering Systems                                                                        | 241        |
| 6.1.5     | Design Procedure                                                                              | 241        |



|           |                                                                    |            |
|-----------|--------------------------------------------------------------------|------------|
| 6.2       | Multiplexers                                                       | 242        |
| 6.2.1     | Multiplexer Design                                                 | 242        |
| 6.2.2     | Combinational Logic Design with MUXs                               | 245        |
| 6.3       | Decoders/Demultiplexers                                            | 248        |
| 6.3.1     | Decoder Design                                                     | 248        |
| 6.3.2     | Combinational Logic Design with Decoders                           | 251        |
| 6.4       | Encoders                                                           | 254        |
| 6.5       | Code Converters                                                    | 257        |
| 6.5.1     | Procedure for Code Converter Design                                | 257        |
| 6.5.2     | Examples of Code Converter Design                                  | 257        |
| 6.6       | Magnitude Comparators                                              | 265        |
| 6.7       | Parity Generators and Error Checking Systems                       | 273        |
| 6.8       | Combinational Shifters                                             | 275        |
| 6.9       | Steering Logic and Tri-State Gate Applications                     | 278        |
| 6.10      | Introduction to VHDL Description of Combinational Primitives       | 279        |
|           | Further Reading                                                    | 287        |
|           | Problems                                                           | 288        |
| <b>7.</b> | <b>Programmable Logic Devices</b>                                  | <b>295</b> |
| 7.1       | Introduction                                                       | 295        |
| 7.2       | Read-Only Memories                                                 | 295        |
| 7.2.1     | PROM Applications                                                  | 299        |
| 7.3       | Programmable Logic Arrays                                          | 301        |
| 7.3.1     | PLA Applications                                                   | 302        |
| 7.4       | Programmable Array Logic Devices                                   | 307        |
| 7.5       | Mixed-Logic Inputs to and Outputs from ROMs, PLAs, and PAL Devices | 310        |
| 7.6       | Multiple PLD Schemes for Augmenting Input and Output Capability    | 312        |
| 7.7       | Introduction to FPGAs and Other General-Purpose Devices            | 317        |
| 7.7.1     | AND–OR–Invert and OR–AND–Invert Building Blocks                    | 317        |
| 7.7.2     | Actel Field Programmable Gate Arrays                               | 319        |
| 7.7.3     | Xilinx FPGAs                                                       | 321        |
| 7.7.4     | Other Classes of General-Purpose PLDs                              | 328        |
| 7.8       | CAD Help in Programming PLD Devices                                | 328        |
|           | Further Reading                                                    | 330        |
|           | Problems                                                           | 331        |
| <b>8.</b> | <b>Arithmetic Devices and Arithmetic Logic Units (ALUs)</b>        | <b>335</b> |
| 8.1       | Introduction                                                       | 335        |
| 8.2       | Binary Adders                                                      | 335        |
| 8.2.1     | The Half Adder                                                     | 336        |
| 8.2.2     | The Full Adder                                                     | 337        |
| 8.2.3     | Ripple-Carry Adders                                                | 338        |
| 8.3       | Binary Subtractors                                                 | 340        |
| 8.3.1     | Adder/Subtractors                                                  | 342        |
| 8.3.2     | Sign-Bit Error Detection                                           | 343        |
| 8.4       | The Carry Look-Ahead Adder                                         | 345        |
| 8.5       | Multiple-Number Addition and the Carry-Save Adder                  | 349        |
| 8.6       | Multipliers                                                        | 350        |
| 8.7       | Parallel Dividers                                                  | 353        |

|            |                                                                                                                |            |
|------------|----------------------------------------------------------------------------------------------------------------|------------|
| 8.8        | Arithmetic and Logic Units                                                                                     | 357        |
| 8.8.1      | Dedicated ALU Design Featuring R-C and CLA Capability                                                          | 358        |
| 8.8.2      | The MUX Approach to ALU Design                                                                                 | 363        |
| 8.9        | Dual-Rail Systems and ALUs with Completion Signals                                                             | 369        |
| 8.9.1      | Carry Look-Ahead Configuration                                                                                 | 378        |
| 8.10       | VHDL Description of Arithmetic Devices                                                                         | 380        |
|            | Further Reading                                                                                                | 383        |
|            | Problems                                                                                                       | 385        |
| <b>9.</b>  | <b>Propagation Delay and Timing Defects in Combinational Logic</b>                                             | <b>391</b> |
| 9.1        | Introduction                                                                                                   | 391        |
| 9.2        | Static Hazards in Two-Level Combinational Logic Circuits                                                       | 392        |
| 9.3        | Detection and Elimination Hazards in Multilevel XOR-Type Functions                                             | 399        |
| 9.3.1      | XOP and EOS Functions                                                                                          | 400        |
| 9.3.2      | Methods for the Detection and Elimination of Static Hazards in Complex Multilevel XOR-type Functions           | 403        |
| 9.3.3      | General Procedure for the Detection and Elimination of Static Hazards in Complex Multilevel XOR-Type Functions | 408        |
| 9.3.4      | Detection of Dynamic Hazards in Complex Multilevel XOR-Type Functions                                          | 409        |
| 9.4        | Function Hazards                                                                                               | 412        |
| 9.5        | Stuck-at Faults and the Effect of Hazard Cover on Fault Testability                                            | 412        |
|            | Further Reading                                                                                                | 413        |
|            | Problems                                                                                                       | 415        |
| <b>10.</b> | <b>Introduction to Synchronous State Machine Design and Analysis</b>                                           | <b>419</b> |
| 10.1       | Introduction                                                                                                   | 419        |
| 10.1.1     | A Sequence of Logic States                                                                                     | 420        |
| 10.2       | Models for Sequential Machines                                                                                 | 421        |
| 10.3       | The Fully Documented State Diagram: The Sum Rule                                                               | 424        |
| 10.4       | The Basic Memory Cells                                                                                         | 428        |
| 10.4.1     | The Set-Dominant Basic Cell                                                                                    | 428        |
| 10.4.2     | The Reset-Dominant Basic Cell                                                                                  | 431        |
| 10.4.3     | Combined Form of the Excitation Table                                                                          | 433        |
| 10.4.4     | Mixed-Rail Outputs of the Basic Cells                                                                          | 434        |
| 10.4.5     | Mixed-Rail Output Response of the Basic Cells                                                                  | 435        |
| 10.5       | Introduction to Flip-Flops                                                                                     | 436        |
| 10.5.1     | Triggering Mechanisms                                                                                          | 437        |
| 10.5.2     | Types of Flip-Flops                                                                                            | 438        |
| 10.5.3     | Hierarchical Flow Chart and Model for Flip-Flop Design                                                         | 438        |
| 10.6       | Procedure for FSM (Flip-Flop) Design and the Mapping Algorithm                                                 | 440        |
| 10.7       | The D Flip-Flops: General                                                                                      | 440        |
| 10.7.1     | The D-Latch                                                                                                    | 441        |
| 10.7.2     | The RET D Flip-Flop                                                                                            | 444        |
| 10.7.3     | The Master-Slave D Flip-Flop                                                                                   | 448        |
| 10.8       | Flip-Flop Conversion: The T, JK Flip-Flops and Miscellaneous Flip-Flops                                        | 450        |
| 10.8.1     | The T Flip-Flops and Their Design from D Flip-Flops                                                            | 451        |
| 10.8.2     | The JK Flip-Flops and Their Design from D Flip-Flops                                                           | 453        |
| 10.8.3     | Design of T and D Flip-Flops from JK Flip-Flops                                                                | 455        |

|            |                                                                                            |            |
|------------|--------------------------------------------------------------------------------------------|------------|
| 10.8.4     | Review of Excitation Tables                                                                | 457        |
| 10.8.5     | Design of Special-Purpose Flip-Flops and Latches                                           | 459        |
| 10.9       | Latches and Flip-Flops with Serious Timing Problems: A Warning                             | 461        |
| 10.10      | Asynchronous Preset and Clear Overrides                                                    | 463        |
| 10.11      | Setup and Hold-Time Requirements of Flip-Flops                                             | 465        |
| 10.12      | Design of Simple Synchronous State Machines with Edge-Triggered Flip-Flops: Map Conversion | 466        |
| 10.12.1    | Design of a Three-Bit Binary Up/Down Counter: D-to-T K-map Conversion                      | 466        |
| 10.12.2    | Design of a Sequence Recognizer: D-to-JK K-map Conversion                                  | 471        |
| 10.13      | Analysis of Simple State Machines                                                          | 476        |
| 10.14      | VHDL Description of Simple State Machines                                                  | 480        |
| 10.14.1    | The VHDL Behavioral Description of the RET D Flip-flop                                     | 480        |
| 10.14.2    | The VHDL Behavioral Description of a Simple FSM                                            | 481        |
|            | Further Reading                                                                            | 482        |
|            | Problems                                                                                   | 483        |
| <b>11.</b> | <b>Synchronous FSM Design Considerations and Applications</b>                              | <b>491</b> |
| 11.1       | Introduction                                                                               | 491        |
| 11.2       | Detection and Elimination of Output Race Glitches                                          | 491        |
| 11.2.1     | ORG Analysis Procedure Involving Two Race Paths                                            | 496        |
| 11.2.2     | Elimination of ORGs                                                                        | 496        |
| 11.3       | Detection and Elimination of Static Hazards in the Output Logic                            | 499        |
| 11.3.1     | Externally Initiated Static Hazards in the Output Logic                                    | 500        |
| 11.3.2     | Internally Initiated Static Hazards in the Output of Mealy and Moore FSMs                  | 502        |
| 11.3.3     | Perspective on Static Hazards in the Output Logic of FSMs                                  | 509        |
| 11.4       | Asynchronous Inputs: Rules and Caveats                                                     | 510        |
| 11.4.1     | Rules Associated with Asynchronous Inputs                                                  | 510        |
| 11.4.2     | Synchronizing the Input                                                                    | 511        |
| 11.4.3     | Stretching and Synchronizing the Input                                                     | 512        |
| 11.4.4     | Metastability and the Synchronizer                                                         | 514        |
| 11.5       | Clock Skew                                                                                 | 517        |
| 11.6       | Clock Sources and Clock Signal Specifications                                              | 520        |
| 11.6.1     | Clock-Generating Circuitry                                                                 | 520        |
| 11.6.2     | Clock Signal Specifications                                                                | 521        |
| 11.6.3     | Buffering and Gating the Clock                                                             | 522        |
| 11.7       | Initialization and Reset of the FSM: Sanity Circuits                                       | 522        |
| 11.7.1     | Sanity Circuits                                                                            | 523        |
| 11.8       | Switch Debouncing Circuits                                                                 | 526        |
| 11.8.1     | The Single-Pole/Single-Throw Switch                                                        | 526        |
| 11.8.2     | The Single-Pole/Double-Throw Switch                                                        | 528        |
| 11.8.3     | The Rotary Selector Switch                                                                 | 529        |
| 11.9       | Applications to the Design of More Complex State Machines                                  | 530        |
| 11.9.1     | Design Procedure                                                                           | 530        |
| 11.9.2     | Design Example: The One- to Three-Pulse Generator                                          | 532        |
| 11.10      | Algorithmic State Machine Charts and State Tables                                          | 536        |
| 11.10.1    | ASM Charts                                                                                 | 537        |
| 11.10.2    | State Tables and State Assignment Rules                                                    | 539        |
| 11.11      | Array Algebraic Approach to Logic Design                                                   | 542        |

|            |                                                                                        |            |
|------------|----------------------------------------------------------------------------------------|------------|
| 11.12      | State Minimization                                                                     | 547        |
|            | Further Reading                                                                        | 549        |
|            | Problems                                                                               | 551        |
| <b>12.</b> | <b>Module and Bit-Slice Devices</b>                                                    | <b>561</b> |
| 12.1       | Introduction                                                                           | 561        |
| 12.2       | Registers                                                                              | 561        |
| 12.2.1     | The Storage (Holding) Register                                                         | 562        |
| 12.2.2     | The Right Shift Register with Synchronous Parallel Load                                | 562        |
| 12.2.3     | Universal Shift Registers with Synchronous Parallel Load                               | 565        |
| 12.2.4     | Universal Shift Registers with Asynchronous Parallel Load                              | 568        |
| 12.2.5     | Branching Action of a 4-Bit USR                                                        | 570        |
| 12.3       | Synchronous Binary Counters                                                            | 572        |
| 12.3.1     | Simple Divide-by- $N$ Binary Counters                                                  | 573        |
| 12.3.2     | Cascadable BCD Up-Counters                                                             | 575        |
| 12.3.3     | Cascadable Up/Down Binary Counters with Asynchronous Parallel Load                     | 579        |
| 12.3.4     | Binary Up/Down Counters with Synchronous Parallel Load and True Hold Capability        | 581        |
| 12.3.5     | One-Bit Modular Design of Parallel Loadable Up/Down Counters with True Hold            | 584        |
| 12.3.6     | Perspective on Parallel Loading of Counters and Registers: Asynchronous vs Synchronous | 588        |
| 12.3.7     | Branching Action of a 4-Bit Parallel Loadable Up/Down Counter                          | 589        |
| 12.4       | Shift-Register Counters                                                                | 590        |
| 12.4.1     | Ring Counters                                                                          | 590        |
| 12.4.2     | Twisted Ring Counters                                                                  | 593        |
| 12.4.3     | Linear Feedback Shift Register Counters                                                | 594        |
| 12.5       | Asynchronous (Ripple) Counters                                                         | 600        |
|            | Further Reading                                                                        | 605        |
|            | Problems                                                                               | 606        |
| <b>13.</b> | <b>Alternative Synchronous FSM Architectures and Systems-Level Design</b>              | <b>613</b> |
| 13.1       | Introduction                                                                           | 613        |
| 13.1.1     | Choice of Components to be Considered                                                  | 613        |
| 13.2       | Architecture Centered around Nonregistered PLDs                                        | 614        |
| 13.2.1     | Design of the One- to Three-Pulse Generator by Using a PLA                             | 615        |
| 13.2.2     | Design of the One- to Three-Pulse Generator by Using a PAL                             | 617        |
| 13.2.3     | Design of the One- to Three-Pulse Generator by Using a ROM                             | 618        |
| 13.2.4     | Design of a More Complex FSM by Using a ROM as the PLD                                 | 622        |
| 13.3       | State Machine Designs Centered around a Shift Register                                 | 626        |
| 13.4       | State Machine Designs Centered around a Parallel Loadable Up/Down Counter              | 632        |
| 13.5       | The One-Hot Design Method                                                              | 636        |
| 13.5.1     | Use of ASMs in One-Hot Designs                                                         | 640        |
| 13.5.2     | Application of the One-Hot Method to a Serial 2's Complementer                         | 643        |
| 13.5.3     | One-Hot Design of a Parallel-to-Serial Adder/Subtractor Controller                     | 645        |
| 13.5.4     | Perspective on the Use of the One-Hot Method: Logic Noise and Use of Registered PLDs   | 647        |

|            |                                                                                         |            |
|------------|-----------------------------------------------------------------------------------------|------------|
| 13.6       | System-Level Design: Controller, Data Path, and Functional Partition                    | 649        |
| 13.6.1     | Design of a Parallel-to-Serial Adder/Subtractor Control System                          | 651        |
| 13.6.2     | Design of a Stepping Motor Control System                                               | 655        |
| 13.6.3     | Perspective on System-Level Design in This Text                                         | 666        |
| 13.7       | Dealing with Unusually Large Controller and System-Level Designs                        | 666        |
|            | Further Reading                                                                         | 668        |
|            | Problems                                                                                | 670        |
| <b>14.</b> | <b>Asynchronous State Machine Design and Analysis: Basic Concepts</b>                   | <b>683</b> |
| 14.1       | Introduction                                                                            | 683        |
| 14.1.1     | Features of Asynchronous FSMs                                                           | 684        |
| 14.1.2     | Need for Asynchronous FSMs                                                              | 685        |
| 14.2       | The Lumped Path Delay Models for Asynchronous FSMs                                      | 685        |
| 14.3       | Functional Relationships and the Stability Criteria                                     | 687        |
| 14.4       | The Excitation Table for the LPD Model                                                  | 688        |
| 14.5       | State Diagrams, K-maps, and State Tables for Asynchronous FSMs                          | 689        |
| 14.5.1     | The Fully Documented State Diagram                                                      | 689        |
| 14.5.2     | Next-State and Output K-maps                                                            | 690        |
| 14.5.3     | State Tables                                                                            | 691        |
| 14.6       | Design of the Basic Cells by Using the LPD Model                                        | 692        |
| 14.6.1     | The Set-Dominant Basic Cell                                                             | 692        |
| 14.6.2     | The Reset-Dominant Basic Cell                                                           | 694        |
| 14.7       | Design of the Rendezvous Modules by Using the Nested Cell Model                         | 695        |
| 14.8       | Design of the RET D Flip-Flop by Using the LPD Model                                    | 698        |
| 14.9       | Design of the RET JK Flip-Flop by Flip-Flop Conversion                                  | 700        |
| 14.10      | Detection and Elimination of Timing Defects in Asynchronous FSMs                        | 701        |
| 14.10.1    | Endless Cycles                                                                          | 702        |
| 14.10.2    | Races and Critical Races                                                                | 703        |
| 14.10.3    | Static Hazards in the NS and Output Functions                                           | 705        |
| 14.10.4    | Essential Hazards in Asynchronous FSMs                                                  | 711        |
| 14.10.5    | Perspective on Static Hazards and E-hazards in Asynchronous FSMs                        | 718        |
| 14.11      | Initialization and Reset of Asynchronous FSMs                                           | 719        |
| 14.12      | Single-Transition-Time Machines and the Array Algebraic Approach                        | 720        |
| 14.13      | Hazard-Free Design of Fundamental Mode State Machines by Using the Nested Cell Approach | 730        |
| 14.14      | One-Hot Design of Asynchronous State Machines                                           | 734        |
| 14.15      | Perspective on State Code Assignments of Fundamental Mode FSMs                          | 738        |
| 14.16      | Design of Fundamental Mode FSMs by Using PLDs                                           | 740        |
| 14.17      | Analysis of Fundamental Mode State Machines                                             | 741        |
|            | Further Reading                                                                         | 758        |
|            | Problems                                                                                | 759        |
| <b>15.</b> | <b>The Pulse Mode Approach to Asynchronous FSM Design</b>                               | <b>773</b> |
| 15.1       | Introduction                                                                            | 773        |
| 15.2       | Pulse Mode Models and System Requirements                                               | 773        |
| 15.2.1     | Choice of Memory Elements                                                               | 774        |
| 15.3       | Other Characteristics of Pulse Mode FSMs                                                | 777        |
| 15.4       | Design Examples                                                                         | 779        |
| 15.5       | Analysis of Pulse Mode FSMs                                                             | 788        |

|            |                                                                                                               |            |
|------------|---------------------------------------------------------------------------------------------------------------|------------|
| 15.6       | Perspective on the Pulse Mode Approach to FSM Design                                                          | 795        |
|            | Further Reading                                                                                               | 796        |
|            | Problems                                                                                                      | 797        |
| <b>16.</b> | <b>Externally Asynchronous/Internally Clocked (Pausable) Systems and Programmable Asynchronous Sequencers</b> | <b>805</b> |
| 16.1       | Introduction                                                                                                  | 805        |
| 16.2       | Externally Asynchronous/Internally Clocked Systems and Applications                                           | 806        |
| 16.2.1     | Static Logic DFLOP Design                                                                                     | 807        |
| 16.2.2     | Domino Logic DFLOP Design                                                                                     | 812        |
| 16.2.3     | Introduction to CMOS Dynamic Domino Logic                                                                     | 814        |
| 16.2.4     | EAIC System Design                                                                                            | 816        |
| 16.2.5     | System Simulations and Real-Time Tests                                                                        | 817        |
| 16.2.6     | Variations on the Theme                                                                                       | 820        |
| 16.2.7     | How EAIC FSMs Differ from Conventional Synchronous FSMs                                                       | 821        |
| 16.2.8     | Perspective on EAIC Systems as an Alternative Approach to FSM Design                                          | 822        |
| 16.3       | Asynchronous Programmable Sequencers                                                                          | 823        |
| 16.3.1     | Microprogrammable Asynchronous Controller Modules and System Architecture                                     | 823        |
| 16.3.2     | Architecture and Operation of the MAC Module                                                                  | 824        |
| 16.3.3     | Design of the MAC Module                                                                                      | 827        |
| 16.3.4     | MAC Module Design of a Simple FSM                                                                             | 830        |
| 16.3.5     | Cascading the MAC Module                                                                                      | 832        |
| 16.3.6     | Programming the MAC Module                                                                                    | 833        |
| 16.3.7     | Metastability and the MAC Module: The Final Issue                                                             | 834        |
| 16.3.8     | Perspective on MAC Module FSM Design                                                                          | 834        |
| 16.4       | One-Hot Programmable Asynchronous Sequencers                                                                  | 835        |
| 16.4.1     | Architecture for One-Hot Asynchronous Programmable Sequencers                                                 | 835        |
| 16.4.2     | Design of a Four-State Asynchronous One-Hot Sequencer                                                         | 837        |
| 16.4.3     | Design and Operation of a Simple FSM by Using a Four-State One-Hot Sequencer                                  | 838        |
| 16.4.4     | Perspective on Programmable Sequencer Design and Application                                                  | 839        |
| 16.5       | Epilogue to Chapter 16                                                                                        | 842        |
|            | Further Reading                                                                                               | 842        |
|            | Problems                                                                                                      | 844        |
| <b>A</b>   | <b>Other Transistor Logic Families</b>                                                                        | <b>849</b> |
| A.1        | Introduction to the Standard NMOS Logic Family                                                                | 849        |
| A.2        | Introduction to the TTL Logic Family                                                                          | 850        |
| A.3        | Performance Characteristics of Important IC Logic Families                                                    | 852        |
|            | Further Reading                                                                                               | 852        |
| <b>B</b>   | <b>Computer-Aided Engineering Tools</b>                                                                       | <b>855</b> |
| B.1        | Productivity Tools Bundled with this Text                                                                     | 855        |
| B.2        | Other Productivity Tools                                                                                      | 855        |
|            | Further Reading                                                                                               | 857        |

|              |                                     |            |
|--------------|-------------------------------------|------------|
| <b>C</b>     | <b>IEEE Standard Symbols</b>        | <b>859</b> |
| C.1          | Gates                               | 859        |
| C.2          | Combinational Logic Devices         | 859        |
| C.3          | Flip-Flops, Registers, and Counters | 860        |
|              | Further Reading                     | 862        |
| <b>Index</b> |                                     | <b>863</b> |

---

# Preface

---

## TEXT OVERVIEW

This text emphasizes the successful engineering design of digital devices and machines from first principles. A special effort has been made *not* to “throw” logic circuits at the reader so that questions remain as to how the circuits came about or whether or not they will function correctly. An understanding of the intricacies of digital circuit design, particularly in the area of sequential machines, is given the highest priority—the emphasis is on error-free operation. From an engineering point of view, the design of a digital device or machine is of little or no value unless it performs the intended operation(s) correctly and reliably.

Both the basics and background fundamentals are presented in this text. But it goes well beyond the basics to provide significant intermediate-to-advanced coverage of digital design material, some of which is covered by no other text. In fact, this text attempts to provide *course coverage at both the first and second levels*—an ambitious undertaking. The aim is to provide the reader with the tools necessary for the successful design of relatively complex digital systems from first principles. In doing so, a firm foundation is laid for the use of CAD methods that are necessary to the design of large systems. In a related sense, VHDL behavioral and architectural descriptions of various machines, combinational and sequential, are provided at various points in the text for those instructors and students who wish to have or require a hardware description language in the study of digital design.

The text is divided into 16 relatively small chapters to provide maximum versatility in its use. These chapters range from introductory remarks to advanced topics in asynchronous systems. In these chapters an attempt is made to replace verbosity by illustration. Students generally do not like to read lengthy verbal developments and explanations when simple illustrations suffice. Well more than 600 figures and tables help to replace lengthy explanations. More than 1000 examples, exercises, and problems (worked and unworked, single and multiple part) are provided to enhance the learning process. They range in complexity from simple algebraic manipulations to multipart system-level designs, each carefully chosen with a specific purpose in mind. Annotated references appear at the end of each chapter, and an appendix at the end of the text provides the details of subjects thought to be peripheral to the main thrust of the text. Chapter 1 breaks with tradition in providing a complete glossary of terms, expressions, and abbreviations that serves as a conspicuous and useful source of information.

## SUBJECT AREAS OF PARTICULAR STRENGTH IN THIS TEXT

Like others, this text has its subject areas of strengths—those that are uniquely presented in sufficient detail as to stand out as significant didactic and edifying contributions. This text



breaks with tradition in providing unique coverage in several important areas. In addition to the traditional coverage, the following 20 subject areas are of particular strength in this text:

1. Thorough coverage of number systems, arithmetic methods and algorithms, and codes
2. Mixed logic notation and symbology used throughout the text
3. Emphasis on CMOS logic circuits
4. Unique treatment of conventional Boolean algebra and XOR algebra as these subjects relate to logic design
5. Entered variable mapping methods as applied throughout the text to combinational and sequential logic design
6. Applications of Reed–Muller transformation forms to function minimization
7. Nonarithmetic combinational logic devices such as comparators, shifters, and FPGAs
8. Arithmetic devices such as carry-save adders, multipliers, and dividers
9. Three uniquely different ALU designs, including an introduction to dual-rail systems and ALUs with completion signal and carry look-ahead capability
10. Detection and elimination methods for static hazards in two-level and multilevel (e.g., XOR-type) circuits including the use of binary decision diagrams (BDDs)
11. Design and analysis of flip-flops provided in a simple, well organized fashion
12. Detection and elimination of timing defects in synchronous sequential circuits
13. Input synchronization and debouncing, and FSM initialization and reset methods
14. Use of unique modular methods in the design of shift registers and counters
15. Complete coverage of ripple counters, ring counters and linear feedback shift register (LFSR and ALFSR) counters
16. Application of the array algebraic and one-hot approaches to synchronous FSM design
17. Detection and elimination of timing defects in asynchronous fundamental mode FSMs
18. Design and analysis of asynchronous FSMs including the nested cell approach, single transition time (STT) machines by using array algebra, and the one-hot code method
19. High speed externally asynchronous/internally clocked systems, including an introduction to dynamic domino logic applications
20. Programmable asynchronous sequencers

## READERSHIP AND COURSE PREREQUISITES

No prior background in digital design is required to enter a *first* course of study by using this text. It is written to accommodate both the first- and second-level user. What is required is that the reader have sufficient maturity to grasp some of the more abstract concepts that are unavoidable in any digital design course of study. It has been the author's experience that digital design makes an excellent introduction to electrical and computer engineering because of the absolute and precise nature of the subjects — there are no approximation signs. This text is designed to make first reading by a user a rewarding experience. However, there is sufficient advanced material to satisfy the needs of the second level students and professionals in the field. A first-level understanding of the subject matter is necessary before entering a second-level course using this text.

## SUGGESTED TEXT USAGE

Perhaps the best advice that can be given to instructors on the use of this text is to study the table of contents carefully and then decide what subject matter is essential to the course under consideration. Once this is done the subject area and order of presentation will usually become obvious. The following two course outlines are offered here as a starting point for instructors in making decisions on course subject usage:

### The Semester System

#### ***[1] First-Level Course — Combinational Logic Design***

##### *Block I*

Introduction (Chapter 1)

Number systems, binary arithmetic and codes (Sections 2.1 through 2.5 or choice)

Binary state terminology, CMOS logic circuits, and mixed-logic symbology  
(Sections 3.1 through 3.7)

Reading and construction of logic circuits (Section 3.8)

XOR and EQV operators and mixed-logic symbology (Section 3.9)

Laws of Boolean and XOR algebra (Sections 3.10 through 3.12)

Review

#### EXAM #1

##### *Block II*

Introduction; logic function representation (Sections 4.1 and 4.2)

Karnaugh map (K-map) function representation and minimization, don't cares,  
and multioutput optimization (Sections 4.3 through 4.5)

Entered variable mapping methods and function reduction of five or more variables  
(Sections 4.6, 4.7 and 4.12)

Introduction to minimization algorithms (Section 4.8)

Factorization and resubstitution methods (Subsections 4.9.1 and 4.9.2)

Function minimization by using XOR K-map patterns (Sections 5.1 through 5.4)

Review

#### EXAM #2

##### *Block III*

Introduction to combinational logic design (Section 6.1)

Multiplexers, decoders, priority encoders, and code converters (Sections 6.2  
through 6.5; Section 2.10)

Magnitude comparators, parity generators and shifters (Sections 6.6 through 6.8)

Programmable logic devices — ROMs, PLAs and PALs (Sections 7.1 through 7.6)

Adders, subtractors, multipliers, and dividers (Section 2.6 and Subsections 2.9.1 through 2.9.5 or choice; Sections 8.1 through 8.7 or choice)  
 Arithmetic and logic units — ALUs (Section 8.8) — may be omitted if time-limited  
 Static hazards in combinational logic devices (Sections 9.1 and 9.2)  
 Review

### EXAM #3 and/or FINAL

## ***[2] Second-Level Course — State Machine Design and Analysis***

### *Block IV*

Introduction; models, the state diagram, and heuristic development of the basic memory cells (Sections 10.1 through 10.4)  
 Design and analysis of flip-flops, flip-flop conversion; timing problems; asynchronous overrides; setup and hold time requirements (Sections 10.5 through 10.11)  
 Design of simple synchronous finite state machines; K-map conversion; analysis of synchronous FSMs (Sections 10.12 and 10.13)  
 Review

### EXAM #1

### *Block V*

Introduction; detection and elimination of timing defects in synchronous state machines (Sections 11.1 through 11.3)  
 Synchronizing and stretching of asynchronous inputs; metastability; clock skew and clock sources (Sections 11.4 through 11.6)  
 Initialization and reset of FSMs, and debouncing circuits (Sections 11.7 and 11.8)  
 Applications to the design and analysis of more complex synchronous FSMs; ASM charts and state assignment rules; array algebraic approach to FSM design; state minimization (Sections 11.9 through 11.12)  
 Review

### EXAM #2

### *Block VI*

Introduction; design of shift registers and synchronous counters; synchronous vs asynchronous parallel loading (Sections 12.1 through 12.3)  
 Shift register counters and ripple counters; special purpose counters (Sections 12.4 through 12.5)  
 Alternative architecture — use of MUXs, decoders, PLDs, counters and shift registers; the one-hot design method (Sections 13.1 through 13.5)  
 The controller, data path, functional partition, and system-level design (Sections 13.6 and 13.7)  
 Introduction to asynchronous sequential machines — fundamental mode FSMs (Sections 14.1 through 14.9)

Pulse mode approach to asynchronous FSM design (Sections 15.1 through 15.6)  
Selected topics in Chapter 16  
Review

### EXAM #3 and/or FINAL

The choice of course content is subject to so many variables that no one course outline will suffice even within a single institution where several instructors may teach a given course. It is for this reason that the text is divided up into 16 relatively small chapters. This offers the instructor somewhat more flexibility in the choice of subject matter. For example, if it is desirable to offer a single (combined) semester course in digital design, it might be desirable to offer both combinational and sequential (synchronous FSM) logic design. Such a course might include the following subject areas taken from Blocks I through VI in sample course outlines [1] and [2]:

#### ***[3] Single (Combined) Semester Course in Digital Design***

Binary state terminology, and mixed-logic symbology (Sections 3.1 through 3.7)  
Reading and construction of logic circuits (Section 3.8)  
XOR and EQV operators and mixed-logic symbology (Section 3.9)  
Laws of Boolean and XOR algebra (Sections 3.10 through 3.12)  
Review

### EXAM #1

Logic function representation (Sections 4.1 and 4.2)  
K-map function representation and minimization, don't cares and multioutput optimization (Sections 4.3 through 4.5)  
Entered variable mapping methods and function reduction of five or more variables (Sections 4.6, 4.7 and 4.12)  
Multiplexers, decoders, priority encoders, and code converters (Sections 6.2 through 6.5)  
Comparators, parity generators, and shifters or choice (Sections 6.6 through 6.8)  
Adders, subtractors, and multipliers (Sections 8.1 through 8.3; Section 8.6)  
Static hazards in combinational logic devices (Sections 9.1 and 9.2)  
Review

### EXAM #2

Heuristic development of the basic memory cells (Sections 10.1 through 10.4)  
Design and analysis of flip-flops, flip-flop conversion (Sections 10.5 through 10.8)  
Asynchronous overrides; setup and hold time requirements; design and analysis of simple synchronous state machines (Sections 10.10 through 10.13)  
Detection and elimination of timing defects in synchronous state machines (Sections 11.1 through 11.3)  
Synchronizing of asynchronous inputs (Section 11.4)  
Initialization and reset of FSMs; debouncing circuits (Sections 11.7 and 11.8)  
Shift registers and counters (Sections 12.1 through 12.3)

Alternative architecture — use of MUXs, decoders, PLDs; the one-hot method  
(Sections 13.1 through 13.3, Section 13.5)

The controller, data path, and functional partition and system-level design  
(Sections 13.6 and 13.7)

Review

### EXAM #3 and/or FINAL

Though the subject coverage for EXAM #3 in course sample outline [3] seems large in proportion to those required for EXAM #2, a close inspection will indicate that the number of sections are the same. The sections required for EXAM #1 number about half that of the other two.

## The Quarter System

Not all courses at colleges and universities are operated on a semester basis. Some are operated on the quarter system. This requires that the course subject areas be divided up in some logical and effective manner, which may require that both combinational and sequential machines be covered within a given quarter course. As a guide to subject area planning on the quarter system when using this text, the following quarter system may be considered (refer to sample course outlines [1] and [2]):

### *First Quarter*

Block I  
EXAM #1  
Block II  
EXAM #2

### *Second Quarter*

Block III  
EXAM #1  
Block IV  
EXAM #2

### *Third Quarter*

Block V  
EXAM #1  
Block VI  
EXAM #2

### *Fourth Quarter (if applicable)*

Chapters 14 and 15  
EXAM #1  
Chapter 16  
PROJECT and/or EXAM #2

Certainly, there are an endless number of ways in which the subject areas can be divided up to meet the requirements of digital design courses that are offered on the basis of a semester, quarter, or trimester system. The presence of 16 relatively small chapters should make the decision process less complicated and lead to a meaningful and productive treatment of digital logic design.

## INSTRUCTIONAL SUPPORT SOFTWARE AND MATERIALS

### For the Student

Bundled with this text on CD-ROM are five important software programs: a logic simulator called *EXL-Sim2002*; two logic minimizers, *BOOZER* and *ESPRESSO II*; and two advanced CAD programs called *ADAM* and *A-OPS*. Complete instructions are included with each software program. The following is a short description of each software program. More detail descriptions are available in Appendix B.

*EXL-Sim2002* is a gate-level, interactive, schematic-capture and simulation program that is ideally suited for use with this text at either the entry or advanced-level of logic design. Its many features include drag-and-drop capability, rubber banding, mixed logic and positive logic simulations, macro generation, individual and global delay assignments, connection features that eliminate the need for wire connections, schematic page sizing and zooming, waveform zooming and scrolling, and a variety of printout capabilities.

*BOOZER* is a software minimization tool that is recommended for use with this text. It accepts entered variable (EV) or canonical (1's and 0's) data from K-maps or truth tables, with or without don't cares, and returns an optimal or near optimal single or multi-output solution. It can handle up to 12 Boolean functions and as many inputs when used on modern computers.

*ESPRESSO II* is another software minimization tool that is in wide use in schools and industry. It supports advanced algorithms for minimization of two-level, multi-output Boolean functions but does not accept entered variables.

*ADAM* (for *Automated Design of Asynchronous Machines*) is a very powerful software tool that permits the automated design of very complex asynchronous and synchronous state machines, all free of timing defects. The input files are state tables for the desired state machines. The output files are given in the Berkeley format appropriate for directly programming PLAs.

*A-OPS* stands for *Asynchronous One-hot Programmable Sequencer* designs of asynchronous and synchronous state machines. *A-OPS* generates output files and VHDL code for the automated timing-defect-free design of 1-Hot sequencers and state machines that can be driven by either PLAs or RAM. This software tool can be used to design systems that permit instant switching between radically different timing-defect-free controllers on a time-shared basis.

### For the Instructor

An instructor's manual is placed on CD-ROM together with all *five* software programs given in the previous paragraphs. The instructor's manual contains the statement of and the detailed solutions for all problems presented in the text, all in PDF format. All figures (also in PDF format) are provided separately in the manual for selective use in creating transparencies or hard copies. Acrobat Reader 5.0, required for reading these files, is free from the Adobe web site <http://www.adobe.com>.

## ERRORS

Any text of this size and complexity is bound to contain errors and omissions that have been overlooked throughout the extensive review and editing process. Identification of any error or omission would be greatly appreciated by the editors of Academic Press and by the author. Constructive comments regarding matters of clarity, organization and coverage of subject matter are also valued. Such information should be directed to the author:

Professor Richard F. Tinder  
School of Electrical Engineering and Computer Science  
Washington State University  
Pullman, WA 99164-2752  
e-mail address: rtinder@eecs.wsu.edu

## ACKNOWLEDGMENTS

Of the many people who have contributed to the completion of this project, certain individuals stand out as having played very significant roles. First, my sincere thanks go to the five reviewers of portions of this text: Professors Ward D. Getty of the University of Michigan, James C. Harden of Mississippi State University, John P. Robinson of the University of Iowa, Harpreet Singh of Wayne State University, and Murali Varanasi of the University of South Florida. Three other persons are here acknowledged for many helpful conversations and suggestions. These are professors Mark Manwaring, Jack Meador, and Mircea Dabacan, all of the School of EECS at Washington State University and friends and colleagues of the author. Special thanks is owed to Professor Manwaring, author of the logic minimizer called BOOZER, for permitting BOOZER to be bundled with this text on CD ROM. A debt of gratitude also goes to Professor Marek Perkowski of Portland State University for his help and suggestions regarding material on Reed–Muller transformation forms covered in Chapter 5. Finally, of great importance to this text is the work of Bob McCurdy, who, with only sketchy ideas from the author, is responsible for the student-friendly but powerful logic simulator, called EXL-Sim2002, that is bundled with this text on CD-ROM.

Four students are gratefully acknowledged for their work in proofing portions of the manuscript: Ryan O’Fallon, Becky Richardson, Rebecca Sheats, and Parag Upadhyaya. Finally, sincere thanks go to the hundreds of students that have over several years made many helpful suggestions and who have helped identify and eliminate many errors and omissions. Furthermore, it must be acknowledged that the students, more than anyone else, have played an essential role in shaping the pedagogical content of this text.

These acknowledgments would not be complete without recognizing the encouragement of and many helpful conversations with Joel Claypool, Executive Editor of Academic Press, a Division of Harcourt, Inc. Most importantly, the person to whom the author owes much more than just a statement of gratitude is his loving wife, his friend and confidant, Gloria.

Richard F. Tinder  
Pullman, Washington

# **Engineering Digital Design**

*Second Edition*



*This page intentionally left blank*

# Introductory Remarks and Glossary

---

## 1.1 WHAT IS SO SPECIAL ABOUT DIGITAL SYSTEMS?

No area of technology has had or is likely to continue to have more of a profound impact on our lives than digital system development. That's quite a statement, but its truth is obvious when one considers the many ways we have become dependent on "digitized" technology. To put this in perspective, let us review the various areas in which digital systems play an important role in our lives. As this is done, keep in mind that there is significant, if not necessary, overlap in the digital system technologies that make possible those areas we have come to take for granted: computing, information retrieval, communication, automatic control systems, entertainment, and instrumentation.

**Computing:** A computer, like the telephone and television, has become almost an essential part of every household. Word processing, information retrieval, communication, finance and business management, entertainment, art and graphics — these are but a few of the functions performed by our beloved computers. In the span of a little more than 10 years, computers in the home and in small businesses have advanced from what was termed microcomputers to the present computers with nearly mainframe capability. Home computers can now perform relatively sophisticated operations in the areas just mentioned. Of course, vastly improved computer speed and memory, together with powerful software development, are primarily responsible for the rapid rise in personal computer capabilities. In addition to the digital computer itself, there are other digital devices or peripherals that are normally part of a computer system. These include disk drives, CD-ROM drives, modems, CRT and LCD monitors, sound cards, scanners, and printers. Then there are the hand-held calculators that now have nearly microcomputer capability and are quite inexpensive. All of these things have been made possible because of the advances in digital system technology. But this is just the beginning.

**Information Retrieval:** The ability to consult one's favorite encyclopedia via CD-ROM or surf (browse) the World Wide Web (WWW) has become a very important part of computer use in the home, at school, and in business. The use of CD-ROMs also permits access to information in the specialized areas of literature, music, religion, health, geography, math,

physical science, biology, and medicine, to name a few. But information retrieval is not limited to these functions. Network communication between computers and our ability to tap into huge university libraries are other important sources of information. Think of where businesses would be without access to data-base information that is critical to day-to-day operation. Local and national security operations depend heavily on data-base information stored on computers that are most likely part of a network. Yes, and then there is education. What an invaluable source of information the computer has become both in the classroom and in the home.

**Communications:** It would be hard to imagine what our world would be like without the ability to send facsimile (fax) communications or e-mail. These are digital transmission methods that were developed to a high degree of sophistication over a period of about 10 years. Of course, the modem, another digital device, has made this possible. Digital communication is hardly limited to fax and e-mail. One's home phone or cellular phone is likely to be digital, permitting a variety of features that were difficult if not impossible to provide by means of an analog transmission device. Scientific data, national security information, and international communications, all of which are collected and transmitted back to earth by satellite, are accomplished by digital transmission methods with accuracy not possible otherwise.

**Automatic Control Systems:** Digital automatic control systems have replaced the old analog methods in almost all areas of industry, the home, and transportation. Typical examples include rapid transit systems, integrated circuit fabrication systems, robot systems of all types in assembly-line production, space vehicle operations, a variety of automobile associated operations, guidance systems, home security systems, heating and air-conditioning systems, many home appliances, and a host of medical systems.

**Entertainment:** Who cannot help but be awed by the impressive computer generated graphics that have become commonplace in movies and in games produced on CDs. Movies such as Jurassic Park and the new Star Wars series will perhaps be remembered as having established a new era in the art of make-believe. The games that are available on the home computer include everything from chess and casino-type games to complex and challenging animated aircraft operations and adventure/fantasy games. Then add to these the high-quality sound that CDs and the Internet produce, and one has a full entertainment center as part of the personal computer. Of course, the incursion of digital systems into the world of entertainment extends well beyond movies and games. For example, one has only to listen to digitally recorded or remastered CDs (from the original analog recordings) to enjoy their clear, noise-free character. Also, don't forget the presence of electronic keyboard instruments ranging from synthesizers to Clavinovas and the like. Then for those who consider photography as entertainment, digital cameras and camcorders fall into this category. And the list goes on and on.

**Instrumentation:** A listing of the many ways in which digital system technology has affected our lives would not be complete without mentioning the myriad of measurement and sensing instruments that have become digitized. Well known examples of electronic laboratory testing equipment include digital voltmeters, ammeters, oscilloscopes, and waveform generators and analyzers. Then there are the sophisticated medical instruments that include MRI and CAT scan devices. Vital signs monitoring equipment, oximeters, IV pumps, patient controlled analgesia (PCA) pumps, digital ear thermometers, and telemetry equipment

are typical examples of the many other ways the medical industry has made use of digital systems technology.

## 1.2 THE YEAR 2002 AND BEYOND?

If one considers what has happened in, say, the past 15 years, the path of future technological development in the field of digital systems would seem to be limited only by one's imagination. It is difficult to know where to begin and where to end the task of forecasting digital system development, but here are a few examples in an attempt to accomplish this:

Computer power will continue to increase as the industry moves to  $0.10\mu$  (and below) CMOS technology with speeds into the terahertz range and with a demand for more efficient ways to sink the heat generated by billions of transistors per processor operated with supply voltages of one volt or below. There will be dramatic changes in the peripherals that are now viewed as part of the computer systems. For example, vacuum (CRT) monitors will eventually be replaced by picture-frame style LCD monitors, or by *micropanel displays* using either DLP (*Digital Light Processing*) or FED (*field emission display*) technologies. Digitized high-definition TV (HDTV) will eventually replace all conventional TV sets, and the World Wide Web (WWW) will be viewed on HDTV via special dedicated computers. In all, larger, sharper, brighter, and clearer computer and TV displays are to be expected, together with a fast-growing and impressive assortment of wireless hand-held and wrist-bound devices.

Expect that the mechanically operated magnetic storage systems (disk drives) of today will soon be replaced by a MR (*magneto-resistive*) technology that will increase the *areal storage density* (gigabits per square inch) by a factor of 100 to 200, or by OAWD (*optically assisted Winchester drive*) and MO (*magneto-optical*) technologies that are expected to increase the areal density even further. Eventually, a holographic storage technology or a *proximal probe* technology that uses a scanning tunneling microscopic technique may provide capabilities that will take mass storage to near its theoretical limit. Thus, expect storage systems to be much smaller with enormously increased storage capacity.

Expect that long-distance video conferencing via computer will become as commonplace as the telephone is today. Education will be a major beneficiary of the burgeoning digital age with schools (K–12, and universities and colleges both public and private) being piped into major university libraries and data banks, and with access to the ever-growing WWW. Look for the common film cameras of today to be replaced by digital cameras having megapixel resolution, audio capability, and with the capability to store a large number of pictures that can be reviewed on camera and later presented on screen by any computer. Expect that certain aspects of laser surgery will be microprocessor controlled and that X-ray imaging methods (e.g., mammography) and radiology generally will be digitally enhanced as a common practice. Also, health facilities and hospitals will be linked for immediate remote site consultation and for specialized robotics surgery.

Expect digital systems to become much more sophisticated and pervasive in our lives. Interconnectivity between “smart” electrically powered systems of all types in the home, automobile, and workplace could be linked to the web together with sophisticated fail-safe and backup systems to prevent large-scale malfunction and possible chaos. Such interconnected systems are expected to have a profound effect on all aspects of our lives — what and when we eat, our exercise habits, comfort and entertainment needs, shopping

activities, medical requirements, routine business transactions, appointment schedules, and many others imaginable.

Optical recognition technology will improve dramatically in the fields of robotics, vehicular operation, and security systems. For example, expect that *iris and retinal pattern recognition* will eventually be used to limit access to certain protected systems and areas, and may even replace digital combination locks, IDs, and licenses for such purposes. Taxation, marketing, and purchasing methods will undergo dramatic changes as digital systems become commonplace in the world of government, commerce, and finance. Even the world of politics, as we now know it, will undergo dramatic change with the use of new and more efficient voting and voter sampling methods. Mass production line manufacturing methods by using robots and other digitally automated mechanical devices will continue to evolve at a rapid pace as dictated by domestic and world market forces. Expect that logic minimization tools and automated digital design tools will become more commonplace and sophisticated, permitting designers with little practical experience to design relatively complex systems.

Business networking will undergo dramatic improvements with the continued development of gigabit Ethernet links and high-speed switching technology. Home connectivity will see vast improvements in *satellite data service downloading* (up to 400 kbps), 56-kbps (and higher) modems that need high-quality digital connections between phones and destination, improved satellite data service with bidirectional data transmission, and DSL (digital subscriber line) cable modem systems.

Finally, there are some really exciting areas to watch. Look for speech recognition, speech synthesis, and handwriting and pattern recognition to dramatically change the manner in which we communicate with and make use of the computer both in business and in the home. Somewhere in the future the computer will be equipped with speech understanding capability that allows the computer to build ideas from a series of spoken words — perhaps like HAL 9000 in the film *2001: A Space Odyssey*. Built-in automatic learning capability may yet prove to be the most challenging undertaking facing computer designers of the future. Thus, expect to see diminished use of the computer keyboard with time as these technologies evolve into common usage.

Revolutionary computer breakthroughs may come with the development of radically different technologies. *Carbon nanotube technology*, for example, has the potential to propel computer speeds well into the gigahertz range together with greatly reduced power dissipation. The creation of carbon nanotube transistors could signal the dawn of a new revolution in chip development. Then there is the specter of the *quantum computer*, whose advent may lead to computing capabilities that are trillions of times faster than those of conventional supercomputers. All of this is expected to be only the beginning of a new millennium of invention limited only by imagination. Remember that radically different technological breakthroughs can appear at any time, even without warning, and can have a dramatic affect on our lives, hopefully for the better.

To accomplish all of the preceding, a new generation of people, technically oriented to cope with the rapidly changing digital systems technology, will result as it must. This new generation of people will have a dramatic impact on education, labor, politics, transportation, and communications, and will most certainly affect domestic and global economies. Thus, expect that more pressure and responsibility will be placed on universities to produce the quality training that can match up to this challenge, not just over a short period but also in the long term.

### 1.3 A WORD OF WARNING

Not yet mentioned are the changes that must take place in the universities and colleges to deal with this rapidly evolving technology. It is fair to say that computer aided design (CAD) or automated design of digital systems is on the upswing. Those who work in the areas of digital system design are familiar with such hardware description languages as VHDL or Verilog, and the means to “download” design data to program PLAs or FPGAs (field programmable gate arrays). It is possible to generate a high-level hardware description of a digital system and introduce that hardware description into circuit layout tools such as Mentor Graphics. The end result would be a transistor-level representation of a CMOS digital system that could be simulated by one of several simulation tools such as HSPICE and subsequently be sent to the foundry for chip creation. The problem with this approach to digital system design is that it bypasses the need to fully understand the intricacies of design that ensure proper and reliable system operation. As is well known, a successful HSPICE simulation does not necessarily ensure a successful design. In the hands of a skilled and experienced designer this approach may lead to success without complications. On the other hand, if care is not taken at the early stages of the design process and if the designer has only a limited knowledge of design fundamentals, the project may fail at one point or another. Thus, as the use of automated (CAD) designs become more attractive to those who lack design detail fundamentals, the chance for design error at the system, device, gate, or transistor level increases. *The word of warning:* Automated design should never be undertaken without a sufficient knowledge of the field and a thorough understanding of the digital system under consideration — *a little knowledge can be dangerous!* This text is written with this warning in mind. The trend toward increasing CAD use is not bad, but automated design methods must be used cautiously with sufficient background knowledge to carry out predictably successful designs. Computer automated design should be used to remove the tedium from the design process and, in many cases, make tractable certain designs that would otherwise not be possible. But CAD is *not* a replacement for the details and background fundamentals required for successful digital system design. It is the goal of this text to provide the reader with the necessary details and background fundamentals so as to permit a successful transition into the CAD domain.

### 1.4 GLOSSARY OF TERMS, EXPRESSIONS, AND ABBREVIATIONS

Upon entering any new field, there is always the problem of dealing with the “jargon” that is peculiar or unique to that field. Conspicuously absent in most texts on digital design is a glossary of terms, expressions, and abbreviations that are used — yes, and even overused — in presenting the subject matter. Readers of these texts are often left leafing through back pages and chapters in search of the meaning of a given term, expression or abbreviation. In breaking with tradition, this text provides an extensive *glossary*, and does so here at the beginning of the text where it can be used — not at the end of the text where it may go unnoticed. In doing this, Chapter 1 serves as a useful source of information.

**ABEL:** advanced Boolean expression language.

**Accumulator:** an adder/register combination used to store arithmetic results.

**Activate:** to assert or make active.

- Activation level:** the logic state of a signal designated to be active or inactive.
- Activation level indicator:** a symbol, (H) or (L), that is attached to a signal name to indicate positive logic or negative logic, respectively.
- Active:** a descriptor that denotes an action condition and that implies logic 1.
- Active device:** any device that provides current (or voltage) gain.
- Active high (H):** indicates a positive logic source or signal.
- Active low (L):** indicates a negative logic source.
- Active state:** the logic 1 state of a logic device.
- Active transition point:** the point in a voltage waveform where a digital device passes from the inactive state to the active state.
- Addend:** an operand to which the augend is added.
- Adder:** a digital device that adds two binary operands to give a sum and a carry.
- Adder/subtractor:** a combinational logic device that can perform either addition or subtraction.
- Adjacent cell:** a K-map cell whose coordinates differ from that of another cell by only one bit.
- Adjacent pattern:** an XOR pattern involving an uncomplemented function in one cell of a K-map and the same function complemented in an adjacent cell.
- ALFSR:** autonomous linear feedback shift register.
- ALFSR counter:** a counter, consisting of an ALFSR, that can sequence through a unique set of pseudo-random states that can be used for test vectors.
- Algorithm:** any special step-by-step procedure for accomplishing a task or solving a problem.
- Alternative race path:** one of two or more transit paths an FSM can take during a race condition.
- ALU:** arithmetic and logic unit.
- Amplify:** the ability of an active device to provide current or voltage gain.
- Analog:** refers to continuous signals such as voltages and current, in contrast to digital or discrete signals.
- AND:** an operator requiring that all inputs to an AND logic circuit symbol be active before the output of that symbol is active — also, Boolean product or intersection.
- AND function:** the function that derives from the definition of AND.
- AND gate:** a physical device that performs the electrical equivalent of the AND function.
- AND laws:** a set of Boolean identities based on the AND function.
- AND-OR-Invert (AOI) gate:** a physical device, usually consisting of two AND gates and one NOR gate, that performs the electrical equivalent of SOP with an active low output.
- AND plane:** the ANDing stage or matrix of a PLD such as a ROM, PLA, or PAL.
- Antiphase:** as used in clock-driven machines to mean complemented triggering of a device relative to a reference system, such as, an FET input device to an RET FSM.
- Apolar input:** an input, such as CK, that requires no activation level indicator to be associated with it.
- Arbiter module:** a device that is designed to control access to a protected system by arbitration of contending signals.
- Arithmetic and logic unit (ALU):** a physical device that performs either arithmetic or logic operations.

- Arithmetic shifter:** a combinational shifter that is capable of generating and preserving a sign bit.
- Array algebra:** the algebra of Boolean arrays and matrices associated with the automated design of synchronous and STT machines.
- Array logic:** any of a variety of logic devices, such as ROMs, PLAs or PALs, that are composed of an AND array and an OR array (see Programmable logic device or PLD).
- ASIC:** application-specific IC.
- ASM:** algorithmic state machine.
- Assert:** activate.
- Assertion level:** activation level.
- Associative law:** a law of Boolean algebra that states that the operational sequence as indicated by the location of parentheses in a p-term or s-term does not matter.
- Associative pattern:** an XOR pattern in a K-map that allows a term or variable in an XOR or EQV function to be looped out (associated) with the same term or variable in an adjacent cell provided that the XOR or EQV connective is preserved in the process.
- Asynchronous:** clock-independent or self-timed — having no fixed time relationship.
- Asynchronous input:** an input that can change at any time, particularly during the sampling interval of the enabling input.
- Asynchronous override:** an input such as preset or clear that, when activated, interrupts the normal operation of a flip-flop.
- Asynchronous parallel load:** the parallel loading of a register or counter by means of the asynchronous PR and CL overrides of the flip-flops.
- Augend:** an operand that is added to the addend in an addition operation.
- Barrel shifter:** a combinational shifter that only rotates word bits.
- Base:** radix. Also, one of three regions in a BJT.
- Basic cell:** a basic memory cell, composed of either cross-coupled NAND gates or cross-coupled NOR gates, used in the design of other asynchronous FSMs including flip-flops.
- BCD:** binary coded decimal.
- BCH:** binary coded hexadecimal.
- BCO:** binary coded octal.
- BDD:** binary decision diagram.
- Bidirectional counter:** a counter that can count up or down.
- Binary:** a number system of radix 2; having two values or states.
- Binary code:** a combination of bits that represent alphanumeric and arithmetic information.
- Binary coded decimal (BCD):** a 4-bit, 10-word decimal code that is weighted 8, 4, 2, 1 and that is used to represent decimal digits as binary numbers.
- Binary coded hexadecimal (BCH):** the hexadecimal number system used to represent bit patterns in binary.
- Binary coded octal (BCO):** the octal number system used to represent bit patterns in binary.
- Binary decision diagram (BDD):** a graphical representation of a set of binary-valued decisions, beginning with an input variable and proceeding down paths that end in either logic 1 or logic 0.
- Binary word:** a linear array of juxtaposed bits that represents a number or that conveys an item of information.



**Bipolar junction transistor (BJT):** an npn or pnp transistor.

**Bipolar PROM:** a PROM that uses diodes as fusible links.

**BIST:** built-in-self-test.

**Bit:** a binary digit.

**Bit slice:** partitioned into identical parts such that each part operates on one bit in a multibit word—part of a cascaded system of identical parts.

**BJT:** bipolar junction transistor.

**BO:** borrow-out.

**Bond set:** in the CRMT method, a disjoint set of bond variables.

**Bond variable:** one of two or more variables that form the axes of an EV K-map used in the CRMT method of function minimization.

**Boolean algebra:** the mathematics of logic attributed to the mathematician George Boole (1815–1864).

**Boolean product:** AND or intersection operation.

**Boolean sum:** OR or union operation.

**BOOZER:** Boolean ZERo-one Reduction—a multioutput logic minimizer that accepts entered variables.

**Borrow-in:** the borrow input to a subtractor.

**Borrow-out:** the borrow output from a subtractor.

**Boundary:** the separation of logic domains in a K-map.

**Bounded pulse:** a pulse with both lower and upper limits to its width.

**Branching condition (BC):** the input requirements that control a state-to-state transition in an FSM.

**Branching path:** a state-to-state transition path in a state diagram.

**Buffer:** a line driver.

**Buffer state:** a state (in a state diagram) whose only purpose is to remove a race condition.

**Bus:** a collection of signal lines that operate together to transmit a group of related signals.

**Byte:** a group of eight bits.

**C:** carry. Also, the collector terminal in a BJT.

**CAD:** computer-aided design.

**CAE:** computer-aided engineering.

**Call module:** a module designed to control access to a protected system by issuing a request for access to the system and then granting access after receiving acknowledgment of that request.

**Canonical:** made up of terms that are either all minterms or all maxterms.

**Canonical truth table:** a 1's and 0's truth table consisting exclusively of minterms or maxterms.

**Capacitance,  $C$ :** the constant of proportionality between total charge on a capacitor and the voltage across it,  $Q = CV$ , where  $C$  is given in farads (F) when charge  $Q$  is given in coulombs and  $V$  in volts.

**Capacitor:** a two-terminal energy storing element for which the current through it is determined by the time-rate of change of voltage across it.

**Cardinality:** the number of prime implements (p-term or s-term cover) representing a function.

**Carry generate:** a function that is used in a carry look-ahead (CLA) adder.

**Carry-in:** the carry input to a binary adder.

**Carry look-ahead (CLA):** same as look-ahead-carry.

- Carry-out:** the carry output from an Adder.
- Carry propagate:** a function that is used in a CLA adder.
- Carry save (CS):** a fast addition method for three or more binary numbers where the carries are saved and added to the final sum.
- Cascade:** to combine identical devices in series such that any one device drives another; to bit-slice.
- Cell:** the intersection of all possible domains of a K-map.
- Central processing unit (CPU):** a processor that contains the necessary logic hardware to fetch and execute instructions.
- CGP:** carry generate/propagate.
- CI:** carry-in.
- Circuit:** a combination of elements (e.g., logic devices) that are connected together to perform a specific operation.
- CK:** clock.
- CL or CLR:** clear.
- CLA:** carry look-ahead.
- CLB:** configurable logic block. Also, a logic cell (LC).
- Clear:** an asynchronous input used in flip-flops, registers, counters and other sequential devices, that, when activated, forces the internal state of the device to logic 0.
- Clock:** a regular source of pulses that control the timing operations of a synchronous sequential machine.
- Clock skew:** a phenomenon that is generally associated with high frequency clock distribution problems in synchronous sequential systems.
- C-module:** an RMOD.
- CMOS:** complementary configured MOSFET in which both NMOS and PMOS are used.
- CNT:** mnemonic for count.
- CO:** carry-out.
- Code:** a system of binary words used to represent decimal or alphanumeric information.
- Code converter:** a device designed to convert one binary code to another.
- Collapsed truth table:** a truth table containing irrelevant inputs.
- Collector:** one of three regions in a BJT.
- Combinational hazard:** a hazard that is produced within a combinational logic circuit.
- Combinational logic:** a configuration of logic devices in which the outputs occur in direct, immediate response to the inputs without feedback.
- Commutative law:** the Boolean law that states that the order in which variables are represented in a p-term or s-term does not matter.
- Comparator:** a combinational logic device that compares the values of two binary numbers and issues one of three outputs indicative of their relative magnitudes.
- Compatibility:** a condition where the input to a logic device and the input requirement of the device are of the same activation level, that is, are in logic agreement.
- Compiler:** converts high-level language statements into typically a machine-coded or assembly language form.
- Complement:** the value obtained by logically inverting the state of a binary digit; the relationship between numbers that allows numerical subtraction to be performed by an addition operation.
- Complementary metal oxide semiconductor (CMOS):** a form of MOS that uses both p- and n-channel transistors (in pairs) to form logic gates.

**Complementation:** a condition that results from logic incompatibility; the mixed-logic equivalent of the NOT operation.

**Complex PLD:** an on-chip array of PAL-like PLDs with I/O blocks and a programmable interconnect structure.

**Composite output map:** a K-map that contains entries representing multiple outputs.

**Computer:** a digital device that can be programmed to perform a variety of tasks (e.g., computations) at extremely high speed.

**Concatenation:** act of linking together or being linked together in a series.

**Conditional branching:** state-to-state transitions that depend on the input status of the FSM.

**Conditional output:** an output that depends on one or more external inputs.

**Conjugate gate forms:** a pair of logic circuit symbols that derive from the same physical gate and that satisfy the DeMorgan relations.

**Connective:** a Boolean operator symbol (e.g., +,  $\oplus$ ,  $\cap$ ).

**Consensus law:** a law in Boolean algebra that allows simplification by removal of a redundant term.

**Consensus term:** the redundant term that appears in a function obeying the consensus law.

**Controlled inverter:** an XOR gate that is used in either the inverter or transfer mode.

**Controller:** that part of a digital system that controls the data path.

**Conventional K-map:** a K-map whose cell entries are exclusively 1's and 0's.

**Counter:** a sequential logic circuit designed to count through a particular sequence of states.

**Counteracting delay:** a delay placed on an external feedback path to eliminate an E-hazard or d-trio.

**Count sequence:** a repeating sequence of binary numbers that appears on the outputs of a counter.

**Coupled term:** one of two terms containing only one coupled variable.

**Coupled variable:** a variable that appears complemented in one term of an expression (SOP or POS) and that also appears uncomplemented in another term of the same expression.

**Cover:** a set of terms that covers all minterms or maxterms of a function.

**CPLD:** complex PLD.

**CPU:** central processing unit.

**Creeping code:** any code whose bit positions fill with 1's beginning at one end, and then fill with 0's beginning at the same end.

**Critical race:** a race condition in an asynchronous FSM that can result in transition to and stable residence in an erroneous state.

**CRMT:** contracted Reed–Muller transformation.

**Cross branching:** multiple transition paths from one or more states in the state diagram (or state table) of a sequential machine whereby unit distance coding of states is not possible.

**CU:** control unit.

**Current,  $I$ :** the flow or transfer of charged matter (e.g., electrons) given in amperes (A).

**Cutoff mode:** the physical state of a BJT in which no significant collector current is permitted to flow.

**Cycle:** two or more successive and uninterrupted state-to-state transitions in an asynchronous sequential machine.

- Data bus:** a parallel set of conductors which are capable of transmitting or receiving data between two parts of a system.
- Data lockout:** the property of a flip-flop that permits the data inputs to change immediately following a reset or set operation without affecting the flip-flop output.
- Data lockout flip-flop:** a one-bit memory device which has the combined properties of a master/slave flip-flop and an edge triggered flip-flop.
- Data path:** the part of a digital system that is controlled by the controller.
- Data path unit:** the group of logic devices that comprise the data path.
- Data selector:** a multiplexer.
- Data-triggered:** referring to flip-flops triggered by external inputs (no clock) as in the pulse mode.
- DCL:** digital combination lock.
- Deactivate:** to make inactive.
- Deassert:** deactivate.
- Debounce:** to remove the noise that is produced by a mechanical switch.
- Debouncing circuit:** a circuit that is used to debounce a switch.
- Decade:** a quantity of 10.
- Decoder:** a combinational logic device that will activate a particular minterm code output line determined by the binary code input. A demultiplexer.
- Decrement:** reduction of a value by some amount (usually by 1).
- Delay:** the time elapsing between related events in process.
- Delay circuit:** a circuit whose purpose it is to delay a signal for a specified period of time.
- Delimiter:** a character used to separate lexical elements and has a specific meaning in a given language. Examples are @, #, +, /, ', >.
- DeMorgan relations:** mixed logic expressions of DeMorgan's laws.
- DeMorgan's laws:** a property that states that the complement of the Boolean product of terms is equal to the Boolean sum of their complements; or that states that the complement of the Boolean sum of terms is the Boolean product of their complements.
- Demultiplexer:** a combinational logic device in which a single input is selectively steered to one of a number of output lines. A decoder.
- Depletion mode:** a normally ON NMOS that has a conducting n-type drain-to-source channel in the absence of a gate voltage but that loses its conducting state when the gate voltage reaches some negative value.
- D flip-flop:** a one-bit memory device whose output value is set to the D input value on the triggering edge of the clock signal.
- D-flop module:** a memory element that is used in an EAIC system and that has characteristics similar to that of a D flip-flop.
- Diagonal pattern:** an XOR pattern formed by identical EV subfunctions in any two diagonally located cells of a K-map whose coordinates differ by two bits.
- Difference:** the result of a subtraction operation.
- Digit:** a single symbol in a number system.
- Digital:** related to discrete quantities.
- Digital combination lock:** a sequence recognizer that can be used to unlock or lock something.
- Digital engineering design:** the design and analysis of digital devices.
- Digital signal:** a logic waveform composed of discrete logic levels (e.g., a binary digital signal).

- Diode:** a two-terminal passive device consisting of a  $p$ - $n$  junction that permits significant current to flow only in one direction.
- Diode-transistor logic:** logic circuits consisting mainly of diodes and BJTs.
- Direct address approach:** an alternative approach to FSM design where PS feedback is direct to the NS logic.
- Disjoint:** as used in “mutually disjoint” to mean a set of  $p$ -terms whose ANDed values taken two at a time are always logic zero; or a set of  $s$ -terms whose ORed values taken two at a time are always logic one.
- Distributed path delays:** a notation in which a path delay is assigned to each gate or inverter of a logic circuit.
- Distributive law:** The dual of the factoring law.
- Divide-by- $n$  counter:** a binary counter of  $n$  states whose MSB output divides the clock input frequency by  $n$ .
- Dividend:** the quantity that is being divided by the divisor in a division operation.
- Divider:** a combinational logic device that performs the binary division operation.
- Divisor:** the quantity that is divided into the dividend.
- DLP:** digital light processing.
- DMUX:** demultiplexer (see decoder).
- Domain:** a range of logic influence or control.
- Domain boundary:** the vertical or horizontal line or edge of a K-map.
- Don't care:** a non-essential minterm or maxterm, denoted by the symbol  $\phi$ , that can take either a logic 1 or logic 0 value. Also, a delimiter  $\phi$  that, when attached to a variable or term, renders that variable or term nonessential to the parent function.
- DPU:** data path unit; also data processing unit.
- Drain:** one of three terminals of a MOSFET.
- DRAM:** dynamic RAM.
- Driver:** a one-input device whose output can drive substantially more inputs than a standard gate. A buffer.
- DTL:** diode-transistor logic.
- D-trio:** a type of essential hazard that causes a fundamental mode machine to transit to the correct state via an unauthorized path.
- Duality:** a property of Boolean algebra that results when the AND and OR operators (or XOR and EQV operators) are interchanged simultaneously with the interchange of 1's and 0's.
- Dual-rail systems:** as used in this text, a system of split signals in an ALU configuration that permits a completion signal to be issued at the end of each process, be it arithmetic or logic.
- Dual relations:** two Boolean expressions that can be derived one from the other by duality.
- Duty cycle:** in a periodic waveform, the percentage of time the waveform is active.
- Dyad:** a grouping of two logically adjacent minterms or maxterms.
- Dynamic domino logic:** buffered CMOS logic that requires complementary precharge and evaluate transistors for proper operation.
- Dynamic hazard:** multiple glitches that occur in the output from a multilevel circuit because of a change in an input for which there are three or more asymmetric paths (delay-wise) of that input to the output.
- Dynamic RAM:** a volatile RAM memory that requires periodic refreshing to sustain its memory.

- EAIC system:** externally asynchronous/internally clocked system.
- ECL:** emitter-coupled logic.
- Edge-triggered flip-flop:** a flip-flop that is triggered on either the rising edge or falling edge of the clock waveform and that exhibits the data-lock-out feature.
- EEPROM:** electrically erasable PROM.
- E-hazard:** essential hazard.
- EI:** enable-in.
- Electron:** the majority carrier in an n-type conducting semiconductor.
- Electronic switch:** a voltage or current controlled switching device.
- Emitter:** one of three terminals of a BJT.
- Emitter-coupled logic (ECL):** a high-speed nonsaturating logic family.
- EN:** enable.
- Enable:** an input that is used to enable (or disable) a logic device, or that permits the device to operate normally.
- Encoder:** a digital device that converts digital signals into coded form.
- Endless cycle:** an oscillation that occurs in asynchronous FSMs.
- Enhancement mode:** a normally OFF NMOS that develops an n-channel drain-to-source conducting path (i.e., turns ON) with application of a sufficiently large positive gate voltage.
- Entered variable (EV):** a variable entered in a K-map.
- EO:** enable-out.
- EPI:** essential prime implicant.
- EPLD:** erasable PLD.
- EPROM:** erasable programmable read-only memory.
- EQPOS:** EQV-product-of-sums.
- Equivalence:** the output of a two-input logic gate that is active if, and only if, its inputs are logically equivalent (i.e., both active or both inactive).
- EQV:** equivalence.
- EQV function:** the function that derives from the definition of equivalence.
- EQV gate:** a physical device that performs the electrical equivalent of the EQV function.
- EQV laws:** a set of Boolean identities based on the EQV function.
- Erasable programmable read-only memory (EPROM):** a ROM that can be programmed many times.
- Error catching:** a serious problem in a JK master/slave flip-flop where a 1 or 0 is caught in the master cell when clock is active and is issued to the slave cell output when clock goes inactive.
- Essential hazard:** a disruptive sequential hazard that can occur as a result of an explicitly located delay in an asynchronous FSM that has at least three states and that is operated in the fundamental mode.
- Essential prime implicant (EPI):** a prime implicant that must be used to achieve minimum cover.
- EU:** execution unit.
- EV:** entered variable.
- EV K-map:** a K-map that contains EVs.
- EV truth table:** a truth table containing EVs.
- Even parity:** an even number of 1's (or 0's) in a binary word depending on how even parity is defined.

- EVM:** entered variable K-map.
- Excess 3 BCD (XS3) code:** BCD plus three.
- Excitation table:** a state transition table relating the branching paths to the branching condition values given in the state diagram for a flip-flop.
- Exclusive OR:** a two-variable function that is active if only one of the two variables is active.
- EXOP:** XOR-sum-of-products.
- Expansion of states:** opposite of merging of states.
- Extender:** a circuit or gate that is designed to be connected to a digital device to increase its fan-in capability — also called an expander.
- Factoring law:** the Boolean law that permits a variable to be factored out of two or more p-terms that contain the variable in an SOP or XOR expression.
- Fall time:** the period of time it takes a voltage signal to change from 90% to 10% of its high value.
- Falling edge-triggered (FET):** activation of a device on the falling edge of the triggering (sampling) variable.
- False carry rejection:** the feature in an ALU where all carry-outs are disabled for all nonarithmetic operations.
- False data rejection (FDR):** the feature of a code converter that indicates when unauthorized data has been issued to the converter.
- Fan-in:** the maximum number of inputs a gate may have.
- Fan-out:** the maximum number of equivalent gate inputs that a logic gate output can drive.
- FDR:** false data rejection.
- FDS diagram:** fully documented state diagram.
- FED:** field emission display.
- Feedback path:** a signal path of a PS variable from the memory output to the NS input.
- FET:** falling edge-triggered. Also, field effect transistor.
- Fetch:** that part of an instruction cycle in which the instruction is brought from the memory to the CPU.
- FF:** flip-flop.
- Field programmable gate array (FPGA):** a complex PLD that may contain a variety of primitive devices such as discrete gates, MUXs and flip-flops.
- Field programmable logic array (FPLA):** one-time user programmable PLA.
- FIFO:** first-in-first-out memory register.
- Fill bit:** the bit of a combinational shifter that receives the fill logic value in a shifting operation.
- Finite state machine (FSM):** a sequential machine that has a finite number of states in which it can reside.
- Flag:** a hardware or software “marker” used to indicate the status of a machine.
- Flip-flop (FF):** a one-bit memory element that exhibits sequential behavior controlled exclusively by a clock input.
- Floating-gate NMOS:** special NMOS used in erasable PROMs.
- Floating point number (FPN) system:** a binary number system expressed in two parts, as a fraction and exponential, and that is used in computers to arithmetically manipulate large numbers.
- Flow chart:** a chart that is made up of an interconnection of action and decision symbols for the purpose of representing the sequential nature of something.

- Flow table:** a tabular realization of a state diagram representing the sequential nature of an FSM.
- Fly state:** a state (in a state diagram) whose only purpose is to remove a race condition. A buffer state.
- Forward bias:** a voltage applied to a  $p$ - $n$  junction diode in a direction as to cause the diode to conduct (turn ON).
- FPGA:** field programmable gate array.
- FPLA:** field programmable logic array.
- FPLS:** field programmable logic sequencer.
- Free set:** variables of a function not used as the bond set in CRMT forms.
- Frequency,  $f$ :** the number of waveform cycles per unit time in Hz or  $s^{-1}$ .
- Frequency division:** the reduction of frequency by a factor of  $f/n$  usually by means of a binary counter, where  $n$  is the number of states in the counter.
- FSM:** finite state machine, either synchronous or asynchronous.
- Full adder (FA):** a combinational logic device that adds two binary bits to a carry-in bit and issues a SUM bit and a carry-out bit.
- Full subtractor (FS):** a combinational logic device that subtracts a subtrahend bit and a borrow-in bit from a minuend bit, and issues a difference bit and a borrow-out bit.
- Fully documented state diagram:** a state diagram that specifies all input branching conditions and output conditions in literal or mnemonic form, that satisfies the sum rule and mutually exclusive requirement, and that has been given a proper state code assignment.
- Function:** a Boolean expression representing a specific binary operation.
- Functional partition:** a diagram that gives the division of device responsibility in a digital system.
- Function generator:** a combinational logic device that generates logic functions (usually via a MUX).
- Function hazard:** a hazard that is produced when two or more coupled variables change in near proximity to each other.
- Fundamental mode:** the operational condition for an asynchronous FSM in which no input change is permitted to occur until the FSM has stabilized following any previous input change.
- Fusible link:** an element in a PLD memory bit location that can be “blown” to store a logic 1 or logic 0 depending on how the PLD is designed.
- Gain element:** a device, such as a buffer, used to boost a signal.
- GAL:** general array logic.
- Gate:** a physical device (circuit) that performs the electrical equivalent of a logic function. Also, one of three terminals of a MOSFET.
- Gated basic cell:** a basic cell that responds to its S and R input commands only on the triggering edge of a gate or clock signal.
- Gate/input tally:** the gate and input count associated with a given logic expression — the gate tally may or may not include inverters, but the input count must include both external and internal inputs.
- Gate-minimum logic:** logic requiring a minimum number of gates; may include XOR and EQV gates in addition to two-level logic.
- Gate path delay:** the interval of time required for the output of a gate to respond to an input signal change.
- Glitch:** an unwanted transient in an otherwise steady-state signal.



- Go/No-Go configuration:** a single input controlling the hold and exit conditions of a state in a state diagram.
- Gray code:** a reflective unit distance code.
- Ground:** a reference voltage level usually taken to be zero volts.
- GS:** group signal.
- Half adder (HA):** a combinational logic device that adds two binary bits and issues a sum bit and a carry-out bit.
- Half subtractor:** a combinational logic device that subtracts one binary bit from another and issues a difference bit and a borrow-out bit.
- Hamming distance:** as used in this text, the number of state variables required to change during a given state-to-state transition in an FSM.
- Handshake interface:** a configuration between two devices whereby the outputs of one device are the inputs to the other and vice versa.
- Hang state:** an isolated state in which an FSM can reside stably but which is not part of the authorized routine.
- Hardware description language (HDL):** a high-level programming language with specialized structures for modeling hardware.
- Hazard:** a glitch or unauthorized transition that is caused by an asymmetric path delay via an inverter, gate, or lead during a logic operation.
- Hazard cover:** the redundant cover that removes a static hazard.
- HDL:** hardware description language.
- Heuristic:** by empirical means or by discovery.
- Hexadecimal (hex):** a base 16 number system in which alphanumeric symbols are used to represent 4-bit binary numbers 0000 through 1111. (See Binary coded hexadecimal.)
- Hold condition:** branching from a given state back into itself or the input requirements necessary to effect such branching action.
- Holding register:** a PIPO (storage) register that is used to filter output signals.
- Hold time:** the interval of time immediately following the transition point during which the data inputs must remain logically stable to ensure that the intended transition of the FSM will be successfully completed.
- Hole:** the absence of a valence electron—the majority carrier in a p-type conducting semiconductor.
- HV:** high voltage.
- Hybrid function:** any function containing both SOP and POS terms.
- IC:** integrated circuit.
- ICS:** iterated carry-save.
- Implicant:** a term in a reduced or minimized expression.
- Inactive:** not active and implying logic 0.
- Inactive state:** the logic 0 state of a logic device.
- Inactive transition point:** the point in a voltage waveform where a digital device passes from the active state to the inactive state.
- Incompatibility:** a condition where the input to a logic device and the input requirement of that device are of opposite activation levels.
- Incompletely specified function:** a function that contains nonessential minterms or maxterms (see Don't care).
- Increment:** to increase usually by 1.

- Indirect address approach:** an alternative approach to FSM design where PS feedback to the NS logic is by way of a converter for the purpose of reducing MUX or PLD size.
- Inertial delay element:** a delay circuit based mainly on an  $R$ - $C$  component.
- Initialize:** to drive a logic circuit into a beginning or reference state.
- Input:** a signal or line into a logic device that controls the operation of that device.
- Input/state map:** a K-map, with inputs as the axes and state identifiers as cell entries, that can be used to determine if the sum rule and the mutually exclusive requirement of any state in an FSM have been violated.
- Integrated circuit (IC):** an electronic circuit that is usually constructed entirely on a single small semiconductor chip called a monolith.
- Intersection:** AND operation.
- Inversion:** the inverting of a signal from HV to LV or vice versa.
- Inverter:** a physical device that performs inversion.
- Involution:** double complementation of a variable or function.
- I/O:** input/output.
- IOB:** I/O block.
- Irredundant:** not redundant, as applied to an absolute minimum Boolean expression.
- Irrelevant input:** an input whose presence in a function is nonessential.
- Island:** a K-map entry that must be looped out of a single cell.
- Iterative:** repeated many times to achieve a specific goal.
- JEDEC:** Joint Electron Device Engineering Council as it pertains to PLD programming format.
- JK flip-flop:** a type of flip-flop that can perform the set, reset, hold, and toggle operations.
- Juxtapose:** to place side by side.
- Karnaugh map (K-map):** graphical representation of a logic function named after M. Karnaugh (1953).
- Keyword:** a word specific to a given HDL.
- Kirchhoff's current law:** the algebraic sum of all currents into a circuit element or circuit section must be zero.
- Kirchhoff's voltage law:** the algebraic sum of all voltages around a closed loop must be zero.
- K-map:** Karnaugh map.
- LAC:** look-ahead-carry (see also CLA).
- Large-scale integrated circuits (LSI):** IC chips that contain 200 to thousands of gates.
- Latch:** a name given to certain types of memory elements as, for example, the D latch.
- Latency:** the time (usually in clock cycles) required to complete an operation in a sequential machine.
- LCA:** logic cell array.
- LD:** mnemonic for load.
- Least significant bit (LSB):** the bit (usually at the extreme right) of a binary word that has the lowest positional weight.
- LED:** light-emitting diode.
- Level:** a term used when specifying the number of gate path delays of a logic function (from input to output) usually exclusive of inverters. See, for example, two-level logic.
- Level triggered:** rising edge triggered (RET) or falling edge triggered (FET).
- Linear state machine:** an FSM with a linear array of states.

- Line driver:** a device whose purpose it is to boost and sharpen a signal so as to avoid fan-out problems.
- LFSR:** linear feedback shift register.
- LFSR counter:** a counter, consisting of an LFSR, that can sequence through a unique set of pseudorandom states controlled by external inputs.
- Logic:** the computational capability of a digital device that is interpreted as either a logic 1 or logic 0.
- Logic adjacency:** two logic states whose state variables differ from each other by only one bit.
- Logic cell:** a configurable logic block (CLB).
- Logic circuit:** a digital circuit that performs the electrical equivalent of some logic function or process.
- Logic diagram:** a digital circuit schematic consisting of an interconnection of logic symbols.
- Logic family:** a particular technology such as TTL or CMOS that is used in the production of ICs.
- Logic instability:** the inability of a logic circuit to maintain a stable logic condition. Also, an oscillatory condition in an asynchronous FSM.
- Logic level:** logic status indicating either positive logic or negative logic.
- Logic level conversion:** the act of converting from positive logic to negative logic or vice versa.
- Logic map:** any of a variety of graphical representations of a logic function.
- Logic noise:** undesirable signal fluctuations produced within a logic circuit following input changes.
- Logic state:** a unique set of binary values that characterize the logic status of a machine at some point in time.
- Logic waveform:** a rectangular waveform between active and inactive states.
- Look-ahead-carry (LAC):** the feature of a “fast” adder that anticipates the need for a carry and then generates and propagates it more directly than does a parallel adder (see also carry look-ahead).
- Loop-out:** the action that identifies a prime implicant in a K-map.
- Loop-out protocol:** a minimization procedure whereby the largest 2 group of logically adjacent minterms or maxterms are looped out in the order of increasing  $n$  ( $n = 0, 1, 2, 3, \dots$ ).
- LPD:** lumped path delay.
- LPDD:** lumped path delay diagram.
- LSB:** least significant bit.
- LSD:** least significant digit.
- LSI:** large-scale integration.
- Lumped path delay diagram (LPDD):** a diagram that replaces discrete gates with other logic symbols for the purpose of comparing path delays from input to output.
- Lumped path delay (LPD) model:** a model, applicable to FSMs that operate in the fundamental mode, that is characterized by a lumped memory element for each state variable/feedback path.
- LV:** low voltage.
- Magnitude comparator:** comparator.

- Majority function:** a function that becomes active when a majority of its variables become active.
- Majority gate:** a logic gate that yields a majority function.
- Mantissa:** the fraction part of a floating point number.
- Map:** usually a Karnaugh map.
- Map compression:** a reduction in the order of a K-map.
- Map key:** the order of K-map compression; hence,  $2^{N-n}$ , where  $N$  is the number of variables in the function to be mapped and  $n$  is the order of the K-map to be used.
- Mapping algorithm:** In FSM design, the procedure to obtain the NS functions by ANDing the memory input logic value in the excitation table with the corresponding branching condition in the state diagram for the FSM to be designed, and entering the result in the appropriate cell of the NS K-map.
- Master/slave (MS) flip-flop:** a flip-flop characterized by a master (input) stage and a slave (output) stage that are triggered by clock antiphase to each other.
- Mask:** to prevent information from passing a certain point in a given process.
- Mask programmed:** refers to the bit patterns produced in a PLD chip at the foundry.
- Maxterm:** a POS term that contains all the variables of the function.
- Maxterm code:** a code in which complemented variables are assigned logic 1 and uncomplemented variables are assigned logic 0 — the opposite of minterm code.
- Mealy machine:** an FSM that conforms to the Mealy model.
- Mealy model:** the general model for a sequential machine where the output state depends on the input state as well as the present state.
- Mealy output:** a conditional output.
- Medium-scale integrated circuits (MSI):** IC chips that contain 20 to 200 gates according to one convention.
- Memory:** the ability of a digital device to store and retrieve binary words on command.
- Memory element:** a device for storing and retrieving one bit of information on command. In asynchronous FSM terminology, a fictitious lumped path delay.
- Merge:** the concatenation of buses to form a larger bus.
- Merging of states:** in a state diagram, the act of combining states to produce fewer states.
- Metal-oxide-semiconductor:** the material constitution of an important logic family (MOS) used in IC construction.
- Metastability:** an unresolved state of an FSM that resides between a Set and a Reset condition or that is logically unstable.
- Metastable exit time:** the time interval between entrance into and exit from the metastable state.
- MEV:** Map entered variable.
- Minimization:** the process of reducing a logic function to its simplest form.
- Minimum cover:** the optimally reduced representation of a logic expression.
- Minterm:** a term in an SOP expression where all variables of the expression are represented in either complemented or uncomplemented form.
- Minterm code:** a logic variable code in which complemented variables are assigned logic 0 while uncomplemented variables are assigned logic 1 — the opposite of maxterm code.
- Minuend:** the operand from which the subtrahend is subtracted in a subtraction operation.
- Mixed logic:** the combined use of the positive and negative logic systems.

**Mixed-rail output:** dual, logically equal outputs of a device (e.g., a flip-flop) where one output is issued active high while the other is issued active low, but the two are not issued simultaneously.

**Mnemonic:** a short single group of symbols (usually letters) that are used to convey a meaning.

**Mnemonic state diagram:** a fully documented state diagram.

**Model:** the means by which the major components and their interconnections are represented for a digital machine or system.

**Module:** a device that performs a specific function and that can be added to or removed from a system to alter the system's capability. A common example is a full adder.

**Modulus- $n$  counter:** (see divide-by- $n$  counter)

**Monad:** a minterm (or maxterm) that is not logically adjacent to any other minterm (or maxterm).

**Moore machine:** a sequential machine that conforms to the Moore model.

**Moore model:** a degenerate form of the Mealy (general) model in which the output state depends only on the present state.

**Moore output:** an unconditional output.

**MOS:** metal-oxide-semiconductor.

**MOSFET:** metal-oxide-semiconductor field effect transistor.

**Most significant bit (MSB):** the extreme left bit of a binary word that has the highest positional weight.

**MSB:** most significant bit.

**MSD:** most significant digit.

**MSI:** medium scale integration.

**MTBF:** mean time between failures.

**Muller C module:** a rendezvous module (RMOD).

**Multilevel logic minimization:** minimization involving more than two levels of path delay as, for example, that resulting from XOR-type patterns in K-maps.

**Multiple-output minimization:** optimization of more than one output expression from the same logic device.

**Multiplex:** to select or gate (on a time-shared basis) data from two or more sources onto a single line or transmission path.

**Multiplexer:** a device that multiplexes data.

**Multiplicand:** the number being multiplied by the multiplier.

**Multiplier:** a combinational logic device that will multiply two binary numbers. Also, the number being used to multiply the multiplicand.

**Mutually exclusive requirement:** a requirement in state diagram construction that forbids overlapping branching conditions (BCs)—i.e., it forbids the use of BCs shared between two or more branching paths.

**MUX:** multiplexer.

**NAND-centered basic cell:** cross-coupled NAND gates forming a basic cell.

**NAND gate:** a physical device that performs the electrical equivalent of the NOT AND function.

**NAND/INV logic:** combinational logic consisting exclusively of NAND gates and inverters.

**Natural binary code:** a code for which the bits are positioned in a binary word according to their positional weight in polynomial notation.

- Natural binary coded decimal:** a 4-bit, 10-word code that is weighted 8, 4, 2, 1 and that is used to represent decimal numbers. Same as binary code.
- NBCD:** natural binary coded decimal. Same as binary coded decimal (BCD).
- n-channel:** an n-type conducting region in a p-type substrate.
- Negative logic:** a logic system in which high voltage (HV) corresponds to logic 0 and low voltage (LV) corresponds to logic 1. The opposite of positive logic.
- Negative pulse:** a 1–0–1 pulse.
- Nested cell:** a basic cell that is used as the memory in an asynchronous FSM design.
- Nested machine:** any asynchronous machine that serves as the memory in the design of a larger sequential machine. Any FSM that is embedded within another.
- Next state (NS):** a state that follows the present state in a sequence of states.
- Next state forming logic:** the logic hardware in a sequential machine whose purpose it is to generate the next state function input to the memory.
- Next state function:** the logic function that defines the next state of an FSM given the present state.
- Next state map:** a composite K-map where the entries for each cell are the next state functions for the present state represented by the coordinates of that cell (see flow table).
- Next state variable:** the variable representing the next state function.
- Nibble:** a group of four bits.
- NMH:** noise margin high — the lower voltage limit of logic 1 and the upper boundary of the uncertainty region.
- NML:** noise margin low — the upper voltage limit of logic 0 and the lower boundary of the uncertainty region.
- NMOS:** an n-channel MOSFET.
- Noise immunity:** the ability of a logic circuit to reject unwanted signals.
- Noise margin:** the maximum voltage fluctuation that can be tolerated in a digital signal without crossing the switching threshold of the switching device.
- Non-restoring logic:** logic that consists of passive switching devices such as diodes or transmission gates that cannot amplify but that dissipate power.
- Nonvolatile:** refers to memory devices that require no power supply to retain information in memory.
- NOR-centered basic cell:** cross-coupled NOR gates forming a basic cell.
- NOR gate:** a physical device that performs the electrical equivalent of the NOT OR function.
- NOR/INV logic:** combinational logic consisting exclusively of NOR gates and inverters.
- NOT function:** an operation that is the logic equivalent of complementation.
- NOT laws:** a set of Boolean identities based on the NOT function.
- npn:** refers to a BJT having a p-type semiconductor base and an n-type semiconductor collector and emitter.
- NS:** next state.
- Octad:** a grouping of eight logically adjacent minterms or maxterms.
- Octal:** a base 8 number system in which numbers 1 through 7 are used to represent 3-bit binary numbers 000 through 111. (See Binary coded octal.)
- Odd parity:** an odd number of 1's or 0's depending on how odd parity is defined.
- Offset pattern:** an XOR pattern in a K-map in which identical subfunctions are located in two nondiagonal cells that differ in cell coordinates by two bits.

**Ohm's law:** voltage is linearly proportional to current,  $V = RI$ , where  $R$  is the constant of proportionality called the resistance (in ohms).

**One-hot code:** a nonweighted code in which there exists only one 1 in each word of the code.

**One-hot design method:** use of the one-hot code for synchronous and asynchronous FSM design.

**One-hot-plus-zero:** one-hot code plus the all-zero state.

**One's complement:** a system of binary arithmetic in which a negative number is represented by complementing each bit of its positive equivalent.

**Operand:** a number or quantity that is to be operated on.

**Operation table:** a table that defines the functionality of a flip-flop or some other device.

**Operator:** a Boolean connective.

**OPI:** optional prime implicant.

**Optional prime implicant (OPI):** a prime implicant whose presence in a minimum function produces alternative minimum cover.

**OR:** an operator requiring that the output of an OR gate be active if one or more of its inputs are active.

**OR-AND-Invert gate:** a physical device, usually consisting of two OR gates and one NAND gate, that performs the electrical equivalent of POS with an active low output.

**Order:** refers to the number of variables on the axes of a K-map.

**OR function:** a function that derives from the definition of OR.

**ORG:** output race glitch.

**OR gate:** a physical device that performs the electrical equivalent of the OR function.

**OR laws:** a set of Boolean identities based on the OR function.

**OR plane:** the ORing stage of a PLD.

**Outbranching:** branching from a state exclusive of the hold branching condition.

**Output:** a concluding signal issued by a digital device.

**Output forming logic:** the logic hardware in a sequential machine whose purpose it is to generate the output signals.

**Output holding register:** a register, consisting of D flip-flops, that is used to filter out output logic noise.

**Output race glitch (ORG):** an internally initiated function hazard that is produced by a race condition in a sequential machine.

**Overflow error:** a false magnitude or sign that results from a left shift in a shifter when there are insufficient word bit positions at the spill end.

**Packing density:** the practical limit to which switches of the same logic family can be packed in an IC chip.

**PAL:** programmable array logic (registered trademark of Advanced Micro Devices, Inc.).

**PALU:** programmable arithmetic and logic unit.

**Parallel adder:** a cascaded array of full adders where the carry-out of a given full adder is the carry-in to the next most significant stage full adder.

**Parallel load:** the simultaneous loading of data inputs to devices such as registers and counters.

**Parity:** related to the existence of an even or odd number of 1's or 0's in a binary word.

**Parity bit:** a bit appended to a binary word to detect, create, or remove even or odd parity.

**Parity detector:** a combinational logic device that will detect an even (or odd) number of 1's (or 0's) in a binary word.

- Parity generator:** a combinational logic device that will append a logic 1 (or logic 0) to a binary word so as to generate an even (or odd) number of 1's (or 0's).
- Passive device:** any device that is incapable of producing voltage or current gain and, thus, only dissipates power.
- Pass transistor switch:** a MOS transistor switch that functions as a nonrestoring switching device and that does not invert a voltage signal. A transmission gate.
- PCB:** printed circuit board.
- p-channel:** a p-type conducting region in an n-type substrate.
- PDF:** portable document format.
- PDP:** power–delay product.
- PE:** priority encoder.
- Period:** the time in seconds (s) between repeating portions of a waveform; hence, the inverse of the frequency.
- Physical truth table:** an I/O specification table based on a physically measurable quantity such as voltage.
- PI:** prime implicant.
- Pipeline:** a processing scheme where each task is allocated to specific hardware (joined in a line) and to a specific time slot.
- PIPO:** parallel-in/parallel-out operation mode of a register.
- PISO:** parallel-in/serial-out operation mode of a register.
- PLA:** programmable logic array.
- Planar format:** a two-dimensional K-map array used to minimize functions of more than four variables.
- PLD:** programmable logic device.
- PLS:** programmable logic sequencer.
- PMOS:** a p-channel MOSFET.
- p–n junction diode:** (see Diode)
- pnp:** refers to a BJT having an n-type semiconductor base and a p-type semiconductor emitter and collector.
- Polarized mnemonic:** a contracted signal name onto which is attached an activation level indicator.
- Port:** an entry or exit element to an entity (e.g., the name given to an input signal in a VHDL declaration).
- POS:** product-of-sums.
- POS hazard:** a static 0-hazard.
- Positional weighting:** a system in which the weight of a bit in a binary word is determined by its polynomial representation.
- Positive logic:** the logic system in which HV corresponds to logic 1 and LV corresponds to logic 0.
- Positive pulse:** a 0–1–0 pulse.
- Power,  $P$ :** the product of voltage,  $V$ , and current,  $I$ , given in units of watts (W).
- Power–delay product (PDP):** the average power dissipated by a logic device multiplied by its propagation delay time.
- PR or PRE:** preset.
- Present state (PS):** the logic state of an FSM at a given instant.
- Present state/next state (PS/NS) table:** a table that is produced from the next state K-maps and that is used to construct a fully documented state diagram in an FSM analysis.



- Preset:** an asynchronous input that is used in flip-flops to set them to a logic 1 condition.
- Prime implicant (PI):** a group of adjacent minterms or maxterms that are sufficiently large that they cannot be combined with other groups in any way to produce terms of fewer variables.
- Primitive:** a discrete logic device such as a gate, MUX, or decoder.
- Priority encoder:** a logic device that generates a coded output based on a set of prioritized data inputs.
- Product-of-sums (POS):** the ANDing of ORed terms in a Boolean expression.
- Programmable logic array (PLA):** any PLD that can be programmed in both the AND and OR planes.
- Programmable logic device (PLD):** any two-level, combinational array logic device from the families of ROMs, PLAs, PALs or FPGAs, etc.
- Programmable read-only memory (PROM):** a once-only user-programmable ROM.
- PROM:** programmable read-only memory.
- Propagation delay:** in a logic device, the time interval of an output response to an input signal.
- PS:** present state.
- PS/NS:** present state/next state.
- P-term:** a Boolean product term—one consisting only of ANDed literals.
- P-term table:** a table that consists of p-terms, inputs, and outputs and that is used to program PLA-type devices.
- Pull-down resistor:** a resistor that causes a signal on a line to remain at low voltage.
- Pull-up resistor:** a resistor that causes a signal on a line to remain at high voltage.
- Pulse:** an abrupt change from one level to another followed by an opposite abrupt change.
- Pulse mode:** an operational condition for an asynchronous FSM where the inputs are required to be nonoverlapping pulse signals.
- Pulse width:** the active duration of a positive pulse or the inactive duration of a negative pulse.
- Quad:** a grouping of four logically adjacent minterms or maxterms.
- Quadratic convergence:** a process as in “fast division” whereby the error per iteration decreases according to the inverse square law.
- Quotient:** the result of a division operation.
- R:** reset.
- Race condition:** a condition in a sequential circuit where the transition from one state to another involves two or more alternative paths.
- Race gate:** the gate to which two or more input signals are in race contention.
- Race path:** any path that can be taken in a race condition.
- Race state:** any state through which an FSM may transit during a race condition.
- Radix:** the number of unique symbols in a number system — same as the base of a number system.
- RAM:** random access memory.
- Random access memory (RAM):** a read/write memory system in which all memory locations can be accessed directly independent of other memory locations.
- R–C:** resistance/capacitance or resistor/capacitor.
- Read only memory (ROM):** a PLD that can be mask programmed only in the OR plane.

- Read/write memory (RWM):** a memory array (e.g., RAM) that can be used to store and retrieve information at any time.
- Redundant cover:** nonessential and nonoptional cover in a function representation.
- Redundant prime implicant:** a prime implicant that yields redundant cover.
- Reflective code:** a code that has a reflection (mirror) plane midway through the code.
- Register:** a digital device, configured with flip-flops and other logic, that is capable of storing and shifting data on command.
- Remainder:** in division, the dividend minus the product of the divisor and the quotient.
- Rendezvous module:** an asynchronous state machine whose output becomes active when all external inputs become active and becomes inactive when all external inputs become inactive.
- Reset:** a logic 0 condition or an input to a logic device that sets it to a logic 0 condition.
- Residue:** the part of term that remains when the coupled variable is removed (see consensus term).
- Resistance,  $R$ :** the voltage drop across a conducting element divided by current through the element (in ohms).
- Resistor-transistor logic:** a logic family that consists of BJTs and resistors.
- Restoring logic:** logic consisting of switching devices such as BJTs and MOSFETs that can amplify.
- RET:** rising edge triggered.
- Reverse bias:** a voltage applied to a  $p$ - $n$  junction diode in a direction that minimizes conduction across the junction.
- Reverse saturation current:** the current through a  $p$ - $n$  junction diode under reverse bias.
- Ring counter:** a configuration of shift registers that generates a one-hot code output.
- Ripple carry (R-C):** the process by which a parallel adder transfers the carry from one full adder to another.
- Ripple counter:** a counter whose flip-flops are each triggered by the output of the next LSB flip-flop.
- Rise time:** the period of time it takes a voltage (or current) signal to change from 10% to 90% of its high value.
- Rising edge triggered (RET):** activation of a logic device on the rising edge of the triggering variable.
- RMOD:** rendezvous module.
- ROM:** read-only memory.
- Round-off error:** the amount by which a magnitude is diminished due to an underflow or spill-off in a shifter undergoing a right shift.
- RPI:** redundant prime implicant.
- RTL:** resistor-transistor logic.
- Runt pulse:** any pulse that barely reaches the switching threshold of a device into which it is introduced.
- S:** set. Also, the source terminal of a MOSFET.
- Sampling interval:** sum of the setup and hold times.
- Sampling variable:** the last variable to change in initiating a state-to-state transition in an FSM.
- Sanity circuit:** a circuit that is used to initialize an FSM into a particular state, usually a resistor/capacitor ( $R$ - $C$ ) type circuit.

- Saturation mode:** the physical state of a BJT in which collector current is permitted to flow.
- Schmitt trigger:** an electronic gate with hysteresis and high noise immunity that is used to “square up” pulses.
- Selector module:** a device whose function it is to steer one of two input signals to either one of two outputs depending on whether a specific input is active or inactive.
- Self-correcting counter:** a counter for which all states lead into the main count sequence or routine.
- Sequence detector (recognizer):** a sequential machine that is designed to recognize a particular sequence of input signals.
- Sequential machine:** any digital machine with feedback paths whose operation is a function of both its history and its present input data.
- Set:** a logic 1 condition or an input to a logic device that sets it to a logic 1 condition.
- Setup time:** the interval of time prior to the transition point during which all data inputs must remain stable at their proper logic level to ensure that the intended transition will be initiated.
- S-hazard:** a static hazard.
- Shift register:** a register that is capable of shifting operations.
- Shift:** the movement of binary words to the left or right in a shifter or shift register.
- Shifter:** a combinational logic device that will shift or rotate data asynchronously upon presentation.
- Sign bit:** a bit appended to a binary number (usually in the MSB position) for the purpose of indicating its sign.
- Sign-complement arithmetic:** 1’s or 2’s complement arithmetic.
- Sign-magnitude representation:** a means of identifying positive and negative binary numbers by a sign and magnitude.
- Single transition time (STT):** a state-to-state transition in an asynchronous FSM that occurs in the shortest possible time, that is, without passing through a race state.
- SIPO:** serial-in/parallel-out operation mode of a register.
- SISO:** serial-in/serial-out operation mode of a register.
- Slice:** that part of a circuit or device that can be cascaded to produce a larger circuit or device.
- Small-scale integration:** IC chips that, by one convention, contain up to 20 gates.
- SOP:** sum-of-products.
- SOP hazard:** a static 1-hazard.
- Source:** one of three terminals of a MOSFET. The origin of a digital signal.
- Spill bit:** the bit in a shifter or shift register that is spilled off (lost) in a shifting operation.
- SPDT switch:** single-pole/double-throw switch.
- SPST switch:** single-pole/single-throw switch.
- Square wave:** a rectangular waveform.
- SRAM:** static RAM.
- SSI:** small-scale integration.
- Stability criteria:** the requirements that determine if an asynchronous FSM, operated in the fundamental mode, is stable or unstable in a given state.
- Stable state:** any logic state of an asynchronous FSM that satisfies the stability criteria.
- Stack format:** a three-dimensional array of conventional fourth-order K-maps used for function minimization of more than four variables.

- State:** a unique set of binary values that characterize the logic status of a machine at some point in time.
- State adjacency set:** any  $2^n$  set of logically adjacent states of an FSM.
- State code assignment:** unique set of code words that are assigned to an FSM to characterize its logic status.
- State diagram:** the diagram or chart of an FSM that shows the state sequence, branching conditions, and output information necessary to describe its sequential behavior.
- State machine:** a finite state machine (FSM). A sequential machine.
- State identifier:** any symbol (e.g., alphabetical) that is used to represent or identify a state in a state diagram.
- State table:** tabular representation of a state diagram.
- State transition table:** (see excitation table).
- State variable:** any variable whose logic value contributes to the logic status of a machine at any point in time. Any bit in the state code assignment of a state diagram.
- Static hazard:** an unwanted glitch in an otherwise steady-state signal that is produced by an input change propagating along asymmetric path delays through inverters or gates.
- Static-1 hazard:** a glitch that occurs in an otherwise steady-state 1 output signal from SOP logic due to a change in an input for which there are two asymmetric paths (delay-wise) to the output.
- Static-0 hazard:** a glitch that occurs in an otherwise steady-state 0 output signal from POS logic due to a change in an input for which there are two asymmetric paths (delay-wise) to the output.
- Static RAM:** a nonvolatile form of RAM — does not need periodic refreshing to hold its information.
- Steering logic:** logic based primarily on transmission gate switches.
- S-term:** a Boolean sum term — one containing only ORed literals.
- Stretcher:** an input conditioning device that catches a short input signal and stretches it.
- STT:** single transition time.
- Stuck-at fault:** an input to a logic gate that is permanently stuck at logic 0 or logic 1 because of a shorted connection, an open connection, or a connection to either ground or a voltage supply.
- Substrate:** the supporting or foundation material in and on which a semiconductor device is constructed.
- Subtractor:** a digital device that subtracts one binary word from another to give a difference and borrow.
- Subtrahend:** the operand being subtracted from the minuend in a subtraction operation.
- Sum-of-products (SOP):** the ORing of ANDed terms in a Boolean expression.
- Sum rule:** a rule in state diagram construction that requires that all possible branching conditions be accounted for.
- Switching speed:** a device parameter that is related to its propagation delay time.
- Synchronizer circuit:** a logic circuit (usually a D flip-flop) that is used to synchronize an input with respect to a clock signal.
- Synchronous machine:** a sequential machine that is clock driven.
- Synchronous parallel load:** parallel loading of a register or counter via a clock signal to the flip-flops.
- System level design:** a design that includes controller and data path sections.
- Tabular minimization:** a minimization procedure that uses tables exclusively.

- T flip-flop:** a flip-flop that operates in either the toggle or hold mode.
- TG:** transmission gate.
- Throughput:** the time required to produce an output response due to an input change.
- Time constant:** the product of resistance and capacitance given in units of seconds (s) — a measure of the recovery time of an  $R$ - $C$  circuit.
- Timing diagram:** a set of logic waveforms showing the time relationships between two or more logic signals.
- Toggle:** repeated but controlled transitions between any two states, as between the Set and Reset states.
- Toggle module:** a flip-flop that is configured to toggle only. Also, a divide-by-2 counter.
- Transfer characteristic:** for a transistor switch, a plot of current ( $I$ ) vs voltage ( $V$ ).
- Trans-HI module:** a transparent high (RET) D latch.
- Trans-LO module:** a transparent low (FET) D latch.
- Transistor:** a three-terminal switching device that exhibits current or voltage gain.
- Transistor-transistor logic:** a logic family in which bipolar junction transistors provide both logic decision and current gain.
- Transition:** in a digital machine, a change from one state (or level) to another.
- Transmission gate:** a pass transistor switch.
- Transparent D latch:** a two-state D flip-flop in which the output,  $Q$ , tracks the input,  $D$ , when clock is active if RET or when clock is inactive if FET.
- Tree:** combining of like gates, usually to overcome fan-in limitations.
- Triggering threshold:** the point beyond which a transition takes place.
- Triggering variable:** sampling (enabling) variable.
- Tri-state bus:** as used in this text, the wire-ORed output lines from a multiplexed scheme of PLDs having tri-state enables. Note: tri-state is a registered trademark of NSC.
- Tri-state driver:** an active logic device that operates in either a disconnect mode or an inverting (or noninverting) mode. Also, three-state driver. Note: tri-state is a registered trademark of NSC.
- True hold:** the condition whereby a device can sustain the same logic output values over any number of clock cycles independent of its input logic status.
- Truth table:** a table that provides an output value for each possible input condition to a combinational logic device.
- TTL:** transistor-transistor (BJT) logic.
- Twisted ring counter:** a configuration of shift registers that generates a creeping code output.
- Two-level logic:** logic consisting of only one ANDing and one ORing stage.
- Two-phase clocking:** two synchronized clock signals that have nonoverlapping active or nonoverlapping inactive waveforms.
- Two's complement:** one's complement plus one added to the LSB.
- Unconditional branching:** state-to-state transitions that take place independent of the input status of the FSM.
- Unconditional output:** an output of an FSM that does not depend on an input signal.
- Union:** OR operation.
- Unit distance code:** a code in which each state in the code is surrounded by logically adjacent states.
- Universal flip-flop:** a JK flip-flop.
- Universal gate:** a NAND or NOR gate.

**Universal shift register:** a shift register capable of performing PIPO, PISO, SIPO, and SISO operations in addition to being capable of performing the true hold condition.

**Unstable state:** any logic state in an asynchronous FSM that does not satisfy the stability criteria.

**Unweighted code:** a code that cannot be constructed by any mathematical weighting procedure.

**USR:** universal shift register.

**UVEPROM:** ultraviolet erasable PROM.

**VEM:** variable entered map.

**Very large scale integrated circuits:** IC chips that contain thousands to millions of gates.

**VHDL:** VHSIC hardware description language.

**VHSIC:** very high speed integrated circuit.

**VLSI:** very large scale integrated circuits.

**Voltage, V:** the potential difference between two points, in units of volts (V). Also, the work required to move a positive charge against an electric field.

**Voltage waveform:** a voltage waveform in which rise and fall times exist.

**Weighted code:** a binary code in which the bit positions are weighted with different mathematically determined values.

**Wired logic:** an arrangement of logic circuits in which the outputs are physically connected to form an “implied” AND or OR function.

**WSI circuits:** wafer-scale integrated circuits.

**XNOR:** (see Equivalence and EQV)

**XOR:** exclusive OR.

**XOR function:** the function that derives from the definition of exclusive OR.

**XOR gate:** a physical device that performs the electrical equivalent of the XOR function.

**XOR laws:** a set of Boolean identities that are based on the XOR function.

**XOR pattern:** any of four possible K-map patterns that result in XOR type functions.

**XS3 code:** BCD code plus three.

**ZBI:** zero-blanking input.

**ZBO:** zero-blanking output.

**Zero banking:** a feature of a BCD-to-seven-segment conversion that blanks out the seven-segment display if all inputs are zero.

*This page intentionally left blank*

# Number Systems, Binary Arithmetic, and Codes

---

## 2.1 INTRODUCTION

Number systems provide the basis for conveying and quantifying information. Weather data, stocks, pagination of books, weights and measures — these are just a few examples of the use of numbers that affect our daily lives. For this purpose we find the decimal (or Arabic) number system to be reliable and easy to use. This system evolved presumably because early humans were equipped with a crude type of calculator, their 10 fingers. But a number system that is appropriate for humans may be intractable for use by a machine such as a computer. Likewise, a number system appropriate for a machine may not be suitable for human use.

Before concentrating on those number systems that are useful in computers, it will be helpful to review those characteristics that are desirable in any number system. There are *four* important characteristics in all:

- Distinguishability of symbols
- Arithmetic operations capability
- Error control capability
- Tractability and speed

To one degree or another the decimal system of numbers satisfies these characteristics for hard-copy transfer of information between humans. Roman numerals and binary are examples of number systems that do not satisfy all four characteristics for human use. On the other hand, the binary number system is preferable for use in digital computers. The reason is simply put: current digital electronic machines recognize only two identifiable states, physically represented by a high voltage level and a low voltage level. These two physical states are logically interpreted as binary symbols 1 and 0.

A fifth desirable characteristic of a number system to be used in a computer should be that it have a minimum number of easily identifiable states. The binary number system satisfies this condition. However, the digital computer must still interface with humankind. This is done by converting the binary data to a decimal and character-based form that can



be readily understood by humans. A minimum number of identifiable characters (say 1 and 0, or true and false) is not practical or desirable for direct human use. If this is difficult to understand, imagine trying to complete a tax form in binary or in any number system other than decimal. On the other hand, use of a computer for this purpose would not only be practical but, in many cases, highly desirable.

## 2.2 POSITIONAL AND POLYNOMIAL REPRESENTATIONS

The *positional form* of a number is a set of side-by-side (juxtaposed) digits given generally in *fixed-point* form as

$$N_r = \underbrace{(a_{n-1} \cdots a_2 a_1 a_0)}_{\text{Integer}} \cdot \underbrace{(a_{-1} a_{-2} a_{-3} \cdots a_{-m})}_\text{Fraction}_r \quad (2.1)$$

$\begin{array}{ccc} \text{MSD} & \text{Radix} & \text{LSD} \\ \downarrow & \text{Point} & \downarrow \end{array}$

where the radix (or base),  $r$ , is the total number of digits in the number system, and  $a$  is a digit in the set defined for radix  $r$ . Here, the radix point separates  $n$  integer digits on the left from  $m$  fraction digits on the right. Notice that  $a_{n-1}$  is the most significant (highest order) digit called MSD, and that  $a_{-m}$  is the least significant (lowest order) digit denoted by LSD.

The *value* of the number in Eq. (2.1) is given in *polynomial form* by

$$N_r = \sum_{i=-m}^{n-1} a_i r^i = (a_{n-1} r^{n-1} + \cdots + a_2 r^2 + a_1 r^1 + a_0 r^0 + a_{-1} r^{-1} + a_{-2} r^{-2} + \cdots + a_{-m} r^{-m})_r, \quad (2.2)$$

where  $a_i$  is the digit in the  $i$ th position with a *weight*  $r^i$ .

Applications of Eqs. (2.1) and (2.2) follow directly. For the decimal system  $r = 10$ , indicating that there are 10 distinguishable characters recognized as decimal numerals 0, 1, 2, ...,  $r - 1 (= 9)$ . Examples of the positional and polynomial representations for the decimal system are

$$\begin{aligned} N_{10} &= (d_3 d_2 d_1 d_0 \cdot d_{-1} d_{-2} d_{-3})_{10} \\ &= 3017.528 \end{aligned}$$

and

$$\begin{aligned} N_{10} &= \sum_{i=-3}^{n-1} d_i 10^i \\ &= 3 \times 10^3 + 0 \times 10^2 + 1 \times 10^1 + 7 \times 10^0 + 5 \times 10^{-1} + 2 \times 10^{-2} + 8 \times 10^{-3} \\ &= 3000 + 10 + 7 + 0.5 + 0.02 + 0.008, \end{aligned}$$

where  $d_i$  is the decimal digit in the  $i$ th position. Exclusive of possible leading and trailing zeros, the MSD and LSD for this number are 3 and 8, respectively. This number could have been written in a form such as  $N_{10} = 03017.52800$  without altering its value but implying greater accuracy of the fraction portion.

## 2.3 UNSIGNED BINARY NUMBER SYSTEM

Applying Eqs. (2.1) and (2.2) to the binary system requires that  $r = 2$ , indicating that there are two distinguishable characters, typically 0 and  $(r - 1) = 1$ , that are used. In positional representation these characters (numbers) are called *binary digits* or *bits*. Examples of the positional and polynomial notations for a binary number are

$$\begin{aligned}
 N_2 &= (b_{n-1} \cdots b_3 b_2 b_1 b_0 \cdot b_{-1} b_{-2} b_{-3} \cdots b_{-m})_2 \\
 &= 1 \ 0 \ 1 \ 1 \ 0 \ 1 \ . \ 1 \ 0 \ 1_2 \\
 &\quad \uparrow \qquad \qquad \qquad \uparrow \\
 &\quad \text{MSB} \qquad \qquad \qquad \text{LSB}
 \end{aligned}$$

and

$$\begin{aligned}
 N &= \sum_{i=-m}^{n-1} b_i 2^i \\
 &= 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 \\
 &\quad + 1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} \\
 &= 32 + 8 + 4 + 1 + 0.5 + 0.125 \\
 &= 45.625_{10},
 \end{aligned}$$

where  $n = 6$  and  $m = 3$ , and  $b_i$  is the bit in the  $i$ th position. Thus, the bit positions are weighted,  $\dots 16, 8, 4, 2, 1, 1/2, 1/4, 1/8, \dots$  for any number consisting of integer and fraction portions. Binary numbers, so represented, are sometimes referred to as *natural* binary. In positional representation, the bit on the extreme left and extreme right are called the MSB (most significant bit) and LSB (least significant bit), respectively. Notice that by obtaining the value of a binary number, a conversion from binary to decimal has been performed. The subject of radix (base) conversion will be dealt with more extensively in a later section.

For reference purposes, Table 2.1 provides the binary-to-decimal conversion for two-, three-, four-, five-, and six-bit binary. The six-bit binary column is only halfway completed for brevity.

In the natural binary system the number of bits in a unit of data is commonly assigned a name. Examples are:

- 4 data-bit unit — nibble (or half byte)
- 8 data-bit unit — byte
- 16 data-bit unit — two bytes (or half word)
- 32 data-bit unit — word (or four bytes)
- 64 data-bit unit — double-word

**Table 2.1 Binary-to-decimal conversion**

| Two-Bit<br>Binary | Decimal<br>Value | Three-Bit<br>Binary | Decimal<br>Value | Four-Bit<br>Binary | Decimal<br>Value | Five-Bit<br>Binary | Decimal<br>Value | Six-Bit<br>Binary | Decimal<br>Value |
|-------------------|------------------|---------------------|------------------|--------------------|------------------|--------------------|------------------|-------------------|------------------|
| 00                | 0                | 000                 | 0                | 0000               | 0                | 10000              | 16               | 100000            | 32               |
| 01                | 1                | 001                 | 1                | 0001               | 1                | 10001              | 17               | 100001            | 33               |
| 10                | 2                | 010                 | 2                | 0010               | 2                | 10010              | 18               | 100010            | 34               |
| 11                | 3                | 011                 | 3                | 0011               | 3                | 10011              | 19               | 100011            | 35               |
|                   |                  | 100                 | 4                | 0100               | 4                | 10100              | 20               | 100100            | 36               |
|                   |                  | 101                 | 5                | 0101               | 5                | 10101              | 21               | 100101            | 37               |
|                   |                  | 110                 | 6                | 0110               | 6                | 10110              | 22               | 100110            | 38               |
|                   |                  | 111                 | 7                | 0111               | 7                | 10111              | 23               | 100111            | 39               |
|                   |                  |                     |                  | 1000               | 8                | 11000              | 24               | 101000            | 40               |
|                   |                  |                     |                  | 1001               | 9                | 11001              | 25               | 101001            | 41               |
|                   |                  |                     |                  | 1010               | 10               | 11010              | 26               | 101010            | 42               |
|                   |                  |                     |                  | 1011               | 11               | 11011              | 27               | 101011            | 43               |
|                   |                  |                     |                  | 1100               | 12               | 11100              | 28               | 101100            | 44               |
|                   |                  |                     |                  | 1101               | 13               | 11101              | 29               | 101101            | 45               |
|                   |                  |                     |                  | 1110               | 14               | 11110              | 30               | 101110            | 46               |
|                   |                  |                     |                  | 1111               | 15               | 11111              | 31               | 101111            | 47               |
|                   |                  |                     |                  |                    |                  |                    |                  | .                 | .                |
|                   |                  |                     |                  |                    |                  |                    |                  | .                 | .                |
|                   |                  |                     |                  |                    |                  |                    |                  | .                 | .                |

The word size for a computer is determined by the number of bits that can be manipulated and stored in registers. The foregoing list of names would be applicable to a 32-bit computer.

## 2.4 UNSIGNED BINARY CODED DECIMAL, HEXADECIMAL, AND OCTAL

Although the binary system of numbers is most appropriate for use in computers, this system has several disadvantages when used by humans who have become accustomed to the decimal system. For example, binary machine code is long, difficult to assimilate, and tedious to convert to decimal. But there exist simpler ways to represent binary numbers for conversion to decimal representation. Three examples, commonly used, are natural binary coded decimal (BCD), binary coded hexadecimal (BCH), and binary coded octal (BCO). These number systems are useful in applications where a digital device, such as a computer, must interface with humans. The BCD code representation is also useful in carrying out computer arithmetic.

### 2.4.1 The BCD Representation

The BCD system is an 8, 4, 2, 1 weighted code. This system uses patterns of four bits to represent each decimal position of a number and is converted to its decimal equivalent by

**Table 2.2 BCD bit patterns and decimal equivalent**

| BCD<br>Bit Pattern | Decimal | BCD<br>Bit Pattern | Decimal |
|--------------------|---------|--------------------|---------|
| 0000               | 0       | 1000               | 8       |
| 0001               | 1       | 1001               | 9       |
| 0010               | 2       | 1010               | NA      |
| 0011               | 3       | 1011               | NA      |
| 0100               | 4       | 1100               | NA      |
| 0101               | 5       | 1101               | NA      |
| 0110               | 6       | 1110               | NA      |
| 0111               | 7       | 1111               | NA      |

NA = not applicable (code words not valid)

polynomials of the form

$$\begin{aligned}
 N_{10} &= b_3 \times 2^3 + b_2 \times 2^2 + b_1 \times 2^1 + b_0 \times 2^0 \\
 &= b_3 \times 8 + b_2 \times 4 + b_1 \times 2 + b_0 \times 1
 \end{aligned}$$

for any  $b_3b_2b_1b_0$  code integer. Thus, decimal 6 is represented as  $(0 \times 8) + (1 \times 4) + (1 \times 2) + (0 \times 1)$  or 0110 in BCD code. As in binary, the bit positional weights of the BCD code are derived from integer powers of  $2^n$ . Table 2.2 shows the BCD bit patterns for decimal integers 0 through 9.

Decimal numbers greater than nine or less than one can be represented by the BCD code if each digit is given in that code and if the results are combined. For example, the number 63.98 is represented by (or converted to) the BCD code

$$\begin{aligned}
 &\quad \quad \quad 6 \quad 3 \quad . \quad 9 \quad 8 \\
 63.98_{10} &= (0110 \ 0011 \ . \ 1001 \ 1000)_{\text{BCD}} \\
 &= 1100011.10011_{\text{BCD}}
 \end{aligned}$$

Here, the code weights are 80, 40, 20, 10; 8, 4, 2, 1; 0.8, 0.4, 0.2, 0.1; and 0.08, 0.04, 0.02, 0.01 for the tens, units, tenths, and hundredths digits, respectively, representing four decades. Notice that the leading and trailing 0's can be dropped. Pencil-and-paper conversion between binary and BCD requires conversion to decimal as an intermediate step. For example, to convert from BCD to binary requires that groups of four bits be selected in both directions from the radix point to form the decimal number. If necessary, leading and trailing zeros are added to the leftmost or rightmost ends to complete the groups of four bits as in the example above. Negative BCD numbers are coded by using 10's complement notation as discussed in a later section.

Another code that is used for number representation and manipulation is called Excess 3 BCD (or XS3 BCD or simply XS3). XS3 is an example of a *biased-weighted* code (a bias of 3). This code is formed by adding  $0011_2 (= 3_{10})$  to the BCD bit patterns in Table 2.2.

Thus,

$$XS3 = BCD + 0011.$$

For example, the number  $63.98_{10}$  is represented in XS3 code as  $1001\ 0110\ .\ 1100\ 1011_{XS3}$ . To convert XS3 to BCD code, 0011 must be subtracted from XS3 code. In 4-bit quantities the XS3 code has the useful feature that when two numbers are added together in XS3 notation, a carry will result and yield the correct value any time a carry results in decimal (i.e., when 9 is exceeded). This feature is not shared by either binary or BCD addition.

## 2.4.2 The Hexadecimal and Octal Systems

The hexadecimal number system requires that  $r = 16$  in Eqs. (2.1) and (2.2), indicating that there are 16 distinguishable characters in the system. By convention, the permissible hexadecimal digits are 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, and F for decimals 0 through 15, respectively. Examples of the positional and polynomial representations for a hexadecimal number are

$$\begin{aligned} N_{16} &= (h_{n-1} \cdots h_3 h_2 h_1 h_0 \cdot h_{-1} h_{-2} h_{-3} \cdots h_{-m})_{16} \\ &= (AF3 \cdot C8)_{16} \end{aligned}$$

with a decimal value of

$$\begin{aligned} N &= \sum_{i=-m}^{n-1} h_i 16^i \\ &= 10 \times 16^2 + 15 \times 16^1 + 3 \times 16^0 + 12 \times 16^{-1} + 8 \times 16^{-2} \\ &= 2803.78125_{10}. \end{aligned}$$

Here, it is seen that a hexadecimal number has been converted to decimal by using Eq. (2.2).

The octal number system requires that  $r = 8$  in Eqs. (2.1) and (2.2), indicating that there are eight distinguishable characters in this system. The permissible octal digits are 0, 1, 2, 3, 4, 5, 6, and 7, as one might expect. Examples of the application of Eqs. (2.1) and (2.2) are

$$\begin{aligned} N_8 &= (o_{n-1} \cdots o_2 o_1 o_0 \cdot o_{-1} o_{-2} o_{-3} \cdots o_{-m})_8 \\ &= 501.74_8, \end{aligned}$$

with a decimal value of

$$\begin{aligned} N &= \sum_{i=-m}^{n-1} o_i 8^i \\ &= 5 \times 8^2 + 0 \times 8^1 + 1 \times 8^0 + 7 \times 8^{-1} + 4 \times 8^{-2} \\ &= 321.9375_{10}. \end{aligned}$$

**Table 2.3 The BCH and BCO number systems**

| Binary | BCH | BCO | Decimal | Binary  | BCH | BCO | Decimal |
|--------|-----|-----|---------|---------|-----|-----|---------|
| 0000   | 0   | 0   | 0       | 1010    | A   | 12  | 10      |
| 0001   | 1   | 1   | 1       | 1011    | B   | 13  | 11      |
| 0010   | 2   | 2   | 2       | 1100    | C   | 14  | 12      |
| 0011   | 3   | 3   | 3       | 1101    | D   | 15  | 13      |
| 0100   | 4   | 4   | 4       | 1110    | E   | 16  | 14      |
| 0101   | 5   | 5   | 5       | 1111    | F   | 17  | 15      |
| 0110   | 6   | 6   | 6       | 10000   | 10  | 20  | 16      |
| 0111   | 7   | 7   | 7       | 11011   | 1B  | 33  | 27      |
| 1000   | 8   | 10  | 8       | 110001  | 31  | 61  | 49      |
| 1001   | 9   | 11  | 9       | 1001110 | 4E  | 116 | 78      |

When the hexadecimal and octal number systems are used to represent bit patterns in binary, they are called binary coded hexadecimal (BCH) and binary coded octal (BCO), respectively. These two number systems are examples of *binary-derived radices*. Table 2.3 lists several selected examples showing the relationships between BCH, BCO, binary and decimal.

What emerges on close inspection of Table 2.3 is that each hexadecimal digit corresponds to four binary digits, and that each octal digit corresponds to three binary digits. The following example illustrate the relationships between these number systems:

$$\begin{aligned}
 & \quad \quad \quad 5 \quad B \quad F \quad . \quad D \quad 8 \\
 10110111111.11011_2 &= 0101 \ 1011 \ 1111 . 1101 \ 1000 \\
 &= 5BF.D8_{16} \\
 & \quad \quad \quad 2 \quad 6 \quad 7 \quad 7 \quad . \quad 6 \quad 6 \\
 &= 010 \ 110 \ 111 \ 111 . 110 \ 110 \\
 &= 2677.66_8 \\
 &= 1471.84375_{10}.
 \end{aligned}$$

To separate the binary digits into groups of four (for BCH) or groups of three (for BCO), counting must begin from the radix point and continue outward in both directions. Then, where needed, zeros are added to the leading and trailing ends of the binary representation to complete the MSDs and LSDs for the BCH and BCO forms.

## 2.5 CONVERSION BETWEEN NUMBER SYSTEMS

It is not the intent of this section to cover all methods for radix (base) conversion. Rather, the plan is to provide general approaches, separately applicable to the integer and fraction portions, followed by specific examples.

### 2.5.1 Conversion of Integers

Since the polynomial form of Eq. (2.2) is a geometrical progression, the integer portion can be represented in *nested radix* form. In source radix  $s$ , the nested representation is

$$\begin{aligned} N_s &= (a_{n-1}s^{n-1} + a_{n-2}s^{n-2} + \cdots + a_1s^1 + a_0s^0)_s \\ &= a_0 + s(a_1 + s(a_2 + \cdots + a_{n-1})) \cdots)_s \\ &= a_0 + s \left( \sum_{i=1}^{n-1} a_i s^{i-1} \right) \end{aligned} \quad (2.3)$$

for digits  $a_i$  having integer values from 0 to  $n-1$ . The nested radix form not only suggests a conversion process, but also forms the basis for computerized conversion.

Consider that the number in Eq. (2.3) is to be represented in nested radix  $r$  form,

$$\begin{aligned} N_r &= b_0 + r(b_1 + r(b_2 + \cdots + b_{m-1})) \cdots)_r \\ &= b_0 + r \left( \sum_{i=1}^{m-1} b_i r^{i-1} \right), \end{aligned} \quad (2.4)$$

where, in general,  $m \neq n$ . Then, if the source number  $N_s$  is divided by  $r$ , the results are of the form

$$\frac{N_s}{r} = Q + \frac{R}{r}, \quad (2.5)$$

where  $Q$  is the integer quotient rearranged as  $Q_0 = b_1 + r(b_2 + \cdots + b_{m-1}) \cdots$  and  $R$  is the remainder  $R_0 = b_0$ . A second division by  $r$  yields  $Q_0/r = Q_1 + R_1/r$ , where  $Q_1$  is arranged as  $Q_1 = b_2 + r(b_3 + \cdots + b_{m-1}) \cdots$ , and  $R_1 = b_1$ . Thus, by repeated division of the integer result  $Q_i$  by  $r$ , the remainders yield  $(b_0, b_1, b_2, \dots, b_{m-1})_r$  in that order.

The conversion method just described, called the *radix divide method*, can be used to convert between any two integers of different radices. However, the requirement is:

*The arithmetic required by  $N_s/r$  must be carried out in source radix,  $s$ .*

Except for source radices 10 and 2, this creates a problem for humans.

Table 2.4 provides the recommended procedures for integer conversion by noncomputer means. The radix divide method is suitable for use in computers only if they are programmed to carry out the arithmetic in different radices. Notice the partitioning required for conversion between binary and BCH and BCO integers.

The following two algorithms offer noncomputer methods for integer conversion:

#### Algorithm 2.1: $N_r \leftarrow N_s$ Positive Integer Conversion

Use Eq. (2.2) and the substitution method with base 10 arithmetic to convert  $N_s$  to  $N_{10}$ , then use the radix divide method and base 10 arithmetic to convert  $N_{10}$  to  $N_r$ .

**Table 2.4** Summary of recommended methods for integer conversion by noncomputer means

| Integer Conversion                              | Conversion Method                                                                                                |
|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| $N_{10} \rightarrow N_r$                        | Radix division by radix $r$ using Eq. (2.5)                                                                      |
| $N_s \rightarrow N_{10}$                        | Eq. (2.2) or (2.3)                                                                                               |
| $N_s)_{s \neq 10} \rightarrow N_r)_{r \neq 10}$ | $N_s \rightarrow N_{10}$ by Eq. (2.2) or (2.3)<br>$N_{10} \rightarrow N_r$ radix division by $r$ using Eq. (2.5) |
| Special Cases for Binary Forms                  |                                                                                                                  |
| $N_2 \rightarrow N_{10}$                        | Positional weighting                                                                                             |
| $N_2 \rightarrow N_{BCH}$                       | Partition $N_2$ into groups of four bits starting from radix point, then apply Table 2.3                         |
| $N_2 \rightarrow N_{BCO}$                       | Partition $N_2$ into groups of three bits starting from radix point, then apply Table 2.3                        |
| $N_{BCH} \rightarrow N_2$                       | Reverse of $N_2 \rightarrow N_{BCH}$                                                                             |
| $N_{BCO} \rightarrow N_2$                       | Reverse of $N_2 \rightarrow N_{BCO}$                                                                             |
| $N_{BCH} \rightarrow N_{BCO}$                   | $N_{BCH} \rightarrow N_2 \rightarrow N_{BCO}$                                                                    |
| $N_{BCO} \rightarrow N_{BCH}$                   | $N_{BCO} \rightarrow N_2 \rightarrow N_{BCH}$                                                                    |
| $N_{BCD} \rightarrow N_{XS3}$                   | Add $0011_2 (= 3_{10})$ to $N_{BCD}$                                                                             |
| $N_{XS3} \rightarrow N_{BCD}$                   | Subtract $0011_2 (= 3_{10})$ from $N_{XS3}$                                                                      |

**Algorithm 2.2:**  $N_{2^k} \leftarrow N_{2^n}$  Positive Integer Conversion

To convert numbers  $N_{2^n}$  to  $N_{2^k}$ , where  $n$  and  $k$  are integers, convert the groups of  $n$  digits in  $N_{2^n}$  to  $N_2$ , then reorganize the result in groups of  $k$  beginning with the LSB and proceeding toward the MSB.<sup>1</sup> Finally, replace each group of  $k$ , reading from the LSB, with the digit appropriate for number system  $N_{2^k}$ .

The integer conversion methods of Table 2.4 and Algorithms 2.1 and 2.2 are illustrated by the following simple examples:

**EXAMPLE 2.1**  $139_{10} \rightarrow N_2$ 

| $N/r$     | $Q$ | $R$ |                         |
|-----------|-----|-----|-------------------------|
| $139/2 =$ | 69  | 1   |                         |
| $69/2 =$  | 34  | 1   |                         |
| $34/2 =$  | 17  | 0   |                         |
| $17/2 =$  | 8   | 1   |                         |
| $8/2 =$   | 4   | 0   |                         |
| $4/2 =$   | 2   | 0   |                         |
| $2/2 =$   | 1   | 0   |                         |
| $1/2 =$   | 0   | 1   | $139_{10} = 10001011_2$ |

**EXAMPLE 2.2**  $10001011_2 \rightarrow N_{10}$ 

By positional weights  $N_{10} = 128 + 8 + 2 + 1 = 139_{10}$ .

<sup>1</sup> Note that leading 0's may be needed to complete the groups of  $k$ .



**EXAMPLE 2.3**  $139_{10} \rightarrow N_8$

$$\begin{array}{rcl} N/r & Q & R \\ 139/8 & = & 17 \quad 3 \\ 17/8 & = & 2 \quad 1 \\ 2/8 & = & 0 \quad 2 \end{array} \quad 139_{10} = 213_8$$

**EXAMPLE 2.4**  $10001011_2 \rightarrow N_{BCO}$

$$\begin{array}{ccc} 2 & 1 & 3 \\ 010 & 001 & 011 = 213_{BCO} \end{array}$$

**EXAMPLE 2.5**  $213_{BCO} \rightarrow N_{BCH}$

$$\begin{array}{ccc} 2 & 1 & 3 \\ 213_{BCO} = 010 & 001 & 011 = 10001011_2 = 1000 \quad 1011 = 8B_{16} \end{array}$$

**EXAMPLE 2.6**  $213_8 \rightarrow N_5$

$$213_8 = 2 \times 8^2 + 1 \times 8^1 + 3 \times 8^0 = 139_{10}$$

$$\begin{array}{rcl} N_r & Q & R \\ 139/5 & = & 27 \quad 4 \\ 27/5 & = & 5 \quad 2 \\ 5/5 & = & 1 \quad 0 \\ 1/5 & = & 0 \quad 1 \end{array} \quad 213_8 = 1024_5$$

$$\text{Check: } 1 \times 5^3 + 0 \times 5^2 + 2 \times 5^1 + 4 \times 5^0 = 125 + 0 + 10 + 4 = 139_{10}$$

## 2.5.2 Conversion of Fractions

By extracting the fraction portion from Eq. (2.2) one can write

$$\begin{aligned} \cdot N &= (a_{-1}s^{-1} + a_{-2}s^{-2} + \cdots + a_{-m}s^{-m})_s \\ &= s^{-1}(a_{-1} + s^{-1}(a_{-2} + \cdots + a_{-m})) \cdots)_s \\ &= s^{-1} \left( a_{-1} + \sum_{i=2}^m a_{-i}s^{-i+1} \right)_s \end{aligned} \quad (2.6)$$

in source radix  $s$ . This is called the *nested inverse radix* form and provides the basis for computerized conversion.

If the fraction in Eq. (2.6) is represented in nested inverse radix  $r$  form, then

$$\begin{aligned} \cdot N &= (b_{-1}r^{-1} + b_{-2}r^{-2} + \cdots + b_{-p}r^{-p})_r \\ &= r^{-1}(b_{-1} + r^{-1}(b_{-2} + \cdots + b_{-p})) \cdots)_r \\ &= r^{-1} \left( b_{-1} + \sum_{i=2}^p b_{-i}r^{-i+1} \right)_r \end{aligned} \quad (2.7)$$

for any fraction represented in radix  $r$ . Now, if source  $N_s$  is multiplied by  $r$ , the result is of the form

$$\cdot N_s \times r = I + F, \quad (2.8)$$

where  $I$  is the product integer,  $I_1 = b_{-1}$ , and  $F_0$  is the product fraction arranged as  $F_1 = r^{-1}(b_{-2} + r^{-1}(b_{-3} + \cdots + b_{-p})) \cdots$ . By repeated multiplication by  $r$  of the remaining fractions  $F_i$ , the resulting integers yield  $(b_{-1}, b_{-2}, b_{-3} \dots b_{-m})_r$  in that order.

The conversion just described is called the *radix multiply method* and is perfectly general for converting between fractions of different radices. However, as in the case of integer conversion, the requirement is that *the arithmetic required by  $\cdot N_s \times r$  must be carried out in source radix,  $s$* . For noncomputer use by humans, this procedure is usually limited to fraction conversions  $N_{10} \rightarrow N_r$ , where the source radix is 10 (decimal). Algorithm 2.3 gives the recommended methods for converting between fractions of different radices. The radix multiply method is well suited to computer use.

**Algorithm 2.3:  $\cdot N_r \leftarrow \cdot N_s$  Fraction Conversion**

- (1) Use Eq. (2.2) and the substitution method with base  $s$  arithmetic, or
- (2) Use the radix multiply method of Eq. (2.8) with source radix  $s$  arithmetic.

In either case for noncomputer means, if the source radix is other than 2 or 10, convert the fraction as follows:  $\cdot N_s \rightarrow \cdot N_{2 \text{ or } 10} \rightarrow \cdot N_r$  so that base 2 or 10 arithmetic can be applied.

Shown in Table 2.5 are the recommended methods given in some detail for fraction conversion by noncomputer means. Notice again the partitioning that is now required for conversion between binary fractions and those for BCH and BCO.

For any integer of source radix  $s$ , there exists an exact representation in radix  $r$ . This is not the case for a fraction whose conversion is a geometrical progression that never converges.

**Table 2.5 Summary of recommended methods for fraction conversion by noncomputer means**

| Fraction Conversion                                         | Conversion Method                                                                                                                    |
|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| $\cdot N_{10} \rightarrow \cdot N_r$                        | Radix multiplication by using Eq. (2.8)                                                                                              |
| $\cdot N_s \rightarrow \cdot N_{10}$                        | Eq. (2.2) or (2.6)                                                                                                                   |
| $\cdot N_s)_{s \neq 10} \rightarrow \cdot N_r)_{r \neq 10}$ | $\cdot N_s \rightarrow \cdot N_{10}$ by Eq. (2.2) or (2.6)<br>$\cdot N_{10} \rightarrow \cdot N_r$ radix multiplication by Eq. (2.5) |
| Special Cases for Binary Forms                              |                                                                                                                                      |
| $\cdot N_2 \rightarrow \cdot N_{10}$                        | Positional weighting                                                                                                                 |
| $\cdot N_2 \rightarrow \cdot N_{BCH}$                       | Partition $\cdot N_2$ into groups of four bits starting from radix point, then apply Table 2.3                                       |
| $\cdot N_2 \rightarrow \cdot N_{BCO}$                       | Partition $\cdot N_2$ into groups of three bits starting from radix point, then apply Table 2.3                                      |
| $\cdot N_{BCH} \rightarrow \cdot N_2$                       | Reverse of $\cdot N_2 \rightarrow \cdot N_{BCH}$                                                                                     |
| $\cdot N_{BCO} \rightarrow \cdot N_2$                       | Reverse of $\cdot N_2 \rightarrow \cdot N_{BCO}$                                                                                     |
| $\cdot N_{BCH} \rightarrow \cdot N_{BCO}$                   | $\cdot N_{BCH} \rightarrow \cdot N_2 \rightarrow \cdot N_{BCO}$                                                                      |
| $\cdot N_{BCO} \rightarrow \cdot N_{BCH}$                   | $\cdot N_{BCO} \rightarrow \cdot N_2 \rightarrow \cdot N_{BCH}$                                                                      |
| $\cdot N_{BCD} \rightarrow \cdot N_{XS3}$                   | Add $0011_2 (= 3_{10})$ to $N_{BCD}$                                                                                                 |
| $\cdot N_{XS3} \rightarrow \cdot N_{BCD}$                   | Subtract $0011_2 (= 3_{10})$ from $N_{XS3}$                                                                                          |

Terminating a fraction conversion at  $n$  digits (to the right of the radix point) results in an error or uncertainty. This error is given by

$$\begin{aligned}\varepsilon &= a_{-n}r^{-n} + a_{-(n+1)}r^{-(n+1)} + a_{-(n+2)}r^{-(n+2)} + \dots \\ &= r^{-n} \left[ a_{-n} + \sum_{i=1}^{\infty} a_{-(n+i)}r^{-(n+i)} \right],\end{aligned}$$

where the quantity in brackets is less than  $(a_{-n} + 1)$ . Therefore, terminating a fraction conversion at  $n$  digits from the radix point results in an error with bounds

$$0 < \varepsilon \leq r^{-n}(a_{-n} + 1). \quad (2.9)$$

Equation (2.9) is useful in deciding when to terminate a fraction conversion.

Often, it is desirable to terminate a fraction conversion at  $n + 1$  digits and then round off to  $n$  from the radix point. A suitable method for rounding to  $n$  digits in radix  $r$  is:

**Algorithm 2.4: Rounding Off to  $n$  Digits for Fraction Conversion in Radix  $r$**

Perform the fraction conversion to  $(n + 1)$  digits from the radix point, then drop the  $(n + 1)$  digit if  $a_{-(n+1)} < r/2$ ; add  $r^{-(n-1)}$  to the result if  $a_{-(n+1)} \geq r/2$ .

After rounding off to  $n$  digits, the maximum error becomes the difference between the rounded result and the smallest value possible. By using Eq. (2.9), this difference is

$$\begin{aligned}\varepsilon_{\max} &= r^{-n}(a_{-n} + 1) - r^{-n} \left( a_{-n} + \frac{a_{-(n+1)}}{r} \right) \\ &= r^{-n} \left( 1 - \frac{a_{-(n+1)}}{r} \right).\end{aligned}$$

Then, by rounding to  $n$  digits, there results an error with bounds

$$0 \leq |\varepsilon| \leq r^{-n} \left( 1 - \frac{a_{-(n+1)}}{r} \right). \quad (2.10)$$

If  $a_{-(n+1)} < r/2$  and the  $(n + 1)$  digit is dropped, the maximum error is  $r^{-n}$ . Note that for  $N_s \rightarrow N_{10} \rightarrow N_r$  type conversions, the bounds of errors aggregate.

The fraction conversion methods given in Table 2.5 and Algorithms 2.3 and 2.4 are illustrated by the following examples:

**EXAMPLE 2.7**  $0.654_{10} \rightarrow N_2$  rounded to 8 bits:

| $\cdot N_s \times r$ | $F$   | $I$ |
|----------------------|-------|-----|
| $0.654 \times 2$     | 0.308 | 1   |
| $0.308 \times 2$     | 0.616 | 0   |
| $0.616 \times 2$     | 0.232 | 1   |
| $0.232 \times 2$     | 0.464 | 0   |

$$\begin{array}{rclcl}
0.464 \times 2 & 0.928 & 0 \\
0.928 \times 2 & 0.856 & 1 \\
0.856 \times 2 & 0.712 & 1 \\
0.712 \times 2 & 0.424 & 1 & 0.654_{10} = 0.10100111_2 \\
0.424 \times 2 & 0.848 & 0 & \varepsilon_{\max} = 2^{-8}
\end{array}$$

**EXAMPLE 2.8**  $0.654_{10} \rightarrow N_8$  terminated at 4 digits:

$$\begin{array}{rclcl}
\cdot N_s \times r & F & I & & \\
0.654 \times 8 & 0.232 & 5 & & \\
0.232 \times 8 & 0.856 & 1 & 0.654_{10} = 0.5166_8 & \\
0.856 \times 8 & 0.848 & 6 & \text{with error bounds} & \\
0.848 \times 8 & 0.784 & 6 & 0 < \varepsilon \leq 7 \times 8^{-4} = 1.71 \times 10^{-3} \text{ by Eq. (2.9)} &
\end{array}$$

**EXAMPLE 2.9** Let  $0.5166_8 \rightarrow N_2$  be rounded to 8 bits and let  $0.5166_8 \rightarrow N_{10}$  be rounded to 4 decimal places:

$$\begin{aligned}
0.5166_8 &= 5 \times 8^{-1} + 1 \times 8^{-2} + 6 \times 8^{-3} + 6 \times 8^{-4} \\
&= 0.625000 + 0.015625 + 0.011718 + 0.001465 \\
&= 0.6538_{10} \text{ rounded to 4 decimal places; } \varepsilon_{10} \leq 10^{-4}
\end{aligned}$$

$$\begin{array}{rclcl}
\cdot N_s \times r & F & I & & \\
0.6538 \times 2 & 0.3076 & 1 & & \\
0.3076 \times 2 & 0.6152 & 0 & & \\
0.6152 \times 2 & 0.2304 & 1 & & \\
0.2304 \times 2 & 0.4608 & 0 & & \\
0.4608 \times 2 & 0.9216 & 0 & & \\
0.9216 \times 2 & 0.8432 & 1 & & \\
0.8432 \times 2 & 0.6864 & 1 & & \\
0.6864 \times 2 & 0.3728 & 1 & 0.5166_8 = 0.10100111_2 \text{ (compare with Example 2.7)} & \\
0.3728 \times 2 & 0.7457 & 0 & \varepsilon_{10} \leq 10^{-4} + 2^{-8} = 0.0040 &
\end{array}$$

**EXAMPLE 2.10**  $0.10100111_2 \rightarrow N_{\text{BCH}}$

$$\begin{array}{rcl}
\cdot A & 7 & \\
0.10100111_2 & = 0.1010 \ 0111 = 0.A7_{\text{BCH}}
\end{array}$$

## 2.6 SIGNED BINARY NUMBERS

To this point only unsigned numbers (assumed to be positive) have been considered. However, both positive and negative numbers must be used in computers. Several schemes have been devised for dealing with negative numbers in computers, but only four are commonly used:

- Signed-magnitude representation
- Radix complement representation

- Diminished radix complement representation
- Excess (offset) code representation

Of these, the radix 2 complement representation, called 2's complement, is the most widely used system in computers.

### 2.6.1 Signed-Magnitude Representation

A signed-magnitude number in radix  $r$  consists of a magnitude  $|N|$  together with a symbol indicating its sign (positive or negative) as follows:

$$N_{rSM} = \left( \underbrace{a_{n-1}}_{\text{Sign}} \underbrace{\overbrace{a_{n-2} \cdots a_2 a_1 a_0}^{\text{Integer}} \cdot \overbrace{a_{-1} a_{-2} a_{-3} \cdots a_{-m}}^{\text{Fraction}}}_{\text{Magnitude } |N|} \right)_{rSM}, \quad (2.11)$$

where the subscript  $rSM$  refers to signed-magnitude in radix  $r$ . Such a number lies in the decimal range of  $-(r^{n-1} - 1)$  through  $+(r^{n-1} - 1)$  for  $n$  integer digits in radix  $r$ . The fraction portion, if it exists, consists of  $m$  digits to the right of the radix point.

The most common examples of signed-magnitude numbers are those in the decimal and binary systems. The sign symbols for decimal (+ or −) are well known. In binary it is established practice to use the following convention:

*0 denotes a positive number*

*1 denotes a negative number.*

One of these (0 or 1) is placed at the MSB position of each SM number. Four examples in 8-bit binary are:

#### EXAMPLE 2.11

$$+45.5_{10} = 0 \overbrace{101101.1}_{\text{Magnitude}}_{2SM}$$

$\uparrow$   
 Sign  
 Bit

#### EXAMPLE 2.12

$$+0_{10} = 0 \ 0000000_{2SM}$$

#### EXAMPLE 2.13

$$-123_{10} = 1 \overbrace{1111011}_{\text{Magnitude}}_{2SM}$$

$\uparrow$   
 Sign  
 Bit

**EXAMPLE 2.14**

$$-0_{10} = 1\ 0000000_{2SM}$$

Although the sign-magnitude system is used in computers, it has two drawbacks. There is no unique zero, as indicated by the previous examples, and addition and subtraction calculations require time-consuming decisions regarding operation and sign, for example,  $(-7)$  minus  $(-4)$ . Even so, the sign-magnitude representation is commonly used in *floating-point* number systems as discussed in Section 2.8.

**2.6.2 Radix Complement Representation**

The *radix complement*  $N_{rC}$  of an  $n$ -digit number  $N_r$  is obtained by subtracting  $N_r$  from  $r^n$ , that is,

$$\begin{aligned} N_{rC} &= r^n - N_r \\ &= \bar{N}_r + 1_{LSD} \end{aligned} \quad (2.12)$$

where

$$\bar{N}_r \equiv \text{Digit complementation in radix } r$$

This operation is equivalent to that of replacing each digit  $a_i$  in  $N_r$  by  $(r - 1) - a_i$  and adding 1 to the LSD of the result as indicated by Algorithm 2.5. The digit complements  $\bar{N}_r$  for three commonly used number systems are given in Table 2.6. Notice that the digit complement of a binary is formed simply by replacing the 1's with 0's and 0's with 1's required by  $2^n - N_2 - 1 = \bar{N}_2$  as discussed in Subsection 2.6.3. The range of representable numbers is  $-(r^{n-1})$  through  $+(r^{n-1} - 1)$ .

Application of Eq. (2.12) or Algorithm 2.5 to the binary and decimal number systems requires that for 2's complement representation  $N_{2C} = \bar{N}_2 + 1_{LSB}$  and for 10's complement  $N_{10C} = \bar{N}_{10} + 1_{LSD}$ , where  $\bar{N}_2$  and  $\bar{N}_{10}$  are the binary and decimal digit complements given in Table 2.6.

**Table 2.6 Digit complements for three commonly used number systems**

| Digit | Complement ( $\bar{N}_r$ ) |         |             |
|-------|----------------------------|---------|-------------|
|       | Binary                     | Decimal | Hexadecimal |
| 0     | 1                          | 9       | F           |
| 1     | 0                          | 8       | E           |
| 2     |                            | 7       | D           |
| 3     |                            | 6       | C           |
| 4     |                            | 5       | B           |
| 5     |                            | 4       | A           |
| 6     |                            | 3       | 9           |
| 7     |                            | 2       | 8           |
| 8     |                            | 1       | 7           |

Algorithm 2.5:  $N_{rC} \leftarrow N_r$

Replace each digit  $a_i$  in  $N_r$  by  $(r - 1) - a_i$  and then add 1 to the LSD of the resultant.

A simpler, “pencil-and-paper” method exists for the 2’s complement of a number  $N_2$  and is expressed by Algorithm 2.6:

Algorithm 2.6:  $N_{2C} \leftarrow N_2$

For any binary number  $N_2$  and beginning with the LSB, proceed toward the MSB until the *first* 1 bit has been reached. Retain that 1 bit and complement the remainder of the bits toward and including the MSB.

With reference to Table 2.6, Eq. (2.12), and Algorithm 2.5 or 2.6, the following examples of radix complement representation are provided:

**EXAMPLE 2.15** The 10’s complement of 47.83 is  $\bar{N}_{10} + 1_{LSD} = 52.17$ .

**EXAMPLE 2.16** The 2’s complement of 0101101.101 is  $\bar{N}_2 + 1_{LSB} = 1010010.011$ .

**EXAMPLE 2.17** The 16’s complement of A3D is  $\bar{N}_{16} + 1_{LSD} = 5C2 + 1 = 5C3$ .

The decimal value of Eq. (2.12) can be found from the polynomial expression as

$$(N_{rC})_{10} = -(a_{n-1}r^{n-1}) + \sum_{i=-m}^{n-2} a_i r^i \quad (2.13)$$

for any  $n$ -digit number of radix  $r$ . In Eqs. (2.12) and (2.13) the MSD is taken to be the position of the sign symbol.

**2’s Complement Representation** The radix complement for binary is the 2’s complement ( $2C$ ) representation. In 2’s complement the MSB is the sign bit, 1 indicating a negative number and 0 a positive one. The decimal range of representation for  $n$  integer bits in 2’s complement is from  $-(2^{n-1})$  through  $+(2^{n-1} - 1)$ . From Eq. (2.12), the 2’s complement is formed by

$$N_{2C} = 2^n - N_2 = \bar{N}_2 + 1_{LSB} \quad (2.14)$$

for any binary number  $N_2$  of  $n$  integer bits. Or by Algorithm 2.5, the 2’s complement of a binary number  $N_2$  is obtained by replacing each bit  $a_i$  in  $N_2$  by  $(1 - a_i)$  and adding 1 to the LSB of the result. The simpler pencil-and-paper method, often used to generate 2’s complement from a binary number  $N_r$ , results from application of Algorithm 2.6. In this case  $\bar{N}_2$  is the bit complement of the number as given in Table 2.6. A few examples of 8-bit 2’s complement numbers are shown in Table 2.7. Notice that application of Eq. (2.14) or Algorithm 2.6 changes the sign of the decimal value of a binary number (+ to – and vice versa), and that only one zero representation exists.

Application of Eq. (2.13) gives the decimal value of any 2’s complement number, including those containing a radix point. For example, the pattern  $N_{2C} = 11010010.011$  has

**Table 2.7 Examples of eight-bit 2's and 1's complement representations (MSB = sign bit)**

| Decimal Value | 2's Complement | 1's Complement |
|---------------|----------------|----------------|
| -128          | 10000000       |                |
| -127          | 10000001       | 10000000       |
| -31           | 11100001       | 11100000       |
| -16           | 11110000       | 11101111       |
| -15           | 11110001       | 11110000       |
| -3            | 11111101       | 11111100       |
| -0            | 00000000       | 11111111       |
| +0            | 00000000       | 00000000       |
| +3            | 00000011       | 00000011       |
| +15           | 00001111       | 00001111       |
| +16           | 00010000       | 00010000       |
| +31           | 00011111       | 00011111       |
| +127          | 01111111       | 01111111       |
| +128          |                |                |

a decimal value of

$$\begin{aligned}
 (N_{2C})_{10} &= -1 \times 2^7 + 1 \times 2^6 + 1 \times 2^4 + 1 \times 2^1 + 1 \times 2^{-2} + 1 \times 2^{-3} \\
 &= -128 + 64 + 16 + 2 + 0.25 + 0.125 \\
 &= -45.625_{10}.
 \end{aligned}$$

But the same result could have easily been obtained by *negation* of  $N_{2C}$  followed by the use of positional weighting to obtain the decimal value. Negation is the reapplication of Eq. (2.12) or Algorithms 2.5 or 2.6 to any 2's complement number  $N_{2C}$  to obtain its *true value*. Thus, from the forgoing example the negation of  $N_{2C}$  is given by

$$\begin{aligned}
 N_{2C})_{2C} &= 00101101.101 \\
 &= 32 + 8 + 5 + 0.5 + 0.125 \\
 &= 45.625_{10},
 \end{aligned}$$

which is known to be a negative number,  $-45.625_{10}$ .

Negative BCD numbers are commonly represented in 10's complement notation with consideration of how BCD is formed from binary. As an example,  $-59.24_{10} = 40.76_{10}$  is represented in BCD 10's complement (BCD,10C) by

$$-0101\ 1001.0010\ 0100)_{BCD} = 0100\ 0000.0111\ 0110)_{BCD,10C},$$

where application of Eq. (2.12), or Algorithm 2.5 or 2.6, has been applied in radix 10 followed by the BCD representation as in Subsections 2.4.1. Alternatively, the sign-magnitude (SM) representation of a negative BCD number simply requires the addition of a sign bit



to the BCD magnitude according to Eq. (2.11). Thus,

$$-0101\ 1001.0010\ 0100)_{BCD} = (1\ 01011001.0010\ 0100)_{BCD,2SM}.$$

### 2.6.3 Diminished Radix Complement Representation

The *diminished radix complement*  $N_{(r-1)C}$  of a number  $N_r$  having  $n$  digits is obtained by

$$N_{(r-1)C} = r^n - N_r - 1, \quad (2.15)$$

where, according to Eq. (2.12),  $N_{(r-1)C} + 1 = N_{rC}$ . Therefore, it follows that

$$N_{(r-1)C} = \bar{N}_r.$$

This means the diminished radix complement of a number is the digits complement of that number as expressed by Algorithm 2.7. The range of representable  $n$  digit numbers in diminished radix complement is  $-(r^{n-1} - 1)$  through  $+(r^{n-1} - 1)$  for radix  $r$ .

#### Algorithm 2.7: $N_{(r-1)C} \leftarrow N_r$

- (1) Replace each digit  $a_i$  of  $N_r$  by  $r - 1 - a_i$  or
- (2) Complement each digit by  $\bar{N}_r$  as in Table 2.6.

In the binary and decimal number systems the diminished radix complement representations are the 1's complement and 9's complement, respectively. Thus, 1's complement is the binary digits complement given by  $N_{1C} = \bar{N}_2$ , while the 9's complement is the decimal digits complement expressed as  $N_{9C} = \bar{N}_{10}$ . Examples of eight-bit 1's complements are shown in Table 2.7 together with their corresponding 2's complement representation for comparison. Notice that in 1's complement there are two representations for zero, one for  $+0$  and the other for  $-0$ . This fact limits the usefulness of the 1's complement representation for computer arithmetic.

Shown in Table 2.8 are examples of 10's and 9's complement representations in  $n$  digits numbering from 3 to 8. Notice that leading 0's are added to the number on the left to meet the  $n$  digit requirement.

**Table 2.8 Examples of 10's and 9's complement representation**

| Number  | $n$ | 10's Complement | 9's Complement |
|---------|-----|-----------------|----------------|
| 0       | 5   | [1]00000        | 99999          |
| 3       | 3   | 997             | 996            |
| 14.59   | 6   | 9985.41         | 9985.40        |
| 225     | 4   | 9775            | 9774           |
| 21.456  | 5   | 78.544          | 78.543         |
| 1827    | 8   | 99998173        | 99998172       |
| 4300.50 | 7   | 95699.50        | 95699.49       |
| 69.100  | 6   | 930.900         | 930.899        |

## 2.7 EXCESS (OFFSET) REPRESENTATIONS

Other systems for representing negative numbers use *excess* or *offset (biased)* codes. Here, a bias  $B$  is added to the true value  $N_r$  of the number to produce an excess number,  $N_{xs}$ , given by

$$N_{xs} = N_r + B. \quad (2.16)$$

When  $B = r^{n-1}$  exceeds the usable bounds of negative numbers,  $N_{xs}$  remains positive. Perhaps the most common use of the excess representation is in floating-point number systems — the subject of the next section. The biased-weighted BCD code, XS3, was discussed in Subsection 2.4.1.

Two examples of excess 127 representation are given below.

### EXAMPLE 2.18

$$\begin{array}{rcl} -43_{10} & 11010101 & N_2\text{'s Compl.} \\ +127_{10} & 01111111 & B \\ \hline 84_{10} & 01010100 & N_{xs} = -43_{10} \text{ in excess 127 representation} \end{array}$$

### EXAMPLE 2.19

$$\begin{array}{rcl} 27_{10} & 00011011 & N_2\text{'s Compl.} \\ +127_{10} & 01111111 & B \\ \hline 154_{10} & 10011010 & N_{xs} = 27_{10} \text{ in excess 127 representation} \end{array}$$

The representable decimal range for an excess  $2^{n-1}$  number system is  $-2^{n-1}$  through  $+(2^{n-1} - 1)$  for an  $n$ -bit binary number. However, if  $N_2 + B > 2^{n-1} - 1$ , *overflow* occurs and  $2^{n-1}$  must be subtracted from  $(N_2 + B)$  to give the correct result in excess  $2^{n-1}$  code.

## 2.8 FLOATING-POINT NUMBER SYSTEMS

In *fixed-point* representation [Eq. (2.1)], the radix point is assumed to lie immediately to the right of the integer field and at the left end of the fraction field. The fixed-point system is the most commonly used system for representing bounded orders of magnitude. For example, with 32 bits a binary number could represent decimal numbers with upper and lower bounds of the order of  $\pm 10^{10}$  and  $\pm 10^{-10}$ . However, for greatly expanded bounds of representation, as in scientific notation, the *floating-point* representation is needed. This form of number representation is commonly used in computers.

A floating-point number (FPN) in radix  $r$  has the general form

$$\text{FPN}_r = M \times r^E, \quad (2.17)$$

where  $M$  is the *fraction* (or *mantissa*) and  $E$  is the *exponent*. Only fraction digits are used for the mantissa! Take, for example, Planck's constant  $h = 6.625 \times 10^{-34}$  J s. This number

can be represented many different ways in floating-point notation:

$$\begin{aligned}\text{Planck's constant, } h &= 0.6625 \times 10^{-33} \\ &= 0.06625 \times 10^{-32} \\ &= 0.006625 \times 10^{-31}.\end{aligned}$$

All three adhere to the form of Eq. (2.17) and are, therefore, legitimate floating-point numbers in radix 10. Thus, as the radix point *floats* to the left, the exponent is *scaled* accordingly. The first form for  $h$  is said to be *normalized* because the most significant digit (MSD) of  $M$  is nonzero, a means of standardizing the radix point position. Notice that the sign for  $M$  is positive while that for  $E$  is negative.

In computers the FPN is represented in binary where the normalized representation requires that the MSB for  $M$  always be 1. Thus, the range in  $M$  in decimal is

$$0.5 \leq M < 1.$$

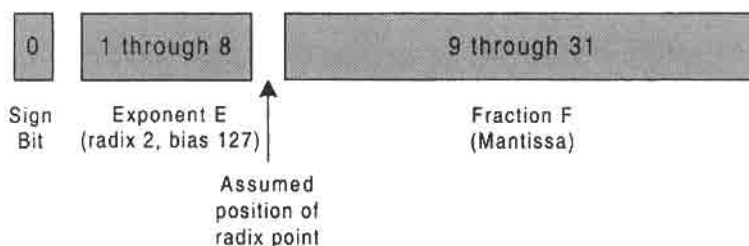
Also, the fraction (mantissa)  $M$  is represented in sign-magnitude form. The normalized format for a 32-bit floating-point number in binary, which agrees with the IEEE standard [3], is shown in Fig. 2.1. Here, the sign bit (1 if negative or 0 if positive) is placed at bit position 0 to indicate the sign of the fraction. Notice that the radix point is assumed to lie between bit positions 8 and 9 to separate the  $E$  bit-field from the  $M$  bit-field.

Before two FPNs can be added or subtracted in a computer, the  $E$  fields must be compared and equalized, and the  $M$  fields adjusted. The decision-making process can be simplified if all exponents are converted to positive numbers by using the excess representation given by Eq. (2.16). For a  $q$ -digit number in radix  $r$ , the exponent in Eq. (2.17) becomes

$$E_{xs} = E_r + r^{q-1}, \quad (2.18)$$

where  $E$  is the actual exponent augmented by a bias of  $B = r^{q-1}$ . The range in the actual exponent  $E_r$  is usually taken to be

$$-(r^{q-1} - 1) \leq E_r \leq +(r^{q-1} - 1).$$



**FIGURE 2.1**

IEEE standard bit format for 32-bit normalized floating-point representation.

In the binary system, required for computer calculations, Eq. (2.18) becomes

$$E_{xs} = E_2 + 2^{q-1}, \quad (2.19)$$

with a range in actual exponent of  $-(2^{q-1} - 1) \leq E_2 \leq +(2^{q-1} - 1)$ . In 32-bit normalized floating-point form, the exponent in Eq. (2.19) is stored in either excess 127 or excess 128 code (depending on the FPN system used) while the mantissa is stored in sign-magnitude form.

There still remains the question of how the number zero is to be represented. If the  $M$  field is zero, then the exponent can be anything and the number will be zero. However, in computers the normalized FPN<sub>2</sub> limits  $M$  to  $(0.5 \leq M < 1)$  since the MSB for  $M$  is always 1. The solution to this problem is to assume that the number is zero if the exponent bits are all zero regardless of the value of the mantissa. But this leads to a discontinuity in normalized FPN<sub>2</sub> representation at the low end.

The IEEE standard for normalized FPN<sub>2</sub> representation attempts to remove the problem just described. The IEEE system stores the exponent in excess  $2^{q-1} - 1$  code and limits the decimal range of the actual exponent to

$$-(2^{q-1} - 2) \leq E_2 \leq +(2^{q-1} - 1).$$

For 32-bit FPN single precision representation, the exponent is stored in excess 127 code as indicated in Fig. 2.1. Thus, the allowable range of representable exponents is from

$$-126_{10} = 00000001_2 \text{ through } +127_{10} = 11111110_2.$$

This system reserves the use of all 0's or all 1's in the exponent for special conditions [3]. So that the  $M$  field magnitude can diminish linearly to zero when  $E = -126$ , the MSB = 1 for  $M$  is not specifically represented in the IEEE system, but is implied.

The following example attempts to illustrate the somewhat confusing aspects of the IEEE normalized representation:

**EXAMPLE 2.20** The number  $101101.11001_2$  is to be represented in IEEE normalized FPN<sub>2</sub> notation:

$$101101.11001_2 = .10110111001 \times 2^6 \text{ Sign bit} = 0 \text{ (positive)}$$

where

$$E_{xs} = 6 + 127 = 133_{10} = 10000101_2$$

$$M = 0110111001 \dots 00 \text{ (the MSB} = 1 \text{ is not shown).}$$

Therefore, the IEEE normalized FPN is

$$\text{FPN}_2 = 0 \ 10000101 \ 0110111001 \dots 0.$$

Still, other forms of FPNs are in use. In addition to the IEEE system, there are the IBM, Cray, and DEC systems of representation, each with their own single- and double-precision

forms. For example, the IEEE double-precision FPN system requires an exponent of 11 bits in excess 1023 code and a mantissa (in sign-magnitude) of 53 bits for a total 64 bits.

## 2.9 BINARY ARITHMETIC

There are many ways in which to manipulate data for the purpose of computation. It is not the intent of this section to discuss all these methods. Rather, the emphasis will be on the basic addition, subtraction, multiplication, and division methods for binary number manipulation as a foundation for the design of the arithmetic units featured in Chapter 8. The coverage will include the basic heuristics involved in fixed-point binary arithmetic together with simple examples. Advanced methods applicable to computer arithmetic operations are provided as needed for the device design. A limited treatment of floating-point binary arithmetic will be given in a later section.

### 2.9.1 Direct Addition and Subtraction of Binary Numbers

The addition of any two positive binary numbers is accomplished in a manner similar to that of two radix (base) 10 numbers. When the addition of two binary bits exceeds  $01_2$ , a carry bit is added to the next MSB, and this process is continued until all bits of the addend and augend have been added together. As an example consider the addition of the following two 8-bit numbers:

#### EXAMPLE 2.21

$$\begin{array}{rcccccccc}
 & & & & 1 & 1 & 1 & 1 & & & \leftarrow \text{Carries} \\
 59_{10} & & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1_2 & = \text{Augend} \\
 +122_{10} & +0 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0_2 & = \text{Addend} \\
 \hline
 181_{10} & & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 1_2 & = \text{Sum}
 \end{array}$$

Notice that in binary addition the carry is rippled to the left in much the same manner as in base 10 addition. The binary numbers are easily converted to base 10 by the method of positional weight described in Section 2.3.

#### Algorithm 2.8: $A_2 + B_2$

- (1) Set operands  $A_2 = a_{n-1}a_{n-2} \cdots a_1a_0$  and  $B_2 = b_{n-1}b_{n-2} \cdots b_1b_0$ , and their sum  $A_2 + B_2 = S_n S_{n-1} \cdots S_1 S_0 = S_2$ .
- (2) Set  $i = 0$  and  $S_2 = 0$ .
- (3) If  $a_0 + b_0 < 10_2$ ,  $S_0 = a_0 + b_0$  and a carry  $C_1 = 0$  is generated for position  $i + 1 = 1$ . If  $a_0 + b_0 \geq 10_2$ , then  $S_0 = a_0 + b_0 - 10_2$  and a carry  $C_{i+1} = 1$  is generated into position  $i + 1 = 1$ .
- (4) Continue steps (2) and (3) in the order  $i = 1, 2, 3, \dots, n - 1$  with carries generated into position  $i + 1$ .
- (5) The most significant sum bit is  $S_n = C_n$ , where  $C_n$  is the carry resulting from the addition of  $a_{n-1}$ ,  $b_{n-1}$ , and  $C_{n-1}$ .

Direct subtraction of two binary numbers also parallels that for base 10 subtraction. Now however, when the subtrahend bit is 1 when the minuend bit is 0, a borrow is required from the next MSB. Thus, the borrowing process begins at the MSB and ends with the LSB—the opposite of the carry process for addition. Remember that a borrow of  $1_2$  from the next MSB, creates a  $10_2$  in the column being subtracted. The following 8-bit example illustrates the subtraction process in base 2:

**EXAMPLE 2.22**

$$\begin{array}{r}
 \begin{array}{cccccccc}
 & 10 & 1 & & & & & \\
 & 0 & \emptyset & 1\emptyset & 10 & 0 & 10 & \\
 101_{10} & 0 & 1 & 1 & \emptyset & \emptyset & 1 & \emptyset & 1_2 = \text{Minuend} \\
 -58_{10} & -0 & 0 & 1 & 1 & 1 & 0 & 1 & 0_2 = \text{Subtrahend} \\
 \hline
 43_{10} & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 = \text{Difference}
 \end{array}
 \end{array}$$

Here, the notation  $\emptyset$  or  $1$  represents denial of the 0 or 1 when a borrow is indicated. Notice, as in the example just given, that the borrowing process may involve more than one level of borrowing as the process proceeds from right to left.

**2.9.2 Two's Complement Subtraction**

Computer calculations rarely involve direct subtraction of binary numbers. Much more commonly, the subtraction process is accomplished by 2's complement arithmetic—a considerable savings in hardware. Here, subtraction involves converting the subtrahend to 2's complement by using Eq. (2.14) in the form  $\bar{N}_2 + 1$  and then adding the result directly to the minuend. For an  $n$ -bit operand subtraction,  $n + 1$  bits are used where the MSB bit is designated the *sign bit*. Also, the *carry overflow* is discarded in 2's complement arithmetic. The following example illustrates the process for two four-bit numbers, A and B:

**EXAMPLE 2.23**

$$\begin{array}{rcl}
 \begin{array}{rcl}
 A & 0:1101 \\
 -B & -0:0111 \\
 \hline
 \end{array}
 & \longrightarrow &
 \begin{array}{rcl}
 0:1101 & = +13_{10} \\
 +1:1001 & = -7_{10} \\
 \hline
 0:0110 & = +6_{10}
 \end{array}
 \end{array}$$

Further illustration continues by interchanging the minuend and subtrahend so as to yield a negative number:

**EXAMPLE 2.24**

$$\begin{array}{rcl}
 \begin{array}{rcl}
 A & 0:0111 \\
 -B & -0:1101 \\
 \hline
 \end{array}
 & \longrightarrow &
 \begin{array}{rcl}
 0:0111 & = +7_{10} \\
 +1:0011 & = -13_{10} \\
 \hline
 1:1010 & = -6_{10}
 \end{array}
 \end{array}$$

In this case, the difference  $1:1010$  is given in 2's complement. The answer in true form is the 2's complement (*negation*) of this result given by  $1:1010 \rightarrow 0:0110_2$  or  $6_{10}$  which is negative. Algorithm 2.9 summarizes the steps required for the 2's complement subtraction.

Algorithm 2.9:  $\{A_2 + (-B_2) \rightarrow A_2 + B_{2C}\}$  or  $\{-A_2 + (-B_2) \rightarrow A_{2C} + B_{2C}\}$

- (1) Set  $n$ -bit operands  $A = a_n a_{n-1} \cdots a_1 a_0$  and  $B = b_n b_{n-1} \cdots b_1 b_0$ , where the MSB  $a_n$  and  $b_n$  are reserved as sign bits, initially both 0.
- (2) Generate  $B_{2C}$  by applying Eq. (2.14).
- (3) Add operands  $A_2 + B_{2C}$  according to Algorithm 2.8.
- (4) If  $|A_2| > |B_2|$ , then the sum ( $S_2 > 0$ ) is the true value with the sign bit 0. If  $|A_2| < |B_2|$ , then the sum ( $S < 0$ ) is given in 2's complement with sign bit equal to 1.
- (5) The true value of a 2's complement sum is obtained by negation,  $S_{2C} \rightarrow 2C$ .
- (6) If  $A_2$  and  $B_2$  are both negative numbers, the sum ( $S_{2C} < 0$ ) is obtained by steps (2), (3), and (5).

### 2.9.3 One's Complement Subtraction

Somewhat less common is the 1's complement subtraction of two binary numbers. In this case the subtrahend is converted to 1's complement by using Eq. (2.15) in the form  $\bar{N}_2$ . The result is then added to the minuend with the carry overflow *carried-end-around* and added to the LSB. Clearly, this is a more complex process than that for 2's complement. The following two examples illustrate the subtraction process:

#### EXAMPLE 2.25

$$\begin{array}{rcl}
 A & 0:1101 & \rightarrow 0:1101 = +13_{10} \\
 -B & -0:0111 & +1:1000 = -7_{10} \\
 & \hline
 & 1:0101 & \\
 & \xrightarrow{1} & \\
 & 0:0110 & = +6_{10} \\
 & \uparrow & \\
 & \text{Sign Bit} & 
 \end{array}$$

Again, the minuend and subtrahend are interchanged for comparison purposes, yielding a negative difference as follows:

#### EXAMPLE 2.26

$$\begin{array}{rcl}
 A & 0:0111 & \rightarrow 0:0111 = +7_{10} \\
 -B & -0:1101 & +1:0010 = -13_{10} \\
 & \hline
 & 1:1001 & \\
 & \xrightarrow{0} & \\
 & 1:1001 & = -6_{10} \\
 & \uparrow & \\
 & \text{Sign Bit} & 
 \end{array}$$

In this latter case the result is given in 1's complement. The true value for the difference is obtained by negation and is  $1:1001 \rightarrow 0:0110_2 = 6_{10}$ , which is known to be negative.

### 2.9.4 Binary Multiplication

Like binary addition and subtraction, binary multiplication closely follows base 10 multiplication. Consider two  $n$ -bit binary integers  $A_2 = (a_{n-1} \cdots a_2 a_1 a_0)_2$  and  $B_2 = (b_{n-1} \cdots b_2 b_1 b_0)_2$ . Their product  $A_2 \times B_2$  is expressed in  $n$ -bits assuming that both numbers are expressible in  $n/2$  bits excluding leading zeros. Under this assumption the product is

$$\begin{aligned} \mu P = A \times B &= \left( \sum_{i=0}^{n-1} a_i \cdot 2^i \right) \cdot B \\ &= a_{n-1} \times 2^{n-1} B + \cdots + a_1 2^1 B + a_0 B, \end{aligned} \quad (2.20)$$

meaning that if  $B = b_{n-1} \cdots b_2 b_1 b_0$ , the product  $2^i \times B$  is

$$2^i \times B = b_{n-1} \cdots b_2 b_1 b_0 \underbrace{00 \cdots 0}_{i \text{ zeros}}.$$

Thus, the product  $A \times B$  is expressed as the sum of the partial products  $p_i$  in the form

$$P = \sum_{i=0}^{n-1} p_i = \sum_{i=0}^{n-1} a_i (b_{n-1} \cdots b_2 b_1 b_0 \underbrace{00 \cdots 0}_{i \text{ zeros}}). \quad (2.21)$$

Therefore it should seem clear that binary multiplication requires addition of all terms of the form  $2^i \times B$  for all  $i$  for which  $a_i = 1$ . The following example illustrates this process.

#### EXAMPLE 2.27

|            |                     |   |                                         |
|------------|---------------------|---|-----------------------------------------|
| $A$        | 00001111            | = | Multiplicand                            |
| $\times B$ | <u>×00001011</u>    | = | Multiplier                              |
|            | 00001111            |   | $2^0 \times B$                          |
|            | 000011110           |   | $2^1 \times B$                          |
|            | 0000000000          |   |                                         |
|            | 00001111000         |   | $2^3 \times B$                          |
|            | 111011              |   | Level 1 Carries                         |
|            | <u>1</u>            |   | Level 2 Carries                         |
|            | 000 <u>10100101</u> | = | Product $P_2$                           |
|            |                     |   | <small>8-bit<br/>representation</small> |

Notice that the carry process may involve more than one level of carry as is true in this example. The following algorithm avoids multiple levels of carry by adding each product to  $P$  as it is formed.



Algorithm 2.10:  $A_2 \times B_2$ 

- (1) Set  $n = 2k$ , where  $k$  is the number of bits of the larger number exclusive of leading zeros.
- (2) Set  $A = a_{n-1} \cdots a_2 a_1 a_0$  and  $B = b_{n-1} \cdots b_2 b_1 b_0$  for an  $n$ -bit multiplier and an  $n$ -bit multiplicand, respectively.
- (3) Set  $P = 0$  and  $i = 0$ .
- (4) If  $a_i = 1$ , calculate  $2^i \times B = (b_{n-1} \cdots b_1 b_0 \underbrace{00 \cdots 0}_{i \text{ zeros}})$  and add it to  $P$ .
- (5) Increment  $i$  by 1.
- (6) Repeat steps (3) and (4) for all  $0 \leq i \leq (n-1)$  ending with a product  $P_2$  of  $n$  bits or less.

If  $A_2^k$  and  $B_2^m$  represent operands expressible by a different number of bits,  $k$  and  $m$ , exclusive of leading zeros, then their product is  $P_2^n = A_2^k \times B_2^m$  given in  $n \leq (k+m)$  bits. For numbers containing both integers and fractions,  $k$  and  $m$  must each include all bits exclusive of leading integer zeros. For example, if  $B_2^m = 1101.11$  ( $m = 6$ ) and  $A_2^k = 110.1$  ( $k = 4$ ), their product  $P_2^n$  will be given in  $n = 6 + 4 = 10$  bits. The following example illustrates the multiplication process for these two operands.

**EXAMPLE 2.28**

|                    |                      |
|--------------------|----------------------|
| 1101.11            | Multiplicand $B_2^m$ |
| <u>× 110.1</u>     | Multiplier $A_2^k$   |
| 110111             | $2^0 \times B$       |
| 0000000            |                      |
| 11011100           | $2^2 \times B$       |
| 110111000          | $2^3 \times B$       |
| 111011             | Level 1 Carries      |
| <u>1</u>           | Level 2 Carries      |
| <u>1011001.011</u> | Product $P_2^n$      |

*10-bit representation*

**2's Complement Multiplication** To understand 2's complement multiplication it is helpful to introduce the concept of *modulo*  $2^n$  ( $\text{Mod } 2^n$ ) arithmetic. In  $\text{Mod } 2^n$  arithmetic multiplication is carried out in 2's complement numbers (if negative) ignoring the overflow beyond  $n$  bits. For example,  $2^4 \times 1111 \pmod{2^4} = 10000 = 2^4$  or generally, for number  $B$  of  $n$  bits,

$$2^n \times B \pmod{2^n} = 2^n.$$

Consider the  $n$ -bit integer operands  $A_2 = a_{n-1} \cdots a_1 a_0$  and  $B_2 = b_{n-1} \cdots b_1 b_0$ . Then, if the product is  $P = A \times (-B)$ , there results, after converting  $B$  to 2's complement,

$$\begin{aligned} P_2 &= A_2 \times (B_{2C}) \\ &= A_2 \times (2^n - B) && \text{Mod } 2^n \\ &= A_2 \times 2^n - A_2 \times B_2 && \text{Mod } 2^n \end{aligned}$$

or

$$P_2 = 2^n - A_2 \times B_2 \quad \text{Mod } 2^n \quad (2.22)$$

Thus,  $P_2 = A_2 \times (B_{2C})$  generates the 2's complement of  $A_2 \times B_2$ —the form in which a computer stores the results. The “true” value of  $P_2$  can be obtained by negation, which is the 2's complement of the result, that is,  $(P_2)_{2C} = (2^n - A_2 \times B_2)_{2C}$ .

Likewise, if both  $A_2$  and  $B_2$  are negative  $n$ -bit operands, then the product  $(-A_2) \times (-B_2)$  becomes, after conversion to 2's complement by Eq. (2.14),

$$\begin{aligned} P_2 &= A_{2C} \times (B_{2C}) \\ &= (2^n - A_2) \times (2^n - B_2) \quad \text{Mod } 2^n \\ &= 2^n \times 2^n - 2^n A_2 - 2^n B_2 + A_2 \times B_2 \quad \text{Mod } 2 \\ &= 2^{2n} - 2^n - 2^n + A_2 \times B_2 \quad \text{Mod } 2^n \end{aligned}$$

$$\text{or} \quad P_2 = A_2 \times B_2, \quad (2.23)$$

where  $2^{2n} - 2^n - 2^n \text{ (Mod } 2^n) = 0$ . Thus, the product of two negative binary numbers in 2's complement notation is the positive product of the two numbers. In dealing with numbers whose bit representations are, say,  $k > m$ , excluding leading zeros for both, the 2's complement product  $P_{2C}^n$  must be given in  $n = 2k$  bits. This count for  $k$  must include fraction bits, if present, but exclude both leading and trailing zeros.

The following example illustrates 2's complement multiplication of two numbers with fractions each of  $k = m = 4$  bits and represented as  $k + m = 8$ -bit operands:

#### EXAMPLE 2.29

$$\begin{array}{rcl} \begin{array}{r} -2.25 \\ \times 6.5 \\ \hline -14.625 \end{array} & \begin{array}{c} -000010.01 \\ \times 0000110.1 \\ \hline \end{array} & \begin{array}{l} \longrightarrow \\ \begin{array}{r} 111101.11 \quad \text{Multiplicand, } B_{2C} \\ \times 0000110.1 \quad \text{Multiplier, } A_2 \\ \hline 11110111 \quad 2^0 \times B \\ 00000000 \\ 1111011100 \quad 2^2 \times B \\ 11110111000 \quad 2^3 \times B \\ 1010101100 \quad \text{Level 1 Carries} \\ 10101 \quad \text{Level 2 Carries} \\ \hline 110010001011 \quad \text{Product, } P_{2C} \text{ Mod } 2^8 \\ \hline \end{array} \\ \text{8-bit representation} \end{array} \end{array}$$

The true value of the 8-bit representation is obtained by negation,

$$10001.011_{2C} = 01110.101_2 = 14.625,$$

which, of course, is a negative number. This example illustrates what is called *Booth's algorithm* for fast signed multiplication, and is expressed as follows:

#### Algorithm 2.11: $A_2 \times B_{2C}$ or $A_{2C} \times B_{2C}$

- (1) Set  $n = 2k$ , where  $k(>m)$  is the larger of two numbers<sup>2</sup> counting both integer and fraction bits in  $k$  but excluding leading and trailing zeros.

<sup>2</sup>The two numbers are initially  $|A_2^k|$  of  $k$  bits and  $|B_2^m|$  of  $m$  bits or vice versa.

- (2) Generate the 2's complement of the negative number(s) by applying Eq. (2.14).  
 (3) Carry out steps (3) through (6) of Algorithm 2.10 applied to operands  $A_2$  and  $B_{2C}$  or  $A_{2C}$  and  $B_{2C}$ , represented as  $n$ -bit operands, to generate the product  $P_{2C}^n$  or  $P_2^n$ . Use Mod  $2^n$  arithmetic where applicable.

### 2.9.5 Binary Division

The division operation is generally more complex than that for multiplication. This is so because the result is often not an integer, though the dividend and divisor may be. Consider that  $A_2$  and  $B_2$  are binary operands each of  $n$  bits and that

$$A \div B = \sum_{i=0}^{n-1} Q_i 2^i + R/B, \quad (2.24)$$

where  $A$  is the *dividend*,  $B$  is the *divisor*,  $Q$  is the *quotient*, and  $R$  is the *remainder* such that  $0 \leq R < B$ . An integer quotient is expressed as the binary number  $Q_{n-1} \cdots Q_1 Q_0$ . From Eq. (2.24) there results the expression

$$A = B \cdot \sum_{i=0}^{n-1} Q_i 2^i + R, \quad (2.25)$$

which forms the basis for a *restoring* type of binary division procedure:

Begin with  $n - 1$  for a  $k$ -bit divisor and a  $(k + n)$ -bit dividend to yield an  $n$ -bit quotient and a  $k$ -bit remainder. If

$$A - 2^{n-1} B = A - b_{n-1} \cdots b_1 b_0 \underbrace{00 \cdots 0}_{n-1 \text{ zeros}} \geq 0,$$

$Q_{n-1} = 1$  or otherwise  $Q_{n-1} = 0$ . If  $Q_{n-1} = 1$ , the remaining quotient bits  $Q_{n-1} \cdots Q_1 Q_0$  are found beginning with  $A' = A - 2^{n-1} B$ . Then, if  $A' - 2^{n-2} B \geq 0$ ,  $Q_{n-2} = 1$  or otherwise  $Q_{n-2} = 0$ . Consequently, if  $Q_{n-2} = 1$ , the remaining quotients  $Q_{n-3} \cdots Q_1 Q_0$  are found beginning with  $A'' = A' - 2^{n-2} B$ , etc. The procedure just described mimics the familiar pencil-and-paper division method.

As an example, consider the following division operation  $A \div B$  with 5-bit operands:

#### EXAMPLE 2.30

$$\begin{array}{r} 00000101 = Q \\ B = 0101 \sqrt{00011011} = A \\ \underline{-00010100} = 2^2 \cdot B \\ 00000111 = A' = A - 2^2 B \\ \underline{-00000101} = 2^0 \cdot B \\ 0010 = R = A' - 2^0 B \end{array}$$

In this example, not all steps are shown, but the implied division process is

$$A - 2^4 B = A - 01010000 < 0, \quad Q_4 = 0$$

$$A - 2^3 B = A - 00101000 < 0, \quad Q_3 = 0$$

$$\begin{aligned} A - 2^2 B &= A - 00010100 > 0, \\ &= 00111 = A', \quad Q_2 = 1 \end{aligned}$$

$$A' - 2^1 B = A' - 00001010 > 0, \quad Q_1 = 0$$

$$\begin{aligned} A' - 2^0 B &= A' - 00000101 > 0, \quad Q_0 = 1 \\ &= 00000010 = A'' = R, \end{aligned}$$

where  $A - 2^5 B < 0$ ,  $A - 2^6 B < 0$ ,  $A - 2^7 B$ , etc., all yield quotient bits  $Q = 0$ . Notice that the subtractions can be carried out with 2's complement arithmetic according to Algorithm 2.9.

The following algorithm generalizes the binary division process as just presented:

Algorithm 2.12:  $A_2 \div B_2$

- (1) Set  $B$  to  $k$ -bits and  $A$  to  $(k + n)$ -bits.
- (2) Set  $i = n - 1$  and the remainder  $= A$ .
- (3) Set  $Q_i = 1$  if  $R - 2^i B \geq 0$  and subtract  $2^i B$  from  $A$ ; otherwise set  $Q_i = 0$  if  $R - 2^i B < 0$ .
- (4) Repeat step (2) for  $i = n - 2, n - 3, \dots, 1, 0$  to generate quotient bits  $Q_{n-2}, Q_{n-3}, \dots, Q_1, Q_0$  ending with the final  $n$ -bit quotient  $Q = Q_{n-1} \dots Q_1 Q_0$ .

Binary division involving numbers with fractions is handled in a manner similar to that for decimals. The bit position of the radix point measured from the LSB in the dividend is the same for the quotient. If a fraction exists in the divisor, the radix point of the quotient is that of the dividend minus that of the divisor all taken from the LSB.

Division involving negative numbers is most easily carried out as unsigned division with the result determined by the normal laws of algebra — that is, operands of like sign produce a positive quotient, while those of unlike sign produce a negative quotient. The remainder is given the same sign as the dividend. Signed division can be performed directly by using 2's complement, but the process requires many decision-making steps and, for this reason, is rarely used.

**High-Speed Division by Direct Quadratic Convergence** A great deal of effort has gone into making multiplication as fast and efficient as possible for use in high-speed computers. So it is only logical that use be made of this fact in generating suitable algorithms for high-speed division. Such methods, commonly used in modern computers, involve *iterative divide algorithms* and are *nonrestoring*. One such method features a system that operates on both the dividend  $D_D$  and the divisor  $D_S$  with equal multipliers so as to cause the divisor to converge quadratically on unity, thereby yielding the dividend as the quotient. This requires that at least the divisor  $D_S$  be represented as a fraction. Therefore, *in binary*

the divisor must be represented in the decimal range  $0.5 \leq \text{operand} < 1$ . If both operands are so represented, the direct quadratic convergence method is ideally suited for use with the mantissas in normalized FPN notation described in Section 2.8.

For this approach the quotient is given by

$$\begin{aligned} Q &= \frac{\text{DIVIDEND}}{\text{DIVISOR}} = \frac{D_D}{D_S} \\ &= \frac{D_D \cdot k_0 \cdot k_1 \cdot k_2 \cdots}{D_S \cdot k_0 \cdot k_1 \cdot k_2 \cdots} \Rightarrow \frac{Q}{1}. \end{aligned} \quad (2.26)$$

The process begins with  $D_S = 1 - \alpha$ , where  $\alpha = 1 - D_S$ . But since  $D_S < 1$ , it follows that  $\alpha < 1$ . Next choose

$$\begin{aligned} k_0 &= 1 + \alpha \\ &= 1 + (1 - D_S) = 2 - D_S, \end{aligned}$$

giving the updated product

$$D_S \cdot k_0 = (1 - \alpha)(1 + \alpha) = 1 - \alpha^2,$$

which is closer to 1 than  $D_S$ . Now set

$$k_1 = 1 + \alpha^2 = 1 + (1 - D_S k_0),$$

giving the updated product

$$D_S \cdot k_0 \cdot k_1 = (1 - \alpha^2)(1 + \alpha^2) = 1 - \alpha^4.$$

Continuing, set

$$k_2 = 1 + \alpha^4 = 1 + (1 - D_S k_0 k_1),$$

so that the updated product becomes

$$D_S \cdot k_0 \cdot k_1 \cdot k_2 = (1 - \alpha^4)(1 + \alpha^4) = 1 - \alpha^8, \text{ etc.}$$

This process continues until the desired number of iterations has been reached or until the updated product  $D_S \cdot k_0 \cdot k_1 \cdot k_2 \cdots = 1$ .

Notice that each  $k_j$  is 1 plus the radix complement of the product of  $D_S$  and all the  $k$  factors to that point. This can be generalized mathematically as

$$k_j = 1 + \left[ D_S \cdot \prod_{i=0}^{j-1} k_i \right]_{rC}. \quad (2.27)$$

Consider the following simple example for the division operation, which will be carried out in both decimal and binary:

$$\frac{D_D}{D_S} = \left[ \frac{0.375}{0.500} \right]_{10} = \left[ \frac{0.011}{0.100} \right]_2.$$

**EXAMPLE 2.31** In decimal:

$$\alpha = 1 - D_S = 0.5$$

$$k_0 = 1 + \alpha = 1.5$$

$$D_S k_0 = 0.75$$

$$k_1 = 1 + (0.75)_{10C} = 1.25$$

$$D_S k_0 k_1 = 0.9375.$$

$$k_2 = 1 + (0.9375)_{10C} = 1.0625$$

$$D_S k_0 k_1 k_2 = 0.99609375$$

$$k_3 = 1 + (0.99609375)_{10C} = 1.00390625$$

$$D_S k_0 k_1 k_2 k_3 = 0.999984741.$$

Therefore, after four iterations the quotient becomes

$$\begin{aligned} Q &= D_D k_0 k_1 k_2 k_3 = 0.749988556 \\ &= 0.749989 \text{ rounded to six places } (10^{-6}). \end{aligned}$$

Note that a fifth iteration with  $k_4 = 1.000015259$  produces  $Q = 0.750000$  rounded to six places.

In binary:

$$\alpha = 1 - D_S = 0.1$$

$$k_0 = 1 + \alpha = 1.1$$

$$D_S k_0 = 0.11$$

$$k_1 = 1 + (0.11)_{2C} = 1.01$$

$$D_S k_0 k_1 = 0.1111$$

$$k_2 = 1 + (0.1111)_{2C} = 1.0001$$

$$D_S k_0 k_1 k_2 = 0.11111111$$

$$k_3 = 1 + (0.11111111)_{2C} = 1.00000001$$

$$D_S k_0 k_1 k_2 k_3 = 0.1111111111111111.$$

Therefore, at the end of four iterations the quotient is

$$Q = D_D k_0 k_1 k_2 k_3 = 0.10111111111111010_2 \\ = 0.75_{10} \text{ after rounding.}$$

In 32-bit FPN notation the quotient  $Q$  would be given as

$$\underbrace{0}_{S} \underbrace{01111111}_{E} \underbrace{01111111111111010 \dots 00}_{M},$$

where the most significant 1 bit in the mantissa  $M$  has been omitted in agreement the IEEE normalized FPN notation discussed in Section 2.8.

In the simple example just given the value of  $\alpha$  was determined by the value of  $D_S$ . Because this is a quadratic convergence algorithm, the process starts off slowly, particularly if the divisor is close to  $\frac{1}{2}$ . The division calculations in computers can be speeded up by using ROM look-up tables to determine  $\alpha$  based on the divisor value. Furthermore, it is common practice to fix the number of iterations and then deal with rounding problem.

With some simplifying assumptions the following algorithm generalizes the quadratic convergence process for iterative division:

Algorithm 2.13:  $Q = D_D \div D_S$

- (1) Set  $D_S$  to normalized FPN form, retain the MSB 1 in the mantissa, and adjust the exponent as required by the FPN notation.
- (2) Calculate  $\alpha = 1 - D_S$  by using Algorithm 2.9.
- (3) Set  $k_0 = 1 + \alpha$  and calculate  $(D_S k_0)_{2C}$  by using Algorithms 2.10 and 2.6.
- (4) Set  $k_1 = 1 + (D_S k_0)_{2C}$  and calculate  $(D_S k_0 k_1)_{2C}$  as in step (3).
- (5) Repeat steps (1) through (4) for  $k_j = 1 + \left[ D_S \cdot \prod_{i=0}^{j-1} k_i \right]$  for all  $j = 2, 3, \dots$
- (6) Calculate  $Q = D_D \prod_{i=0}^{j-1} k_i$  when  $D_S \prod_{i=0}^{j-1} k_i = 1$ .

### 2.9.6 BCD Addition and Subtraction

Compared to binary arithmetic, BCD arithmetic is more complex, particularly with regard to hardware implementation. This is true since not all possible four-bit binary number states correspond to the BCD number system. The six number patterns 1010, 1011, 1100, 1101, 1110, and 1111 are not valid BCD states, as is indicated in Table 2.2.

**BCD Addition** BCD addition is similar to unsigned binary addition, except that a correction must be made any time a sum exceeds  $9_{10} = 1001_2$ . Summation begins with the least significant digit (LSD) and ends with the most significant digit (MSD). If the sum exceeds 1001 for any given digit, that sum is corrected by adding  $6_{10} = 0110_2$  with a carry of 0001 to the next MSD. The following example illustrates the addition process for two-decade BCD integers  $A_{BCD} = A_{10}A_1$  and  $B_{BCD} = B_{10}B_1$  represented in three-decade form:

**EXAMPLE 2.32**

$$\begin{array}{rcl}
056_{10} & \rightarrow & 0000\ 0101\ 0110_{BCD} \quad \text{Augend } A_{BCD} \\
+069_{10} & \rightarrow & +0000\ 0110\ 1001_{BCD} \quad \text{Addend } B_{BCD} \\
\hline
125_{10} & & 0000\ 1011\ 1111 \quad \text{Sum} \\
& & \quad 0110\ 0110 \quad \text{Correction} \\
& & \quad 1\ 1111\ 110 \quad \text{Carries} \\
& & \hline
& & 0001\ 0010\ 0101_{BCD} \quad \text{Result} = 125_{10}
\end{array}$$

The following algorithm generalizes the process just given for operands having both integers and fractions.

Algorithm 2.14:  $A_{BCD} + B_{BCD}$

- (1) Set BCD operands in descending decade order,  $A_{BCD} = \cdots A_{100}A_{10}A_1A_{.1}A_{.01}\cdots$  and  $B_{BCD} = \cdots B_{100}B_{10}B_1B_{.1}B_{.01}\cdots$  such that the MSDs for  $A$  and  $B$  are 0000 (null).
- (2) Set  $i = \text{LSD}$  for matching operand decades.
- (3) If  $A_i + B_i > 1001$  by Algorithm 2.8, add 0110 to that result and carry 0001 over to the next MSD. If  $A_i + B_i < 1001$ , add 0000 to the result and carry 0000 to the next MSD.
- (4) Repeat steps (2) and (3) for matching decades in the order of  $10^1i, 10^2i, 10^3i, \dots$ , MSD.

**BCD Subtraction** Negative BCD numbers are most conveniently represented in 10's complement (10C). This permits negative BCD numbers to be added according to Algorithm 2.14. The result, if negative, will be represented in 10C form requiring negation to obtain the true value.

**EXAMPLE 2.33**

$$\begin{array}{rcl}
08.25_{10} & \rightarrow & 08.25_{10} & \rightarrow & 0000\ 1000\ .0010\ 0101_{BCD} \\
-13.52_{10} & \rightarrow & +86.48_{10C} & \rightarrow & +1000\ 0110\ .0100\ 1000_{BCD)_{10C}} \\
-05.27_{10} & & 94.73_{10C} & & \hline
& & & & 1000\ 1110\ .0110\ 1101 \quad \text{Sum} \\
& & & + & \quad 0110 \quad \quad 0110 \quad \text{Correction} \\
& & & & \quad 1\ 11 \quad \quad 1\ 1 \quad \text{Carries} \\
& & & & \hline
& & & & 1001\ 0100\ .0111\ 0011_{BCD)_{10C}} \quad \text{Result}
\end{array}$$

The true (absolute) value of the result is found by negation to be

$$94.73_{10C)_{10C}} = 05.27_{10} \text{ or } 0000\ 0101\ .0010\ 0111_{BCD}.$$

Note that to convert directly from the BCD form,  $BCD)_{10C} = N_{2C} + 1010 = \bar{N}_2 + 1_{LSB} + 1010$  for the LSD but thereafter is  $BCD)_{10C} = \bar{N}_2 + 1010 = N_{9C}$ , discarding any carry overflow in either case.

The following algorithm generalizes the process of BCD subtraction.

Algorithm 2.15:  $A_{BCD} + B_{BCD)_{10C}}$  or  $A_{BCD)_{10C}} + B_{BCD)_{10C}}$

- (1) Convert any negative decimal number to its 10's complement (10C) by Algorithm 2.5 with  $r = 10$ .



- (2) Represent each operand in BCD form.  
 (3) Carry out steps (1) through (4) of Algorithm 2.14. If the result is negative, the true value is found by negation:  $[(result)_{10C}]_{10C} = (result)_{BCD}$ . If the result is positive, that result is the true value.

### 2.9.7 Floating-Point Arithmetic

Up to this point the arithmetic operations have involved fixed-point representation in which all bits of a binary number were represented. In many practical applications the numbers may require many bits for their representation. In Section 2.8 the floating-point number (FPN) system was discussed for just that reason. Now it is necessary to deal with the arithmetic associated with the FPN system, namely addition, subtraction, multiplication, and division.

**FPN Addition and Subtraction** Before two numbers can be added or subtracted one from the other, it is necessary that they have the same exponent. This is equivalent to aligning their radix points. From Eq. (2.17) for radix 2, consider the following two FPNs:

$$X = M_X \cdot 2^{E_X}$$

and

$$Y = M_Y \cdot 2^{E_Y}$$

Now, if for example  $E_X \geq E_Y$ , then  $Y$  is represented as  $M'_Y \cdot 2^{E'_Y}$ , where

$$M'_Y = \underbrace{.00 \cdots 0}_{E_X - E_Y \text{ zeros}} f_{-1} f_{-2} \cdots f_{-m} \quad \text{and} \quad E'_Y = E_Y + (E_X - E_Y) = E_X,$$

so that  $X + Y = (M_X + M'_Y) \cdot 2^{E_X}$  or  $X - Y = (M_X - M'_Y) \cdot 2^{E_X}$ , etc. Here,  $M_Y = .f_{-1} f_{-2} \cdots f_{-m}$  originally, but is now adjusted so that the exponents for both operands are the same. The addition or subtraction of the fractions  $M_X$  and  $M'_Y$  is carried out according to Algorithm 2.8 or Algorithm 2.9, respectively.

Consider the following examples of FPN addition:

#### EXAMPLE 2.34 — Addition

$$\begin{array}{rcl} 145.500_{10} & \rightarrow & 10010001.100_2 = X \\ + 27.625_{10} & & 00011000.101_2 = Y \\ \hline 173.125_{10} & & \end{array}$$

Comparing and equalizing the exponents  $E_X$  and  $E_Y$  gives

$$\begin{aligned} 145.500_{10} &= .10010001100 \times 2^8 \\ 27.625_{10} &= .00011011101 \times 2^8. \end{aligned}$$

In FPN notation the addition operation becomes

$$\begin{array}{rcl}
 145.500 & \rightarrow & \begin{array}{c} S \quad \quad E \quad \quad M \\ \overbrace{0} \quad \overbrace{10000111} \quad \overbrace{100100011000 \dots 00} \end{array} \\
 +27.625 & \rightarrow & \begin{array}{c} S \quad \quad E \quad \quad M \\ +0 \quad \overbrace{10000111} \quad \overbrace{000110111010 \dots 00} \end{array} , \\
 \hline
 173.125 & & \begin{array}{c} S \quad \quad E \quad \quad M \\ 0 \quad \overbrace{10000111} \quad \overbrace{101011010010 \dots 00} \end{array}
 \end{array}$$

where the exponents given in excess 127 form are  $127 + 8 = 135$  as discussed in Section 2.7. To represent the result in normalized FPN notation, the most significant 1 bit of the mantissa is omitted, yielding the result 0 10000111 010110100100  $\dots$  00 for the sign, exponent, and mantissa fields, respectively.

**EXAMPLE 2.35** — Subtraction in 2's complement

$$\begin{array}{rcl}
 -145.500 & \rightarrow & \begin{array}{c} S \quad \quad E \quad \quad M \\ \overbrace{0} \quad \overbrace{10000111} \quad \overbrace{011011101000 \dots 00} \end{array} \\
 +27.625 & \rightarrow & \begin{array}{c} S \quad \quad E \quad \quad M \\ +0 \quad \overbrace{10000111} \quad \overbrace{000110111010 \dots 00} \end{array} \\
 \hline
 -117.875 & & \begin{array}{c} S \quad \quad E \quad \quad M \\ 0 \quad \overbrace{10000111} \quad \overbrace{100010100010 \dots 00} \end{array}
 \end{array}$$

The true value is obtained by negation of the mantissa (taking its 2's complement), giving the sign magnitude result 1 10000111 011101011110  $\dots$  00, which is  $-117.875_{10}$ . In normalized FPN notation the MSB 1 bit of the mantissa would be omitted, giving the 2's complement result 1 10000111 00010100010  $\dots$  00.

**FPN Multiplication and Division** In some respects multiplication and division of FPNs is simpler than addition and subtraction from the point of view of the decision-making problem. Multiplication and division operations can be carried out by using the standard methods for such operations without the need to compare and equalize the exponents of the operands. The following generalizations of these processes illustrates the procedure involved.

The product of two operands  $X$  and  $Y$  in radix  $r$  is represented as follows:

$$\begin{aligned}
 P &= X \times Y \\
 &= (M_X \cdot r^{E_X}) \times (M_Y \cdot r^{E_Y}) \\
 &= (M_X \times M_Y) \cdot r^{(E_X + E_Y)} \\
 &= M_P \cdot r^{E_P},
 \end{aligned}$$

where the exponents are added following Algorithm 2.8 while the mantissas are multiplied by using Algorithm 2.10. The addition and multiplication of signed numbers is covered by Algorithms 2.9 and 2.11, respectively.

Similarly, for division in radix  $r$  the quotient is given by

$$\begin{aligned}
 Q &= X \div Y \\
 &= (M_X \cdot r^{E_X}) \div (M_Y \cdot r^{E_Y}) \\
 &= (M_X \div M_Y) \cdot r^{(E_X - E_Y)}.
 \end{aligned}$$

Here, the exponents are subtracted (added) in radix 2 (binary) by using the same algorithms as for addition, namely Algorithms 2.8 and 2.9. The division of the mantissas, on the other hand, is best accomplished by using the quadratic convergence iterative divide method discussed in Subsection 2.9.5, since the mantissas are usually represented in normalized FPN form. The following examples illustrate the multiplication and division of FPNs.

**EXAMPLE 2.36** — FPN signed-magnitude multiplication

$$\begin{array}{rcl}
 -7.25_{10} & \rightarrow & \begin{array}{c} S \\ 1 \end{array} \quad \begin{array}{c} E \\ 10000010 \end{array} \quad \begin{array}{c} M \\ 11101000 \dots 00 \end{array} \\
 \times 4.50_{10} & \rightarrow & +0 \quad +10000010 \quad \times 10010000 \dots 00 \\
 \hline
 -32.625_{10} & & 1 \quad 10000101 \quad 00000000 \dots 00 \\
 & & 000000000 \dots 00 \\
 & & 0000000000 \dots 00 \\
 & & 00000000000 \dots 00 \\
 & & 111010000000 \dots 00 \\
 & & 0000000000000 \dots 00 \\
 & & 00000000000000 \dots 00 \\
 & & 111010000000000 \dots 00 \\
 & & \hline
 & & 1000001010000000 \dots 00 \\
 & & \hline
 & & \text{23-bit representation}
 \end{array}$$

The result, given in normalized signed-magnitude 32-bit FPN form, is

$$1 \ 10000101 \ 000001010000000 \dots 0_{FPN} = -32.625_{10},$$

where the MSB 1 bit of the mantissa is omitted. Note that the mantissa has a magnitude  $.10000010100 \dots 00 \times 2^6 = 100000.101$ .

**EXAMPLE 2.37** — FPN signed-magnitude division

$$\frac{4.5}{-0.625} \Rightarrow -\frac{0.1001 \times 2^3}{0.1010} = \frac{X}{Y}$$

In FPN notation the operands are

$$X = 0 \ 10000010 \ 100100 \dots 00$$

$$Y = 1 \ 01111111 \ 101000 \dots 00.$$

Division of the mantissas  $M_X/M_Y = D_D/D_S$  by Algorithm 2.13:

$$\alpha = 1 - D_S = 1 - .101000 \dots 00 = .011000 \dots 00$$

$$k_0 = 1 + \alpha = 1.011000 \dots 00$$

$$D_S k_0 = .11011100 \dots 00$$

$$\begin{aligned}
 k_1 &= 1 + (D_S k_0)_{2C} = 1.00100100 \dots 00 \\
 D_S k_0 k_1 &= .11111010111100 \dots 00 \\
 k_2 &= 1 + (D_S k_0 k_1)_{2C} = 1.00000101000100 \dots 00 \\
 D_S k_0 k_1 k_2 &= .1111111111001100101111 \\
 k_3 &= 1 + (D_S k_0 k_1 k_2)_{2C} = 1.00000000000110011010001.
 \end{aligned}$$

After four iterations the quotient is given by

$$Q = (D_D k_0 k_1 k_2 k_3) = .11100110011001100011000 \times 2^3,$$

which is truncated to 23 bits. The quotient has a decimal value of

$$(0.899996755 \times 2^3)_{10} = 7.1999740450_{10}.$$

In normalized FPN signed-magnitude 32 bit form the quotient is given by

$$Q = 1 \ 10000010 \ 11001100110011000110000,$$

where the MSB 1 bit of the mantissa has been omitted as discussed in Section 2.8. Note that the subtraction of exponents  $E_X - E_Y$  is  $130 - 127 = 003_{10}$  or  $10000010_2$  in excess 127 code. The sign bits are added in binary giving  $S_X + S_Y = 0 + 1 = 1$ , where any carry (in this case 0) is discarded.

Algorithm 2.16: Signed-Magnitude  $(X \times Y)_{FPN}$  or  $(X \div Y)_{FPN}$

- (1) Set operands  $X$  and  $Y$  in IEEE normalized FPN form (see Section 2.7).
- (2) Add the exponents  $E_X$  and  $E_Y$  according to Algorithms 2.8 or 2.9.
- (3) If  $X \times Y$ , then multiply mantissa fractions according to Algorithm 2.10.
- (4) If  $X \div Y$ , then divide mantissas according to Algorithm 2.13.
- (5) Add the sign bits  $S_X + S_Y$  and discard the carry.
- (6) Set result in IEEE normalized FPN form.

### 2.9.8 Perspective on Arithmetic Codes

It should seem clear to the reader that certain arithmetic operations are more easily executed than others, depending on whether or not the operands are signed and depending on the number code used to carry out the operation. Table 2.9 is intended to show the general degree of difficulty of certain arithmetic operations relative to the arithmetic code (signed-magnitude, 2's complement, etc.) used.

Not indicated in Table 2.9 are the subdivisions within a given type of arithmetic operation. For example, no distinction is made between a direct division (restoring) algorithm and an iterative divide (nonrestoring) algorithm, which may differ significantly with regard to difficulty — the latter being easier for operands represented in FPN notation. As a general rule,

**Table 2.9 Arithmetic codes vs the degree of difficulty of arithmetic operations**

| Arithmetic Operation        | Unsigned Numbers | Signed-Magnitude | Two's Complement |
|-----------------------------|------------------|------------------|------------------|
| Unsigned addition           | Easy             | Easy             | Easy             |
| Signed addition/subtraction | —                | Difficult        | Easy             |
| Unsigned multiplication     | Fairly difficult | Fairly difficult | Difficult        |
| Signed multiplication       | —                | Fairly difficult | Difficult        |
| Unsigned division           | Difficult        | Difficult        | Very difficult   |

addition/subtraction of signed numbers involves relatively simple arithmetic manipulations of the operands compared to multiplication; and division requires more decision-making steps than multiplication. Also not shown in Table 2.9 are the 1's complement, BCD, and XS3 number codes, since they are not commonly used in computer numeric operations. Finally, a direct correspondence is implied between degree of difficulty and the hardware requirements to carry out a given arithmetic operation.

## 2.10 OTHER CODES

Most binary codes of value in digital design fall into one or more of the following ten categories:

|                          |                       |
|--------------------------|-----------------------|
| Weighted binary codes    | Unit distance codes   |
| Unweighted binary codes  | Reflective codes      |
| Biased codes             | Number codes          |
| Decimal codes            | Alphanumeric codes    |
| Self-complementing codes | Error detecting codes |

The previous sections have introduced examples of weighted binary codes, number codes, biased codes, and decimal codes. Number codes are those such as 2's and 1's complement that are used in addition/subtraction operations. Now, other codes (excluding alphanumeric codes) will be briefly discussed so as to provide a foundation for the developments in later chapters.

### 2.10.1 The Decimal Codes

Shown in Table 2.10 are seven decimal (10 state) codes that can be classified as either weighted or unweighted codes. All but one of these codes is weighted as indicated in the table. A weighted code can be converted to its decimal equivalent by using positional weights in a polynomial series as was done for the BCD code (1) discussed in Subsection 2.4.1. Code (2), the XS3 code, is a biased-weighted code considered in Subsection 2.4.1 and in Section 2.7. An unweighted code, such as code (7), cannot be converted to its decimal equivalent by any mathematical weighting procedure.

Not all weighted decimal codes are natural in the sense that their code weights cannot be derived from positive powers of 2 as is done for codes (1) and (2). Codes (3) through (6) in Table 2.10 are of this type. Code weights such as  $-1$ ,  $-2$ ,  $5$ , and  $6$  cannot be generated

**Table 2.10 Weighted and unweighted decimal codes**

| Dec.<br>Value | Weighted codes |              |             |               |              |              | Unweighted           |
|---------------|----------------|--------------|-------------|---------------|--------------|--------------|----------------------|
|               | (1)<br>(BCD)   | (2)<br>(XS3) | (3)<br>2421 | (4)<br>84-2-1 | (5)<br>86421 | (6)<br>51111 | (7)<br>Creeping Code |
| 0             | 0000           | 0011         | 0000        | 0000          | 00000        | 00000        | 00000                |
| 1             | 0001           | 0100         | 0001        | 0111          | 00001        | 00001        | 10000                |
| 2             | 0010           | 0101         | 0010        | 0110          | 00010        | 00011        | 11000                |
| 3             | 0011           | 0110         | 0011        | 0101          | 00011        | 00111        | 11100                |
| 4             | 0100           | 0111         | 0100        | 0100          | 00100        | 01111        | 11110                |
| 5             | 0101           | 1000         | 1011        | 1011          | 00101        | 10000        | 11111                |
| 6             | 0110           | 1001         | 1100        | 1010          | 01000        | 11000        | 01111                |
| 7             | 0111           | 1010         | 1101        | 1001          | 01001        | 11100        | 00111                |
| 8             | 1000           | 1011         | 1110        | 1000          | 10000        | 11110        | 00011                |
| 9             | 1001           | 1100         | 1111        | 1111          | 10001        | 11111        | 00001                |

by any positive integer power of 2, but they can still serve as code weights. As an example, consider how decimal 5 is represented by code (4):

$$\text{Decimal equivalent} = 5$$

$$\begin{aligned} 84\text{-}2\text{-}1 \text{ code representation} &= (1 \times 8) + (0 \times 4) + [1 \times (-2)] + [1 \times (-1)] \\ &= 1011. \end{aligned}$$

Note that there may be more than one combination of weighted bits that produce a given state. When this happens, the procedure is usually to use the fewest 1's. For example, decimal 7 can be represented by 00111 in code (5), 86421 code, but 01001 is preferred. An exception to this rule is the 2421 code discussed next.

Codes (2), (3), and (4) are examples of codes that have the unusual property of being *self-complementing*. This means that the 1's complement of the code number is the code for the 9's complement of the corresponding decimal number. In other words, the 1's complement of any state  $N$  (in decimal) is the same as the  $(9 - N)$  state in the same self-complementing code. As an example, the 1's complement of state 3 in XS3 (0110) is state 6 (1001) in that same code. The 1's and 9's complement number codes were discussed in Subsection 2.6.3 and are presented in Tables 2.7 and 2.8, respectively.

### 2.10.2 Error Detection Codes

There is another class of weighted or semiweighted codes with the special property that their states contain either an even number or an odd number of logic 1's (or 0's). Shown in Table 2.11 are four examples of such codes. This unique feature make these codes attractive as error-detecting (parity-checking) codes. Notice that both the 2-out-of-5 code (semiweighted) and the biquinary code (weighted 50 43210) must have two 1's in each of their 10 states and are, therefore, even-parity codes. In contrast, the one-hot code (weighted 9876543210) is an odd-parity code, since by definition it is allowed to have only a single 1 for any given state. Code (d) is no more than the BCD code with an attached odd parity-generating bit,  $P$ .

**Table 2.11 Error detection codes**

| Decimal Value | (a)                                  | (b)                                 | (c)                                 | (d)                        |
|---------------|--------------------------------------|-------------------------------------|-------------------------------------|----------------------------|
|               | Even Parity<br>2-out-of-5<br>(74210) | Even Parity<br>Biquinary<br>5043210 | Odd Parity<br>One-Hot<br>9876543210 | Odd Parity<br>BCD<br>P8421 |
| 0             | 11000                                | 01 00001                            | 0000000001                          | 10000                      |
| 1             | 00011                                | 01 00010                            | 0000000010                          | 00001                      |
| 2             | 00101                                | 10 00100                            | 0000000100                          | 00010                      |
| 3             | 00110                                | 10 01000                            | 0000001000                          | 10011                      |
| 4             | 01001                                | 01 10000                            | 0000010000                          | 00100                      |
| 5             | 01010                                | 10 00001                            | 0000100000                          | 10101                      |
| 6             | 01100                                | 10 00010                            | 0001000000                          | 10110                      |
| 7             | 10001                                | 10 00100                            | 0010000000                          | 00111                      |
| 8             | 10010                                | 10 01000                            | 0100000000                          | 01000                      |
| 9             | 10100                                | 10 10000                            | 1000000000                          | 11001                      |

The advantage of using an error-detecting code is that single-bit errors (those most likely to occur) are easily detected by a parity detector placed at the receiving end of a data bus. If a single error occurs, the parity is changed (odd-to-even or vice versa) and further processing can be delayed until the error is corrected. On the other hand, if two errors occur, the error cannot be detected by any simple means.

### 2.10.3 Unit Distance Codes

The last class of codes that will be discussed here are called *unit distance* codes, so called because only *one* bit is permitted to change between any two of their states—recall that in natural binary, adjacent states may differ by one or more bits. Three examples of unit distance codes are given in Table 2.12: (1) a decimal code, (2) a *reflective* unit distance code called *Gray code*, and (3) an XS3 decimal Gray code formed from the inner 10 states of code (2). The reflective character of the Gray and XS3 Gray codes are easily revealed by the fact that all bits except the MSB are mirrored across an imaginary plane located midway in the 16 states and 10 states, respectively, as indicated by the dashed lines. The unit distance property of the Gray code will be used in logic function graphics discussed at length in Chapter 4. Also, the unit distance and reflective character of the Gray code make it uniquely suitable as a position indicator code for rotating disks and shafts. Encoding errors produced by rotational irregularities can be detected and corrected by the use of such a code.

Although only a 4-bit Gray code is represented in Table 2.12, it should be noted that a Gray code of any number of bits is possible. Also, there are other unit distance codes that can be generated with any number of bits but they will most likely not be reflective.

### 2.10.4 Character Codes

The most common character code is called *ASCII* (pronounced “as-key”), the acronym for *American Standard Code for Information and Interchange*. ASCII code represents each character as a 7-bit binary string, hence a total of  $2^7 = 128$  characters, and is given in Table 2.13. This code encodes numerals, punctuation characters, all upper- and lowercase

**Table 2.12 Unit distance codes: (1) a decimal code (nonreflective); (2) four-bit Gray code (reflective); (3) XS3 Gray decimal code (reflective)**

| Decimal Value | (1) Decimal Code | (2) 4-Bit Gray Code | (3) XS3 Gray Decimal Code |
|---------------|------------------|---------------------|---------------------------|
| 0             | 0000             | 0000                | 0010                      |
| 1             | 0001             | 0001                | 0110                      |
| 2             | 0011             | 0011                | 0111                      |
| 3             | 0010             | 0010                | 0101                      |
| 4             | 0110             | 0110                | 0100                      |
| 5             | 1110             | 0111                | 1100                      |
| 6             | 1111             | 0101                | 1101                      |
| 7             | 1101             | 0100                | 1111                      |
| 8             | 1100             | 1100                | 1110                      |
| 9             | 0100             | 1101                | 1010                      |
| 10            | —                | 1111                | —                         |
| 11            | —                | 1110                | —                         |
| 12            | —                | 1010                | —                         |
| 13            | —                | 1011                | —                         |
| 14            | —                | 1001                | —                         |
| 15            | —                | 1000                | —                         |

**Table 2.13 ASCII character code**

|                                                             |           | a <sub>6</sub> a <sub>5</sub> a <sub>4</sub> (column) |          |          |          |          |          |          |          |
|-------------------------------------------------------------|-----------|-------------------------------------------------------|----------|----------|----------|----------|----------|----------|----------|
| a <sub>3</sub> a <sub>2</sub> a <sub>1</sub> a <sub>0</sub> | Row (Hex) | 000<br>0                                              | 001<br>1 | 010<br>2 | 011<br>3 | 100<br>4 | 101<br>5 | 110<br>6 | 111<br>7 |
| 0000                                                        | 0         | NUL                                                   | DLE      | SP       | 0        | @        | P        | '        | p        |
| 0001                                                        | 1         | SOH                                                   | DC1      | !        | 1        | A        | Q        | a        | q        |
| 0010                                                        | 2         | STX                                                   | DC2      | ”        | 2        | B        | R        | b        | r        |
| 0011                                                        | 3         | ETX                                                   | DC3      | #        | 3        | C        | S        | c        | s        |
| 0100                                                        | 4         | EOT                                                   | DC4      | \$       | 4        | D        | T        | d        | t        |
| 0101                                                        | 5         | ENQ                                                   | NAK      | %        | 5        | E        | U        | e        | u        |
| 0110                                                        | 6         | ACK                                                   | SYN      | &        | 6        | F        | V        | f        | v        |
| 0111                                                        | 7         | BEL                                                   | ETB      | '        | 7        | G        | W        | g        | w        |
| 1000                                                        | 8         | BS                                                    | CAN      | (        | 8        | H        | X        | h        | x        |
| 1001                                                        | 9         | HT                                                    | EM       | )        | 9        | I        | Y        | i        | y        |
| 1010                                                        | A         | LF                                                    | SUB      | *        | :        | J        | Z        | j        | z        |
| 1011                                                        | B         | VT                                                    | ESC      | +        | ;        | K        | [        | k        | {        |
| 1100                                                        | C         | FF                                                    | FS       | ,        | <        | L        | \        | l        |          |
| 1101                                                        | D         | CR                                                    | GS       | -        | =        | M        | ]        | m        | }        |
| 1110                                                        | E         | SO                                                    | RS       | .        | >        | N        | ^        | n        | ~        |
| 1111                                                        | F         | SI                                                    | US       | /        | ?        | O        | _        | o        | DEL      |



alphabet letters, and a variety of printer and typewriter control abbreviations. An eighth bit (not shown) is often used with the ASCII code for error detection purposes.

Another common character code is known as *EBCDIC* (pronounced “ebb-see-dick”), the acronym for *extended BCD interchange code*. It uses 8-bit BCD strings so as to encode a 256-character set.

## FURTHER READING

Literature on number systems and arithmetic is extensive. Many journal articles and most texts on digital logic design cover these subjects to one extent or another. Portions of this chapter regarding number systems are taken from contributions by Tinder to *The Electrical Engineering Handbook*, cited here. Recognized classic treatments of number systems and arithmetic include those of Garner, Hwang, and Knuth. The IEEE publication on the standard for floating-point arithmetic is also frequently cited. These references, together with recent texts covering the subject areas, are cited here.

- [1] H. L. Garner, “Number Systems and Arithmetic,” in *Advances in Computers*, Vol. 6. Academic Press, New York, 1965, pp. 131–194.
- [2] K. Hwang, *Computer Arithmetic*. John Wiley & Sons, New York, 1978.
- [3] IEEE, *IEEE Standard for Binary Floating-Point Arithmetic* (ANSI/IEEE Std 754-1985) The Institute of Electrical and Electronic Engineers, New York, 1985.
- [4] D. E. Knuth, *The Art of Computer Programming: Seminumerical Algorithms*, Vol. 2. Addison-Wesley, Reading, MA, 1969.
- [5] V. P. Nelson, H. T. Nagle, B. D. Carroll, and J. D. Irwin, *Digital Logic Circuit Analysis and Design*. Prentice Hall, Englewood Cliffs, NJ, 1995.
- [6] L. H. Pollard, *Computer Design and Architecture*. Prentice Hall, Englewood Cliffs, NJ, 1990.
- [7] A. W. Shaw, *Logic Circuit Design*. Saunders College Publishing, Fort Worth, TX, 1993.
- [8] R. F. Tinder, *Digital Engineering Design: A Modern Approach*, Prentice Hall, Englewood Cliffs, NJ, 1991.
- [9] R. F. Tinder, “Number Systems,” in *The Electrical Engineering Handbook*, 2nd ed. (R. C. Dorf, Ed.). CRC Press, 1997, pp. 1991–2006.
- [10] C. Tung, “Arithmetic,” *Computer Science* (A. F. Cardenas *et al.*, Eds.), Chapter 3. Wiley-Interscience, New York, 1972.
- [11] J. F. Wakerly, *Digital Design Principles and Practice*, 2nd ed. Prentice Hall, Englewood Cliffs, NJ, 1994.

## PROBLEMS

*Note:* Use Tables P2.1, P2.2, and P2.3 as needed in working the following problems.

**2.1** Convert the following decimal numbers to binary:

- (a) 5
- (b) 14
- (c) 39
- (d) 107.25
- (e) 0.6875

- 2.2** Convert the following binary numbers to decimal by using the method of positional weights:
- (a) 0110
  - (b) 1011
  - (c) 11001
  - (d) 11011001.11
  - (e) 0.01011
- 2.3** Convert the decimal numbers of Problem 2.1 to BCD.
- 2.4** Convert the binary numbers of Problem 2.2 to BCD. To do this, add trailing and leading 0's as required.
- 2.5** Convert the following BCD numbers to binary:
- (a) 00010011
  - (b) 01010111
  - (c) 0101000110
  - (d) 1001000.00100101
  - (e) 0.100001110101
- 2.6** Convert the decimal numbers in Problem 2.1 to XS3.
- 2.7** Convert the BCD numbers in Problem 2.5 to XS3.
- 2.8** Convert the binary numbers in Problem 2.2 to BCH.
- 2.9** Convert the BCD numbers in Problem 2.5 to BCO.
- 2.10** Convert the following numbers to binary:
- (a)  $613520_8$
  - (b)  $2FD6A25B_{16}$
  - (c)  $11110011100.011_{XS3}$
  - (d)  $6!$
- 2.11** Convert the following fractions as indicated:
- (a)  $0.534_{10} \rightarrow N_2$  rounded to 8 bits.
  - (b)  $0.3DF2_{16} \rightarrow N_2$  rounded to 8 bits.
  - (c)  $0.534_{10} \rightarrow N_{16}$  terminated at 4 digits.
  - (d)  $0.5427_8 \rightarrow N_2$  rounded to 8 bits.
- 2.12** Convert the following numbers to *signed-magnitude* binary form:
- (a)  $+56.25_{10}$
  - (b)  $-94.625_{10}$
  - (c)  $-7BD.5_{16}$
  - (d)  $+125_8$
  - (e)  $-0110101.10011_{BCD}$
- 2.13** Give the radix complement representation for the following numbers:
- (a) The 10's complement of  $47.63_{10}$

- (b) The 2's complement of  $011011101.1101_2$
- (c) The 8's complement of  $501.74_8$
- (d) The 16's complement of  $AF3.C8_{16}$

**2.14** Represent the following numbers in IEEE normalized FPN<sub>2</sub> form:

- (a)  $1101011.1011_2$
- (b)  $+27.6875_{10}$
- (c)  $-145.500_{10}$

**2.15** Add the following binary additions and verify in decimal:

- (a)  $10 + 11$
- (b)  $101 + 011$
- (c)  $10111 + 01110$
- (d)  $101101.11 + 011010.10$
- (e)  $0.1100 + 1.1101$

**2.16** Carry out the following binary subtraction operations in 2's complement and verify in decimal:

- (a)  $01100 - 00101$
- (b)  $0111011 - 0011001$
- (c)  $01001000 - 01110101$
- (d)  $010001.0101 - 011011.1010$
- (e)  $00.011010 - 01.110001$

**2.17** Repeat Problem 2.16 in 1's complement.

**2.18** Carry out the following binary multiplication operations and verify in decimal:

- (a)  $11 \times 0101$
- (b)  $11101 \times 1111011$
- (c)  $1001.10 \times 11101.11$
- (d)  $110.011 \times 1101.0101$
- (e)  $0.1101 \times 0.01111$

**2.19** Carry out the following complement multiplications and verify in decimal:

- (a)  $00000111 \times -00001101$
- (b)  $110 \times -11101$  ( $k = 5$ )
- (c)  $-11.01 \times 101.11$  ( $k = 5$ )
- (d)  $111.111 \times -1.101$  ( $k = 6$ )

(Hint: Consider switching minuend and subtrahend operands if it yields less work.)

**2.20** Find the quotient for each of the following division operations by using the binary equivalent of the familiar "pencil-and-paper" method used in long division of decimal numbers. Show work details.

- (a)  $1100 \div 100$
- (b)  $111111 \div 1001$
- (c)  $11001.1 \div 011.11$  (Carry out quotient to the  $2^{-2}$  bit and give the remainder)
- (d)  $100 \div 1010$  (Carry out quotient to the  $2^{-6}$  bit and give remainder)

- 2.21** Use the direct quadratic convergence method to obtain the quotient for the following fractions. To do this, use Eqs. (2.26) and (2.27).
- $(0.25 \div 0.75)_{10}$  in decimal. Find  $Q$  after three iterations and rounded to  $10^{-5}$ .
  - $(0.01 \div 0.11)_2$  in binary. Compare  $Q$  after two iterations rounded to  $2^{-8}$  with  $Q$  after three iterations rounded to  $2^{-16}$ . For comparison, use decimal values derived from the binary results.
- 2.22** Carry out the following hexadecimal operations and verify in decimal:
- $1A8 + 67B$
  - $ACEF1 + 16B7D$
  - $1273_{16} - 3A8$
  - $89_{16} \times 1A3$
  - $A2 \times 15BE3$
  - $1EC87 \div A5$  (Hint: Use decimal  $\leftrightarrow$  hex methods with Table P2.3.)
- 2.23** Convert the following decimal numbers to BCD with the MSDs null (0000), then carry out the indicated arithmetic in BCD by using Algorithms 2.14 and 2.15 in Subsection 2.9.6:
- $049_{10} + 078_{10}$
  - $168.6_{10} + 057.5_{10}$
  - $093_{10} - 067_{10}$
  - $034.79_{10} - 156.23_{10}$
- 2.24** Perform the FPN arithmetic indicated below. To do this follow the examples in Subsection 2.9.7.
- $135.25_{10} + 54.625_{10}$
  - $54.625_{10} - 135.25_{10}$
  - $3.75_{10} \times 5.0625_{10}$
  - $4.50_{10} \times (-2.3125_{10})$
  - $6.25 \div (-0.375_{10})$
- Note: Use the sign-magnitude FPN system for parts (d) and (e) following Examples 2.36 and 2.37.
- 2.25** To add XS3 numbers, a correction by either adding or subtracting 0011 is necessary depending on whether or not a 1 carry is generated. Study, then write an algorithm for the addition in XS3 numbers.
- 2.26** Prove that a self-complementing unit-distance code is not possible.
- 2.27** An inspection of the binary and Gray codes in Tables 2.1 and 2.12 indicates a unique relationship between these codes. Examine these codes and devise a simple algorithm that will permit direct "pencil-and-paper" conversion between them, binary-to-Gray or vice versa.
- 2.28** Decipher the following ASCII code. It is given in hexadecimal, MSD first.

57 68 61 74 69 73 79 6F 75 72 6E 61 6D 65 3F

### Table P2.1 Powers of 2

|   |     | 2 <sup>n</sup> | n   | 2 <sup>-n</sup> |       |     |     |     |     |     |     |     |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|---|-----|----------------|-----|-----------------|-------|-----|-----|-----|-----|-----|-----|-----|-----|-----|----|--|--|--|--|--|--|--|--|--|--|--|--|--|--|
|   |     | 1              | 0   | 1.0             |       |     |     |     |     |     |     |     |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   |     | 2              | 1   | 0.5             |       |     |     |     |     |     |     |     |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   |     | 4              | 2   | 0.25            |       |     |     |     |     |     |     |     |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   |     | 8              | 3   | 0.125           |       |     |     |     |     |     |     |     |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   |     | 16             | 4   | 0.062           | 5     |     |     |     |     |     |     |     |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   |     | 32             | 5   | 0.031           | 25    |     |     |     |     |     |     |     |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   |     | 64             | 6   | 0.015           | 625   |     |     |     |     |     |     |     |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   |     | 128            | 7   | 0.007           | 812   | 5   |     |     |     |     |     |     |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   |     | 256            | 8   | 0.003           | 906   | 25  |     |     |     |     |     |     |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   |     | 512            | 9   | 0.001           | 953   | 125 |     |     |     |     |     |     |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   | 1   | 024            | 10  | 0.000           | 976   | 562 | 5   |     |     |     |     |     |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   | 2   | 048            | 11  | 0.000           | 488   | 281 | 25  |     |     |     |     |     |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   | 4   | 096            | 12  | 0.000           | 244   | 140 | 625 |     |     |     |     |     |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   | 8   | 192            | 13  | 0.000           | 122   | 070 | 312 | 5   |     |     |     |     |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   | 16  | 384            | 14  | 0.000           | 061   | 035 | 156 | 25  |     |     |     |     |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   | 32  | 768            | 15  | 0.000           | 030   | 517 | 578 | 125 |     |     |     |     |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   | 65  | 536            | 16  | 0.000           | 015   | 258 | 789 | 062 | 5   |     |     |     |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   | 131 | 072            | 17  | 0.000           | 007   | 629 | 394 | 531 | 25  |     |     |     |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   | 262 | 144            | 18  | 0.000           | 003   | 814 | 697 | 265 | 625 |     |     |     |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   | 524 | 288            | 19  | 0.000           | 001   | 907 | 348 | 632 | 812 | 5   |     |     |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   | 1   | 048            | 576 | 20              | 0.000 | 000 | 953 | 674 | 316 | 406 | 25  |     |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   | 2   | 097            | 152 | 21              | 0.000 | 000 | 476 | 837 | 158 | 203 | 125 |     |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   | 4   | 194            | 304 | 22              | 0.000 | 000 | 238 | 418 | 579 | 101 | 562 | 5   |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   | 8   | 388            | 608 | 23              | 0.000 | 000 | 119 | 209 | 289 | 550 | 781 | 25  |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   | 16  | 777            | 216 | 24              | 0.000 | 000 | 059 | 604 | 644 | 775 | 390 | 625 |     |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   | 33  | 554            | 432 | 25              | 0.000 | 000 | 029 | 802 | 322 | 387 | 695 | 312 | 5   |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   | 67  | 108            | 864 | 26              | 0.000 | 000 | 014 | 901 | 161 | 193 | 847 | 656 | 25  |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   | 134 | 217            | 728 | 27              | 0.000 | 000 | 007 | 450 | 580 | 596 | 923 | 828 | 125 |     |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   | 268 | 435            | 456 | 28              | 0.000 | 000 | 003 | 725 | 290 | 298 | 461 | 914 | 062 | 5   |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|   | 536 | 870            | 912 | 29              | 0.000 | 000 | 001 | 862 | 645 | 149 | 230 | 957 | 031 | 25  |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
| 1 | 073 | 741            | 824 | 30              | 0.000 | 000 | 000 | 931 | 322 | 574 | 615 | 478 | 515 | 625 |    |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
| 2 | 147 | 483            | 648 | 31              | 0.000 | 000 | 000 | 465 | 661 | 287 | 307 | 739 | 257 | 812 | 5  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
| 4 | 294 | 967            | 296 | 32              | 0.000 | 000 | 000 | 232 | 830 | 643 | 653 | 869 | 628 | 906 | 25 |  |  |  |  |  |  |  |  |  |  |  |  |  |  |

**Table P2.2 Hexadecimal addition table**

|   | 0 | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  |
|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0 | 0 | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  |
| 1 | 1 | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  | 10 |
| 2 | 2 | 3  | 4  | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  | 10 | 11 |
| 3 | 3 | 4  | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  | 10 | 11 | 12 |
| 4 | 4 | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  | 10 | 11 | 12 | 13 |
| 5 | 5 | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  | 10 | 11 | 12 | 13 | 14 |
| 6 | 6 | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  | 10 | 11 | 12 | 13 | 14 | 15 |
| 7 | 7 | 8  | 9  | A  | B  | C  | D  | E  | F  | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
| 8 | 8 | 9  | A  | B  | C  | D  | E  | F  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 |
| 9 | 9 | A  | B  | C  | D  | E  | F  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 |
| A | A | B  | C  | D  | E  | F  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 |
| B | B | C  | D  | E  | F  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 1A |
| C | C | D  | E  | F  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 1A | 1B |
| D | D | E  | F  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 1A | 1B | 1C |
| E | E | F  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 1A | 1B | 1C | 1D |
| F | F | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 1A | 1B | 1C | 1D | 1E |

**Table P2.3 Hexadecimal multiplication table**

|   | 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  |
|---|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0 | 0 | 0 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
| 1 | 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | A  | B  | C  | D  | E  | F  |
| 2 | 0 | 2 | 4  | 6  | 8  | A  | C  | E  | 10 | 12 | 14 | 16 | 18 | 1A | 1C | 1E |
| 3 | 0 | 3 | 6  | 9  | C  | F  | 12 | 15 | 18 | 1B | 1E | 21 | 24 | 27 | 2A | 2D |
| 4 | 0 | 4 | 8  | C  | 10 | 14 | 18 | 1C | 20 | 24 | 28 | 2C | 30 | 34 | 38 | 3C |
| 5 | 0 | 5 | A  | F  | 14 | 19 | 1E | 23 | 28 | 2D | 32 | 37 | 3C | 41 | 46 | 4B |
| 6 | 0 | 6 | C  | 12 | 18 | 1E | 24 | 2A | 30 | 36 | 3C | 42 | 48 | 4E | 54 | 5A |
| 7 | 0 | 7 | E  | 15 | 1C | 23 | 2A | 31 | 38 | 3E | 46 | 4D | 54 | 5B | 62 | 69 |
| 8 | 0 | 8 | 10 | 18 | 20 | 28 | 30 | 38 | 40 | 48 | 50 | 58 | 60 | 68 | 70 | 78 |
| 9 | 0 | 9 | 12 | 1B | 24 | 2D | 36 | 3E | 48 | 51 | 5A | 63 | 6C | 75 | 7E | 87 |
| A | 0 | A | 14 | 1E | 28 | 32 | 3C | 46 | 50 | 5A | 64 | 6E | 78 | 82 | 8C | 96 |
| B | 0 | B | 16 | 21 | 2C | 37 | 42 | 4D | 58 | 63 | 6E | 79 | 84 | 8F | 9A | A5 |
| C | 0 | C | 18 | 24 | 30 | 3C | 48 | 54 | 60 | 6C | 78 | 84 | 90 | 9C | A8 | B4 |
| D | 0 | D | 1A | 27 | 34 | 41 | 4E | 5B | 68 | 75 | 82 | 8F | 9C | A9 | B6 | C3 |
| E | 0 | E | 1C | 2A | 38 | 46 | 54 | 62 | 70 | 7E | 8C | 9A | A8 | B6 | C4 | D2 |
| F | 0 | F | 1E | 2D | 3C | 4B | 5A | 69 | 78 | 87 | 96 | A5 | B4 | C3 | D2 | E1 |

*This page intentionally left blank*

# Background for Digital Design

---

## 3.1 INTRODUCTION

The contents of this chapter are considered all important to the reader's understanding of the remainder of this text and, hence, to an understanding of modern digital design methods. In this chapter the reader will learn mixed logic notation and symbology, Boolean algebra, and the reading and construction of logic circuits. Besides becoming the industrial standard, mixed logic notation and symbology, once learned, offers a remarkably simple, direct means of reading and constructing logic circuits and timing diagrams. Use will be made of the CMOS logic family to develop this symbology. Other logic families, such as NMOS and TTL, are discussed in Appendix A.

This chapter goes beyond the usual treatment of Boolean algebra to present what is called XOR algebra, an extension of Boolean algebra that deals with functions that have become very important in circuit design, particularly in arithmetic circuit design. CMOS realizations of XOR functions have, in a special sense, revolutionized thinking along these lines, making the use of such functions much more appealing to the logic designer.

## 3.2 BINARY STATE TERMINOLOGY AND MIXED LOGIC NOTATION

Digital systems are switching devices that operate in only one of two possible states at any given time, but that can be switched back and forth from one state to another at very high speed (millions of times per second). The two states are high voltage (HV) and low voltage (LV). The LV and HV levels are usually taken as 0 V and 2 to 5 V, respectively, for common CMOS logic circuits.

To design a useful digital device, meaningful logic names must be assigned to the inputs and outputs of a logic circuit so that their physical interpretation in terms of voltage levels can be made unambiguously. This requires the use of a notation that can easily *bridge the gap* between the logic domain in which the device is designed, and the physical domain in which the device is to be operated. The following subsection defines this notation.

### 3.2.1 Binary State Terminology

A state is said to be *active* if it is the condition for causing something to happen. And for every active state there must exist one that is *inactive*. In the binary (base 2) system of 1's



and 0's, these descriptors take the following meaning:

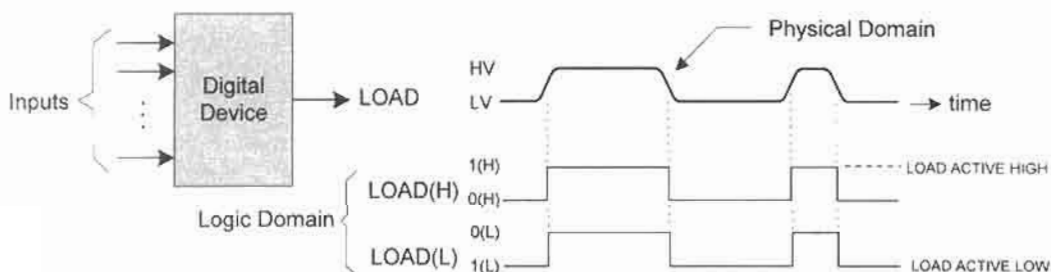
*Logic 1 is the ACTIVE state*  
*Logic 0 is the INACTIVE state*

Thus, in the *logic domain*, logic 1 is assigned to the active condition while logic 0 is assigned to the inactive condition. This will always be so.

A symbol that is attached to the name of a signal and that establishes which physical state, HV or LV, is to be the active state for that signal, is called the *activation level indicator*. The activation level indicators used in this text are

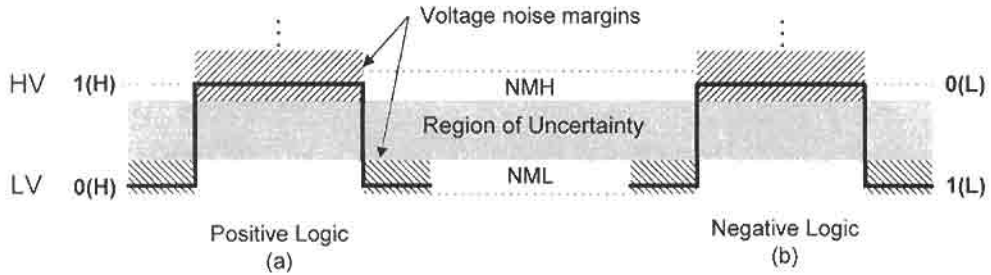
(H) meaning ACTIVE HIGH  
 (L) meaning ACTIVE LOW

Thus, a line signal LOAD(H) is one for which the active state occurs at high voltage (HV), and LOAD(L) is one for which the active state occurs at low voltage (LV). This is illustrated in Fig. 3.1. Here, the name LOAD is the physical waveform output of a digital device, and LOAD(H) and LOAD(L) are equivalent logical interpretations of that physical waveform. Notice that logic waveforms are rectangular (i.e., with zero rise and fall times), whereas physical waveforms must have finite rise and fall times. Finite rise and fall times are a consequence of the fact that changes in the physical state of anything cannot occur instantaneously. Logic level transitions, on the other hand, are non-physical and occur abruptly at the active and inactive transition points of the physical waveform, as indicated by the vertical dotted lines in Fig. 3.1. Also, the physical waveforms in Fig. 3.1 have amplitudes measured in terms of voltage whereas logic waveforms have amplitudes indicated by the logic levels 0 and 1. Labels such as LOAD(H) or LD(H) and LOAD(L) or LD(L) are commonly referred to as *polarized mnemonics*. The word “polarized” refers to the use of activation level indicator symbols, (H) and (L). Thus, LD(L) means LOAD active (or asserted) low, and LD(H) refers to LOAD active (or asserted) high.



**FIGURE 3.1**

Mixed logic interpretation of a physical waveform showing a digital device with its voltage waveform and the positive and negative logic interpretations of the waveform.

**FIGURE 3.2**

Logic interpretation of voltage waveforms. (a) Positive logic. (b) Negative logic.

What results from an inspection of the physical and logic waveforms in Fig. 3.1 is the important conclusion

$$\begin{aligned} \text{HV corresponds to } 1(\text{H}) = 0(\text{L}) \\ \text{LV corresponds to } 0(\text{H}) = 1(\text{L}). \end{aligned} \quad (3.1)$$

Relations (3.1) represent the essence of *mixed logic notation* and are used throughout this text for the purpose of converting from the logic domain to the physical domain or vice versa. Thus, the physical state of HV is represented by either 1(H) or 0(L) in the logic domain while the physical state of LV is represented by either 0(H) or 1(L) in the logic domain. The expression “mixed logic” applies to the use of both the *positive logic* and *negative logic* systems within a given application.

The positive and negative logic systems, which follow from Eqs. (3.1), are presented in Fig. 3.2. Here, the two systems are shown on logic waveform pulses similar to those shown in Fig. 3.1. The high noise margin (NMH) and low noise margin (NML) are included as a reminder that their inner boundaries are also the inner limits of the logic states (1 and 0) as well as the outer limits of the uncertainty region. A signal whose value lies in the uncertainty region cannot be taken as either logic 1 or logic 0.

The digital device shown in Fig. 3.3 illustrates the use of *polarized mnemonics* in the mixed logic digital system. Shown here are two inputs, LD(H) and CNT(L), and one output, DONE(H). The input LD(H) is said to arrive from a positive logic source (active high) while CNT(L) arrives from a negative logic source (hence, active low). The output DONE(H) is delivered to the next stage as a positive logic source (active high). LD and CNT,

**FIGURE 3.3**

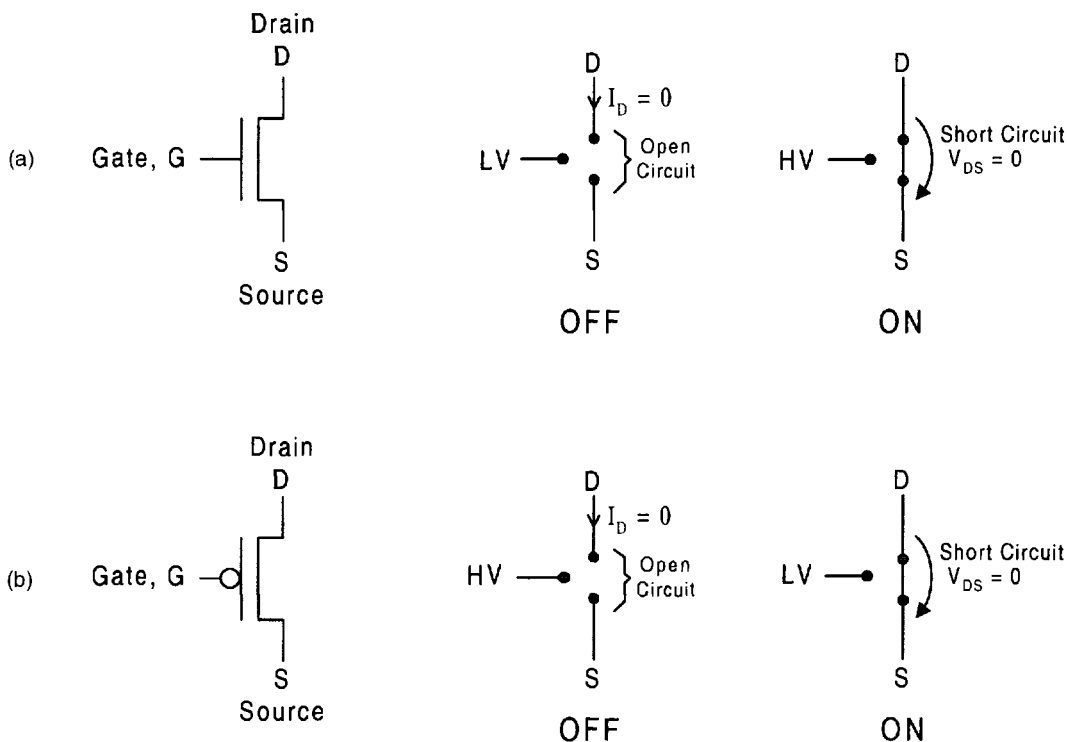
Polarized mnemonics applied to a digital device.

which represent LOAD and COUNT, respectively, are meaningful abbreviations called mnemonics.

### 3.3 INTRODUCTION TO CMOS TERMINOLOGY AND SYMBOLOGY

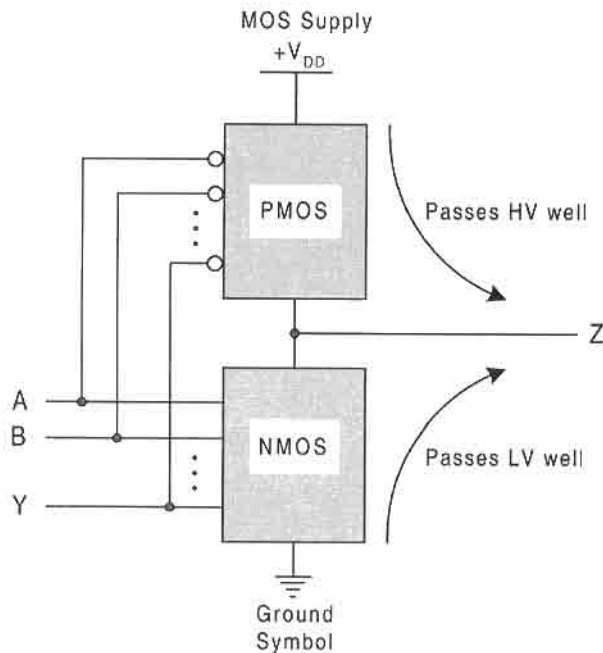
Complementary MOSFET (CMOS) switching circuits are composed of n-type MOSFETs (NMOS for short) and p-type MOSFETs (PMOS). As a help in reading and constructing CMOS switching circuits, the simplified symbols and ideal equivalent circuits for both types are given in Fig. 3.4. Thus, for either, the OFF condition is always an *open circuit* while the ON condition is always a *short circuit*. But the voltage levels causing the ON and OFF conditions for NMOS and PMOS are opposite; hence, they are called *complementary*. Notice that the voltage to produce the ON or OFF condition is always applied to the gate, G, and that the drain-to-source is either nonconducting ( $I_{Drain} = 0$ ) for the OFF condition or conducting ( $V_{DS} = 0$ ) for the ON condition. Use of Fig. 3.4 makes reading and construction of CMOS circuits very easy. However, knowledge of which terminal is the drain and which is the source is important only when configuring at the transistor circuit layout level.

Proper CMOS circuit construction requires that the NMOS and PMOS sections be positioned as shown in Fig. 3.5. The reason for this particular configuration is that NMOS passes



**FIGURE 3.4**

Symbols and ideal equivalent circuits for n and p MOSFETs: (a) NMOS. (b) PMOS.

**FIGURE 3.5**

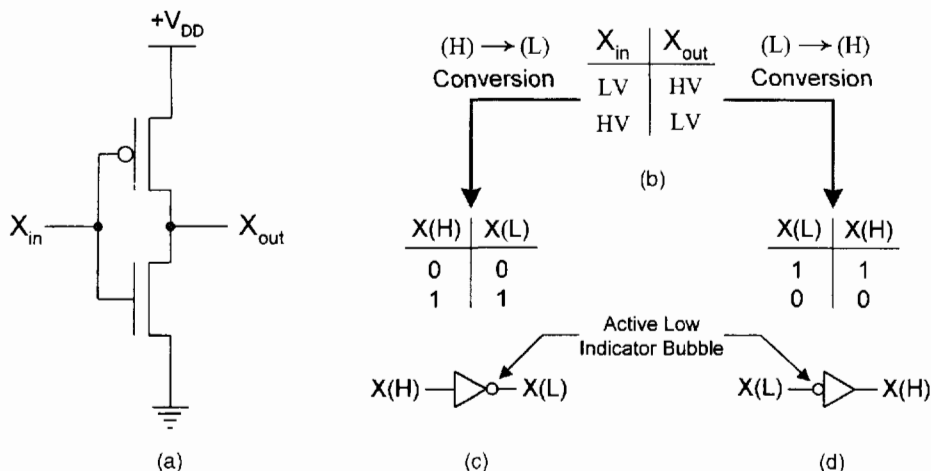
Proper PMOS and NMOS placement for generalized CMOS gate configurations.

LV well but not HV. Conversely, PMOS passes HV well but not LV. The proper placement of the NMOS and PMOS sections results in a sharp, relatively undistorted waveform. Inverting this configuration would require that the NMOS and PMOS sections pass voltage levels that they do not pass well, resulting in a distortion of the voltage waveform. Therefore, the PMOS section is always placed on the HV end with the NMOS on the LV side, as in Fig. 3.5.

### 3.4 LOGIC LEVEL CONVERSION: THE INVERTER

When a positive logic source is converted to a negative logic source, or vice versa, *logic level conversion* is said to occur. The physical device that performs logic level conversion is called the *inverter*. Shown in Fig. 3.6a is the CMOS version of the inverter. It is a CMOS inverter because it is composed of both NMOS and PMOS cast in the complementary configuration of Fig. 3.5. The physical truth table, shown in Fig. 3.6b, is easily understood by referring to Fig. 3.4. The logic interpretations and *conjugate logic symbols* that derive from the physical truth table are shown in Figs. 3.6c and 3.6d. The conjugate logic circuit symbols are used to indicate the logic level conversion  $X(H) \rightarrow X(L)$  or  $X(L) \rightarrow X(H)$  depending on where the *active low indicator bubble* is located. The designation “conjugate” indicates that the symbols are interchangeable, as they must be since they are derived from the same physical device (the inverter).

The CMOS inverter is used here for the purpose of developing the concept of logic level conversion. However, there are versions of the inverter that belong to logic families other than the CMOS family. These include the NMOS and TTL families, all of which yield

**FIGURE 3.6**

The inverter, its I/O behavior, and its two logic interpretations. (a) The CMOS transistor circuit. (b) Physical truth table. (c) Active-high-to-active-low conversion and logic circuit symbol. (d) Active-low-to-active-high conversion and logic circuit symbol.

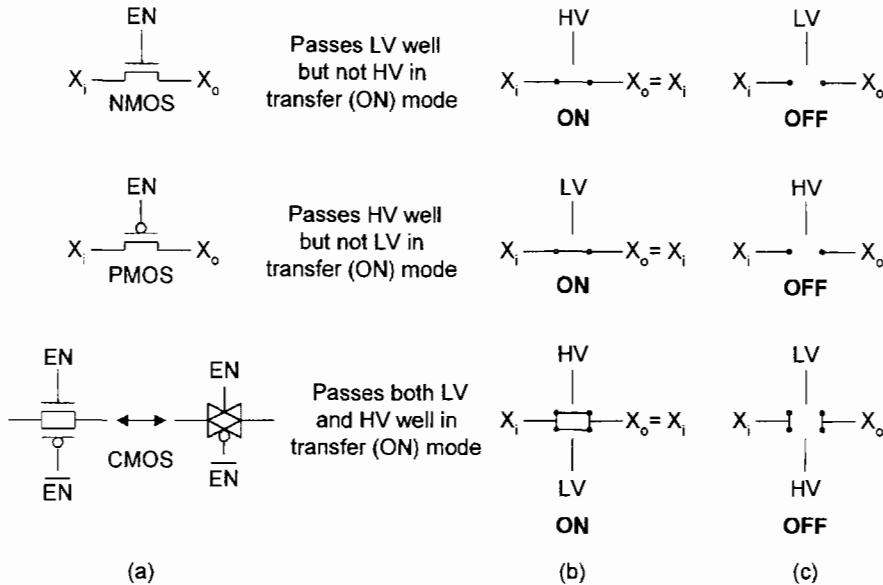
the physical truth table and logic interpretations given in Figs. 3.6b, 3.6c, and 3.6d. More detailed information on these logic families is presented in Appendix A.

### 3.5 TRANSMISSION GATES AND TRI-STATE DRIVERS

A MOS transistor switch that functions as a *passive* (non-amplifying) switching device and that does not invert a voltage signal is called a *transmission gate* or *pass transistor*. Logic circuits composed of transmission gates are called *steering logic* and are discussed in Section 6.9. Shown in Fig. 3.7 are the circuit symbols and equivalent circuits for the NMOS, PMOS, and CMOS transmission gates. Here, it can be seen that the ON condition in Fig. 3.7b permits an input signal  $X_i$  to be transferred to the output; hence,  $X_o = X_i$ . Conversely, the OFF condition disconnects the output from the input, allowing no signal to be transferred. Notice that the CMOS transmission gate requires complementary “enable” inputs,  $EN$  and  $\overline{EN}$ , to the NMOS and PMOS gates, respectively. This simply means that when one enable input is at high voltage (HV) the other must be at low voltage (LV) and vice versa.

As indicated earlier, an NMOS switch passes LV well but not HV, the reverse being true for a PMOS switch. Consequently, some distortion of the transmitted waveform is to be expected in NMOS and PMOS transmission gates operated in the transfer mode (ON condition). The CMOS switch, on the other hand, combines the best of both worlds, thereby minimizing waveform distortion.

An *active* (restoring) switching device that operates in either a transfer or disconnect mode is called a *tri-state driver* or *tri-state buffer*. If in the transfer mode it is designed to invert, it is called an inverting tri-state driver. These devices are called “tri-state” or “three-state” because they operate in one of three states — logic 0, logic 1, or high-impedance (Hi-Z) state. In the Hi-Z or disconnect state the tri-state driver is functionally “floating,” as if it were not there. Tri-state drivers are used to interface various IC devices to a common

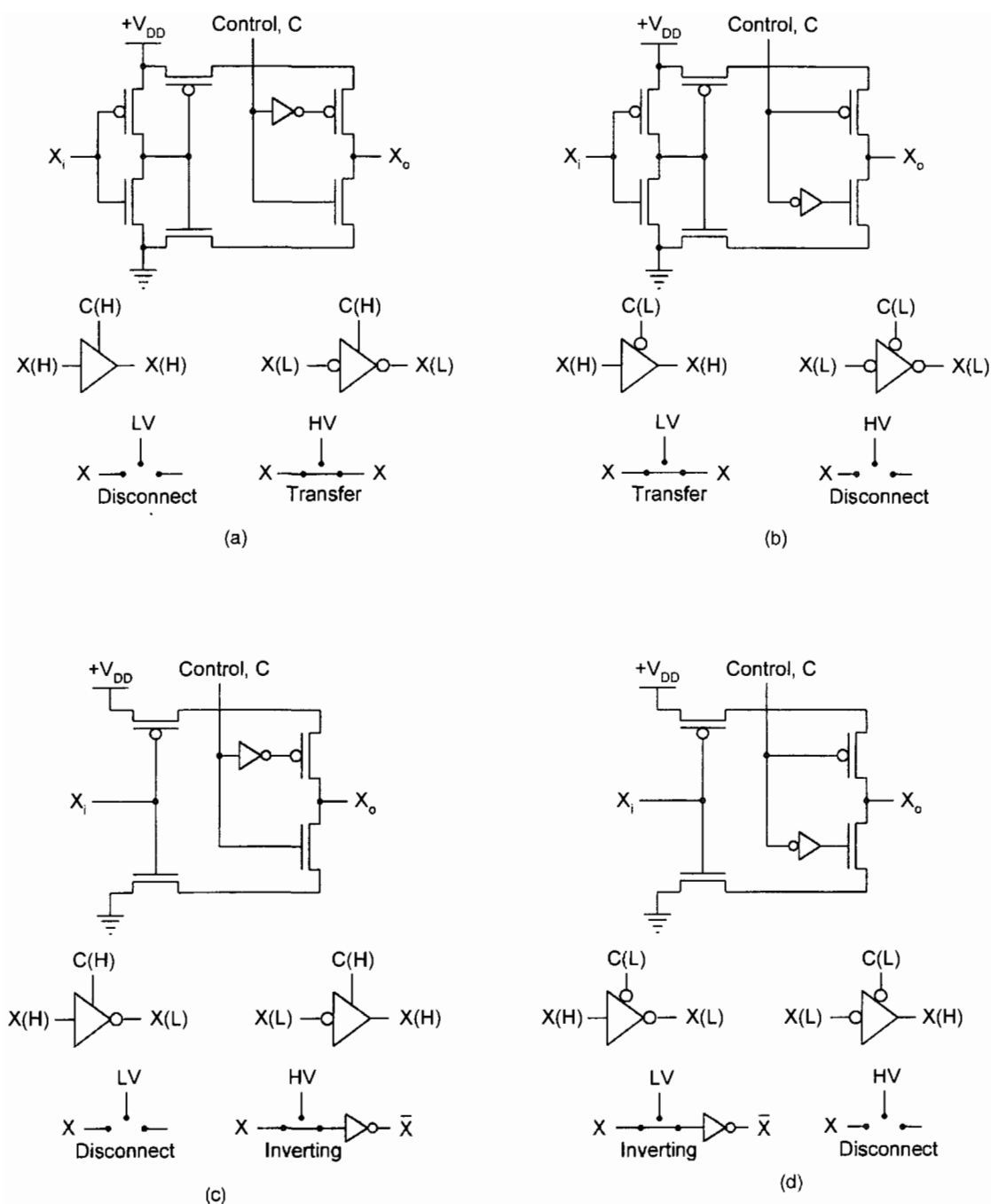
**FIGURE 3.7**

Transmission gate circuit symbols and their idealized equivalent circuits. (a) Simplified circuit symbols for NMOS, PMOS, and CMOS transmission gates. (b) ON (transfer) mode equivalent circuit. (c) OFF (disconnect) mode equivalent circuit.

data bus so that the devices will not interfere with each other. By this means, tri-state drivers permit multiple signal sources to share a single line if only one of the signals is active at any given time. These drivers also serve as a controlled enable on the output of some devices. Note that the term “tri-state” is a trademark of National Semiconductor Corporation. Thus, the use of the term “tri-state” in this text acknowledges NSC’s right of trademark. The terms tri-state and three-state are often used interchangeably.

Shown in Fig. 3.8 are four types of CMOS tri-state drivers constructed from the equivalent of two or three inverters. They differ in the activation levels of the control input,  $C$ , and the output,  $X_o$ , indicated by the logic circuit symbols. The choices are inverting or noninverting tri-state drivers with active high or active low control inputs, as provided in Figs. 3.8a–d. The buffering (driving) strength is the same for all tri-state drivers, however. This is so because during the transfer stage the outputs  $X_o$  are connected to the supply  $+V_{DD}$  depending on the character of the driver and the  $X_i$  voltage level. For example, in the case of the inverting tri-state driver of Fig. 3.8c, a control  $C = HV$  connects the output  $X_o$  to  $+V_{DD}$  if the input is  $X_i = LV$  or connects  $X_o$  to ground if  $X_i = HV$ . Thus, in the transfer mode, the transistors of a tri-state driver serve as transmission gates, thereby permitting an input signal to be enhanced (or refreshed); hence the meaning of the term driver. Of course, in the disconnect mode the tri-state driver produces a very large impedance (Hi-Z) between its input and output, virtually disconnecting the input from the output.

Note that the conjugate logic circuit symbols are provided for each tri-state driver shown in Fig. 3.8 and that these symbols are interchangeable — as they must be, since they are derived from the same physical device (the tri-state driver). The idea here parallels that of the inverter and its conjugate logic circuit symbols shown in Fig. 3.6. Symbol  $\bar{X}$  appearing



**FIGURE 3.8**

CMOS tri-state drivers, conjugate circuit symbols, and ideal equivalent circuits. (a) Noninverting tri-state driver with active high control,  $C$ . (b) Noninverting tri-state driver with active low control. (c) Inverting tri-state driver with active high control. (d) Inverting tri-state driver with active low control.

on the output of an inverting tri-state driver in the transfer mode indicates an inverted voltage signal. Thus, if  $X$  is at LV, then  $\bar{X}$  is at HV and vice versa.

Buffers, or *line drivers* as they are sometimes called, may be composed of a series of inverters or gates used as inverters, or they may be simply a tri-state driver operated in the transfer mode. Remember, it is the function of a line driver to boost and sharpen signals that might otherwise degrade below switching levels or be distorted. The mixed logic circuit symbols for buffers are given in Fig. 3.20a.

### 3.6 AND AND OR OPERATORS AND THEIR MIXED-LOGIC CIRCUIT SYMBOLOGY

There are just two binary logic operations that underlie all of logic design and Boolean algebra (after George Boole, 1815–1864, English mathematician). These are the AND and OR operations. The following are the operator symbols (or connectives) that are used for AND and OR:

$$(\cdot) \rightarrow \text{AND} \quad (+) \rightarrow \text{OR}$$

So, if one writes  $X \cdot Y$ ,  $XY$ , or  $(X)(Y)$ , it is read as  $X$  AND  $Y$ . Note that the AND operator  $(\cdot)$  is also called the *Boolean product* (or *intersection*) and may be represented by the alternative symbol  $(\wedge)$ . Thus,  $X \cdot Y = X \wedge Y$  is the intersection or Boolean product of  $X$  and  $Y$ . In contrast,  $X + Y$  is read as  $X$  OR  $Y$ . The operator  $(+)$  is often called the *Boolean sum* (or *union*) and may be represented by the alternative symbol  $(\vee)$ . Hence,  $X + Y = X \vee Y$  is the union or Boolean sum of  $X$  and  $Y$ .

By using the two Boolean operators, an endless variety of Boolean expressions can be represented. Simple examples are expressions such as

$$F = X + Y \cdot Z \quad \text{and} \quad G = X \cdot (Y + Z).$$

The first is read as  $F$  equals  $X$  OR  $(Y$  AND  $Z)$ . In this expression the Boolean quantity  $Y \cdot Z$  must first be evaluated before it is “ORed” with  $X$ . The second expression is read as  $G$  equals  $X$  AND  $(Y$  OR  $Z)$ . In this case the quantity  $(Y + Z)$  must first be evaluated before it can be “ANDed” with  $X$ . Thus, the hierarchy of Boolean operation is similar to that of Cartesian algebra for multiplication and addition.

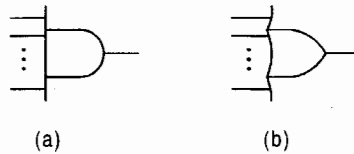
#### 3.6.1 Logic Circuit Symbolology for AND and OR

The meanings of the AND and OR operators (functions) are best understood in terms of their logic circuit symbols. Shown in Fig. 3.9 are the distinctively shaped logic circuit symbols commonly used to represent the AND and OR operators, which may have multiple inputs and a single output. The functional descriptions of these symbols are stated as follows:

*The output of a logic AND circuit symbol is active if, and only if, all inputs are active.*

*The output of a logic OR circuit symbol is active if one or more of the inputs are active.*



**FIGURE 3.9**

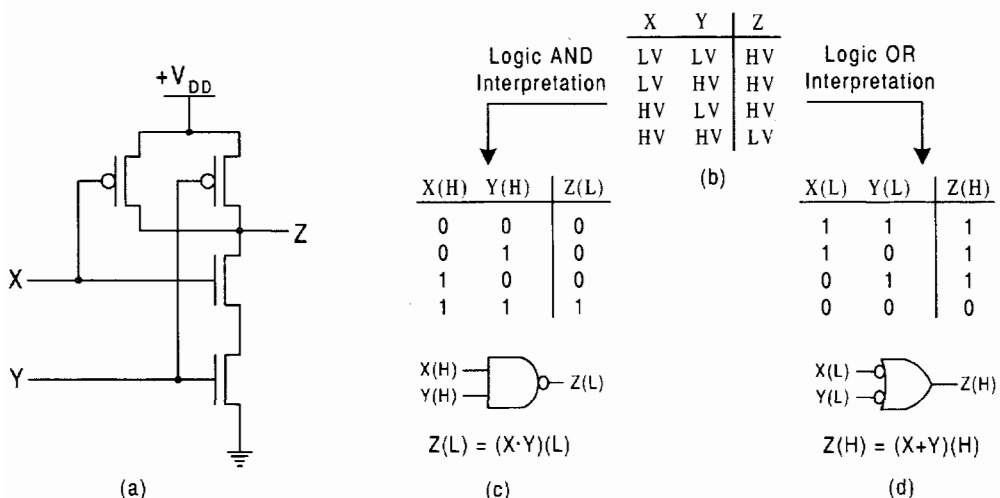
Distinctive logic circuit symbols for the binary operators. (a) AND symbol. (b) OR symbol.

The functional descriptions may be taken as the *definitions for AND and OR*. Remember that the term active implies logic 1.

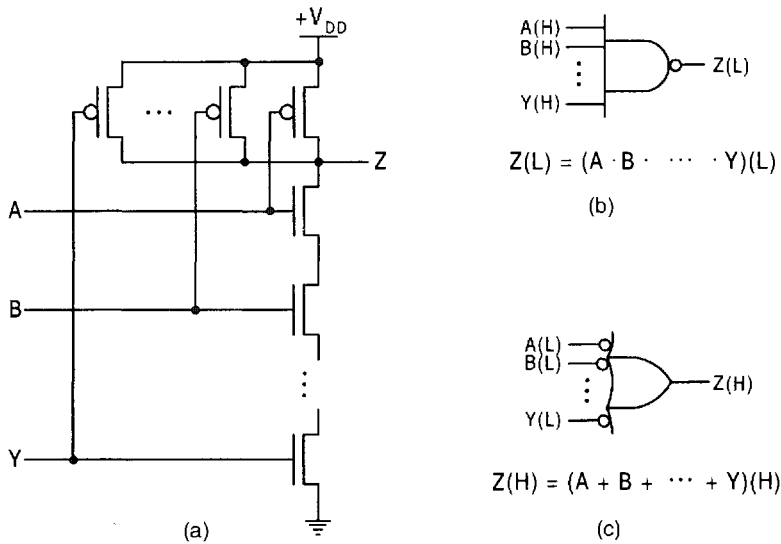
The distinctively shaped symbols in Fig. 3.9 represent the functional descriptions for AND and OR and provide the logical interpretation for a variety of physical devices called *gates*. That is, each gate must satisfy the logical AND or logical OR functional description. By definition: *A gate is an interconnection of electronic switches and other circuit elements arranged in such a way as to produce the electrical equivalent of a logic operation.* The inputs and outputs of a gate are measured in terms of voltages (LV or HV), whereas the inputs and outputs of a logic symbol, as in Fig. 3.9, are expressed in terms of logic 1 or logic 0 together with the appropriate activation level indicators, (H) and (L).

### 3.6.2 NAND Gate Realization of Logic AND and OR

The physical device shown in Fig. 3.10a is a two-input NAND gate. NAND is short for NOT-AND. Because this version of NAND gate complies with the generalized CMOS gate configuration in Fig. 3.5, it is called a CMOS NAND gate. The physical truth table for this

**FIGURE 3.10**

The two-input NAND gate, its I/O behavior, and its two logic interpretations. (a) CMOS transistor circuit. (b) Physical truth table. (c) Logic AND interpretation and circuit symbol. (d) Logic OR interpretation and circuit symbol.

**FIGURE 3.11**

Multiple input NAND gates and logic circuit symbols. (a) CMOS logic circuit. (b) AND interpretation. (c) OR interpretation.

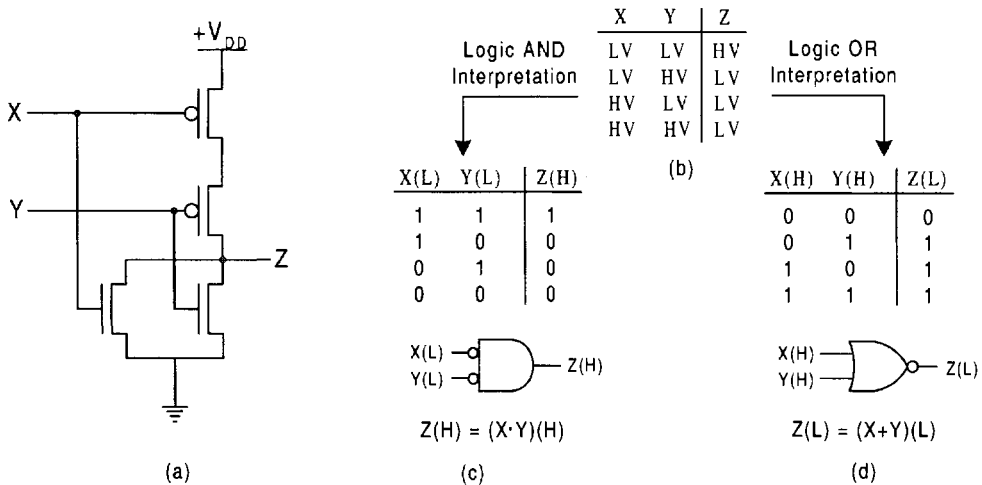
or any NAND gate is given in Fig. 3.10b. It is easily generated by applying the ON and OFF equivalent circuits for NMOS and PMOS given in Fig. 3.4.

The two logical AND and OR interpretations of the NAND gate and their mixed logic circuit symbols are given in Figs. 3.10c and 3.10d. They, too, apply to NAND gates belonging to logic families other than CMOS, as explained further in Appendix A. Notice that by applying Eqs. (3.1) the truth tables for the AND and OR interpretations satisfy the definitions for AND and OR given earlier in connection with Fig. 3.9—no other combination of activation level symbols applied to inputs  $X$  and  $Y$  satisfies these definitions. But both logic circuit symbols represent (in the logic domain) the physical NAND gate, since both are derived from it. Thus, one symbol (c) performs the AND operation with active low output, while the other symbol (d) performs the OR operation with active low inputs. The symbols are interchangeable in a logic circuit and, for that reason, are called *conjugate NAND gate symbols* even though, strictly speaking, they are only logic circuit symbols.

Multiple input CMOS NAND gates result by adding more PMOS in parallel and an equal number of NMOS in series, as shown in Fig. 3.11. The logic circuit symbols and output expressions shown in Figs. 3.11b and 3.11c result. The number of inputs is usually limited to eight or fewer, mainly because of an increase in resistance of the series N-MOSFETs, each of which has a small ON channel resistance associated with it. Therefore, too many inputs causes an increase in gate propagation delay and a degradation of the signal. The number of inputs that a gate can have is called the *fan-in*. For example, a four-input NAND gate would have a fan-in of 4.

### 3.6.3 NOR Gate Realization of Logic AND and OR

The transistor circuit for the two-input CMOS NOR gate is shown in Fig. 3.12a. NOR is short for NOT-OR. The physical truth table and the AND and OR logical interpretations

**FIGURE 3.12**

The two-input NOR gate, its I/O behavior and its two logic interpretations. (a) CMOS transistor circuit. (b) Physical truth table. (c) Logic AND interpretation and circuit symbol. (d) Logic OR interpretation and circuit symbol.

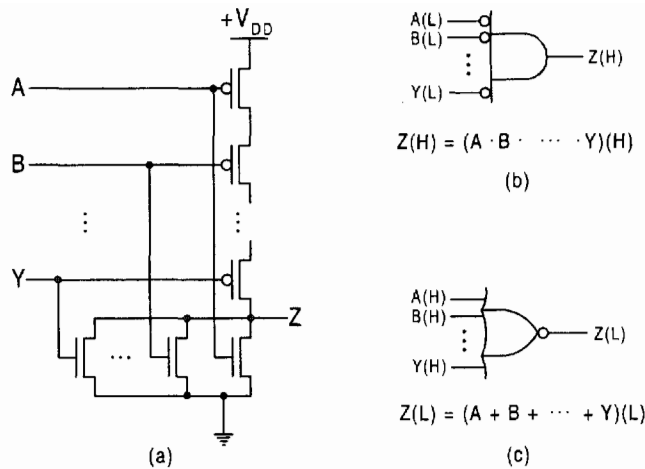
that derive from it are given in Figs. 3.12b, 3.12c, and 3.12d, respectively, and these also apply to NOR gates belonging to other logic families, as discussed in Appendix A. As before, the physical truth table is obtained by applying the equivalent circuits given in Fig. 3.4 to the transistors in Fig. 3.12a. The AND and OR logic interpretations in parts (c) and (d) derive from the application of Eqs. (3.1) to the physical truth table and are observed to agree with the definitions of AND and OR given in connection with Fig. 3.9 — again, no other combination of activation level symbols applied to inputs  $X$  and  $Y$  satisfies these definitions. Thus, there results two logic circuit symbols, one performing the AND operation with active low inputs (c) and the other performing the OR operation with active low output. Since the logic symbols are interchangeable, they are called *conjugate NOR gate symbols*.

Multiple input NOR gates are produced by adding more PMOS in series and an equal number of NMOS in parallel, as indicated in Fig. 3.13a. The logic symbols for multiple input NOR gates are shown in Figs. 3.13b and 3.13c. As in the case of multiple NAND gates, there exists a practical limit to the number of NOR gate inputs (fan-in) because of the channel resistance effect. Thus, too many inputs to a NOR gate will increase the gate propagation delay and degrade the signal.

When fan-in restrictions become a problem, a gate *tree* structure (e.g., a NOR gate tree) can be used. A gate tree is a combination of like gates that form a multilevel array (see Fig. 4.49). Thus, a tree composed of OR gates and an inverter can replace a multiple-input NOR gate when the number of inputs exceeds the fan-in limit for that gate.

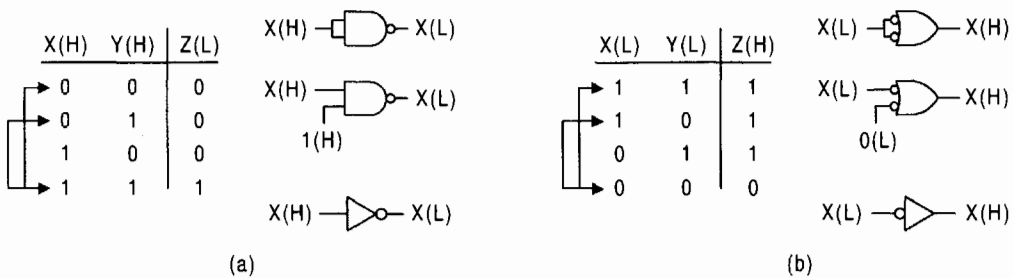
### 3.6.4 NAND and NOR Gate Realization of Logic Level Conversion

Inherent in any NAND or NOR gate is the ability to function as an inverter. Thus, under the proper conditions, the NAND or NOR gate can perform the equivalent of logic level conversion as in Fig. 3.6. Shown in Figs. 3.14 and 3.15 are the various input connections



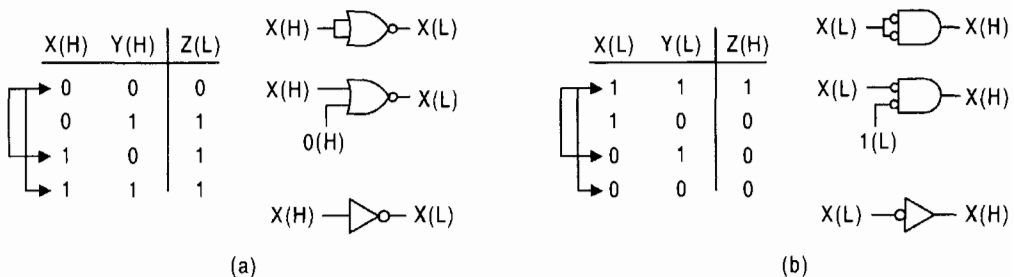
**FIGURE 3.13**

Multiple input NOR gates and logic circuit symbols. (a) CMOS logic circuit. (b) AND interpretation. (c) OR interpretation.



**FIGURE 3.14**

Nand gate realization of logic level conversion and equivalent symbology. (a) (H) → (L) conversion. (b) (L) → (H) conversion.



**FIGURE 3.15**

NOR gate realization of logic level conversion and equivalent symbology. (a) (H) → (L) conversion. (b) (L) → (H) conversion.

that permit this to happen. These input connections result directly from the logic truth tables in Figs. 3.10 and 3.12. The arrows on the left side of each truth table are given to draw attention to those portions of the table that dictate how the connections to the NAND and NOR gates must be made. For example, the extreme upper and lower arrows in Fig. 3.14a indicate that the two inputs to the NAND gate can be connected for the  $X(H) \rightarrow Z(L) = X(L)$  conversion. The lower two arrows indicate that the same conversion can be achieved by setting  $Y(H) = 1(H)$ . It is not likely that a NAND or NOR gate would be used as a replacement for an inverter if the latter were available, but the substitution is permissible if the need is there. Obviously, the NAND or NOR gate inverter is more costly (spacewise) and is slower than the inverter in Fig. 3.6.

### 3.6.5 The AND and OR Gates and Their Realization of Logic AND and OR

NAND and NOR CMOS gates are natural electrical realizations of the AND and OR logic operators, but the AND and OR CMOS gates are not. This can be understood if one recalls that a transistor switch is, by its nature, an inverter. Thus, it might be expected that proper CMOS realizations of NOT-AND and NOT-OR would be simpler (by transistor count) than the equivalent CMOS realizations of AND and OR, and this is the case.

Shown in Fig. 3.16a is the transistor circuit for the CMOS version of the two-input AND gate. It is seen to be composed of the NAND gate followed by an inverter, hence NAND-NOT or NOT-AND-NOT. By application of Eqs. (3.1), the physical truth table for the AND gate, given in Fig. 3.16b, yields the AND and OR interpretations shown in Figs. 3.16c, and 3.16d. From these interpretations there results the two *conjugate AND gate symbols*, one performing the AND operation with active high inputs and output (c) and the other performing the OR operation with active low inputs and output as indicated by the active

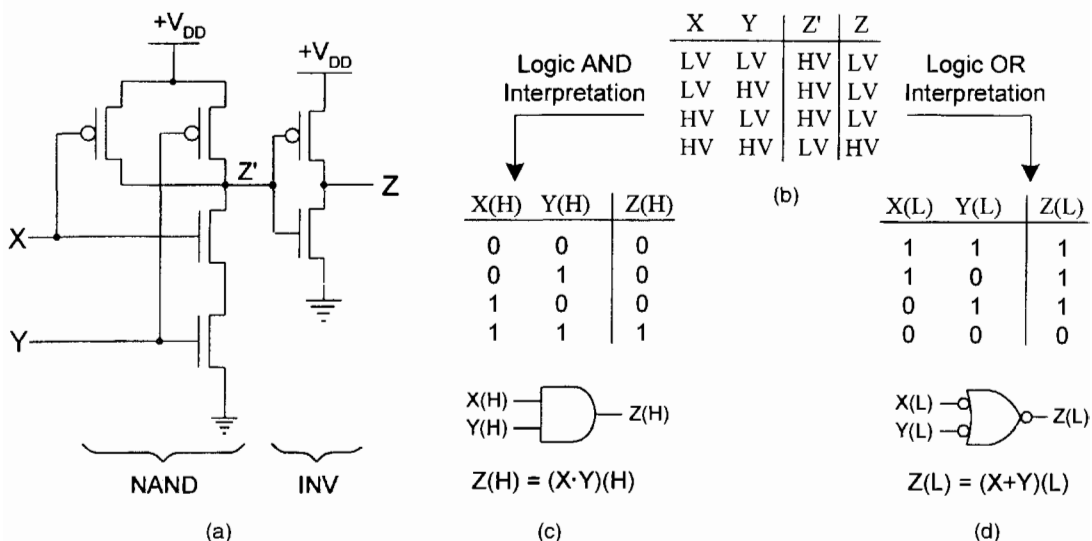
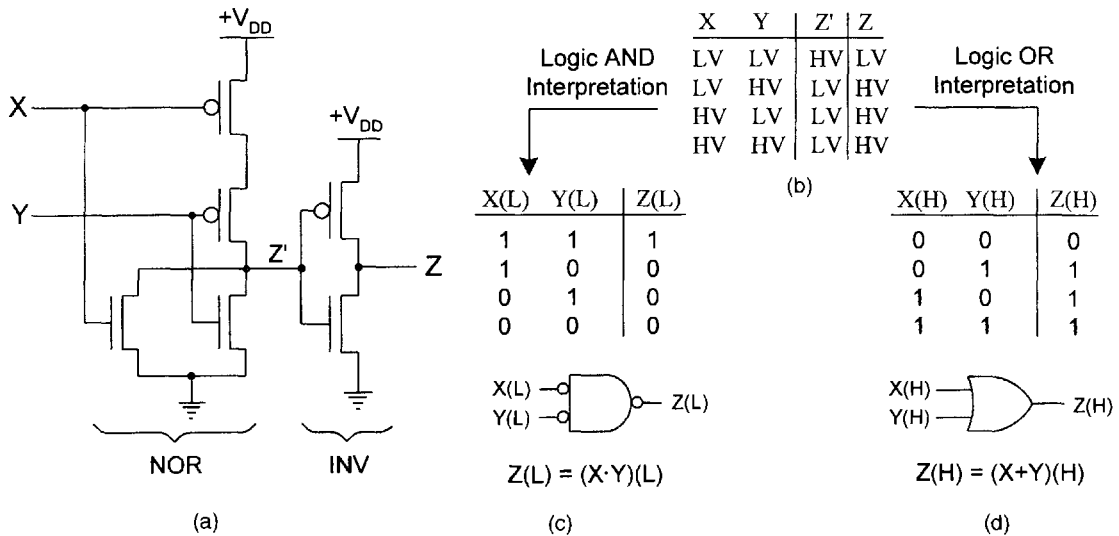


FIGURE 3.16

The two-input AND gate, its I/O behavior, and its two logic interpretations. (a) CMOS transistor circuit. (b) Physical truth table. (c) Logic AND interpretation and circuit symbol. (d) Logic OR interpretation and circuit symbol.



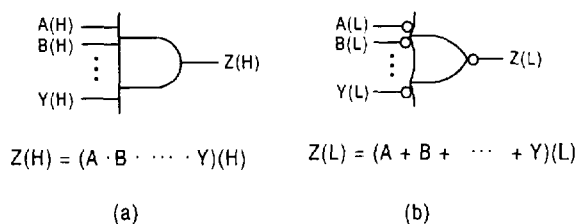
**FIGURE 3.17**

The two-input OR gate, its I/O behavior, and its two logic interpretations. (a) CMOS transistor circuit. (b) Physical truth table. (c) Logic AND interpretation and circuit symbol. (d) Logic OR interpretation and circuit symbol.

low indicator bubbles. The logic interpretations and mixed logic circuit symbols also apply to the AND gate of any logic family.

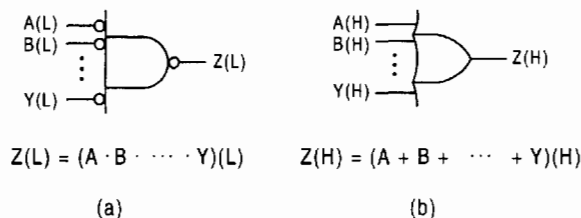
The CMOS version of the two-input OR gate, its physical truth table, and its two logic interpretations are shown in Fig. 3.17. Again, Eqs. (3.1) and the functional descriptions associated with Fig. 3.9 have been applied to the physical truth table, Fig. 3.17b, to yield the AND and OR interpretations and the mixed logic circuit symbols presented in Figs. 3.17c and 3.17d. The two logic circuit symbols are interchangeable and hence are *conjugate OR gate symbols*. One symbol performs the AND operation and has active low inputs and output, while the other performs the OR operation and has active high inputs and output. As before, the truth table and logic AND and OR interpretations apply also to an OR gate of any logic family.

Multiple input CMOS AND and OR gates are possible by combining the transistor circuit in either Fig. 3.11 or Fig. 3.13 with an inverter. The conjugate gate symbols for AND and OR that result are shown in Figs. 3.18 and 3.19. The same limitations on numbers of inputs that apply to CMOS NAND and NOR gates also apply to CMOS AND and OR gates.



**FIGURE 3.18**

Logic symbols for multiple input AND gates. (a) AND interpretation. (b) OR interpretation.

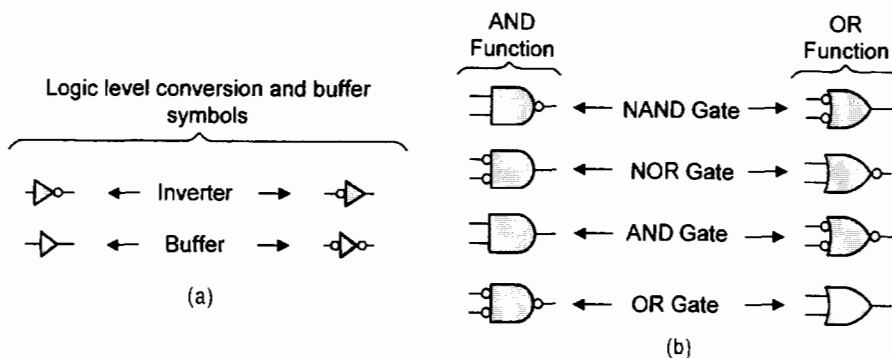
**FIGURE 3.19**

Logic circuit symbols for multiple input OR gates. (a) AND interpretation. (b) OR interpretation.

AND and OR gates can be configured by “flipping” end-for-end the NAND and NOR gates shown in Figs. 3.10a and 3.12a, respectively, such that the NMOS occupy the HV end while the PMOS reside at the LV end. However, to do this requires that the NMOS pass HV, which they do not do well, and that the PMOS pass LV, which they do not do well. Thus, although such flipped configurations logically satisfy the AND and OR interpretations for the respective gates, their output signals would be somewhat distorted. For minimum output signal distortion the PMOS and NMOS portions for any gate should be configured as in Fig. 3.5.

### 3.6.6 Summary of Logic Circuit Symbols for the AND and OR Functions and Logic Level Conversion

For reference purposes, a summary is now provided for the mixed logic symbology that has been covered so far. Shown in Fig. 3.20 are the conjugate mixed logic circuit symbols together with the physical gate names they represent. The conjugate mixed logic circuit symbols for the inverter and buffer are given in Fig. 3.20a. Notice that the conjugate pairs of logic circuit symbols in Fig. 3.20b are split into two groups, one group performing the AND function and the other performing the OR function. The buffer, not previously discussed, is included here for completeness. It functions as an amplifier to boost the signal to meet

**FIGURE 3.20**

Summary of conjugate mixed logic circuit symbols and the gates they represent. (a) Logic level conversion and buffer symbols. (b) AND and OR function symbols.

fan-in requirements. For reference purposes, the ANSI/IEEE Standard logic symbols for gates are provided in Appendix C.1.

### 3.7 LOGIC LEVEL INCOMPATIBILITY: COMPLEMENTATION

The eight conjugate gate symbols in Fig. 3.20b perform one of two logic operations, AND or OR, regardless of the presence or absence of the active low indicator bubbles that serve to associate the symbols to the physical gates from which they are derived. However, the presence or absence of an active low indicator bubble on the input to a given circuit symbol indicates the activation level of the input, (L) or (H), that is “required” by that gate. Thus, the presence of active low indicator bubbles on the inputs to a logic symbol requires that all inputs arrive from negative logic sources while the absence of bubbles requires that the inputs arrive from positive logic sources. When these requirements are met the inputs are said to have *logic compatibility* with the logic symbol.

But suppose an input signal arrives at the input to a logic symbol with an activation level that is of opposite polarity to that required by the logic circuit symbol. When this happens a condition of *logic incompatibility* exists, and this requires that the signal name in the output be complemented.

The operation of *complementation* is defined by the following important relations applied to a logic function  $\alpha$ :

$$\alpha(L) = \bar{\alpha}(H) \quad \text{and} \quad \alpha(H) = \bar{\alpha}(L) \quad (3.2)$$

such that

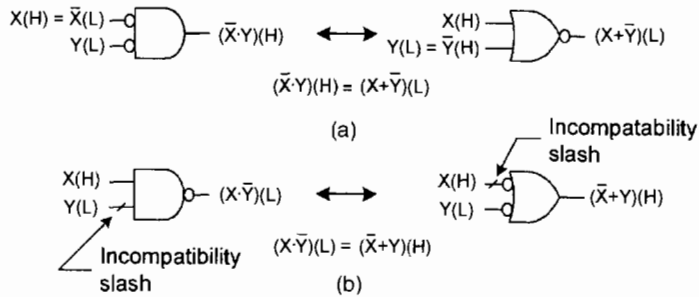
$$\begin{aligned} (\alpha \cdot \bar{\alpha})(H) &= 0(H) \\ (\alpha \cdot \bar{\alpha})(L) &= 0(L) \\ (\alpha + \bar{\alpha})(H) &= 1(H) \\ (\alpha + \bar{\alpha})(L) &= 1(L). \end{aligned} \quad (3.3)$$

The overbar is read as “the complement of.” Thus, in the logic domain a logic function  $\alpha$  ANDed with its complement  $\bar{\alpha}$  is logic 0, or the function ORed with its complement is logic 1.

In Fig. 3.21 are four typical examples of input logic level incompatibility each requiring the complementation of the incompatible input name in the output expression. Note that this is indicated in two ways. In Fig. 3.21a, Eqs. (3.2) are applied directly to satisfy the logic level compatibility requirements of the logic symbol. In Fig. 3.21b, an incompatibility slash “/” is placed on the input line containing the logic incompatibility as a visual reminder that a logic level incompatibility exists and that the input name must be complemented in the output expression.

The pairs of logic circuit symbols in Figs. 3.21a and 3.21b are conjugate symbol forms as in Fig. 3.20. Because these conjugate circuit symbols are interchangeable, their output expressions are equal and are representative of a set of such equations called the *DeMorgan relations*. This subject will be considered further in Section 3.10.



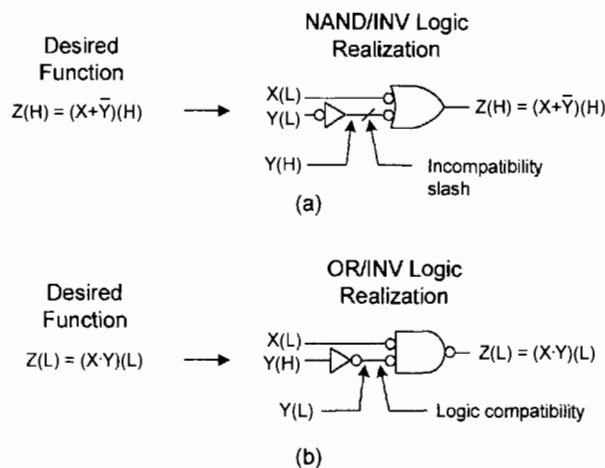
**FIGURE 3.21**

Examples of logic level incompatibility. (a) Applications of Eqs. (3.2). (b) Use of the incompatibility slash (/) as an alternative.

Earlier it was stated that the only logic function of the inverter is to perform logic level conversion,  $(H) \rightarrow (L)$  or  $(L) \rightarrow (H)$ , and this is true. But to what purpose? The answer is simply stated:

*The logical function of the inverter is to create or remove an input logic level incompatibility depending on the output function requirements of the logic symbol to which the input is attached.*

Consider two examples of logic level conversion in Figs. 3.22a and 3.22b. Here, NAND and OR logic realizations of logic functions require the use of inverters to create and remove logic level incompatibilities, respectively.

**FIGURE 3.22**

Examples of logic level conversion. (a) Creation of a logic incompatibility with active low inputs. (b) Removal of a logic incompatibility with inputs  $X(L)$  and  $Y(H)$ .

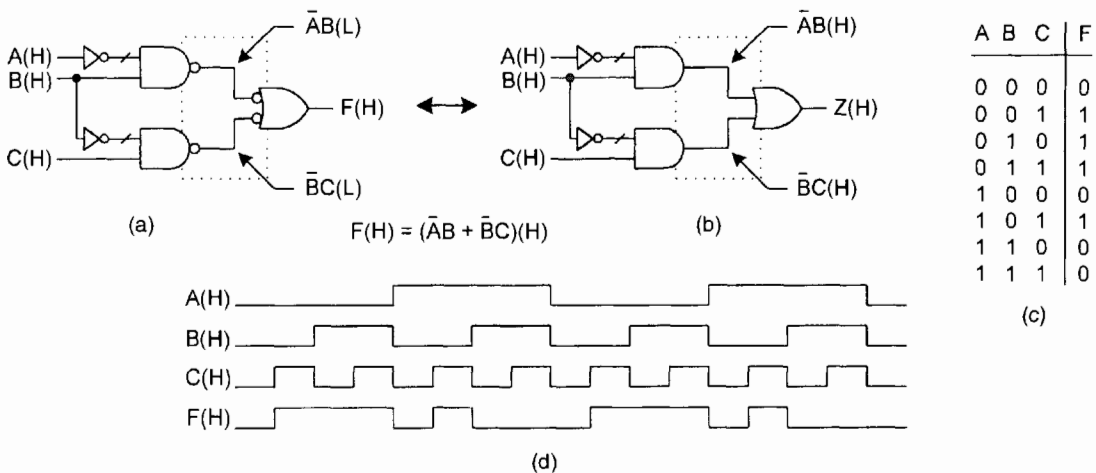
### 3.8 READING AND CONSTRUCTION OF MIXED-LOGIC CIRCUITS

The very simple procedures that are necessary to construct and read mixed-logic circuits are now demonstrated by more complex examples. Consider the function  $F(H)$ :

$$F(H) = [\bar{A} \cdot B + \bar{B} \cdot C](H)$$

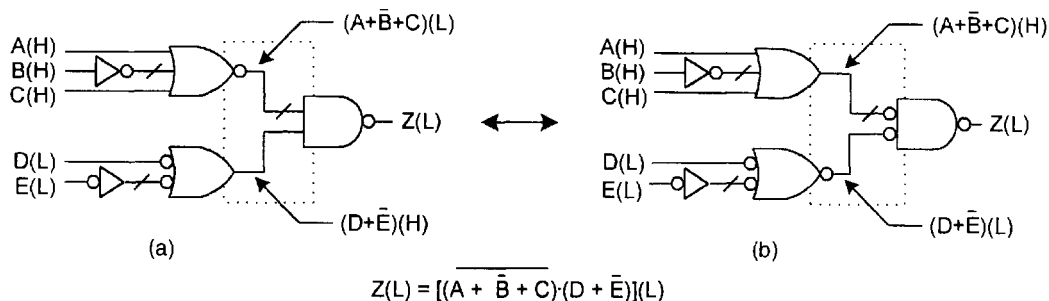
Notice that this function is formed by ORing together two ANDed terms and is read as  $F$  equals  $A$  “bar” AND  $B$  ORed with  $B$  “bar” AND  $C$ , all active high. The logic circuit for this function is shown in Fig. 3.23, where it is assumed that the inputs arrive active high ( $H$ ), that is, from positive logic sources. Two logic realizations are shown, one NAND/INV logic and the other AND/OR/INV logic, both yielding the function  $F(H)$ . Thus, by complementing between the AND and OR stages (area enclosed by dotted lines), the physical realization is altered but without changing the original function—logic level compatibility has been preserved. Observe that an incompatibility slash (“/”) is placed on a given symbol input as a reminder that an input logic incompatibility requires complementation of the input name in the output expression. In Figs. 3.23c and 3.23d are two additional means of representing the function  $F$ —namely, the truth table and logic waveforms. Here, a binary input sequence is assumed and no account is taken of the path delays through the gates and inverters.

A second more complex example is shown in Fig. 3.24, where a function  $Z(L)$  has been implemented by using NAND/NOR/INV logic in (a) and by using AND/OR/INV logic in (b).



**FIGURE 3.23**

Examples of the reading, construction, and waveform analysis of a logic circuit. (a) NAND/INV and (b) AND/OR/INV logic realizations of the function  $F(H)$  with active high inputs. (c) Truth table for the function  $F$ . (d) Logic waveforms for the function  $F$  assuming a binary input pattern.

**FIGURE 3.24**

Logic realizations of the function  $Z(L)$  with inputs  $A(H)$ ,  $B(H)$ ,  $C(H)$ ,  $D(L)$ , and  $E(L)$ . (a) NAND/NOR/INV logic. (b) AND/OR/INV.

logic in (b). In this example the ORing input stages receive inputs that are assumed to arrive as  $A(H)$ ,  $B(H)$ ,  $C(H)$ ,  $D(L)$ , and  $E(L)$ . Here again, by complementing between the AND and OR stages (dotted boxes), the physical realization is changed without altering the original function. Notice that incompatibilities exist on inputs and between ORing and ANDing stages requiring (in each case) complementation of the signal name in the output expression as indicated by the “/” symbol.

**Reading vs Construction of Logic Circuits** Implied by Figs. 3.23 and 3.24 is the procedure to be followed when reading or constructing a logic circuit:

*The reading of a logic circuit always begins with the inputs and ends at the output, hence “input-to-output.”*

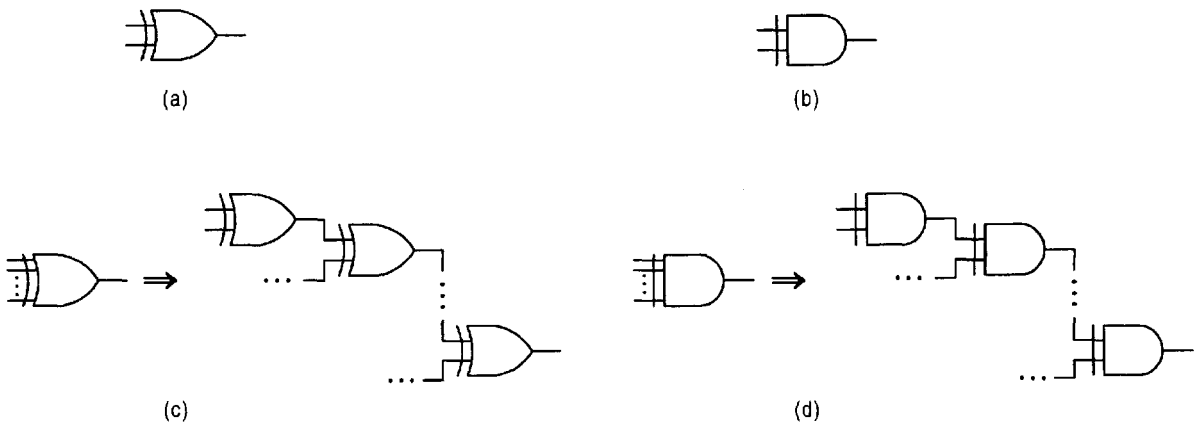
*Construction of a logic circuit begins at the output stage and continues to the inputs, hence “top down.”*

One must not begin construction of a mixed-logic circuit until the activation levels of the inputs and outputs are known and the output and input stage operators have been identified. If a circuit has been presented in positive logic form (no mixed logic symbology), it is advisable for the reader to convert the circuit to mixed logic form before reading it. This will speed up the reading process and minimize the probability for error.

### 3.9 XOR AND EQV OPERATORS AND THEIR MIXED-LOGIC CIRCUIT SYMBOLOGY

Certain functions consisting of AND and OR operations occur so often in digital logic design that special names and operator symbols have been assigned to them. By far the most common of these are the *exclusive or* (XOR) and *equivalence* (EQV) functions represented by the following operator symbols:

$\oplus \rightarrow \text{XOR}$ , meaning “one or the other but not both equivalent.”  
 $\odot \rightarrow \text{EQV}$ , meaning “both the same (equivalent).”

**FIGURE 3.25**

Distinctive logic circuit symbols for XOR and EQV. (a) The XOR function circuit symbol. (b) The EQV function circuit symbol. (c) and (d) illustrate the meaning of multiple input symbols.

Thus, if one writes  $X \oplus Y$ , it is read as  $X$  XOR  $Y$ ;  $X \odot Y$  is read as  $X$  EQV  $Y$ . The EQV operator is also known as XNOR (for EXCLUSIVE NOR), a name that will *not* be used in this text.

Like the AND and OR functions, the XOR and EQV functions are best understood in terms of the logic circuit symbols representing them. Figures 3.25a and 3.25b give the commonly used XOR and EQV circuit symbols for which the following functional descriptions apply:

*The output of a logic XOR circuit symbol is active if one or the other of two inputs is active but not both active or inactive — that is, if the inputs are not logically equivalent.*

*The output of a logic EQV circuit symbol is active if, and only if, both inputs are active or both inputs are inactive — that is, if both inputs are logically equivalent.*

A circuit symbol for either XOR or EQV consists of *two and only two* inputs. Multiple input XOR or EQV circuit symbols are understood to have the meaning indicated in Figs. 3.25c and 3.25d and are known as *tree forms*.

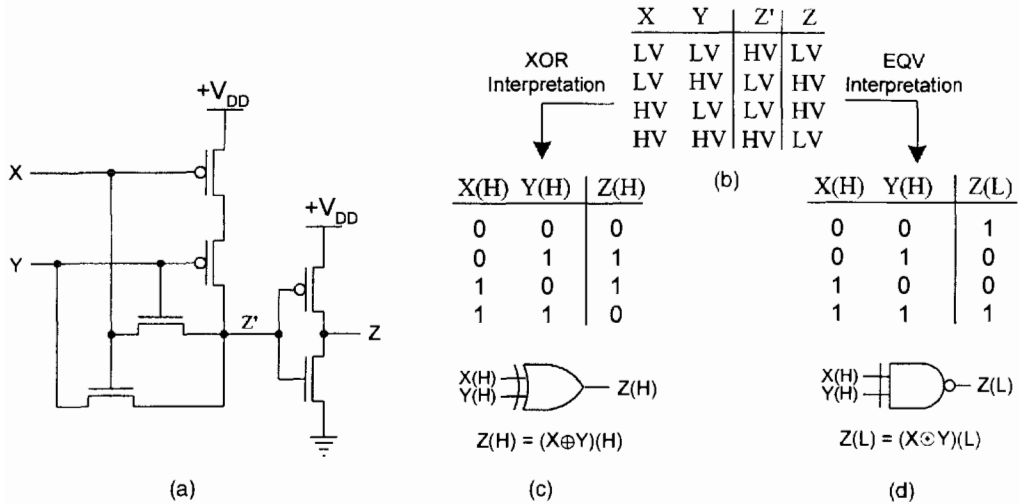
The *defining relations* for XOR and EQV are obtained from the functional descriptions just given. In Boolean sum-of-products and Boolean product-of-sums form these defining relations are

$$A \oplus B \equiv \bar{A} \cdot B + A \cdot \bar{B} = (\bar{A} + \bar{B}) \cdot (A + B) \quad (3.4)$$

and

$$A \odot B \equiv \bar{A} \cdot \bar{B} + A \cdot B = (\bar{A} + B) \cdot (A + \bar{B}). \quad (3.5)$$

In words, the XOR function in Eq. (3.4) is active if only one of the two variables in its defining relation is active but not both active or both inactive. Thus,  $A \oplus B = 1$  if only one

**FIGURE 3.26**

The XOR gate, its I/O behavior, and its two logic interpretations. (a) A CMOS transistor circuit. (b) Physical truth table. (c) Logic XOR interpretation. (d) Logic EQV interpretation.

of the two variables takes logic 1 at any given time. Conversely, the EQV function in Eq. (3.5) is active only if both variables in its defining relation are active or both are inactive. In this case  $A \odot B = 1$  only if both variables are logically equivalent (both logic 1 or both logic 0).

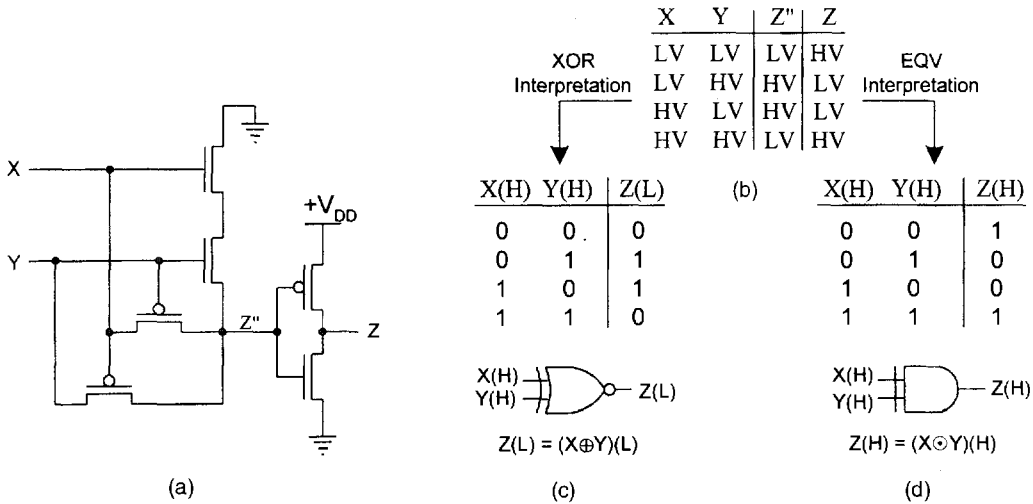
### 3.9.1 The XOR and EQV Functions of the XOR Gate

Shown in Fig. 3.26a is one of several CMOS versions of the XOR gate. This version makes rather clever use of NMOS and PMOS totaling six transistors. Notice that the output stage is an inverter that acts not only to invert the signal, but also as a *gain element* to boost the signal. This is important since the remainder of the circuit is composed of NMOS and PMOS *transmission gates* (Fig. 3.7) that lack the ability to amplify. In Fig. 3.26b is the physical truth table for the XOR gate. Observe that all but the  $X, Y = LV, LV$  input conditions produce a  $Z'$  voltage that is the voltage level from one or both inputs. This is characteristic of logic gates composed of pass transistors.

Presented in Figs. 3.26c and 3.26d are the XOR and EQV logic interpretations of the XOR gate together with their distinctively shaped circuit symbols. The logic truth tables for these interpretations derive from the defining relations given by Eqs. (3.4) and (3.5), respectively, and from Eqs. (3.1). Observe that the XOR symbol with active high inputs and output is interchangeable with an EQV symbol with active low output. Thus, it follows that  $(X \oplus Y)(H) = (X \odot Y)(L)$ .

### 3.9.2 The XOR and EQV Functions of the EQV Gate

A version of the CMOS EQV gate is shown in Fig. 3.27a. It is obtained from the XOR version in Fig. 3.26a by “complementing” the MOS transistors in the  $Z'$  circuit to obtain the  $Z''$  circuit in Fig. 3.27a. Notice that all input conditions except  $X, Y = HV, HV$  produce a  $Z''$  output directly from one or both of the inputs. Also note that  $Z''$  is an XOR output,

**FIGURE 3.27**

The EQV gate, its I/O behavior, and its two logic interpretations. (a) A CMOS transistor circuit. (b) Physical truth table. (c) Logic XOR interpretation. (d) Logic EQV interpretation.

whereas  $Z'$  is an EQV output, and that they are the inverse of each other. In each case an inverter is added to invert the signal as well as to buffer the output  $Z$ .

The physical truth table for the EQV gate and its two logic interpretations are given in parts (b), (c), and (d) of Fig. 3.27. The same procedure used for the XOR gate is used here to obtain the logic truth tables and circuit symbols for the EQV gate. In this case, the XOR symbol with active low output is interchangeable with the EQV symbol with active high inputs and output resulting in the relation  $(X \oplus Y)(L) = (X \odot Y)(H)$ .

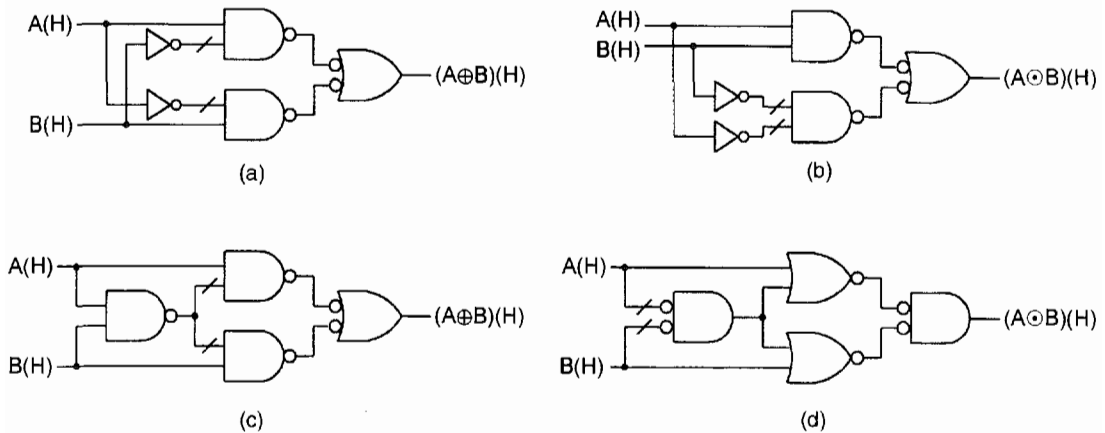
### 3.9.3 Multiple Gate Realizations of the XOR and EQV Functions

The CMOS transistor circuits for the XOR and EQV functions given in Figs. 3.26 and 3.27 represent the most efficient use of MOSFETs for such purposes. However, there are occasions when such MOS implementations of these functions are not possible. One example is the use of programmable logic arrays (PLAs), as discussed in Section 7.3, to implement arithmetic-type circuits discussed in Chapter 8. PLAs are devices that must use two-level gate forms to implement XOR or EQV functions—XOR or EQV gates are not commonly part of the PLA architecture. Shown in Fig. 3.28 are four multiple-gate realizations of the XOR and EQV functions. The circuits in Figs. 3.28a and 3.28b have been derived from the defining relations for XOR and EQV given by Eqs. (3.4) and (3.5), respectively and are suitable for two-level circuit design. The three-level circuits in Figs. 3.28c and 3.28d are not suitable for two-level circuit design. These three-level circuits result from derivatives of the defining relations:

$$A \cdot (\overline{AB}) + B(\overline{AB}) = A\bar{B} + \bar{A}B \quad \text{XOR form} \quad (3.4a)$$

$$(A + \bar{A}\bar{B})(B + \bar{A}\bar{B}) = (A + \bar{B})(\bar{A} + B) \quad \text{EQV form} \quad (3.4b)$$

The applications of CMOS AND-OR-invert and OR-AND-invert gates to the implementation of XOR and EQV functions are given later in Subsection 7.7.1. Such CMOS

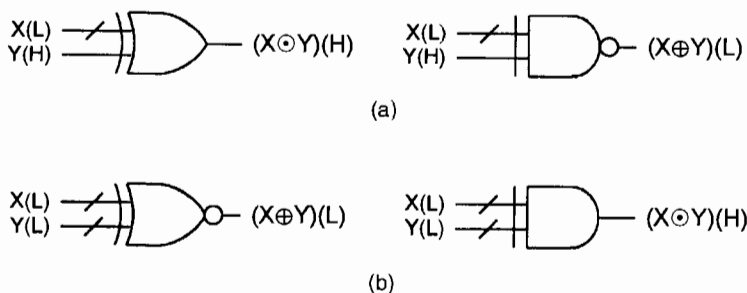
**FIGURE 3.28**

Multiple gate realizations of the XOR and EQV functions. (a, b) Two-level NAND implementations of the XOR and EQV functions. (c, d) Three-level NAND and NOR implementations of the XOR and EQV functions.

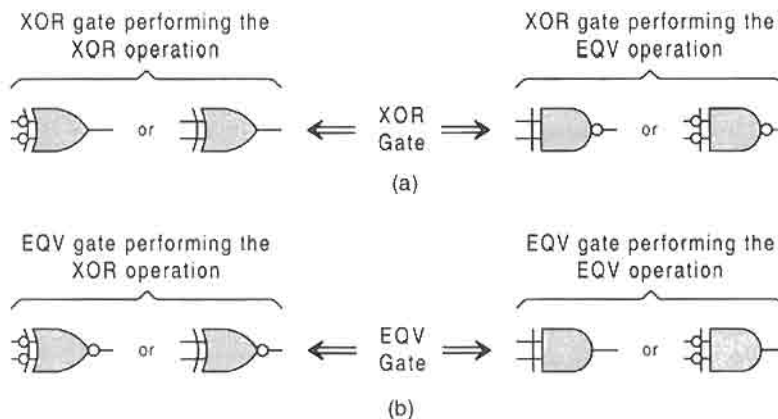
realizations of these functions are shown to be closer to a single level of path delay rather than two levels, as is true for those in Figs. 3.26a and 3.27a.

### 3.9.4 The Effect of Active Low Inputs to the XOR and EQV Circuit Symbols

An interesting property of the XOR and EQV logic symbols is the fact that when the two inputs are of opposite polarity the output function is complemented, but when the inputs are of the same polarity (both active high or both active low) the function remains unchanged. This is illustrated by the four examples in Fig. 3.29. Thus, a single incompatibility complements the function (changing an XOR output function to an EQV output function or vice versa), whereas a double incompatibility or no incompatibility retains the output function. These results may be proven by altering the appropriate logic truth table in Figs. 3.26 and 3.27 to agree with the input activation levels indicated for each logic circuit symbol.

**FIGURE 3.29**

Effect of active low inputs to XOR and EQV logic circuit symbols. (a) Single logic incompatibilities. (b) Dual logic incompatibilities.

**FIGURE 3.30**

Summary of conjugate mixed logic circuit symbols for the XOR and EQV gates. (a) XOR gate symbols. (b) EQV gate symbols.

### 3.9.5 Summary of Conjugate Logic Circuit Symbols for XOR and EQV Gates

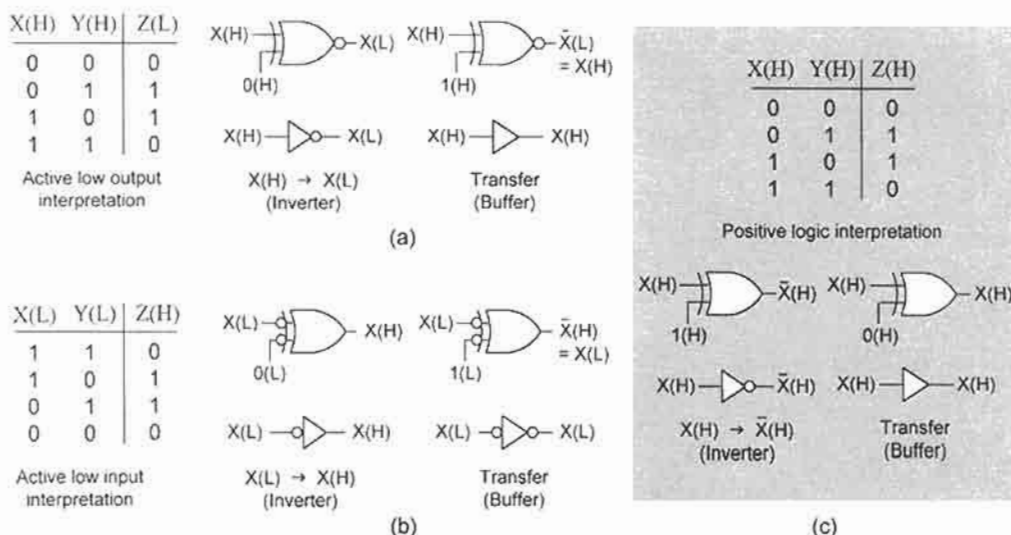
For reference purposes the logic circuit symbols representing the XOR and EQV gates are summarized here. Shown in Fig. 3.30a are the four conjugate forms for the XOR gate and in (b) the four conjugate forms for the EQV gate. The conjugate logic circuit symbol forms are interchangeable, as indicated by the two-way arrows. This information can be very useful when synthesizing logic circuits containing XOR or EQV functions. Notice that dual logic low indicator bubbles, representing double incompatibilities, have no effect on the logic function. The reader is referred to Fig. 3.29b for reinforcement of this fact.

### 3.9.6 Controlled Logic Level Conversion

In Section 3.4 the concept of logic level conversion was introduced in connection with the inverter. Here, the subject of logic level conversion is revisited as it relates to the XOR or EQV gate. Another interesting and useful property of the XOR and EQV gates is that they can be operated in either one of two modes: the *inverter mode* or the *transfer mode*. These modes are illustrated in Fig. 3.31, where exclusive use is made of the XOR symbol to represent the XOR and EQV gates. In Fig. 3.31a the XOR interpretation of the EQV gate is used for  $(H) \rightarrow (L)$  logic level conversion or for logic transfer depending on the logic level of the controlling input. Notice that the buffer symbol is used to represent the transfer mode. These two modes are easily deduced from the truth table given at left in Fig. 3.31a. Similarly, in Fig. 3.31b, the XOR interpretation of the XOR gate is used for the  $(L) \rightarrow (H)$  conversion mode or for the logic transfer mode depending on the logic level of the controlling input. Here again, these results are easily deduced from the truth table to the left in Fig. 3.31b, which has been altered from that in Fig. 3.26c to account for the active low inputs.

The positive logic interpretation of an XOR gate used as a *controlled inverter* is given in Fig. 3.31c. This is included to add greater understanding of the XOR gate and its operation. Although all three cases in Fig. 3.31 physically represent *controlled inversion*, it is common to find controlled inverters represented as in Fig. 3.31c. A typical example is in the design



**FIGURE 3.31**

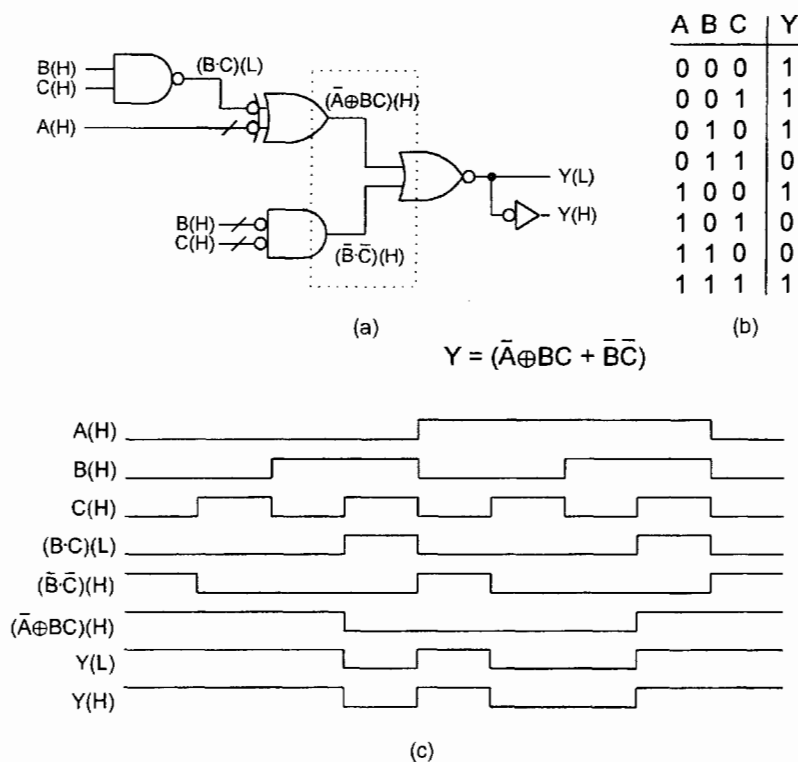
Controlled logic level conversion. (a) The EQV gate used for (H)  $\rightarrow$  (L) conversion and logic transfer. (b) The XOR gate in mixed logic notation used for (L)  $\rightarrow$  (H) conversion. (c) Positive logic interpretation of the XOR gate used as a controlled inverter.

of the adder/subtractor featured in Fig. 8.9. In making the transition from Fig. 3.31b to Fig. 3.31c, it should be recalled that complementation of both inputs to an XOR or EQV circuit symbol leaves the output function unaltered. Notice that the inverter and buffer symbols in Fig. 3.31 are the same as those given in Fig. 3.20a.

### 3.9.7 Construction and Waveform Analysis of Logic Circuits Containing XOR-Type Functions

As an extension of Section 3.8, the reading and construction of a *multilevel* logic circuit containing an XOR function is demonstrated by the NAND/XOR/NOR/INV circuit in Fig. 3.32a representing the function  $Y = \bar{A} \oplus BC + \bar{B}\bar{C}$ . A multilevel logic function is one that has more than two gate path delays from input to output. In this case there are three levels of path delay. Here, an XOR gate performs the XOR operation to yield the  $(\bar{A} \oplus BC)(H)$  input to the NOR output stage performing the OR operation. The waveform for  $\bar{B}\bar{C}(H)$  is obtained by ANDING the complement of the  $B(H)$  waveform with the complement of the  $C(H)$  waveform by using a NOR gate to perform the AND operation. Thus, there are three logic incompatibilities, one for the  $A(H)$  input and the other two for the  $B(H)$  and  $C(H)$  inputs, but inverters are not needed to create these logic level incompatibilities.

Presented in Figs. 3.32b and 3.32c are the truth table and logic waveforms for the circuit in Fig. 3.32a. The inputs are arbitrarily given in binary sequence, and the output waveforms from the intermediate stages are given to reveal the advantage of the mixed logic method. No account is taken of the propagation delays of the gates and inverters. Notice that the  $(\bar{A} \oplus BC)(H)$  and  $\bar{B}\bar{C}(H)$  logic signals are logically compatible with the requirements of the ORing operation of the NOR gate output stage. If complementation is carried out within the dashed box, the waveforms for the resulting  $(\bar{A} \oplus BC)(L)$  and  $\bar{B}\bar{C}(L)$  signals would

**FIGURE 3.32**

(a) NAND/NOR/XOR/INV logic circuit, (b) truth table, and (c) logic waveforms for the function  $Y$  with active high inputs and mixed logic outputs.

remain the same as those shown for the active high signals, but with opposite activation levels. However, the physical nature of the devices required for implementation would now become NAND/EQV/OR/AND/INV as taken from input to output.

Another interesting facet of the mixed logic method is the fact that an inverter on the output stage permits the generation of *mixed rail output* signals. The  $Y(L)$  and  $Y(H)$  signals constitute the mixed rail outputs and have waveforms that are identical if account is not taken of the propagation delay through the inverter. In the physical domain, however, the voltage waveforms represented by  $Y(L)$  and  $Y(H)$  would be the inverse of one another.

### 3.10 LAWS OF BOOLEAN ALGEBRA

To design a digital circuit that will perform a given function, it may be necessary to manipulate and combine the various switching variables in certain ways that are in agreement with mathematical logic. Use of the laws of Boolean algebra makes these manipulations and combinations relatively simple. This special mathematical logic, named in recognition of the English mathematician George Boole, can be rigorously and eloquently presented by using axioms, theorems, and corollaries. However, for our purposes there is no need for such a formal approach. The laws of Boolean algebra are relatively few and can be deduced

from the truth tables for NOT, AND, and OR. In this section these laws are developed exclusively within the logic domain with only passing reference to activation levels.

### 3.10.1 NOT, AND, and OR Laws

**NOT Laws** The *unary operator* NOT is the logic equivalent of complementation and connotes inversion in the sense of supplying the lack of something. Although NOT is purely a logic concept and complementation arises more from a physical standpoint, the two terms, NOT and complementation, will be used interchangeably following established practice.

The truth table for NOT is the positive logic interpretation of the physical truth table given in Fig. 3.6b. It is from this truth table that the NOT laws are derived.

| NOT<br>Truth Table |           |   | NOT Laws            |       |
|--------------------|-----------|---|---------------------|-------|
| $X$                | $\bar{X}$ |   |                     |       |
| 0                  | 1         | → | $\bar{0} = 1$       | (3.6) |
| 1                  | 0         |   | $\bar{1} = 0$       |       |
|                    |           |   | $\bar{\bar{X}} = X$ |       |

The NOT operation, like complementation, is designated by the overscore (or “bar”). A double bar (or double complementation) of a function, sometimes called *involution*, is the function itself, as indicated in Eqs. (3.6).

As examples of the applications of the NOT laws, suppose that  $X = A\bar{B}$ . Then the function  $\bar{X} = \overline{A\bar{B}}$  is read as *A AND B bar the quantity complemented*, and  $\bar{\bar{X}} = \overline{\overline{A\bar{B}}} = A\bar{B}$ . Or, if  $Y = 0$ , then  $\bar{Y} = \bar{0} = 1$ , etc. Finally, notice that Eqs. (3.2) can be generated one from the other by involution—even in mixed logic notation. Thus,  $\bar{\alpha}(L) = \bar{\bar{\alpha}}(H) = \alpha(H)$ , and so on.

**AND Laws** The AND laws are easily deduced by taking the rows two at a time from the truth table representing the logic AND interpretation of the AND gate given in Fig. 3.16c. Thus, by taking  $Y$  equal to logic values 0, 1,  $X$ , and  $\bar{X}$ , the four AND laws result and are given by Eqs. (3.7).

| AND<br>Truth Table |     |             |   | AND Laws              |       |
|--------------------|-----|-------------|---|-----------------------|-------|
| $X$                | $Y$ | $X \cdot Y$ |   |                       |       |
| 0                  | 0   | 0           | → | $X \cdot 0 = 0$       | (3.7) |
| 0                  | 1   | 0           |   | $X \cdot 1 = X$       |       |
| 1                  | 0   | 0           |   | $X \cdot X = X$       |       |
| 1                  | 1   | 1           |   | $X \cdot \bar{X} = 0$ |       |

To illustrate the application of the AND laws, let  $X$  be the function  $X = A + \bar{B}$  so that  $(A + \bar{B}) \cdot 0 = 0$ ,  $(A + \bar{B}) \cdot 1 = A + \bar{B}$ ,  $(A + \bar{B}) \cdot (A + \bar{B}) = A + \bar{B}$ , and  $(A + \bar{B}) \cdot (A + \bar{B}) = 0$ . These laws are valid regardless of the complexity of the function  $X$ , which can represent any multivariable function.

**OR Laws** The four OR laws are deduced from the logic OR interpretation of the OR gate given in Fig. 3.17d by assigning to  $Y$  the values 0, 1,  $X$ , and  $\bar{X}$  and are given by Eqs. (3.8). The OR laws are illustrated by letting  $X$  represent the function  $X = \bar{B}C$ . Then, according to the OR laws,  $\bar{B}C + 0 = \bar{B}C$ ,  $\bar{B}C + 1 = 1$ ,  $\bar{B}C + \bar{B}C = \bar{B}C$ , and  $\bar{B}C + \bar{\bar{B}C} = 1$ . Here again, use has been made of a multivariable function  $X$  to demonstrate the applicability of a fundamental Boolean law, the OR law.

| <b>OR</b>          |     |         |   | <u><b>OR Laws</b></u> |       |
|--------------------|-----|---------|---|-----------------------|-------|
| <b>Truth Table</b> |     |         |   |                       |       |
| $X$                | $Y$ | $X + Y$ |   |                       |       |
| 0                  | 0   | 0       | → | $X + 0 = X$           | (3.8) |
| 0                  | 1   | 1       |   | $X + 1 = 1$           |       |
| 1                  | 0   | 1       |   | $X + X = X$           |       |
| 1                  | 1   | 1       |   | $X + \bar{X} = 1$     |       |

Notice that the AND and OR laws are easily verified by substituting 0 and 1 for the multivariable function  $X$  in the examples just given, and then comparing the results with the AND and OR truth tables.

### 3.10.2 The Concept of Duality

An inspection of the AND and OR laws reveals an interesting relationship that may not be obvious at first glance. If the 1's and 0's are interchanged while the AND and OR operators,  $(\cdot)$  and  $(+)$ , are interchanged, the AND laws generate the OR laws and vice versa. For reference purposes, the interchange of 1's and 0's simultaneously with the interchange of operators is represented by the double arrows ( $\leftrightarrow$ ) as follows:

$$\left\{ \begin{array}{l} 0 \leftrightarrow 1 \\ (\cdot) \leftrightarrow (+) \\ \odot \leftrightarrow \oplus \end{array} \right\}$$

This simultaneous interchange of logic values and operators is called logic *duality*. The duality between the AND and OR laws is given by Eqs. (3.9).

| <u><b>AND Laws</b></u> |                    | <u><b>OR Laws</b></u> |       |
|------------------------|--------------------|-----------------------|-------|
| $X \cdot 0 = 0$        | By<br>Duality<br>↔ | $X + 0 = X$           | (3.9) |
| $X \cdot 1 = X$        |                    | $X + 1 = 1$           |       |
| $X \cdot X = X$        |                    | $X + X = X$           |       |
| $X \cdot \bar{X} = 0$  |                    | $X + \bar{X} = 1$     |       |

Perhaps the best way to demonstrate duality is by the two dual sets

$$(A \odot B)[A\bar{B} + \bar{A}B] = 0 \quad \longleftrightarrow \quad (A \oplus B) + [(A + \bar{B}) \cdot (\bar{A} + B)] = 1$$

and

$$X \odot (\bar{X} + Y) = X \cdot Y \quad \longleftrightarrow \quad X \oplus (\bar{X} \cdot Y) = X + Y,$$

where the double arrow ( $\longleftrightarrow$ ) again represents the duality relationship of the set. For each dual set it can be seen that an operator in the left side equation has been replaced by its dual in the right side while the logic 0 and 1 (in the first dual set) are interchanged. Note that the two equations in a given set are *not* algebraically equal—they are duals of each other. However, a dual set of equations are complementary if an equation is equal to logic 1 or logic 0 as in the first example. Such is not the case for the second set. The concept of duality pervades the entire field of mathematical logic, as will become apparent with the development of Boolean algebra.

### 3.10.3 Associative, Commutative, Distributive, Absorptive, and Consensus Laws

The associative, commutative, distributive, absorptive, and consensus laws are presented straightforwardly in terms of the multivariable functions  $X$ ,  $Y$ , and  $Z$  to emphasize their generality, but the more formal axiomatic approach is avoided for the sake of simplicity. These laws are given in a *dual* form that the reader may find useful as a mnemonic tool:

$$\text{Associative Laws} \quad \left\{ \begin{array}{l} (X \cdot Y) \cdot Z = X \cdot (Y \cdot Z) = X \cdot Y \cdot Z \\ (X + Y) + Z = X + (Y + Z) = X + Y + Z \end{array} \right\} \quad (3.10)$$

$$\text{Commutative Laws} \quad \left\{ \begin{array}{l} X \cdot Y \cdot Z = X \cdot Z \cdot Y = Z \cdot X \cdot Y = \dots \\ X + Y + Z = X + Z + Y = Z + X + Y = \dots \end{array} \right\} \quad (3.11)$$

$$\text{Distributive Laws} \quad \left\{ \begin{array}{ll} (X \cdot Y) + (X \cdot Z) = X \cdot (Y + Z) & \text{Factoring Law} \\ (X + Y) \cdot (X + Z) = X + (Y \cdot Z) & \text{Distributive Law} \end{array} \right\} \quad (3.12)$$

$$\text{Absorptive Laws} \quad \left\{ \begin{array}{l} X \cdot (\bar{X} + Y) = X \cdot Y \\ X + (\bar{X} \cdot Y) = X + Y \end{array} \right\} \quad (3.13)$$

$$\text{Consensus Laws} \quad \left\{ \begin{array}{l} (X \cdot Y) + (\bar{X} \cdot Z) + (Y \cdot Z) = (X \cdot Y) + (\bar{X} \cdot Z) \\ (X + Y) \cdot (\bar{X} + Z) \cdot (Y + Z) = (X + Y) \cdot (\bar{X} + Z) \end{array} \right\} \quad (3.14)$$

Notice that for each of the five sets of laws, duality exists whereby the AND and OR operators are simultaneously interchanged. The dual set of distributive laws in Eqs. (3.12) occur so often that they are sometimes given the names *factoring law* and *distributive law* for the first and second, respectively. The factoring law draws its name from its similarity to the factoring law of Cartesian algebra.

Although rigorous proof of these laws will not be attempted, they are easily verified by using truth tables. Shown in Figs. 3.33 and 3.34 are the truth table verifications for the AND form of the associative law and the factoring law. Their dual forms can be verified in a similar manner.

Proof of the commutative laws is obtained simply by assigning logic 0 and logic 1 to the  $X$ 's and  $Y$ 's in the two variable forms of these laws and then comparing the results with the AND and OR truth tables given by Eqs. (3.7) and (3.8), respectively.

The distributive law can also be verified by using truth tables. However, having verified the factoring law, it is simpler to prove this law with Boolean algebra by using the factoring

| Decimal | X | Y | Z | $X \cdot Y$ | $Y \cdot Z$ | $(X \cdot Y) \cdot Z$ | $X \cdot (Y \cdot Z)$ | $X \cdot Y \cdot Z$ |
|---------|---|---|---|-------------|-------------|-----------------------|-----------------------|---------------------|
| 0       | 0 | 0 | 0 | 0           | 0           | 0                     | 0                     | 0                   |
| 1       | 0 | 0 | 1 | 0           | 0           | 0                     | 0                     | 0                   |
| 2       | 0 | 1 | 0 | 0           | 0           | 0                     | 0                     | 0                   |
| 3       | 0 | 1 | 1 | 0           | 1           | 0                     | 0                     | 0                   |
| 4       | 1 | 0 | 0 | 0           | 0           | 0                     | 0                     | 0                   |
| 5       | 1 | 0 | 1 | 0           | 0           | 0                     | 0                     | 0                   |
| 6       | 1 | 1 | 0 | 1           | 0           | 0                     | 0                     | 0                   |
| 7       | 1 | 1 | 1 | 1           | 1           | 1                     | 1                     | 1                   |

**FIGURE 3.33**

Truth table for the AND form of the associative laws in Eqs. (3.10).

law together with the AND and OR laws. This is done in the following sequence of steps by using square brackets to draw attention to those portions where the laws indicated on the right are applied:

$$\begin{aligned}
 [(X + Y)(X + Z)] &= [X \cdot (X + Z)] + [Y \cdot (X + Z)] && \text{Factoring law (applied twice)} \\
 &= [X \cdot X] + (X \cdot Z) + (Y \cdot X) + (Y \cdot Z) && \text{AND law } (X \cdot X = X) \\
 &= [X + (X \cdot Z) + (Y \cdot X)] + (Y \cdot Z) && \text{Factoring law} \\
 &= X \cdot [1 + Z + Y] + (Y \cdot Z) && \text{OR law } (1 + Z + Y = 1) \\
 &= X + (Y \cdot Z).
 \end{aligned}$$

In similar fashion the second of the absorptive laws is proven as follows:

$$\begin{aligned}
 X + \bar{X}Y &= [(X + \bar{X})(X + Y)] && \text{Distributive and OR laws} \\
 &= 1 \cdot (X + Y) && \text{AND law } (1 \cdot (X + Y) = X + Y) \\
 &= X + Y.
 \end{aligned}$$

The remaining absorptive law is easily proved by first applying the factoring law followed by the AND law  $X \cdot \bar{X} = 0$ . Duality can also be used as a validation of one form once its dual is proven.

| Decimal | X | Y | Z | $X \cdot Y$ | $X \cdot Z$ | $Y + Z$ | $(X \cdot Y) + (X \cdot Z)$ | $X \cdot (Y + Z)$ |
|---------|---|---|---|-------------|-------------|---------|-----------------------------|-------------------|
| 0       | 0 | 0 | 0 | 0           | 0           | 0       | 0                           | 0                 |
| 1       | 0 | 0 | 1 | 0           | 0           | 1       | 0                           | 0                 |
| 2       | 0 | 1 | 0 | 0           | 0           | 1       | 0                           | 0                 |
| 3       | 0 | 1 | 1 | 0           | 0           | 1       | 0                           | 0                 |
| 4       | 1 | 0 | 0 | 0           | 0           | 0       | 0                           | 0                 |
| 5       | 1 | 0 | 1 | 0           | 1           | 1       | 1                           | 1                 |
| 6       | 1 | 1 | 0 | 1           | 0           | 1       | 1                           | 1                 |
| 7       | 1 | 1 | 1 | 1           | 1           | 1       | 1                           | 1                 |

**FIGURE 3.34**

Truth table for the factoring law given in Eqs. (3.12).

The first of the consensus laws in Eqs. (3.14) is proven by applying the OR and factoring laws:

$$\begin{aligned}
 XY + \bar{X}Z + YZ &= XY + \bar{X}Z + [(X + \bar{X})YZ] && \text{OR law and factoring law} \\
 &= XY + \bar{X}Z + [XYZ + \bar{X}YZ] && \text{Factoring law} \\
 &= [XY(1 + Z)] + [\bar{X}Z(1 + Y)] && \text{Factoring law (applied twice); OR law} \\
 &= XY + \bar{X}Z.
 \end{aligned}$$

Proof of the second of the consensus laws follows by duality.

### 3.10.4 DeMorgan's Laws

In the latter half of the nineteenth century, the English logician and mathematician Augustus DeMorgan proposed two theorems of mathematical logic that have since become known as DeMorgan's theorems. The Boolean algebraic representations of these theorems are commonly known as DeMorgan's laws. In terms of the two multivariable functions  $X$  and  $Y$ , these laws are given in dual form by

$$\text{DeMorgan's Laws} \quad \left\{ \begin{array}{l} \overline{X \cdot Y} = \bar{X} + \bar{Y} \\ \overline{X + Y} = \bar{X} \cdot \bar{Y} \end{array} \right\} \quad (3.15)$$

More generally, for any number of functions, the DeMorgan laws take the following form:

$$\overline{X \cdot Y \cdot Z \cdot \dots \cdot N} = \bar{X} + \bar{Y} + \bar{Z} + \dots + \bar{N} \quad (3.15a)$$

and

$$\overline{X + Y + Z + \dots + N} = \bar{X} \cdot \bar{Y} \cdot \bar{Z} \cdot \dots \cdot \bar{N}.$$

DeMorgan's laws are easily verified by using truth tables. Shown in Fig. 3.35 is the truth table for the first of Eqs. (3.15).

Application of DeMorgan's laws can be demonstrated by proving the absorptive law  $X + \bar{X}Y = X + Y$ :

$$\overline{\overline{X + \bar{X}Y}} = \overline{\bar{X} \cdot (\bar{\bar{X}Y})} = \overline{\bar{X} \cdot (\bar{X + Y})} = \overline{\bar{X} \cdot X + \bar{X} \cdot \bar{Y}} = \overline{\bar{X} \cdot \bar{Y}} = X + Y.$$

Notice that the double bar over the term  $X + \bar{X}Y$  is a NOT law and does not alter the term. Here, DeMorgan's laws are first applied by action of the "inner" bar followed by simplification under the "outer" bar. Final application of DeMorgan's law by application

| $X$ | $Y$ | $X \cdot Y$ | $\overline{X \cdot Y}$ | $\bar{X}$ | $\bar{Y}$ | $\bar{X} + \bar{Y}$ |
|-----|-----|-------------|------------------------|-----------|-----------|---------------------|
| 0   | 0   | 0           | 1                      | 1         | 1         | 1                   |
| 0   | 1   | 0           | 1                      | 1         | 0         | 1                   |
| 1   | 0   | 0           | 1                      | 0         | 1         | 1                   |
| 1   | 1   | 1           | 0                      | 0         | 0         | 0                   |

FIGURE 3.35

Truth table for DeMorgan's Law  $\overline{X \cdot Y} = \bar{X} + \bar{Y}$ .

of the outer bar takes place only after simplification. As a general rule, DeMorgan's laws should be applied to a function only after it has been sufficiently reduced so as to avoid unnecessary Boolean manipulation.

### 3.11 LAWS OF XOR ALGEBRA

The laws of XOR algebra share many similarities with those of conventional Boolean algebra discussed in the previous section and can be viewed as a natural extension of the conventional laws. Just as the AND and OR laws are deduced from their respective truth tables, the XOR and EQV laws are deduced from their respective truth tables in Figs. 3.26c and 3.27d and are given by Eqs. (3.16) together with their truth tables:

| XOR<br>Truth Table |   |              |  | EQV<br>Truth Table |   |             |
|--------------------|---|--------------|--|--------------------|---|-------------|
| X                  | Y | $X \oplus Y$ |  | X                  | Y | $X \odot Y$ |
| 0                  | 0 | 0            |  | 0                  | 0 | 1           |
| 0                  | 1 | 1            |  | 0                  | 1 | 0           |
| 1                  | 0 | 1            |  | 1                  | 0 | 0           |
| 1                  | 1 | 0            |  | 1                  | 1 | 1           |

**XOR Laws**

$$\begin{aligned}
 X \oplus 0 &= X \\
 X \oplus 1 &= \bar{X} \\
 X \oplus X &= 0 \\
 X \oplus \bar{X} &= 1
 \end{aligned}$$

**By**

 $\longleftrightarrow$ 

**Duality**

**EQV Laws**

$$\begin{aligned}
 X \odot 1 &= X \\
 X \odot 0 &= \bar{X} \\
 X \odot X &= 1 \\
 X \odot \bar{X} &= 0
 \end{aligned}$$

(3.16)

Here, the dual relationship between the XOR and EQV laws is established by interchanging the 1's and 0's while simultaneously interchanging the XOR and EQV operators, as indicated by the double arrow.

The associative and commutative laws for EQV and XOR follow from the associative and commutative laws for AND and OR given by Eqs. (3.10) and (3.11) by exchanging operator symbols:  $\odot$  for  $(\cdot)$  and  $\oplus$  for  $(+)$ . The distributive, absorptive, and consensus laws of XOR algebra follow from their AND/OR counterparts in Eqs. (3.12), (3.13), and (3.14) by replacing the appropriate  $(+)$  operator symbols with the  $\oplus$  operator symbols, and by replacing the appropriate  $(\cdot)$  symbols with the  $\odot$  symbol, but not both in any given expression. In similar fashion, DeMorgan's laws in XOR algebra are produced by substituting  $\odot$  for  $(\cdot)$  and  $\oplus$  for  $(+)$  in Eqs. (3.15). These laws are presented as follows in dual form and in terms of variables  $X$ ,  $Y$ , and  $Z$ , which may represent single or multivariable functions:

$$\text{Associative Law} \quad \left\{ \begin{aligned} (X \odot Y) \odot Z &= X \odot (Y \odot Z) = X \odot Y \odot Z \\ (X \oplus Y) \oplus Z &= X \oplus (Y \oplus Z) = X \oplus Y \oplus Z \end{aligned} \right\} \quad (3.17)$$

$$\text{Commutative Laws} \quad \left\{ \begin{aligned} X \odot Y \odot Z &= X \odot Z \odot Y = Z \odot X \odot Y = \dots \\ X \oplus Y \oplus Z &= X \oplus Z \oplus Y = Z \oplus X \oplus Y = \dots \end{aligned} \right\} \quad (3.18)$$



$$\text{Distributive Laws} \quad \left\{ \begin{array}{ll} (X \cdot Y) \oplus (X \cdot Z) = X \cdot (Y \oplus Z) & \text{Factoring Law} \\ (X + Y) \odot (X + Z) = X + (Y \odot Z) & \text{Distributive Law} \end{array} \right\} \quad (3.19)$$

$$\text{Absorptive Laws} \quad \left\{ \begin{array}{l} X \cdot (\bar{X} \oplus Y) = X \cdot Y \\ X + (\bar{X} \odot Y) = X + Y \end{array} \right\} \quad (3.20)$$

$$\text{Consensus Laws} \quad \left\{ \begin{array}{l} (X \cdot Y) \oplus (\bar{X} \cdot Z) + (Y \cdot Z) = (X \cdot Y) \oplus (\bar{X} \cdot Z) \\ (X + Y) \odot (\bar{X} + Z) \cdot (Y + Z) = (X + Y) \odot (\bar{X} + Z) \end{array} \right\} \quad (3.21)$$

$$\text{DeMorgan's Laws} \quad \left\{ \begin{array}{l} \overline{X \odot Y} = \bar{X} \oplus \bar{Y} = X \oplus Y \\ \overline{X \oplus Y} = \bar{X} \odot \bar{Y} = X \odot Y \end{array} \right\}. \quad (3.22)$$

Notice that each of the six sets of equations are presented in dual form. Thus, by interchanging AND and OR operators while simultaneously interchanging EQV and XOR operators, duality of the set is established. The first of the distributive laws given in Eqs. (3.19) can be termed the *factoring law* of XOR algebra owing to its similarity with the factoring law of Cartesian algebra and that of Eqs. (3.12).

Generalizations of DeMorgan's XOR laws follow from Eqs. (3.15a) and (3.22) and are given by

$$\overline{X \odot Y \odot Z \odot \cdots \odot N} = \bar{X} \oplus \bar{Y} \oplus \bar{Z} \oplus \cdots \oplus \bar{N} \quad (3.22a)$$

and

$$\overline{X \oplus Y \oplus Z \oplus \cdots \oplus N} = \bar{X} \odot \bar{Y} \odot \bar{Z} \odot \cdots \odot \bar{N}.$$

Verification of the associative, commutative, and distributive laws is easily accomplished by using truth tables. For example, the second of the distributive laws in Eqs. (3.19) is verified by the truth table in Fig. 3.36. Here, Eq. (3.5) is used together with the OR laws [Eqs. (3.8)] to show the identity of the terms  $(X + Y) \odot (X + Z)$  and  $X + (Y \odot Z)$ .

The distributive laws may also be proven by using Boolean algebra. For example, the factoring law of Eqs. (3.19) is proven by applying the defining relation of the XOR function

| X | Y | Z | X + Y | X + Z | Y ⊙ Z | (X + Y) ⊙ (X + Z) | X + (Y ⊙ Z) |
|---|---|---|-------|-------|-------|-------------------|-------------|
| 0 | 0 | 0 | 0     | 0     | 1     | 1                 | 1           |
| 0 | 0 | 1 | 0     | 1     | 0     | 0                 | 0           |
| 0 | 1 | 0 | 1     | 0     | 0     | 0                 | 0           |
| 0 | 1 | 1 | 1     | 1     | 1     | 1                 | 1           |
| 1 | 0 | 0 | 1     | 1     | 1     | 1                 | 1           |
| 1 | 0 | 1 | 1     | 1     | 0     | 1                 | 1           |
| 1 | 1 | 0 | 1     | 1     | 0     | 1                 | 1           |
| 1 | 1 | 1 | 1     | 1     | 1     | 1                 | 1           |

**FIGURE 3.36**

Truth table for the XOR distributive law given in Eqs. (3.19).

given by Eq. (3.4) and by using the AND and OR laws of Eqs. (3.9):

$$\begin{aligned}
 [(XY) \oplus (XZ)] &= [(\overline{XY})(XZ) + (XY)(\overline{XZ})] && \text{Eq. (3.4) and Eq. (3.15)} \\
 &= [(\bar{X} + \bar{Y})(XZ)] + [(XY)(\bar{X} + \bar{Z})] && \text{Factoring law [Eqs. (3.12)]} \\
 &= [\bar{X}XZ + \bar{Y}\bar{X}Z] + [X\bar{X}Y + XY\bar{Z}] && \text{AND and OR laws [Eqs. (3.9)]} \\
 &= [X\bar{Y}Z + XY\bar{Z}] && \text{Factoring law [Eqs. (3.12)]} \\
 &= X[\bar{Y}Z + Y\bar{Z}] && \text{Eq. (3.4)} \\
 &= X(Y \oplus Z).
 \end{aligned}$$

In these equations, the square brackets [ ] are used to draw attention to those portions where the laws or equations indicated on the right are to be applied in going to the next step. Equation (3.4) refers to the defining relation for XOR given by  $X \oplus Y = \bar{X}Y + X\bar{Y}$ .

The absorptive laws are also easily proven by using Boolean algebra. Beginning with the first of Eqs. (3.20), there follows

$$\begin{aligned}
 X \cdot [(X \oplus Y)] &= X \cdot (\bar{X}\bar{Y} + XY) && \text{Eq. (3.4)} \\
 &= [X \cdot (\bar{X}\bar{Y} + XY)] && \text{Factoring law [Eqs. (3.12)]} \\
 &= [X \cdot \bar{X}\bar{Y} + X \cdot XY] && \text{AND and OR laws [Eqs. (3.9)]} \\
 &= XY,
 \end{aligned}$$

where the square brackets [ ] are again used to draw attention to those portions where the laws or equations indicated on the right are to be applied. The second of Eqs. (3.20) is proven by the following sequence of steps:

$$\begin{aligned}
 X + [(X \odot Y)] &= X + (X\bar{Y}) + (\bar{X}Y) && \text{Eq. (3.5)} \\
 &= [X + (X\bar{Y})] + \bar{X}Y && \text{Factoring law [Eqs. (3.12)]} \\
 &= [X(1 + \bar{Y}) + \bar{X}Y] && \text{OR and AND laws} \\
 &= [X + \bar{X}Y] && \text{Absorptive law [Eqs. (3.13)]} \\
 &= X + Y.
 \end{aligned}$$

Notice that in the foregoing proofs, use is tacitly made of the important dual relations

$$\begin{aligned}
 \bar{X} \oplus Y &= X \oplus \bar{Y} = \overline{X \oplus \bar{Y}} = X \odot Y \\
 \bar{X} \odot Y &= X \odot \bar{Y} = \overline{X \odot \bar{Y}} = X \oplus Y.
 \end{aligned} \tag{3.23}$$

These relations are easily verified by replacing the variable ( $X$  or  $Y$ ) by its complement ( $\bar{X}$  or  $\bar{Y}$ ) in the appropriate defining relation, (3.4) or (3.5).

An inspection of Eqs. (3.23) reveals what should already be understood — that complementing one of the connecting variables complements the function, and that the complement of an XOR function is the EQV function and vice versa. A generalization of this can be stated as follows:

*In any string of terms interconnected only with XOR and/or EQV operators, an odd number of complementations (variable or operator complementations) complements the function, whereas an even number of complementations preserves the function.*

To illustrate, consider a function  $F$  consisting of a string of four multivariable terms  $W$ ,  $X$ ,  $Y$ , and  $Z$  interconnected initially by XOR operators:

$$\begin{aligned}
 F &= W \oplus X \oplus Y \oplus Z = W \oplus \bar{X} \oplus Y \odot Z \\
 &= W \odot \bar{X} \oplus \bar{Y} \oplus \bar{Z} = W \odot X \odot Y \oplus Z = \dots \\
 \bar{F} &= W \oplus \bar{X} \oplus Y \oplus Z = W \oplus \bar{X} \oplus Y \odot \bar{Z} \\
 &= W \odot X \odot Y \odot Z = \bar{W} \odot X \oplus \bar{Y} \oplus Z = \dots
 \end{aligned} \tag{3.24}$$

An examination of Eqs. (3.24) reveals that there are 64 possible expressions representing  $F$  and 64 for  $\bar{F}$ , all generated by repeated applications of Eqs. (3.23). The number 64 is derived from combinations of seven different objects taken an even or odd number at a time for  $F$  and  $\bar{F}$ , respectively.

Application of Eqs. (3.24) is illustrated by

$$\begin{aligned}
 A \odot (A \odot D + \bar{C}) \odot \bar{B} &= A \odot [(A \oplus D)C] \odot B \\
 &= A \oplus [(A \oplus D)C] \oplus B,
 \end{aligned}$$

where the original function has been converted from one having only EQV operators and two complemented variables to one having only XOR operators with no complemented variables. The two alternative forms (right side) differ from each other by only two complementations. Notice also that the first alternative form involved applications of DeMorgan's laws given by Eqs. (3.15) and (3.22).

### 3.11.1 Two Useful Corollaries

Interesting and useful relationships result between XOR algebra and conventional Boolean algebra by recognition of the following two dual corollaries, which follow directly from the definitions of the XOR and EQV operations:

**COROLLARY I** *If two functions,  $\alpha$  and  $\beta$ , never take the logic 1 value at the same time, then*

$$\alpha \cdot \beta = 0 \quad \text{and} \quad \alpha + \beta = \alpha \oplus \beta \tag{3.25}$$

*and the logic operators  $(+)$  and  $(\oplus)$  are interchangeable.*

**COROLLARY II** *If two functions,  $\alpha$  and  $\beta$ , never take the logic 0 value at the same time, then*

$$\alpha + \beta = 1 \quad \text{and} \quad \alpha \cdot \beta = \alpha \odot \beta \tag{3.26}$$

*and the logic operators  $(\cdot)$  and  $(\odot)$  are interchangeable.*

Corollary I requires that  $\alpha$  and  $\beta$  each be terms consisting of ANDed variables called *product terms* (p-terms) and that they be *disjoint*, meaning that the two terms never take logic 1 simultaneously. By duality, Corollary II requires that  $\alpha$  and  $\beta$  each be terms consisting of ORed variables called *sum terms* (s-terms) and that they be *disjoint*, meaning that the two terms never take logic 0 simultaneously. The subject of these corollaries will be revisited in Section 5.5 where their generalizations will be discussed.

The most obvious application of Corollaries I and II is in operator interchange as demonstrated by the following four examples:

$$[1] \quad AB + \bar{B}C = (AB) \oplus (\bar{B}C),$$

where  $\alpha = AB$ ,  $\beta = \bar{B}C$ , and  $\alpha \cdot \beta = 0$  by Corollary I.

$$[2] \quad (\bar{A} + B + X) \cdot (A + B + C + Y) = (\bar{A} + B + X) \odot (A + B + C + Y),$$

where  $\alpha = (\bar{A} + B + X)$ ,  $\beta = (A + B + C + Y)$  and  $\alpha + \beta = 1$  according to Corollary II.

$$[3] \quad a + \bar{b} \oplus bc = a + \bar{b} + bc = a + \bar{b} + c,$$

where Corollary I has been applied followed by the absorptive law in Eqs. (3.13).

$$[4] \quad (X\bar{Y}) \odot (\bar{X} + Y + \bar{Z}) = (X\bar{Y})(\bar{X} + Y + \bar{Z}) = X\bar{Y}\bar{Z}$$

Here, Corollary II is applicable since  $\overline{X\bar{Y}} = \bar{X} + Y$ , and the result follows by using the AND and OR laws given by Eqs. (3.9).

### 3.11.2 Summary of Useful Identities

The laws of XOR algebra have been presented in the foregoing subsections. There are several identities that follow directly or indirectly from these laws. These identities are useful for function simplification and are presented here in *dual* form for reference purposes.

$$\left\{ \begin{array}{l} \bar{X} \oplus Y = X \oplus \bar{Y} = \overline{X \oplus Y} = \bar{X} \odot \bar{Y} = X \odot Y \\ \bar{X} \odot Y = X \odot \bar{Y} = \overline{X \odot Y} = \bar{X} \oplus \bar{Y} = X \oplus Y \end{array} \right\} \quad (3.27)$$

$$\left\{ \begin{array}{l} X \oplus \bar{X} = X \odot X = 1 \\ X \odot \bar{X} = X \oplus X = 0 \end{array} \right\} \quad (3.28)$$

$$\left\{ \begin{array}{l} 1 \oplus X = \bar{X} \\ 0 \odot X = \bar{X} \end{array} \right\} \quad \left\{ \begin{array}{l} 0 \oplus X = X \\ 1 \odot X = X \end{array} \right\} \quad (3.29)$$

$$\left\{ \begin{array}{l} X\bar{Y} \oplus X = \bar{X}Y \oplus Y = XY \\ (X + \bar{Y}) \odot X = (\bar{X} + Y) \odot Y = X + Y \end{array} \right\} \quad (3.30)$$

$$\left\{ \begin{array}{l} \bar{X}Y \oplus X = X\bar{Y} \oplus Y = 1 \oplus \bar{X}\bar{Y} \\ (\bar{X} + Y) \odot X = (X + \bar{Y}) \odot Y = 0 \odot (\bar{X} + \bar{Y}) \end{array} \right\} \quad (3.31)$$

$$\left\{ \begin{array}{l} (XY) \oplus (X + Y) = X \oplus Y \\ (X + Y) \odot (XY) = X \odot Y \end{array} \right\} \quad (3.32)$$

$$\left\{ \begin{array}{l} XY + YZ + XZ = XY \oplus YZ \oplus XZ \\ (X + Y)(Y + Z)(X + Z) = (X + Y) \odot (Y + Z) \odot (X + Z) \end{array} \right\} \quad (3.33)$$

Note that in these identities, either  $X$  or  $Y$  or both may represent multivariable functions or single variables of any polarity (i.e., either complemented or uncomplemented).

By applying the laws and corollaries previously given, the *majority functions* identity of three variables expressed by Eqs. (3.33) is proven as follows:

$$\begin{aligned}
 XY + YZ + XZ &= XY(\bar{Z} + Z) + (\bar{X} + X)YZ + X(\bar{Y} + Y)Z && \text{OR Laws} \\
 &= XYZ + XY\bar{Z} + XYZ + \bar{X}YZ + XYZ + X\bar{Y}Z && [\text{Eqs. (3.8)}] \\
 &= XYZ \oplus XY\bar{Z} \oplus (\bar{X}YZ + X\bar{Y}Z) && \text{Eqs. (3.19) and OR Laws} \\
 &= XY \oplus (Y \oplus X)Z && \text{Corollary I} \\
 &= XY \oplus YZ \oplus XZ. && \text{Eqs. (3.19), OR Law, Eq. (3.4) Eq. (3.8)}
 \end{aligned}$$

Proof of the second identity of Eqs. (3.33) follows by duality, that is, simply by interchanging all (+) with (·) operators while simultaneously interchanging all  $\oplus$  with  $\odot$  operators. The *generalized majority function* identity is given by

$$\begin{aligned}
 [WXY \cdots + WXZ \cdots + WYZ \cdots + XYZ \cdots + \cdots] \\
 = [WXY \cdots \oplus WXZ \cdots \oplus WYZ \cdots \oplus XYZ \cdots \oplus \cdots],
 \end{aligned}$$

which also has its dual formed by the simultaneous interchange of the operators.

This concludes the treatment of Boolean algebra. While not intended to be an exhaustive coverage of the subject, it is adequate for the needs of digital design as presented in this text. Additional references on Boolean algebra are available in the list of further reading that follows.

### 3.12 WORKED EXAMPLES

**EXAMPLE 3.1** Given the waveforms (heavy lines) at the top of Figs. 3.37a and 3.37b, draw the two waveforms for the two terminals below each.

**EXAMPLE 3.2** Complete the physical truth table in Fig. 3.38b for the CMOS logic circuit given in Fig. 3.38a. Name the gate and give the two conjugate logic circuit symbols for this gate in part (c).

**EXAMPLE 3.3** The logic circuit in Fig. 3.39 is a redundant circuit, meaning that excessive logic is used to implement the function  $Z(H)$ . (a) Name the physical gates that are used in the logic circuit in Fig. 3.39. (b) Read the circuit in mixed-logic notation and express the results in reduced, polarized Boolean form at nodes  $W$ ,  $X$ ,  $Y$ , and  $Z$ .

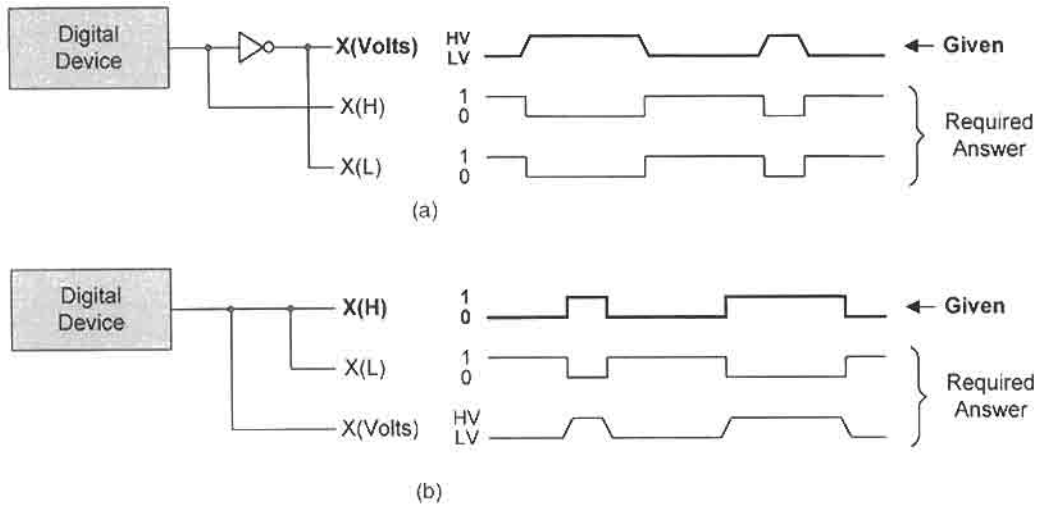
(a) (1) NAND, (2) NOR, (3) NOR, (4) OR, (5) AND, (6) NOR

(b)  $W(H) = A\bar{B}(H)$

$$X(L) = \bar{B}\bar{C}(L)$$

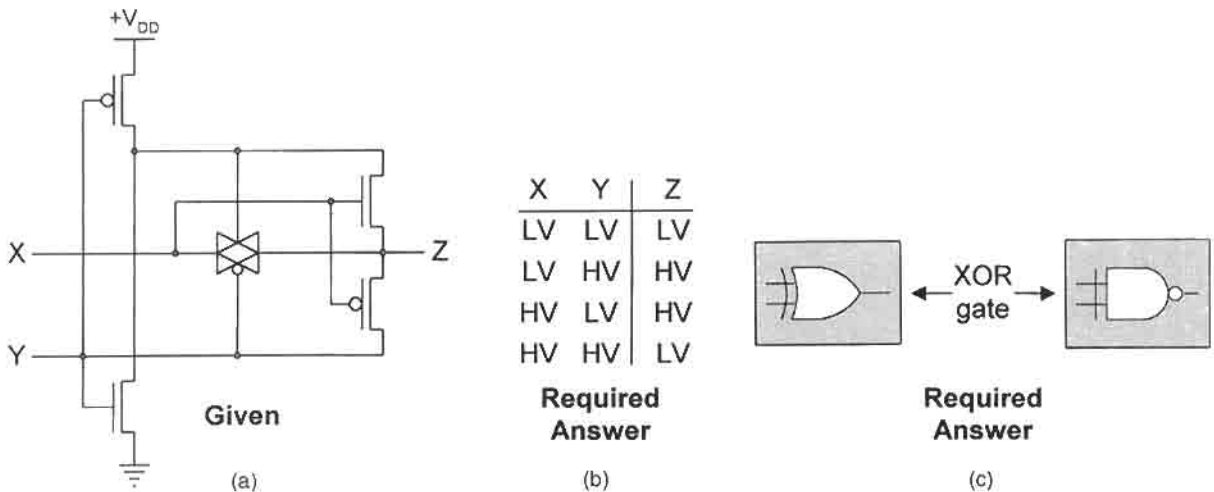
$$Y(L) = (\bar{C} + D)(L)$$

$$\begin{aligned}
 Z(H) &= \bar{W}XY(H) = (\overline{A\bar{B}})(\bar{B}\bar{C})(\bar{C} + D)(H) = (\bar{A} + B)(\bar{B}\bar{C})(\bar{C} + D)(H) \\
 &= (\bar{A} + B)(\bar{B}\bar{C} + \bar{B}\bar{C}D)(H) \\
 &= \bar{A}\bar{B}\bar{C}(H)
 \end{aligned}$$



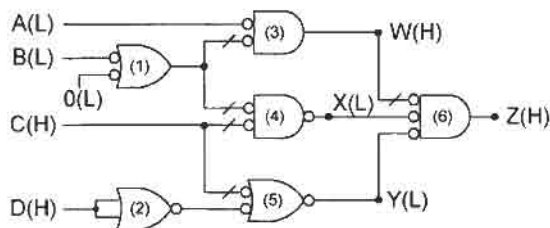
**FIGURE 3.37**

Physical (voltage) waveforms and mixed-logic notation. (a) Effect of logic level conversion. (b) Absence of logic level conversion.



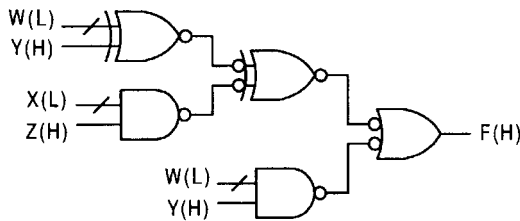
**FIGURE 3.38**

Physical truth table and logic circuit symbols for a CMOS logic gate. (a) Given logic circuit. (b) Physical truth table for an XOR gate. (c) Conjugate logic circuit symbols for the XOR gate.



**FIGURE 3.39**

A redundant logic circuit.

**FIGURE 3.40**

Logic circuit for the function given in Example 3.4.

**EXAMPLE 3.4** Use three NAND gates and two EQV gates (nothing else) to implement the following function *exactly* as written:

$$F(H) = [(\bar{W} \oplus Y) \oplus (\bar{X}Z) + \bar{W}Y](H)$$

The solution is shown in Fig. 3.40.

**EXAMPLE 3.5** A simple function of three inputs is given by the following expression:

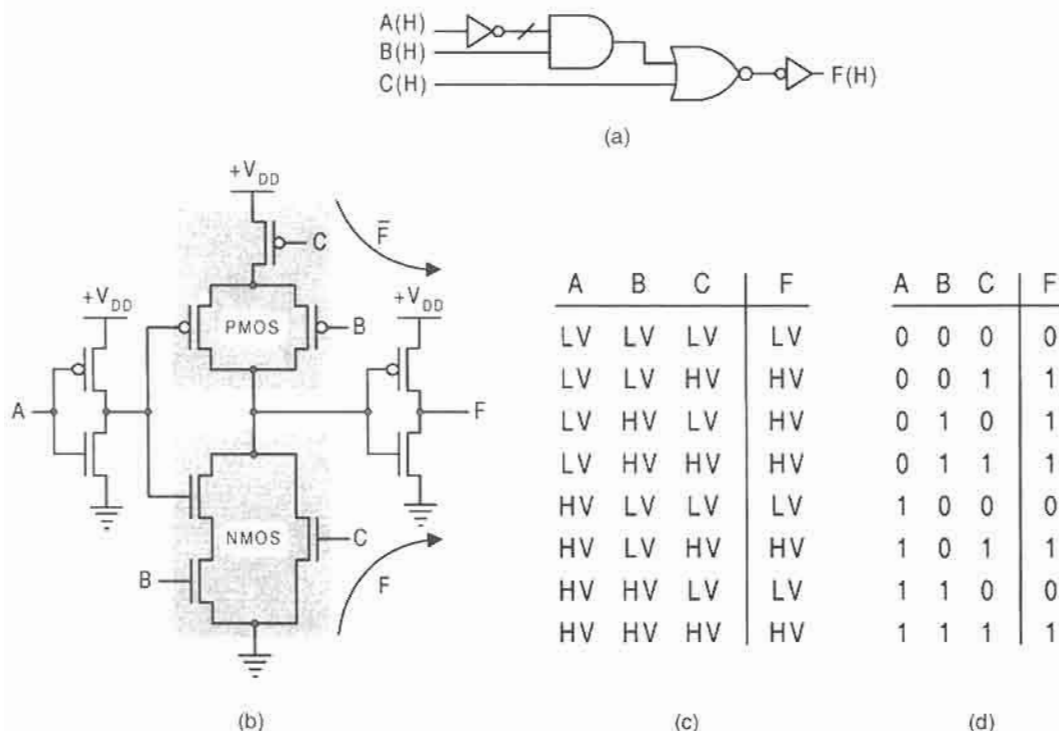
$$F(H) = (\bar{A}B + C)(H).$$

- Construct the logic circuit by using AND/NOR/INV logic. Assume that the inputs arrive active high.
- Construct the CMOS circuit for the function given in Part (a).
- Obtain the physical truth table for the circuit of Part (b).
- Obtain the positive logic truth table for the circuit of Part (b).

The solutions to Example 3.5 are given in Fig. 3.41. Notice that PMOS and NMOS are organized according to Fig. 3.5, and that the PMOS section generates the complement of that of the NMOS section, hence the complementary MOS. Also note that the output of the *A* inverter is connected to both the PMOS and NMOS inputs of the complementary sections for *F*.

**EXAMPLE 3.6** Use the laws of Boolean algebra, including XOR algebra and corollaries, to reduce each of the following expressions to their *simplest* form. Name the law(s) in each step.

- [1]  $\bar{A} + A\bar{B}\bar{C} + \overline{(B + \bar{C})} = \bar{A} + A\bar{B}\bar{C} + \bar{B}C$  DeMorgan's law [Eqs. (3.15)]  
 $= \bar{A} + \bar{B}\bar{C} + \bar{B}C$  Absorptive law [Eqs. (3.13)]  
 $= \bar{A} + \bar{B}(\bar{C} + C)$  Factoring law [Eqs. (3.12)]  
 $= \bar{A} + \bar{B}$  AND and Or laws [Eqs. (3.7) and (3.8)]
- [2]  $(a + b)(\bar{a} + c)(\bar{a} + \bar{c}) = (a + b)(\bar{a} + c \cdot \bar{c})$  Distributive law [Eqs. (3.12)]  
 $= (a + b)\bar{a}$  AND and OR laws [Eqs. (3.7) and (3.8)]  
 $= \bar{a}a + \bar{a}b$  Factoring law [Eqs. (3.12)]  
 $= \bar{a}b$  AND and OR laws

**FIGURE 3.41**

Circuit and truth table representations for the function  $F$  of Example 3.5. (a) Logic circuit. (b) CMOS circuit. (c) Physical truth table. (d) Logic truth table.

$$\begin{aligned}
 [3] \quad (X + Y)(X + \bar{Z})[Y(X + \bar{Z}) + \bar{Y}] &= (X + Y\bar{Z})[Y(X + \bar{Z}) + \bar{Y}] && \text{Distributive law} \\
 & && [\text{Eqs. (3.12)}] \\
 &= (X + Y\bar{Z})(X + \bar{Y} + \bar{Z}) && \text{Absorptive law} \\
 & && [\text{Eqs. (3.13)}] \\
 &= X + Y\bar{Z}\bar{Y} + Y\bar{Z}\bar{Z} && \text{Distributive law} \\
 &= X + Y\bar{Z} && \text{AND and OR laws}
 \end{aligned}$$

$$\begin{aligned}
 [4] \quad (b \oplus \bar{c}) + (ab)(\bar{a} + \bar{c}) &= (b \odot c) + (\bar{a} + \bar{b})(a\bar{c}) && \text{Eqs. (3.23); DeMorgan's laws} \\
 & && [\text{Eqs. (3.15)}] \\
 &= (b \odot c) + \bar{a}a\bar{c} + \bar{a}\bar{b}\bar{c} && \text{Factoring law [Eqs. (3.12)]} \\
 &= (bc + \bar{b}\bar{c} + a\bar{b}\bar{c}) && \text{Eq. (3.5); AND and OR laws} \\
 &= (bc + \bar{b}\bar{c}) && \text{Factoring law; AND and OR laws} \\
 &= b \oplus c && \text{Eqs. (3.5) and (3.23)}
 \end{aligned}$$

$$\begin{aligned}
 [5] \quad (\bar{X} + Y) \odot (X \oplus Y) &= (\bar{X} + Y) \odot (\bar{X}Y + X\bar{Y}) && \text{Eq. (3.4)} \\
 &= (\bar{X} + Y)(\bar{X}Y + X\bar{Y}) && \text{Corollary II [Eq. (3.26)]} \\
 &= \bar{X}Y + \bar{X}Y && \text{Factoring law; AND and OR laws} \\
 &= \bar{X}Y && \text{OR laws}
 \end{aligned}$$



$$\begin{aligned}
 [6] \quad (A + \bar{B} + \bar{A}C) \odot (\bar{A}B + C) &= (A + \bar{B} + \bar{A}C)(\bar{A}B + C) && \text{Corollary II} \\
 &= (A + \bar{B} + C)(\bar{A}B + C) && \text{Absorption [Eqs. (3.13)]} \\
 &= C + (A + \bar{B})(\bar{A}B) && \text{Distributive law} \\
 & && \text{[Eqs. (3.12)]} \\
 &= C && \text{Factoring law; AND and OR laws}
 \end{aligned}$$

$$\begin{aligned}
 [7] \quad \bar{a}c + (\bar{a} + b) \odot (\bar{a} + bc) &= \bar{a}c + (a\bar{b}) \oplus (\bar{a} + bc) && \text{DeMorgan's law [Eqs. (3.15); Eqs. (3.23)]} \\
 &= \bar{a}c + (a\bar{b}) + (\bar{a} + bc) && \text{Corollary I [Eq. (3.25)]} \\
 &= a\bar{b} + \bar{a} + bc && \text{Factoring law; AND and OR laws} \\
 &= \bar{a} + \bar{b} + bc && \text{Absorption [Eqs. (3.13)]} \\
 &= \bar{a} + \bar{b} + c && \text{Absorption}
 \end{aligned}$$

$$\begin{aligned}
 [8] \quad w\bar{x}\bar{y} + wx\bar{z} + \bar{w}x\bar{z} + w\bar{y}\bar{z} + x\bar{z} &= w\bar{x}\bar{y} + wx\bar{z} + \bar{w}x\bar{z} && \text{Consensus law [Eqs. (3.14)]} \\
 &= w\bar{x}\bar{y} + x\bar{z}(w + \bar{w}) && \text{Factoring law [Eqs. (3.12)]} \\
 &= w\bar{x}\bar{y} + x\bar{z} && \text{Or laws}
 \end{aligned}$$

$$\begin{aligned}
 [9] \quad A \oplus B \oplus (A + B) &= A \oplus [\bar{B} \oplus (\bar{A}\bar{B})] && \text{Eqs. (3.27)} \\
 &= A \oplus [\bar{B}(1 \oplus \bar{A})] && \text{XOR Factoring law [Eqs. (3.19)]} \\
 &= A \oplus (A\bar{B}) && \text{Eqs. (3.29)} \\
 &= A(1 \oplus \bar{B}) && \text{XOR Factoring law} \\
 &= AB && \text{Eqs. (3.29)}
 \end{aligned}$$

$$\begin{aligned}
 [10] \quad f &= d \oplus \bar{b}\bar{c}\bar{d} \oplus ab\bar{d} \oplus cd \oplus a\bar{d} \oplus \bar{a}\bar{b}\bar{c}\bar{d} \\
 &= [d \oplus cd] \oplus [ab\bar{d} \oplus a\bar{d}] \oplus [\bar{b}\bar{c}\bar{d} \oplus \bar{a}\bar{b}\bar{c}\bar{d}] && \text{Rearranging terms} \\
 &= [d(1 \oplus c)] \oplus [a\bar{d}(b \oplus 1)] \oplus [\bar{b}\bar{c}\bar{d}(1 \oplus \bar{a})] && \text{XOR Factoring law [Eqs. (3.19)]} \\
 &= \bar{c}d \oplus a\bar{b}\bar{d} \oplus \bar{a}\bar{b}\bar{c}\bar{d} && \text{Repeated applications of Eqs. (3.29)} \\
 &= \bar{c}d \oplus [a\bar{b}\bar{d}(1 \oplus c)] && \text{XOR Factoring law [Eqs. (3.19)]} \\
 &= \bar{c}d \oplus a\bar{b}\bar{c}\bar{d} && \text{Application of Eqs. (3.29)}
 \end{aligned}$$

Notice that the gate/input tally of  $f$  has been reduced from 10/24 to 3/8 in the final expression. Application of Corollary I further reduces  $f$  to  $(ab\bar{c} + \bar{c}d)$ .

## FURTHER READING

Additional reading on the subject of mixed logic notation and symbology can be found in the texts of Comer, Fletcher, Shaw and Tinder.

- [1] D. J. Comer, *Digital Logic and State Machine Design*, 3rd ed. Saunders College Publishing, Fort Worth, TX, 1995.

- [2] W. I. Fletcher, *An Engineering Approach to Digital Design*. Prentice Hall, Englewood Cliffs, NJ, 1980.
- [3] A. W. Shaw, *Logic Circuit Design*. Sanders College Publishing, Fort Worth, TX, 1993.
- [4] R. F. Tinder, *Digital Engineering Design: A Modern Approach*. Prentice Hall, Englewood Cliffs, NJ, 1991.

Virtually every text on digital or logic design provides some coverage of Boolean algebra. The texts of McCluskey and Dietmeyer are noteworthy for their coverage of both conventional Boolean algebra and XOR algebra including a very limited treatment of the Reed–Muller expansion theorem.

- [5] D. L. Dietmeyer, *Logic Design of Digital Systems*, 2nd ed. Allyn and Bacon, Boston, MA, 1978.
- [6] E. J. McCluskey, *Logic Design Principles*. Prentice-Hall, Englewood Cliffs, NJ, 1986.

A more formal treatment of XOR algebra can be found in the work of Fisher.

- [7] L. T. Fisher, “Unateness Properties of AND-EXCLUSIVE OR,” *IEEE Trans. on Computers* **C-23**, 166–172 (1974).

A brief history of Boolean algebra is provided in Chapter 2 of Hill and Peterson.

- [8] F. J. Hill and G. R. Peterson, *Digital Logic and Microprocessors*, John Wiley, NY, 1984.

CMOS logic, which is emphasized in this text, is adequately covered by Weste and Eshraghian in Chapter 1 and portions of Chapter 5. But an excellent coverage of experimental work on various XOR and EQV gates on the MOS transistor level is given by Wang et al.

- [9] N. H. E. Weste and K. Eshraghian, *Principles of CMOS VLSI Design*, Addison-Wesley, Reading, MA, 1985.
- [10] J. Wang, S. Fang, and W. Feng, “New Efficient Designs for EXOR and XNOR Functions on the Transistor Level,” *IEEE Journal of Solid-State Circuits* **29**(7), 780–786 (1994).

## PROBLEMS

### 3.1 Define the following:

- (a) Mixed logic
- (b) Polarized mnemonic
- (c) Logic level conversion
- (d) Active and inactive states
- (e) Inverter
- (f) Gate

### 3.2 Identify the gate appropriate to each of the physical truth tables in Fig. P3.1. Note: It may be necessary to search this chapter for the answers.

| A  | B  | Y  | A  | B  | Y  | A  | B  | Y  | A  | B  | Y  | A  | B  | Y  |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| LV | LV | HV | LV | LV | LV | LV | LV | LV | LV | LV | HV | LV | LV | LV |
| LV | HV | HV | LV | HV | HV | LV | HV | LV | LV | HV | LV | LV | HV | HV |
| HV | LV | HV | HV | LV | HV | HV | LV | LV | HV | LV | LV | HV | LV | HV |
| HV | HV | LV | HV | HV | HV | HV | HV | HV | HV | HV | LV | HV | HV | LV |

(a)                      (b)                      (c)                      (d)                      (e)

FIGURE P3.1

- 3.3** By using a sketch, indicate how the CMOS inverter of Fig. 3.6a can be converted to a two-transistor noninverting switch. What would be the disadvantage (if any) of such a device?
- 3.4** Given the waveforms from the two logic devices in Fig. P3.2, sketch the waveforms for  $X$ (voltage),  $X$ (L),  $Y$ (H), and  $Y$ (voltage). Keep the logic and voltage levels as shown.
- 3.5** With reference to Problem 3.4, explain the differences between the logic levels for the  $X$ (H) and  $X$ (L) waveforms and those for the  $Y$ (H) and  $Y$ (L) waveforms. Do these differences represent a contradiction in the definition of positive and negative logic? Explain.
- 3.6** Use the inverter, its I/O behavior, and the logic circuit symbols in Fig. 3.6 to explain how the PMOS indicator bubble in the inverter circuit is related to the active low indicator bubble appearing on the inverter symbols.

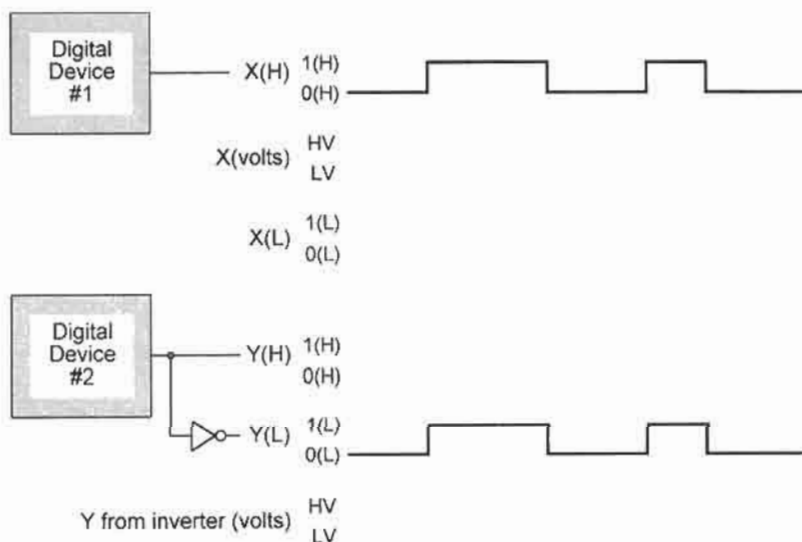


FIGURE P3.2

- 3.7** Use the definitions of positive and negative logic and Fig. 3.6 as a guide to construct the physical truth table and its two mixed logic truth tables for each of the tri-state drivers in Fig. 3.8 as listed below. Remember to include both inputs  $X_i$  and the control  $C$ . Use the letter  $D$  to represent the disconnect mode.
- (a) Noninverting tri-state driver with  $C(H)$ .
  - (b) Noninverting tri-state driver with  $C(L)$ .
  - (c) Inverting tri-state driver with  $C(H)$ .
  - (d) Inverting tri-state driver with  $C(L)$ .
- 3.8** By adding an inverter to each, reconfigure the tri-state drivers in Figs. 3.8c and 3.8d so that they become noninverting tri-state enable switches with driver capability. Give the circuit symbols and ideal equivalent circuits for these two reconfigured tri-state switches.
- 3.9** Reconfigure the NAND gate in Fig. 3.10a by flipping it end-for-end such that the two series NMOS are on the  $+V_{DD}$  (HV) end and the two parallel PMOS on the ground (LV) end.
- (a) Construct the physical and mixed logic truth tables for this reconfigured CMOS circuit. Is this a valid gate form and, if so, what logic function does it perform? (Hint: Compare with Fig. 3.16.)
  - (b) What, if any, are the disadvantages of this new configuration? Explain.
- 3.10** Repeat Problem 3.9 for the NOR gate in Fig. 3.12a, but with the two parallel NMOS on the HV end and the series PMOS on the LV end. (Hint: Compare with Fig. 3.17.)
- 3.11** Explain why the AND and OR gates of Figs. 3.16 and 3.17 cannot be used for logic level conversion as is done for the NAND and NOR gates of Figs. 3.14 and 3.15.
- 3.12** Write the logic expressions for the action indicated by the situations given below. Use mnemonics or abbreviations where appropriate.
- (a) Bob will go fishing in a boat only if the boat does not leak and if it is not windy. Otherwise, he will fish from the bank, but only if the fish are biting.
  - (b) A laboratory class consists of five students (A, B, C, D, and E) each from a different discipline. An experiment has been assigned that must be carried out with any one of the following combinations of students:
    - A and C but not D
    - A or B but not both (see Section 3.9)
    - D but only if E is present
  - (c) A robot is activated only if a majority of its three switches (X, Y, and Z) are turned ON and is deactivated if a majority of its three switches are turned OFF.
- 3.13** Archie (A), Betty (B), Cathy (C), and David (D) may attend a school dance, but will dance only with the opposite sex and then only under the following conditions: Archie will dance with either Betty or Cathy. However, Cathy will dance with Archie only

if Betty is not present at the dance. David will dance only with Betty. Obtain the logic expression representing the active state of dancing for A, B, C, and D.

- 3.14** Use a minimum number of gates and inverters to implement the functions below with NAND/INV logic. Give the gate/input tally for each logic circuit, excluding inverters. Implement the function exactly as presented — make no alterations. Assume that all inputs arrive from positive logic sources. Use the inverters for logic level conversion.

(a)  $Z(H) = (X\bar{Y} + W)(H)$

(b)  $F(H) = [\bar{A}\bar{D} + (B + \bar{E})](H)$

(c)  $g(L) = (\bar{w}\bar{y} + x + z)(L)$

(d)  $G(L) = [(AB + \bar{C})(\bar{D} + \bar{E})](L)$

(e)  $Y(H) = [(a + \bar{b}c) \cdot (\bar{d} + e)](H)$

- 3.15** Repeat Problem 3.14 by using NOR/INV logic. Assume that all inputs arrive from positive logic sources.

- 3.16** Repeat Problem 3.14 by using AND/OR/INV logic. Assume that all inputs arrive from positive logic sources.

- 3.17** Use three NOR gates (nothing else) to implement the function  $Y(H)$  below exactly as written. Assume the inputs arrive as follow:  $A(H)$ ,  $B(H)$ ,  $C(H)$ ,  $D(L)$ , and  $E(L)$ .

$$Y(H) = [(\bar{A}\bar{D}) \cdot (B + C + \bar{E})](H)$$

- 3.18** Use three NAND gates (nothing else) to implement the function  $Z(H)$  below exactly as written. Assume the inputs arrive as follow:  $A(H)$ ,  $B(H)$ ,  $C(H)$ ,  $D(L)$ , and  $E(L)$ .

$$Z(H) = [(\bar{A} + D) + (BC\bar{E})](H)$$

- 3.19** Name the gates used in each of the logic circuits shown in Fig. P3.3 and give the mixed logic expression at each node in mixed logic notation. Use Figs. 3.20, 3.23, and 3.24 as a guide if needed.

- 3.20** The CMOS circuits in Fig. P3.4 perform specific logic functions. Construct the physical and mixed logic truth tables for each circuit, indicate what logic function it performs and give its two conjugate logic circuit symbols. Note that  $\bar{B}$  is the inverse voltage of  $B$ .

- 3.21** Use two NOR gates and one XOR gate (nothing else) to implement the function  $Y(H)$  below exactly as written. Assume the inputs arrive as  $A(H)$ ,  $B(H)$ ,  $C(H)$ ,  $D(L)$ , and  $E(L)$ .

$$Y(H) = [(A \oplus D) \cdot (B + C + \bar{E})](H)$$

- 3.22** Use three NAND gates, one EQV gate, and one inverter (nothing else) to implement the function  $G(H)$  below exactly as written. Assume the inputs all arrive from positive

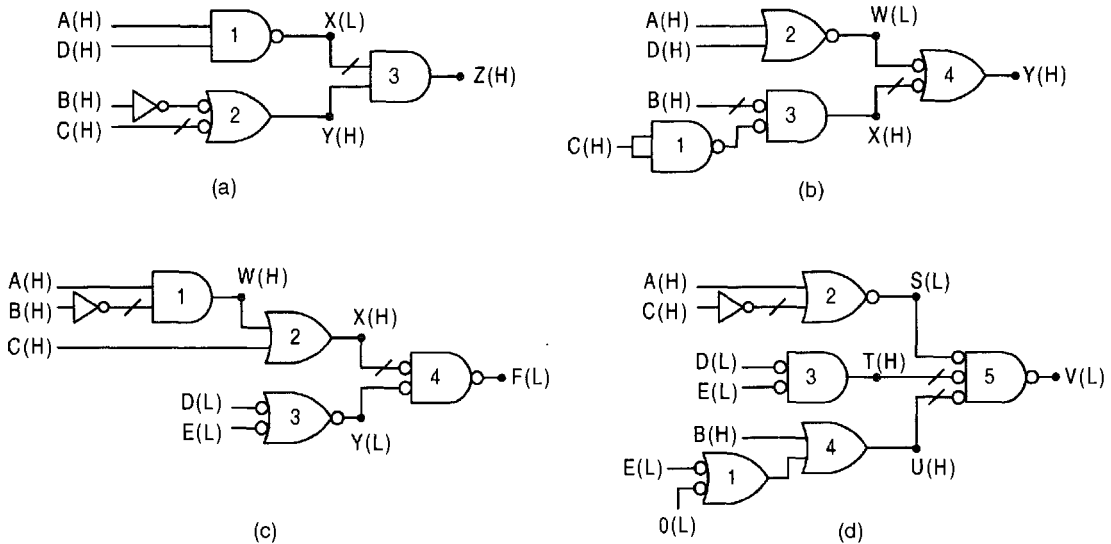


FIGURE P3.3

logic sources.

$$G(H) = [(X \bar{Y}) \oplus \bar{Z} + XYZ](H)$$

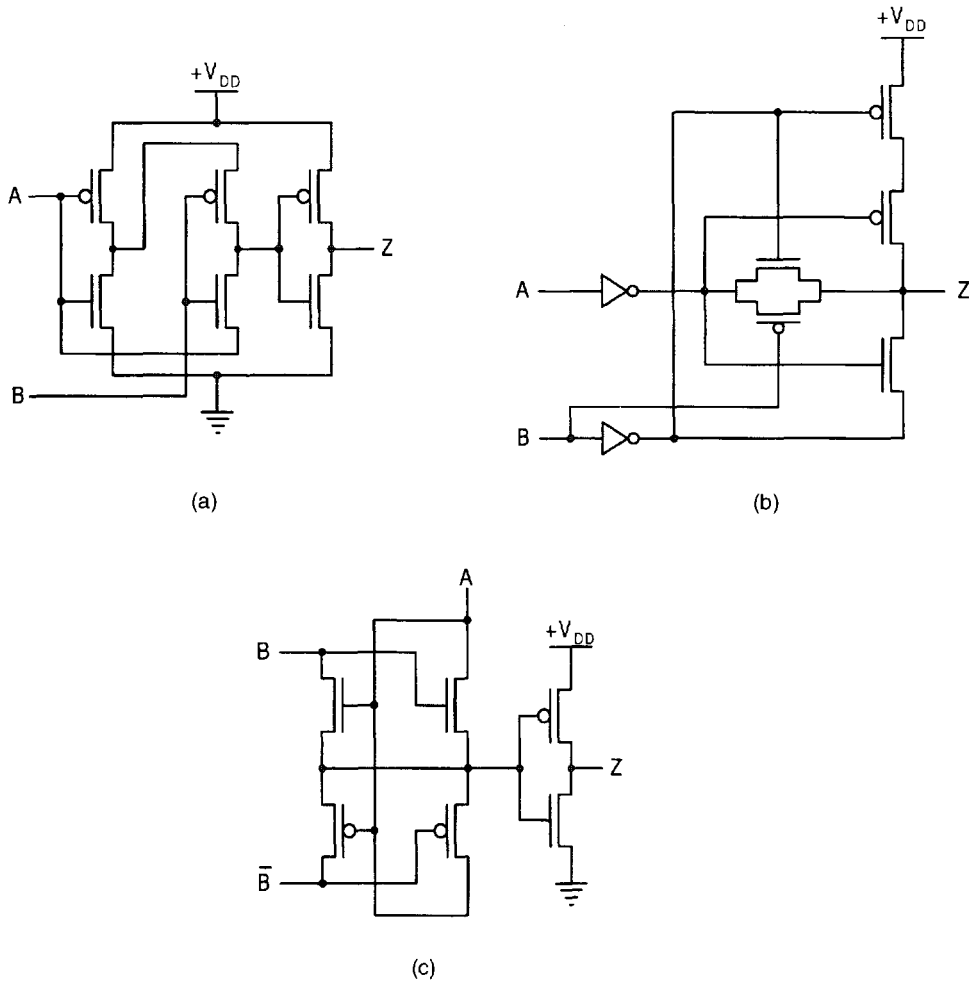
- 3.23** Use three NAND gates and two EQV gates (nothing else) to implement the function  $F(H)$  below exactly as written. Assume the inputs arrive as  $W(L)$ ,  $X(L)$ ,  $Y(H)$ , and  $Z(H)$ .

$$F(H) = [(\bar{W} \oplus Y) \oplus (\bar{X}Z) + \bar{W}Y](H)$$

- 3.24** Unused inputs must not be left dangling. Instead, they must be tied to other inputs, or be connected to HV or LV depending on the logic operations involved. Implement the following functions with the logic indicated.

- A four-input NOR gate performing the  $(A\bar{B})(H)$  operation with inputs  $A(L)$  and  $B(H)$ .
- A three-input NAND gate performing the  $X(H) \rightarrow X(L)$  logic level conversion operation.
- An XOR gate performing the controlled  $X(L) \rightarrow X(H)$  logic level conversion operation.
- A four-input AND gate performing the  $(\bar{A} + B)(L)$  operation with inputs  $A(H)$  and  $B(L)$ .

- 3.25** Construct the truth table and the mixed logic waveforms for the functions below by using a binary input sequence in alphabetical order, all inputs active high. Use Table 2.1 in Section 2.3 if needed, and follow the format of Fig. 3.32 in constructing the waveforms. Do not take into account the propagation delays through gates and inverters.

**FIGURE P3.4**

- (a) Function  $Z(H)$  in Fig. P3.3a.  
 (b) Function  $G(H)$  in Problem 3.22.  
 (c) Function  $F(H)$  of Problem 3.23.

**3.26** Reduce the following expressions to their simplest form and name the Boolean laws used in each step. (Note: Follow the procedure of Examples 3.6 in Section 3.12.)

- (a)  $ab(c + \bar{b}) + a\bar{b}$   
 (b)  $(X + \bar{Y})(XZ + \bar{Y})$  (Hint: First use the distributive law.)  
 (c)  $A + \overline{A\bar{C}} + B$   
 (d)  $(x + y)(x + \bar{z})[y(x + \bar{z}) + \bar{y}]$   
 (e)  $AB + \bar{A}CD + BC + \bar{A}C$

**3.27** Reduce the following expressions to their simplest form, but *do not* name the Boolean laws used in each step.

- (a)  $\bar{A} + AB\bar{C} + \overline{\bar{A} + C}$
- (b)  $(a\bar{c}d + a\bar{d})(a\bar{d} + \bar{c}d)$
- (c)  $(WX + \bar{Y} + \bar{W})(\bar{W}X + \bar{Y} + WX)$
- (d)  $(x + y)(\bar{x} + z)(y + z)$
- (e)  $\overline{(A + \bar{B}\bar{C})(A\bar{B} + \bar{A}\bar{B}\bar{C})}$  (Hint: Simplify under short complementation bars first.)
- (f)  $\bar{a} + b + a(\bar{b} + bc) + (b + \bar{c}) \cdot ab\bar{c}d$
- (g)  $(\bar{A} + B + \bar{C} + D)(\bar{A} + \bar{C} + D)(\bar{A} + \bar{B} + D)$  (Hint: First use consensus.)

**3.28** Reduce the following expressions to their simplest form and name the Boolean laws used in each step.

- (a)  $\overline{(a \oplus b + \bar{b})}(a + b)$
- (b)  $(XY) \oplus (X + \bar{Y})$
- (c)  $x \odot y \odot (xy)$
- (d)  $[(X + Y) \odot (\bar{X} + Z)] + X$
- (e)  $[(\bar{A} + B) \cdot \bar{C}] \oplus [\bar{A} + B + AC]$  (Hint: Find a way to use Corollary II.)

**3.29** Reduce the following expressions to their simplest form, but *do not* name the Boolean laws used in each step.

- (a)  $\bar{A} + A \oplus B + \bar{A}B$
- (b)  $\{\bar{S} + [S \oplus (ST)]\}(H) = [?](L)$
- (c)  $(X + Y) \odot (X \oplus Y)$
- (d)  $(a \odot b) \oplus (a\bar{b})$
- (e)  $(\bar{x} + \bar{y})(\bar{x} \oplus y + \bar{y})$
- (f)  $[1 \oplus (\bar{1} + \bar{0}.1) + 1 \odot 0](H) = [?](L)$

**3.30** Use the laws of Boolean algebra, including the XOR laws, identities, and corollaries given by Eqs. (3.17) through (3.33), to prove whether the following equations are true (T) or false (F). *Do not* name the laws used.

- (a)  $X \odot (\bar{X} + Y) = X\bar{Y}$
- (b)  $ab(\bar{b} + bc) + b\bar{c} + ab\bar{c}d = bc$
- (c)  $\bar{A} \oplus \bar{B} \oplus (A\bar{B}) = \bar{A}B$
- (d)  $X \oplus (\bar{X}Y) = X + (\bar{X} \odot Y)$
- (e)  $[(A\bar{B})(A \odot B)](L) = \bar{A}B(H)$
- (f)  $AX\bar{Y} + A\bar{X}Y + \bar{A}Y = (AX) \oplus Y$  (Hint: First apply Corollary I.)

**3.31** Use whatever conjugate gate forms are necessary to obtain a gate-minimum implementation of the following functions exactly as written (do not alter the functions):

- (a)  $F(H) = \{[A \oplus B] \cdot [(B\bar{C}) \odot D]\}(H)$  with inputs as  $A(H)$ ,  $B(H)$ ,  $C(L)$ , and  $D(L)$ .
- (b)  $K(L) = [A \oplus C \oplus (\bar{B}\bar{D}) \oplus (\bar{A}\bar{B}\bar{C}\bar{D})](L)$  with inputs from negative logic sources.



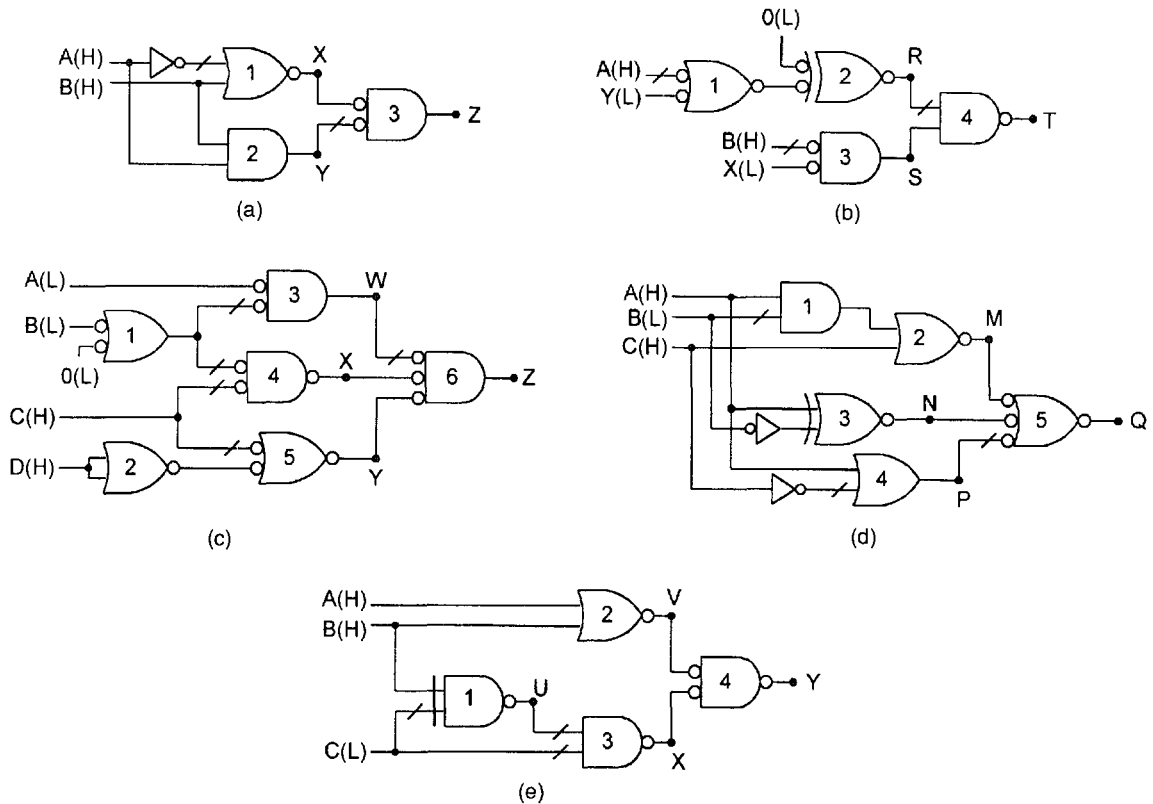


FIGURE P3.5

**3.32** Use NOR/XOR/INV logic to implement the function below exactly as written by using the fewest number of gates and inverters possible, assuming the inputs  $A$  and  $B$  arrive active low and inputs  $X$  and  $Y$  arrive active high.

$$Z(H) = \{[\bar{X} \odot (\bar{A} + Y)] \cdot \bar{B}\}(H)$$

**3.33** A room has two doors and a light that is controlled by three switches,  $A$ ,  $B$ , and  $C$ . There is a switch beside each door and a third switch in another room. The light is turned on (LTON) any time an odd number of switches are closed (active). Find the function LTON( $H$ ) and implement it with a gate-minimum circuit. Assume that the inputs are all active high.

**3.34** The logic circuits shown in Fig. P3.5 are redundant circuits, meaning that they contain more logic than is necessary to implement the output function. Identify each numbered gate and give the *simplest* mixed logic result at each node indicated. To do this, it will be necessary to use the various laws of Boolean algebra together with mixed logic notation.

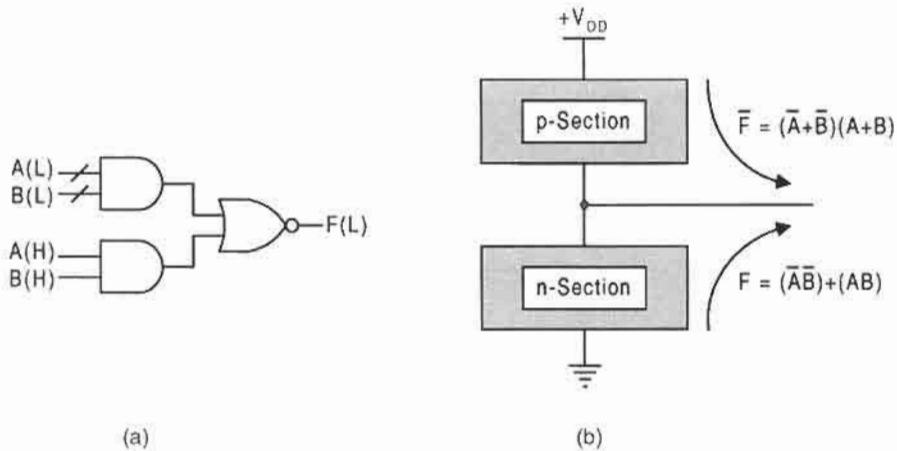


FIGURE P3.6

- 3.35** By following Subsection 3.10.2, write the dual forms for the functions  $Y(H)$ ,  $G(H)$ , and  $Z(H)$  in Problems 3.21, 3.22, and 3.32.
- 3.36** Use the laws of XOR algebra and identities given in Eqs. (3.17) through (3.33) to reduce the following function to its simplest (gate-minimum) form:

$$F = D \oplus B \oplus BD \oplus BCD \oplus A \oplus AD \oplus AC \oplus ACD \oplus AB.$$

- 3.37** The mixed logic circuit for the multiple gate realization of the XOR function  $F(L) = (\bar{A}\bar{B} + AB)(L) = (A \odot B)(L) = (A \oplus B)(H)$  is shown in Fig. P3.6a, together with its CMOS organization in Fig. P3.6b. It derives from the defining relations given by Eqs. (3.4) and (3.5). Construct the CMOS circuit (excluding inverters) for this function by using the proper placement of the PMOS and NMOS as indicated in Figs. 3.5 and P3.6b. Also, construct the physical and logic truth tables for this function.
- 3.38** The logic circuit for the function  $Y(H) = [A(B + CD)](H)$  is given in Fig. P3.7.
- Assuming that inputs  $A$ ,  $B$ ,  $C$ , and  $D$  arrive from positive logic sources, construct the CMOS circuit for the function  $Y(H)$ .
  - Obtain the physical and positive logic truth table for this function.
- 3.39** Shown in Fig. P3.8 is a CMOS circuit having three inputs and one output.

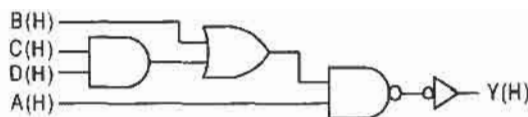


FIGURE P3.7

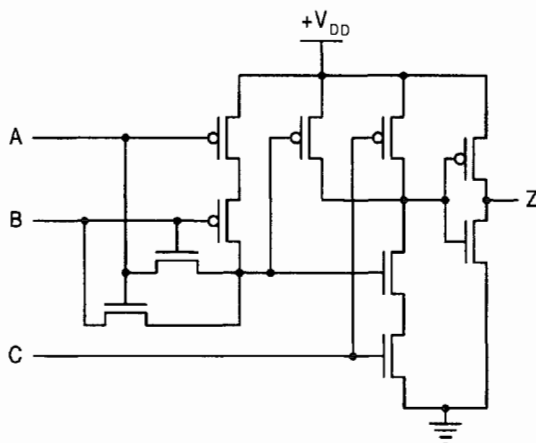


FIGURE P3.8

- Construct the physical truth table for this circuit taking into account all possible combinations of LV and HV inputs.
- If the inputs and outputs are all assumed to be active high, find the logic function for  $Z(H)$  and its logic circuit.

**3.40** The CMOS circuit in Fig. P3.9 is an example of a *gate-matrix layout*. The circuit has four inputs,  $A$ ,  $B$ ,  $C$ , and  $D$ , and one output  $Z$ . Note that  $X$  indicates an internal connection.

- Construct the physical truth table for this circuit taking into account all possible combinations of LV and HV inputs.
- If the inputs and outputs are all assumed to be active high, find the logic function for  $Z(H)$  and construct the logic circuit for  $Z(H)$ .

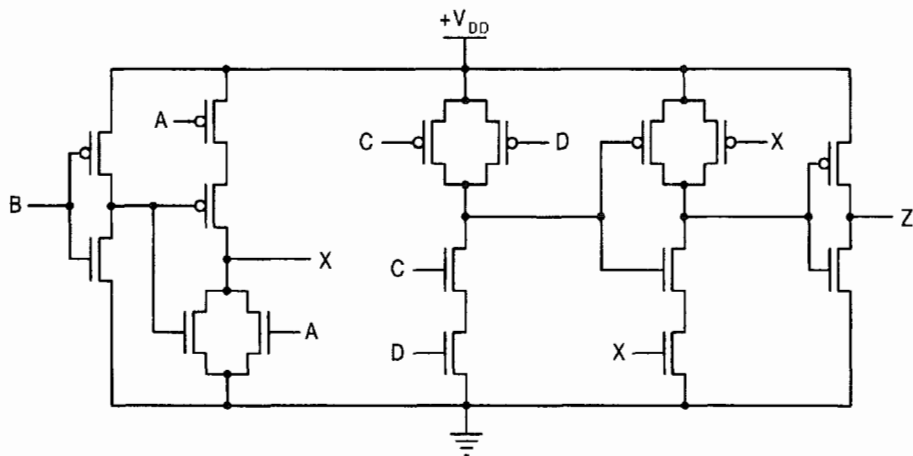


FIGURE P3.9

# Logic Function Representation and Minimization

---

## 4.1 INTRODUCTION

A given logic function can be represented in a variety of different forms, and often one of these forms proves to be the best for the application under consideration. It is the purpose of this chapter to consider the different forms of logic representation. It is also the purpose of this chapter to consider the reduction and minimization of these different forms. Knowing how to reduce or minimize a logic function is important so as to reduce design area, power consumption, and cost by eliminating unnecessary hardware. Also, the minimized function often reveals information that is not readily apparent from a nonminimized form. In short, the information in this chapter is essential to good design practices and specifically to an understanding of the remainder of this text.

## 4.2 SOP AND POS FORMS

Without specific mention of it, SOP and POS forms have already been used in the discussions of Chapter 3. Typical examples are the defining relations for XOR and EQV given by Eqs. (3.4) and (3.5) in Section 3.9, where each is given in both SOP and POS form. To understand what is meant by SOP and POS, the AND operation is taken as the *Boolean product* and the OR operation represents the *Boolean sum*. Thus, SOP means *sum-of-products* while POS denotes *product-of-sums*. These definitions will be used throughout the text.

### 4.2.1 The SOP Representation

Consider the function of three variables given by the Boolean expression

$$f(A, B, C) = \bar{A}B + BC + \underbrace{A\bar{B}\bar{C}}_{\text{minterm}}. \quad (4.1)$$

The function in Eq. (4.1) is written in *sum-of-products* (SOP) form, meaning ORing of ANDed terms also called *p-terms* (product-terms). Although there are three p-terms in this expression, only the term  $A\bar{B}\bar{C}$  is called a *minterm*. A minterm is defined as follows:

**Minterm:** Any ANDed term containing all the variables of a function in complemented or uncomplemented form.

Use will be made of the symbol

$$m_i = m_i(A, B, C, \dots) \quad (4.2)$$

to represent the *i*th minterm of a function. Notice that two of the three p-terms in Eq. (4.1) cannot be minterms by this definition.

To simplify minterm representation, a shorthand notation is used and is based on the following *minterm code*:

**MINTERM CODE**

Complmented variables: logic 0  
Uncomplemented variables: logic 1

Once the logic 0's and 1's have been assigned to all variables in a given minterm, a minterm code is established where the subscript in  $m_i$  becomes the decimal equivalent of the binary code formed by the logic state assignments. For example, the minterm in Eq. (4.1) is represented by

$$\begin{array}{c} A\bar{B}\bar{C} = m_4, \\ 100 \end{array}$$

since the binary of 100 has a decimal value of 4. A complete minterm code table for four variables is given in Fig. 4.1. A similar minterm code table can be constructed for any number of variables.

A function composed completely of a logical sum of minterms is said to be in *canonical SOP form*. A typical example is given by the following expressions, where use has been made of the minterm code shorthand notation and the operator symbol  $\sum$  to represent the logical sum of minterms:

$$\begin{aligned} Y(A, B, C) &= \underbrace{\bar{A}\bar{B}\bar{C}}_{000} + \underbrace{\bar{A}BC}_{011} + \underbrace{ABC}_{111} + \underbrace{A\bar{B}\bar{C}}_{100} + \underbrace{ABC}_{110} \\ &= m_0 + m_3 + m_7 + m_4 + m_6 \\ &= \sum m(0, 3, 4, 6, 7). \end{aligned}$$

A reduced SOP function such as that in Eq. (4.1) can be expanded to canonical form by applying the factoring law and the AND and OR laws given in Section 3.10. This is

| SOP Term                          | Binary | Decimal | $m_i$ | SOP Term                    | Binary | Decimal | $m_i$    |
|-----------------------------------|--------|---------|-------|-----------------------------|--------|---------|----------|
| $\bar{A} \bar{B} \bar{C} \bar{D}$ | 0000   | 0       | $m_0$ | $A \bar{B} \bar{C} \bar{D}$ | 1000   | 8       | $m_8$    |
| $\bar{A} \bar{B} \bar{C} D$       | 0001   | 1       | $m_1$ | $A \bar{B} \bar{C} D$       | 1001   | 9       | $m_9$    |
| $\bar{A} \bar{B} C \bar{D}$       | 0010   | 2       | $m_2$ | $A \bar{B} C \bar{D}$       | 1010   | 10      | $m_{10}$ |
| $\bar{A} \bar{B} C D$             | 0011   | 3       | $m_3$ | $A \bar{B} C D$             | 1011   | 11      | $m_{11}$ |
| $\bar{A} B \bar{C} \bar{D}$       | 0100   | 4       | $m_4$ | $A B \bar{C} \bar{D}$       | 1100   | 12      | $m_{12}$ |
| $\bar{A} B \bar{C} D$             | 0101   | 5       | $m_5$ | $A B \bar{C} D$             | 1101   | 13      | $m_{13}$ |
| $\bar{A} B C \bar{D}$             | 0110   | 6       | $m_6$ | $A B C \bar{D}$             | 1110   | 14      | $m_{14}$ |
| $\bar{A} B C D$                   | 0111   | 7       | $m_7$ | $A B C D$                   | 1111   | 15      | $m_{15}$ |

**FIGURE 4.1**

Minterm code table for four variables.

demonstrated by expanding Eq. (4.1) as follows:

$$\begin{aligned}
 f(A, B, C) &= \bar{A}B + BC + A\bar{B}\bar{C} \\
 &= \bar{A}B(\bar{C} + C) + (\bar{A} + A)BC + A\bar{B}\bar{C} \\
 &= \bar{A}B\bar{C} + \bar{A}BC + \bar{A}B\bar{C} + ABC + A\bar{B}\bar{C} \\
 &= m_2 + m_3 + m_3 + m_7 + m_4 \\
 &= \sum m(2, 3, 4, 7).
 \end{aligned} \tag{4.3}$$

Note that the OR law  $X + \bar{X} = 1$  has been applied twice and that the two identical minterms  $\bar{A}B\bar{C}$  are combined according to the OR law  $X + X = X$ .

The canonical truth table for Eqs. (4.3), shown in Fig. 4.2, is easily constructed from the minterm code form. However, the truth table can also be constructed directly from the original reduced form given by Eqs. (4.1). Notice that a logic 1 is placed in the  $f$  column each time an  $\bar{A}B = 01$  occurs, each time a  $BC$  occurs, and for  $A\bar{B}\bar{C}$ . Thus, construction

| A B C | f       |
|-------|---------|
| 0 0 0 | 0       |
| 0 0 1 | 0       |
| 0 1 0 | 1 $m_2$ |
| 0 1 1 | 1 $m_3$ |
| 1 0 0 | 1 $m_4$ |
| 1 0 1 | 0       |
| 1 1 0 | 0       |
| 1 1 1 | 1 $m_7$ |

**FIGURE 4.2**

Truth table for Eq. (4.3).

of the truth table from a reduced form permits a simple means of obtaining the canonical representation without having to use the Boolean manipulation given by Eqs. (4.3).

### 4.2.2 The POS Representation

An alternative means of representing a logic expression is to cast it in *product-of-sums* (POS) form, meaning the ANDing of ORed terms, also called s-terms (sum-terms). An example of POS representation is given by the function

$$f(A, B, C, D) = (A + \bar{B})(\underbrace{\bar{A} + \bar{B} + C + \bar{D}}_{\text{Maxterm}})(B + \bar{C} + D) \quad (4.4)$$

where, of the three s-terms, only the term  $(\bar{A} + \bar{B} + C + \bar{D})$  is called a *maxterm*.

A maxterm is defined as follows:

**Maxterm:** Any ORed term containing all the variables of a function in complemented or uncomplemented form.

The symbol

$$M_i = M_i(A, B, C, \dots) \quad (4.5)$$

will be used to represent the  $i$ th maxterm of a function.

Maxterm representation can be simplified considerably by using the *maxterm code*:

| <b>MAXTERM CODE</b>      |         |
|--------------------------|---------|
| Complemented variable:   | logic 1 |
| Uncomplemented variable: | logic 0 |

The assignment of the logic 1's and 0's in this manner to all variables in each maxterm establishes the maxterm code, where the subscript in  $M_i$  is the decimal equivalent of the binary number formed by the logic state assignments. The maxterm code table for four variables is given in Fig. 4.3. Use of this table is illustrated by maxterm in Eq. (4.4),

$$\bar{A} + \bar{B} + C + \bar{D} = M_{13}$$

1
1
0
1

where  $1101_2 = 13_{10}$ .

A comparison of the minterm and maxterm code tables in Figs. 4.1 and 4.3 indicates that

$$M_i = \bar{m}_i \quad (4.6)$$

and

$$m_i = \bar{M}_i,$$

| POS Term                    | Binary | Decimal | $M_i$ |  | POS Term                          | Binary | Decimal | $M_i$    |
|-----------------------------|--------|---------|-------|--|-----------------------------------|--------|---------|----------|
| $A+B+C+D$                   | 0000   | 0       | $M_0$ |  | $\bar{A}+B+C+D$                   | 1000   | 8       | $M_8$    |
| $A+B+C+\bar{D}$             | 0001   | 1       | $M_1$ |  | $\bar{A}+B+C+\bar{D}$             | 1001   | 9       | $M_9$    |
| $A+B+\bar{C}+D$             | 0010   | 2       | $M_2$ |  | $\bar{A}+B+\bar{C}+D$             | 1010   | 10      | $M_{10}$ |
| $A+B+\bar{C}+\bar{D}$       | 0011   | 3       | $M_3$ |  | $\bar{A}+B+\bar{C}+\bar{D}$       | 1011   | 11      | $M_{11}$ |
| $A+\bar{B}+C+D$             | 0100   | 4       | $M_4$ |  | $\bar{A}+\bar{B}+C+D$             | 1100   | 12      | $M_{12}$ |
| $A+\bar{B}+C+\bar{D}$       | 0101   | 5       | $M_5$ |  | $\bar{A}+\bar{B}+C+\bar{D}$       | 1101   | 13      | $M_{13}$ |
| $A+\bar{B}+\bar{C}+D$       | 0110   | 6       | $M_6$ |  | $\bar{A}+\bar{B}+\bar{C}+D$       | 1110   | 14      | $M_{14}$ |
| $A+\bar{B}+\bar{C}+\bar{D}$ | 0111   | 7       | $M_7$ |  | $\bar{A}+\bar{B}+\bar{C}+\bar{D}$ | 1111   | 15      | $M_{15}$ |

**FIGURE 4.3**

Maxterm code table for four variables.

revealing a complementary relationship between minterms and maxterms. The validity of Eqs. (4.6) is easily demonstrated by the following examples:

$$\bar{m}_5 = \overline{ABC} = \bar{A} + B + \bar{C} = M_5$$

and

$$\bar{M}_{12} = \overline{\bar{A} + \bar{B} + C + D} = ABC\bar{D} = m_{12},$$

where use has been made of DeMorgan's laws given by Eqs. (3.15a).

A function whose terms are *all* maxterms is said to be given in *canonical POS form* as indicated next by using maxterm code.

$$\begin{aligned}
 f(A, B, C) &= \underbrace{(A + B + \bar{C})}_{001} \cdot \underbrace{(\bar{A} + B + \bar{C})}_{101} \cdot \underbrace{(\bar{A} + B + C)}_{100} \cdot \underbrace{(A + B + C)}_{000} \\
 &= M_1 \cdot M_5 \cdot M_4 \cdot M_0 \\
 &= \prod M(0, 1, 4, 5)
 \end{aligned}$$

Note that the operator symbol  $\prod$  is used to denote the ANDing (Boolean product) of maxterms  $M_0$ ,  $M_1$ ,  $M_4$ , and  $M_5$ .

Expansion of a reduced POS function to canonical POS form can be accomplished as indicated by the following example:

$$\begin{aligned}
 f(A, B, C) &= (A + \bar{C})(\bar{B} + \bar{C})(\bar{A} + B + C) \\
 &= (A + \bar{B}B + \bar{C})(\bar{A}\bar{A} + \bar{B} + \bar{C})(\bar{A} + B + C) \\
 &= (A + \bar{B} + \bar{C})(A + B + \bar{C})(\bar{A} + \bar{B} + \bar{C})(A + \bar{B} + \bar{C})(\bar{A} + B + C) \\
 &\quad \quad \quad \begin{matrix} M_3 & M_1 & M_7 & M_3 & M_4 \end{matrix} \\
 &= \prod M(1, 3, 4, 7). \tag{4.7}
 \end{aligned}$$

Here, use is made of multiple applications of the distributive, AND, and OR laws in the



| A | B | C | f                   |
|---|---|---|---------------------|
| 0 | 0 | 0 | 1 $m_0$             |
| 0 | 0 | 1 | 0 $\rightarrow M_1$ |
| 0 | 1 | 0 | 1 $m_2$             |
| 0 | 1 | 1 | 0 $\rightarrow M_3$ |
| 1 | 0 | 0 | 0 $\rightarrow M_4$ |
| 1 | 0 | 1 | 1 $m_5$             |
| 1 | 1 | 0 | 1 $m_6$             |
| 1 | 1 | 1 | 0 $\rightarrow M_7$ |

**FIGURE 4.4**

Truth table for Eqs. (4.8).

form of  $(X + Y)(X + \bar{Y}) = X$ . Notice that the AND law  $M_3 \cdot M_3 = M_3$  is applied since this maxterm occurs twice in the canonical expression.

The results expressed by Eq. (4.7) are represented by the truth table in Fig. 4.4, where use is made of both minterm and maxterm codes. Function  $f$  values equal to logic 1 are read as minterms, while function values equal to logic 0 are read as maxterms. From this there emerges the result

$$f(A, B, C) = \sum m(0, 2, 5, 6) = \prod M(1, 3, 4, 7), \quad (4.8)$$

which shows that a given function can be represented in either canonical SOP or canonical POS form. Moreover, this shows that *if one form is known, the other is found simply by using the missing code numbers from the former.*

By applying DeMorgan's laws given by Eqs. (3.15a), it is easily shown that the complement of Eqs. (4.8) is

$$\bar{f}(A, B, C) = \prod M(0, 2, 5, 6) = \sum m(1, 3, 4, 7). \quad (4.9)$$

This follows from the result

$$\begin{aligned} \bar{f} &= \overline{\sum m(0, 2, 5, 6)} = \overline{m_0 + m_2 + m_5 + m_6} \\ &= \bar{m}_0 \cdot \bar{m}_2 \cdot \bar{m}_5 \cdot \bar{m}_6 \\ &= M_0 \cdot M_2 \cdot M_5 \cdot M_6 \\ &= \prod M(0, 2, 5, 6) = \sum m(1, 3, 4, 7) \end{aligned}$$

A similar set of equations exist for  $\bar{f} = \overline{\prod M(1, 3, 4, 7)}$ . Equations (4.8) and (4.9), viewed as a set, illustrate the type of interrelationship that always exists between canonical forms.

There is more information that can be gathered from the interrelationship between canonical forms. By applying the OR law,  $X + \bar{X} = 1$ , and the OR form of the commutative laws

to Eqs. (4.8) and (4.9), there results

$$\begin{aligned} f + \bar{f} &= \sum m(0, 2, 5, 6) + \sum m(1, 3, 4, 7) \\ &= \sum m(0, 1, 2, 3, 4, 5, 6, 7) \\ &= 1. \end{aligned}$$

Generally, the Boolean sum of all  $2^n$  minterms of a function is logic 1 according to

$$\sum_{i=0}^{2^n-1} m_i = 1. \quad (4.10)$$

Similarly, by using the AND law,  $X \cdot \bar{X} = 0$ , and the AND form of the commutative laws, there results

$$\begin{aligned} f \cdot \bar{f} &= \prod M(1, 3, 4, 7) \cdot \prod M(0, 2, 5, 6) \\ &= \prod M(0, 1, 2, 3, 4, 5, 6, 7) \\ &= 0. \end{aligned}$$

Or generally, the Boolean product of all  $2^n$  maxterms of a function is logic 0 according to

$$\prod_{i=0}^{2^n-1} M_i = 0. \quad (4.11)$$

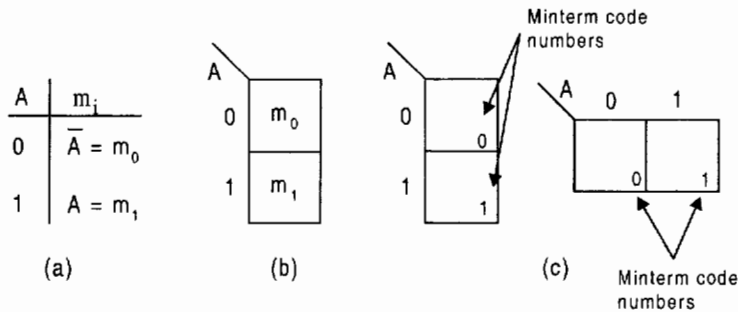
Equations (4.10) and (4.11) are dual relations by the definition of duality given in Subsection 3.10.2.

To summarize, the following may be stated:

*Any function ORed with its complement is logic 1 definite, and any function ANDed with its complement is logic 0 definite — the form of the function is irrelevant.*

### 4.3 INTRODUCTION TO LOGIC FUNCTION GRAPHICS

Graphical representation of logic truth tables are called *Karnaugh maps (K-maps)* after M. Karnaugh, who, in 1953, established the map method for combinational logic circuit synthesis. K-maps are important for the following reasons: (1) K-maps offer a straightforward method of identifying the minterms and maxterms inherent in relatively simple minimized or reduced functions. (2) K-maps provide the designer with a relatively effortless means of function minimization through pattern recognition for relatively simple functions. These two advantages make K-maps extremely useful in logic circuit design. However, it must be pointed out that the K-map method of minimization becomes intractable for very large complex functions. Computer assisted minimization is available for logic systems too complex for K-map use. The following is a systematic development of the K-map methods.

**FIGURE 4.5**

(a) Minterm code table for one variable and (b) its graphical equivalent. (c) Alternative formats for first order K-maps showing minterm positions.

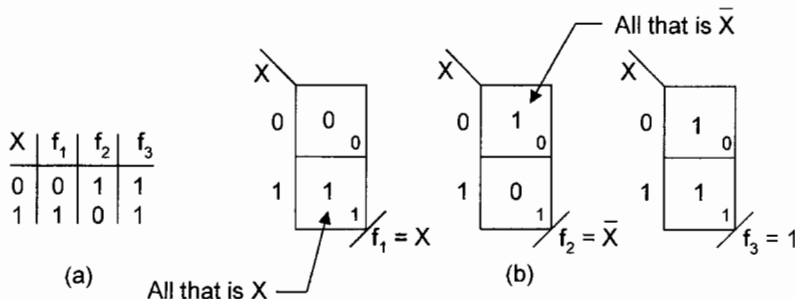
### 4.3.1 First-Order K-maps

A first-order K-map is the graphical representation of a truth table of one variable and is developed from the minterm code table shown in Fig. 4.5a. The minterm positions in a first-order K-map are shown in Fig. 4.5b, leading to the alternative formats for a first-order K-map given in Fig. 4.5c. The number in the lower right-hand corner of a K-map in Fig. 4.5c indicates the position into which a minterm with that code number must be placed.

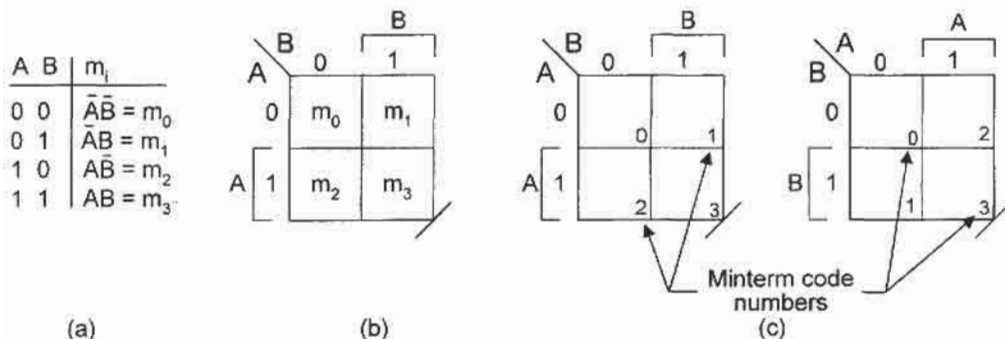
Consider the three functions given by the truth tables in Fig. 4.6a. Notice that all information contained within a given truth table is present in the corresponding K-map in Fig. 4.6b and that the functions are read as  $f_1 = X$ ,  $f_2 = \bar{X}$ , and  $f_3 = 1$  from either the truth tables or K-maps. Thus, a logic 1 indicates presence of a minterm and a logic 0 (the absence of a minterm) is a maxterm.

### 4.3.2 Second-Order K-maps

A second-order K-map is the graphical representation of a truth table for a function of two variables and is developed from the minterm code table for two variables given in Fig. 4.7a. The graphical equivalent of the minterm code table in Fig. 4.7a is given in Fig. 4.7b, where the minterm code decimal for each of four  $m_i$  is the binary equivalent of the *cell coordinates*

**FIGURE 4.6**

(a) Truth table and (b) first order K-maps for functions  $f_1$ ,  $f_2$ , and  $f_3$  of one variable  $X$ .

**FIGURE 4.7**

(a) Minterm code table for two variables and (b) its graphical equivalent. (c) Alternative formats for second-order K-maps showing minterm positions.

(read in alphabetical order  $AB$ ) of the cell into which that minterm is placed. From these figures there result the two alternative K-map formats shown in Fig. 4.7c, where the number in the lower right-hand corner of each cell is the decimal equivalent of the coordinates for that cell given in binary.

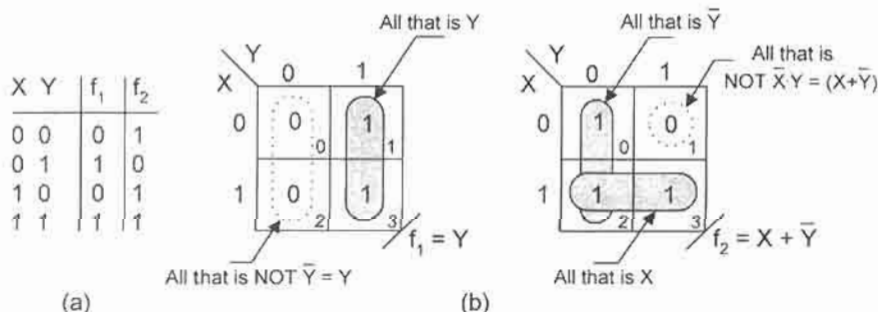
As examples, functions  $f_1$  and  $f_2$  of two variables ( $X$  and  $Y$ ) are represented by truth tables in Fig. 4.8a and by K-maps in Fig. 4.8b. Function  $f_1$  is shown to have two minterms and two maxterms while function  $f_2$  has three minterms and one maxterm. From the truth tables the functions can be read in SOP form as

$$f_1(X, Y) = \sum m(1, 3) = \bar{X}Y + XY = Y \quad (4.12)$$

and

$$f_2(X, Y) = \sum m(0, 2, 3) = \bar{X}\bar{Y} + X\bar{Y} + XY = X + \bar{Y}.$$

However, by combining ("looping out") adjacent minterms these results are immediately obvious as indicated in Fig. 4.8b.

**FIGURE 4.8**

(a) Truth tables for functions  $f_1$  and  $f_2$ . (b) K-maps for functions  $f_1$  and  $f_2$ , showing minimum SOP cover (shaded) and POS cover (dashed loops).

The same results could have been obtained by reading the maxterms in the K-maps of Fig. 4.8b. Thus, in maxterm code the canonical and minimum POS forms become

$$f_1 = \prod M(0, 2) = (X + Y)(\bar{X} + Y) = Y$$

(4.13)

and

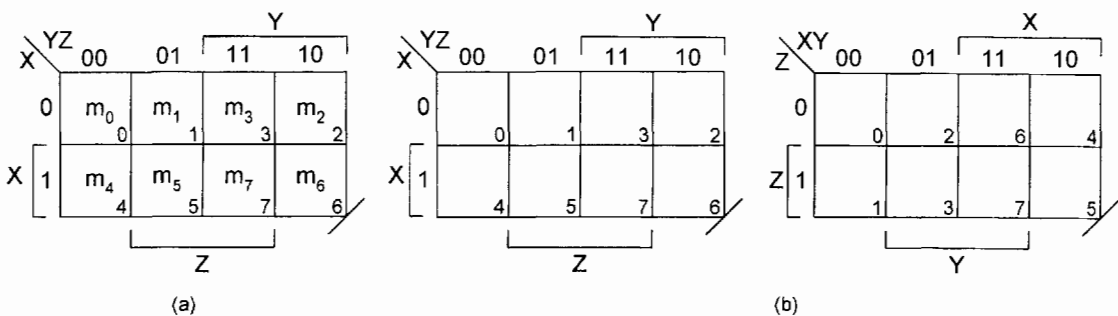
$$f_2 = \prod M(1) = (X + \bar{Y}),$$

where for  $f_1$  the distributive law in Eqs. (3.12) has been applied. Another way of looking at the results given by Eqs. (4.13) is to read groups of adjacent maxterms. For  $f_1$  the two adjacent maxterms (0's) are read as "all that is NOT  $\bar{Y}$ " or  $\bar{Y} = Y$ . Similarly, for function  $f_2$  the 0 in cell 1 is read as "all that is NOT  $\bar{X}Y$ " or simply  $\bar{X}\bar{Y} = X + \bar{Y}$ .

### 4.3.3 Third-Order K-maps

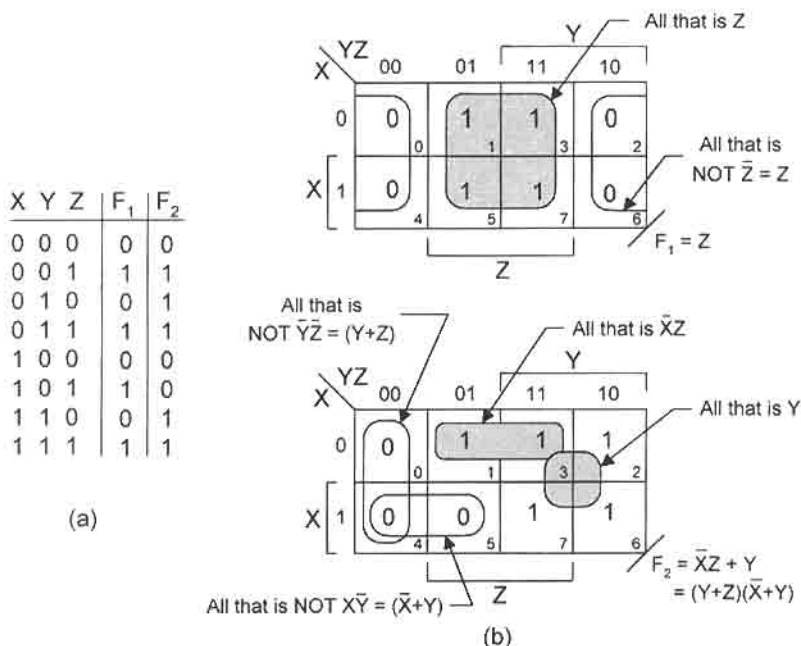
In dealing with functions of three variables, a suitable graphical representation and K-map format must be decided. One choice would be to use a three-dimensional graphic having one axis for each variable. However, such a graphical representation would be difficult to construct and equally difficult to read. A much better choice would be to maintain the domain concept in two dimensions. To do this requires the use of two variables for one axis. Shown in Fig. 4.9a is the graphical representation for the minterm code table of three variables as deduced from Fig. 4.1. Again, the minterm positions are those given by the coordinates of the cells read in alphabetical order  $XYZ$ . From this there results the two alternative formats for a third-order K-map given in Fig. 4.9b, where the minterm code numbers in decimal are shown in the lower right-hand corners of the cells.

Notice that the two-variable axes in the third-order K-maps of Fig. 4.9b are laid out in 2-bit Gray code, a unit distant code featured in Subsection 2.10.3. This is important so that each cell along the two-variable axis is surrounded by logically adjacent cells. The result is that the  $Y$  and  $Z$  domains in Fig. 4.9 are maintained intact. Notice that in Fig. 4.9a the logic adjacency along the two-variable axis is continuous as though the K-map were formed into a cylinder about the  $X$  axis (orthogonal to the  $YZ$  axis). Had the  $YZ$  axis been laid out in binary, the  $Z$  domain would be split into two separate sections, making map plotting and reading difficult. For this reason all axes of two or more variables are laid out in Gray code so as to maximize axis coherency.



**FIGURE 4.9**

(a) Minterm positions and (b) alternative formats for third-order K-maps.

**FIGURE 4.10**

Truth tables for functions  $F_1$  and  $F_2$ . (b) K-map representations for functions  $F_1$  and  $F_2$  showing minimum SOP cover (shaded) and minimum POS cover (unshaded).

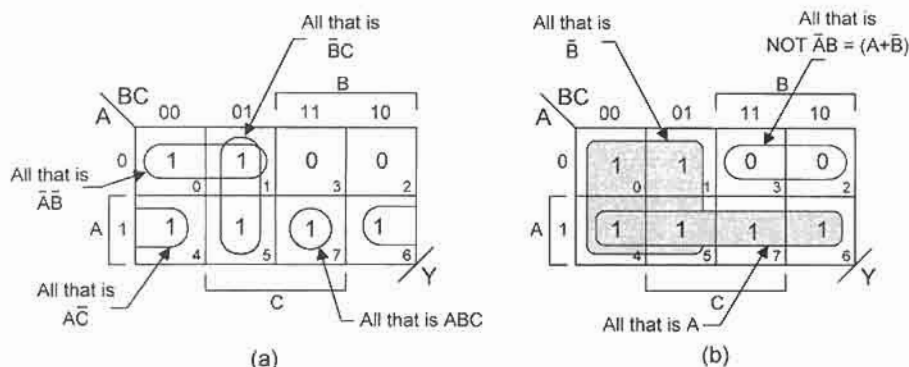
To illustrate the application of third-order K-maps, two simple functions are presented in Fig. 4.10. Here, the truth tables for functions  $F_1$  and  $F_2$  are presented in Fig. 4.10a and their K-map representations together with minimum cover are given in Fig. 4.10b. Notice that the 1's and 0's are placed at the proper coordinates within the K-maps, in agreement with the truth table. From the K-maps, the canonical and minimum SOP forms for functions  $F_1$  and  $F_2$  are read as

$$\begin{aligned}
 F_1(X, Y, Z) &= \sum m(1, 3, 5, 7) = \bar{X}\bar{Y}Z + \bar{X}YZ + X\bar{Y}Z + XYZ \\
 &= Z \\
 F_2(X, Y, Z) &= \sum m(1, 2, 3, 6, 7) = \bar{X}\bar{Y}Z + \bar{X}Y\bar{Z} + \bar{X}YZ + XY\bar{Z} + XYZ \\
 &= \bar{X}Z + Y.
 \end{aligned} \tag{4.14}$$

By grouping minterms in Fig. 4.10b, the minimum SOP expressions,  $F_1 = Z$  and  $F_2 = \bar{X}Z + Y$ , become immediately apparent.

The 0's in the K-maps of Fig. 4.10b can be given in canonical and minimum POS forms:

$$\begin{aligned}
 F_1(X, Y, Z) &= \prod M(0, 2, 4, 6) \\
 &= (X + Y + Z)(X + \bar{Y} + Z)(\bar{X} + Y + Z)(\bar{X} + \bar{Y} + Z) \\
 &= Z \\
 F_2(X, Y, Z) &= \prod M(0, 4, 5) = (X + Y + Z)(\bar{X} + Y + Z)(\bar{X} + Y + \bar{Z}) \\
 &= (Y + Z)(\bar{X} + Y),
 \end{aligned} \tag{4.15}$$

**FIGURE 4.11**

(a) K-map for the reduced function  $Y$  of Eq. (4.16). (b) K-map showing minimum SOP and POS cover for function  $Y$ .

as read in maxterm code. The minimum POS results are easily read from the K-maps of Fig. 4.10b by combining adjacent 0's as indicated. Thus,  $F_1$  is read as "all that is NOT  $\bar{Z}$ " or  $\bar{\bar{Z}} = Z$ . Similarly,  $F_2$  can be read as "all that is NOT  $\bar{Y}\bar{Z} + X\bar{Y}$ " or  $\bar{\bar{Y}\bar{Z} + X\bar{Y}} = (Y + Z)(\bar{X} + Y)$ . Notice that the distributive law in Eqs. (3.12) is applied as  $(Y + Z)(\bar{X} + Y) = Y + \bar{X}Z$ , demonstrating that the SOP and POS forms for  $F_2$  are algebraically equal, as they must be. The minimum POS results given by Eqs. (4.15) can also be obtained by applying the Boolean laws given in Section 3.10, but with somewhat more effort. For example,  $F_1$  is minimized to give the result  $Z$  after three applications of the distributive law in Eqs. (3.12) together with the AND and OR laws.

The use of third-order K-maps is further illustrated by placing the reduced SOP function

$$Y = ABC + A\bar{C} + \bar{B}C + \bar{A}\bar{B} \quad (4.16)$$

into the third-order K-map in Fig. 4.11a. Then by grouping adjacent minterms (shaded loops) as in Fig. 4.11b, there results the minimum expression for Eq. (4.16),

$$Y = A + \bar{B}. \quad (4.17)$$

As expected, the same results could have been obtained by grouping the adjacent maxterms (0's) in Fig. 4.11b, which is equivalent to saying "all that is NOT  $\bar{A}B$ " or  $\bar{\bar{A}B} = A + \bar{B}$ .

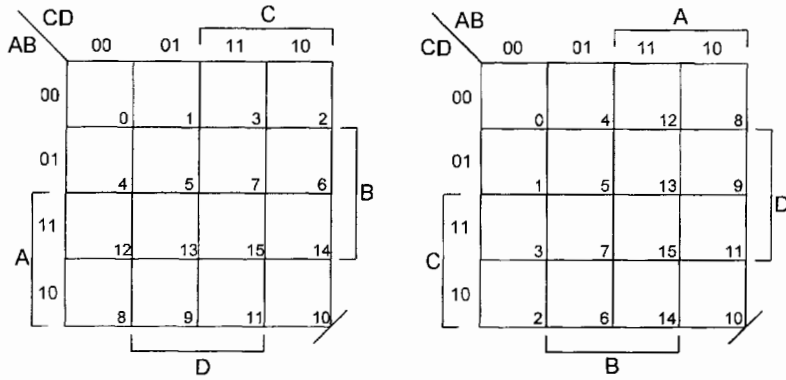
Other information may be gleaned from Fig. 4.11. Extracting canonical information is as easy as reading the minterm code numbers in the lower right-hand corner of each cell. Thus, the canonical SOP and canonical POS forms for function  $Y$  are given by

$$\begin{aligned} Y &= \sum m(0, 1, 4, 5, 6, 7) \\ &= \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + A\bar{B}\bar{C} + A\bar{B}C + ABC\bar{C} + ABC \end{aligned} \quad (4.18)$$

or

$$\begin{aligned} Y &= \prod M(2, 3) \\ &= (A + \bar{B} + C)(A + \bar{B} + \bar{C}) \end{aligned}$$

as read in minterm code and maxterm code, respectively.

**FIGURE 4.12**

Alternative formats for fourth-order K-maps.

#### 4.3.4 Fourth-Order K-maps

At this point it is expected that the reader is familiar with the formats for first-, second-, and third-order K-maps. Following the same development, two alternative formats for fourth-order K-maps are presented in Fig. 4.12, where use of the minterm code table in Fig. 4.1 is implied and where *A* is the MSB and *D* is the LSB. Here, both two-variable axes have logic coordinates that are unfolded in Gray code order so that all juxtaposed minterms (those separated by any single domain boundary) are logically adjacent. Notice that each cell in the K-maps of Fig. 4.12 has a number assigned to it that is the decimal equivalent of the binary coordinates for that cell (read in the order *ABCD*), and that each cell has four other cells that are logically adjacent to it. For example, cell 5 has cells 1, 4, 7, and 13 logically adjacent to it.

Just as a third-order K-map forms an imaginary cylinder about its single variable axis, a fourth-order K-map whose axes are laid out in Gray code will form an imaginary toroid (doughnut-shaped figure), the result of trying to form two cylinders about perpendicular axes. Thus, cells (0, 8) and (8, 10) and (1, 9) are examples of logically adjacent pairs, while cells (0, 2, 8, 10) and (0, 1, 4, 5) and (3, 7, 11, 15) are examples of logically adjacent groups of four.

To illustrate the application of fourth-order K-maps, consider the reduced SOP function

$$F(A, B, C, D) = \bar{A}\bar{C}\bar{D} + \bar{C}D + \bar{A}\bar{B}C\bar{D} + \bar{B}\bar{C}\bar{D} + \bar{A}BCD \quad (4.19)$$

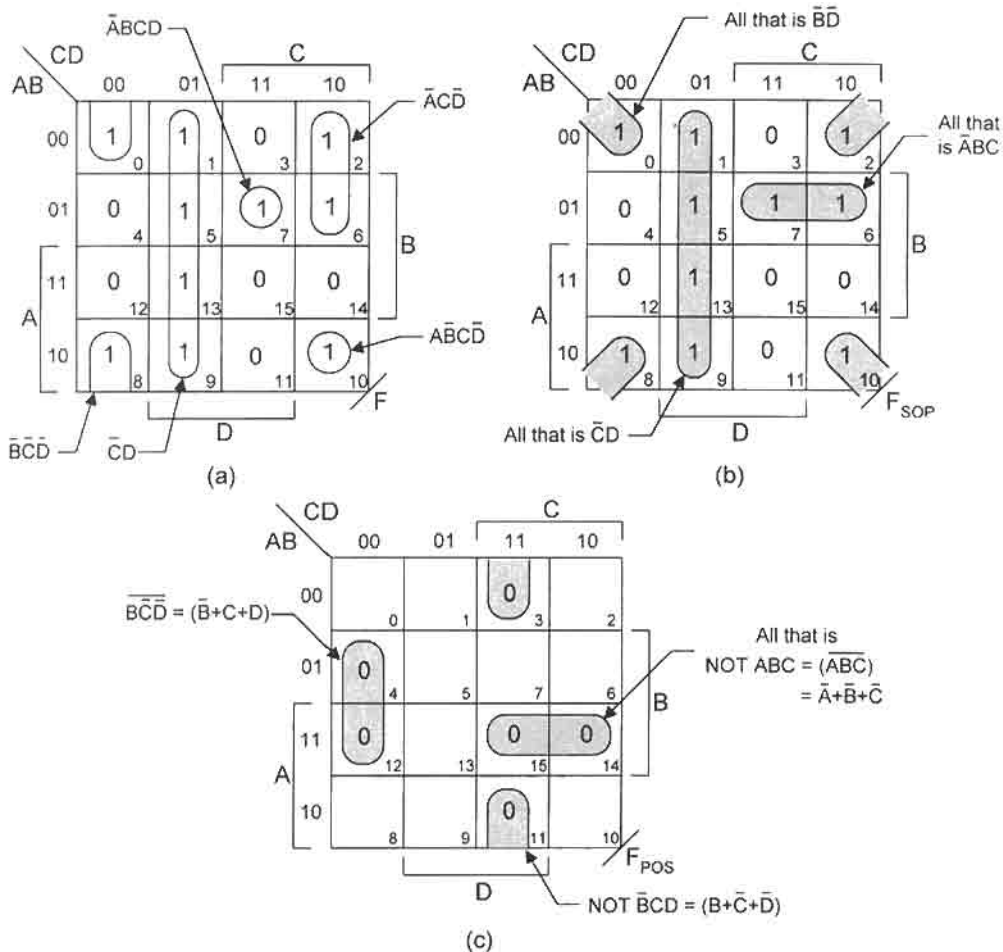
and its K-map representation in Fig. 4.13a. By grouping logically adjacent minterms as in Fig. 4.13b, a minimum SOP result is found to be

$$F_{SOP} = \bar{A}BC + \bar{C}D + \bar{B}\bar{D}. \quad (4.20)$$

Notice that the original function in Eq. (4.19) requires six gates, whereas the minimum result in Eq. (4.20) requires only four gates. In both cases the gate count includes the final ORing operation of the p-terms. The minimum POS cover for function *F* is obtained by grouping the logically adjacent 0's as in Fig. 4.13c, giving

$$F_{POS} = (\bar{B} + C + D)(\bar{A} + \bar{B} + \bar{C})(B + \bar{C} + \bar{D}), \quad (4.21)$$





which is saying “all that is NOT ( $\bar{B}\bar{C}\bar{D} + ABC + \bar{A}BC\bar{D}$ )” as indicated in Fig. 4.13c. The gate tally for Eq. (4.21) is four, including the final ANDing of s-terms, which is less than the original function in Eq. (4.19). Canonical minterm and maxterm representations are easily determined by reading the 1's and 0's in the K-maps of Fig. 4.13 to give

$$\begin{aligned}
 F &= \sum m(0, 1, 2, 5, 6, 7, 8, 9, 10, 13) \\
 &= \prod M(3, 4, 11, 12, 14, 15).
 \end{aligned}
 \tag{4.22}$$

#### 4.4 KARNAUGH MAP FUNCTION MINIMIZATION

Use of the K-map offers a simple and reliable method of minimizing (or at least greatly reducing) logic expressions. In fact, this is the most important application of K-maps. In

Section 4.3, simple examples serve to demonstrate how K-maps can be used to extract both canonical and minimum SOP and POS forms depending on whether 1's or 0's are read. Now it is necessary to present certain important information that was not explicitly stated earlier, but was implied.

It should be clear from Section 4.3 that each line or edge of a K-map forms the boundary between two complementary domains. As a result, minterms or maxterms that are separated by a line or edge are logically adjacent and can be combined to form a reduced function. The following rule generalizes this point:

#### **Reduction Rule**

Each variable domain boundary crossed in an adjacent group (looping) requires the absence of that variable in the reduced term.

Thus, a pair of logically adjacent minterms or maxterms crosses one domain boundary and eliminates the domain variable in the reduced term; a grouping of four logically adjacent minterms or maxterms crosses two domain boundaries and eliminates the two domain variables in the reduced function. In this way  $2^n$  logic adjacencies ( $n = 1, 2, 3, \dots$ ) can be extracted (looped out) to produce a reduced  $(N - n)$ -variable term of an  $N$ -variable function.

To help ensure a minimized result from K-map extraction, thereby avoiding possible costly redundancies, the following loop-out protocol is recommended but not required:

#### **Loop-out Protocol**

Loop out the largest  $2^n$  group of logically adjacent minterms or maxterms in the order of increasing  $n = 0, 1, 2, 3, \dots$

When following this protocol, single isolated minterms or maxterms (*monads*), if present, should be looped out first. This should be followed by looping out groups of any two logically adjacent minterms or maxterms (*dyads or duads*) that cannot be looped out in any other way. The process continues with groups of four logic adjacencies (*quads*), then groups of eight (*octads*), etc. — always in groups of  $2^n$  logic adjacencies.

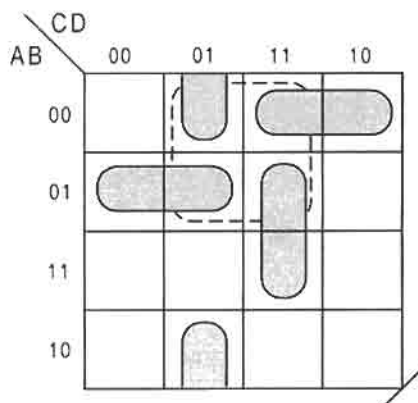
As an example of the pitfalls that can result from failure to follow the loop-out protocol, consider the function represented in the K-map of Fig. 4.14. Instinctively, one may be tempted to loop out the quad (dashed loop) because it is so conspicuous. However, to do so creates a redundancy, since all minterms of that grouping are covered by the four dyads shown by the shaded loops.

Since K-maps are minterm-code based, minimum POS cover can be extracted directly, avoiding the “NOT” step indicated in Figs. 4.8, 4.10, 4.11, and 4.13, by using the following procedure:

#### **Simplified POS Extraction Procedure**

Take the *union* (ORing) of the complemented domains in which the  $2^n$  groups of logically adjacent maxterms exist.

Groups of minterms or maxterms other than  $2^n$  groups (e.g., groups of three, five, six, and seven) are forbidden since such groups are *not* continuously adjacent. Examples of such

**FIGURE 4.14**

Minimum cover (shaded) by using the loop-out protocol avoids the redundant quad (dashed loop).

forbidden groups are presented in Fig. 4.15, which has been “crossed out” to indicate that such groupings are not allowed.

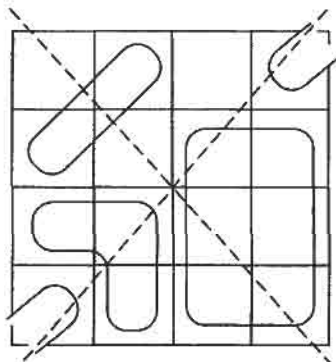
#### 4.4.1 Examples of Function Minimization

Just as canonical forms can be read from a K-map in two ways (SOP and POS), so also can a function be read from a K-map in either minimum SOP form or minimum POS form. To illustrate, consider the function

$$G(A, X, Y) = \sum m(0, 3, 5, 7), \quad (4.23)$$

which is mapped and minimized in Fig. 4.16. Noting that the 1's are looped out as two dyads and a monad as are the 0's, there results

$$G_{SOP} = \bar{A}\bar{X}\bar{Y} + XY + AY \quad (\text{minimum SOP cover})$$

**FIGURE 4.15**

Examples of forbidden (non- $2^n$ ) groupings of minterms and maxterms.

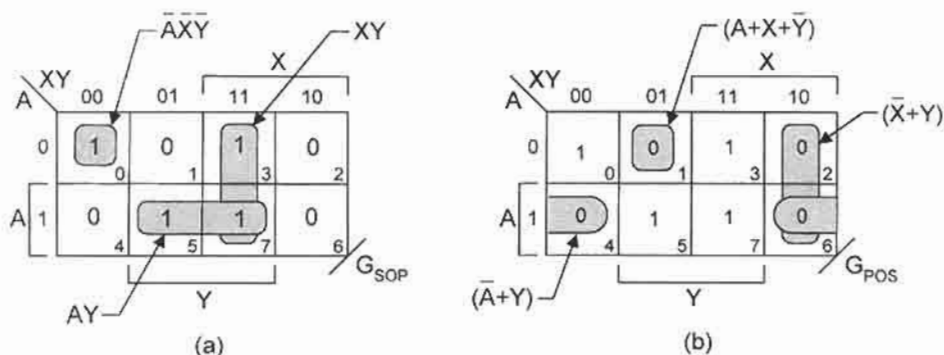


FIGURE 4.16

K-maps for Eq. (4.23). (a) Minimum SOP cover. (b) Minimum POS cover.

and

$$G_{POS} = (A + X + \bar{Y})(\bar{X} + Y)(\bar{A} + Y) \quad (\text{minimum POS cover}).$$

Application of the laws of Boolean algebra shows that the SOP and POS minima are algebraically equal:

$$\begin{aligned} (A + X + \bar{Y})(\bar{X} + Y)(\bar{A} + Y) &= (A + X + \bar{Y})(\bar{A}\bar{X} + \bar{X}Y + \bar{A}Y + Y) \\ &= (A + X + \bar{Y})(\bar{A}\bar{X} + Y) \\ &= \bar{A}\bar{X}\bar{Y} + XY + AY. \end{aligned}$$

As a second example, consider the reduced function given in POS form:

$$Y(A, B, C, D) = (A + \bar{B} + \bar{D})(B + \bar{C})(A + \bar{C} + D)(\bar{A} + \bar{B} + C + \bar{D}). \quad (4.24)$$

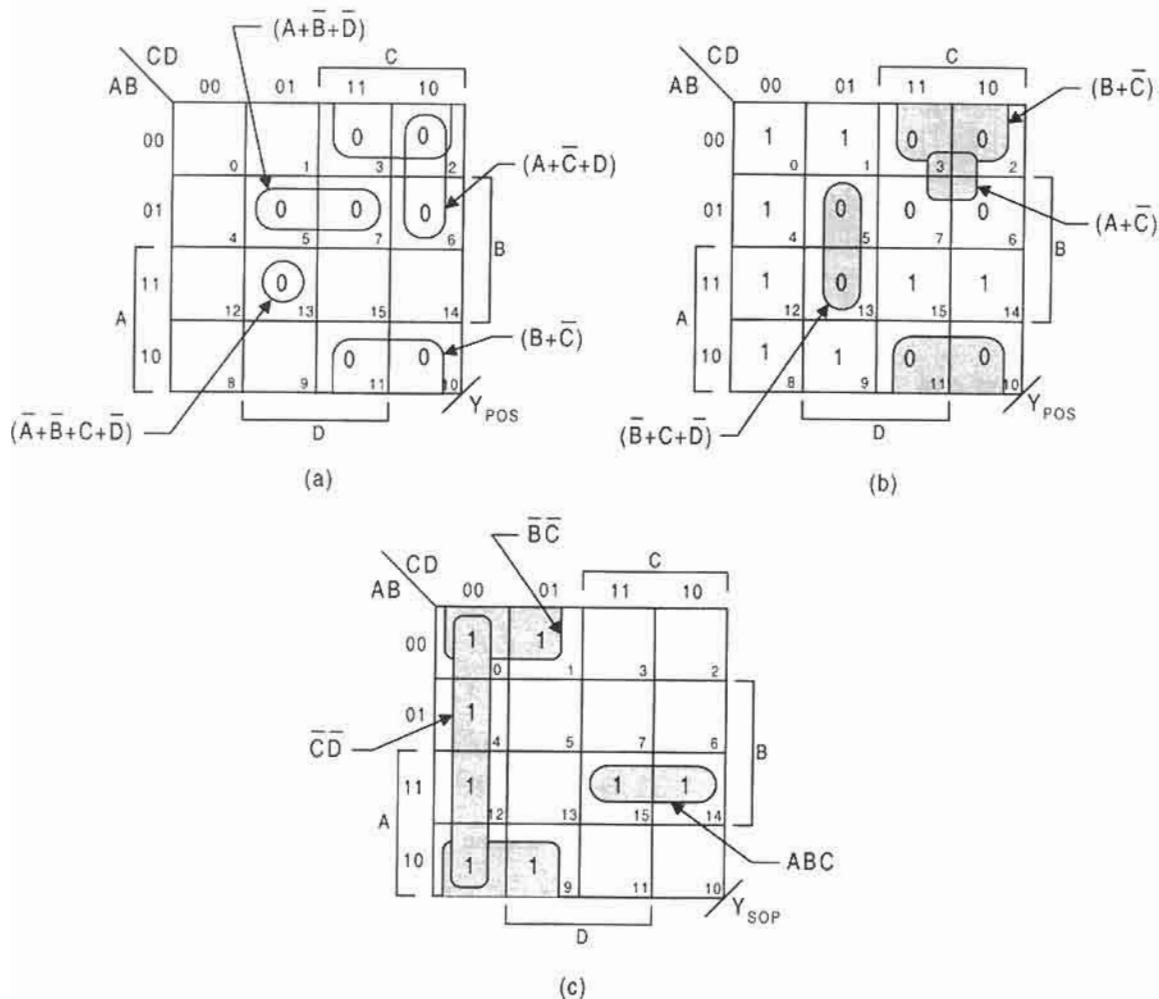
To map this function, one simply maps the 0's in maxterm code, as indicated in Fig. 4.17a. The representation in Fig. 4.17a is not minimum. However, after the maxterms are regrouped, a minimum POS representation is shown in Fig. 4.17b. Notice that the dyad  $M(5, 13)$  crosses the  $A$  boundary, permitting  $(A\bar{A} + \bar{B} + C + \bar{D}) = (\bar{B} + C + \bar{D})$  as the reduced s-term. Similarly, the quad  $M(2, 3, 6, 7)$  crosses the  $B$  and  $D$  boundaries to yield  $(A + B\bar{B} + \bar{C} + D\bar{D}) = (A + \bar{C})$ . Also, the quad  $M(2, 3, 10, 11)$  crosses the  $A$  and  $D$  boundaries, eliminating these variables to give  $(B + \bar{C})$  as the reduced s-term.

The minimum SOP cover for the function  $Y$  of Eq. (4.24) is shown in Fig. 4.17c and consists of one dyad and two quads. The dyad  $m(14, 15)$  crosses the  $D$  boundary, permitting  $ABC(D + \bar{D}) = ABC$ , while the quad  $m(0, 4, 8, 12)$  crosses the  $A$  and  $B$  boundaries, yielding  $(A + \bar{A})(B + \bar{B})\bar{C}\bar{D} = \bar{C}\bar{D}$ . Likewise, the quad  $m(0, 1, 8, 9)$  crosses the  $A$  and  $D$  boundaries to give  $\bar{B}\bar{C}$  as the reduced p-term. The minimized results that are extracted from Figs. 4.17b and 4.17c are now presented as

$$Y_{POS} = (\bar{B} + C + \bar{D})(A + \bar{C})(B + \bar{C}) \quad \text{Minimum POS cover}$$

and

$$Y_{SOP} = ABC + \bar{C}\bar{D} + \bar{B}\bar{C}, \quad \text{Minimum SOP cover}$$

**FIGURE 4.17**

(a) K-map representing the cover as given by Eq. (4.24). (b) Minimum POS cover. (c) Minimum SOP cover.

which are shown to be algebraically equal if one carries out the required algebraic manipulations.

#### 4.4.2 Prime Implicants

Groups of  $2^n$  minterms or maxterms that cannot be combined with other  $2^n$  groups in any way to produce terms of fewer variables are called *prime implicants* (PIs). The loop-out protocol described in the previous section offers a procedure for achieving minimum cover by systematically extracting PIs in the order of increasing  $n$  ( $n = 0, 1, 2, 3, \dots$ ). But the task of achieving minimum cover following the loop-out protocol (or any procedure for that matter) is not quite as straightforward as one might believe. Difficulties can arise when optional and redundant groupings of adjacent minterms or maxterms are present. To deal

with these problems, it will be helpful to identify the following three subsets of PIs:

- *Essential Prime Implicants* (EPIs): Single-way PIs that must be used to achieve minimum cover
- *Optional Prime Implicants* (OPIs): Optional-way PIs that are used for alternative minimum cover
- *Redundant Prime Implicants* (RPIs): Superfluous PIs that cannot be used if minimum cover is to result.

Any grouping of  $2^n$  adjacencies is an *implicant*, including a single minterm or maxterm, but it may not be a PI. For example, a solitary quad EPI contains eight RPIs, four monads, and four dyads, none of which are PIs.

To illustrate a simple mapping problem with optional coverage, consider the function

$$Z(A, B, C, D) = \sum m(2, 4, 6, 8, 9, 10, 11, 15), \quad (4.25)$$

which is mapped in Fig. 4.18a. Noting first the minterm adjacencies that form the three dyads (no monads exist) and the single quad, there results the SOP minimum expression

$$Z_{SOP} = ACD + \bar{A}B\bar{D} + \left\{ \begin{array}{l} \bar{A}C\bar{D} \\ \bar{B}C\bar{D} \end{array} \right\} + A\bar{B}, \quad (4.26)$$

which has three EPI p-terms (two dyads and one quad), and two OPI dyads indicated in braces. The minterm  $m_2$  can be covered in two ways to form the OPI dyads  $m(2, 6)$  and  $m(2, 10)$  shown with dashed loops in Fig. 4.18a. Remember that when one OPI is selected

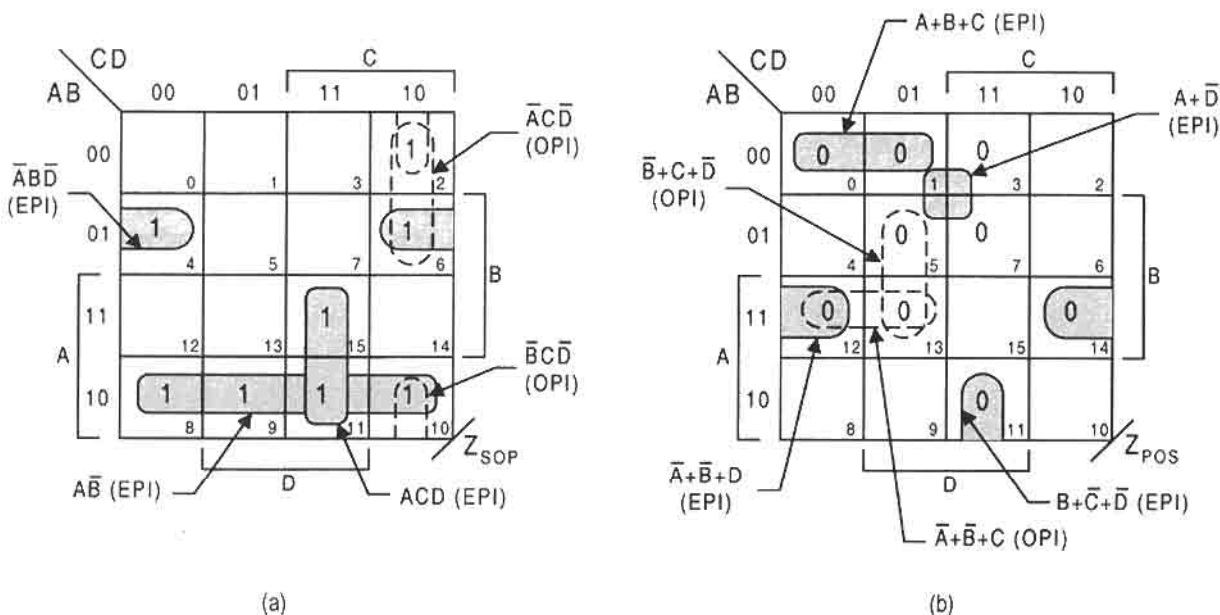


FIGURE 4.18

K-maps showing EPIs and OPIs for the function  $Z$  in Eq. (4.25). (a) SOP minimum cover. (b) POS minimum cover.

to be an EPI, the other OPI becomes redundant (an RPI). In similar fashion a minimum POS cover is extracted as shown in Fig. 4.18b, giving the result

$$Z_{POS} = (A + B + C)(B + \bar{C} + \bar{D})(\bar{A} + \bar{B} + D) \left\{ \begin{array}{l} (\bar{A} + \bar{B} + C) \\ (\bar{B} + C + \bar{D}) \end{array} \right\} (A + \bar{D}), \quad (4.27)$$

which is seen to have four dyads (including one of two OPIs) and one quad. The maxterm  $M_{13}$  can be looped out in two ways (dashed loops in Fig. 4.18b) to form the OPI dyads  $M(12, 13)$  and  $M(5, 13)$  represented by the bracketed s-terms in Eq. (4.27).

#### 4.4.3 Incompletely Specified Functions: Don't Cares

In the design of logic circuits *nonessential* minterms or maxterms may be introduced so as to simplify the circuit. Such nonessential minterms or maxterms are called *don't cares* and are represented by the symbol

$$\phi = \text{Min/Max} = \text{don't care}.$$

Thus, the don't care can be taken as logic 0 or logic 1, take your choice. The symbol  $\phi$  can be thought of as a logic 0 with a logic 1 superimposed on it.

Don't cares can arise under the following two conditions:

- When certain combinations of input logic variables can never occur, the output functions for such combinations are nonessential and are assigned don't cares.
- When all combinations of input logic variables occur but certain combinations of these variables are irrelevant, the output functions for such combinations are assigned don't cares.

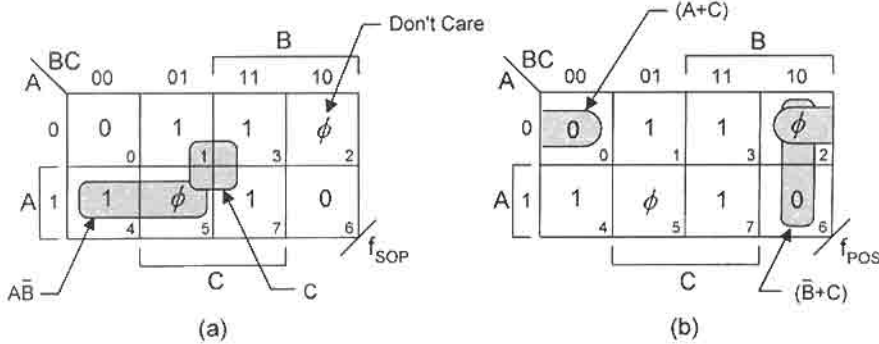
As an example of the second condition, the BCD number system discussed in Subsection 2.4.1 has 10 4-bit binary patterns for decimal integers 0 through 9. Thus, there are six 4-bit patterns, representing decimal integers 10 through 15 that are never used — that is, we “don't care” about them. Accordingly, the don't care symbol  $\phi$  can be assigned to any output generated by one of the six nonessential 4-bit patterns. This will be demonstrated in Subsection 6.5.2 for conversion from BCD to XS3 decimal codes.

Consider the three-variable function

$$f(A, B, C) = \underbrace{\sum m(1, 3, 4, 7)}_{\text{Essential minterms}} + \underbrace{\phi(2, 5)}_{\substack{\text{Nonessential} \\ \text{minterms} \\ \text{(don't cares)}}} \quad (4.28)$$

written in canonical SOP form showing essential minterms and nonessential minterms (don't cares). The K-maps representing minimum SOP and POS cover are shown in Figs. 4.19a and 4.19b, giving the results

$$\begin{aligned} F_{SOP} &= A\bar{B} + C \\ F_{POS} &= (A + C)(\bar{B} + C). \end{aligned} \quad (4.29)$$

**FIGURE 4.19**

K-maps for Eq. (4.28) showing EPIs containing don't cares. (a) Minimum SOP cover. (b) Minimum POS cover.

Notice that the don't cares  $\phi_2$  and  $\phi_5$  are purposely used differently to obtain the minimum SOP and POS expressions of Eqs. (4.29). The result is that the  $F_{SOP}$  and  $F_{POS}$  expressions are algebraically equal since there is no *shared use* of don't cares between the two functions ( $\phi_5 = 1$  and  $\phi_2 = 0$  in both cases). Thus,  $F_{SOP}$  can be produced by algebraically manipulating  $F_{POS}$ . Had no use been made of the two don't cares in Fig. 4.19, the results would be quite different, namely  $F_{SOP} = \bar{A}\bar{B}\bar{C} + \bar{A}C + BC$  and  $F_{POS} = (A + B + C)(\bar{A} + \bar{B} + C)$ , which are logically equivalent but not algebraically equal.

As a second example consider the four-variable function given in canonical POS form showing essential and nonessential maxterms:

$$Y(A, B, C, D) = \underbrace{\prod M(0, 1, 2, 4, 6, 9, 11, 15)}_{\text{Essential maxterms}} \cdot \underbrace{\phi(3, 8, 10, 12)}_{\substack{\text{Nonessential} \\ \text{maxterms} \\ \text{(don't cares)}}} \quad (4.30)$$

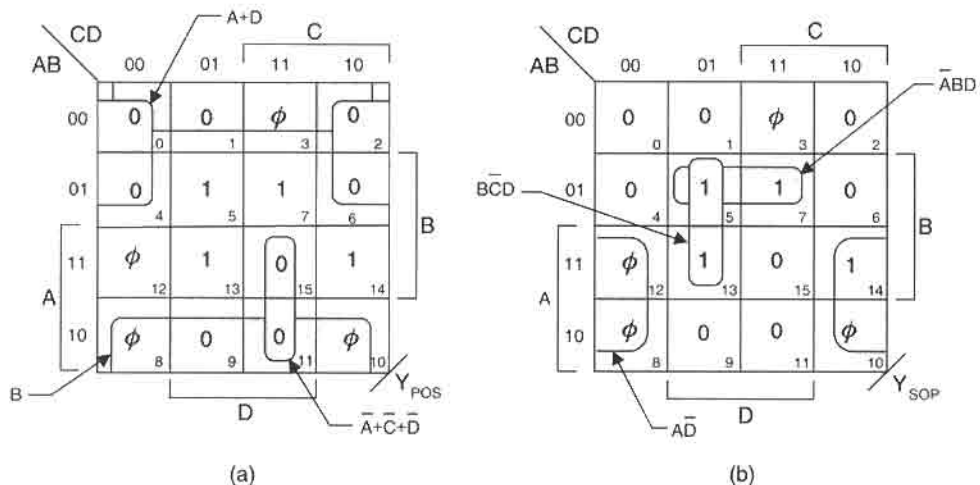
In Fig. 4.20 the 0's and  $\phi$ 's of Eq. (4.30) are mapped in maxterm code, and the minimum covers for  $Y_{POS}$  and  $Y_{SOP}$  are shown by the shaded loops in Figs. 4.20a and 4.20b, respectively. The resulting minimum POS and SOP expressions for Eq. (4.30) are

$$\begin{aligned} Y_{POS} &= (\bar{A} + \bar{C} + \bar{D})(A + D)B \\ Y_{SOP} &= \bar{A}BD + B\bar{C}D + A\bar{D} \end{aligned} \quad (4.31)$$

Again it is noted that these expressions are logically equivalent. However, they are algebraically unequal because of the *shared use* of don't cares ( $\phi_8$  and  $\phi_{10}$ ) in the loop-out process. Notice also that  $Y_{SOP}$  contains OPIs  $B\bar{C}D$  and  $A\bar{B}C$  with  $\bar{A}BD$  as an EPI, since minterm  $m_{13}$  can be looped out in two ways (with  $m_5$  or with  $\phi_{12}$ ). Similarly, OPIs  $\bar{A}BD$  and  $\bar{A}CD$  result if  $B\bar{C}D$  is an EPI, since minterm  $m_7$  can be looped out in two ways (with  $m_5$  and with  $\phi_3$ ). No OPIs exist for  $Y_{POS}$ .

**The Gate/Input Tally vs Cardinality of a Function** Throughout this text use will be made of the ratio of the gate tally to the input tally (*gate/input tally*) as a measure of function complexity in terms of hardware cost. Input tallies include both external and internal inputs



**FIGURE 4.20**

K-maps for Eq. (4.30) containing don't cares showing (a) minimum POS cover and (b) minimum SOP cover containing OPIs for minterms in cells 7 and 13 but not shown.

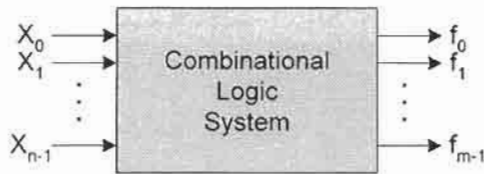
(connections) to gates. Gate tallies are weighted more heavily than input tallies. Inverters can be included in the gate/input tally of a given function only if the activation levels of the inputs are known. Unless otherwise stated, the gate/input tallies will be given exclusive of inverters. An inverter is considered to be a gate with one input.

The number of p-terms or s-terms representing a function is called the *cardinality* of the function. Thus, the cardinality of a cover represents the number of prime implicants of the function, and a minimum cardinality (or cover) consists only of EPIs. When significant numbers of don't cares are present in a function, there may exist several alternative covers of minimum cardinality that may differ in gate/input tally.

As an example of the use of the gate/input tally and cardinality, consider the minimized expressions in Eqs. (4.29). Here,  $F_{SOP}$  has a gate/input tally of 2/4, whereas the gate/input tally for  $F_{POS}$  is 3/6, both exclusive of inverters and both with a minimum cardinality of 2. Thus, the SOP expression is the simpler of the two. However, this may not always be true. Taking a gate and input count of Eqs. (4.31) reveals that the gate/input tally for  $Y_{POS}$  is 3/8 while that for  $Y_{SOP}$  is 4/11, again both exclusive of possible inverters. Thus, in this case, the POS expression is the simpler hardware-wise, but both expressions have a minimum cardinality of 3. Notice that a single variable EPI contributes to the cardinality count of the function but not to the gate tally.

#### 4.5 MULTIPLE OUTPUT OPTIMIZATION

Frequently, logic system design problems require optimization of multiple output functions, all of which are functions of the same input variables. For complex systems this is generally regarded as a tedious task to accomplish without the aid of a computer, and for this reason computer programs have been written to obtain the optimum cover for multioutput functions of many variables. Examples of such computer programs are discussed in Appendix B.1. In this section a simple approach to this process will be presented but limited to two or

**FIGURE 4.21**

Block diagram for an  $n$ -input/ $m$ -output combinational logic system.

three outputs, each output being limited to four variables or less. Beyond these limitations, computer-aided optimization is recommended.

Consider the  $n$ -input/ $m$ -output system illustrated by the block diagram in Fig. 4.21. Suppose the object is to minimize each of the  $m$  output functions in such a way as to make use of as many of the shared terms between them as possible, thus optimizing the combinational logic of this system. The recommended procedure is given in four steps that follow.

### **Multiple-Output Minimization Procedure**

Step 1. Obtain the canonical SOP or POS forms. If necessary, K-maps can be used for this purpose.

Step 2. AND the canonical SOP forms or OR the canonical POS forms in some systematic way (for example,  $f_1 \cdot f_2$ ,  $f_2 \cdot f_3$ ,  $f_3 \cdot f_4, \dots$ , or  $f_1 + f_2$ ,  $f_2 + f_3$ ,  $f_3 + f_4, \dots$ ) and map each ANDed or Ored expression separately, looping out all shared PIs (common terms).

Minterm ANDing rules:

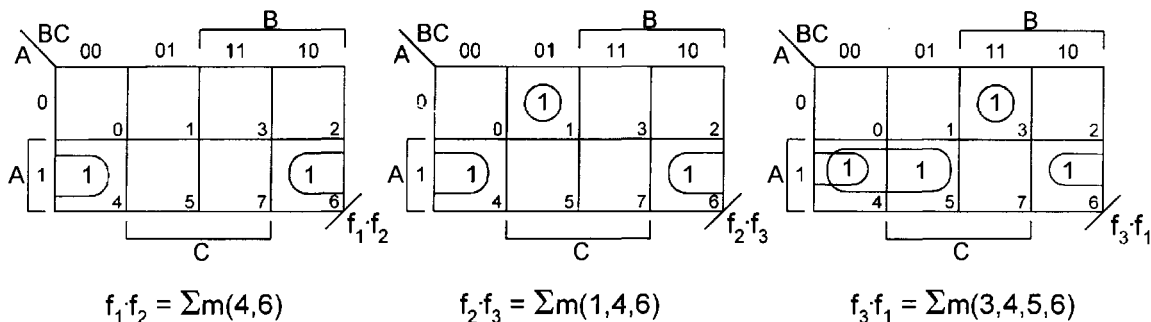
$$\begin{aligned}
 m_i \cdot m_i &= m_i \\
 m_i \cdot m_j &= 0 \quad (i \neq j) \\
 m_i \cdot \phi_i &= m_i \\
 \phi_i \cdot \phi_i &= \phi_i \\
 m_i \cdot \phi_j &= \phi_i \cdot \phi_j = 0 \quad (i \neq j)
 \end{aligned} \tag{4.32}$$

Maxterm ORing rules:

$$\begin{aligned}
 M_i + M_i &= M_i \\
 M_i + M_j &= 1 \quad (i \neq j) \\
 M_i + \phi_i &= M_i \\
 \phi_i + \phi_i &= \phi_i \\
 M_i + \phi_j &= \phi_i + \phi_j = 1 \quad (i \neq j)
 \end{aligned} \tag{4.33}$$

Step 3. Make a table of the results of step 2 giving all shared PIs in literal form.

Step 4. From K-maps of the original functions, loop out the shared PIs given in step 3, then loop out the remaining EPIs following the loop-out protocol with

**FIGURE 4.22**

ANDed functions, their K-maps, and minimum cover for Eqs. (4.34).

one exception. If the adjacencies of the shared PIs are part of a larger  $2^n$  grouping of adjacencies, use the larger grouping, but only if it leads to simpler form.

For simple systems the four-step procedure just given can be shortened considerably by simply comparing the K-maps of the functions. Often the adjacency patterns lead to an immediate recognition of the shared PIs that should be included for optimum cover.

To illustrate the four-step procedure given previously, consider the system of three outputs, each a function of three variables:

$$\left\{ \begin{array}{l} f_1(A, B, C) = \sum m(0, 3, 4, 5, 6) \\ f_2(A, B, C) = \sum m(1, 2, 4, 6, 7) \\ f_3(A, B, C) = \sum m(1, 3, 4, 5, 6) \end{array} \right\}. \quad (4.34)$$

Equations (4.34) satisfy step 1 of the multiple-output minimization procedure. Then, moving on to step 2, the ANDed functions are given in Fig. 4.22, together with their respective K-maps and minimum cover. The minimum cover in each ANDed K-map indicates the common terms that must be included in the optimized expressions for the three-output system.

The table of shared PIs for each of the ANDed forms and the appropriate transfer of these shared PIs into the K-maps of the original functions are given in Fig. 4.23, in agreement with steps 3 and 4 of the multiple-output minimization procedure. Notice that the dyad  $A\bar{C}$  is common to all three ANDed functions, as is evident from the ANDed function  $f_1 \cdot f_2 \cdot f_3 = m(4, 6)$  indicated in the table of shared PIs of Fig. 4.23.

By looping out the shared PIs first in Fig. 4.23 followed by the remaining EPIs, there result the optimal expressions

$$\left\{ \begin{array}{l} f_1 = \bar{A}BC + A\bar{C} + A\bar{B} + \bar{B}\bar{C} \\ f_2 = \bar{A}\bar{B}C + A\bar{C} + AB + B\bar{C} \\ f_3 = \bar{A}BC + \bar{A}\bar{B}C + A\bar{C} + A\bar{B} \end{array} \right\}. \quad (4.35)$$

Notice that the dyad  $m(1, 3)$  in the  $f_3$  K-map is avoided, hence also an individual minimum for  $f_3$ , so that the expression for  $f_3$  can be completely generated from the terms in  $f_1$  and  $f_2$ , the optimal solution. The optimum gate/input tally is 10/28 for this system of three outputs, each output having a cardinality of 4.

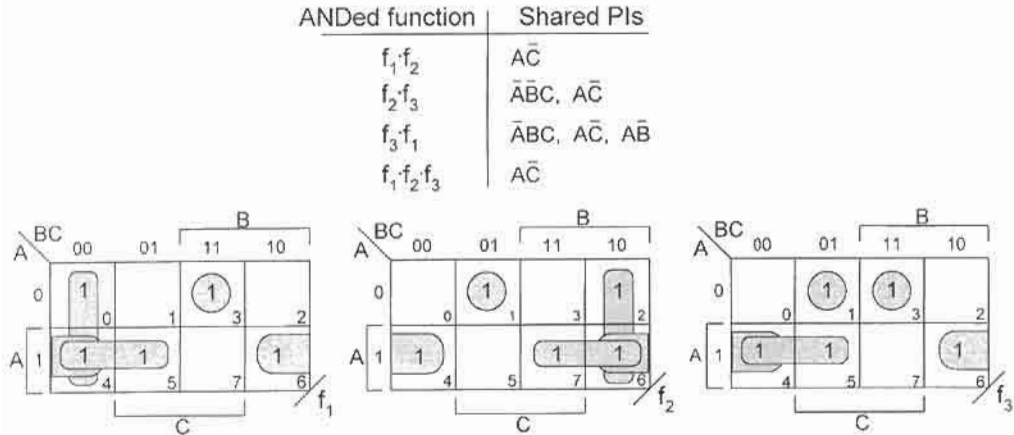
**FIGURE 4.23**

Table of shared PIs and the K-maps for the functions  $f_1$ ,  $f_2$ , and  $f_3$ , showing optimal cover for the three-output system of Eqs. (4.34).

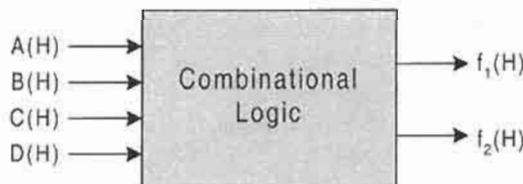
As a second example, consider the output functions for the four-input/two-output logic system represented by Eqs. (4.36) and by the block diagram in Fig. 4.24. It is required that this system be optimized with respect to both POS cover and SOP cover following the four-step multiple output minimization procedure and that the results be compared to determine which, if either, is the more optimum. The optimized system is to be implemented with either NOR/INV logic or NAND/INV logic.

$$\begin{aligned}
 f_1(A, B, C, D) &= \prod M(1, 2, 3, 4, 5, 9, 10) \cdot \phi(6, 11, 13) \\
 &= \sum m(0, 7, 8, 12, 14, 15) + \phi(6, 11, 13) \\
 f_2(A, B, C, D) &= \prod M(2, 5, 9, 10, 11, 15) \cdot \phi(3, 4, 13, 14) \\
 &= \sum m(0, 1, 6, 7, 8, 12) + \phi(3, 4, 13, 14)
 \end{aligned}
 \tag{4.36}$$

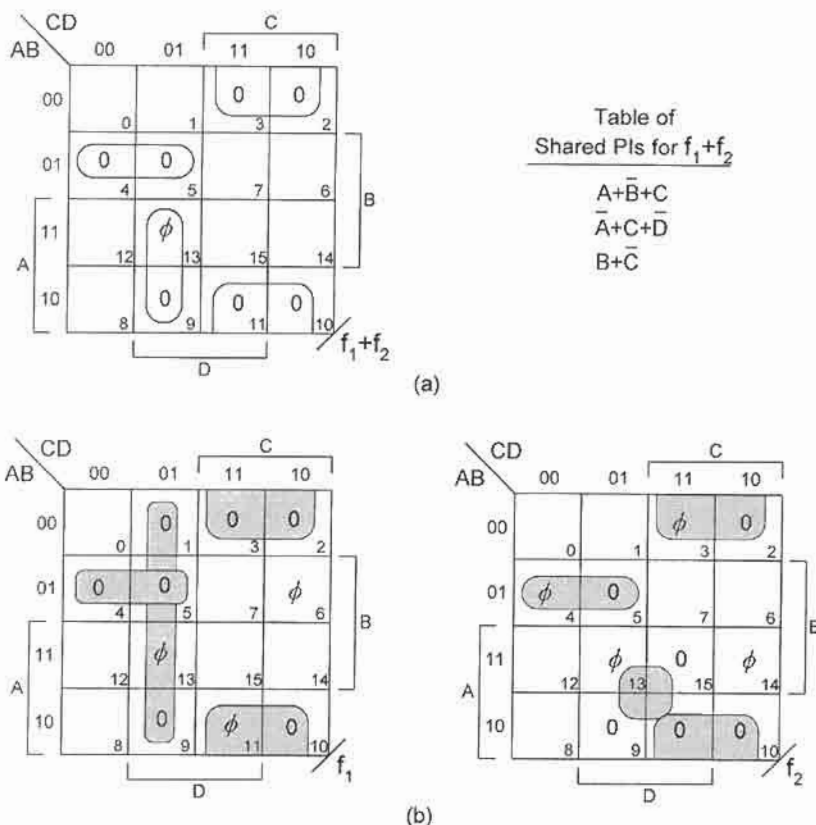
*Optimized POS Cover.* ORing of the canonical forms of Eqs. (4.36) yields

$$f_1 + f_2 = \prod M(2, 3, 4, 5, 9, 10, 11) \cdot \phi(13),$$

where use has been made of the ORing rules given by Eqs. (4.33) at the beginning of this

**FIGURE 4.24**

Block diagram for a four-input/two-output combinational logic system represented by Eqs. (4.36).

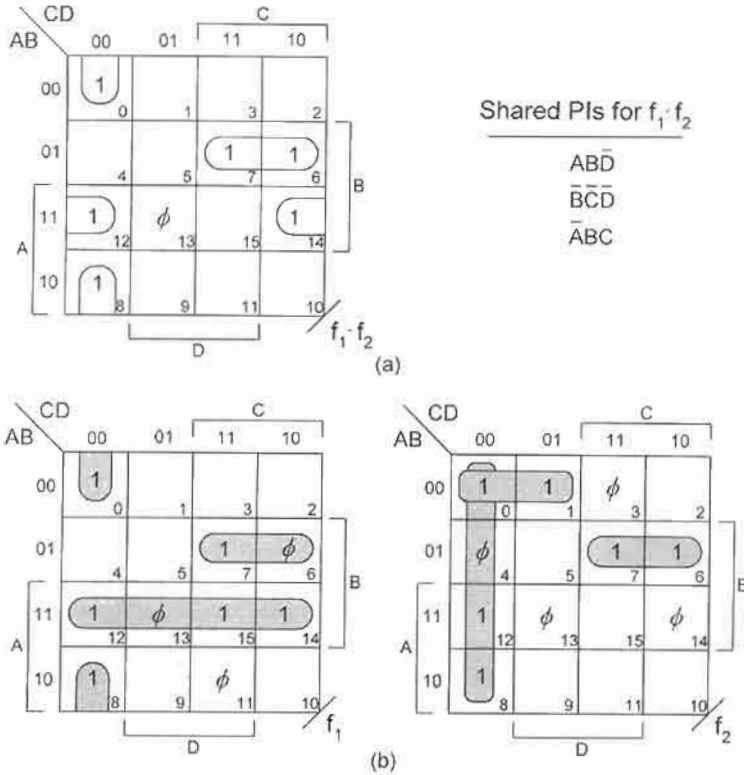
**FIGURE 4.25**

Multioutput POS optimization for the system represented by Eqs. (4.36) and Fig. (4.24). (a) K-map and shared PIs for  $(f_1 + f_2)$ . (b) K-maps showing optimal POS cover for functions  $f_1$  and  $f_2$ .

section. Figure 4.25a gives the K-map and table of shared s-term PIs for  $f_1 + f_2$ . The final step involves looping out the individual functions by following the loop-out protocol in such a manner as to incorporate as many shared PIs as necessary to achieve optimum cover for the two outputs. Reading the K-maps in Fig. 4.25b for  $f_1$  and  $f_2$  produces the results

$$\begin{cases} f_1 = (A + \bar{B} + C)(B + \bar{C})(C + \bar{D}) \\ f_2 = (A + \bar{B} + C)(B + \bar{C})(\bar{A} + \bar{D}) \end{cases} \quad (4.37)$$

which yields a combined gate/input tally of 6/15 exclusive of possible inverters. Notice that the shared PI dyad  $(\bar{A} + C + \bar{D})$  is covered by the quads  $(C + \bar{D})$  and  $(\bar{A} + \bar{D})$  in the expressions for  $f_1$  and  $f_2$ , respectively. Thus, the optimum coverage for both  $f_1$  and  $f_2$  is, in this case, that of the individual minimum forms. This is not usually the case, as is demonstrated next for the optimum SOP results. Note that if strict use had been made of all the shared PIs in the table of Fig. 4.25a together with a required dyad for each output, the combined gate/input tally would become 7/22, significantly greater than that of Eqs. (4.37).

**FIGURE 4.26**

Multioutput SOP optimization for the system represented by Eqs. (4.36) and Fig. 4.24. (a) K-map and shared PIs for  $f_1 \cdot f_2$ . (b) K-maps showing optimal SOP cover for functions  $f_1$  and  $f_2$ .

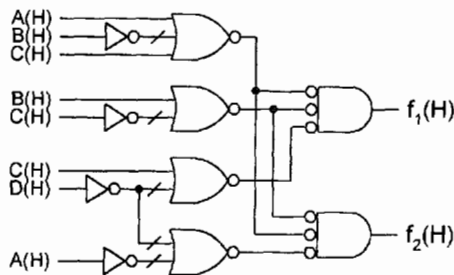
*Optimized SOP Cover.* ANDING the canonical SOP forms of Eqs. (4.36) by using the ANDING rules given by Eqs. (4.32) produces

$$f_1 \cdot f_2 = \sum m(0, 6, 7, 8, 12, 14) + \phi(13).$$

The K-map for  $f_1 \cdot f_2$  and the table of shared p-term PIs is given in Fig. 4.26a. The K-maps in Fig. 4.26b show the optimized cover for the two-function system. The results are

$$\begin{cases} f_1 = \bar{A}BC + \bar{B}\bar{C}\bar{D} + AB \\ f_2 = \bar{A}BC + \bar{A}\bar{B}\bar{C} + \bar{C}\bar{D} \end{cases}, \quad (4.38)$$

which represent a combined gate/input tally of 7/19 making use of only one of the three shared PIs. Here, shared PI dyads  $AB\bar{D}$  and  $\bar{B}\bar{C}\bar{D}$  are rejected in favor of quads  $AB$  and  $\bar{C}\bar{D}$  in the  $f_1$  and  $f_2$  K-maps, respectively. Notice that function  $f_1$  is not an individual minimum, but combined with the individual minimum for function  $f_2$  results in an optimized system. An individual minimum for function  $f_1$  is achieved by replacing the shared PI  $m(6, 7)$  with the quad  $m(6, 7, 14, 15)$  in Fig. 4.26b. When combined with the individual minimum for function  $f_2$ , there results a gate/input tally of 8/21, which is not optimal. Also, note that

**FIGURE 4.27**

NOR/INV logic circuit for the optimized POS system of Fig. 4.25.

making use of all shared PIs in the table of Fig. 4.26a together with the required additional p-term cover yields a combined gate/input tally of 7/22.

Comparing the POS and SOP results with optimum system covers of cardinality 4 and 5, respectively, it is clear that the POS result is the more optimum (gate/input tally of 6/15 or 10/19 including inverters). Shown in Fig. 4.27 is the optimal NOR/INV logic implementation of the POS results given by Eqs. (4.37).

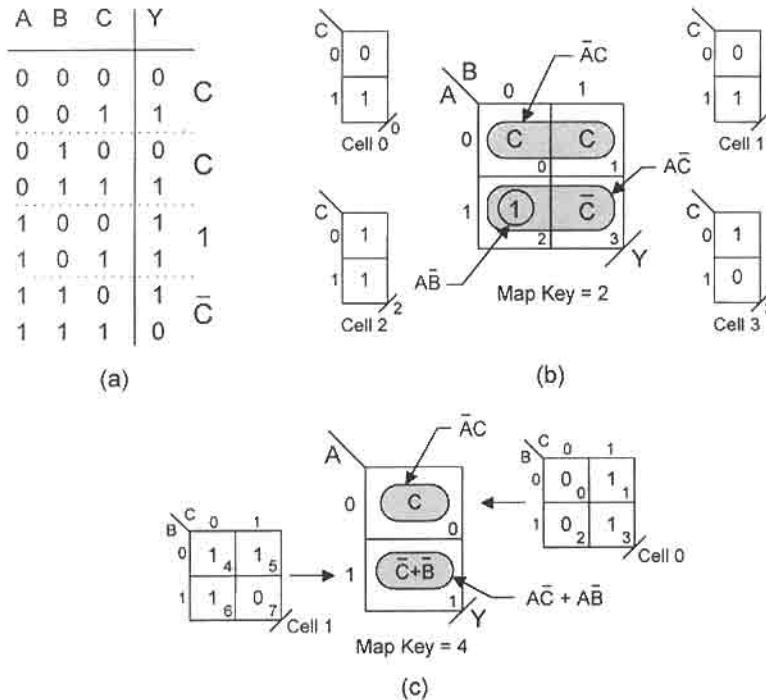
The simple search method used here to obtain optimum results becomes quite tedious when applied to multiple output systems more complicated than those just described. For example, a four-input/four-output SOP optimization problem would require at least 10 ANDed fourth-order K-maps, including one for each of six ANDed pairs. For systems this large and larger it is recommended that a computer optimization program (Appendix B) be used, particularly if a guaranteed optimum cover is sought. Optimum cover, as used here, means the least number of gates required for implementation of the multiple output system. Obviously, the number of inverters required and fan-in considerations must also be taken into account when appraising the total hardware cost.

## 4.6 ENTERED VARIABLE K-MAP MINIMIZATION

Conspicuously absent in the foregoing discussions on K-map function minimization is the treatment of function minimization in K-maps of lesser order than the number of variables of the function. An example of this would be the function reduction of five or more variables in a fourth-order K-map. In this section these problems are discussed by the subject of *entered variable (EV) mapping*, which is a “logical” and very useful extension of the conventional (1’s and 0’s) mapping methods developed previously.

Properly used, EV K-maps can significantly facilitate the function reduction process. But function reduction is not the only use to which EV K-maps can be put advantageously. Frequently, the specifications of a logic design problem lend themselves quite naturally to EV map representation from which useful information can be obtained directly. Many examples of this are provided in subsequent chapters. In fact, EV (entered variable) K-maps are the most common form of graphical representation used in this text.

If  $N$  is the number of variables in the function, then map entered variables originate when a conventional  $N$ th-order K-map is compressed into a K-map of order  $n < N$  with terms of  $(N - n)$  variables entered into the appropriate cells of the  $n$ th-order K-map. Thus,

**FIGURE 4.28**

(a) Truth table for function  $Y$  in Eq. (4.39) showing subfunctions for a first-order map compression. (b), (c) Second and first-order EV K-maps showing submaps and minimum SOP cover extracted in minterm code.

each cell of the  $n$ th-order K-map becomes a submap of order  $(N - n)$ , hence K-maps within K-maps.

To illustrate, consider the three-variable function

$$Y(A, B, C) = \sum m(1, 3, 4, 5, 6), \quad (4.39)$$

which has been placed in a truth table and mapped into a second-order EV K-map, as shown in Figs. 4.28a and 4.28b. The subfunctions indicated to the right of the truth table are also represented as first-order submaps corresponding to the cells 0, 1, 2, and 3 in the EV K-map of Fig. 4.28b. The minimum cover is then obtained by looping out the cell entries, as shown by the shaded loops, giving the minimum result

$$Y_{SOP} = \bar{A}C + A\bar{C} + A\bar{B}. \quad (4.40)$$

Notice that the term  $A\bar{C}$  covers only the  $\bar{C}$  in the  $1 = C + \bar{C}$  of cell 2. This requires that the  $C$  in the 1 be covered by one of the two OPIs,  $A\bar{B}$  or  $\bar{B}C$ , and the former is chosen.

The same result can be obtained from a second-order compression if the expression of Eq. (4.39) is compressed into a first-order K-map. This is done in Fig. 4.28c, where  $B$  and  $C$  are now the EVs. The minimum cover is indicated by the shaded loops, yielding



the expression in Eq. (4.40). The OPI  $\bar{B}C$  is not easily seen in the first-order EV K-map, but can be found by observing the 1's representing  $\bar{B}C$  in the two submaps shown in Fig. 4.28c.

**Map Key** It has already been pointed out that each cell of the compressed  $n$ th-order K-map represents a submap of order  $(N - n)$  for an  $N > n$  variable function. Thus, each submap covers  $2^{N-n}$  possible minterms or maxterms. This leads to the conclusion that any compressed  $n$ th-order K-map, representing a function of  $N > n$  variables, has a Map Key defined by

$$\text{Map Key} = 2^{N-n} \quad N > n \quad (4.41)$$

The Map Key has the special property that when multiplied by a cell code number of the compressed  $n$ th-order K-map there results the code number of the first minterm or maxterm possible for that cell. Furthermore, the Map Key also gives the maximum number of minterms or maxterms that can be represented by a given cell of the compressed  $n$ th-order K-map. These facts may be summarized as follows:

Conventional K-map: Map Key = 1 (no EVs, 1's and 0's only)

First-order compression K-map: Map Key = 2 (one EV)

Second-order compression K-map: Map Key = 4 (two EVs)

Third-order compression K-map: Map Key = 8 (three EVs), etc.

As an example, the first-order compressed K-map in Fig. 4.28b has a Map Key of  $2^{3-2} = 2$ . So each of its cells represents two possible minterms (first-order submaps) beginning with minterm code number equal to (Map Key = 2)  $\times$  (Cell Number). This is evident from an inspection of the truth table in Fig. 4.28a. Similarly, the second-order compression in Fig. 4.28c has a Map Key of  $2^{3-1} = 4$ . Therefore, each cell represents four possible minterms represented by the conventional second-order submaps shown to the sides of Fig. 4.28c.

The compressed K-maps in Fig. 4.28 can also be read in maxterm code as indicated by the shaded loops in Fig. 4.29. In this case the logic 1 in cell 2 must be excluded. The result for either the first-order or second-order compressed K-maps is

$$Y_{POS} = (\bar{A} + \bar{B} + \bar{C})(A + C). \quad (4.42)$$

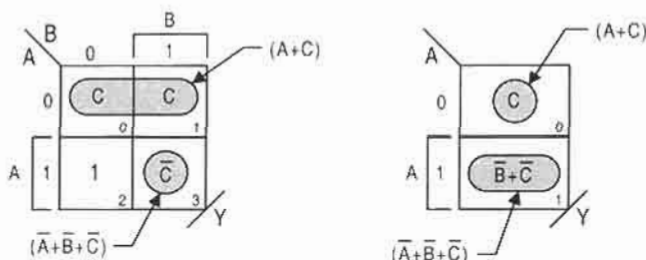
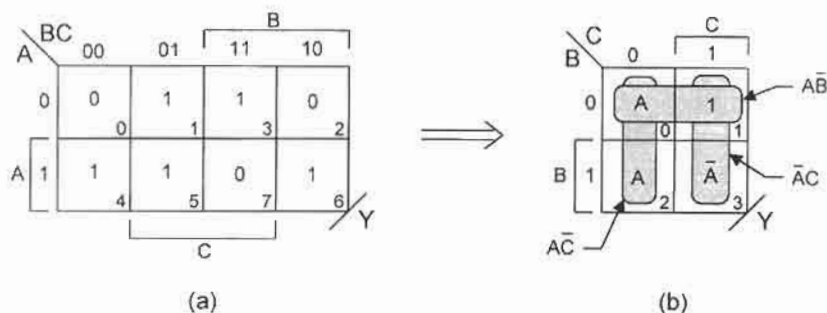


FIGURE 4.29

Second- and first-order EV K-maps showing minimum POS cover for function  $Y$  extracted in maxterm code.

**FIGURE 4.30**

(a) Conventional K-map for function  $Y$  of Eq. (4.39). (b) Second-order EV K-map with entered variable  $A$  showing minimum cover for  $Y$  as extracted in minterm code.

That  $Y_{POS}$  in Eq. (4.40) and  $Y_{SOP}$  in Eq. (4.42) are algebraically equal is made evident by carrying out the following Boolean manipulation:

$$(\bar{A} + \bar{B} + \bar{C})(A + C) = \bar{A}C + A\bar{C} + [A\bar{B} + \bar{B}C],$$

where the two p-terms in brackets are OPIs, thereby rendering one to be redundant.

In the second-order K-maps of Figs. 4.28 and 4.29,  $C$  is taken to be the EV. However, any of the three variables could have been chosen as the EV in the first-order compression K-maps. As an example, variable  $A$  is the EV in Fig. 4.30, where the columns in the conventional K-map of (a) form the submaps of the cells in the compressed K-map of Fig. 4.30b. Minimum cover extracted in minterm code then yields the same result as Eq. (4.40). Or, if extracted in maxterm code, Eq. (4.42) would result. Thus, one concludes that the choice of EVs in a compressed K-map does not affect the extracted minimum result.

Reduced but nonminimum functions can be easily compressed into EV K-maps. This is demonstrated by mapping the four-variable function

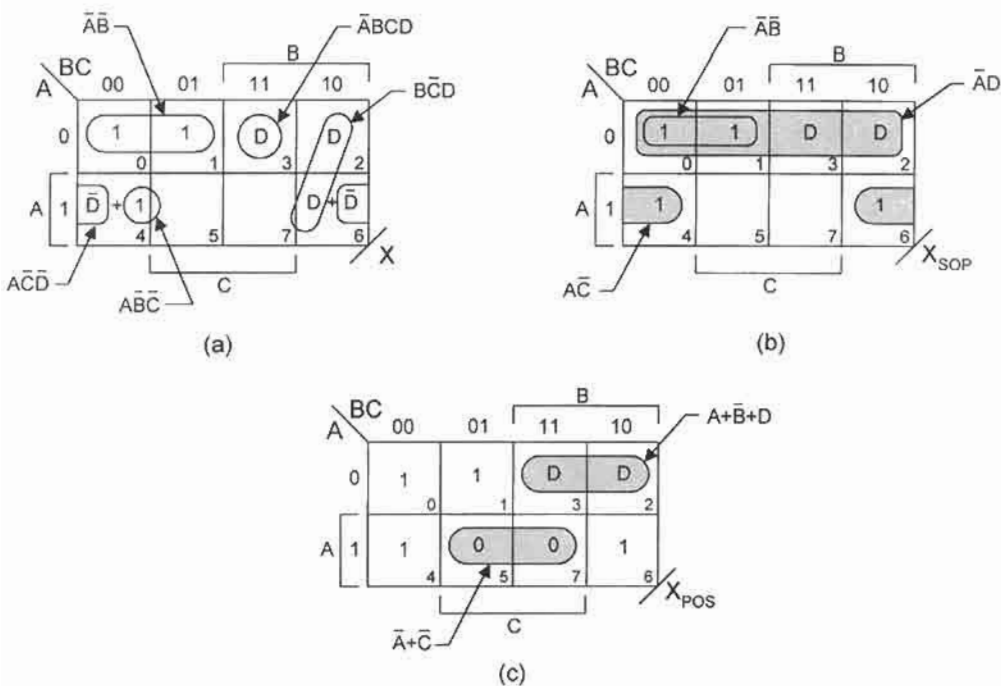
$$X = B\bar{C}D + \bar{A}\bar{B} + A\bar{C}\bar{D} + \bar{A}BCD + A\bar{B}\bar{C} \quad (4.43)$$

into the third-order EV K-maps shown in Fig. 4.31, where the Map Key is 2. Here,  $D$  is the EV and  $1 = (D + \bar{D})$ . Figure 4.31a shows the p-terms (loops) exactly as presented in Eq. (4.43). However, regrouping of the logic adjacencies permits minimum SOP and POS cover to be extracted. This is done in Figs. 4.31b and 4.31c, yielding

$$\begin{aligned} X_{SOP} &= \bar{A}D + A\bar{C} + \bar{A}\bar{B} \\ X_{POS} &= (A + \bar{B} + D)(\bar{A} + \bar{C}), \end{aligned} \quad (4.44)$$

where the expressions for  $X_{SOP}$  and  $X_{POS}$  represent gate/input tallies of 4/9 and 3/7, respectively, excluding possible inverters.

The four-variable function  $X$  in Eq. (4.43) can also be minimized in a second-order EV K-map. Shown in Fig. 4.32 is the second-order compression and minimum SOP and POS cover for this function, giving the same results as in Eqs. (4.44). Notice that after covering the  $D$  in cell 1 of Fig. 4.32a, it is necessary to cover all that remains in cell 0 by looping out the 1 as an island to give  $\bar{A}\bar{B}$ . In this case the 1 has the value  $1 = C + \bar{C} = D + \bar{D}$ . Clearly, the 1 in cell 0 cannot be used in extracting minimum cover in maxterm code.

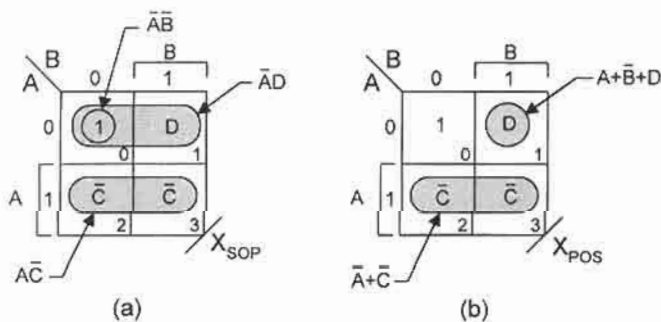
**FIGURE 4.31**

(a) First-order compression plot of the function  $X$  in Eq. (4.43) showing original p-terms. (b) Minimum SOP cover. (c) Minimum POS cover.

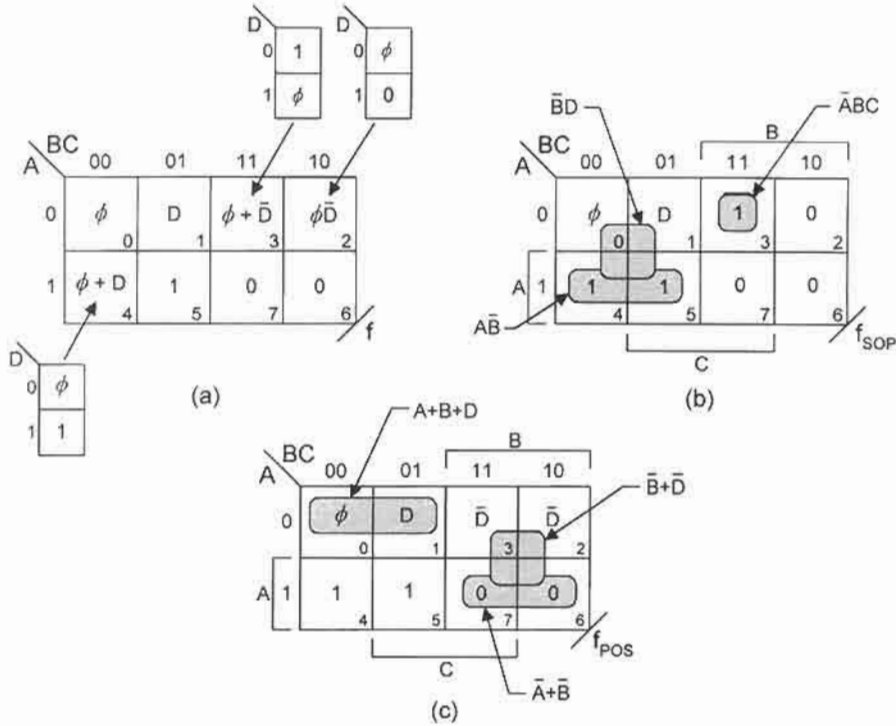
#### 4.6.1 Incompletely Specified Functions

The EV mapping method is further illustrated by compressing the incompletely specified function

$$f(A, B, C, D) = \sum m(3, 6, 9, 10, 11) + \phi(0, 1, 4, 7, 8) \quad (4.45)$$

**FIGURE 4.32**

Second-order compressions of the function  $X$  showing (a) minimum SOP cover and (b) minimum POS cover.

**FIGURE 4.33**

(a) First-order compression plot and submaps for the function  $f$  in Eq. (4.45). (b) Minimum SOP cover and (c) minimum POS cover.

into the third-order K-map in Fig. 4.33a, a first-order compression with a Map Key of 2. Here, the subfunctions are presented in their simplest form yet preserving all canonical information. In Figs. 4.33b and 4.33c are shown the minimum SOP and POS covers for this function, which produce the expressions

$$\begin{aligned} f_{SOP} &= \bar{B}\bar{D} + \bar{A}BC + A\bar{B} \\ f_{POS} &= (A + B + D)(\bar{B} + \bar{D})(\bar{A} + \bar{B}), \end{aligned} \quad (4.46)$$

both of which have a gate/input tally of 4/10. In extracting the minimum expressions of Eqs. (4.46), the loop-out protocol is first applied to the entered variable  $D$  and then applied to the 1's or 0's.

Some observations are necessary with regard to Fig. 4.33 and Eqs. (4.46). First, these expressions are logically equivalent but are *not* algebraically equal. The reason is that the don't cares  $\phi_4$  and  $\phi_7$  in cells 2 and 3 are used differently for the  $f_{SOP}$  and  $f_{POS}$ . For example,  $(\phi_7 + \bar{D})_{SOP} = 1$  for  $\phi_7 = 1$  but  $(\phi_7 + \bar{D})_{POS} = \bar{D}$ , since, in this case,  $\phi_7 = 0$ . Second, the extraction process involved some techniques in dealing with  $\phi$ 's that have not been discussed heretofore. These techniques are set off for reference purposes by the following:

**Remember:**

- Treat the don't care ( $\phi$ ) as an *entered variable* — which it is.
- In simplifying incompletely specified subfunctions, apply the absorptive laws:

$$X + \phi \bar{X} = X + \phi$$

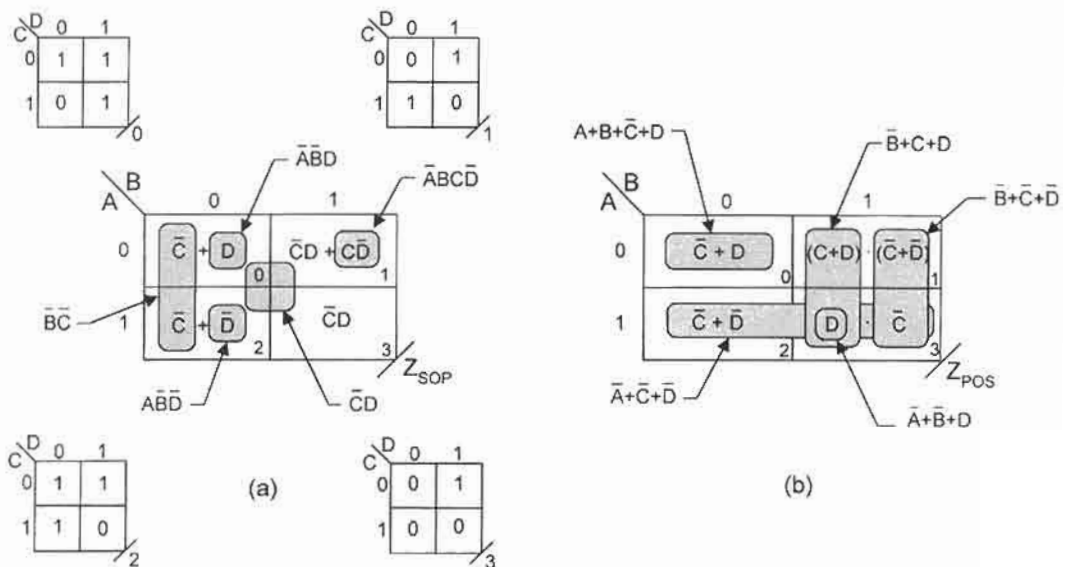
$$X \cdot (\phi + \bar{X}) = \phi X.$$

- Subfunctions of the type  $(\phi + X)$  have an essential SOP component but no essential POS component. (Proved by substituting the set  $\{0, 1\}$  for  $\phi$ .)
- Subfunctions of the type  $\phi X$  have an essential POS component but no essential SOP component. (Proved by substituting the set  $\{0, 1\}$  for  $\phi$ .)

Concluding this section is the function

$$\begin{aligned} Z(A, B, C, D) &= \prod M(2, 4, 7, 11, 12, 14, 15) \\ &= \sum m(0, 1, 3, 5, 6, 8, 9, 10, 13), \end{aligned} \quad (4.47)$$

which is represented by the second-order EV K-maps in Fig. 4.34, where  $C$  and  $D$  are the EVs and the Map Key is 4. This example is interesting because of the XOR function in cell 1, which must be represented by both the SOP and POS defining relations, given in Eqs. (3.4), so as to extract minimum SOP and POS cover. To assist the reader in identifying the subfunctions, second-order conventional submaps in  $C$  and  $D$  axes are shown for each cell. Thus, the subfunction for cell 0 is  $\sum m(0, 1, 3) = \bar{C} + D$ , while that for cell 1 is



**FIGURE 4.34**

Second-order EV K-maps and submaps for Eqs. (4.47) showing (a) minimum SOP cover and (b) minimum POS cover.

$\sum m(5, 6) = C \oplus D = C\bar{D} + \bar{C}D = (C + D)(\bar{C} + \bar{D})$ . The minimum SOP and POS results are given by

$$\begin{aligned} Z_{SOP} &= \bar{A}BC\bar{D} + \bar{A}\bar{B}D + A\bar{B}\bar{D} + \bar{C}D + \bar{B}\bar{C} \\ Z_{POS} &= (A + B + \bar{C} + D)(\bar{B} + \bar{C} + \bar{D})(\bar{B} + C + D)(\bar{A} + \bar{B} + D)(\bar{A} + \bar{C} + \bar{D}). \end{aligned} \quad (4.48)$$

From the results depicted in Fig. 4.34, certain conclusions are worth remembering and are set off by the following:

- In minterm code, subfunctions of the type  $XY$  are subsets of forms of the type  $X + Y$ .
- In maxterm code, subfunctions of the type  $X + Y$  are subsets of forms of the type  $XY$ .

What this means is that subfunctions of the type  $XY$  can be looped out from terms of the type  $X + Y$  to produce reduced SOP cover. For reduced POS cover, subfunctions of the type  $X + Y$  can be looped out from terms of the type  $XY$  (there are more 0's in  $XY$  than in  $X + Y$ ). For example, in Fig. 4.34  $\bar{C}D$  is looped out of both  $\bar{C} + D$  and  $\bar{C} + \bar{D}$  to contribute to minimum SOP cover. However, in Fig. 4.34b both  $C + D$  and  $\bar{C} + \bar{D}$  are looped out of  $\bar{C}D$ , leaving  $\bar{C} + D$  to be covered by  $\bar{A} + \bar{B} + D$ .

## 4.7 FUNCTION REDUCTION OF FIVE OR MORE VARIABLES

Perhaps the most powerful application of the EV mapping method is the minimization or reduction of functions having five or more variables. However, beyond eight variables the EV method could become too tedious to be of value, given the computer methods available. The subject of computer-aided minimization tools is covered in Appendix B.

Consider the function

$$F(A, B, C, D, E) = \sum m(3, 11, 12, 19, 24, 25, 26, 27, 28, 30), \quad (4.49)$$

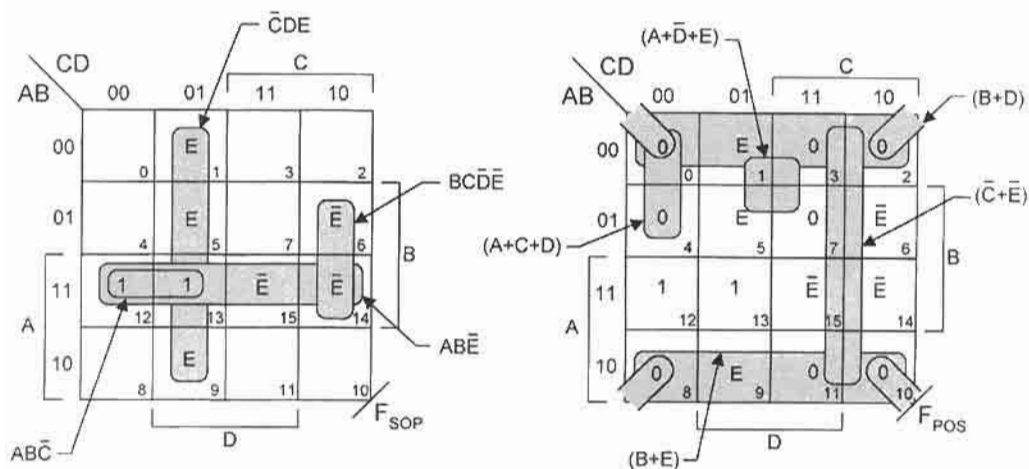
which is to be compressed into a fourth-order K-map. Shown in Fig. 4.35 is the first-order compression (Map Key = 2) and minimum SOP and POS cover for the five variable function in Eqs. (4.49). The minimized results are

$$\begin{aligned} F_{SOP} &= BC\bar{D}\bar{E} + \bar{C}DE + AB\bar{E} + AB\bar{C} \\ F_{POS} &= (A + \bar{D} + E)(\bar{C} + \bar{E})(B + E)(A + C + D)(B + D), \end{aligned} \quad (4.50)$$

which have gate input tallies of 5/17 and 6/17, respectively. Thus, the SOP result is the simpler of the two. Also, since there are no don't cares involved, the two expressions are algebraically equal. Thus, one expression can be derived from the other by Boolean manipulation.

A more complex example is presented in Fig. 4.36, where the six-variable function

$$\begin{aligned} Z(A, B, C, D, E, F) \\ = \sum m(0, 2, 4, 6, 8, 10, 12, 14, 16, 20, 23, 32, 34, 36, 38, 40, \\ 42, 44, 45, 46, 49, 51, 53, 54, 55, 57, 59, 60, 61, 62, 63) \end{aligned} \quad (4.51)$$

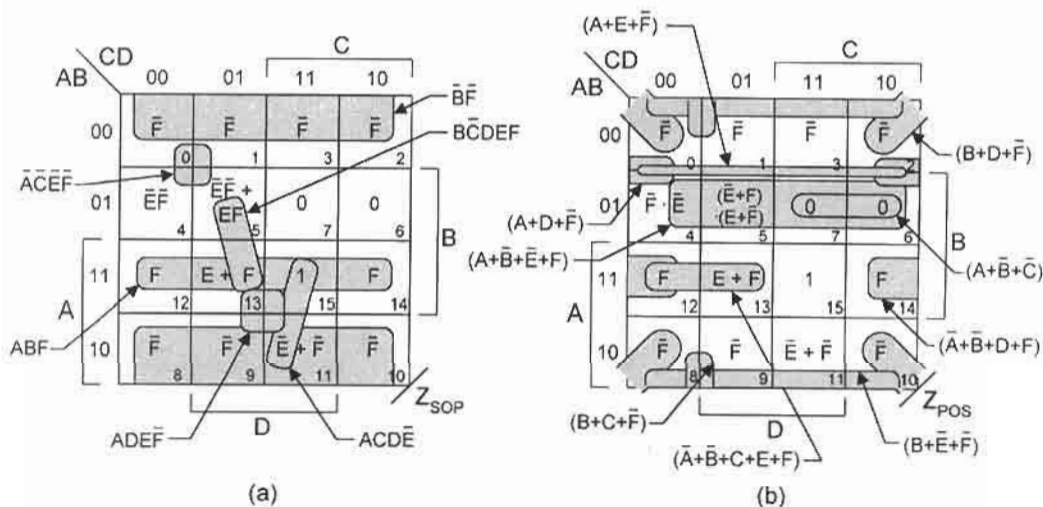
**FIGURE 4.35**

Minimum SOP and POS cover for the five-variable function given by Eq. (4.49).

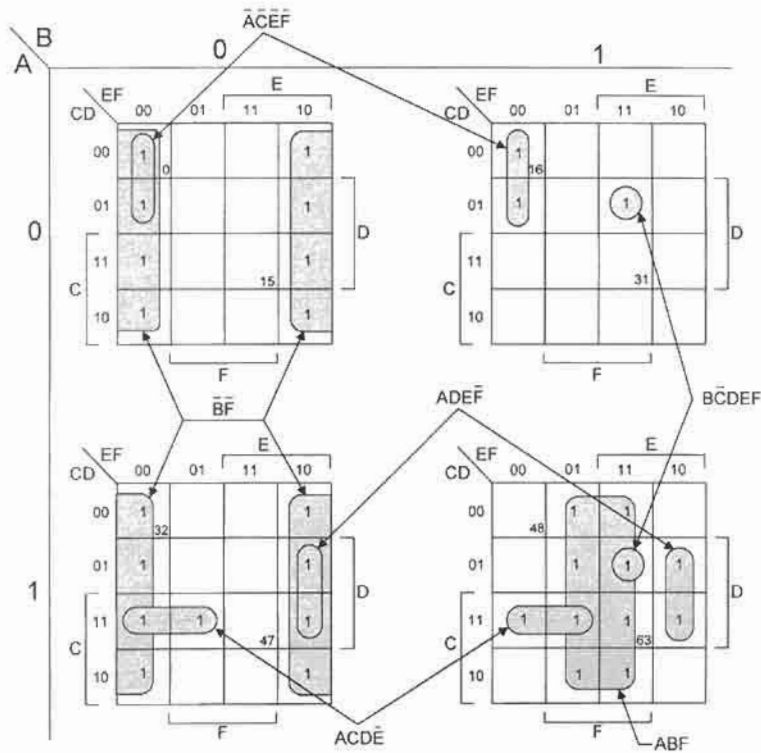
is compressed into a fourth-order K-map, a second-order compression (Map Key = 4). The minimum SOP and POS cover is indicated by the shaded loops in Figs. 4.36a and 4.36b and yield the following minimum expressions for function  $Z$ :

$$Z_{SOP} = \bar{B}\bar{C}DE\bar{F} + \bar{A}\bar{C}\bar{E}\bar{F} + ACD\bar{E} + ADE\bar{F} + ABF + \bar{B}\bar{F}$$

$$Z_{POS} = (A + \bar{B} + \bar{E} + F)(A + E + \bar{F})(A + D + \bar{F})(\bar{A} + \bar{B} + C + E + F) \cdot (\bar{A} + \bar{B} + D + F)(B + D + \bar{F})(B + \bar{E} + \bar{F})(B + C + \bar{F})(A + \bar{B} + \bar{C}). \quad (4.52)$$

**FIGURE 4.36**

Fourth-order EV K-maps for the six-variable function  $Z$  in Eq. (4.51), showing (a) minimum SOP cover and (b) minimum POS cover.

**FIGURE 4.37**

An  $A/B||CD/EF$  format for the six-variable function of Eq. (4.51) and Fig. 4.36a, showing minimum SOP cover.

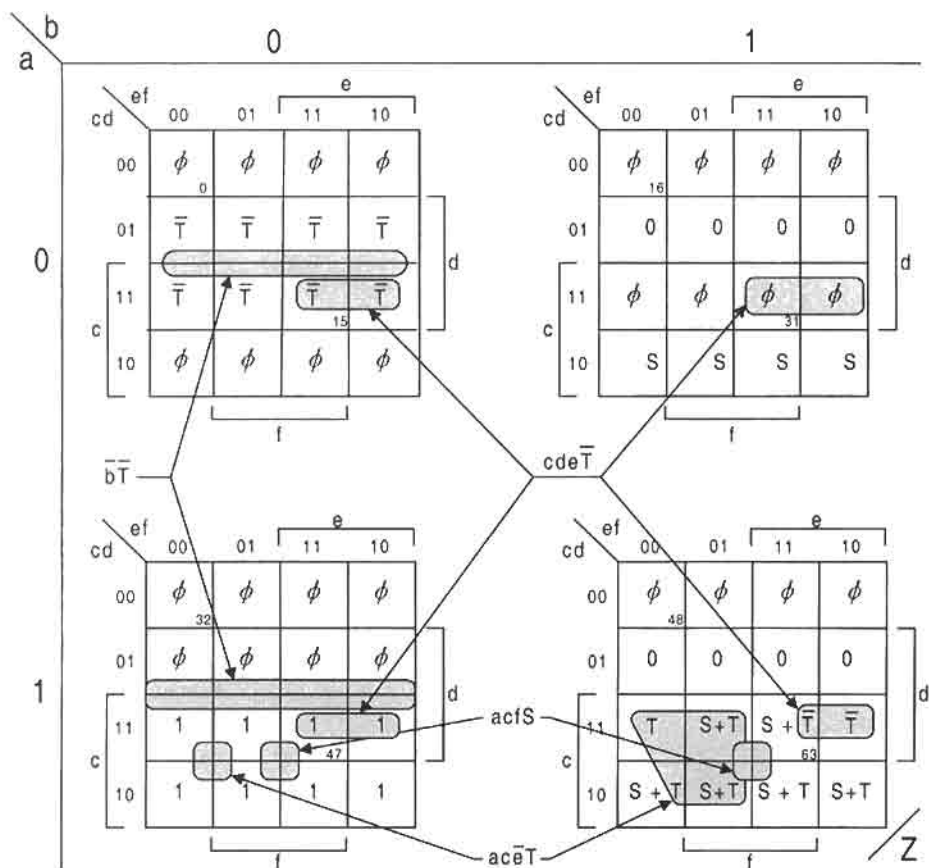
Notice that the SOP form of the equivalence function,  $E \odot F = \bar{E}\bar{F} + EF$  given by Eq. (3.5), is used in cell 5 to extract minimum cover in minterm code and that the POS form  $(\bar{E} + F) \cdot (E + \bar{F})$  is required for extraction of minimum cover in maxterm code. Note also that the loop-out protocol is applied first to the EVs and then to the 1's (in minterm code) and 0's (in maxterm code) as "clean-up" operations. This protocol procedure is recommended to avoid possible redundancy.

There are other K-map formats that can be used to extract reduced or minimum cover for a given function. Consider again the six-variable function given in Eq. (4.51). Presented in Fig. 4.37 is the  $A/B||CD/EF$  format for the conventional (1's and 0's) mapping of this function where only minterm code extraction is considered. Observe that extraction of the EPIs takes on a three-dimensional (3-D) character in a 2-D layout, which can be somewhat perplexing.

As a final example, the format of Fig. 4.37 is used to deal with the following incompletely specified function of eight variables:

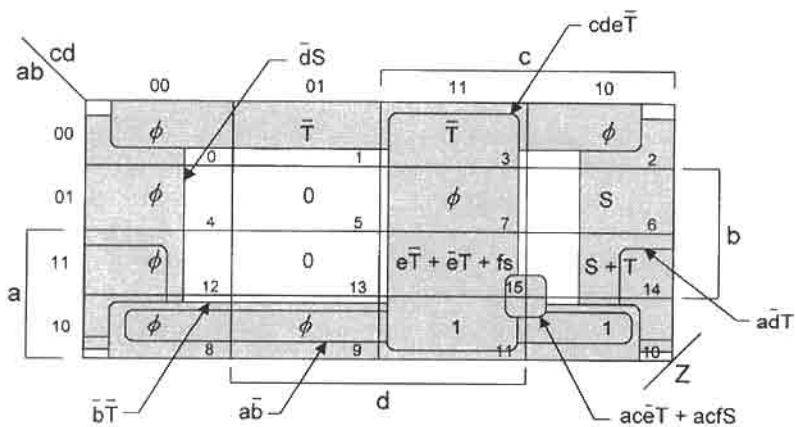
$$\begin{aligned}
 Z(a, b, c, d, e, f, S, T) = \sum m(16, 18, 20, 22, 24, 26, 28, 30, 48, 50, 52, 54, 56, 58, 60, \\
 62, 98, 99, 102, 103, 106, 107, 110, 111, 160-191, \\
 225-227, 229-231, 233-235, 237-239, \\
 241, 243, 245-247, 248, 250, 252, 254, 255) \\
 + \phi(0-15, 32-47, 64-79, 112-159, 192-207). \quad (4.53)
 \end{aligned}$$





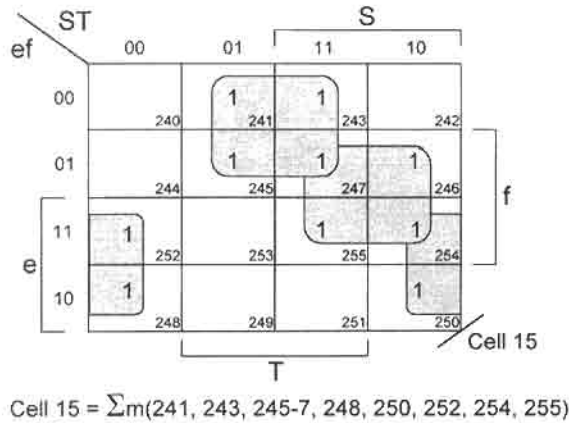
**FIGURE 4.38**

An  $a/b||cd/ef$  EV format for an eight-variable function  $Z$  given by Eq. (4.53).



**FIGURE 4.39**

$ab/cd$  EV K-map showing minimum SOP cover for the eight-variable function represented in Fig. 4.38 and by Eq. (4.53).

**FIGURE 4.40**

Submap for Cell 15 of Fig. 4.39 showing minimum subfunction cover.

Presented in Fig. 4.38 is the second-order compression of this function (Map Key = 4) by using the format  $a/b||cd/ef$ , where  $S$  and  $T$  are the EVs. The minimized result, as extracted from Fig. 4.38, is given by

$$Z_{SOP} = cde\bar{T} + ac\bar{e}T + acfS + a\bar{d}T + \bar{d}S + \bar{b}\bar{T} + \bar{a}\bar{b}, \quad (4.54)$$

where, for clarity's sake, only the loopings for the first three and sixth p-terms are shown. Here again the p-terms are given in the order determined by the loop-out protocol first for the EVs then for the 1's as a "clean-up" operation. Note that the term  $\bar{a}\bar{b}$  covers all the 1's and don't cares in cells 32 through 47 of Fig. 4.38, but is not shown.

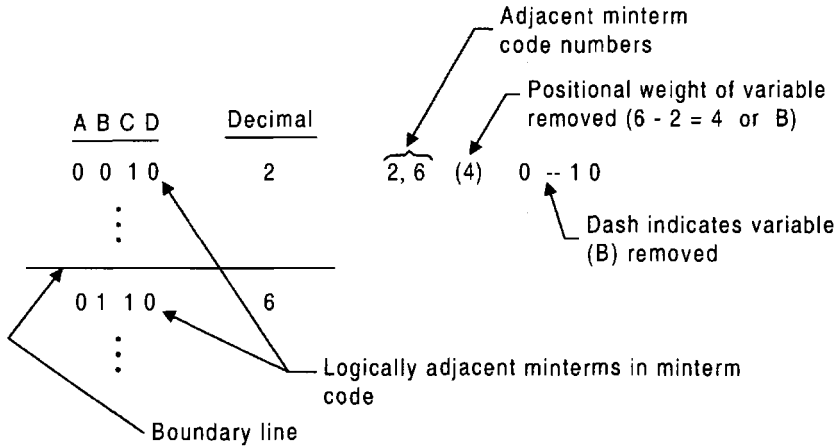
Next, the function of Eq. (4.53) is compressed into the fourth-order K-map of Fig. 4.39, a fourth-order compression (Map Key = 16). The same minimum result given by Eq. (4.54) is easily obtained from Fig. 4.39 as indicated by the shaded loopings and verified by Boozer. To understand the entry in Cell 15 of Fig. 4.39, a submap for this cell is provided in Fig. 4.40. The last line of essential minterms in Eq. (4.53) pertains to Fig. 4.40.

## 4.8 MINIMIZATION ALGORITHMS AND APPLICATION

Tabular methods for function minimization have been devised that can be implemented by a computer and can therefore be used to minimize functions having a large number of input variables. One such method has become known as the *Quine–McCluskey* (Q-M) algorithm. Typical of these methods, the Q-M algorithm first finds the prime implicants (PIs) and then generates the minimum cover. Another important minimization algorithm is a heuristic-type algorithm called *Espresso*. This section will provide a description of these two algorithms together with simple illustrative applications.

### 4.8.1 The Quine–McCluskey Algorithm

To understand the Q-M algorithm, it is helpful to review the tabular format and notation that is unique to it. In Fig. 4.41 is shown the Q-M notation that will be used in the two examples

**FIGURE 4.41**

Quine-McCluskey (Q-M) notation for PI determination.

that follow. Notice that the Q-M notation uses minterm code, minterm code numbers, and positional weights for PI determination.

**EXAMPLE 1** Consider the minimization of the incompletely specified function

$$Y(A, B, C, D) = \sum m(0, 1, 4, 6, 8, 14, 15) + \phi(2, 3, 9). \quad (4.55)$$

In the Q-M algorithm the  $\phi$ 's are treated as essential minterms, and minterm sets  $k$  are compared with sets  $(k + 1)$  in a linear and exhaustive manner. The first step in the application of the Q-M algorithm is presented in Fig. 4.42. Here, a check mark ( $\checkmark$ ) indicates that an implicant is covered by a PI in the column to the right and, therefore, cannot itself be a PI. Thus, unchecked terms in columns 4 and 6 are the indicated PIs and those that are lined out are redundant.

The second step in the application of the Q-M method is the identification of the essential prime implicants (EPIs). Presented in Fig. 4.43 is a table of the PIs (taken from Fig. 4.42) vs the essential minterms in Eq. (4.55). The check mark ( $\checkmark$ ) within the table indicates that a given minterm is covered by a PI. The EPIs are selected from a minimum set of PIs that cover all of the essential minterms of the function  $Y$  in Eq. (4.55) and are presented in Eq. (4.56):

$$Y = ABC + \bar{B}\bar{C} + \bar{A}\bar{D}. \quad (4.56)$$

This result can be easily verified by the conventional K-map extraction method described in Section 4.4.

**EXAMPLE 2** In this example a minimum POS result is required for the incompletely specified function

$$F(W, X, Y, Z) = \sum m(0, 1, 4, 5, 11, 12, 13, 14, 15) + \phi(2, 7, 9) \quad (4.57)$$

$$= \prod M(3, 6, 8, 10) \cdot \phi(2, 7, 9). \quad (4.58)$$

No. of  
1's

|   |   |             |    |                  |                                               |   |                    |                                                   |           |
|---|---|-------------|----|------------------|-----------------------------------------------|---|--------------------|---------------------------------------------------|-----------|
| 0 | { | <u>0000</u> | 0  | 0,1 (1)          | $\bar{A} \bar{B} \bar{C} \bar{D}$             | ✓ | 0,1,2,3            | $\bar{A} \bar{B} \bar{C} \bar{D}$                 | 0 0 -- -- |
| 1 | { | 0001        | 1  | 0,2 (2)          | $\bar{A} \bar{B} \bar{C} D$                   | ✓ | 0,1,8,9            | $\bar{A} \bar{B} \bar{C} D$                       | -- 0 0 -- |
|   |   | 0010        | 2  | 0,4 (4)          | $\bar{A} \bar{B} C \bar{D}$                   | ✓ | <del>0,2,1,3</del> | <del><math>\bar{A} \bar{B} C \bar{D}</math></del> |           |
|   |   | 0100        | 4  | <u>0,8 (8)</u>   | <u><math>\bar{A} B \bar{C} \bar{D}</math></u> | ✓ | 0,2,4,6            | $\bar{A} B \bar{C} \bar{D}$                       | 0 -- -- 0 |
|   |   | 1000        | 8  | 1,3 (2)          | $A \bar{B} \bar{C} \bar{D}$                   | ✓ | <del>0,4,2,6</del> | <del><math>A \bar{B} \bar{C} \bar{D}</math></del> |           |
| 2 | { | 0011        | 3  | 1,9 (8)          | $\bar{A} \bar{B} C D$                         | ✓ | <del>0,8,1,9</del> | <del><math>\bar{A} \bar{B} C D</math></del>       |           |
|   |   | 0110        | 6  | 2,3 (1)          | $\bar{A} B C \bar{D}$                         | ✓ |                    |                                                   |           |
|   |   | 1001        | 9  | 2,6 (4)          | $A \bar{B} C \bar{D}$                         | ✓ |                    |                                                   |           |
| 3 | { | <u>1110</u> | 14 | 4,6 (2)          | $A B \bar{C} \bar{D}$                         | ✓ |                    |                                                   |           |
| 4 | { | <u>1111</u> | 15 | 8,9 (1)          | $A B C \bar{D}$                               | ✓ |                    |                                                   |           |
|   |   |             |    | <u>6,14 (8)</u>  | <u><math>\bar{A} B C \bar{D}</math></u>       |   | 6,14               | $\bar{A} B C \bar{D}$                             |           |
|   |   |             |    | <u>14,15 (1)</u> | <u><math>A B C \bar{D}</math></u>             |   | 14,15              | $A B C \bar{D}$                                   |           |

✓ Indicates that an implicant is covered by a Prime Implicant in the columns to the right.

FIGURE 4.42

Determination of PIs for the 1's in the function  $Y$  of Eq. (4.55).

To do this, the 0's of Eq. (4.58) will be treated as 1's, as required by the Q-M algorithm, to yield  $\bar{F}_{POS}$  in minimum SOP form. Then, application of DeMorgan's law, given by Eqs. (3.15), yields the results  $\bar{\bar{F}}_{POS} = F_{POS}$  by involution. Here, the  $\phi$ 's in Eq. (4.58) are treated as essential minterms, not as nonessential maxterms. Shown in Fig. 4.44 is the tabular determination of PIs for the 0's, treated as 1's, in the maxterm form of function  $F$  given by Eq. (4.58).

The final step is to tabulate the PIs of Fig. 4.44 with the maxterms (now treated as minterms) in Eq. (4.58) to obtain the EPIs for the function  $\bar{F}_{POS}$ . This is done in Fig. 4.45,

|         | 0 | 1 | 4 | 6 | 8 | 14 | 15 | Essential PIs                                 |
|---------|---|---|---|---|---|----|----|-----------------------------------------------|
| PIs     |   |   |   |   |   |    |    |                                               |
| 0,1,2,3 | ✓ | ✓ |   |   |   |    |    |                                               |
| 0,1,8,9 | ✓ | ✓ |   |   | ✓ |    |    | → $\bar{B} \bar{C} \bar{D} = \bar{B} \bar{C}$ |
| 0,2,4,6 | ✓ |   | ✓ | ✓ |   |    |    | → $\bar{A} \bar{C} \bar{D} = \bar{A} \bar{D}$ |
| 6,14    |   |   |   | ✓ |   | ✓  |    |                                               |
| 14,15   |   |   |   |   |   | ✓  | ✓  | → $A B C \bar{D} = A B C$                     |

FIGURE 4.43

Table of PIs (from Fig. 4.42) vs minterms for the function  $Y$  of Eq. (4.55) showing the resulting EPIs.

| No.<br>of 1's |   |         |          |                             |   |                    |                                                   |
|---------------|---|---------|----------|-----------------------------|---|--------------------|---------------------------------------------------|
| 1             | { | 0010 2  | 2,3 (1)  | $\bar{W} \bar{X} Y \bar{Z}$ | ✓ | 2,3,6,7            | $\bar{W} \bar{Z} Y \bar{Z}$                       |
|               |   | 1000 8  | 2,6 (4)  | $\bar{W} \bar{Z} Y \bar{Z}$ | ✓ | <del>2,6,3,7</del> | <del><math>\bar{W} \bar{Z} Y \bar{Z}</math></del> |
| 2             | { | 0011 3  | 2,10 (8) | $\bar{W} \bar{X} Y \bar{Z}$ | ✓ | 2,10               | $\bar{W} \bar{X} Y \bar{Z}$                       |
|               |   | 0110 6  | 8,9 (1)  | $W \bar{X} \bar{Y} \bar{Z}$ | ✓ | 8,9                | $W \bar{X} \bar{Y} \bar{Z}$                       |
|               |   | 1001 9  | 8,10 (2) | $W \bar{X} \bar{Z}$         | ✓ | 8,10               | $W \bar{X} \bar{Z}$                               |
|               |   | 1010 10 | 3,7 (4)  | $\bar{W} \bar{Z} Y \bar{Z}$ | ✓ |                    |                                                   |
| 3             | { | 0111 7  | 6,7 (1)  | $\bar{W} X Y \bar{Z}$       | ✓ |                    |                                                   |

✓ Indicates that an implicant is covered by a Prime Implicant in the columns to the right.

**FIGURE 4.44**

Tabular determination of PIs for the 0's (treated as 1's) in function  $F$  of Eq. (4.58).

giving the final results

$$\begin{aligned}\bar{F}_{POS} &= \bar{W}Y + W\bar{X}\bar{Z} \\ \bar{\bar{F}}_{POS} = F_{POS} &= (W + \bar{Y})(\bar{W} + X + Z).\end{aligned}\quad (4.59)$$

Notice that the PI (2, 3, 6, 7) is the EPI  $\bar{W}Y$  and that the remaining maxterms (treated as minterms) are covered by the PI (8,10), the minimum set of PIs covering all minterms.

Had the Q-M algorithm been applied to Eq. (4.57), the minimum SOP result would be

$$F_{SOP} = \bar{W}\bar{Y} + WX + WZ, \quad (4.60)$$

which is algebraically equal to the POS result of Eq. (4.59). The reason for this is that the application of the Q-M algorithm uses the three  $\phi$ 's in the same way for the two cases, a feature of the Q-M method. As a general rule, this is rarely the case for SOP and POS minimized forms of incompletely specified functions obtained by other methods.

|            | 3 | 6 | 8 | 10 |                                           |
|------------|---|---|---|----|-------------------------------------------|
| <u>Pis</u> |   |   |   |    | <u>Essential Pis</u>                      |
| 2,3,6,7    | ✓ | ✓ |   |    | → $\bar{W} \bar{Z} Y \bar{Z} = \bar{W} Y$ |
| 2,10       |   |   |   | ✓  |                                           |
| 8,9        |   |   | ✓ |    |                                           |
| 8,10       |   |   | ✓ | ✓  | → $W \bar{X} \bar{Z} = W \bar{X} \bar{Z}$ |

**FIGURE 4.45**

Table of PIs (from Fig. 4.44) vs maxterms treated as minterms for function  $F$  of Eq. (4.58) showing the essential PIs.

### 4.8.2 Cube Representation and Function Reduction

The *cube notation* is commonly used in CAD programs and, in fact, is the notation that is used in the computer implementation of the Q-M algorithm described in Subsection 4.8.1. In this notation an  $n$ -dimensional cube has  $2^n$  vertices formed by the intersection of  $n$  dimensional lines. Most commonly one thinks of a cube as three-dimensional (a 3-cube) having  $2^3 = 8$  vertices. But the concept is much more general, extending to  $n$  dimensions that generally cannot be easily visualized by a geometrical figure.

Cube representation is usually based on minterm code. Thus, the minterms of a switching function can be mapped onto the  $2^n$  vertices of an  $n$ -dimensional cube such that each pair of adjacent vertices differ by exactly one bit position. As an example, consider implicants (2, 3) and (6, 7) listed in the Q-M example of Fig. 4.44. In minterm code cube notation, these implicants would be represented as (0010, 0011) and (0110, 0111), respectively. Reduction of these implicants to PI ( $r$ -cube) form occurs between adjacencies (adjacent vertices) as follows:

$$0010 + 0011 = 001- = \bar{W}\bar{X}Y \quad \text{and} \quad 0110 + 0111 = 011- = \bar{W}XY$$

or, finally,

$$(001-) + (011-) = 0-1- = \bar{W}Y,$$

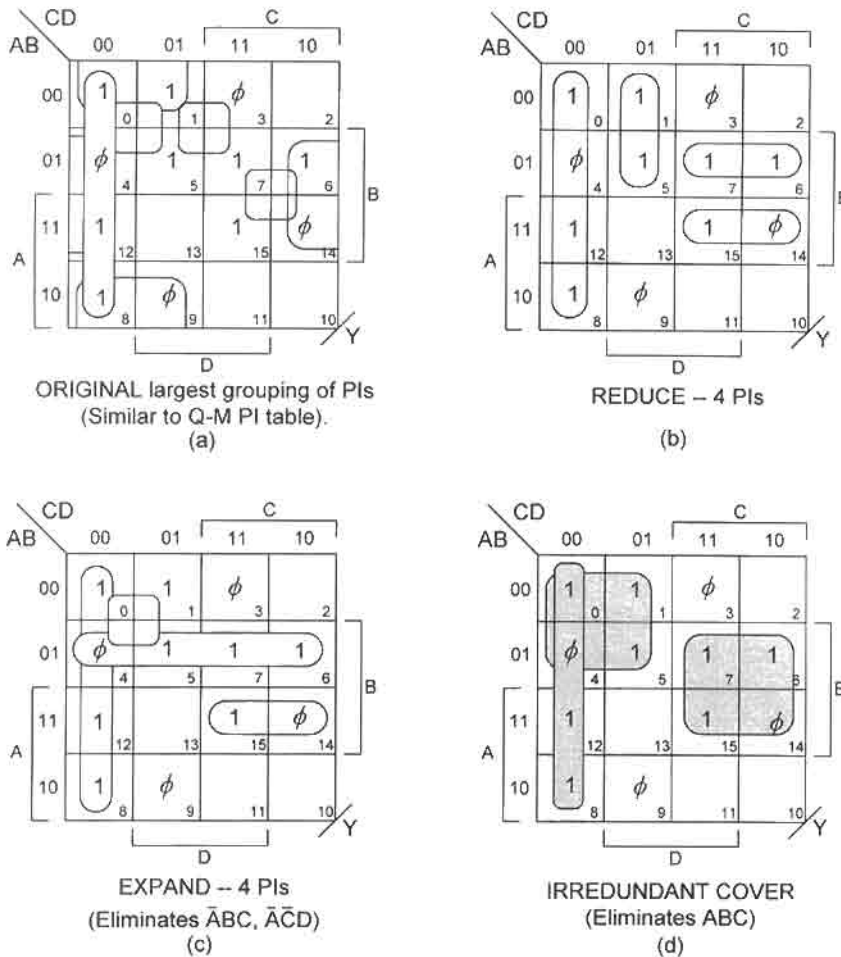
where 0 represents the complemented variable, 1 is the uncomplemented variable, and the “-” symbol represents an irrelevant input variable (representing both 1 and 0). Thus, in general, an  $r$ -cube of an  $n$ -variable function is produced by combining  $2^r$  adjacent minterms, thereby eliminating  $r$  variables in a function reduction process.

### 4.8.3 Qualitative Description of the Espresso Algorithm

The two-level minimization algorithm called *Espresso* belongs to a class of minimization algorithms that use heuristic logic methods as opposed to the linear exhaustive PI search of the Q-M method. In effect, all heuristic methods group, expand and regroup adjacent minterms over a number of iterations until an optimal or near-optimal grouping, called the *irredundant set*, is found. The exact strategies used and the order in which they are used depends on the particular algorithm.

Though a detailed description of the Espresso algorithm is beyond the scope of this text, the principal steps involved can be qualitatively understood by the K-maps in Fig. 4.46. Here, the four basic steps of the Espresso algorithm are represented by four fourth-order K-maps labeled ORIGINAL, REDUCE, EXPAND and IRRREDUNDANT. The ORIGINAL function, plotted in Figure 4.46a, is the graphical equivalent to the PI table of the Q-M method since it represents the largest number of prime implicants, that is, six PIs. The original function is then regrouped to form a smaller (REDUCED) number of prime implicants (four PIs) in Fig. 4.46b and then EXPANDED (RESHAPED) to form four PIs by eliminating two PIs. Notice that the cardinality is preserved in the REDUCE-to-EXPAND step. Finally, an IRRREDUNDANT set is found by regrouping and eliminating yet another PI, resulting in only three EPis. This irredundant set is said to have minimum *cardinality*, that is, minimum cover.

The Espresso algorithm just described qualitatively is usually called Espresso-II. Since its inception, various improvements have been made, adding to the speed and multiple-output

**FIGURE 4.46**

Four-variable K-maps for function  $Y$  illustrating four steps of the Espresso algorithm: (a) ORIGINAL, (b) REDUCE, (c) EXPAND, and (d) IRREDUNDANT COVER.

capability. These improved Espresso algorithms include the two referred to as Espresso-EXACT and Espresso-MV. A detailed description of these and other closely related algorithms can be found in references cited at the end of this chapter.

#### 4.9 FACTORIZATION, RESUBSTITUTION, AND DECOMPOSITION METHODS

Beyond the SOP or POS minimum result, it is possible to further reduce the gate/input tally, reduce the number of inverters, and reduce the gate fan-in requirements for some functions by using a technique called *factoring* or *factorization*. The results of factorization lead to *multilevel* forms that are *hybrids*, since they cannot be classified as either purely SOP or POS. A multilevel logic realization of a function is one involving more than two levels of

gate path delay excluding possible inverters. The logic circuits considered to this point are classified as two-level.

#### 4.9.1 Factorization

The method of factorization is illustrated by using a simple example. Consider the minimum SOP function

$$F = ABC\bar{C} + AD + \bar{B}D + \bar{C}D, \quad (4.61)$$

which requires a gate/input tally of 5/13 excluding inverters. This requires four ANDing operations with a maximum of three inputs per gate, and one ORing operation requiring four inputs. Also, if it is assumed that the inputs arrive active high, two inverters are required, bringing the total gate/input tally to 7/15. Now suppose that it is desirable to limit the fan-in to a maximum of three inputs per gate and to eliminate the need to use inverters in generating the complements of  $B$  and  $C$  variables. This can be accomplished by factorization of Eq. (4.61) in the following manner:

$$\begin{aligned} F &= ABC\bar{C} + AD + \bar{B}D + \bar{C}D \\ &= AB(\bar{B} + \bar{C}) + AD + (\bar{B} + \bar{C})D \\ &= AB(\overline{BC}) + AD + (\overline{BC})D. \end{aligned} \quad (4.62)$$

The term  $ABC\bar{C}$  is factored as  $ABC\bar{C} = AB(\bar{B} + \bar{C})$ . Notice that if the function of Eq. (4.62) is implemented as a NAND circuit, a gate/input tally of 5/12 would result with a maximum fan-in of 3 with no inverters.

An interesting feature of the factorization method is that there is usually more than one way in which to factor a given function. So it is with Eq. (4.61), which can be factored as a three-level hybrid form in the following alternative way:

$$\begin{aligned} F &= ABC\bar{C} + AD + \bar{B}D + \bar{C}D \\ &= ABC\bar{C} + D(A + \bar{B} + \bar{C}). \end{aligned}$$

If implemented with NAND gates, the gate/input becomes 4/10 plus two inverters, assuming that the inputs arrive active high.

The factorization method can be extended to multioutput systems of the type considered in Section 4.5. The process is illustrated by the following simple example where three optimized functions are factored as indicated:

$$\begin{aligned} f_1 &= \bar{A}\bar{B} + \bar{A}\bar{C} + AB = \bar{A}(\bar{B} + \bar{C}) + AB = \bar{A}(\overline{BC}) + AB \\ f_2 &= \bar{A}\bar{B}C + \bar{B}\bar{C} + AB = \bar{B}(\bar{A} + \bar{C}) + AB = \bar{B}(\overline{AC}) + AB \\ f_3 &= \bar{B}C + \bar{A}BC + B\bar{C} = C(\bar{B} + \bar{C}) + B(\bar{A} + \bar{C}) = C(\overline{BC}) + B(\overline{AC}). \end{aligned} \quad (4.63)$$

Here, terms in expressions for  $f_2$  and  $f_3$  are factored as  $\bar{A}\bar{B}C + \bar{B}\bar{C} = \bar{B}(\bar{A}C + \bar{C}) = \bar{B}(\bar{A} + \bar{C}) = \bar{B}(\overline{AC})$  and  $\bar{B}C = C(\bar{B} + \bar{C}) = C(\overline{BC})$ . With NAND logic and assuming the inputs arrive active high, the total gate/input tally for the factored expressions is 12/20, including one inverter, with fan-in requirements of two inputs per gate. In comparison, the



original optimized SOP expressions require a gate/input tally of 14/30, including three inverters, and have a maximum fan-in requirement of 3.

The factorized expressions of Eqs. (4.63) are three-level functions, whereas the original SOP expressions are two-level. This brings up other aspects of the optimization problem, namely the *design area* (real estate usage) vs *delay* (performance), as discussed in Section 4.10.

#### 4.9.2 Resubstitution Method

The Boolean resubstitution method possesses a close resemblance to polynomial division and works to generate multilevel functions that have improved fan-in (hence improved area) requirements. The process of resubstitution begins by finding a good, if not optimal, divisor  $P$  in the expression

$$F = PQ + R, \quad (4.64)$$

where  $F$  is the dividend,  $Q$  is the quotient, and  $R$  is the remainder. Heuristic algorithms exist that can accomplish this, but they are complex and fall outside the scope of this text. However, an attempt will be made to illustrate the resubstitution method with a simple example. Consider the minimized SOP five-variable function

$$F = AB\bar{E} + ABC\bar{D} + CDE + \bar{A}\bar{C}\bar{E} + \bar{A}\bar{B}CD + \bar{A}\bar{B}E + \bar{C}\bar{D}E. \quad (4.65)$$

Noting the appearance of  $AB$ ,  $\bar{A}\bar{B}$ ,  $CD$ ,  $\bar{C}\bar{D}$ ,  $E$ , and  $\bar{E}$  in six of the seven p-terms, the divisor is chosen to be  $P = AB + CD + E$ . The process continues by repeating three steps for each of the seven p-terms:

Step 1. Select term  $AB\bar{E}$ .

Step 2. AND (Boolean multiply)  $AB\bar{E} \cdot P = AB\bar{E} + ABCDE + AB\bar{E}E = AB\bar{E}$ .

Step 3. Delete  $AB$  in  $AB\bar{E} \cdot P$  to yield term  $\bar{E} \cdot P$ .

Step 1. Select term  $ABC\bar{D}$ .

Step 2. AND  $ABC\bar{D} \cdot P = ABC\bar{D} + ABC\bar{D}E = ABC\bar{D}$ .

Step 3. Delete  $AB$  in  $ABC\bar{D} \cdot P$  to yield term  $\bar{C}\bar{D} \cdot P$ .

Repeat Steps 1, 2, and 3 for the remaining five terms in the order given by Eq. (4.65):

$CDE \cdot P = ABCDE + CDE = CDE$ . Delete  $CD$  in  $CDE \cdot P$  to yield  $E \cdot P$ .

$\bar{A}\bar{C}\bar{E} \cdot P = 0$ . Thus, no literals can be deleted in  $\bar{A}\bar{C}\bar{E} \cdot P$ .

$\bar{A}\bar{B}CD \cdot P = \bar{A}\bar{B}CD + \bar{A}\bar{B}CDE = \bar{A}\bar{B}CD$ . Delete  $CD$  in  $\bar{A}\bar{B}CD \cdot P$  yield  $\bar{A}\bar{B} \cdot P$ .

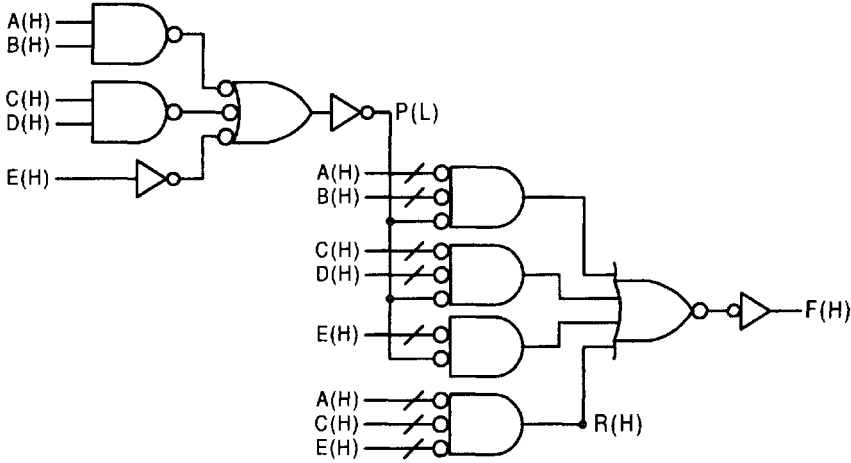
$\bar{A}\bar{B}E \cdot P = \bar{A}\bar{B}CDE + \bar{A}\bar{B}E = \bar{A}\bar{B}E$ . Delete  $E$  in  $\bar{A}\bar{B}E \cdot P$  to yield  $\bar{A}\bar{B} \cdot P$ .

$\bar{C}\bar{D}E \cdot P = ABC\bar{D}E + \bar{C}\bar{D}E = \bar{C}\bar{D}E$ . Delete  $E$  in  $\bar{C}\bar{D}E \cdot P$  to yield  $\bar{C}\bar{D} \cdot P$ .

In the preceding set of steps it should be observed that the only literals that can be deleted are those that appear as p-terms in the divisor  $P$ . Also, it should be noted that the choice of divisor  $P$  is somewhat arbitrary, since there are other combination of terms that can be used in the resubstitution process.

The final results of resubstitution are expressed by the partition

$$\begin{aligned} F &= \bar{A}\bar{B}P + \bar{C}\bar{D}P + \bar{E}P + \bar{A}\bar{C}\bar{E} \\ &= PQ + R, \end{aligned} \quad (4.66)$$

**FIGURE 4.47**

NAND/NOR/INV realization of the partitioned function given by Eq. (4.66).

where  $P = AB + CD + E$ ,  $Q = \bar{A}\bar{B} + \bar{C}\bar{D} + \bar{E}$  and  $R = \bar{A}\bar{C}\bar{E}$ . Function  $F$ , expressed by Eqs. (4.66), represents four levels of path delay, as shown implemented by NAND/NOR/INV logic in Fig. 4.47 where it is assumed that all inputs arrive active high. Notice that the gate/input tally is now 11/25, including three inverters, and that only one gate has a fan-in of 4. If a fan-in limitation of 4 is also applied to the original two-level SOP expression in Eq. (4.65), a three-level circuit results having a gate/input tally of 14/35, including five inverters, and four gates with a fan-in of 4. Thus, the partitioned function of Fig. 4.47 has an improved design area factor but not necessarily an improved performance. A discussion of the design area vs performance factors is given in Section 4.10.

The resubstitution method just described bears similarity to portions of some heuristic two-level minimization algorithms such as Espresso II, qualitatively described in Subsection 4.8.3. In particular, the introduction of a new literal, divisor term  $P$  in step 2 and the subsequent deletion of literals in step 3 of resubstitution is a generalization of the REDUCE and EXPAND processes in Espresso II. In these processes, Espresso seeks to add literals existing in one product term of the original expression to other candidate terms so that implicants covered by a given expanded implicant can be deleted. Thus, by repeated introduction of divisor  $P$  followed by deletions of redundant terms, the resubstitution process seeks a more optimum result, not unlike the heuristic processes in Espresso.

#### 4.9.3 Decomposition by Using Shannon's Expansion Theorem

Shannon's expansion theorem states that any Boolean function of  $n$  variables  $f(x_{n-1}, \dots, x_2, x_1, x_0)$  can be decomposed into the SOP form

$$\begin{aligned}
 f(x_{n-1}, \dots, x_2, x_1, x_0) &= \bar{x}_i f(x_{n-1}, \dots, x_{i+1}, 0, x_{i-1}, \dots, x_2, x_1, x_0) \\
 &\quad + x_i f(x_{n-1}, \dots, x_{i+1}, 1, x_{i-1}, \dots, x_2, x_1, x_0) \\
 &= \bar{x}_i f_{\bar{x}_i}^{SOP} + x_i f_{x_i}^{SOP}
 \end{aligned} \tag{4.67}$$

or into its dual POS form

$$\begin{aligned} f(x_{n-1}, \dots, x_2, x_1, x_0) &= [\bar{x}_i + f(x_{n-1}, \dots, x_{i+1}, 1, x_{i-1}, \dots, x_2, x_1, x_0)] \\ &\quad \cdot [x_i + f(x_{n-1}, \dots, x_{i+1}, 0, x_{i-1}, \dots, x_2, x_1, x_0)] \\ &= [\bar{x}_i + f_{\bar{x}_i}^{POS}] \cdot [x_i + f_{x_i}^{POS}], \end{aligned} \quad (4.68)$$

where  $f_{\bar{x}_i}^{SOP}$  and  $f_{x_i}^{SOP}$  are the cofactors for  $\bar{x}_i$  and  $x_i$  in Eq. (4.67), and  $f_{\bar{x}_i}^{POS}$  and  $f_{x_i}^{POS}$  are the cofactors for  $\bar{x}_i$  and  $x_i$  in Eq. (4.68).

Proof of Eqs. (4.67) and (4.68) is easily obtained by setting  $x_i = 1$  and then  $x_i = 0$  and observing that in each case the surviving cofactors are identical to the left side of the respective equation. For example, setting  $x_i = 1$  in Eq. (4.67) leads to

$$f(x_{n-1}, \dots, x_{i+1}, 1, x_{i-1}, \dots, x_2, x_1, x_0) = f(x_{n-1}, \dots, x_{i+1}, 1, x_{i-1}, \dots, x_2, x_1, x_0),$$

since  $\bar{x}_i = 0$  when  $x_i = 1$ .

Multiple applications of Eqs. (4.67) and (4.68) are possible. For example, if decomposition is carried out with respect to two variables,  $x_1$  and  $x_0$ , Eq. (4.67) becomes

$$\begin{aligned} f(x_{n-1}, \dots, x_2, x_1, x_0) &= \bar{x}_1 \bar{x}_0 f(x_{n-1}, \dots, x_2, 0, 0) + \bar{x}_1 x_0 f(x_{n-1}, \dots, x_2, 0, 1) \\ &\quad + x_1 \bar{x}_0 f(x_{n-1}, \dots, x_2, 1, 0) + x_1 x_0 f(x_{n-1}, \dots, x_2, 1, 1) \\ &= m_0 f(x_{n-1}, \dots, x_2, \underline{m_0}) + m_1 f(x_{n-1}, \dots, x_2, \underline{m_1}) \\ &\quad + m_2 f(x_{n-1}, \dots, x_2, \underline{m_2}) + m_3 f(x_{n-1}, \dots, x_2, \underline{m_3}), \end{aligned}$$

or generally for decomposition with respect to  $(x_{k-1}, \dots, x_2, x_1, x_0)$ ,

$$f(x_{n-1}, \dots, x_2, x_1, x_0) = \sum_{i=0}^{2^k-1} m_i(x_{n-1}, \dots, x_2, x_1, x_0) \cdot f(x_{n-1}, \dots, x_k, \underline{m_i}). \quad (4.69)$$

Here,  $m_i$  are the canonical ANDed forms of variables  $x_j$  taken in ascending minterm code order from  $i = 0$  to  $(2^k - 1)$ , and  $\underline{m_i}$  represents their corresponding minterm code. As an example, decomposition with respect to variables  $(x_2, x_1, x_0)$  gives

$$\begin{aligned} f(x_{n-1}, \dots, x_2, x_1, x_0) &= \bar{x}_2 \bar{x}_1 \bar{x}_0 f(x_{n-1}, \dots, x_3, 0, 0, 0) \\ &\quad + \bar{x}_2 \bar{x}_1 x_0 f(x_{n-1}, \dots, x_3, 0, 0, 1) + \dots \end{aligned}$$

for  $k = 3$ .

In similar fashion, the dual of Eq. (4.69) is the generalization of Eq. (4.68) given by

$$f(x_{n-1}, \dots, x_2, x_1, x_0) = \prod_{i=0}^{2^k-1} [M_i(x_{n-1}, \dots, x_2, x_1, x_0) + f(x_{n-1}, \dots, x_k, \underline{M_i})], \quad (4.70)$$

where now  $M_i$  represents the canonical ORed forms of variables  $x_j$  and  $\underline{M_i}$  represents their corresponding maxterm code.

As a practical example of the application of Shannon's expansion theorem, consider the function

$$F(A, B, C, D) = \sum m(1, 3, 4, 5, 9, 10, 13, 14, 15), \quad (4.71)$$

which is represented in the truth table of Fig. 4.48a and in the K-map of Fig. 4.48b. Applying Eq. (4.69) for decomposition with respect to variables  $C$  and  $D$  gives the cofactors

$$\left\{ \begin{array}{l} F_0 = (A, B, 0, 0) = \bar{A}B \\ F_1 = (A, B, 0, 1) = 1 \\ F_2 = (A, B, 1, 0) = A \\ F_3 = (A, B, 1, 1) = A \odot B \end{array} \right\},$$

from which the function  $F$  can be written as

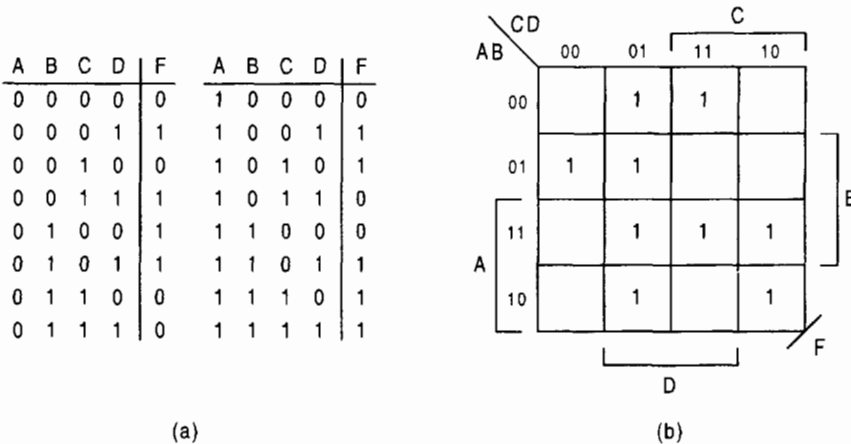
$$\begin{aligned} F_{CD}(A, B, C, D) &= \bar{C}\bar{D}(\bar{A}B) + \bar{C}D(1) + C\bar{D}(A) + CD(A \odot B) \\ &= \bar{A}B\bar{C}\bar{D} + \bar{C}D + AC\bar{D} + (A \odot B)CD, \end{aligned}$$

which could have been deduced directly from an inspection of the truth table or K-map in Fig. 4.48.

But the variables about which the function is to be decomposed are a matter of choice. If it is required that the function  $F$  be decomposed with respect to variables  $A$  and  $B$ , the result would be

$$F_{AB}(A, B, C, D) = \bar{A}\bar{B}(D) + \bar{A}B(\bar{C}) + A\bar{B}(C \oplus D) + AB(C + D),$$

which, like the previous result, can be read directly from either the truth table or the K-map. Note that decompositions of the type just described can be very useful in implementing



**FIGURE 4.48**

Truth table (a) and K-map (b) for the function  $F$  given by Eq. (4.71).

functions of a large number of variables by using devices with lesser input capability. The use of multiplexers discussed in Section 6.2 offers a good example of this fact.

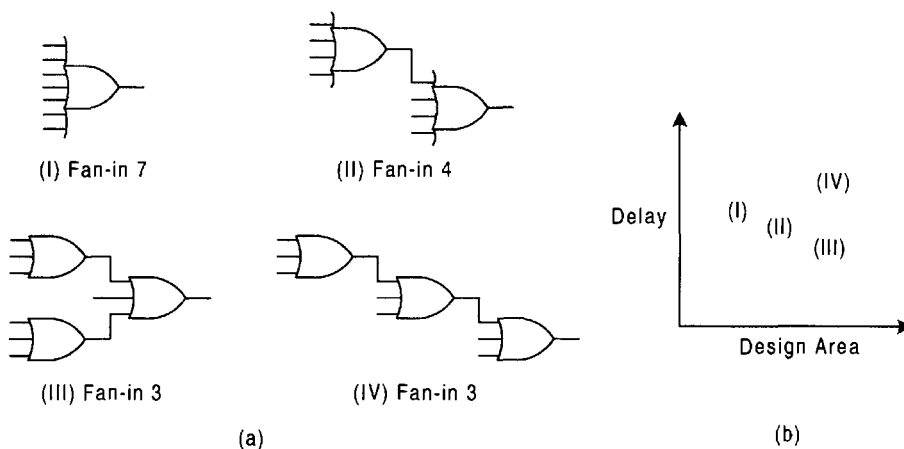
The process of decomposition can be reversed to yield a purely SOP or purely POS expression from the decomposed expression. This is illustrated by factoring out  $\bar{A}$ ,  $A$ ,  $\bar{B}$ , and  $B$  in turn from  $F_{AB}$  to yield the SOP expression

$$\begin{aligned}
 F_{AB} &= \bar{A}\bar{B}(D) + \bar{A}B(\bar{C}) + A\bar{B}(C \oplus D) + AB(C + D) \\
 &= \bar{A}[\bar{B}D + B\bar{C}] + A[\bar{B}(C \oplus D) + B(C + D)] \\
 &= \bar{A}\bar{B}[D] + \bar{A}B[\bar{C}] + A\bar{B}[\bar{C}D + C\bar{D}] + AB[C + D] \\
 &= \bar{A}\bar{B}D + \bar{A}B\bar{C} + A\bar{B}\bar{C}D + A\bar{B}C\bar{D} + ABC + ABD,
 \end{aligned}$$

where  $C \oplus D = \bar{C}D + C\bar{D}$  follows from Eq. (3.4). A cursory inspection of the SOP form of  $F_{AB}$  verifies its agreement with Fig. 4.48.

#### 4.10 DESIGN AREA VS PERFORMANCE

It is common to observe an inverse relationship between design area and performance (delay). That is, circuit realizations with improved design area commonly suffer from poorer performance and vice versa. It is known that CMOS gate performance decreases (i.e., delay increases) with increasing numbers of inputs (fan-in). The larger the fan-in, the greater is the path delay through the gate. As an example, consider the function of Eq. (4.65). It has a cardinality of 7 that must be ORed. Shown in Fig. 4.49a are four alternative ORing configurations for seven inputs. It is expected that there exists a *trade-off* between design area and delay for these four configurations, as illustrated in Fig. 4.49b.



**FIGURE 4.49**

Area/delay trade-off for the seven-input function of Eq. (4.65). (a) Four alternative ORing configurations. (b) Possible area/delay trade-off points for configurations (I), (II), (III), and (IV), showing effect of treeing and cascading configurations.

Here, *tree* configuration (III) is expected to show the least delay, but at the cost of greater design area. Tree configuration (II) would seem to have the most favorable area/delay trade-off, while gate (I) and *cascade* configuration (IV) are expected to have the least favorable trade-off. A dual set of ANDing operations would show the same area/delay trade-offs.

#### 4.11 PERSPECTIVE ON LOGIC MINIMIZATION AND OPTIMIZATION

The EV mapping methods described in Sections 4.6 and 4.7 are useful up to three or four orders of map compression. However, with increasing compression order beyond third order, the gap usually widens between the reduced forms obtained and the absolute minimum result. This is especially true if reduced or minimized subfunctions are used to extract cover from such EV K-maps. For this reason a practical limit of four orders of K-map compression (eight variables) is set, and use of reduced or minimum subfunctions is highly recommended. The use of submaps can narrow the gap between a reduced result and one that is an absolute or exact minimum. This fact is implied by the simple examples given in the sections on EV mapping methods.

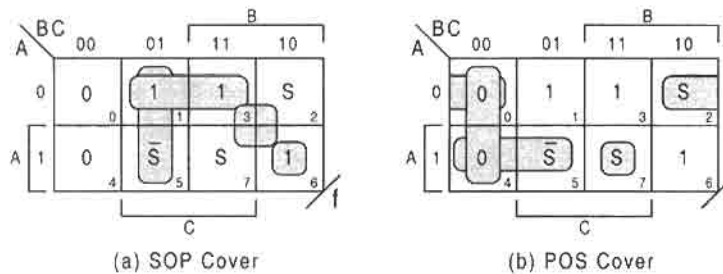
Beyond four orders of compression in fourth-order K-maps, the use of computer algorithmic methods for logic minimization becomes necessary. But even these computer programs have their limitations, particularly with regard to multiple output systems having a large number of input variables. It is an established fact that the generalized optimal solution for an  $n$ -variable function is impossible. The reason for this is that  $2^n$  minterms must be dealt with in some manner or another. Minimization problems in the class referred to as  $\mathcal{P}$  are called *tractable* problems — those for which an optimum or near-optimum solution is possible. Those that are *intractable* belong to the class of problems referred to as  $\mathcal{NP}$ -complete. The search for faster, more robust algorithms to optimize very large multiple output systems continues. These algorithms are most likely to be of the heuristic type. Though the Q-M linear tabular method is useful in introducing readers to the subject of logic minimization, it is of little practical importance given the much-improved heuristic methods now in use.

Finally, it must be said that the SOP (or POS) minimization of a function may not be an end in itself. Section 4.9 demonstrates that optimization may continue beyond minimization by techniques such as factorization and resubstitution that generate multilevel functions. To do this, however, brings into play other factors such as area/delay trade-offs. Thus, there emerge two approaches to function optimization from which the designer must choose: *Optimize design area under delay constraints* or *optimize delay under design area constraints*. It is unlikely that a system can be optimized with respect to both design area and delay, although it may be possible to come close to this for some systems.

#### 4.12 WORKED EV K-MAP EXAMPLES

**EXAMPLE 4.1** Compress the following four-variable function into a third-order K-map and extract minimum SOP and minimum POS cover from it.

$$\begin{aligned} f(A, B, C, S) &= \sum m(2, 3, 5, 6, 7, 10, 12, 13, 15) \\ &= \prod M(0, 1, 4, 8, 9, 11, 14). \end{aligned} \quad (4.72)$$

**FIGURE 4.50**

(a) Minimum SOP cover and (b) minimum POS cover for the function  $f$ .

The compressed third-order K-maps representing SOP and POS cover are shown in Fig. 4.50. Applying the loop-out protocol first to the EV and then to the 1's and 0's gives

$$f_{SOP} = \bar{B}\bar{C}\bar{S} + BS + ABC\bar{C} + \bar{A}\bar{C}$$

$$f_{POS} = (\bar{A} + \bar{B} + \bar{C} + S)(\bar{A} + B + \bar{S})(A + C + S)(B + C)$$

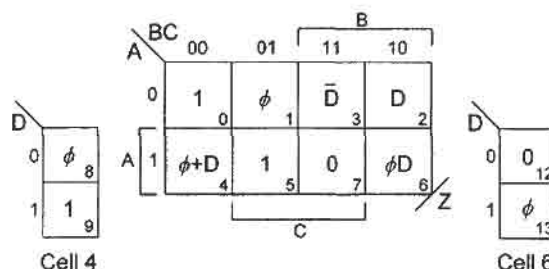
**EXAMPLE 4.2** A four-variable function  $Z$  containing don't cares is shown in the compressed third-order K-map of Fig. 4.51. Two first-order submaps for cells 4 and 6 are also shown to demonstrate that the don't care ( $\phi$ ) is treated as an EV, which it is.

(a) Represent the function  $Z$  in canonical SOP and POS form by using coded notation.

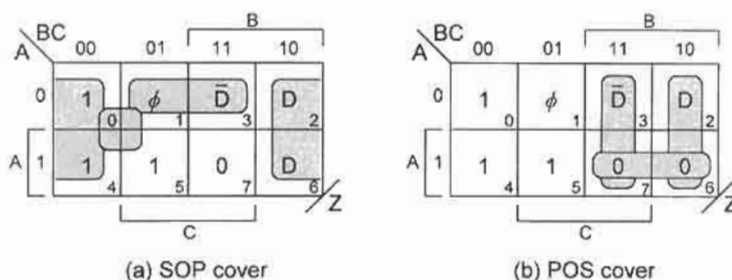
Noting that the Map Key is  $2^{4-3} = 2$ , the results can be written directly in canonical SOP and POS form by counting by 2's or by making use of first-order submaps in D, and by applying the minterm and maxterm codes, respectively. For example, cell 3 represents  $m_6$  or  $M_7$ , cell 4 represents  $(\phi m_8 + m_9)$  or  $\phi M_8$ , and so on. Proceeding in this manner, the results are given by

$$\begin{aligned} Z(A, B, C, D) &= \sum m(0, 1, 5, 6, 9, 10, 11) + \phi(2, 3, 8, 13) \\ &= \prod M(4, 7, 12, 14, 15) \cdot \phi(2, 3, 8, 13), \end{aligned} \quad (4.73)$$

where knowing one canonical form yields the other through observation of the missing numbers in the former.

**FIGURE 4.51**

Compressed K-map for Example 4.2 showing sample first-order submaps.

**FIGURE 4.52**

(a) Minimum SOP cover and (b) minimum POS cover for function  $Z$  of Example 4.2.

(b) Extract minimum SOP and minimum POS cover for the function  $Z$ .

In Fig. 4.52 are the third-order K-maps showing the minimum SOP and minimum POS cover for the function  $Z$ . Notice that the subfunction in cell 6 is interpreted differently in the SOP and POS K-maps.

From reading this cover, the results are

$$Z_{SOP} = \bar{A}C\bar{D} + \bar{C}D + \bar{B}$$

$$Z_{POS} = (\bar{B} + C + D)(\bar{B} + \bar{C} + \bar{D})(\bar{A} + \bar{B}),$$

which are seen to be logically equivalent but not algebraically equal. Notice that the 1's in the SOP K-map are looped out as the octad  $\bar{B}$  by using  $\phi_8 = 1$  in cell 4 of Fig. 4.51 to give  $\phi_8 + D = 1$ . Also, note that the 0 in cell 6 of the POS K-map in Fig. 4.51 is looped out as the quad  $\bar{A} + \bar{B}$  by using  $\phi_{13} = 0$  to give  $\phi_{13}D = 0$ . Thus,  $\phi_{13}$  is used as a 1 for minimum SOP extraction but as a 0 for minimum POS extraction, meaning that the SOP and POS expressions cannot be algebraically equal.

**EXAMPLE 4.3** A four-variable function  $F(A, B, C, D)$  containing don't cares is compressed into the truth table given in Fig. 4.53.

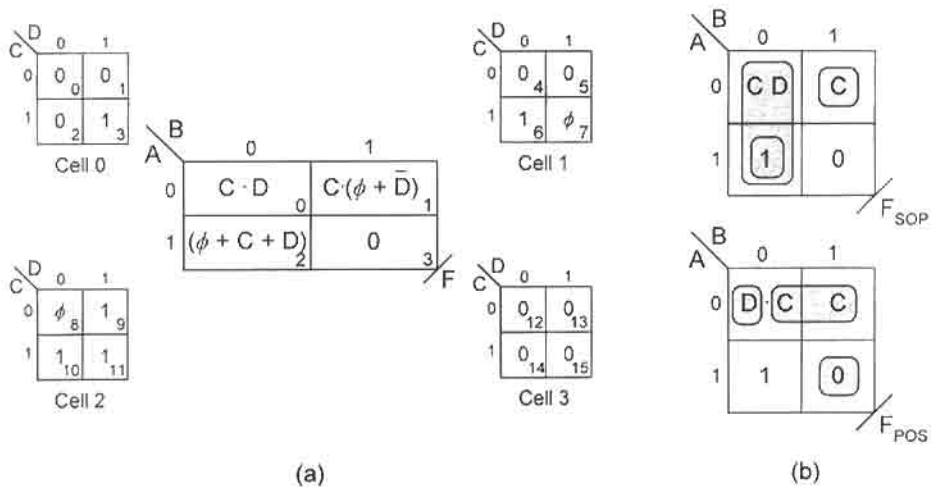
- Represent the function  $F$  in a second-order K-map, and express  $F$  in canonical SOP and POS form by using coded notation.
- By proper interpretations of the don't care subfunctions, loop out the minimum SOP and POS cover from the second-order K-map and give the gate/input tallies for each.

| A | B | F                   |
|---|---|---------------------|
| 0 | 0 | $C\bar{D}$          |
| 0 | 1 | $C(\phi + \bar{D})$ |
| 1 | 0 | $(\phi + C + D)$    |
| 1 | 1 | 0                   |

**FIGURE 4.53**

Compressed truth table for a function  $F$  of four variables.



**FIGURE 4.54**

(a) Second-order compressed K-map and its submaps for the four-variable function given in the EV truth table of Fig. 4.53. (b) EV K-maps showing minimum SOP cover and minimum POS cover.

(a) The simplest means of obtaining the canonical forms from Fig. 4.53 is to use a second-order K-map. Shown in Fig. 4.54a is the second-order compressed K-map together with its submaps for a Map Key of  $2^{4-2} = 4$ . By reading the submaps directly, the canonical forms become

$$\begin{aligned}
 F &= \sum m(3, 6, 9, 10, 11) + \phi(7, 8) \\
 &= \prod M(0, 1, 2, 4, 5, 12, 13, 14, 15) \cdot \phi(7, 8).
 \end{aligned} \quad (4.74)$$

(b) The compressed second-order K-maps for the function  $F$  are given in Fig. 4.54b. From these K-maps the minimum SOP and minimum POS expressions are found to be

$$\begin{aligned}
 F_{SOP} &= \bar{B}CD + \bar{A}BC + A\bar{B} \\
 F_{POS} &= (A + B + D)(A + C)(\bar{A} + \bar{B}),
 \end{aligned}$$

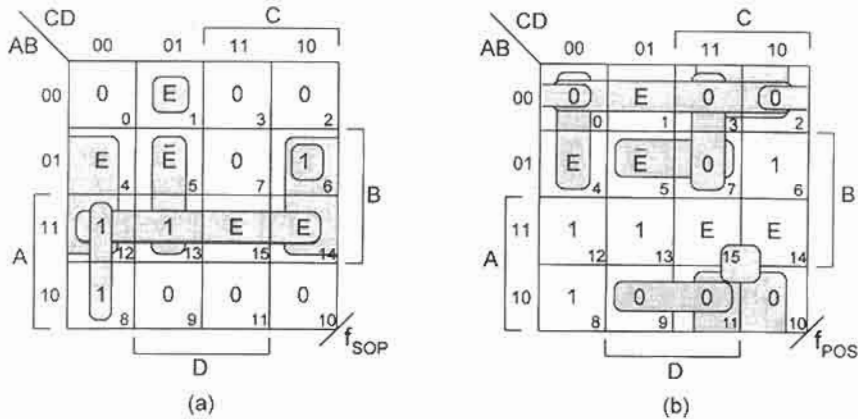
with gate/input tallies of 4/11 and 4/10, respectively, excluding possible inverters. Notice that the minimum SOP and POS cover results from these K-maps by taking  $\phi_7 = 1$  to give  $C(\phi_7 + \bar{D}) = C$  in cell 1, and by taking  $\phi_8 = 1$  to give  $(\phi_8 + C + D) = 1$  in cell 2. Because the don't cares,  $\phi_7$  and  $\phi_8$ , are used in the same way (no shared use) in both K-maps of Fig. 4.54b, the minimum SOP and POS expressions are algebraically equal.

**EXAMPLE 4.4** A five-variable function  $f$  is given in the canonical form:

$$f(A, B, C, D, E) = \sum m(3, 9, 10, 12, 13, 16, 17, 24, 25, 26, 27, 29, 31). \quad (4.75)$$

(a) Use a fourth-order EV K-map to minimize this function in both SOP and POS form.

A compression of one order requires that the Map Key be 2. Therefore, each cell of the fourth-order EV map represents a first-order submap covering two possible minterm or

**FIGURE 4.55**

Fourth-order EV K-maps for the five-variable function  $f$  in Eq. (4.75). (a) Minimum SOP cover and (b) minimum POS cover.

maxterm positions. In Fig. 4.55 are the fourth-order EV K-maps showing minimum SOP and minimum POS cover for which the expression are

$$f_{SOP} = \bar{A}\bar{B}\bar{C}\bar{D}E + \bar{B}\bar{C}\bar{D}\bar{E} + \bar{B}\bar{D}E + ABE + \bar{A}BC\bar{D} + A\bar{C}\bar{D}$$

$$f_{POS} = (A + C + D + E)(A + \bar{B} + \bar{D} + \bar{E})(\bar{A} + \bar{C} + E)(A + B + E)(\bar{A} + B + \bar{D}) \\ \cdot (A + \bar{C} + \bar{D})(A + B + D)(B + \bar{C}).$$

Notice that the loop-out protocol is applied first to the EVs and then to the 1's or 0's as a "cleanup" operation, a practice that should always be followed. Also, notice that for the POS result, the term  $(B + \bar{D} + E)$  is an OPI for the term  $(A + B + E)$ .

(b) Find the minimum SOP cover for the five-variable function in Eq. (4.75) by using conventional (1's and 0's)  $A||BC/DE$  format K-map similar to that used for a six-variable function in Fig. 4.37.

Shown in Fig. 4.56 is the conventional (1's and 0's) K-map indicating minimum SOP cover.

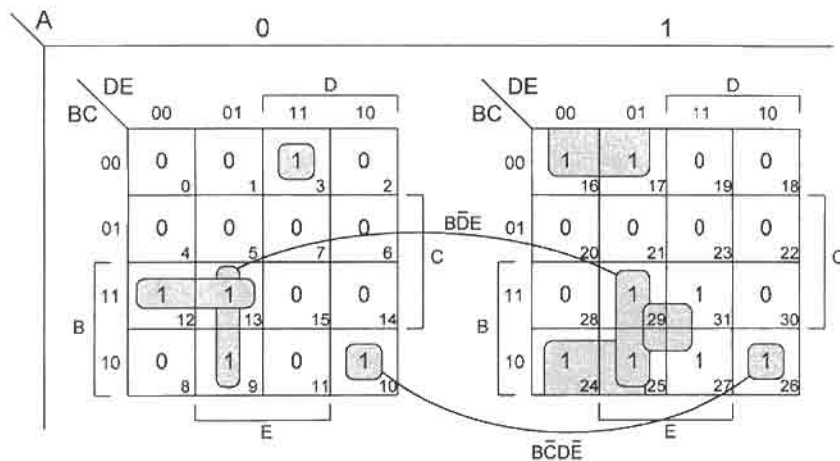
**EXAMPLE 4.5** Map the reduced function in Eq. (4.76) into a fourth-order K-map and extract minimum SOP and POS cover. Give the gate/input tally for each result, exclusive of possible inverters.

$$Y = \bar{A}\bar{B}\bar{C}\bar{D}E + \bar{A}BCD + \bar{A}BDE + \bar{B}\bar{C}\bar{D}\bar{E} + \bar{A}\bar{B}\bar{C}\bar{D}\bar{E} + \bar{A}BDE + \bar{A}BC\bar{E} \\ + \bar{B}\bar{C}\bar{D}E + \bar{A}CDE + \bar{A}\bar{B}CE \quad (4.76)$$

The function of Eq. (4.76) is mapped into the fourth-order K-map shown in Fig. 4.57, and the minimum SOP and minimum POS covers are indicated with shaded loops. The resulting minimum expressions are given by

$$Y_{SOP} = \bar{A}\bar{B}\bar{D}E + \bar{A}\bar{B}\bar{D}\bar{E} + \bar{B}\bar{C}\bar{E} + \bar{B}DE + \bar{B}CE$$

$$Y_{POS} = (\bar{B} + D + \bar{E})(C + \bar{D} + E)(B + E)(\bar{A} + C + E)(B + C + \bar{D})(A + B + C),$$

**FIGURE 4.56**

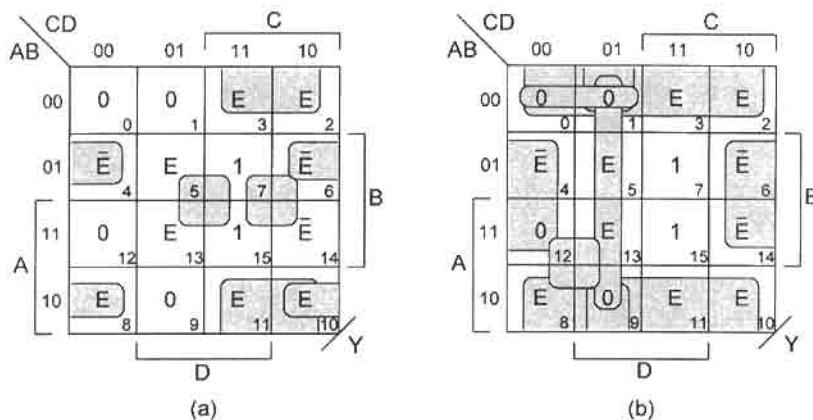
Conventional (1's and 0's) format of the type  $A||BC/DE$  for the function  $f$  in Eq. (4.75).

which represent gate/input tallies of 6/22 and 7/23, respectively, for the  $Y_{SOP}$  and  $Y_{POS}$  results, exclusive of inverters. Notice that the 1's in the SOP K-map are covered by the quads  $B\bar{C}\bar{E}$  and  $B\bar{D}\bar{E}$ , and that the 0 in cell 12 of the POS K-map is covered by the quads  $(\bar{B} + D + \bar{E})$  and  $(\bar{A} + C + E)$ .

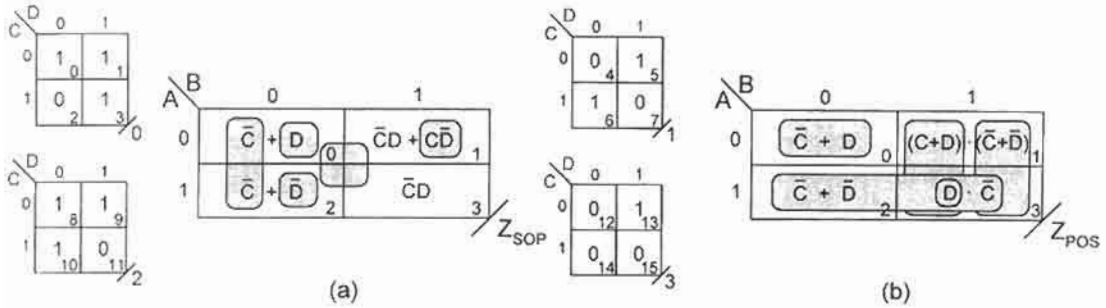
**EXAMPLE 4.6** Compress the following function into a second-order K-map and extract minimum SOP and POS cover:

$$\begin{aligned} Z(A, B, C, D) &= \prod M(2, 4, 7, 11, 12, 14, 15) \\ &= \sum m(0, 1, 3, 5, 6, 8, 9, 10, 13). \end{aligned} \quad (4.77)$$

In Fig. 4.58 are the second-order EV K-maps and submaps showing minimum SOP and

**FIGURE 4.57**

(a) Minimum SOP cover and (b) minimum POS cover for the function  $Y$  in Eq. (4.76).

**FIGURE 4.58**

Second-order EV K-maps and submaps for the function  $Z$  in Eq. (4.77) showing (a) minimum SOP cover and (b) minimum POS cover.

minimum POS cover for the function  $Z$  with the results given by

$$Z_{SOP} = \bar{A}BC\bar{D} + \bar{A}\bar{B}D + A\bar{B}\bar{D} + \bar{B}\bar{C} + \bar{C}D$$

$$Z_{POS} = (A + B + \bar{C} + D)(\bar{B} + \bar{C} + \bar{D})(\bar{B} + C + D)(\bar{A} + \bar{B} + D)(\bar{A} + \bar{C} + \bar{D})$$

Notice how easy it is to read a subfunction when accompanied by a submap. Thus, the SOP term  $\bar{C}D$  is easily observed to be present in each of the four submaps of Fig. 4.58a. Similarly,  $\bar{C}D$ , read as a POS term in Fig. 4.58b, is seen to contain both the  $(C + D)$  and  $(\bar{C} + \bar{D})$  terms by a cursory inspection of the submaps.

**EXAMPLE 4.7** Compress the following six-variable function into a fourth-order EV K-map and extract minimum SOP and minimum POS cover for it:

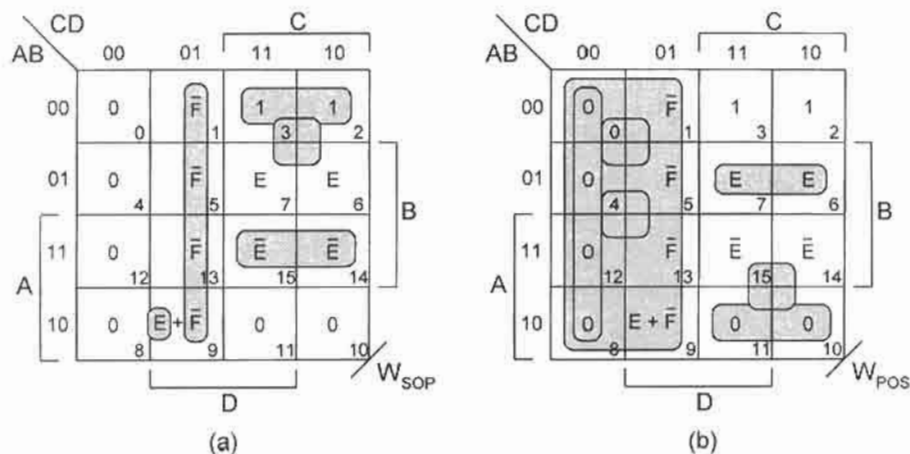
$$W(A, B, C, D, E, F) = \sum m(4, 6, 8, 9, 10, 11, 12, 13, 14, 15, 20, 22, 26, 27, 30, 31, 36, 38, 39, 52, 54, 56, 57, 60, 61). \quad (4.78)$$

Compressing a six-variable function into a fourth-order K-map requires a Map Key of  $2^{6-4} = 4$ , hence four possible minterms per K-map cell. This is a second-order compression meaning that each cell of the K-map contains subfunctions from a second-order K-map. Shown in Fig. 4.59 are the fourth-order K-maps for the function  $W$  in Eq. (4.78) where the EVs are  $E$  and  $F$ . The minimum covers for the SOP and POS functions are indicated by shaded loops and yield

$$W_{SOP} = \bar{A}\bar{B}\bar{C}DE + ABCE + \bar{C}D\bar{F} + \bar{A}CE + \bar{A}\bar{B}C$$

$$W_{POS} = (A + \bar{B} + \bar{C} + E)(C + E + \bar{F})(\bar{B} + C + \bar{F})(A + C + \bar{F})(\bar{A} + \bar{C} + \bar{E}) \cdot (\bar{A} + B + \bar{C})(C + D),$$

which represent gate/input tallies of 6/23 and 8/28, respectively. Note that the loop-out protocol is applied first to the EVs and then to the 1's and 0's as cleanup operations, a procedure that should always be followed. Observe also that these expressions are algebraically equal since no don't cares are involved.

**FIGURE 4.59**

Fourth-order EV K-maps for the six-variable function  $W$  in Eq. (4.78) showing (a) minimum SOP cover and (b) minimum POS cover.

### FURTHER READING

Nearly every text on digital or logic design provides some coverage of conventional (1's and 0's) K-map methods. Typical among these are texts of Hill and Peterson; Nelson, Nagle, Carroll and Irwin; and Roth.

- [1] F. J. Hill and G. R. Peterson, *Digital Logic and Microprocessors*. John Wiley & Sons, New York, 1984.
- [2] V. P. Nelson, H. T. Nagle, B. D. Carroll, and J. D. Irwin, *Digital Logic Circuit Analysis and Design*. Prentice Hall, Englewood Cliffs, NJ, 1995.
- [3] C. H. Roth, *Fundamentals of Logic Design*, 4th ed., West, St. Paul, MN, 1992.

References on entered variable (EV) K-map methods are limited to only a few texts. The best sources appear to be the texts of Comer, Shaw, and Tinder.

- [4] D. J. Comer, *Digital Logic and State Machine Design*, 3rd ed., Sanders College Publishing, Fort Worth, TX, 1995.
- [5] A. W. Shaw, *Logic Circuit Design*. Sanders College Publishing, Fort Worth, TX, 1993.
- [6] R. F. Tinder, *Digital Engineering Design: A Modern Approach*. Prentice Hall, Englewood Cliffs, NJ, 1991.

A classic contribution to logic minimization generally, is found in the text of Kohavi. There, can be found early treatment of the algebraic, tabular, and mapping methods. Entered variable K-map methods are not treated in this text.

- [7] Z. Kohavi, *Switching and Finite Automata Theory*. McGraw-Hill, New York, 1978.

The two logic minimization algorithms briefly discussed in this chapter, the Quinn-McCluskey method and the Espresso approach (which is a heuristic algorithm), have been

cited in countless publications. A few representative sources of these methods are presented here. Included are some of the original references as well as some of the more current ones, which often provide useful summaries of the methods.

- [8] E. J. McCluskey, *Logic Design Principles*. Prentice-Hall, Englewood Cliffs, NJ, 1986.
- [9] E. J. McCluskey, "Minimization of Boolean Functions," *Bell Syst. Tech. J.* 35(5), 1417–1444 (1956).
- [10] W. V. Quine, "The Problem of Simplifying Truth Functions," *Am. Math Monthly* 59(8), 521–531 (1952).
- [11] R. K. Brayton, G. Hachtel, C. McMullen, and A. Sangiovanni-Vincentelli, *Logic Minimization Algorithms for VLSI Synthesis*. Kluwer Academic Publishers, Boston, 1984.
- [12] R. Rudell and A. Sangiovanni-Vincentelli, "Multiple-valued Minimization for PLA Optimization," *IEEE Transactions on CAD/CAS* CAD-6(5), 727–750 (1987).
- [13] R. K. Brayton, P. C. McGeer, J. V. Sanghavi, and A. L. Sangiovanni-Vincentelli, "A New Exact Minimizer for Two-Level Logic Synthesis," in *Logic Synthesis and Optimization* (T. Sasao, Ed.). Kluwer Academic Publishers, Boston, 1993.

References on the factorization, resubstitution, and decomposition methods of optimization of multilevel circuits are fairly numerous but are set in fairly advanced notation. Perhaps the most useful are those found in texts by De Micheli, Kohavi, and Dietmeyer, and in the reference book edited by Sasao. Advanced preparation by the reader is recommended for use of these references. The text of De Micheli also has useful discussions of the area/delay trade-off factors.

- [14] G. De Micheli, *Synthesis and Optimization of Digital Circuits*. McGraw-Hill, New York, 1994.
- [15] D. L. Dietmeyer, *Logic Design of Digital Systems*, 2nd ed., Allyn and Bacon, Boston, 1971.
- [16] M. Fujita, Y. Matsunaga, Y. Tamiya, and K.-C. Chen, "Multi-Level Logic Minimization of Large Combinational Circuits by Partitioning," in *Logic Synthesis and Optimization* (T. Sasao, Ed.). Kluwer Academic Publishers, Boston, 1993.
- [17] Z. Kohavi, *Switching and Finite Automata Theory*. McGraw-Hill, New York, 1978.

## PROBLEMS

- 4.1** Expand each of the following expressions into canonical (literal) form by using the appropriate Boolean laws:
- (a)  $e(a, b) = a + \bar{b}$
  - (b)  $f(x, y) = x + \bar{x}y$
  - (c)  $g(A, B, C) = A\bar{B}C + \bar{A}BC + AB + BC + \bar{A}B\bar{C}$
  - (d)  $h(X, Y, Z) = (X + Y)(\bar{X} + \bar{Y} + Z)(Y + \bar{Z})(\bar{X} + Y + \bar{Z})$
  - (e)  $E(A, B, C, D) = (\bar{A} + \bar{B}C)(B + D)(\bar{A} + C + D)(A + \bar{B} + C + \bar{D})(\bar{B} + D)$
  - (f)  $F(w, x, y, z) = wxy\bar{z} + \bar{w}\bar{x}z + x\bar{y}z + w\bar{x}yz + xz + w\bar{x}\bar{y}\bar{z} + \bar{w}x\bar{y}z$
  - (g)  $G(a, b, c, d) = (a + \bar{b} + c + \bar{d})(b + \bar{c} + \bar{d})(\bar{a} + b)(\bar{b} + d)(\bar{a} + c + d)$
  - (h)  $H(V, W, X, Y) = VW\bar{X}\bar{Y} + \bar{X}Y + WX\bar{Y} + V\bar{W}XY + VXY + \bar{V}\bar{W}XY + \bar{W}\bar{X}\bar{Y}$
- 4.2** Place each of the three-variable functions below in a canonical truth table and in a conventional (1's and 0's) K-map. Place the variables on the K-map axes in alphabetical

order beginning with the ordinate (vertical) axis, as has been done throughout this text.

(a)  $P(A, B, C) = (A + B + \bar{C})(A + \bar{B} + \bar{C})(A + B + C)(\bar{A} + B + \bar{C})(\bar{A} + \bar{B} + \bar{C})$

(b)  $Q(a, b, c) = \sum m(1, 2, 4, 5, 6)$

(c)  $W(a, b, c) = a\bar{b}c + \bar{a}\bar{b}\bar{c} + ab\bar{c} + abc + a\bar{b}\bar{c}$

(d)  $X(A, B, C) = \prod M(0, 1, 2, 6, 7)$

(e)  $Y(w, x, y) = wx + \bar{x}\bar{y} + w(x \oplus y) + \bar{w}\bar{y}$  (Hint: Expand first.)

(f)  $Z(A, B, C) = (A + B) \odot (\bar{A}C) + A\bar{B}$  (Hint: First construct a truth table with input A.)

(g)  $F(X, Y, Z) = XY \oplus YZ \oplus XZ + X\bar{Y}$  [Hint: See Eq. (3.33).]

- 4.3** Place each of the four-variable functions below in a canonical truth table and in a conventional (1's and 0's) K-map. Place the variables on the K-map axes in alphabetical order beginning with the ordinate (vertical) axis, as has been done throughout this text.

(a)  $R(u, v, w, x) = \sum m(0, 2, 3, 7, 8, 9, 10, 11, 13)$

(b)  $S(a, b, c, d) = (a + \bar{b})(\bar{a} + b\bar{c})(\bar{b} + \bar{c})(\bar{a} + \bar{b} + c)$

(c)  $T(W, X, Y, Z) = YZ + \bar{W}\bar{X}Y + W\bar{X}\bar{Y}\bar{Z} + \bar{X}\bar{Y}Z + \bar{W}\bar{Y}Z + WXY\bar{Z} + X\bar{Y}\bar{Z}$

(d)  $U(A, B, C, D) = \prod M(0, 5, 8, 9, 11, 12, 15)$

(e)  $V(a, b, c, d) = \sum m(0, 4, 5, 7, 8, 9, 13, 15)$

(f)  $W(u, v, w, x) = [(v + w) \odot x](u + w)(u + \bar{v})(u + x)$

(g)  $X(A, B, C, D) = (A \oplus B)\bar{C}\bar{D} + \bar{B}\bar{C}D + BCD + (\bar{A} + \bar{B})C\bar{D} + \bar{A}B(C \odot D)$   
(Hint: First construct a truth table for  $CD$ , then map the result into a 1's and 0's K-map.)

(h)  $F(W, X, Y, Z) = (X \oplus Z) \oplus [W(Y \oplus Z)] + X\bar{Y}\bar{Z}$   
(Hint: First construct a truth table for  $WX$ , then map the result into a 1's and 0's K-map.)

- 4.4** Place each function of Problem 4.1 into a conventional (1's and 0's) K-map and extract canonical (coded) SOP and POS expressions from that K-map.

- 4.5** Minimize each function of Problem 4.2 in both SOP and POS form with a third-order K-map. By using the gate/input tally (exclusive of possible inverters) determine which is simpler, the SOP or POS expression. Do not implement with logic gates.

- 4.6** Minimize each function of Problem 4.3 in both SOP and POS form with a fourth-order K-map. By using the gate/input tally (exclusive of possible inverters), determine which is simpler, the SOP or POS expression. Do not implement with logic gates.

- 4.7** The following three-variable functions are incompletely specified functions, that is, they contain don't cares. By using a third-order K-map, minimize each function in both SOP and POS form *with and without* the use of the don't cares in each case. Identify any OPIs that may be present.

(a)  $e(A, B, C) = \sum m(0, 1, 2, 7) + \phi(3, 5)$

(b)  $f(X, Y, Z) = \prod M(3, 4, 6) \cdot \phi(0, 2)$

(c)  $g(a, b, c) = \sum m(0, 1, 5, 7) + \phi(2, 4)$

(d)  $h(x, y, z) = \prod M(3, 4, 5) \cdot \phi(0, 1, 2)$

(e)  $i(X, Y, Z) = \sum m(0, 5) + \phi(1, 2, 3, 7)$

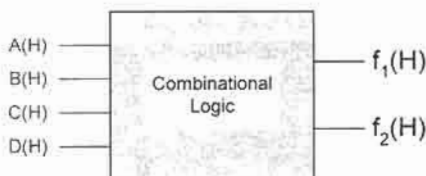


FIGURE P.4.1

- 4.8** The following four-variable functions are incompletely specified functions — they contain don't cares. Use a conventional (1's and 0's) K-map to minimize each function in both SOP and POS form and, with the help of the gate/input tally (exclusive of possible inverters), indicate which is simpler. Also, identify any OPIs that may be present.

(a)  $E(a, b, c, d) = \sum m(6, 11, 12, 13, 14) + \phi(0, 1, 2, 3, 4, 5)$

(b)  $F(A, B, C, D) = \prod M(0, 3, 6, 11, 13, 15) \cdot \phi(5, 8, 10, 14)$

(c)  $G(W, X, Y, Z) = \sum m(0, 4, 6, 8, 9, 10, 11, 14, 15) + \phi(1, 5)$

(d)  $H(w, x, y, z) = \prod M(1, 2, 3, 9, 10, 14) \cdot \phi(11, 13)$

(e)  $I(A, B, C, D) = \sum m(4, 5, 7, 12, 14, 15) + \phi(3, 8, 10)$

(f)  $J(a, b, c, d) = \prod M(0, 1, 2, 5, 7, 9) \cdot \phi(4, 6, 10, 13)$

- 4.9** Find the optimum cover (either SOP or POS) for the following four-input/two-output system (see Fig. P4.1). Base your choice on the total gate/input tally (including inverters) for the system. Assume the inputs and outputs are all active high. Do not construct the logic circuit.

$$f_1 = \sum m(0, 2, 4, 5, 9, 10, 11, 13, 15)$$

$$f_2 = \sum m(2, 5, 10, 11, 12, 13, 14, 15)$$

- 4.10** Three functions, each of three inputs, are given in canonical SOP form. Follow the discussion in Section 4.5 and find the optimized SOP minimum for the three functions taken as a system. Give the total gate/input tally for the system, exclusive of inverters.

$$f_1(A, B, C) = \sum m(1, 3, 5, 6, 7)$$

$$f_2(A, B, C) = \sum m(0, 1, 3, 6)$$

$$f_3(A, B, C) = \sum m(0, 5, 7)$$

- 4.11** Two functions, each of four variables, are given in canonical SOP form. Follow the discussion in Section 4.5 and find the optimized SOP and POS minima for the two functions taken as a system. By using the gate/input tally, exclusive of inverters, indicate which is simpler, the SOP result or the POS result.

$$F_1(A, B, C, D) = \sum m(7, 8, 10, 14, 15) + \phi(1, 2, 5, 6)$$

$$F_2(A, B, C, D) = \sum m(1, 5, 7, 8, 11, 14, 15) + \phi(2, 3, 10)$$



- 4.12** The two four-variable functions shown are presented in canonical POS form. Follow the discussion in Section 4.5 and find the optimized SOP and POS minima for the two functions taken as a system. Use the gate/input tally, including inverters, to determine which is simpler, the SOP result or the POS result. Implement the simpler of the two forms in either NAND/INV or NOR/INV logic. Assume that the inputs and outputs are all active high.

$$g_1(A, B, C, D) = \prod M(0, 3, 4, 11, 12, 13, 15) \cdot \phi(2, 5, 6)$$

$$g_2(A, B, C, D) = \prod M(0, 1, 9, 12, 13) \cdot \phi(2, 3, 4, 10)$$

- 4.13** Given below is a set of three functions, each of four variables. Follow the discussion in Section 4.5 and find the optimized SOP and POS minima for the three functions taken as a system. Use the gate/input tally, excluding inverters, to determine which is simpler, the SOP result or the POS result. [Hint: In determining the shared PIs, don't forget to include the ANDed and ORed functions ( $y_1 \cdot y_2 \cdot y_3$ ) and ( $y_1 + y_2 + y_3$ ).]

$$y_1(a, b, c, d) = \sum m(0, 1, 2, 5, 7, 8, 10, 14, 15)$$

$$y_2(a, b, c, d) = \sum m(0, 2, 4, 5, 6, 7, 10, 12)$$

$$y_3(a, b, c, d) = \sum m(0, 1, 2, 3, 4, 6, 8, 9, 10, 11)$$

- 4.14** Extract minimum SOP and POS expressions (cover) from the K-maps shown in Fig. P4.2. Where appropriate, application of the loop-out protocol discussed in Section 4.4 will help to avoid redundancy.
- 4.15** Following the discussion in Section 4.6, compress each function in Problem 4.2 into a second-order K-map (Map Key = 2) and extract minimum SOP and POS cover. Use the LSB variable as the entered variable (EV).
- 4.16** Following the discussion in Section 4.6, compress each function in Problem 4.3 into a third-order K-map (Map Key = 2) and extract minimum SOP and POS cover. Use the LSB variable as the entered variable (EV).
- 4.17** Following the discussion in Section 4.6, compress each function in Problem 4.7 into a second-order K-map (Map Key = 2) and extract minimum SOP and POS cover. Use the LSB variable as the entered variable (EV).

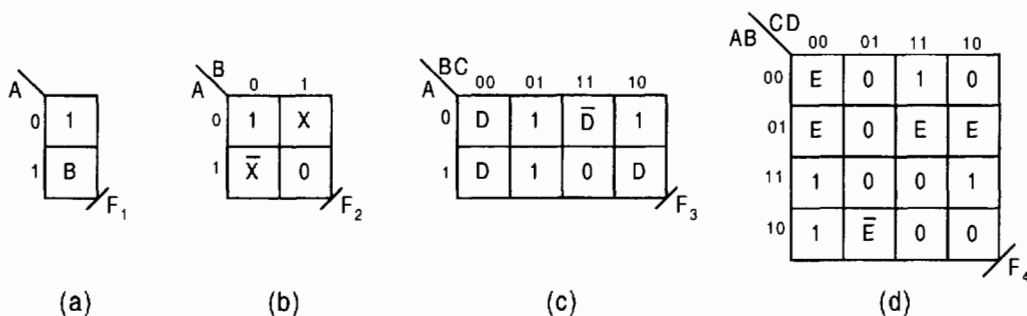


FIGURE P4.2

- 4.18** Following the discussion in Section 4.6, compress each function in Problem 4.8 into a third-order K-map (Map Key = 2) and extract minimum SOP and POS cover. Use the LSB variable as the entered variable (EV).
- 4.19** Following the discussion in Section 4.7, compress each of the following five-variable functions into a fourth-order K-map (Map Key = 2) and extract minimum SOP and POS cover. Use the LSB variable as the entered variable (EV).
- (a)  $q(A, B, C, D, E) = \prod M(0, 1, 2, 5, 14, 16, 17, 18, 19, 21, 26, 27, 30)$
- (b)  $r(A, B, C, D, E) = A\bar{B}CE + \bar{A}CDE + \bar{B}C\bar{D}E + \bar{A}BC\bar{E} + \bar{A}BDE$   
 $+ \bar{A}B\bar{C}\bar{D}\bar{E} + BC\bar{D}\bar{E} + ABD + B\bar{C}D + ABCDE$
- (c)  $s(A, B, C, D, E) = \sum m(0, 2, 4, 5, 7, 10, 13, 15, 21, 23, 24, 25, 28, 29, 30)$
- (d)  $t(A, B, C, D) = (A + B + D + \bar{E})(\bar{B} + \bar{C} + \bar{D} + E)(\bar{A} + \bar{B} + E)(\bar{A} + C + D + \bar{E})$   
 $\cdot (\bar{B} + C + D)(B + C + \bar{D} + E)(A + B + \bar{C} + \bar{D} + \bar{E})(\bar{B} + \bar{C} + D + E)$   
 $\cdot (A + \bar{B} + C)(\bar{B} + C + \bar{D})(B + \bar{C} + D + \bar{E})$
- 4.20** Minimize each function of Problem 4.19 in both SOP and POS by using a conventional (1's and 0's) K-map. To do this follow the example in Fig. 4.56.
- 4.21** Following the discussion in Section 4.6, compress each function in Problem 4.2 into a first-order K-map (Map Key = 4) and extract a minimum SOP and POS expression for each. Use the last two significant bit variables as EVs.
- 4.22** Following the discussion in Section 4.6, compress each function in Problem 4.3 into a second-order K-map (Map Key = 4) and extract a minimum SOP and POS expression for each. Use the last two significant bit variables as EVs.
- 4.23** Following the discussion in Section 4.6, compress each function in Problem 4.7 into a first-order K-map (Map Key = 4) and extract a minimum SOP and POS expression for each. Use the last two significant bit variables as EVs.
- 4.24** Following the discussion in Section 4.6, compress each function in Problem 4.8 into a second-order K-map (Map Key = 4) and extract a minimum SOP and POS expression for each. Use the last two significant bit variables as EVs.
- 4.25** Compress each function in Problem 4.19 into a third-order K-map (Map Key = 4) and extract a minimum SOP and POS expression for each. Use the last two significant bit variables as EVs.
- 4.26** Shown in Fig. P4.3 are two functions,  $F$  and  $Z$ , each of four variables, that have been compressed into third-order K-maps. (Hint: It will help to first simplify the

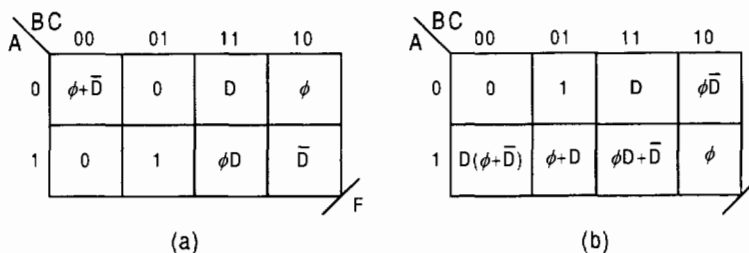


FIGURE P4.3

| AB \ CD |    |    |    |    |
|---------|----|----|----|----|
|         | 00 | 01 | 11 | 10 |
| 00      | 1  | 0  | 0  | 1  |
| 01      | 1  | 1  | 0  | 0  |
| 11      | 0  | 1  | 1  | 0  |
| 10      | 1  | 0  | 0  | 1  |

FIGURE P.4.4

subfunctions in cells 4 and 7 of function Z by applying the laws of Boolean algebra while treating the  $\phi$  as an entered variable.)

- (1) By following the example in Fig. 4.33a, construct the first-order submap for each of the eight cells in each K-map.
- (2) Give the canonical SOP and POS expressions in *code* form for each function.
- (3) Extract the minimum SOP and POS forms from each third-order K-map, keeping in mind the discussion on the use of don't cares in Subsection 4.6.1.

**4.27** Compress the fourth-order K-map in Fig. P4.4 into a second-order K-map (Map Key = 4) and loop out minimum SOP and POS expressions according to the following format:

- (a) Set A, B as the axis variables.
- (b) Set B, C as the axis variables.
- (c) Set A, C as the axis variables.

**4.28** Compress the following function into a fourth-order K-map (Map Key = 4) and extract minimum SOP and POS forms. By using the gate/input tally (exclusive of inverters), indicate which form is simpler.

$$Y(A, B, C, D, E, F) = \prod M(0, 1, 5, 7, 9, 15, 16, 18, 21, 24, 29, 31, 35, 37, 39, 40, 45, 49, 50, 56, 58, 60, 61, 63)$$

**4.29** Shown in Fig. P4.5 is a function of six variables that has been compressed into a third-order K-map, hence a third-order compression (Map Key = 8).

| A \ BC |    |                   |             |    |
|--------|----|-------------------|-------------|----|
|        | 00 | 01                | 11          | 10 |
| 0      | 1  | $\bar{X} + Y$     | $X \cdot Y$ | Z  |
| 1      | 1  | $\bar{X} \cdot Y$ | 0           | Z  |

FIGURE P.4.5

- (a) From the third-order K-map, write the canonical coded SOP and POS for this function.
- (b) Use this K-map to extract the minimum SOP and POS expressions for this function.
- 4.30** Find the minimum SOP and POS expressions (cover) for each of the following sub-functions and give the cell location of each sub-function in the fourth-order K-map.
- (a)  $P(A, B, C, D, E, F, G) = \sum m(33, 34, 36, 38) + \phi(32, 29)$
- (b)  $Q(a, b, c, d, e, f, g, h) = \sum m(114, 116, 118, 122, 124, 126)$
- (c)  $R(A, B, C, D, E, F, G) = \prod M(105, 107, 108, 109, 110)$
- (d)  $S(a, b, c, d, e, f, g, h) = \prod M(176, 181, 182, 183, 184, 189, 191) \cdot \phi(177, 185, 190)$
- 4.31** Minimize each of the following functions in both SOP and POS form by using the Quine–McCluskey (Q-M) algorithm discussed in Section 4.8.
- (a)  $f(w, x, y) = \sum m(0, 1, 3, 5, 7)$
- (b)  $g(a, b, c) = \prod M(2, 3, 4, 6)$
- (c)  $F(W, X, Y, Z) = \sum m(0, 2, 4, 5, 6, 8, 10, 11, 13, 14)$
- (d)  $G(A, B, C, D) = \prod M(1, 2, 3, 5, 7, 9, 11, 12, 14)$
- 4.32** Minimize each of the functions of Problem 4.7 in both SOP and POS form by using the Quine–McCluskey (Q-M) algorithm discussed in Section 4.8. Keep in mind the manner in which the Q-M algorithm treats don't cares.
- 4.33** Minimize each of the functions of Problem 4.8 in both SOP and POS form by using the Quine–McCluskey (Q-M) algorithm discussed in Section 4.8. Keep in mind the manner in which the Q-M algorithm treats don't cares.
- 4.34** Use the method of *factorization* to obtain a gate-minimum SOP and POS result for the following two-level functions. Find the gate/input tally (including inverters) for each and compare the results with the two-level minimum forms. Assume the inputs all arrive from positive logic sources. (Hint: First minimize the functions in two-level form and then apply the factorization method.)
- (a)  $Y = \bar{A}B + B\bar{D} + \bar{A}C + ABC + AC\bar{D}$
- (b)  $F = A\bar{B}\bar{D}\bar{E} + AB\bar{C}\bar{E} + C\bar{D}E + BCDE + ABCD + (A \odot C)(B + \bar{D})$
- 4.35** Use the *resubstitution method* discussed in Subsection 4.9.2 to obtain a gate minimum for each of the following functions. Compare the gate/input tally (excluding inverters) of the result with that for the two-level minimum. Also, comment on fan-in and inverter requirements for each, and on the gate propagation delay level for each. Assume that all inputs are active high. (Hint: First obtain the two-level SOP minimum expression, then plan to use the suggested divisor given for each.)
- (a)  $F(W, X, Y, Z) = \sum m(0, 4, 5, 7, 10, 13, 14, 15)$  (Use divisor  $X + \bar{Z}$ )
- (b)  $G(A, B, C, D) = \sum m(0, 1, 2, 3, 4, 9, 10, 11, 13, 14, 15)$  (Use divisor  $\bar{A} + C + D$ )
- (c)  $H(W, X, Y, Z) = \prod M(0, 2, 4, 6, 9)$  (Your choice of divisor)
- 4.36** Decompose each function in Problem 4.31 by applying *Shannon's expansion theorem* discussed in Subsection 4.9.3. Try at least two sets of two-variable axes about which

each expansion is to be performed. Compare the best expansion result for each with its two-level K-map minimum result.

- 4.37** Use BOOZER, the logic minimizer bundled with this text, to verify or compare (if applicable) the results with any of the previously stated problems. For example, Problem 4.37/4.6c would require use of BOOZER to minimize Problem 4.3c, since Problem 4.6 refers to Problem 4.3. [Hint: To obtain a minimum POS result by using BOOZER, map the function, complement each cell of the K-map, enter the results into BOOZER and minimize as an SOP function, and then complement the BOOZER result. Note that either entered variables (EVs) or 1's and 0's can be entered into the BOOZER algorithm — your choice.] Follow the entry protocol contained in the Readme.doc accompanying the BOOZER software.
- 4.38** Repeat Problem 4.37 by using the ESPRESSO logic minimizer bundled with this text. For example, Problem 4.38/4.19b would require the use of ESPRESSO to minimize the function  $r$  after it has been properly expressed in minterm code form.

# Function Minimization by Using K-map XOR Patterns and Reed–Muller Transformation Forms

---

## 5.1 INTRODUCTION

In this chapter it will be shown how simple “pencil-and-paper” methods can be used to extract gate-minimum multilevel logic designs not yet possible by any conventional method, including the use of CAD techniques. The methods described here make possible multilevel IC designs that occupy much less real estate than would be possible for an equivalent two-level design, and often with little or no sacrifice in speed — an advantage for VLSI design.

There are a variety of approaches to logic function minimization, which can be divided into two main categories: two-level and multilevel approaches. Chapter 4 was devoted primarily to the two-level approach to minimization. Combining entered variable (EV) sub-functions and the XOR patterns (described in the following section) in a K-map extraction process is a special and powerful form of multilevel function minimization. Used with two-level logic forms (AND and OR functions) this multilevel minimization approach leads to XOR/SOP, EQV/POS, and hybrid forms that can represent a substantial reduction in the hardware not possible otherwise. XOR/SOP and EQV/POS forms are those connecting p-terms (product terms) with XOR operators or s-terms (sum-terms) with EQV operators, respectively. Hybrid forms are those containing a mixture of these.

Another approach to multilevel logic minimization involves the use of Reed–Muller transformation forms (discussed in Sections 5.5 through 5.12) that are partitioned (broken up) into tractable parts with the assistance of entered variable Karnaugh maps (EV K-maps). The process is called the contracted Reed–Muller transformation (CRMT) minimization method and is expressly amenable to classroom (or pencil-and-paper) application. General information covering the subjects associated with Reed–Muller minimized logic synthesis are cited in Further Reading at the end of this chapter.

The word *level* (meaning level of a function) refers to the number of gate path delays from input to output. In the past the XOR gate (or EQV gate) has been viewed as a two-level

device, meaning two units of path delay as implied by the defining relations for XOR and EQV given by Eqs. (3.4) and (3.5). But the emergence of CMOS IC technology has moved the XOR and EQV gates close to single-level gates with respect to compactness and speed, as is evident from Figs. 3.26 and 3.27. The term *multilevel*, as used in this text, means the use of XOR and/or EQV gates together with two-level logic to form multiple levels of path delay as measured from input to output.

The concept of minimization, as used in this text, is presented in terms of three degrees. A *minimum* result is one that yields the lowest gate/input tally for a particular method used, for example, a two-level minimum result, but may not be the lowest possible. An *exact* minimization designates a result that has the fewest p-terms possible in an expression or the fewest s-terms possible in an expression. An *absolute* minimum expression is one that has the lowest possible gate/input tally considering all possible methods of minimization. Thus, an absolute minimum is a *gate/input-tally minimum* (or simply *gate-minimum*) and is usually the result of a specific or unique method of minimization. As a reminder, the gate/input tally (defined in Subsection 4.4.3) will usually be given exclusive of possible inverters. Only when the input activation levels are known can the gate/input tally include the inverter count.

Where appropriate to do so, reference will be made to the defining relations for XOR and EQV given by Eqs. (3.4) and (3.5) and to the XOR and EQV laws, corollaries, and identities presented in Section 3.10. Reference will also be made to minterm code (logic 0 for a complemented variable and logic 1 for an uncomplemented variable), and to maxterm code which is the dual of minterm code as discussed in Section 4.2. The EV K-map methods used in this chapter may be considered as an extension of the conventional methods discussed in Sections 4.6 and 4.7.

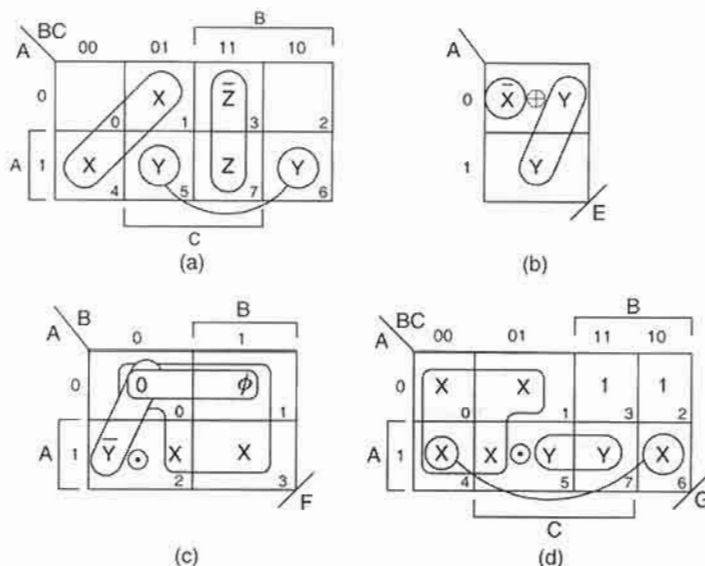
## 5.2 XOR-TYPE PATTERNS AND EXTRACTION OF GATE-MINIMUM COVER FROM EV K-MAPS

There are four types of XOR patterns that can be easily identified in EV K-maps:

1. *Diagonal patterns*
2. *Adjacent patterns*
3. *Offset patterns*
4. *Associative patterns*

References will frequently be made to the so-called *XOR-type patterns* in EV K-maps. These are references to the diagonal, adjacent, offset, and associative patterns listed above and are found only in compressed K-maps. A  $k$ th-order K-map compression results when an  $N$ -variable function is represented in an  $n$ th-order K-map—that is,  $k = N - n$ . Of the XOR-type patterns, only the offset pattern requires third and higher order K-maps for its appearance. K-maps used in the following discussions are all minterm code based, but are used to extract gate-minimum functions in both minterm code and maxterm code.

Simple examples of the first three patterns are shown in Fig. 5.1a, where a six-variable function has been compressed into a third-order EV K-map. Empty cells 0 and 2 in Fig. 5.1a are to be disregarded so as to focus attention on the patterns: The diagonal pattern formed by cells 1 and 4 is read in minterm code as  $\bar{B}X(A \oplus C)$  or in maxterm code as  $B + X + A \odot C$ .

**FIGURE 5.1**

Examples of XOR patterns in compressed K-maps. (a) Diagonal, adjacent, and offset patterns. (b), (c), (d) Associate patterns.

Notice that the diagonal pattern lies in the  $\bar{B}$  domain ( $B$  domain in maxterm code) “for all that is  $X$ ,” and that the defining relations for XOR and EQV, Eqs. (3.4) and (3.5), are used for cells 1 and 4 to give  $A \oplus C$  and  $A \odot C$ , respectively, for minterm code and maxterm code. The adjacent pattern is formed by cells 3 and 7 and is read  $BC(A \odot Z)$  in minterm code or as  $\bar{B} + \bar{C} + A \odot Z$  in maxterm code. Here, the adjacent pattern lies at the intersection of domains  $B$  and  $C$  in minterm code ( $\bar{B} + \bar{C}$  in maxterm code), and again the defining relations for XOR and EQV are used to obtain the minterm and maxterm extraction, respectively. The offset pattern is formed by cells 5 and 6 and is read in minterm code as  $AY(B \oplus C)$  and in maxterm code as  $\bar{A} + Y + B \odot C$ . In this case, the offset pattern lies in the  $A$  domain ( $\bar{A}$  in maxterm code) “for all that is  $Y$ ,” and the defining relations, Eqs. (3.4) and (3.5), are used for cells 5 and 6 to obtain  $B \oplus C$  and  $B \odot C$ , respectively. Throughout this discussion it is assumed that any entered variable, for example  $X$ ,  $Y$ , or  $Z$ , may represent a single variable or a multivariable function of any complexity.

Each of the three XOR-type patterns extracted from Fig. 5.1a has a gate/input tally of 2/5 (excluding inverters). The *gate/input tally* is a measure of logic circuit cost (in hardware and real estate) and is defined in Subsection 4.4.3. The gate count is, of course, the more significant of the two tallies and the *input tally* includes the inputs to all gates in the logic circuit. Unless stated otherwise, the gate/input tally will exclude inverters and their inputs. By comparison, the two-level logic gate/input tally for each of the patterns in Fig. 5.1a is 3/8.

The associative patterns shown in Figs. 5.1b, 5.1c, and 5.1d may combine with any or all of the other three patterns to form *compound patterns*. For this reason the associative patterns require special consideration and will be dealt with separately in the following subsection.



### 5.2.1 Extraction Procedure and Examples

Before illustrating the extraction process by example, it will be instructive to outline the extraction procedure. In this procedure, reference will be made to minterm and maxterm codes for clarification purposes. Since all K-maps are minterm code based, extraction of EQV/POS cover from them requires that the K-map domains be complemented, but not the entered variables. Extraction of XOR/SOP cover follows conventional procedure. The following six-step extraction procedure applies generally to all four types of XOR-type patterns.

#### Extraction Procedure

**Step I.** Identify the type of EV XOR pattern that exists in the K-map. A diagonal pattern requires identical K-map cell entries in diagonally located cells. An adjacent pattern requires complementary K-map cell entries in logically adjacent cells. An offset pattern requires identical cell entries in cells whose coordinates differ by two bits (a Hamming distance of 2). Associative patterns require terms associated by an XOR or EQV connective in at least one cell.

**Step II.** Write down the K-map domains in which the XOR pattern exists and any subfunctions that are the same in the pattern. Remember that in maxterm code the domains are complemented, whereas in minterm code they are not.

**Step III.** Extract the XOR pattern of type 1, 2, or 3 that exists by using the defining SOP or POS relations for XOR and EQV given by Eqs. (3.4) and (3.5). Associative patterns, of type 4, are extracted in a manner similar to the extraction of EV *s*- and *p*-terms as discussed in Section 4.6. Thus, associative patterns with XOR connectives are extracted in minterm code while those with EQV connectives are extracted in maxterm code. Compound associative patterns involve some combination of associative pattern with one or more of the other three patterns. They may also include the intersection (ANDing) of patterns or the union (ORing) of patterns. In all cases involving an associative pattern, the associating connective must be preserved in the resulting expression.

**Step IV.** Extract any remaining two-level SOP or POS cover that may exist.

**Step V.** Combine into SOP or POS form the results of steps I through IV. The resulting expression may be altered as follows: Single complementation of an XOR/EQV-associated term complements the XOR or EQV connective while double complementation of the associated terms retains the original connective.

**Step VI.** If necessary, test the validity of the extraction process. This can be done by introducing the K-map cell coordinates into the resulting expression. Generation of each cell subfunction of the K-map validates the extraction procedure.

**Examples** The simplest associative patterns are formed between XOR-associated or EQV-associated variables and like variables in adjacent cells. Three examples are presented in Figs. 5.1b, 5.1c, and 5.1d, all representing second-order K-map compressions (two EVs). For the first-order EV K-map, shown in Fig. 5.1b, the function *E* is read in minterm code as

$$E_{XOR/SOP} = (\bar{A} \cdot \bar{X}) \oplus Y = (A + X) \odot Y \quad (5.1)$$

and is seen to be a two-level function. Here, according to step III of the extraction procedure, the associative XOR pattern is extracted in minterm code in SOP form with  $\bar{X}$  located in the  $\bar{A}$  domain, hence  $\bar{A} \cdot \bar{X}$ . The  $E_{XOR/SOP}$  form can be converted to the  $E_{EQV/POS}$  form by double complementation as required by Eqs. (3.24), or can be read in maxterm code directly from the K-map.

The function  $F$  in the second-order K-map of Fig. 5.1c is read in maxterm code, according to step III and is given by

$$F_{EQV/POS} = [(B + \bar{Y}) \odot X] \cdot A, \quad (5.2)$$

which is a three-level function. In this case the EQV connective associates the  $\bar{Y}$  in cells 0 and 2 (hence  $B + \bar{Y}$  in maxterm code) with the  $X$  in all four cells. The remaining POS cover in cell 0 is extracted with the don't care ( $\phi$ ) in cell 1 by ANDing the previous result with  $A$  as required by step IV in the extraction procedure.

The function  $G$  in the third-order EV K-map, shown in Fig. 5.1d, is also read in maxterm code. Here, the EQV connective associates the  $X$ 's in cells 0, 1, 4, and 5 (thus,  $B + X$  in maxterm code) with the  $Y$ 's in cells 5 and 7 (hence,  $\bar{A} + \bar{C} + Y$ ), giving the result

$$G_{EQV/POS} = [(B + X) \odot (\bar{A} + \bar{C} + Y)](\bar{A} + C + X), \quad (5.3)$$

which is also a three-level function. The term  $(\bar{A} + C + X)$  removes the remaining POS cover in cells 4 and 6, as required by step IV.

For comparison purposes the two-level minimum results for  $E_{SOP}$ ,  $F_{POS}$ , and  $G_{POS}$  are

$$E_{SOP} = XY + \bar{A}\bar{X}\bar{Y} + AY \quad (5.4)$$

$$F_{POS} = (X + Y)(B + \bar{X} + \bar{Y})(\bar{B} + X)A \quad (5.5)$$

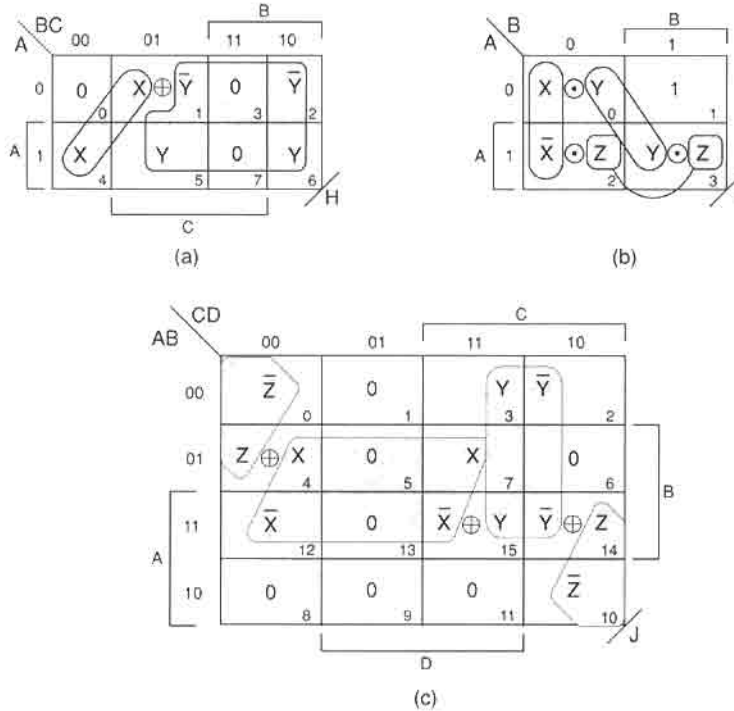
$$G_{POS} = (B + X + Y)(\bar{A} + \bar{C} + \bar{X} + \bar{Y})(\bar{A} + C + X) \cdot (A + B + X)(\bar{A} + \bar{B} + X + \bar{Y}). \quad (5.6)$$

The use of associative patterns often leads to significant reduction in hardware compared to the two-level SOP and POS forms. For example, function  $E_{XOR/SOP}$  has a minimum gate/input tally of 2/4 compared to 4/10 for  $E_{SOP}$ , the two-level SOP minimum form. The gate/input tally for  $F_{EQV/POS}$  is 3/6 compared to 4/11 for the  $F_{POS}$  expression, and function  $G_{EQV/POS}$  has a minimum gate/input tally of 4/12 compared to 6/22 for  $G_{POS}$ , the two-level POS minimum result, all excluding inverters.

XOR patterns may be combined very effectively to yield gate-minimum results. Shown in Fig. 5.2a is a second-order compression where diagonal, adjacent, and offset patterns are associated in minterm code by the XOR operator in cell 1. Here, the defining relation for XOR, given in Eqs. (3.4), is applied to the diagonal pattern (cells 1 and 4) in the  $\bar{B}$  domain for all that is  $X$  to yield  $\bar{B}X(A \oplus C)$ . This pattern is then associated with the intersection (ANDing) of the adjacent pattern ( $A \odot Y$ ) and the offset pattern ( $B \oplus C$ ) in cells 1, 2, 5, and 6 to give the gate-minimum, three-level result

$$H_{XOR/SOP} = [\bar{B}X(A \oplus C)] \oplus [(A \odot Y)(B \oplus C)] \quad (5.7)$$

with a gate/input tally of 6/13. The defining relation for EQV, given in Eqs. (3.5), is used

**FIGURE 5.2**

Examples of complex XOR patterns. (a) Combined XOR-type patterns. (b), (c) Compound associative patterns.

for the adjacent pattern ( $\bar{Y}$  in the  $\bar{A}$  domain and  $Y$  in the  $A$  domain), while Eqs. (3.4) are applied to the offset pattern ( $\bar{Y}$  and  $Y$  in the  $\bar{B}\bar{C}$  domain, and  $\bar{Y}$  and  $Y$  in the  $B\bar{C}$  domain). Notice that the 0's in cells 3 and 7 play no role in this offset/adjacent pattern even though they are included in the shaded loop covering this pattern.

For comparison purposes, the two-level minimum result is

$$H_{SOP} = \bar{A}\bar{B}\bar{C}\bar{X}\bar{Y} + A\bar{B}\bar{C}X + \bar{A}\bar{B}CY + \bar{B}CXY + \bar{A}\bar{B}\bar{C}\bar{Y} + A\bar{B}\bar{C}Y, \quad (5.8)$$

which has a gate/input tally of 7/31. Comparison with Eq. (5.7) makes it clear that the three-level result provides a better gate-minimum result but not necessarily a better performance. To evaluate the relative performance of the two approaches, fan-in restrictions and gate propagation delays would have to be established.

Compound (interconnected) associative patterns are also possible and may lead to gate-minimum functions, although often of a higher level (hence slower) than those where there is no interconnection between associative patterns. Two examples are given in Figs. 5.2b and 5.2c, both third-order compressions (hence three EVs). Function  $I$  is extracted in maxterm code, yielding the four-level, gate-minimum result

$$I_{EQVPOS} = [B + (A \oplus X)] \odot [Y + (A \oplus B)] \odot [\bar{A} + Z], \quad (5.9)$$

which has a gate/input tally of 7/14. Extraction involves the association of an adjacent pattern and a diagonal pattern with the three EQV connectives. The adjacent pattern in domain  $B$  (cells 0 and 2) requires the use of Eqs. (3.5) to give  $[B + (A \oplus X)]$ . This is associated with the diagonal pattern in cells 0 and 3, by using Eqs. (3.5), “for all that is  $Y$ ” to give  $[Y + (A \oplus B)]$ , but is also associated with the cell 3 connective in domain  $\bar{A}$  for all that is  $Z$ . Notice that the terms in square brackets are commutative. For comparison purposes the two-level POS result for function  $I$  is given by

$$I_{POS} = (A + B + X + \bar{Y})(A + B + \bar{X} + Y)(\bar{A} + B + X + Z)(\bar{A} + B + \bar{X} + \bar{Z}) \\ \times (\bar{A} + \bar{B} + Y + \bar{Z})(\bar{A} + \bar{B} + \bar{Y} + Z) \quad (5.10)$$

and has a gate/input tally of 7/30.

The function  $J$  in Fig. 5.2c is extracted in minterm code, giving the four-level, gate-minimum result

$$J_{XOR/SOP} = [\bar{D}(B \odot Z)(A \odot C)] \oplus [B(A \oplus X)(C \odot D)] \oplus [C(D \odot Y)(A \odot B)] \quad (5.11)$$

with a gate/input tally of 11/25. This function is extracted as three sets of two intersecting patterns, all associated by the three XOR connectives. The “ $Z$ ” set consists of adjacent and diagonal patterns where application of Eqs. (3.5) yields  $(B \odot Z)$  and  $(A \odot C)$ , respectively, which intersect (AND) in the  $\bar{D}$  domain. The “ $X$ ” set consists of adjacent and offset patterns that are read as  $(A \oplus X)$  and  $(C \odot D)$ , by application of Eqs. (3.4) and (3.5), and that intersect in the  $B$  domain. Here, as in Fig 5.2a, the 0’s (now in cells 5 and 13) are disregarded in the development of the offset/adjacent pattern. Finally, the “ $Y$ ” set also consists of adjacent and offset patterns such that the application of Eqs. (3.5) yields  $(D \odot Y)$  and  $(A \odot B)$ , respectively, which intersect in the  $C$  domain. As in the previous example, the terms in square brackets of Eq. (5.11) are commutative. In comparison, the two-level SOP minimum for function  $J$  is given by

$$J_{SOP} = ABCD\bar{X}\bar{Y} + ABC\bar{D}YZ + \bar{A}\bar{B}\bar{C}\bar{D}\bar{Z} + \bar{A}\bar{B}CDY + \bar{A}\bar{B}C\bar{D}\bar{Y} + \bar{A}BCDX + AB\bar{C}\bar{D}\bar{X} \\ + \bar{A}\bar{B}C\bar{D}\bar{Z} + \bar{A}\bar{C}\bar{D}X\bar{Z} + B\bar{C}\bar{D}\bar{X}Z + BCDXY + AC\bar{D}\bar{Y}\bar{Z} \quad (5.12)$$

and has a gate/input tally of 13/74. Again, the gate/minimum advantage of the multilevel function over its two-level counterpart in Eq. (5.12) is evident.

Both four-level functions,  $I_{EQV/POS}$  and  $J_{XOR/SOP}$ , are easily verified by introducing in turn the coordinates for each cell into the particular expression. For example, if one sets  $ABCD = 1111$  in Eq. (5.11), the subfunction  $\bar{X} \oplus Y$  is generated for cell 15 as required by Fig. 5.2c. Generation of the subfunctions in each cell validates the extraction process.

The gate/input tallies for all six functions represented previously are given exclusive of inverters. When account is taken of the inverters required for inputs assumed to arrive active high, the gate/input tally differentials between the multilevel results and the two-level results increases significantly. These gate/input tallies from previous examples are compared in Table 5.1, where all inputs are assumed to arrive active high.

There are other factors that may significantly increase the gate/input tally and throughput time differentials between multilevel and standard two-level SOP and POS minimum forms. These include gate fan-in restrictions and static hazard cover considerations. Static hazards

**Table 5.1 Gate/input tallies including *inverters* for functions *E, F, G, H, I,* and *J* represented as multilevel logic forms and as two-level logic forms**

| Function   | <i>E</i> | <i>F</i> | <i>G</i> | <i>H</i> | <i>I</i> | <i>J</i> |
|------------|----------|----------|----------|----------|----------|----------|
| Multilevel | 2/4      | 4/7      | 5/13     | 7/14     | 8/15     | 12/36    |
| Two-level  | 7/13     | 7/14     | 11/21    | 12/36    | 12/35    | 20/81    |

are a type of timing defect that will be discussed at length in Chapter 9. The term *fan-in* refers to the number of inputs required by a given gate. For logic families such as CMOS, propagation delay is increased significantly with increasing numbers of gate inputs, and it is here where the multilevel XOR forms often have a distinct advantage over their two-level counterparts. For example, the largest number of inputs to any gate in the implementation of function  $J_{XOR/SOP}$  is 3, whereas for the two-level function  $J_{SOP}$  it is 12. Thus, depending on how such a function is implemented, the gate/input tally and throughput time differentials between the multilevel and two-level results could increase significantly. An example of how multiple output optimization considerations may further increase the gate/input tally differential between the multilevel and two-level approaches to design is given in Section 8.8.

### 5.3 ALGEBRAIC VERIFICATION OF OPTIMAL XOR FUNCTION EXTRACTION FROM K-MAPS

Verification of the multilevel XOR forms begins by direct K-map extraction of the function in SOP or POS form by using minterm code for XOR connectives and maxterm code for EQV connectives. It then proceeds by applying Corollary I [Eq. (3.25)] or Corollary II [Eq. (3.26)] together with commutivity, distributivity, and the defining relations for XOR and EQV given by Eqs. (3.18), (3.19), (3.4), and (3.5).

As an example, consider the function *H* in Fig. 5.2a, which is extracted in minterm code. Verification of this function is accomplished in six steps:

$$\begin{aligned}
 H &= \bar{A}\bar{B}C(X \oplus \bar{Y}) + A\bar{B}\bar{C}X + \bar{A}\bar{B}\bar{C}\bar{Y} + A\bar{B}C\bar{Y} & (1) \text{ From K-map} \\
 &= [\bar{A}\bar{B}C(X \oplus \bar{Y})] \oplus (A\bar{B}\bar{C}X) \oplus (\bar{A}\bar{B}\bar{C}\bar{Y}) \\
 &\quad \oplus (A\bar{B}C\bar{Y}) \oplus (A\bar{B}\bar{C}Y) & (2) \text{ By Eq. (3.25)} \\
 &= (\bar{A}\bar{B}CX) \oplus (\bar{A}\bar{B}C\bar{Y}) \oplus (A\bar{B}\bar{C}X) \oplus (\bar{A}\bar{B}\bar{C}\bar{Y}) \\
 &\quad \oplus (A\bar{B}C\bar{Y}) \oplus (A\bar{B}\bar{C}Y) & (3) \text{ By Eqs. (3.19)} \\
 &= [\bar{B}X\{(\bar{A}C) \oplus (A\bar{C})\}] \oplus [\bar{B}C\{(\bar{A}\bar{Y}) \oplus (A\bar{Y})\}] \\
 &\quad \oplus [\bar{B}\bar{C}\{(\bar{A}\bar{Y}) \oplus (A\bar{Y})\}] & (4) \text{ By Eqs. (3.19)} \\
 &= [\bar{B}X(A \oplus C)] \oplus [\bar{B}C(A \odot Y)] \oplus [\bar{B}\bar{C}(A \odot Y)] & (5) \text{ By Eqs. (3.25),} \\
 &\quad (3.19), (3.4), \text{ and (3.5)} \\
 &= [\bar{B}X(A \oplus C)] \oplus [(A \odot Y)(B \oplus C)] & (6) \text{ By Eqs. (3.19)} \\
 &\quad \text{and (3.4)}
 \end{aligned}$$

Notice that in going from step 3 to step 4 the commutative law of XOR algebra is used.

As a second example, consider function  $I$  in Fig. 5.2b, which has been extracted in maxterm code. Verification of this function is also accomplished in six steps:

$$\begin{aligned}
 I &= (A + B + X \odot Y)(\bar{A} + B + \bar{X} \odot Z)(\bar{A} + \bar{B} + Y \odot Z) & (1) \text{ From K-map} \\
 &= (A + B + X \odot Y) \odot (\bar{A} + B + \bar{X} \odot Z) \odot (\bar{A} + \bar{B} + Y \odot Z) & (2) \text{ By Eq. (3.26)} \\
 &= (A + B + X) \odot (A + B + Y) \odot (\bar{A} + B + \bar{X}) \odot (\bar{A} + B + Z) \\
 &\quad \odot (\bar{A} + \bar{B} + Y) \odot (\bar{A} + \bar{B} + Z) & (3) \text{ By Eqs. (3.19)} \\
 &= [B + (A + X) \odot (\bar{A} + \bar{X})] \odot [Y + (A + B) \odot (\bar{A} + \bar{B})] \\
 &\quad \odot [\bar{A} + Z + (B \odot \bar{B})] & (4) \text{ By Eqs. (3.19)} \\
 &= [B + (A + X)(\bar{A} + \bar{X})] \odot [Y + (A + B)(\bar{A} + \bar{B})] \odot [\bar{A} + Z] & (5) \text{ By Eq. (3.26)} \\
 &= [B + (A \oplus X)] \odot [Y + (A \oplus B)] \odot [\bar{A} + Z] & (6) \text{ By Eqs. (3.4)}
 \end{aligned}$$

In going from step 3 to step 4, commutivity was applied before application of Eqs. (3.19). Also, in step 4,  $B \odot \bar{B} = 0$ .

#### 5.4 K-MAP PLOTTING AND ENTERED VARIABLE XOR PATTERNS

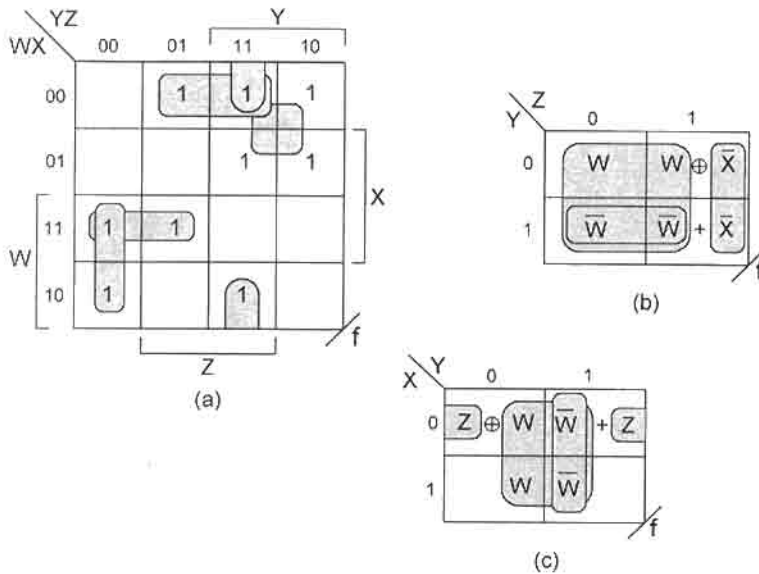
At the onset let it be understood that one does not usually hunt for applications of the XOR pattern minimization methods described here. It is possible to do this, as the example in this section illustrates, but it is more likely that such methods would be applied to EV XOR patterns that occur naturally in the design of a variety of combinational logic devices. Examples of these include a  $2 \times 2$  bit “fast” multiplier, comparator design, Gray-to-binary code conversion, XS3-to-BCD code conversion, dedicated ALU design, binary-to-2’s complement conversion, and BCD to 84-2-1 code conversion, to name but a few, most covered in later chapters. EV XOR patterns may also occur quite naturally in the design of some state machines as, for example, the linear feedback shift register counters discussed in Subsection 12.4.3.

EV K-map plotting for the purpose of extracting a gate-minimum cover by using XOR patterns is not an exact science, and it is often difficult to find the optimum K-map compression involving specific EVs, hence specific K-map axis variables. However, for some functions it is possible to plot the map directly from the canonical form, as illustrated by the example that follows. For some relatively simple functions, the K-map plotting process can be deduced directly from the canonical expression. Consider the simple function given in canonical code form:

$$f(W, X, Y, Z) = \Sigma m(1, 2, 3, 6, 7, 8, 11, 12, 13). \quad (5.13)$$

Shown in Fig. 5.3 are the conventional (1’s and 0’s) K-map and the second-order compression (two EVs) K-map derived directly from the conventional K-map. The two-level SOP minimum and the multilevel XOR/SOP gate-minimum forms are

$$f_{SOP} = \bar{W}\bar{X}Z + WX\bar{Y} + W\bar{Y}\bar{Z} + \bar{X}YZ + \bar{W}Y \quad (5.14)$$

**FIGURE 5.3**

Compressed K-maps for extraction of gate-minimum XOR forms. (a) Conventional K-map for function  $f$  in Eq. (5.13). Second-order compression K-map deduced from K-map in (a) showing XOR patterns. (c) Alternative second-order K-map.

and

$$f_{XOR/SOP} = [(Y \oplus W)] \oplus (\bar{X}Z) + \bar{W}Y, \quad (5.15)$$

which have gate/input tallies of 6/19 and 5/10, respectively. The second-order K-map in Fig. 5.3b is deduced from the K-map in Fig. 5.3a by observing that  $W \odot X$  exists in the  $YZ = 01$  column, with  $W$  and  $\bar{W}$  located in adjacent  $YZ$  columns. Thus, by taking  $Y$  and  $Z$  as the axis variables and  $W$  and  $X$  as the EVs for the compressed K-map, the XOR patterns appear, allowing one to easily extract gate-minimum results.

Notice that the  $\bar{W}$  in the EV K-map of Fig. 5.3b must be looped out a second time to give the  $\bar{W}Y$  term in Eq. (5.15). This is necessary because cover remains in  $\bar{W} + \bar{X}$  after the associative pattern involving  $W$  and  $\bar{X}$  in cell 1 has been extracted. That is, only  $\bar{W} \oplus \bar{X}$  has been looped out of  $\bar{W} + \bar{X}$ , making it necessary to cover either  $\bar{W}$  or  $\bar{X}$  a second time. This is easily verified by introducing the coordinates of the cell 3 ( $Y = 1, Z = 1$ ) into Eq. (5.15). Without the term  $\bar{W}Y$  the subfunction  $\bar{W} + \bar{X}$  cannot be generated. The residual cover in  $\bar{W} + \bar{X}$  can also be looped out of cell 3 by extracting  $\bar{X}YZ$  and using it in place of  $\bar{W}Y$  in Eq. (5.15).

Only in one other compressed K-map are the gate-minimum XOR patterns and Eq. (5.15) results obvious, and that is shown in Fig. 5.3c. In all four other compressed K-map possibilities, those having axes  $W/X, X/Z, W/Z$ , and  $W/Y$ , the XOR patterns shown in Figs. 5.3b and 5.3c disappear, making a gate-minimum extraction without extensive Boolean manipulation very difficult if not impossible. Notice that the compressed K-map in Fig. 5.3c is

easily obtained from that of Fig. 5.3a by introducing the coordinate values for  $X$  and  $Y$  into Fig. 5.3b to obtain the subfunctions in terms of  $W$  and  $Z$ .

For complex functions involving five or more variables, the process of generating a gate-minimum result by using XOR EV patterns becomes increasingly more a matter of trial and error as the number of variables increases. Again, the application of the EV XOR pattern approach to design is left more to the natural occurrence of such patterns than it is to the hunt-and-choose method. However, if it is known that XOR patterns occur naturally in some functions and if one is familiar with conventional (1's and 0's) K-map methods for five or more variables, it is possible to deduce a compressed K-map that will yield XOR/SOP or EQV/POS forms, but that may not necessarily represent gate-minimum results.

To overcome the obvious problem of dealing with complex K-map XOR patterns in functions having more the five variables, an algebraic approach can be used, a subject that is discussed at length in the remaining sections of this chapter.

## 5.5 THE SOP-TO-EXSOP REED–MULLER TRANSFORMATION

A generalization of Corollary I (Subsection 3.11.1) can be expressed in canonical form as

$$\begin{aligned}
 F_n(x_0, x_1, \dots, x_{n-1}) &= \sum_{i=0}^{2^n-1} (m_i \cdot f_i) \\
 &= \bigoplus_{i=0}^{2^n-1} (m_i \cdot f_i) \\
 &= (m_0 f_0) \oplus (m_1 f_1) \oplus (m_2 f_2) \oplus \dots \oplus (m_{2^n-1} f_{2^n-1}), \quad (5.16)
 \end{aligned}$$

where the  $2^n$   $m_i$  represent minterms read in minterm code, and the  $f_i$  represent their respective coefficients whose values derive from the binary set  $\{0, 1\}$ . The  $m_i$  symbols represent minterms that are, by definition, *mutually disjoint*, since only one minterm can be active (logic 1) for the same values of inputs. For this reason, it is permissible to interchange the OR and XOR operators as in Eq. (5.16). Thus, Eq. (5.16) expresses a transformation of an SOP expression to an EXSOP (EXclusive OR-sum-of-products) expression. Notice that if all  $f_i$  are logic 1, then  $F_n = 1$ .

By setting  $\bar{x}_i = x_i \oplus 1$ , all  $\bar{x}_i$  are eliminated from the EXSOP form of Eq. (5.16) and a function of positive polarity results. Then after considerable Boolean manipulation involving multiple applications of the XOR form of the distributive law given by Eqs. (3.19), the EXSOP expression of Eq. (5.16) is recast as the *Reed–Muller expansion* in the EXSOP form

$$\begin{aligned}
 F_n(x_0, x_1, \dots, x_{n-1}) &= g_0 \oplus g_1 x_{n-1} \oplus g_2 x_{n-2} \oplus g_3 x_{n-2} x_{n-1} \\
 &\quad \oplus g_4 x_{n-3} \dots \oplus g_{2^n-1} x_0 x_1 \dots x_{n-1}, \quad (5.17)
 \end{aligned}$$

where the  $g_i$  are called the Reed–Muller (R-M) coefficients for a positive polarity



(uncomplemented  $f_i$ ) R-M expansion (PPRME). Each R-M coefficient is the set

$$g_i = \bigoplus_{j \subseteq i} f_j. \quad (5.18)$$

obtained from the subnumbers of  $i$  by replacing  $m$  1's with 0's in  $2^m$  possible ways in the binary number corresponding to decimal  $i$ :

$$\begin{aligned} g_0 &= f_0 & \dots 000 \\ g_1 &= \oplus f(1, 0) = f_1 \oplus f_0 & \dots 001 \dots 000 \\ g_2 &= \oplus f(2, 0) = f_2 \oplus f_0 & \dots 010 \dots 000 \\ g_3 &= \oplus f(3, 2, 1, 0) = f_3 \oplus f_2 \oplus f_1 \oplus f_0 & \dots 011 \dots 010 \dots 001 \dots 000 \\ g_4 &= \oplus f(4, 0) = f_4 \oplus f_0 & \dots 100 \dots 000 \\ g_5 &= \oplus f(5, 4, 1, 0) = f_5 \oplus f_4 \oplus f_1 \oplus f_0 & \dots 101 \dots 100 \dots 001 \dots 000 \\ &\vdots & \\ &\vdots & \\ &\vdots & \\ g_{2^n-1} &= \bigoplus_{i=0}^{2^n-1} f_i. \end{aligned}$$

Note that any  $g_i$  in Eq. (5.18) is 1 if an *odd* number of  $f$  coefficients are logic 1, but is 0 if an even number of  $f$  coefficients are logic 1. If a Karnaugh map (K-map) of  $F_n$  is available, the values for the  $g_i$  are easily determined by counting the 1's in the map domains defined by the 0's in the binary number representing  $i$  in  $g_i$ . For example, the value of  $g_5$  is found by counting the 1's present in the  $\bar{x}_0\bar{x}_2$  domain for a function  $F_4 = (x_0x_1x_2x_3)$ . Thus,  $g_5 = 1$  if an odd number of 1's exists or  $g_5 = 0$  otherwise. Similarly, to determine the logic value for  $g_8$  one would count the number of 1's present in the  $\bar{x}_1\bar{x}_2\bar{x}_3$  domain for the same function, etc. All terms in the PPRME expansion whose  $g$  coefficients are logic 0 are disregarded.

## 5.6 THE POS-TO-EQPOS REED-MULLER TRANSFORMATION

The dual of Eqs. (5.16) is the generalization of Corollary II (Subsection 3.11.1) and is expressed as

$$\begin{aligned} F_n(x_0, x_1, x_2, \dots, x_{n-1}) &= \prod_{i=0}^{2^n-1} (M_i + f_i) \\ &= \bigodot_{i=0}^{2^n-1} (M_i + f_i) \\ &= (M_0 + f_0) \odot (M_1 + f_1) \odot (M_2 + f_2) \\ &\quad \odot \dots \odot (M_{2^n-1} + f_{2^n-1}), \end{aligned} \quad (5.19)$$

where the  $2^n M_i$  in Eq. (5.19) represent maxterms read in maxterm code, and the  $f_i$  represent their respective coefficients whose values derive from the binary set  $\{0, 1\}$ . The  $2^n M_i$  maxterms are *mutually disjoint* since only one maxterm can be inactive (logic 0) for the same values of inputs. For this reason it is permissible to interchange the AND and EQV operators in Eq. (5.19). Thus, Eq. (5.19) expresses the transformation of a POS expression to an EQV-product-of-sums (EQPOS) expression. Note that if all  $f_i$  are logic 0, then  $F_n = 0$ .

Setting  $x_i = \bar{x}_i \odot 0$  eliminates all  $x_i$  from Eq. (5.19), resulting in a negative polarity expression for the function  $F_n$ , which is simplified by multiple applications of the EQV form of the distributive law given by Eqs. (3.19). The result is the Reed–Muller expansion in the EQPOS form

$$\begin{aligned} F_n(x_0, x_1, x_2, \dots, x_{n-1}) = & g_0 \odot (g_1 + \bar{x}_{n-1}) \odot (g_2 + \bar{x}_{n-2}) \odot (g_3 + \bar{x}_{n-2} + \bar{x}_{n-1}) \\ & \odot (g_4 + \bar{x}_{n-3}) \odot \dots \odot (g_{2^n-1} + \bar{x}_0 + \bar{x}_1 + \dots + \bar{x}_{n-1}) \end{aligned} \quad (5.20)$$

where the  $g_i$  are now the R-M coefficients for an EQPOS (negative polarity) R-M expansion. Each EQPOS R-M coefficient is the set

$$g_i = \bigodot_{i \subseteq j} f_j \quad (5.21)$$

obtained from the subnumbers of  $i$  by replacing  $m$  1's with 0's in  $2^m$  possible ways in the binary number corresponding to decimal  $i$ , as in Eq. (5.18). Thus, the array of  $g_i$  for an EQPOS expansion is the same as that for an EXSOP expansion except that the  $\oplus$  operator is replaced by the  $\odot$  operator. In Eq. (5.21) any  $g_i$  is 0 if an odd number of  $f$  coefficients are logic 0, but is 1 otherwise. Again, the use of a K-map can be helpful in obtaining the values by counting the 0's within a given domain similar to the procedure explained earlier for the case of the EXSOP expansion. Thus, any  $g_i$  is 0 if an odd number of 0's exist within a given domain defined by the 0's in the binary number. All terms in the R-M EQPOS expansion whose  $g$  coefficients are logic 1 are ignored.

## 5.7 EXAMPLES OF MINIMUM FUNCTION EXTRACTION

In this section two examples of minimum function extraction are presented that bear the same relationship to each other as do the conventional and EV K-map methods—that is, one is based on canonical forms (conventional method) while the other is equivalent to the use of entered variables in K-maps (called the CRMT method).

**A SIMPLE EXSOP EXAMPLE** Consider the function

$$\begin{aligned} F_3 = \bar{A}BC + A\bar{B} + A\bar{C} &= \sum m(3, 4, 5, 6) \\ &= \bigoplus m(3, 4, 5, 6), \end{aligned} \quad (5.22)$$

where, for this example,  $f_3 = f_4 = f_5 = f_6 = 1$  and  $f_0 = f_1 = f_2 = f_7 = 0$ . Therefore, the  $g_i$  are found as follows:

$$\begin{aligned} g_0 &= f_0 = 0 & g_4 &= \oplus f_4(4, 0) = 1 \\ g_1 &= \oplus f_1(1, 0) = 0 & g_5 &= \oplus f_5(5, 4, 1, 0) = 0 \\ g_2 &= \oplus f_2(2, 0) = 0 & g_6 &= \oplus f_6(6, 4, 2, 0) = 0 \\ g_3 &= \oplus f_3(3, 2, 1, 0) = 1 & g_7 &= \oplus f_7(7-0) = 0. \end{aligned}$$

Here, the notation (7-0) means (7, 6, 5, 4, 3, 2, 1, 0). From Eq. (5.17) the result is an exact minimum given directly as

$$F_3 = BCg_3 \oplus Ag_4 = BC \oplus A, \quad (5.23)$$

which is a much simplified result compared to the original function and has a gate/input tally of 2/4. This function is said to be a positive polarity R-M expression or PPRME.

The same result is achieved, but with less effort, if the variables of function  $F_3$  are partitioned into two distinct sets: a disjoint set of *bond variables* (called the *bond set*) and a free set of variables (called the *free set*), both chosen from the set  $\{A, B, C\}$  and recast into a contracted (reduced) form for application of Eqs. (5.16) and (5.17). Here,  $\{A, B\}$  is chosen as the *bond set* to be coupled with the remaining (orthogonal) *free set*,  $\{C\}$ . In this particular case, any combination of bond and free sets would achieve the same desired result with equal ease. When a function is recast into bond and free sets it is said to be in a contracted Reed–Muller transformation (CRMT) form. For this example, the CRMT form of function  $F_3$  is

$$\begin{aligned} F_{AB} &= (\bar{A}B)C + (A\bar{B}) + (AB)\bar{C} = (\bar{A}B)C \oplus (A\bar{B}) \oplus (AB)\bar{C} \\ &= (\bar{A}\bar{B})f_0 \oplus (\bar{A}B)f_1 \oplus (A\bar{B})f_2 \oplus (AB)f_3 \\ &= g_0 \oplus Bg_1 \oplus Ag_2 \oplus ABg_3, \end{aligned} \quad (5.24)$$

where the subscript in  $F_{AB}$  identifies the bond set  $\{A, B\}$ . Notice that use was made of  $A\bar{B}\bar{C} + A\bar{B}C = A\bar{B}$  and that all terms of the bond set are mutually disjoint. Now, the  $f$  coefficients are  $f_0 = 0$ ,  $f_1 = C$ ,  $f_2 = 1$ , and  $f_3 = \bar{C}$ . Therefore, the resulting CRMT coefficients become

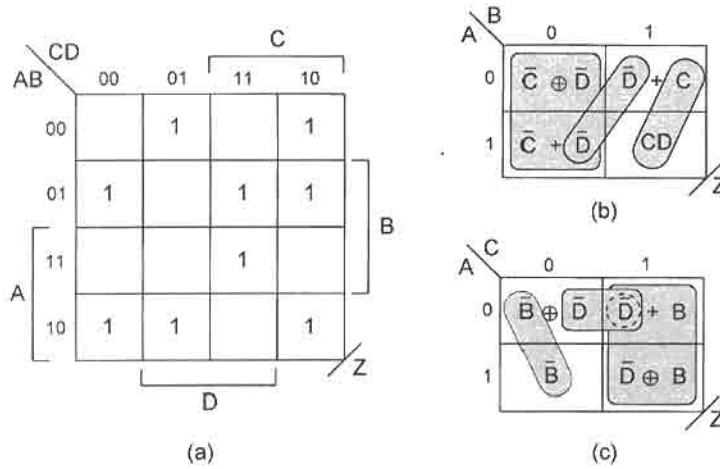
$$\begin{aligned} g_0 &= f_0 = 0 & g_2 &= f_2 \oplus f_0 = 1 \\ g_1 &= f_1 \oplus f_0 = C & g_3 &= f_3 \oplus f_2 \oplus f_1 \oplus f_0 = \bar{C} \oplus 1 \oplus C \oplus 0 = 0. \end{aligned}$$

Introducing these coefficients into the CRMT expression gives the result

$$\begin{aligned} F_2 &= g_0 \oplus Bg_1 \oplus Ag_2 \oplus ABg_3 \\ &= 0 \oplus BC \oplus A \oplus 0 \\ &= BC \oplus A \end{aligned} \quad (5.25)$$

as before.

This simple example has been given to illustrate the application of the CRMT method of function minimization. The examples that follow are designed to establish the foundation

**FIGURE 5.4**

(a) Conventional K-map for function Z. (b), (c) Compressed EV K-maps for function Z representing bond sets  $\{A, B\}$  and  $\{A, C\}$  showing minimum cover by using XOR-type patterns.

for a tractable CRMT minimization procedure, one that is suitable for classroom (or pencil-and-paper) application.

**A MORE COMPLEX EXSOP EXAMPLE** Consider the function and its canonical R-M transformation

$$\begin{aligned} Z_4(A, B, C, D) &= \sum m(1, 2, 4, 6, 7, 8, 9, 10, 15) \\ &= \bigoplus m(1, 2, 4, 6, 7, 8, 9, 10, 15). \end{aligned} \quad (5.26)$$

In Fig. 5.4a is shown the conventional K-map for function Z. In Figs. 5.4b and 5.4c are shown the second-order compressed K-maps of function Z for bond sets  $\{A, B\}$  and  $\{A, C\}$ , respectively, which are representative of the six possible bond sets for two variables.

Considering first the bond set  $\{A, B\}$ , as depicted in Fig. 5.4b, and noting that the cell entries are the  $f$  coefficients, the function  $Z_4$  is recast into the following CRMT form:

$$\begin{aligned} Z_{AB} &= (\bar{A}\bar{B})f_0 \oplus (\bar{A}B)f_1 \oplus (A\bar{B})f_2 \oplus (AB)f_3 \\ &= (\bar{A}\bar{B})(C \oplus D) \oplus (\bar{A}B)(C + \bar{D}) \oplus (A\bar{B})(\bar{C} + \bar{D}) \oplus (AB)CD \\ &= g_0 \oplus Bg_1 \oplus Ag_2 \oplus ABg_3 \end{aligned} \quad (5.27)$$

for which the CRMT coefficients are

$$\begin{aligned} g_0 &= f_0 = C \oplus D \\ g_1 &= \oplus f(1, 0) = (C + \bar{D}) \oplus C \oplus D = \bar{C}D \oplus \bar{C} \oplus D = \bar{C}\bar{D} \oplus D = \bar{C} \oplus CD = 1 \oplus C\bar{D} \\ g_2 &= \oplus f(2, 0) = (\bar{C} + \bar{D}) \oplus C \oplus D = CD \oplus C \oplus \bar{D} = C\bar{D} \oplus \bar{D} = \bar{C}\bar{D} \\ g_3 &= \oplus f(3, 2, 1, 0) = CD \oplus (\bar{C} + \bar{D}) \oplus 1 \oplus C\bar{D} = CD \oplus CD \oplus 0 \oplus C\bar{D} = C\bar{D}, \end{aligned}$$

where use has been made of the XOR form of Eqs. (3.31), and the XOR DeMorgan identities given in Eqs. (3.27) from which, for example, there results  $(C + \bar{D}) \oplus C = \bar{C}D \oplus \bar{C}$  in  $g_1$ . Introducing these coefficients into Eq. (5.27) and simplifying by using the XOR form of Eqs. (3.30) gives the minimum result

$$\begin{aligned} Z_{AB} &= C \oplus D \oplus B \oplus BCD \oplus A\bar{C}\bar{D} \oplus ABC\bar{D} \\ &= B \oplus C \oplus D \oplus A\bar{C}\bar{D} \oplus \bar{A}BC\bar{D}, \end{aligned} \quad (5.28)$$

which is a three-level function with a gate/input tally of 6/15.

Repeating the same procedure for Fig. 5.4c and bond set  $\{A, C\}$ , the function  $Z$  is recast into the CRMT form

$$\begin{aligned} Z_{AC} &= (\bar{A}\bar{C})f_0 \oplus (\bar{A}C)f_1 \oplus (A\bar{C})f_2 \oplus (AC)f_3 \\ &= (\bar{A}\bar{C})(B \oplus D) \oplus (\bar{A}C)(B + \bar{D}) \oplus (A\bar{C})\bar{B} \oplus (AC)(B \oplus \bar{D}) \\ &= g_0 \oplus g_1C \oplus g_2A \oplus g_3AC, \end{aligned} \quad (5.29)$$

where the  $g$  coefficients become

$$\begin{aligned} g_0 &= B \oplus D \\ g_1 &= (B + \bar{D}) \oplus B \oplus D = \bar{B}D \oplus \bar{B} \oplus D = \bar{B}\bar{D} \oplus D = \bar{B} \oplus BD = 1 \oplus B\bar{D} \\ g_2 &= \bar{B} \oplus B \oplus D = \bar{D} \\ g_3 &= B \oplus \bar{D} \oplus \bar{B} \oplus 1 \oplus B\bar{D} = \bar{D} \oplus B\bar{D} = \bar{B}\bar{D}. \end{aligned}$$

Then by introducing these  $g$  coefficients into Eq. (5.29) and simplifying with the XOR form of Eqs. (3.30), there results the exact minimum

$$\begin{aligned} Z_{AC} &= B \oplus D \oplus \bar{B}C\bar{D} \oplus CD \oplus A\bar{D} \oplus A\bar{B}C\bar{D} \\ &= B \oplus A\bar{D} \oplus \bar{C}D \oplus \bar{A}\bar{B}C\bar{D}, \end{aligned} \quad (5.30)$$

which is a three-level function with a gate/input tally of 6/14 excluding possible inverters.

The results for  $Z_{AB}$  and  $Z_{AC}$  are typical of the results for the remaining four two-variable bond sets  $\{C, D\}$ ,  $\{B, D\}$ ,  $\{A, D\}$ , and  $\{B, C\}$ . All yield gate/input tallies of 6/14 or 6/15 and are three-level functions. Thus, in this case, the choice of bond set does not significantly affect the outcome, but the effort required in achieving an exact minimum may vary with the choice of the bond set. No attempt is made to use single- or three-variable bond sets for function  $Z$ .

A comparison is now made between the CRMT minimization method and other approaches to the minimization of function  $Z$ . Beginning with the canonical R-M approach and from Fig. 5.4a, the  $f$  coefficients easily seen to be

$$\begin{aligned} f_1 = f_2 = f_4 = f_6 = f_7 = f_8 = f_9 = f_{10} = f_{11} = f_{14} = f_{15} = 1 \quad \text{and} \\ f_0 = f_3 = f_5 = f_{12} = f_{13} = 0. \end{aligned}$$

Then from Eq. (5.16) the R-M  $g_i$  coefficients are evaluated as follows:

$$\begin{aligned}
 g_0 &= f_0 = 0 & g_8 &= \oplus f(8, 0) = 1 \\
 g_1 &= \oplus f(1, 0) = 1 & g_9 &= \oplus f(9, 8, 1, 0) = 1 \\
 g_2 &= \oplus f(2, 0) = 1 & g_{10} &= \oplus f(10, 8, 2, 0) = 1 \\
 g_3 &= \oplus f(3, 2, 1, 0) = 0 & g_{11} &= \oplus f(11 - 8, 3 - 0) = 1 \\
 g_4 &= \oplus f(4, 0) = 1 & g_{12} &= \oplus f(12, 8, 4, 0) = 0 \\
 g_5 &= \oplus f(5, 4, 1, 0) = 0 & g_{13} &= \oplus f(13, 12, 9, 8, 5, 4, 1, 0) = 0 \\
 g_6 &= \oplus f(6, 4, 2, 0) = 1 & g_{14} &= \oplus f(14, 12, 10, 8, 6, 4, 2, 0) = 1 \\
 g_7 &= \oplus f(7 - 0) = 1 & g_{15} &= \oplus f(15 - 0) = 1.
 \end{aligned}$$

Note that the  $g_i$  coefficients are immediately realized by counting 1's within the domains of the conventional K-map shown in Fig. 5.4a. Thus,  $g_{13} = 0$  since an even number of 1's exist in the  $\bar{C}$  domain (determined from 1101), or  $g_9 = 1$  because an odd number of 1's exist in the  $\bar{B}\bar{C}$  domain (from 1001). Disregarding the  $g = 0$  coefficients, there results the positive polarity R-M expression and its simplified mixed polarity form

$$\begin{aligned}
 Z_4 &= Dg_1 \oplus Cg_2 \oplus Bg_4 \oplus BCg_6 \oplus BCDg_7 \oplus Ag_8 \oplus ADg_9 \oplus ACg_{10} \oplus ACDg_{11} \\
 &\quad \oplus ABCg_{14} \oplus ABCDg_{15} \\
 &= D \oplus C \oplus B \oplus BC \oplus BCD \oplus A \oplus AD \oplus AC \oplus ACD \oplus ABC \oplus ABCD \\
 &= D \oplus C \oplus B \oplus BCD \oplus A\bar{D} \oplus AC\bar{D} \oplus ABC\bar{D} \\
 &= B \oplus C \oplus D \oplus A\bar{C}\bar{D} \oplus \bar{A}BC\bar{D}, \tag{5.31}
 \end{aligned}$$

which is a three-level function having a gate/input tally of 6/15 excluding possible inverters. The function in Eq. (5.31) is seen to be the same as that in Eq. (5.28), but it is not an exact minimum. Here, multiple applications of the XOR identities in Eqs. (3.30) have been applied to excise terms.

Other comparisons are now made between the CRMT method and the EV K-map and conventional K-map methods presented in Sections 4.6 and 4.4. From Figs. 5.4b and 5.4c, the minimum cover extraction by using XOR type patterns (shown by loops) gives

$$Z_{K\text{-map } AB} = \bar{B}(C \oplus D) + \bar{D}(A \oplus B) + BCD \tag{5.32}$$

and

$$Z_{K\text{-map } AC} = (\bar{B}\bar{C}) \oplus (\bar{A}\bar{D}) + C(B \oplus \bar{D}) \tag{5.33}$$

representing three-level functions with gate/input tallies of 6/14 and 6/12, respectively, excluding possible inverters. The function  $Z_{K\text{-map } AC}$  is a gate/input-tally minimum for function  $Z$ . The results in Eqs. (5.32) and (5.33) are hybrid forms classified as mixed AND/OR/EXSOP expressions. The two-level SOP minimum, obtained from Fig. 5.4a,

offers another comparison and is

$$Z = \bar{B}\bar{C}D + \bar{A}B\bar{D} + BCD + \bar{B}C\bar{D} + A\bar{B}\bar{C} \quad (5.34)$$

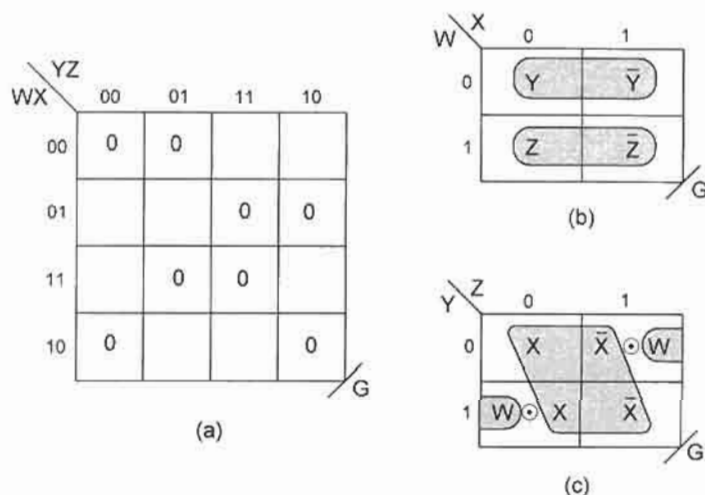
with a gate/input tally of 6/20, again excluding inverters.

Comparing the results for the four methods used to minimize function  $Z$  given by Eqs. (5.28) through (5.34), it is clear that the CRMT results in Eqs. (5.28) and (5.30) are competitive with the other methods. However, only the CRMT result in Eq. (5.30) is an exact EXSOP minimum result. As will be demonstrated further by other examples, the CRMT and EV K-map methods of minimization tend to yield results that are typically competitive with or more optimum than those obtained by the other methods, including computerized two-level results. These observations are valid for relatively simple expressions amenable to classroom methods. No means are yet available for making a fair comparison of the CRMT approach with related computer algorithmic methods.

**AN EQPOS EXAMPLE** Consider the four variable function  $G$  and its canonical R-M transformation

$$G(W, X, Y, Z) = \prod M(0, 1, 6, 7, 8, 10, 13, 15) = \bigodot M(0, 1, 6, 7, 8, 10, 13, 15), \quad (5.35)$$

which follows from Eqs. (5.19). The conventional (1's and 0's) K-map for this function is shown in Fig. 5.5a. Begin with the CRMT minimization method applied to bond set  $\{W, X\}$  as depicted in Fig. 5.5b, which is a second-order compression of the function  $G$ . From Eqs. (5.20) and (5.21) and for bond set  $\{W, X\}$ , this function is represented in the



**FIGURE 5.5**

(a) Conventional K-map for function  $G$ . (b), (c) Compressed EV K-maps of function for bond sets  $\{W, X\}$  and  $\{Y, Z\}$  showing minimum cover by using XOR-type patterns.

negative-polarity CRMT form

$$\begin{aligned}
 G_{WX} &= (W + X + f_0) \odot (W + \bar{X} + f_1) \odot (\bar{W} + X + f_2) \odot (\bar{W} + \bar{X} + f_3) \\
 &= (W + X + Y) \odot (W + \bar{X} + \bar{Y}) \odot (\bar{W} + X + Z) \odot (\bar{W} + \bar{X} + \bar{Z}) \\
 &= g_0 \odot (\bar{X} + g_1) \odot (\bar{W} + g_2) \odot (\bar{W} + \bar{X} + g_3),
 \end{aligned} \tag{5.36}$$

which are read in maxterm code. From Eq. (5.21) the  $g$  coefficients become

$$\begin{aligned}
 g_0 &= Y & g_2 &= Z \odot Y \\
 g_1 &= \bar{Y} \odot Y = 0 & g_3 &= Z \odot \bar{Z} \odot 0 = 1.
 \end{aligned}$$

Introducing these coefficients into Eq. (5.36) yields the absolute minimum EQPOS expression

$$\begin{aligned}
 G_{WX} &= Y \odot \bar{X} \odot (\bar{W} + Z \odot Y) \\
 &= Y \odot \bar{X} \odot (\bar{W} + Y) \odot (\bar{W} + Z) \\
 &= \bar{X} \odot (W + Y) \odot (\bar{W} + Z)
 \end{aligned} \tag{5.37}$$

that is seen to be a three-level function with a gate/input tally of 4/8.

The CRMT minimization process is repeated for the bond set  $\{Y, Z\}$  as depicted in Fig. 5.5c. The CRMT expression now becomes

$$\begin{aligned}
 G_{YZ} &= (Y + Z + g_0) \odot (Y + \bar{Z} + g_1) \odot (\bar{Y} + Z + g_2) \odot (\bar{Y} + \bar{Z} + g_3) \\
 &= (Y + Z + X) \odot (Y + \bar{Z} + W \odot \bar{X}) \odot (\bar{Y} + Z + W \odot X) \odot (\bar{Y} + \bar{Z} + \bar{X}) \\
 &= g_0 \odot (\bar{Z} + g_1)(\bar{Y} + g_2)(\bar{Y} + \bar{Z} + g_3)
 \end{aligned} \tag{5.38}$$

for which the  $g$  coefficients are found to be

$$\begin{aligned}
 g_0 &= f_0 = X \\
 g_1 &= \odot f(1, 0) = W \odot \bar{X} \odot X = \bar{W} \\
 g_2 &= \odot f(2, 0) = W \odot X \odot X = W \\
 g_3 &= \odot f(3, 2, 1, 0) = \bar{X} \odot W \odot X \odot \bar{W} = 1,
 \end{aligned}$$

where use is made of  $g_1 = \odot f(1, 0) = \bar{W}$  in the last term for  $g_3$ . Then, introducing these coefficients into Eq. (5.38) gives the absolute minimum result

$$G_{YZ} = X \odot (\bar{W} + \bar{Z}) \odot (W + \bar{Y}), \tag{5.39}$$

which is again a three-level function with a gate/input tally of 4/8, inverters excluded.

The same result is, in this case, obtained by minimizing a canonical R-M expansion of Eq. (5.35), which becomes

$$\begin{aligned}
 G_4(W, X, Y, Z) &= \bigodot M(0, 1, 6, 7, 8, 10, 13, 15) \\
 &= g_0 \odot (M_1 + g_1) \odot (M_2 + g_2) \odot (M_3 + g_3) \odot \cdots \odot (M_{15} + g_{15}).
 \end{aligned}$$



From the conventional K-map in Fig. 5.5a and counting 0's within a given domain, the  $g$  coefficients are found to be  $g_0 = g_2 = g_4 = g_9 = g_{10} = 0$  with all the rest being logic 1. Introducing these values in the R-M expansion gives the minimum result

$$\begin{aligned} G_4 &= 0 \odot \bar{Y} \odot \bar{X} \odot (\bar{W} + \bar{Z}) \odot (\bar{W} + \bar{Y}) \\ &= 0 \odot \bar{X} \odot (\bar{W} + \bar{Z}) \odot (W + \bar{Y}) \\ &= X \odot (\bar{W} + \bar{Z}) \odot (W + \bar{Y}), \end{aligned} \quad (5.40)$$

as before in Eq. (5.39).

The result previously obtained for  $G_{YZ}$  can also be obtained by using the CRMT approach in a somewhat different way. The plan is to obtain the result for the SOP CRMT expansion ("for the 0's") and then complement that result to produce the EQPOS CRMT expansion form. Complementing each of the four EV cell entries in Fig. 5.5b gives

$$\begin{aligned} \bar{G}_{YZ}(\text{EPOS}) &= (\bar{Y}\bar{Z})\bar{X} + (\bar{Y}Z)(W \oplus \bar{X}) + (Y\bar{Z})(W \oplus X) + (YZ)X \\ &= g_0 \oplus Zg_1 \oplus Yg_2 \oplus YZg_3, \end{aligned} \quad (5.41)$$

with  $g$  values

$$\begin{aligned} g_0 &= f_0 = \bar{X} & g_2 &= \oplus f(2, 0) = W \oplus X \oplus \bar{X} = \bar{W} \\ g_1 &= \oplus f(1, 0) = W \oplus \bar{X} \oplus \bar{X} = W & g_3 &= \oplus f(3 - 0) = X \oplus W \oplus X \oplus W = 0, \end{aligned}$$

where use is made of  $g_1 = \oplus f(1, 0) = W$  in the last term for  $g_3$ . Introducing these values into Eq. (5.41) gives

$$\bar{G}_{YZ}(\text{EPOS}) = \bar{X} \oplus ZW \oplus Y\bar{W},$$

resulting in the EQPOS expression

$$\begin{aligned} G_{YZ} &= \overline{\bar{X} \oplus ZW \oplus Y\bar{W}} \\ &= X \odot (\bar{W} + \bar{Z}) \odot (W + \bar{Y}), \end{aligned} \quad (5.42)$$

where an odd number of complementations (operators and operands) have been performed to complement the function. Notice that the  $f$  coefficients are also the complements of those required for the EQPOS expansion, as they must be, since the cells of the EV K-map in Fig. 5.5b were complemented.

It is interesting to compare the results just obtained for  $G$  with those read from the EV K-maps in Figs. 5.5b and c, and with two-level POS minimization. Following the procedure given by [3, 4], the results for  $G_{WX}$  and  $G_{YZ}$  are read directly in maxterm code from the K-maps (see K-map loopings) as

$$G_{K\text{-map } WX} = [W + (X \oplus Y)][(\bar{W} + (X \oplus Z))] \quad (5.43)$$

and

$$G_{K\text{-map } YZ} = [W + (Y \odot Z)] \odot (X \oplus Z) \quad (5.44)$$

with gate/input tallies of 5/10 and 4/8, respectively. Note that reading a K-map in maxterm code requires that the domains (not the entered variables) be complemented, since the K-maps are minterm-code based [3]. In comparison, the two-level minimum result from Fig. 5.5a is

$$G = (W + X + Y)(W + \bar{X} + \bar{Y})(\bar{W} + \bar{X} + \bar{Z})(\bar{W} + X + Z), \quad (5.45)$$

which has a gate/input tally of 5/16 excluding possible inverters.

Notice that all CRMT minimization results, the canonical R-M minimization result, and one EV K-map result for  $G$  all represent three-level functions with minimum gate/input tallies of 4/8 (excluding possible inverters). In comparison, the best two-level result that can be obtained for  $G$  yields a gate/input tally of 5/16, again excluding inverters. Notice also that the two-level result requires that four s-terms be ANDed in the output stage, whereas all other results mentioned earlier have a fan-in limit of two. Increased fan-in can slow the throughput of a circuit, particularly in CMOS, as was discussed in Subsections 3.6.2 and 3.6.3, and in Section 4.10.

## 5.8 HEURISTICS FOR CRMT MINIMIZATION

A given minimization method can yield a guaranteed exact minimum for a function if, and only if, an exhaustive search is carried out. Applied to the CRMT method this involves finding the optimum (bond set)/(free set) combination for the optimal reduction process of minimization required by the CRMT method. As the number of inputs to a function increases, the task of performing an exhaustive search becomes more difficult, eventually requiring computer algorithmic means. Even then, an intractable problem eventually ensues when the number of inputs becomes excessively large for the minimization algorithm used. When this happens, the minimization problem is known to be  $\mathcal{NP}$ -complete (see Section 4.11).

Fortunately, variation in the choice of bond set for CRMT minimization often results in little or no difference in minimum gate/input tally for a minimized function. However, the effort required to achieve a minimum result may vary considerably with bond set choice. Thus, if a guaranteed absolute minimum is not required, alternative choices of bond set should yield an acceptable minimum, but with some limits placed on the number of bond set variables. By the pencil-and-paper method this means that for practical reasons the number of bond set variables should not exceed four for most applications of the CRMT method. The limit on the total number of variables is placed between eight and twelve depending on one's ability to use entered variables. In any case, experience in the application of the laws and identities of XOR algebra is an invaluable asset in achieving a minimized result.

Given these preliminary comments, the following procedure should be helpful in applying the CRMT "hand" minimization method to functions of 12 variables or less:

**Step 1.** Choose a bond set and construct an entered variable (EV) K-map with the K-map axes as the bond set. The starting point in this process can be a canonical SOP or

POS expression, a conventional (1's and 0's) K-map, or a truth table. The cell subfunctions of the EV K-map become the  $f_i$  coefficients in the CRMT form of Eq. (5.16) or (5.19). Thus, the entered variables make up the free set. As a caveat, try to avoid bond sets that generate  $f$  coefficients like  $\cdots Z(X + Y) \cdots$ , since such coefficients do not produce simple  $g$  coefficients. Note that an EV truth table, such as that in Fig. 8.26, will also suffice for the purpose of the CRMT minimization method if the table-heading variables are taken as the bond set variables.

**Step 2.** For the choice of bond set used, obtain a set of minimum CRMT  $g_i$  coefficients from Eq. (5.18) or (5.21) by using the EV K-map cell entries as the  $f_i$  coefficients and by applying Eqs. (3.19). If alternative minimum expressions exist for a given  $g$  coefficient, choose among these for the “best” one in consideration of steps 3 and 4 that follow. Thus, if an exact minimum result is required for a given bond set, an exhaustive search for an optimum  $g$  set must be carried out.

**Step 3.** Recast the function in positive or negative CRMT form by using the  $g$  set from step 2 in Eq. (5.17) or (5.20).

**Step 4.** Reduce the results of STEP (3) by applying the laws and identities given by Eqs. (3.19) and (3.27)–(3.33). Keep in mind that identical EXSOP terms in the form  $\cdots \oplus X \oplus X \oplus \cdots$  or EQPOS terms in the form  $\cdots \odot X \odot X \odot \cdots$  can be excised immediately in their respective CRMT expressions.

**Step 5.** If an exact minimum result is required, an exhaustive search must be carried out by repeating Steps (1) through (4) for all possible bond sets. As a practical matter for pencil-and-paper application of the CRMT method, the exhaustive search process should not be conducted on functions exceeding five variables. For example, a five-variable function would have to be partitioned into 10 two- or 10 three-variable bond sets in addition to the partitioning for the remaining bond sets. Of course, if an exact minimum is not required, most any choice of CRMT bond set will yield an acceptable minimum for many applications — one that may even be a near-exact minimum.

**Step 6.** Don't cares, if present, must be considered during the bond set selection process. This is usually done with the intent of reducing the complexity of the CRMT  $g$  coefficients, if not optimizing them. It is often the case that simple  $f$  coefficients (EV K-map cell entries such as 0, 1,  $X$ , or  $X \oplus Y$ ) yield simple  $g$  coefficients that lead to near-exact minimizations. In any case, the presence of don't cares will complicate considerably an exhaustive search process.

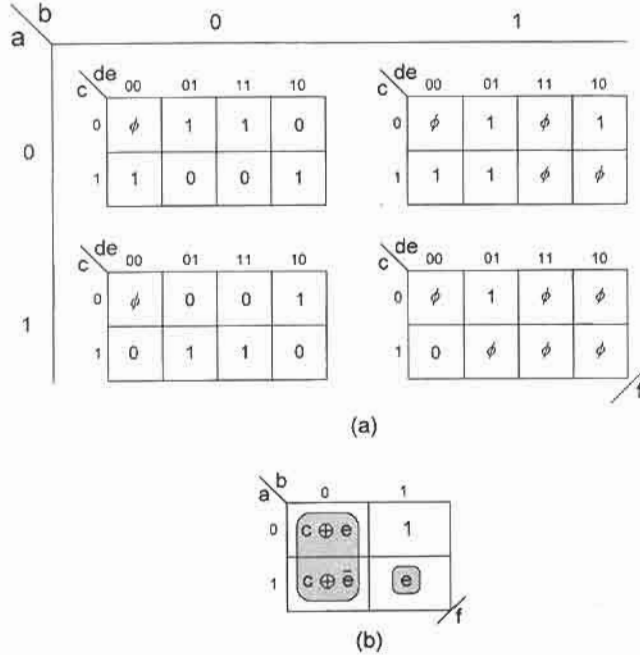
**Step 7.** If more than one function is to be optimized, the procedure is to set up the CRMT forms separately as in steps 1–5 and then follow a systematic reduction process for each, taking care to use shared terms in an optimal fashion.

## 5.9 INCOMPLETELY SPECIFIED FUNCTIONS

Consider the five-variable function

$$f(a, b, c, d, e) = \sum m(1, 3, 4, 6, 9, 10, 12, 13, 18, 21, 23, 25) \\ + \phi(0, 8, 11, 14, 15, 16, 24, 26, 27, 29, 30, 31), \quad (5.46)$$

where  $\phi$  is the symbol representing don't cares (nonessential minterms). Shown in Fig. 5.6a is the conventional K-map for this function and in Fig. 5.6b its second-order EV K-map

**FIGURE 5.6**

K-maps for the five-variable function  $f$  given in Eq. (5.46). (a) Conventional K-map array for bond set  $\{a, b\}$ . (b) Third-order compression of function  $f$  for bond set  $\{a, b\}$  showing minimum cover by using the adjacent XOR pattern in maxterm code.

(third-order compression) for bond set  $\{a, b\}$ . Recasting this function into the CRMT form of Eqs. (5.16) and (5.17) gives

$$\begin{aligned}
 f_{ab} &= \bar{a}\bar{b}f_0 \oplus \bar{a}bf_1 \oplus a\bar{b}f_2 \oplus abf_3 \\
 &= \bar{a}\bar{b}(c \oplus e) \oplus \bar{a}b \oplus a\bar{b}(c \oplus \bar{e}) \oplus abe \\
 &= g_0 \oplus bg_1 \oplus ag_2 \oplus abg_3
 \end{aligned} \tag{5.47}$$

with  $g$  values

$$\begin{aligned}
 g_0 &= c \oplus e & g_2 &= c \oplus \bar{e} \oplus c \oplus e = 1 \\
 g_1 &= 1 \oplus c \oplus e = \bar{c} \oplus e = c \oplus \bar{e} & g_3 &= e \oplus c \oplus \bar{e} \oplus c \oplus \bar{e} = e.
 \end{aligned}$$

Here, don't cares are set as  $\phi_0 = \phi_{24} = \phi_{26} = \phi_{30} = 0$  with the remainder equal to logic 1. Introducing the values into Eq. (5.47) and simplifying yields the mixed polarity result

$$\begin{aligned}
 f_{ab} &= c \oplus e \oplus b\bar{c} \oplus be \oplus a \oplus abe \\
 &= a \oplus e \oplus c \oplus b\bar{c} \oplus \bar{a}be \\
 &= a \oplus e \oplus (b + c) \oplus \bar{a}be \\
 &= a \oplus \bar{e} \oplus \bar{b}\bar{c} \oplus \bar{a}be,
 \end{aligned} \tag{5.48}$$

which is a three-level minimum with a gate/input tally of 5/11 excluding possible inverters. No attempt is made to examine bond sets other than  $\{a, b\}$ . Consequently, Eq. (5.48) cannot necessarily be regarded as an exact minimum.

The EQPOS CRMT form for bond set  $\{a, b\}$  is obtained from Eqs. (5.20) and (5.21) and is

$$f_{ab} = g_0 \odot (\bar{b} + g_1) \odot (\bar{a} + g_2) \odot (\bar{a} + \bar{b} + g_3), \quad (5.49)$$

for which the  $g$  coefficients are

$$\begin{aligned} g_0 &= c \oplus e = c \odot \bar{e} & g_2 &= \bar{c} \oplus e \odot c \oplus e = 0 \\ g_1 &= 1 \odot (c \odot \bar{e}) = c \odot \bar{e} = \bar{c} \odot e & g_3 &= e \odot e \odot c \odot c \odot \bar{e} = \bar{e}. \end{aligned}$$

After introducing these coefficients into Eq. (5.49) there results the mixed polarity CRMT result

$$\begin{aligned} f_{ab} &= c \odot \bar{e} \odot (\bar{b} + c \odot \bar{e}) \odot \bar{a} \odot (\bar{a} + \bar{b} + \bar{e}) \\ &= \bar{a} \odot \bar{e} \odot c \odot (\bar{b} + c) \odot (\bar{b} + \bar{e}) \odot (\bar{a} + \bar{b} + \bar{e}) \\ &= \bar{a} \odot \bar{e} \odot (b + c) \odot (a + \bar{b} + \bar{e}). \end{aligned} \quad (5.50)$$

This is a three-level EQPOS minimum with a gate/input tally of 5/11.

Now it is desirable to compare the CRMT minimum forms of Eqs. (5.48) and (5.50) with the EV K-map and two-level results. Reading the loops of Fig. 5.6b in maxterm code (or the submaps in Fig. 5.6a) gives

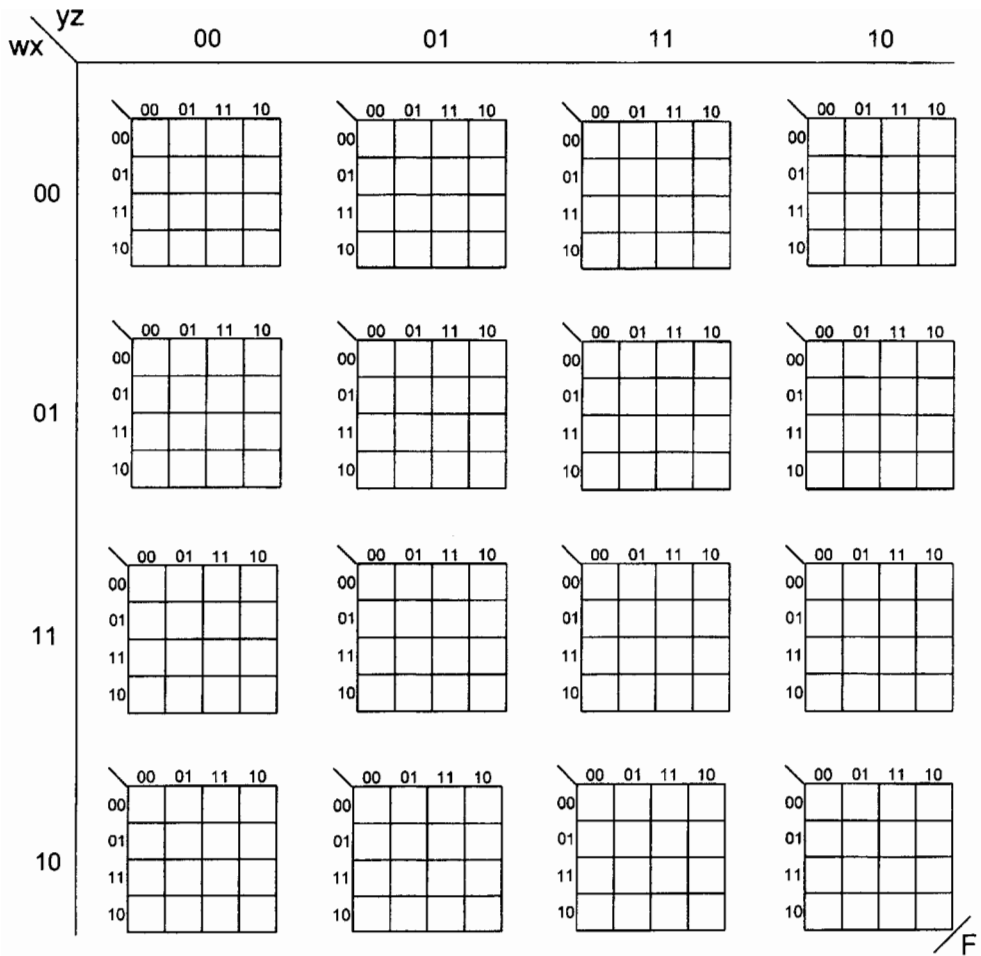
$$f_{K\text{-map}} = (b + a \oplus c \oplus e)(\bar{a} + \bar{b} + e), \quad (5.51)$$

which is a four-level function having a gate/input tally of 5/11 excluding possible inverters. By comparison, the computer-minimized two-level POS minimum result is

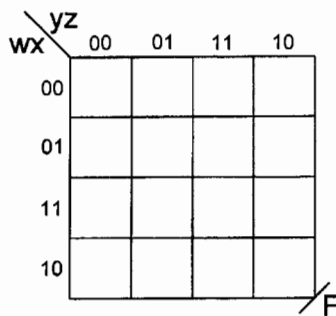
$$f_{POS} = (a + b + \bar{c} + \bar{e})(\bar{a} + b + c + \bar{e})(\bar{a} + \bar{c} + e)(a + b + c + e) \quad (5.52)$$

and has a gate/input tally of 5/19. The SOP minimum result (not shown) has a gate/input tally of 7/22. No attempt is made to minimize function  $f$  by the EXSOP minimization approach, which is best accomplished by computer algorithmic means.

Figure 5.6 illustrates how the CRMT method can be applied to a five-variable function having a two-variable bond set,  $\{a, b\}$  in this case. Shown in Fig. 5.7a is the conventional K-map array suitable for an eight variable function  $F$  having the bond set  $\{w, x, y, z\}$ , and in Fig. 5.7b its fourth-order compression, also for bond set  $\{w, x, y, z\}$ . These K-map formats also suggest a means by which functions with more than eight variables can be minimized by the CRMT method, providing one can deal with EV K-maps within EV K-maps. Large numbers of don't cares would greatly reduce the complexity of the K-map cell entries (and hence  $f$  coefficients) in Fig. 5.7b.



(a)



(b)

**FIGURE 5.7**

(a) K-map array for an eight variable function  $F$  having a bond set  $\{w, x, y, z\}$ . (b) K-map required for a fourth-order compression of function  $F$  having a bond set  $\{w, x, y, z\}$ .

### 5.10 MULTIPLE OUTPUT FUNCTIONS WITH DON'T CARES

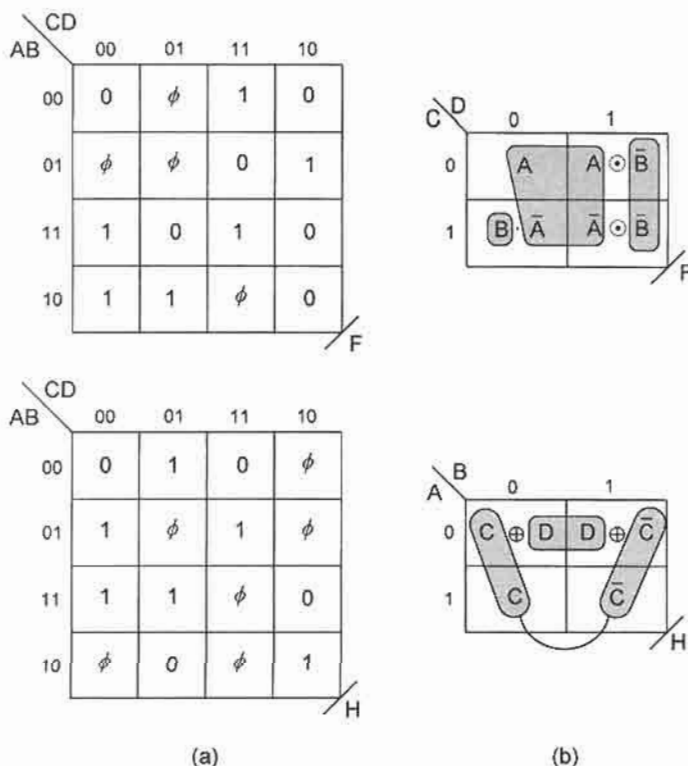
The problem of selecting an optimum bond set is further complicated by the presence of don't cares in multiple output systems. Application of the CRMT minimization procedure given earlier to such systems is illustrated by minimizing the following two four-variable functions containing don't cares:

$$F(A, B, C, D) = \sum m(3, 6, 8, 9, 12, 15) + \phi(1, 4, 5, 11) \quad (5.53)$$

and

$$H(A, B, C, D) = \sum m(1, 4, 7, 10, 12, 13) + \phi(2, 5, 6, 8, 11, 15).$$

The conventional fourth-order K-maps for functions  $F$  and  $H$  are shown in Fig. 5.8a. The don't cares are so chosen as to best meet the requirements of the CRMT minimization procedure for both functions, but with no guarantee of a two-function optimal result. The bond sets are arbitrarily chosen to be  $\{C, D\}$  and  $\{A, B\}$ , respectively, for functions  $F$  and



**FIGURE 5.8**

(a) Conventional K-maps for functions  $F$  and  $H$  of Eqs. (5.53) and (b) their second-order compressions for bond sets  $\{C, D\}$  and  $\{A, B\}$  showing minimum cover for each by using XOR-type patterns.

$H$ , and their canonical SOP/EXSOP transformations now become

$$F = \sum m(3, 5, 6, 8, 9, 12, 15) = \bigoplus (3, 5, 6, 8, 9, 12, 15)$$

and

$$H = \sum m(1, 2, 4, 7, 10, 11, 12, 13) = \bigoplus m(1, 2, 4, 7, 10, 11, 12, 13).$$

Here, for function  $F$ , the don't cares are chosen to be  $\phi_1 = \phi_4 = \phi_{11} = 0$  and  $\phi_5 = 1$ . For function  $H$  the don't cares are assigned the values  $\phi_5 = \phi_6 = \phi_8 = \phi_{15} = 0$  and  $\phi_2 = \phi_{11} = 1$ . The don't-care values are chosen in agreement with STEP (6) of the heuristics for CRMT minimization given previously. Thus, the  $CD$  columns and the  $AB$  rows have simple subfunctions of the type  $X$  and  $X \oplus Y$  to improve chances for an optimum result.

Function  $F$  in Eqs. (5.53) is now recast as the contracted CRMT form

$$\begin{aligned} F_{CD} &= (\bar{C}\bar{D}f_0) \oplus (\bar{C}Df_1) \oplus (C\bar{D}f_2) \oplus (CDf_3) \\ &= (\bar{C}\bar{D})A \oplus (\bar{C}D)(A \oplus B) \oplus (C\bar{D})\bar{A}B \oplus (CD)(\bar{A} \oplus B) \\ &= g_0 \oplus Dg_1 \oplus Cg_2 \oplus CDg_3 \end{aligned} \quad (5.55)$$

for bond set  $\{C, D\}$  and with CRMT coefficients

$$\begin{aligned} g_0 &= A & g_2 &= \bar{A}B \oplus A = 1 \oplus \bar{A}\bar{B} \\ g_1 &= A \oplus B \oplus A = B & g_3 &= \bar{A} \oplus B \oplus \bar{A}B \oplus B = \bar{A}\bar{B}. \end{aligned}$$

Introducing these coefficients into Eq. (5.55) gives the minimized result for  $F_{CD}$

$$\begin{aligned} F_{CD} &= A \oplus BD \oplus C \oplus \bar{A}\bar{B}C \oplus \bar{A}\bar{B}CD \\ &= A \oplus C \oplus BD \oplus \bar{A}\bar{B}C\bar{D}. \end{aligned} \quad (5.56)$$

Following the same procedure for function  $H_{AB}$ , there results

$$\begin{aligned} H_{AB} &= (\bar{A}\bar{B}f_0) \oplus (\bar{A}Bf_1) \oplus (A\bar{B}f_2) \oplus (ABf_3) \\ &= (\bar{A}\bar{B})(C \oplus D) \oplus (\bar{A}B)(\bar{C} \oplus D) \oplus (A\bar{B})C \oplus (AB)\bar{C} \\ &= g_0 \oplus Bg_1 \oplus Ag_2 \oplus ABg_3 \end{aligned} \quad (5.57)$$

for bond set  $\{A, B\}$ . From Fig. 5.8b and Eq. (5.18), the CRMT  $g$  coefficients become

$$\begin{aligned} g_0 &= C \oplus D & g_2 &= C \oplus C \oplus D = D \\ g_1 &= \bar{C} \oplus D \oplus C \oplus D = 1 & g_3 &= \bar{C} \oplus C \oplus 1 = 0, \end{aligned}$$

which, when introduced into Eq. (5.57), give the absolute minimum result

$$\begin{aligned} H_{AB} &= C \oplus D \oplus B \oplus AD \\ &= C \oplus B \oplus \bar{A}D. \end{aligned} \quad (5.58)$$



The two CRMT optimized functions are now expressed together as

$$\begin{cases} F_{CD} = A \oplus C \oplus BD \oplus \bar{A}\bar{B}\bar{C}\bar{D} \\ H_{AB} = C \oplus B \oplus \bar{A}D \end{cases}, \quad (5.59)$$

representing a three-level system with a combined gate/input tally of 8/18, but with no shared terms.

A comparison is now made with other approaches to the minimization of these functions. The EV K-map minimum results read directly from the cover (shown by the loopings) in Figs. 5.8b are

$$\begin{cases} F_{K-map} = [A \odot \bar{C} \odot (\bar{B} + \bar{D})](B + \bar{C} + D) \\ H_{K-map} = B \oplus C \oplus \bar{A}D \end{cases}, \quad (5.60)$$

representing a three-level system having a gate/input tally of 8/17 with no shared terms. Notice that function  $F$  is extracted (looped out) in maxterm code, whereas function  $H$  is extracted in minterm code. The computer-optimized two-level SOP result is

$$\begin{cases} F = A\bar{C}\bar{D} + \bar{A}B\bar{D} + ACD + \bar{B}D \\ H = \bar{B}\bar{C}\bar{D} + \bar{A}\bar{C}D + \bar{A}B + B\bar{C} \end{cases}, \quad (5.61)$$

with a total gate/input tally of 10/29 excluding possible inverters.

For further comparison, these two functions are minimized together as a system by using canonical R-M forms. As a practical matter, an exhaustive search is not carried out on the choice of don't cares and, consequently, an exact EXSOP result cannot be guaranteed. However, a few trial-and-error attempts at minimization indicate that an exact or near-exact result is obtained for function  $F$  if all  $\phi$ 's are taken as logic 1, but that for function  $H$  the don't-care values are taken to be the same as those used by the CRMT method. Therefore, from the conventional K-maps in Fig. 5.8a there result the following canonical R-M forms: For  $F$  the R-M coefficients are

$$g_1 = g_4 = g_5 = g_7 = g_8 = g_9 = g_{10} = g_{11} = g_{12} = 1,$$

and for  $H$  they are

$$g_1 = g_2 = g_4 = g_9 = 1.$$

Introducing the  $g$  values for functions  $F$  and  $H$  into Equation (5.17) separately gives

$$\begin{aligned} F &= D \oplus B \oplus BD \oplus BCD \oplus A \oplus AD \oplus AC \oplus ACD \oplus AB \\ &= \bar{A}D \oplus A\bar{C}\bar{D} \oplus B\bar{C}D \oplus A \oplus AB \oplus B \\ &= \bar{A}D \oplus A\bar{C}\bar{D} \oplus B\bar{C}D \oplus A\bar{B} \oplus B \end{aligned} \quad (5.62)$$

for function  $F$  and

$$H = D \oplus C \oplus B \oplus AD = \bar{A}D \oplus C \oplus B \quad (5.63)$$

for function  $H$ . Then, combining an optimum set of shared EXSOP terms results in the expressions

$$\left\{ \begin{array}{l} F = [B \oplus \bar{A}D] \oplus A\bar{B} \oplus AC\bar{D} \oplus B\bar{C}D \\ H = [B \oplus \bar{A}D] \oplus C \end{array} \right\}. \quad (5.64)$$

This is a four-level system having a total gate/input tally of 9/20, including shared term  $B \oplus \bar{A}D$ .

Comparing results in Eqs. (5.60), (5.61), and (5.64) with those for the minimized CRMT forms in Eqs. (5.59) clearly shows that the CRMT method is competitive with the K-map and two-level minimization methods and illustrates the advantage of simplicity that the CRMT minimization approach has over that of the EXSOP minimization as a pencil-and-paper method.

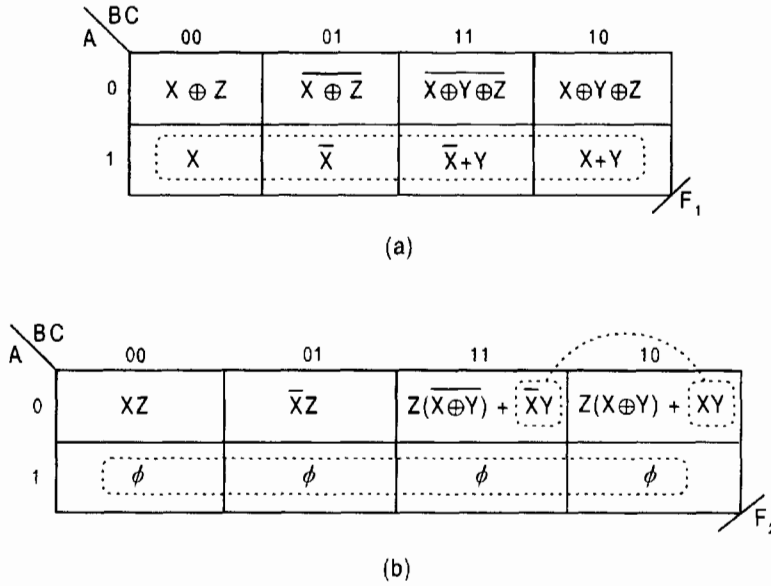
### 5.11 K-MAP SUBFUNCTION PARTITIONING FOR COMBINED CRMT AND TWO-LEVEL MINIMIZATION

Any function can be partitioned in a manner that permits it to be minimized by a combination of the CRMT and two-level methods. Function partitioning for this purpose is best carried out within an EV K-map, hence subfunction partitioning. This partitioning process is significant because with K-map assistance it makes possible the selection of the most tractable (if not optimal) parts of a function for the combined two methods of minimization. This can be of great advantage for a multioutput function where shared term usage is important. There still remains the problem of knowing what is the “best” choice of function partitioning for optimal results. An absolute minimum result in the CRMT approach not only would require an exhaustive search of the best CRMT bond set minimum, but must be accompanied by an exhaustive two-level search. This is no easy task except for, perhaps, relatively simple functions. However, if an absolute minimum result is not sought, there may exist a variety of ways in which a given function can be partitioned without significant change in the cost (complexity) of the resulting minimized function. In any case, the combined minimum forms are classified as partitioned EXSOP/SOP forms or their dual EQPOS/POS.

As a simple example of K-map function partitioning, consider function  $Z_{AC}$  in Fig. 5.4c. Here, the literal  $\bar{D}$  in cell 1 (see dashed loop in domain  $\bar{A}C$ ) is separated out to be later ORed to the CRMT solution as the EPI  $\bar{A}C\bar{D}$ . After removal of the literal  $\bar{D}$ , the CRMT  $g$  coefficients become

$$\begin{array}{ll} g_0 = B \oplus D & g_2 = \bar{B} \oplus B \oplus D = \bar{D} \\ g_1 = B \oplus B \oplus D = D & g_3 = B \oplus \bar{D} \oplus \bar{B} \oplus D = 0. \end{array}$$

Introducing these coefficients into Eq. (5.29) and adding the two level result  $\bar{A}C\bar{D}$  yields

**FIGURE 5.9**

Combined CRMT and two-level minimization of a two-output system of six variables. (a) Third-order compression of function  $F_1$ , (b) third-order compression of function  $F_2$ .

the minimum result

$$\begin{aligned}
 Z_{AC} &= [B \oplus D \oplus \bar{C}\bar{D} \oplus A\bar{D}] + \bar{A}\bar{C}\bar{D}, \\
 &= [B \oplus CD \oplus A\bar{D}] + \bar{A}\bar{C}\bar{D},
 \end{aligned} \tag{5.65}$$

where  $Z_{AC}$  is now a four-level function with a gate/input tally of 6/13. Recall that the CRMT result in Eq. (5.30) is only a three-level function. The extra level in Eq. (5.65) compared to Eq. (5.30) is due to the OR operator of the mixed form.

A more interesting example is the EXSOP/SOP partitioning of the two function system shown in Figs. 5.9a and 5.9b. In this case, all entries in cells 100, 101, 111, and 110 for function  $F_1$  are partitioned out (set to logic 0) for CRMT minimization, but are extracted optimally as shown by the two-level minimum cover. Similarly, for function  $F_2$ , terms  $\bar{X}Y$  and  $XY$  in cells 011 and 010 are partitioned out of the CRMT minimization but are extracted optimally in SOP logic. Also, the don't cares in the  $F_2$  K-map are set to logic 0 for both the CRMT and two-level minimizations.

The minimization process is now carried out on both the function  $F_1$  and  $F_2$  in such a manner as to make effective use of any shared terms that may occur. By using Figs. 5.9a and 5.9b, the function  $F_1$  and  $F_2$  are cast in the form of Eq. (5.17) to give

$$F_1, F_2 = g_0 \oplus Cg_1 \oplus Bg_2 \oplus BCg_3 \oplus Ag_4 \oplus ACg_5 \oplus ABg_6 \oplus ABCg_7. \tag{5.66}$$

After partitioning, CRMT coefficients for function  $F_1$  become

$$\begin{aligned} g_0 &= X \oplus Z & g_4 &= X \oplus Z \\ g_1 &= 1 & g_5 &= 1 \\ g_2 &= Y & g_6 &= Y \\ g_3 &= 0 & g_7 &= 0. \end{aligned}$$

The two-level minimization result for cells 100, 101, 111, and 110 is simply

$$A\bar{C}X + AC\bar{X} + ABY = A(X \oplus C + BY). \quad (5.67)$$

Introducing the  $g$  coefficients into Eq. (5.66) and adding the two-level result gives the mixed minimum result

$$\begin{aligned} F_1 &= X \oplus Z \oplus C \oplus BY \oplus AX \oplus AZ \oplus AC \oplus ABY + A(X \oplus C + BY) \\ &= (X \oplus C) \oplus \bar{A}Z \oplus \bar{A}BY \oplus A(X \oplus C) + A(X \oplus C + BY) \\ &= \bar{A}[Z \oplus (X \oplus C) \oplus BY] + A(X \oplus C + BY). \end{aligned} \quad (5.68)$$

Applying the same procedure to the partitioned  $F_2$  function gives the CRMT  $g$  coefficients

$$\begin{aligned} g_0 &= g_4 = XZ & g_2 &= g_6 = YZ \\ g_1 &= g_5 = Z & g_3 &= g_7 = 0. \end{aligned}$$

From the K-map for  $F_2$ , the two-level minimum result is easily seen to be

$$\bar{A}BC\bar{X}Y + \bar{A}B\bar{C}XY = \bar{A}BY(X \oplus C). \quad (5.69)$$

Now introducing the  $g$  coefficients into Eq. (5.66) and adding the two-level result yields an EXSOP/SOP minimum,

$$\begin{aligned} F_2 &= XZ \oplus CZ \oplus BYZ \oplus AXZ \oplus ACZ \oplus ABYZ + \bar{A}BY(X \oplus C) \\ &= \bar{A}XZ \oplus \bar{A}CZ \oplus \bar{A}BYZ + \bar{A}BY(X \oplus C) \\ &= \bar{A}Z[(X \oplus C) \oplus (BY)] + \bar{A}(BY)(X \oplus C). \end{aligned} \quad (5.70)$$

The combined two-function minimum result is now given by

$$\left\{ \begin{aligned} F_1 &= \bar{A}[Z \oplus (X \oplus C) \oplus (BY)] + A(X \oplus C + BY) \\ F_2 &= \bar{A}Z[(X \oplus C) \oplus (BY)] + \bar{A}(BY)(X \oplus C) \end{aligned} \right\}, \quad (5.71)$$

which represents a five-level system with a combined gate/input tally of 11/24 excluding possible inverters.

The OR operators in Eqs. (5.71) add an extra level of path delay compared to forms that are exclusively EXSOP/SOP. This can be demonstrated by avoiding the partitioning of function  $F_1$ . When this is done the CRMT  $g$  coefficients become

$$\begin{aligned} g_0 &= X \oplus Z & g_4 &= Z \\ g_1 &= 1 & g_5 &= 0 \\ g_2 &= Y & g_6 &= Y \oplus (X + Y) \oplus X = Y \oplus \bar{X}\bar{Y} \oplus \bar{X} = Y \oplus \bar{X}Y = XY \\ g_3 &= 0 & g_7 &= 0 \oplus 1 \oplus X\bar{Y} \oplus \bar{X}\bar{Y} = 1 \oplus \bar{Y} = Y. \end{aligned}$$

Introducing these  $g$  coefficients into Eq. (5.66) gives the EXSOP/SOP result

$$\begin{aligned} F_1 &= X \oplus Z \oplus C \oplus BY \oplus AZ \oplus ABXY \oplus ABCY \\ &= (X \oplus C) \oplus \bar{A}Z \oplus BY \oplus ABY(X \oplus C) \\ &= [\overline{ABY}(X \oplus C)] \oplus (\bar{A}Z) \oplus (BY), \end{aligned} \quad (5.72)$$

which is a four-level function with a gate/input tally of 7/15, exclusive of inverters. This compares to the mixed five-level function  $F_1$  in Eqs. (5.71), which has a gate/input tally of 8/16.

Subfunction partitioning in maxterm code is equally effective in facilitating the minimization process. As a simple example, consider the function  $F_{CD}$  in Fig. 5.8b and the EQPOS CRMT form

$$F_{CD} = g_0 \odot (\bar{D} + g_1) \odot (\bar{C} + g_2) \odot (\bar{C} + \bar{D} + g_3), \quad (5.73)$$

which follows Eq. (5.20). Proceeding with the CRMT minimization, with  $B$  partitioned out of the term  $\bar{A} \cdot B$  in cell 10, gives the CRMT  $g$  coefficients

$$\begin{aligned} g_0 &= A & g_2 &= \bar{A} \odot A = 0 \\ g_1 &= A \odot \bar{B} \odot A = \bar{B} & g_3 &= \bar{A} \odot \bar{B} \odot \bar{A} \odot \bar{B} = 1. \end{aligned}$$

Introducing these coefficients into Eq. (5.73) and adding the two-level result gives

$$F_{CD} = [A \odot (\bar{B} + \bar{D}) \odot \bar{C}] \cdot (B + \bar{C} + D), \quad (5.74)$$

which is exactly the same as the K-map minimum result in Eqs. (5.60).

Notice that the mixed CRMT/two-level method requires that the partitioning be carried out in either minterm or maxterm code form. Thus, if subfunctions of the type  $X + Y$  are partitioned, the entire minimization process must be carried out in minterm code. Or, if terms such as  $X \cdot Y$  are partitioned, the minimization process must be carried out in maxterm code. Note that either  $X$  or  $Y$  or both may represent multivariable functions or single literals of any polarity.

### 5.12 PERSPECTIVE ON THE CRMT AND CRMT/TWO-LEVEL MINIMIZATION METHODS

The main advantage of the CRMT method of function minimization lies in the fact that it breaks up the minimization process into tractable parts that are amenable to pencil-and-paper or classroom application. The CRMT minimization process can be thought of as consisting of three stages: the selection of a suitable bond set, the optimization of the CRMT  $g$  coefficients (for the chosen bond set), and the final minimization stage once the  $g$  coefficients have been introduced into the CRMT form. If an exact minimum is not required, a suitable bond set can be easily found, permitting the CRMT method to be applied to functions of as many as eight variables or more. Knowledge of the use of EV K-maps and familiarity with XOR algebra are skills essential to this process. A properly conducted CRMT minimization can yield results competitive with or more optimum than those obtained by other means.

It has been shown that minimization by the CRMT method yields results that are often similar to those obtained by the EV K-map method described in Section 5.4. This is particularly true when the EV K-map subfunctions are partitioned so as to take advantage of both the CRMT and two-level (SOP or POS) minimization methods. In fact, when subfunction partitioning is carried out in agreement with the minimum K-map cover (as indicated by loopings), the CRMT/two-level result is often the same as that obtained from the K-map. It is also true that when a function is partitioned for CRMT and two-level minimizations, an extra level results because of the OR (or AND) operator(s) that must be present in the resulting expression. Thus, a CRMT/two-level (mixed) result can be more optimum than the CRMT method (alone) only if the reduction in the gate/input tally of the CRMT portion of the mixed result more than compensates for the addition of the two-level part. At this point, this can be known only by a trial-and-error-method that is tantamount to an exhaustive search.

If an exact or absolute minimum CRMT result is sought, an exhaustive search must be undertaken for an optimum bond set. Without computer assistance this can be a tedious task even for functions of four variables, particularly if the function contains don't cares. Multiple-output systems further complicate the exhaustive search process and make computer assistance all the more necessary. One advantage of the mixed CRMT/two-level approach to function minimization is that each method can be carried out independently on more tractable parts.

### FURTHER READING

Additional information on XOR algebra, XOR function extraction from K-maps, and logic synthesis with XOR and EQV gates can be found in the texts of Roth, Sasao (Ed.), and Tinder.

- [1] C. H. Roth, *Fundamentals of Logic Design*, 4th ed. West, St. Paul, MN 1992 (Chapter 3).
- [2] T. Sasao, "Logic Synthesis with XOR Gates," in *Logic Synthesis and Optimization* (T. Sasao, Ed). Kluwer, 1993 (see, e.g., Chapter 12).
- [3] R. F. Tinder, *Digital Engineering Design: A Modern Approach*. Prentice Hall, 1991 (see, e.g., Chapter 3).

- [4] R. F. Tinder, "Multilevel Logic Minimization by Using K-map XOR Patterns," *IEEE Trans. on Ed.* **38**(4), 370–375 (1995).

Earlier work on Reed–Muller expansions and the use of conventional K-map methods to obtain Reed–Muller coefficient values can be found in the work of Dietmeyer and Wu *et al.*

- [5] D. L. Dietmeyer, *Logic Design of Digital Systems*. Allyn and Bacon, 1978 (Chapter 2).  
 [6] X. Wu, X. Chen, and S. L. Hurst, "Mapping of Reed–Muller Coefficients and the Minimization of Exclusive-OR Switching Functions," *Proc. IEE, Part E*, **129**, 15–20 (1982).

An excellent experimental study of the various XOR and EQV (XNOR) CMOS gate configurations can be found in the work of Wang, Fang, and Feng.

- [7] J. Wang, S. Fang, and W. Feng, "New Efficient Designs for XOR and XNOR Functions on the Transistor Level," *IEEE Journal of Solid-State Circuits* **29**(7), 780–786 (1994).

Many approaches to the decomposition and minimization of multilevel (Reed–Muller) forms can be found in literature. A few representative works are cited below:

- [8] D. Bochman, F. Dresig, and B. Steinbach, "A New Decomposition Method for Multilevel Circuit Design," The European Conference on Design Automation, Amsterdam, The Netherlands, 25–28 Feb. 1991, pp. 374–377.  
 [9] H. M. Fleisher and J. Yeager, "A Computer Algorithm for Minimizing Reed–Muller Canonical Forms," *IEEE Trans. Comput.* **36**(2), 247–250 (1987).  
 [10] J. M. Saul, "An Algorithm for the Multi-level Minimization of Reed–Muller Representations," *IEEE Int. Conf. on Computer Design: VLSI in Computers and Processors* (Cat. No. 91CH3040-3), pp. 634–637. IEEE Computer Soc. Press, Los Alamitos, CA, 1991.  
 [11] T. Sasao, "Logic Synthesis with XOR Gates," in *Logic Synthesis and Optimization* (T. Sasao, Ed.), Kluwer, 1993, pp. 259–285.  
 [12] N. Song and M. A. Perkowski, "EXORCISM-MV-2: Minimization of Exclusive Sum of Products Expressions for Multiple-valued Input Incompletely Specified Functions," Proc. of the 23rd International Symposium on Multiple-Valued Logic, ISMVL '93, Sacramento, CA, May 24–27, 1993, pp. 132–137.  
 [13] W. Wan and M. A. Perkowski, "A New Approach to the Decomposition of Incompletely Specified Functions Based on Graph-Coloring and Local Transformations and its Application to FPGA Mapping," Proc. of the IEEE EURO-DAC '92 European Design Automation Conference, Hamburg, Sept. 7–10, Hamburg, 1992, pp. 230–235.

## PROBLEMS

**Note:** Most K-map minimization results of problems that follow can be verified by introducing the binary coordinates of each K-map cell into the resulting expression. Generation of each cell subfunction by this means validates the extraction results. In some cases, it may be necessary to construct a suitable EV K-map for this purpose. Also, to obtain correct answers for these problems, the reader will be required to make frequent use of the laws, corollaries, and identities of XOR algebra given in Section 3.11.

- 5.1** Compress the following function into a first-order K-map of axis  $A$ , and loop out a gate minimum expression by using XOR-type patterns in minterm code. Next, obtain the SOP minimum from the same K-map and compare the gate/input tallies for both the XOR and SOP forms. Finally, construct the logic circuits for the XOR and SOP results assuming that the inputs and output are all active high. What do you conclude from these comparisons?

$$E = \bar{A}\bar{X}\bar{Y} + \bar{A}XY + AY$$

- 5.2** The output  $F$  of a logic circuit is a function of three inputs  $A$ ,  $B$ , and  $C$ . The output goes active under any of the following conditions as read in the order ABC:

All inputs are logic 1

An odd number of inputs are logic 1

None of the inputs are logic 1

- Construct a truth table for output function  $F$  and inputs  $ABC$ .
  - Map the result in a second-order K-map and extract a gate-minimum expression by using XOR-type patterns.
  - Further compress this function into a first-order K-map of axis  $A$  and again extract a gate-minimum expression by using XOR-type patterns. Compare the result with that of (b).
  - Finally, place this function in a conventional (1's and 0's) K-map and extract minimum two-level SOP and POS logic expressions. By using the gate/input tally (exclusive of inverters), compare the results with those of (b) and (c). What do you conclude from this comparison?
- 5.3** Compress the following function into a *second-order* K-map with axes as indicated and extract a gate-minimum expression for each set of axes by using XOR patterns. Use the gate/input tally, exclusive of possible inverters, to compare this result with the minimum expressions for the two-level SOP and POS results. What do you conclude from this comparison? What is the gate delay level for the XOR pattern results? (Hint: It will be helpful to first plot this function into a conventional 1's and 0's K-map.)

$$F(W, X, Y, Z) = \sum m(0, 2, 5, 7, 9, 11, 12)$$

- Axes  $W, X$
- Axes  $Y, Z$
- Axes  $X, Y$

- 5.4** Shown in Fig. P5.1 are six EV K-maps that contain XOR-type patterns and that represent two and three levels of compression. Use *minterm code* to loop out a gate-minimum cover for each by using XOR patterns (where appropriate). For comparison, loop out a minimum two-level SOP cover for each and compare their relative complexity by using the gate/input tally exclusive of possible inverters. Also, as part of the comparison, comment on the fan-in requirements for each.



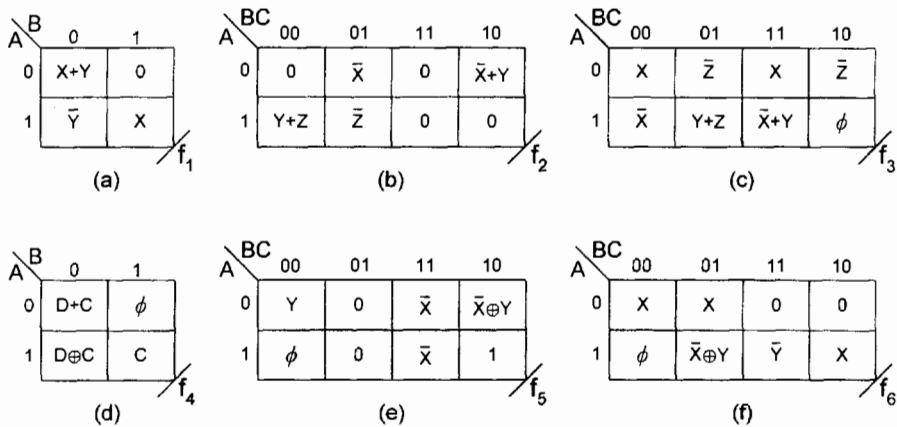


FIGURE P5.1

- 5.5** Use *maxterm code* and XOR-type patterns to loop out a gate-minimum expression for each of the five functions represented in Fig. P5.2. Give the gate/input tally for each and compare that value with the gate/input tallies for the two-level SOP and POS minimum expressions obtained from the same K-maps. [Hint: To obtain the two-level expressions from the K-maps in Figs. 5.2d and 5.2e, it will be necessary to expand the XOR and EQV subfunctions by using their defining relations given by Eqs. (3.4) and (3.5).]
- 5.6** Compress each of the following functions into a second-order K-map with axes  $A$ ,  $B$  and loop out a gate-minimum expression for each by using XOR-type patterns where appropriate. Obtain the two-level SOP and POS minimum result and use the gate/input tally (exclusive of possible inverters) to compare the multi-level result. (Hint: Consider both minterm and maxterm codes when looping out XOR-type patterns for gate-minimum results.)

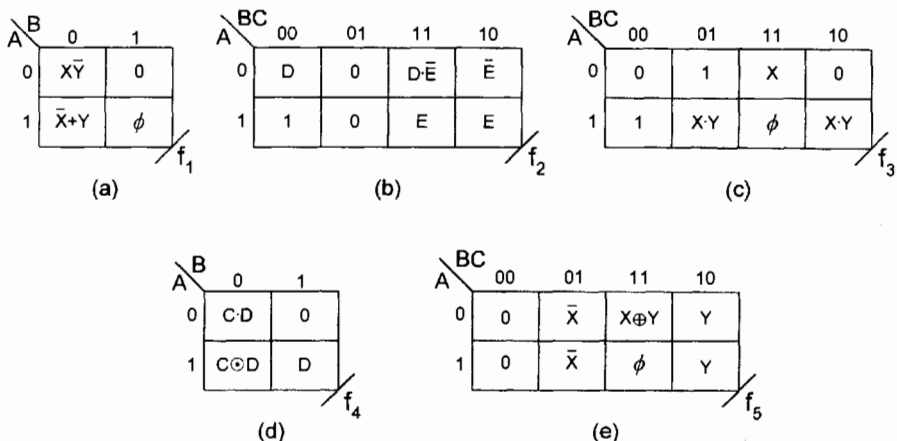


FIGURE P5.2

- (a)  $W(A, B, C, D) = \sum m(3, 6, 9, 12)$   
 (b)  $X(A, B, C, D) = \prod M(2, 3, 4, 5, 7, 8, 9, 11, 14, 15)$   
 (c)  $Y(A, B, C, D) = \sum m(1, 2, 4, 7, 11, 13, 14)$   
 (d)  $Z(A, B, C, D) = \prod M(0, 3, 4, 6, 9, 10, 13)$

**5.7** The following incompletely specified function contains XOR-type patterns:

$$G(A, B, C, D) = \prod M(0, 1, 2, 3, 8, 11, 12, 13) \cdot \phi(4, 5, 6, 7).$$

- (a) Compress the following function into a second-order K-map of axes  $A, B$  and loop out a gate-minimum expression by using XOR-type patterns where appropriate. (Hint: Consider both minterm and maxterm codes and the best use of the don't cares when looping out XOR-type patterns for a gate-minimum result.)
- (b) Use the same K-map to extract minimum SOP and POS expressions for this function. Compare the gate/input tallies (exclusive of possible inverters) for the XOR result with those for the SOP and POS results. What do you conclude from these comparisons?
- (c) Construct the logic circuit for both the XOR result and the SOP result, assuming that the inputs and output are all active high.
- 5.8** Use XOR-type patterns to extract a gate-minimum expression for each of the three functions represented in Fig. P5.3. Use the gate/input tally (exclusive of inverters) to compare the multilevel result with that for the two-level SOP and POS minimum result. Note that compound XOR-type patterns may exist. [Hint: For  $f_2$  and  $f_3$ , it will be necessary to make use of Eqs. (3.27).]
- 5.9** A computer program has been written that will yield a minimum solution to a combinational logic function, but only in SOP form. It accepts the data in either conventional (1's and 0's) form or in two-level EV SOP form—it does not recognize the XOR or EQV operators.
- (1) Given the functions  $F_1$  and  $F_2$  represented by the EV K-maps in Fig. P5.4, extract a gate-minimum expression from each in maxterm code by using the pencil-and-paper method and XOR-type patterns.
- (2) By following Example 2 in Section 4.8, outline the procedure required to “trick” the computer program into yielding a two-level minimum expression from the K-maps in Fig. P5.4, that can be easily converted to minimum POS form. (Hint: It will be necessary to complement the subfunction in each cell of the K-map and represent it in SOP form.)

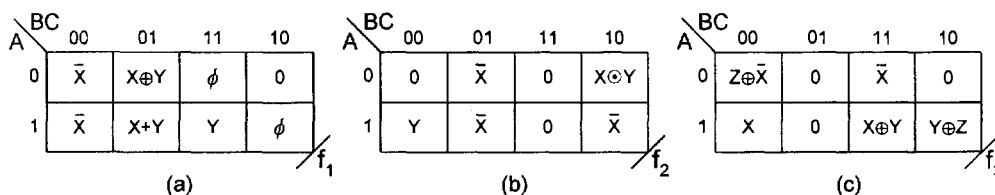


FIGURE P5.3

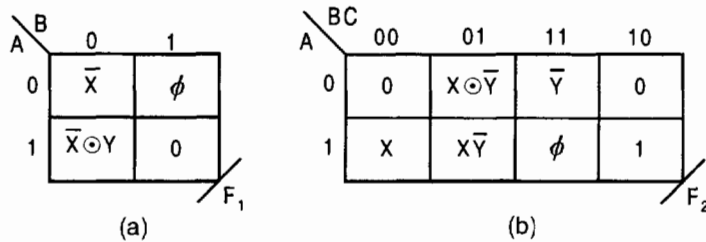


FIGURE P5.4

- (3) Use the procedure in part 2 to obtain the two-level POS expression for function  $F_1$  in Fig. P5.4a. Next, convert each cell of the original K-map to two-level POS subfunction form and extract a two-level POS minimum expression from it by using maxterm code. Should it agree with the results obtained by using the procedure of part 2? Explain.
- (4) Repeat part 3 for function  $F_2$  in Fig. P5.4b.
- 5.10** Repeat Problem 5.3 by using the CRMT method, taking each bond set as the axis indicated in the problem. Use the gate/input tally (exclusive of possible inverters) to compare the CRMT results with the two-level SOP minimum in each case.
- 5.11** Use the canonical Reed–Muller (R–M) approach to obtain an absolute minimum for the function  $F$  given in Problem 5.3. Compare the results with the two-level SOP minimum result by using the gate/input tally (exclusive of possible inverters).
- 5.12** Use the CRMT method to obtain an absolute minimum for the function  $G$  in Problem 5.7 by taking axes  $A, B$  as the bond set. Use the gate/input tally (exclusive of possible inverters) to compare the CRMT result with the two-level SOP minimum result.
- 5.13** Use the CRMT method to obtain an absolute minimum for each of the four functions given in Problem 5.6. Take axes  $A, B$  as the bond set for each. Construct the logic circuit for each CRMT minimum function assuming that all inputs and outputs are active high. Also, for comparison, construct the logic circuit for the minimum two-level SOP or POS minimum result, whichever is the simpler in each case.
- 5.14** Use the canonical R–M approach to obtain a gate-minimum for the four functions given in Problem 5.6. Then, by using the gate/input tally (exclusive of possible inverters), compare these results with the two-level SOP or POS minimum results, whichever is the simpler in each case.
- 5.15** (a) The following two functions are to be optimized together as a system by using the multiple-output CRMT method discussed in Section 5.10. To do this, collapse each function into a third-order K-map with axes  $A, B, C$  and then use the CRMT approach in minterm code to minimize each function while making the best use possible of shared terms. Plan to use  $\{A, B, C\}$  as the bond set.

$$F_1(A, B, C, D, E) = \sum m(2, 3, 4-7, 9, 11, 12, 15, 21, 23, 25, 27)$$

$$F_2(A, B, C, D, E) = \sum m(4, 5, 10, 11, 13, 15-17, 20, 23-25, 30, 31)$$

- (b) Obtain the optimized two-level SOP results for the two functions and compare them with the results of part (a) by using the gate/input tally (including inverters) assuming that the inputs and outputs are all active high.
- (c) Construct the logic circuits for the circuits of parts (a) and (b).

**5.16** (a) Use *subfunction partitioning* of the following function for CRMT/two-level minimization in minterm code. To do this, collapse this function into a third-order K-map of axes  $A, B, C$  and follow the discussion given in Section 5.11. Choose  $\{A, B, C\}$  as the bond set for the CRMT portion.

$$F(A, B, C, D, E) = \sum m(4, 7, 10-12, 14, 16-19, 21, 23, 24-27, 28, 30)$$

- (b) Without partitioning, use the CRMT method to obtain a gate-minimum for this function. Compare this result with that of (a) by using the gate/input tally exclusive of inverters.
- 5.17** A function  $F$  is to be activated by the use of three switches,  $A, B$ , and  $C$ . It is required that the function  $F$  be active iff a single switch is active. Thus, if any two or three of the switches are active the function must be inactive. Design a gate minimum circuit for the function  $F$  consisting of three XOR gates and an AND gate (nothing else). Assume that the inputs and output are all active high. (Hint: Apply the CRMT method.)

*This page intentionally left blank*

# Nonarithmetic Combinational Logic Devices

---

## 6.1 INTRODUCTION AND BACKGROUND

It is the purpose of combinational logic design to build larger, more sophisticated logic circuits by using the most adaptable and versatile building blocks available. The choice of discrete gates as the building blocks is not always a good one, owing to the complex nature of the circuits that must be designed and to the fact that there are integrated circuit (IC) packages available that are much more adaptable. It is the plan of this chapter to develop these building blocks and demonstrate their use in construction of larger combinational logic systems. Brief discussions of the various device performance characteristics and a design procedure are provided in advance of the logic device development.

### 6.1.1 The Building Blocks

It is well understood that the digital designer must be able to create combinational circuits that will perform a large variety of tasks. Typical examples of these tasks include the following:

- Data manipulation (logically and arithmetically)
- Code conversion
- Combinational logic design
- Data selection from various sources
- Data busing and distribution to various destinations
- Error detection

To implement circuits that will perform tasks of the type listed, the logic designer can draw upon an impressive and growing list of combinational logic devices that are commercially available in the form of IC packages called *chips*. Shown in Fig. 6.1 is a partial listing of the combinational logic chips, those that are of a nonarithmetic type (a) and those that are arithmetic in character (b). Only the devices in Fig. 6.1a will be considered in this chapter.

| (a)<br>Non-Arithmetic Combinational Logic<br>Devices                                                                                                          | (b)<br>Arithmetic-Type Combinational Logic<br>Circuits                         |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Multiplexers (Data Selectors)<br>Decoders/Demultiplexers<br>Priority Encoders<br>Code Converters<br>Comparators<br>Parity Detectors<br>Combinational Shifters | Adders<br>Subtractors<br>Arithmetic and Logic Units<br>Multipliers<br>Dividers |

**FIGURE 6.1**

Partial lists of available nonarithmetic IC devices and arithmetic IC devices.

### 6.1.2 Classification of Chips

IC chips for the devices of the type listed in Fig. 6.1 can be classified as small-scale integrated (SSI) circuits, medium-scale integrated (MSI) circuits, large-scale integrated (LSI) circuits, very-large-scale integrated (VLSI) circuits, and wafer-scale integrated (WSI) circuits. It has become customary to assign one of the preceding acronyms to a given IC circuit on the basis of the number of equivalent fundamental gates (meaning AND, OR, Inverter or NAND, NOR, Inverter) that are required to implement it. By one convention, these acronyms may be assigned the following gate count ranges:

SSI circuits: up to 20 gates

MSI circuits: 20 to about 200 gates

LSI circuits: 200 to thousands of gates

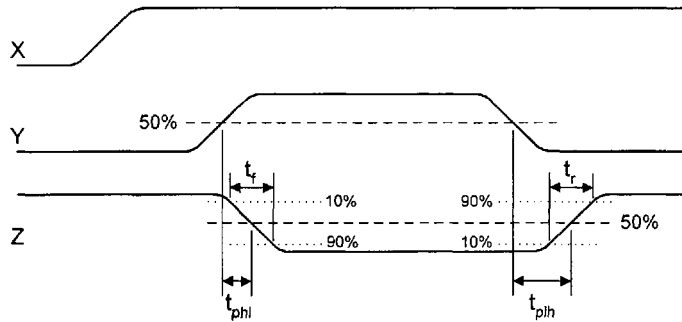
VLSI circuits: thousands to millions of gates

WSI chips might contain tens to hundreds of VLSI circuits. This classification scheme is obviously ineffective in revealing the true complexity of a given IC relative to the digital system in which it operates. For example, an LSI chip might be a 64-bit adder or it might be a moderately complex microprocessor. Thus, the reader should exercise caution when evaluating the complexity of a chip based on some count system. Finally, it is now common practice for logic designers to design chips for a limited, specific application. Such chips are called *application-specific* ICs, or ASICs, and may differ greatly from the usual commercial chips. ASICs can reduce total manufacturing costs and can often provide higher performance than is possible by combining commercially available devices.

### 6.1.3 Performance Characteristics and Other Practical Matters

The most desirable features a designer would want in a switching device, say, for integrated circuit applications are as follows:

- Fast switching speed
- Low power dissipation
- Wide noise margins

**FIGURE 6.2**

Voltage waveforms showing propagation delays and rise and fall times for a 2-input NAND gate with output Z as in Fig. 3.10.

- High fan-out capability
- High packing density
- Low cost

Although no single family or technology has all these desirable features, some may come close, at least for most of those listed above. A summary of these and other practical matters now follows.

**Propagation Delay (Switching Speed) and Rise and Fall Times** The propagation delay or switching speed of a device is the *measured output response to an input change*. Typically, a given logic circuit will have many outputs and many inputs with various input-to-output paths, each with a different path delay. Furthermore, propagation delays usually differ for output changes that are low-to-high ( $t_{plh}$ ) compared to those that are high-to-low ( $t_{phl}$ ), but both of which are measured from the 50% point of the input signal to the 50% point of the output response signal as illustrated in Fig. 6.2. The average propagation delay for a given input-to-output path is then given by

$$\tau_{p(avg)} = \frac{t_{plh} + t_{phl}}{2}, \quad (6.1)$$

where, typically,  $t_{plh} > t_{phl}$ . Since several input-to-output paths may be involved, the timing specifications given by manufacturers often include typical extremes in propagation delay data. A *minimum* value for  $\tau_p$  is the smallest propagation delay that the logic device will ever exhibit; the *maximum* value is the delay that will “never” be exceeded. The maximum value is the one of most interest to designers since it is used to determine useful factors of safety. For modern CMOS, these values lie in the range of 0.1 to 10 ns. Also shown in Fig. 6.2 are the rise and fall times,  $t_r$  and  $t_f$ , as measured between the 10% and 90% marks of a given waveform.

**Power Dissipation** Logic devices consume power when they perform their tasks, and this power is dissipated in the form of heat, Joule heat. Of the various logic families,



CMOS devices consume the least power and then only during switching intervals — that is, dynamic power dissipation. Thus, CMOS power consumption is frequency dependent and may become quite large at very high frequencies. The bipolar families of devices (see Appendix A) consume the most power, mainly due to leakage current, but are much faster than CMOS. Other logic families tend to fall in between these two extremes.

A useful figure of merit for logic devices is called the *power–delay product* (PDP) given by

$$PDP = P_{Consumption} \times \tau_{p(avg)}, \quad (6.2)$$

which is the product of the power consumed by the device and its average propagation delay given by Eq. (6.1). The PDP of a device is sometimes called the *speed–power product* and is usually expressed in picojoules ( $1 \text{ mW} \times 1 \text{ ns} = 1 \text{ pJ} = 10^{-12} \text{ joules}$ ). Since it is desirable for a given logic device to have both a low power consumption and a small propagation delay, a low PDP is also desirable.

**Noise Margins** The noise margin of a logic device is the largest voltage that can be added to or subtracted from the logic voltage and still maintain the required logic level. The noise margins are defined as

$$\begin{cases} NML = V_{IL\max} - V_{OL\max} \\ NMH = V_{OH\min} - V_{IH\min} \end{cases} \quad (6.3)$$

and are shown in Fig. 3.2. The voltage parameters defined by manufacturers are expressed as follows:

|              |                                                                   |
|--------------|-------------------------------------------------------------------|
| $V_{IL\max}$ | Maximum input voltage guaranteed to be recognized as LOW level.   |
| $V_{OL\max}$ | Maximum output voltage guaranteed to be recognized as LOW level.  |
| $V_{OH\min}$ | Minimum output voltage guaranteed to be recognized as HIGH level. |
| $V_{IH\min}$ | Minimum input voltage guaranteed to be recognized as HIGH level.  |

As an example, typical values for high speed (HC) CMOS are  $V_{IL\max} = 0.3V_{DD}$ ,  $V_{IH\min} = 0.7V_{DD}$ , with  $V_{OL\max}$  being slightly above zero voltage and  $V_{OH\min}$  being slightly below the supply level  $V_{DD}$ .

CMOS logic has always been considered as having good noise margins. However, in the low submicron ranges, CMOS noise margins have been reduced to relatively low values. The bipolar families are usually considered to have good noise margins. It is important that the noise margins of logic devices be wider than any noise transients that may occur so as to prevent unrecoverable errors in the output signals. Thus, noise margins may be regarded as the margins of safety within which digital systems must be operated if their behavior is to be predictable.

**Fan-out and Fan-in** Since the output from a switching device (for example, a gate) has a definite limit to the amount of current it can supply or absorb, there is a definite limit to the number of other switching devices that can be driven by a single output from that switch.

This limit is called the *fan-out* of a given device and is, in effect, the worst-case loading specification for that device. The fan-out limit is usually given in microamps ( $\mu\text{A}$ ). If the fan-out limit of a device is exceeded, the signal can be degraded. MOS circuits are least affected by fan-out restrictions, whereas members of the bipolar families are dramatically affected by such restrictions. Propagation delay is essentially unaffected by fan-out limitations.

The maximum number of inputs permitted to control the operation of a digital device (usually a gate) is called the *fan-in*. Thus, a gate with four inputs has a fan-in of 4. In general for CMOS gates propagation delay increases with increasing fan-in. Fan-in and its consequences are discussed in Subsections 3.6.2 and 3.6.3 and in Section 4.10.

**Cost** The cardinality or cover of a function is a measure of the *cost* of that function. Design area is also a measure of the cost of a function and is called *area cost*. Thus, the cardinality or design area of a function can be given a monetary value, and this is what is of particular interest to manufacturers of digital devices. But there are more factors that contribute to the monetary cost of an IC. To one extent or another all of the factors previously mentioned directly or indirectly affect the cost of an IC. Appendix A gives qualitatively the performance characteristics as a measure of cost for commonly used IC logic families.

#### 6.1.4 Part Numbering Systems

Typically, parts in data books are given specific part numbers indicative of the logic function they perform and the logic family to which they belong. Commercially available digital devices belonging to the CMOS and TTL (*transistor-transistor logic*) families are given the part prefix “74xnnn”, where the “x” represents a string of literals indicating the logic family or subfamily and “nnn” is the part number. To understand this nomenclature the following literals are defined: H = High-speed, L = Low-power, A = Advanced, F = Fast, C = CMOS, and S = Schottky. For example, 74HC00 is a two-input high-speed CMOS NAND gate and a 74AS27 is a three-input advanced Schottky NOR gate. To avoid referring to any specific logic family or subfamily, the “x” descriptor is used along with the part number. For example, a 74x151 is an 8-to-1 multiplexer of a generic type, meaning that it belongs to any of the families for which the prefix “74 . . .” is applicable. The TTL subfamilies designated 74nnn, 74Lnnn, and 74Hnnn have been made obsolete by the presence of modern Schottky subfamilies.

Another member of the bipolar family is called ECL for *emitter-coupled logic*. The ECL family is currently the fastest of the logic families but has an extremely high power consumption and high PDP. ECL parts are named either with a 5-digit number system (10nnn) or a 6-digit system (100nnn), depending on which subfamily is being referenced. In either case all part numbers “nnn” are always three digits in length, unlike those for CMOS and TTL families, which can be two or three digits in length. Appendix A qualitatively summarizes the performance characteristics of TTL, ECL, NMOS, and CMOS families.

#### 6.1.5 Design Procedure

The design of any combinational logic device generally begins with the description of and specifications for the device and ends with a suitable logic implementation. To

assist the reader in developing good design practices, the following six-step sequence is recommended:

- Step 1: Understand the device.** Describe the function of the device; then clearly indicate its input/output (I/O) specifications and timing constraints, and construct its block diagram(s).
- Step 2: State any relevant algorithms.** State all algorithms and/or binary manipulations necessary for the design. Include a general operations format if necessary.
- Step 3: Construct the truth tables.** From step 2, construct the truth tables that detail the I/O relationships. Truth tables are usually presented in positive logic form.
- Step 4: Obtain the output functions.** Map or use a minimization program to obtain any minimum or reduced expressions that may be required for the output functions.
- Step 5: Construct the logic diagrams.** Use either a gate or modular level approach (or both) to implement the logic expressions obtained in step 4. Implement from output to input, taking into account any mixed logic I/O conditions and timing constraints that may be required.
- Step 6: Check the results.** Check the final logic circuit by simulation before implementation as a physical device. Real-time tests of the physical device should be the final test stage.

This text follows the six-step sequence where appropriate and does so without specifically mentioning each step.

## 6.2 MULTIPLEXERS

There is a type of device that performs the function of selecting one of many data input lines for transmission to some destination. This device is called a *multiplexer* (*MUX* for short) or *data selector*. It requires  $n$  data select lines to control  $2^n$  data input lines. Thus, a MUX is a  $2^n$ -input/1-output device (excluding the  $n$  data select inputs) identified by the block diagram in Fig. 6.3a. Shown in Fig. 6.3b is the mechanical switch equivalent of the MUX. Notice that the function of the enable ( $EN = G$ ) is to provide a disable capability to the device. Commercial MUX ICs usually come with active low enable,  $EN(L)$ .

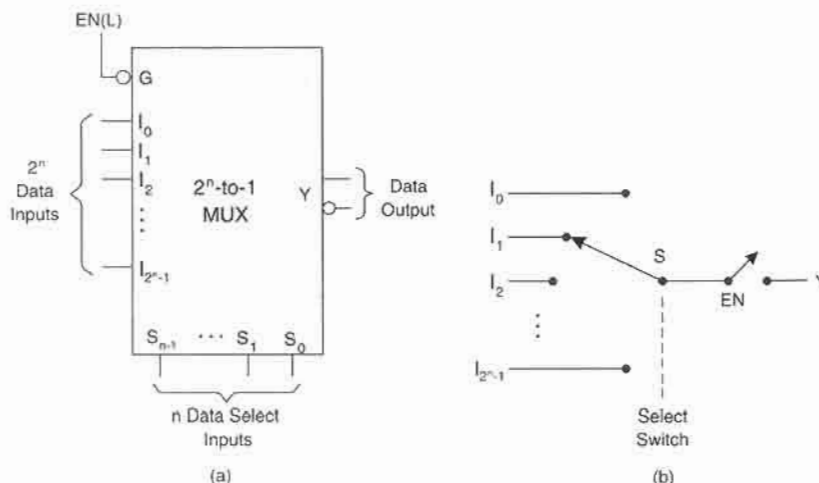
The general logic equation for the MUX of Fig. 6.3 can be expressed as

$$Y = \sum_{i=0}^{2^n-1} (m_i \cdot I_i) \cdot EN, \quad (6.4)$$

where  $m_i$  represents the  $i$ th minterm of the data select inputs (e.g.,  $m_2 = \bar{S}_{n-1} \cdots \bar{S}_2 S_1 \bar{S}_0$ ). The validity of this equation will be verified in the following subsection on multiplexer design.

### 6.2.1 Multiplexer Design

The easiest and most “logical” way to design a MUX is to represent the MUX by a compressed, entered variable (EV) truth table. This is illustrated by the design of a 4-to-1

**FIGURE 6.3**

A  $2^n$ -to-1 multiplexer (MUX) or data selector. (a) Block diagram symbol. (b) Mechanical switch equivalent.

( $2^2$ -to-1) MUX with active low enable. Shown in Fig. 6.4 are the logic symbol, EV truth table, and K-map for the 4-to-1 MUX. From the K-map there result the output expressions given by

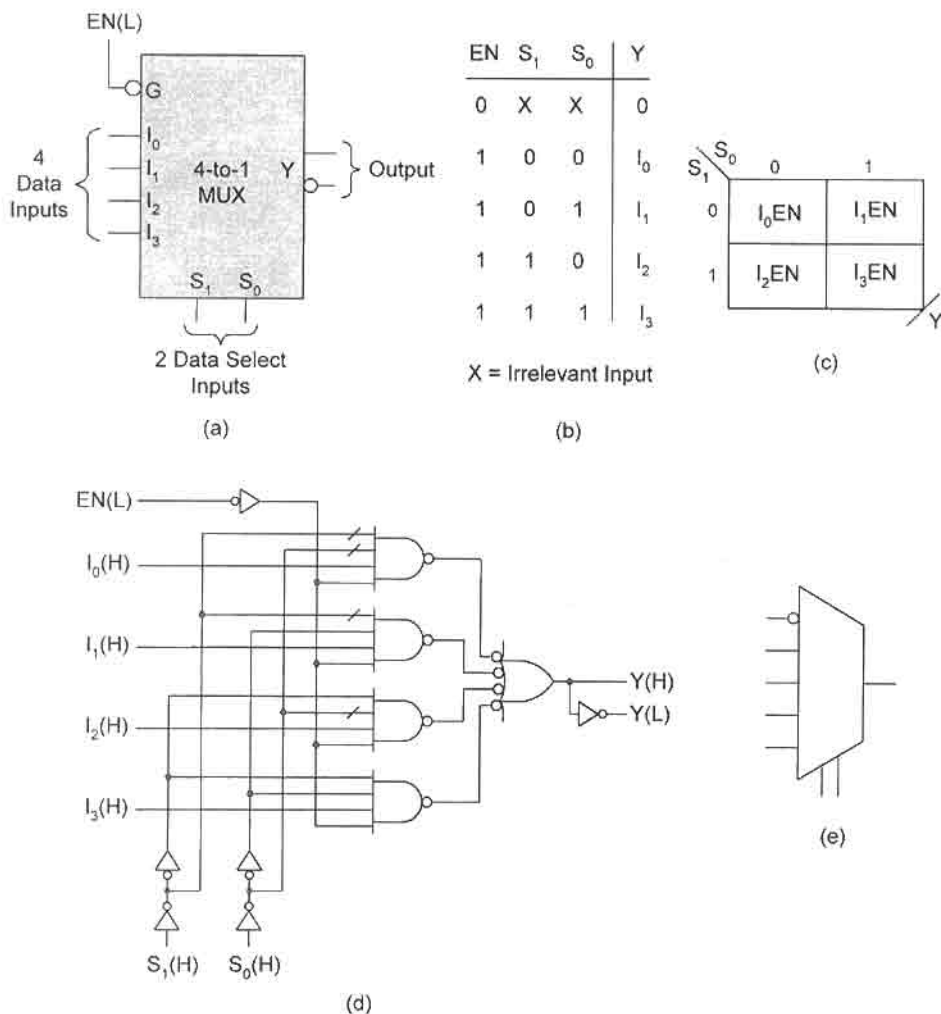
$$\begin{aligned}
 Y &= \bar{S}_1 \bar{S}_0 I_0 EN + \bar{S}_1 S_0 I_1 EN + S_1 \bar{S}_0 I_2 EN + S_1 S_0 I_3 EN \\
 &= m_0 I_0 EN + m_1 I_1 EN + m_2 I_2 EN + m_3 I_3 EN \\
 &= \sum_{i=0}^{2^2-1} (m_i \cdot I_i) \cdot EN,
 \end{aligned} \tag{6.5}$$

representing four data input lines, two data select lines, and one output. The circuit for the 4-to-1 MUX is obtained directly from Eq. (6.5) and is presented in Fig. 6.4d, together with its shorthand circuit symbol given in Fig. 6.4e.

An  $m \times 2^n$  input MUX can be produced by stacking  $m$   $2^n$ -to-1 MUXs with outputs to an  $m$ -to-1 MUX output stage. This is illustrated in Fig. 6.5, where four 8-to-1 (74x151) MUXs are stacked to produce a 32-to-1 MUX. Notice that this MUX can be disabled simply by using the EN(L) line to the output stage MUX. For an explanation of the part identification notation (e.g., 74x...), see Subsection 6.1.4.

Many variations of the stacked MUX configuration are possible, limited only by the availability of different MUX sizes. For example, two 16-to-1 MUXs combine to form a 32-to-1 MUX or four 4-to-1 MUXs combine to produce a 16-to-1 MUX. In the former case a 2-to-1 MUX must be used to complete the stack configuration, whereas in the latter case a 4-to-1 MUX is required. There are other ways in which to stack and package MUXs. One variation is illustrated in the discussion that follows.

More than one MUX can be packaged in an IC, and this can be done in a variety of ways. One configuration is illustrated by the design of the 74x153 4-input/2-bit MUX

**FIGURE 6.4**

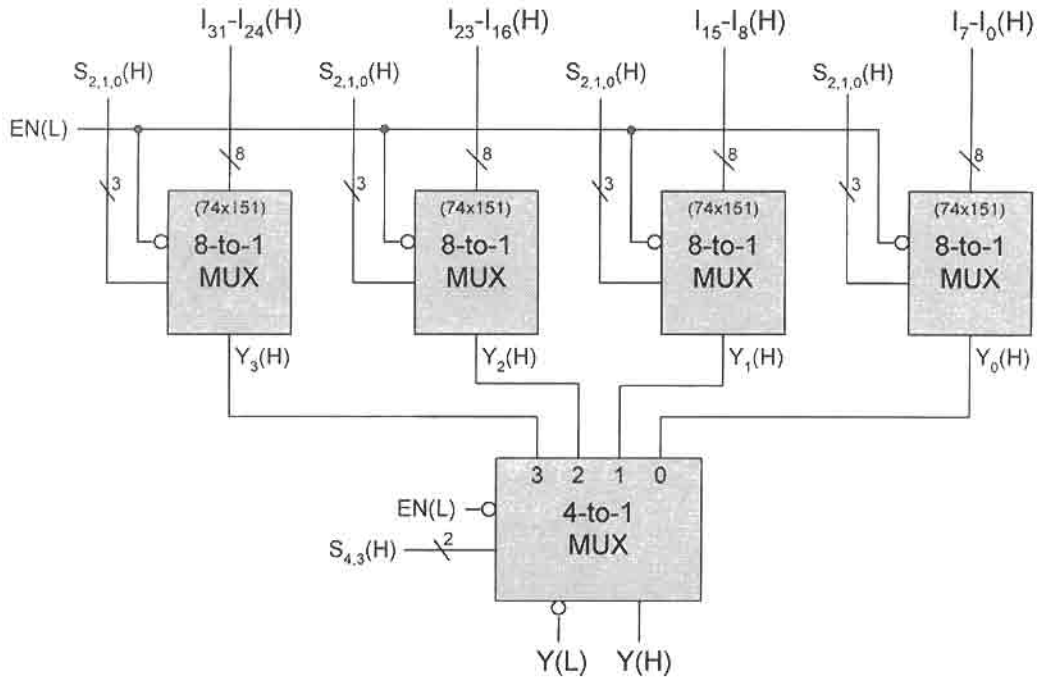
Design of a 4-to-1 MUX or data selector. (a) Block diagram symbol. (b) Compressed EV truth table. (c) MUX EV K-map. (d) Logic circuit for a 4-to-1 MUX with active low enable and mixed logic output. (e) Shorthand 4-to-1 MUX circuit symbol.

shown in Fig. 6.6. Reading directly from the EV truth table in Fig. 6.6a yields the output expressions

$$\begin{aligned}
 1Y &= \bar{S}_1\bar{S}_0I_0 \cdot 1G + \bar{S}_1S_0I_1 \cdot 1G + S_1\bar{S}_0I_2 \cdot 1G + S_1S_0I_3 \cdot 1G \\
 2Y &= \bar{S}_1\bar{S}_02I_0 \cdot 2G + \bar{S}_1S_02I_1 \cdot 2G + S_1\bar{S}_02I_2 \cdot 2G + S_1S_02I_3 \cdot 2G,
 \end{aligned}
 \tag{6.6}$$

which are implemented with NAND/INV logic in Fig. 6.6c.

The traditional logic symbol for the '153 MUX is given in Fig. 6.6b. Notice that there are two data select inputs that simultaneously control data selection to both outputs, and

**FIGURE 6.5**

Four 8-to-1 MUXs and a 4-to-1 MUX combine to produce a 32-to-1 MUX having five data select inputs  $S_4, S_3, S_2, S_1$ , and  $S_0$  and an active low enable.

that there are two independently operated enable inputs,  $1G$  and  $2G$ , that enable or disable either or both of the MUXs.

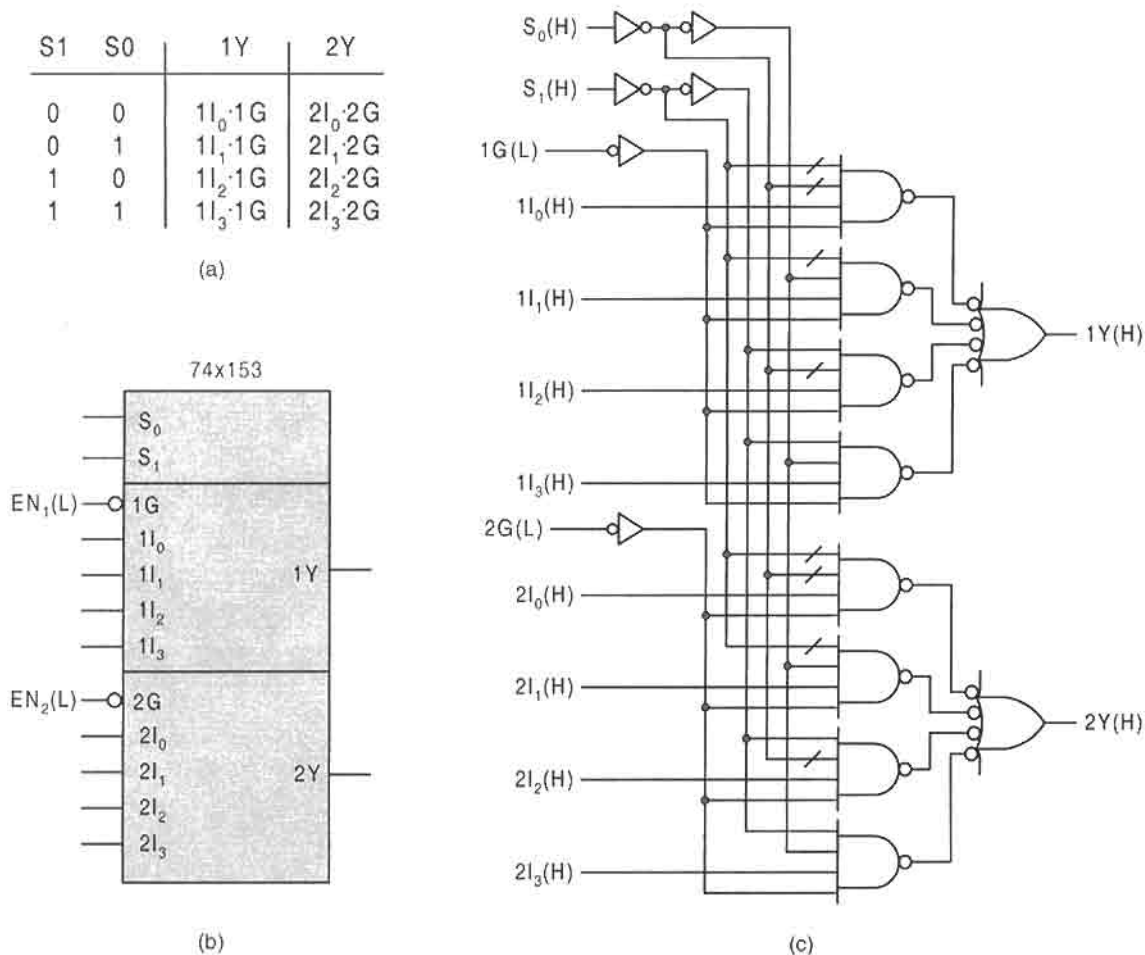
### 6.2.2 Combinational Logic Design with MUXs

A MUX is a *function generator* and can be used to implement a function in canonical form or in compressed (reduced or minimum) form. To understand this, consider the function

$$\begin{aligned} Y(A, B, C, D) &= \sum m(3, 4, 5, 6, 7, 9, 10, 12, 14, 15) \\ &= \prod M(0, 1, 2, 8, 11, 13). \end{aligned} \quad (6.7)$$

If this function is implemented with a 16-to-1 MUX, then all inputs representing minterms in Eq. (6.7) are connected to logic 1 (HV) while all maxterms are connected to logic 0 (LV). In this case the data select variables are the four function variables. But if it is desirable to implement Eq. (6.7) with a 4-to-1 MUX, two levels of K-map compression are needed, as in Fig. 6.7b.

Notice that the data select variables,  $S_1 = A$  and  $S_0 = B$ , form the axes of the MUX K-map in Fig. 6.7b and that the functions generated by the 4-to-1 MUX in Fig. 6.7 are

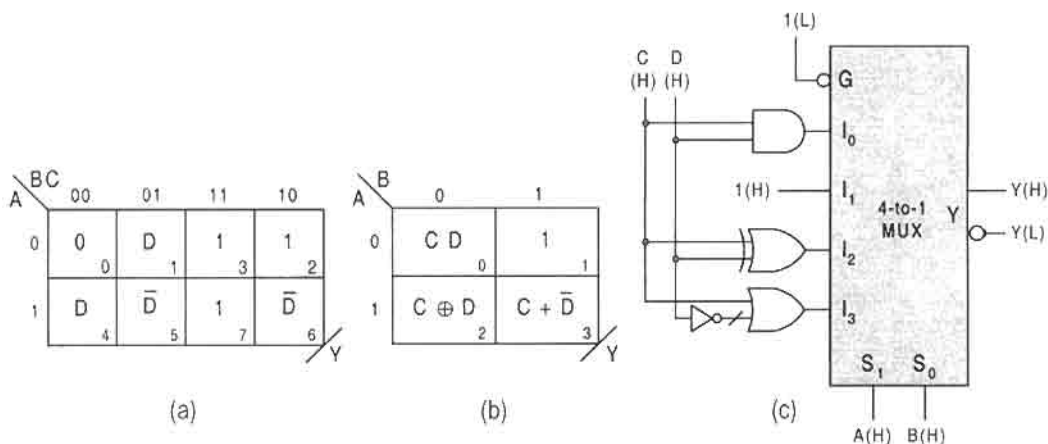
**FIGURE 6.6**

Design of the 74x153 4-input/2-bit MUX. (a) EV truth table. (b) Traditional logic symbol. (c) Logic diagram.

given by

$$\left\{ \begin{array}{l} Y = [\bar{A}\bar{B}CD + \bar{A}B + A\bar{B}(C \oplus D) + AB(C + \bar{D})] \\ \text{or} \\ Y = [(A + B + CD)(\bar{A} + B + C \oplus D)(\bar{A} + \bar{B} + C + \bar{D})] \end{array} \right\}, \quad (6.8)$$

both of which are three-level hybrid forms that are generated (H) or (L) from the  $Y(H)$  and  $Y(L)$  outputs of the MUX. The latter of the two may not seem obvious but is easily verified by extracting cover in maxterm code from the second-order K-map in Fig. 6.7b. Note also that if an 8-to-1 MUX is used to implement the function in Eq. (6.7), the K-map in Fig. 6.7a applies, where the data select variables are now  $A$ ,  $B$ , and  $C$ . In this case inputs  $I_1$  and  $I_4$

**FIGURE 6.7**

Implementation of the function in Eq. (6.7) by using a 4-to-1 MUX. (a), (b) First- and second-order K-map compressions for the function Y. (c) Logic circuit.

are connected to  $D(H)$ , inputs  $I_5$  and  $I_6$  are connected to  $D(L)$ , inputs  $I_2, I_3$ , and  $I_7$  connect to  $1(H)$ , and  $I_0$  goes to ground  $0(H)$ .

There still remains the problem of dealing with active low inputs to a MUX. The rules are simply stated:

#### For Active Low Data Select Inputs to MUXs

(1) Complement the MUX K-map axis of any active low data select input and renumber the K-map cells. The new cell numbers identify the MUX inputs to which they must be connected.

or

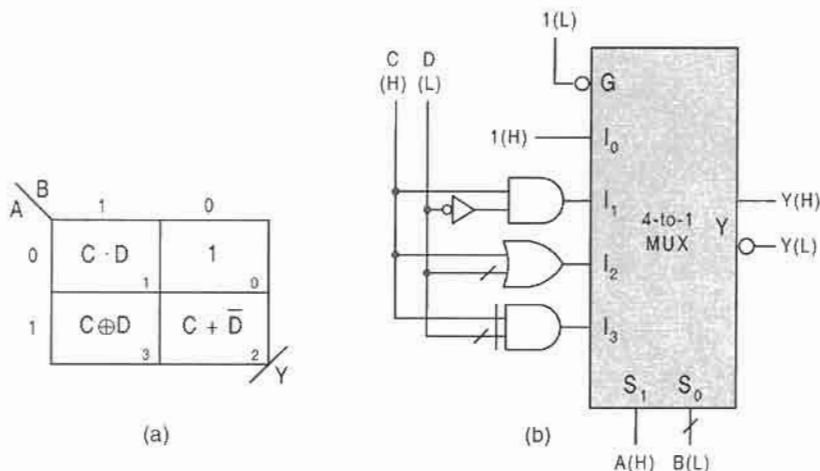
(2) Use an inverter on the input of an active low data select input and *do not* complement the MUX K-map axis.

#### For All Other Active Low Inputs

Active low nondata select inputs are dealt with as any combinational logic problem with mixed-logic inputs (see, e.g., Section 3.7). Therefore, *do not* complement any EV subfunction in a MUX K-map.

To illustrate the problem of mixed-logic inputs to a MUX, consider the function of Eq. (6.7) with inputs that arrive as  $A(H)$ ,  $B(L)$ ,  $C(H)$ , and  $D(L)$ . Implementation with a 4-to-1 MUX follows as in Fig. 6.8, where the  $B$  axis of the MUX K-map is complemented since no inverter is used on the  $B(L)$  data select input line. Notice that no additional inverters are required when compared to the implementation of Fig. 6.7, and that the resulting outputs are identical to those of Eqs. (6.8). The use of an EQV gate in place of an XOR gate is a consequence of the  $D(L)$  and the fact that only one inverter is used.



**FIGURE 6.8**

Implementation of the function in Eq. (6.7) with mixed-logic inputs by using a 4-to-1 MUX. (a) Second-order K-map compressions for the function  $Y$  showing the renumbered cells due to  $B(L)$ . (b) Logic circuit.

### 6.3 DECODERS/DEMULPLEXERS

A decoder is an  $n$ -input/ $2^n$ -output combinational logic device that has the function of activating one of the  $2^n$  outputs for every unique binary input pattern of  $n$  bits. The circuit symbol for an  $n$ -to- $2^n$  decoder is shown in Fig. 6.9a, where  $I_{n-1} - I_0$  are the data inputs,  $Y_{2^n-1} - Y_0$  are the outputs, and  $G$  is the enable. Each output is identified by the minterm code  $m_i$  of the binary input pattern it represents and can be represented, generally, as

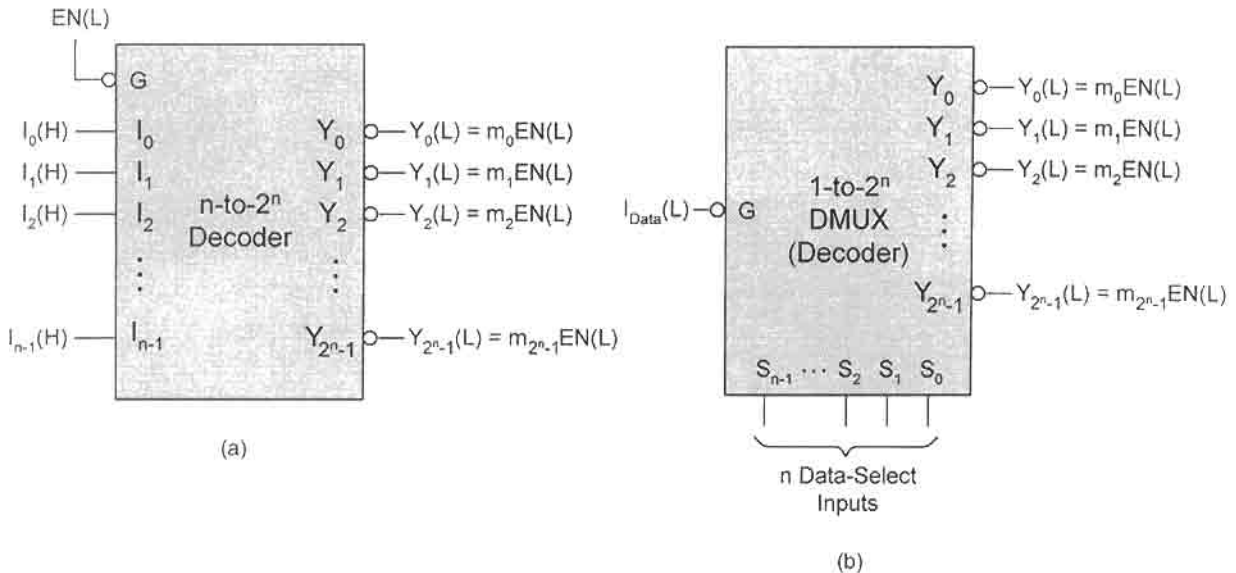
$$Y_i = m_i \cdot EN, \quad (6.9)$$

where  $m_0 = \bar{I}_{n-1} \cdots \bar{I}_2 \bar{I}_1 \bar{I}_0$ ,  $m_1 = \bar{I}_{n-1} \cdots \bar{I}_2 \bar{I}_1 I_0$ ,  $m_2 = \bar{I}_{n-1} \cdots \bar{I}_2 I_1 \bar{I}_0$ , and so on. For this reason a decoder can be called a *minterm code generator*. Commercial decoders are available in a variety of sizes and packaged configurations, but most all feature active low outputs and an active low enable.

Shown in Fig. 6.9b is the same decoder used as a demultiplexer (DMUX). Now, the active low enable  $EN(L)$  to the decoder becomes the single data input  $I_{Data}(L)$  to the DMUX, and the data inputs  $I_0, I_1, I_2, \dots, I_{n-1}$  for the decoder become the data select inputs  $S_0, S_1, S_2, \dots, S_{n-1}$  for the DMUX. The outputs for the decoder and DMUX are the same if it is understood that  $I_i$  is replaced by  $S_i$  in Eq. (6.9). The active low outputs and active low enable are of particular importance when a decoder is used as a DMUX, since the DMUX is often paired with a MUX for data routing as explained later.

#### 6.3.1 Decoder Design

Decoder design is illustrated by the design of a 3-to-8 decoder. Shown in Fig. 6.10 is the collapsed canonical I/O truth table for the enable input ( $EN$ ), the three data inputs ( $I_2, I_1,$

**FIGURE 6.9**

Generalization of a decoder/demultiplexer (DMUX). (a) An  $n$ -to- $2^n$  decoder with an active low enable. (b) The decoder of (a) used as a 1-to- $2^n$  DMUX with data input  $I_{Data}(L)$ .

and  $I_0$ ) and the eight outputs  $Y_7, \dots, Y_2, Y_1$ , and  $Y_0$ . The truth table is termed a *collapsed truth table* because of the *irrelevant input* symbol  $X$  used to represent either logic 0 or logic 1. Thus,  $X X X$  in the first row of the table represents eight minterms in variables  $I_i$ . Notice that only one minterm code line is activated for each corresponding three-bit binary pattern that appears on the input with active  $EN$ .

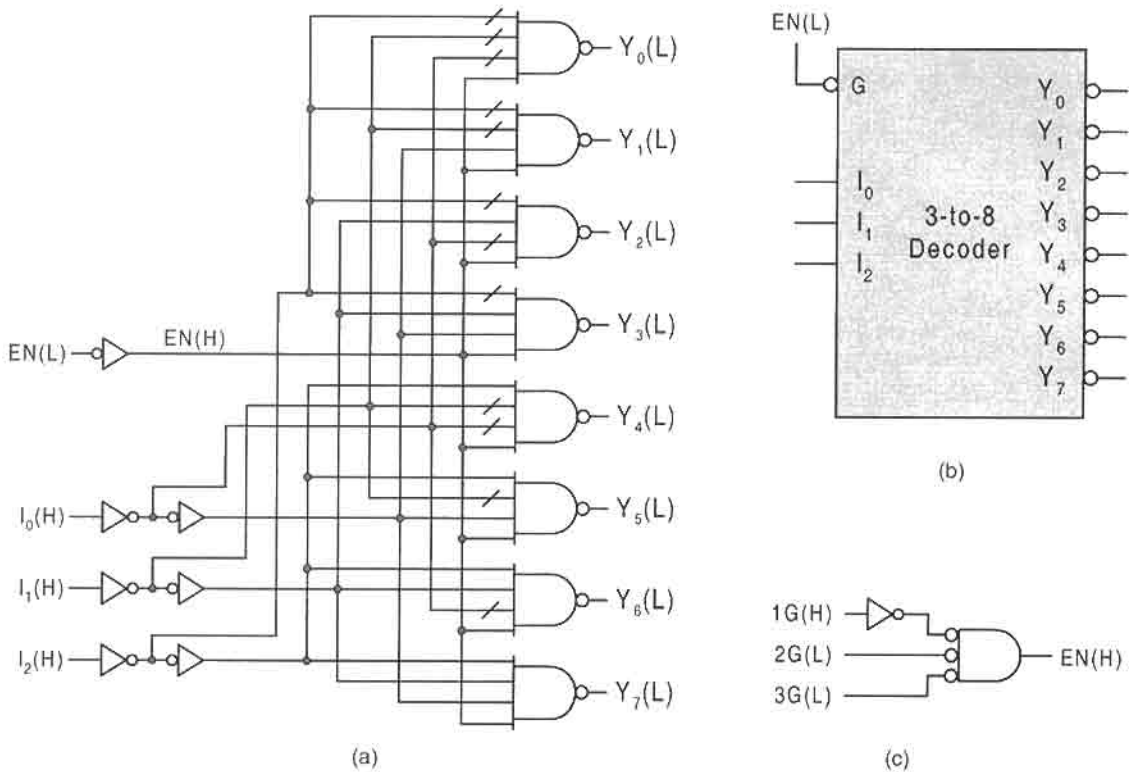
Each output ( $Y_i$ ) column in Fig. 6.10 represents a third-order K-map containing a single minterm ANDed with  $EN$ . However, it is not necessary to construct eight EV K-maps to obtain the eight output expression for  $Y_i$ , since this information can be read directly from

| EN | $I_2$ | $I_1$ | $I_0$ | $Y_7$ | $Y_6$ | $Y_5$ | $Y_4$ | $Y_3$ | $Y_2$ | $Y_1$ | $Y_0$ |                                                                                                                                                                                                                                                                                                                                      |
|----|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0  | X     | X     | X     | 0     | 0     | 0     | 0     | 0     | 0     | 0     | 0     | $Y_0 = \bar{I}_2 \bar{I}_1 \bar{I}_0 \cdot EN$<br>$Y_1 = \bar{I}_2 \bar{I}_1 I_0 \cdot EN$<br>$Y_2 = \bar{I}_2 I_1 \bar{I}_0 \cdot EN$<br>$Y_3 = \bar{I}_2 I_1 I_0 \cdot EN$<br>$Y_4 = I_2 \bar{I}_1 \bar{I}_0 \cdot EN$<br>$Y_5 = I_2 \bar{I}_1 I_0 \cdot EN$<br>$Y_6 = I_2 I_1 \bar{I}_0 \cdot EN$<br>$Y_7 = I_2 I_1 I_0 \cdot EN$ |
| 1  | 0     | 0     | 0     | 0     | 0     | 0     | 0     | 0     | 0     | 0     | 1     |                                                                                                                                                                                                                                                                                                                                      |
| 1  | 0     | 0     | 1     | 0     | 0     | 0     | 0     | 0     | 0     | 1     | 0     |                                                                                                                                                                                                                                                                                                                                      |
| 1  | 0     | 1     | 0     | 0     | 0     | 0     | 0     | 0     | 1     | 0     | 0     |                                                                                                                                                                                                                                                                                                                                      |
| 1  | 0     | 1     | 1     | 0     | 0     | 0     | 0     | 1     | 0     | 0     | 0     |                                                                                                                                                                                                                                                                                                                                      |
| 1  | 1     | 0     | 0     | 0     | 0     | 0     | 1     | 0     | 0     | 0     | 0     |                                                                                                                                                                                                                                                                                                                                      |
| 1  | 1     | 0     | 1     | 0     | 0     | 1     | 0     | 0     | 0     | 0     | 0     |                                                                                                                                                                                                                                                                                                                                      |
| 1  | 1     | 1     | 0     | 0     | 1     | 0     | 0     | 0     | 0     | 0     | 0     |                                                                                                                                                                                                                                                                                                                                      |
| 1  | 1     | 1     | 1     | 1     | 0     | 0     | 0     | 0     | 0     | 0     | 0     |                                                                                                                                                                                                                                                                                                                                      |

$X$  indicates an irrelevant input and represents either logic 0 or logic 1.

**FIGURE 6.10**

Collapsed truth table for a 3-to-8 decoder/demultiplexer with enable showing output expressions that derived directly from the truth table.

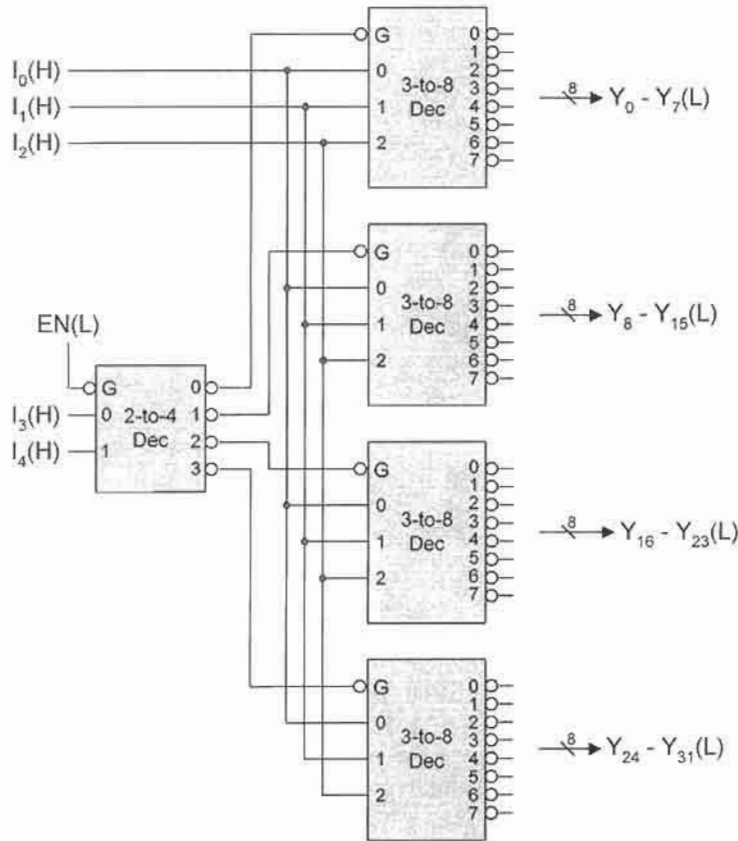
**FIGURE 6.11**

Implementation of the 3-to-8 decoder/demultiplexer in Fig. 6.10. (a) Logic circuit with active low outputs and active low enable. (b) Logic symbol. (c) An alternative three-enable configuration used by the commercial 74x138 decoder.

the truth table and is provided in the third column of Fig. 6.10. When the requirements of active low outputs and active low enable are introduced, the  $Y_i$  expressions for the 3-to-8 decoder/demultiplexer in Fig. 6.10 are implemented with NAND/INV logic as shown in Fig. 6.11a. Its logic symbol is given in Fig. 6.11(b).

A single enable input is used to enable or disable the decoder of Fig. 6.11. But other enable configurations are common. For example, a commercially available 3-to-8 decoder, the 74x138, has the same decoder logic as in Fig. 6.11a, except the commercial unit features three enable inputs as indicated in Fig. 6.11c. Multiple enable inputs permit greater versatility when controlling a given decoder from various sources.

Decoders can be stacked (cascaded) in hierarchical configurations to produce much larger decoders. This requires that the  $2^n$  outputs of one decoder drive the EN controls of  $2^n$  other decoders, assuming that all outputs of the leading decoders are used. As an example, four 3-to-8 decoders are enable/selected by a 2-to-4 decoder in Fig. 6.12 to produce a 5-to-32 decoder. Similarly, two 4-to-16 decoders can be stacked to produce a 5-to-32 decoder when enable/selected by a single inverter. Or cascading four 4-to-16 decoders produces a 6-to-64 decoder when enable/selected by a 2-to-4 decoder. Note that stacking any two decoders requires only an inverter acting as a 1-to-2 enable/select decoder.

**FIGURE 6.12**

Stacking of four 3-to-8 decoders to produce a 5-to-32 decoder.

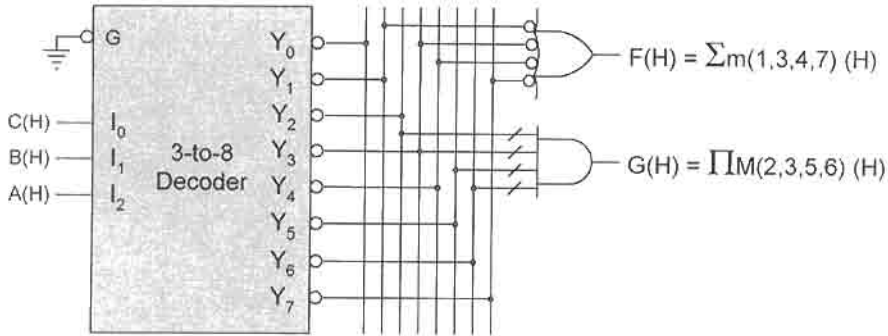
Decoders can be packaged in pairs such as the dual 2-to-4 74x139 decoder. Such a dual set will have two independent enable inputs, one for each of the two 2-to-4 decoders. The 2-to-4 decoder in Fig. 6.12 is actually one half of the 74x139, indicated as  $\frac{1}{2}$  74x139.

### 6.3.2 Combinational Logic Design with Decoders

Decoders can be used effectively to implement any function represented in canonical form. All that is needed is the external logic required to OR minterms for SOP representation or to AND maxterms for POS representation. As a simple example, consider the two functions in canonical form:

$$\begin{aligned} F(A, B, C) &= \sum m(1, 3, 4, 7) \quad \text{SOP} \\ G(A, B, C) &= \prod M(2, 3, 5, 6) \quad \text{POS} \end{aligned} \quad (6.10)$$

Assuming that the inputs arrive as  $A(H)$ ,  $B(H)$ ,  $C(H)$ , and that the outputs are delivered

**FIGURE 6.13**

Decoder implementations of Eqs. (6.10) assuming inputs and outputs are all active high.

active high, these two functions are implemented as given in Fig. 6.13. To understand why function  $G$  is implemented with an AND gate, consider what is generated by the ANDing operation:

$$G = \bar{m}_2 \cdot \bar{m}_3 \cdot \bar{m}_5 \cdot \bar{m}_6 = M_2 \cdot M_3 \cdot M_5 \cdot M_6 = \prod M(2, 3, 5, 6).$$

If it is desirable to issue  $G$  active low, a NAND gate would be used in place of the AND gate. Or if  $F$  is to be issued active low, an AND would be needed in place of the NAND. Actually, to fully understand the versatile nature of function implementation with decoders, the reader should experiment by replacing the NAND and AND gates in Fig. 6.13 with a variety of gates, including treed XOR and EQV gates.

The problem of mixed-logic inputs can be dealt with in a manner similar to those issued to MUXs. The rules are similar to those for MUXs and are stated as follows:

**For Mixed-Logic Data Inputs to Decoders**

(1) Complement the bit of any active low input to a decoder and renumber the minterms accordingly.

or

(2) Use an inverter on the input line of an active low input and do not complement the bit.

Consider, as an example, the two functions of Eqs. (6.10) with inputs that arrive as  $A(H)$ ,  $B(L)$ , and  $C(L)$ . Functionally, the mixed-logic forms of Eqs. (6.10) become

$$F_{SOP}[A(H), B(L), C(L)] = F_{SOP}[A, \bar{B}, \bar{C}](H)$$

and

$$G_{POS}[A(H), B(L), C(L)] = G_{POS}[A, \bar{B}, \bar{C}](H).$$

(6.11)

Then by Eqs. (6.11) and if inverters are not to be used, the  $B$  and  $C$  bits must be complemented:

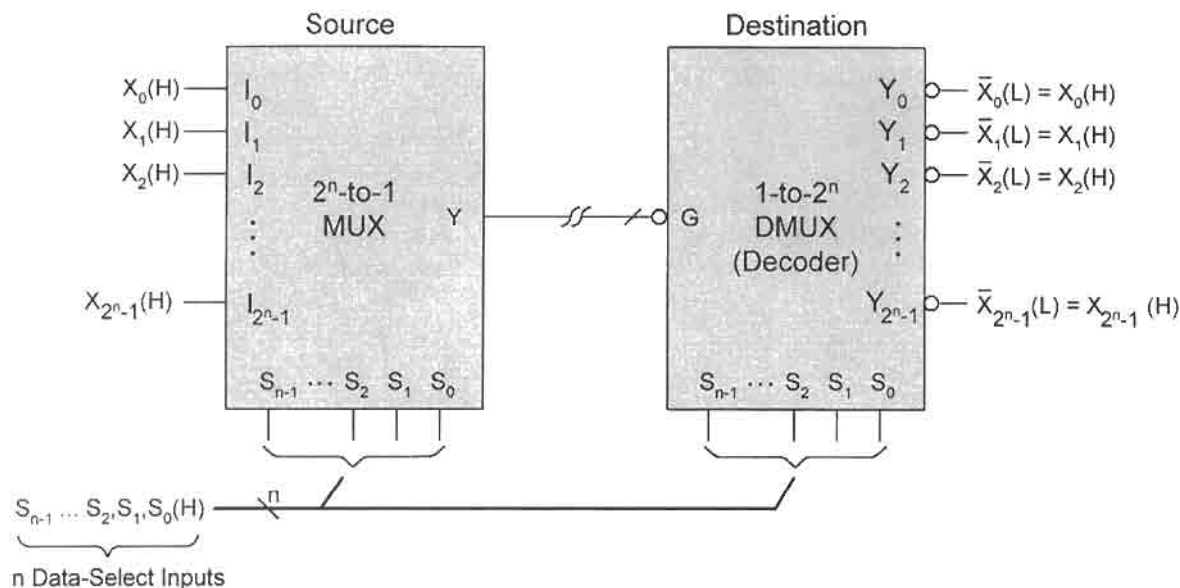
$$\begin{array}{ll} m_0 = 000 \rightarrow 011 = m_3 & m_4 = 100 \rightarrow 111 = m_7 \\ m_1 = 001 \rightarrow 010 = m_2 & m_5 = 101 \rightarrow 110 = m_6 \\ m_2 = 010 \rightarrow 001 = m_1 & m_6 = 110 \rightarrow 101 = m_5 \\ m_3 = 011 \rightarrow 000 = m_0 & m_7 = 111 \rightarrow 100 = m_4. \end{array}$$

Thus, to accommodate the mixed-logic inputs, the two functions in Eqs. (6.10) must be connected to the decoder according to the renumbered functions

$$\begin{aligned} F[A(H), B(L), C(L)] &= \sum m(0, 2, 4, 7) \quad \text{and} \\ G[A(H), B(L), C(L)] &= \prod M(0, 1, 5, 6). \end{aligned}$$

Of course, if inverters are used on the  $B(L)$  and  $C(L)$  inputs, no complementation is necessary and the functions are implemented according to Eqs. (6.10).

Decoders, used as demultiplexers (DMUXs), are simply reoriented so that the active low enable is the only data input. Now the  $I_i$  inputs become the data select inputs  $S_i$  as in Fig. 6.9b. Used in conjunction with MUXs, the MUX/DMUX system offers a means of time-shared bussing of multiple-line data  $X_i$  on a single line as illustrated in Fig. 6.14. Bussing data over large distances by using this system results in a significant savings on hardware, but is a relatively slow process.



**FIGURE 6.14**

Generalization of a MUX/DMUX system for bussing data on  $2^n - 1$  lines over a single time-shared line from the MUX source to the DMUX destination.

## 6.4 ENCODERS

By definition an *encoder* performs an operation that is the opposite to that of a decoder. That is, an encoder must generate a different output bit pattern (code) for each input line that becomes active. For a binary encoder, this requirement can be enforced only if one output binary word of  $n$  bits is associated with only one of  $2^n$  “decimal” input lines (0, 1, 2, 3, ...,  $2^n - 1$ ). Obviously, if more than one input line is active, the output becomes ambiguous in such an encoder. The ambiguity problem is overcome by prioritizing the input. When this is done the result is called a *priority encoder* (PE), which assigns a priority to each input according to a PE schedule for that encoder. Most encoders are PEs.

A priority encoder is generally an  $n$ -input/ $m$ -output ( $n \leq 2^m$ ) device as indicated by the circuit symbol in Fig. 6.15. In addition to the  $n$  address inputs and  $m$  outputs, a commercial PE will usually have three other input and output lines that are used to cascade (stack) PEs: an *enable-in* input ( $EI$ ), an *enable-out* output ( $EO$ ), and a *group signal* output ( $GS$ ). The purpose of the  $GS$  output is to indicate any legitimate encoding condition, meaning that  $EI$  is active concurrently with a single active address input. All inputs and outputs of a PE are active low as shown.

The design of a simple 3-input/2-output PE with cascading capability is illustrated in Fig. 6.16. Shown in Fig. 6.16a is the priority schedule and the collapsed I/O truth table for this encoder. The EV K-maps in Fig. 6.16b are plotted from the truth table, and the minimum cover yields the following output expressions:

$$\begin{aligned} Y_1 &= I_2 EI + I_1 EI & Y_0 &= \bar{I}_1 I_0 EI + I_2 EI \\ EO &= \bar{I}_2 \bar{I}_1 \bar{I}_0 EI & GS &= (I_2 + I_1 + I_0)EI = \overline{EO} \cdot EI. \end{aligned} \quad (6.12)$$

These expressions are implemented with minimum logic in Fig. 6.16c for active low inputs and outputs as required in Fig. 6.15. Notice that this circuit represents a simple multioutput optimization, which is deduced by inspection of expressions for  $EO$  and  $GS$ .

The outputs  $EO$  and  $GS$  require special attention since their logic values have been specifically chosen to make cascading of PEs possible. When the address inputs for the  $n$ th stage are *all* inactive (logic 0), it is the function of  $EO_n$  to activate the  $(n - 1)$ th stage. This assumes that prioritization is assigned from highest active input (decimal-wise) to lowest. Therefore,  $EO$  can be active only for inactive address inputs and active  $EI$ . It is the function

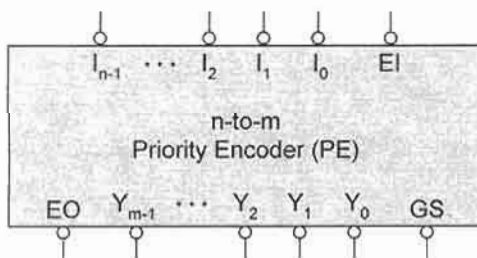
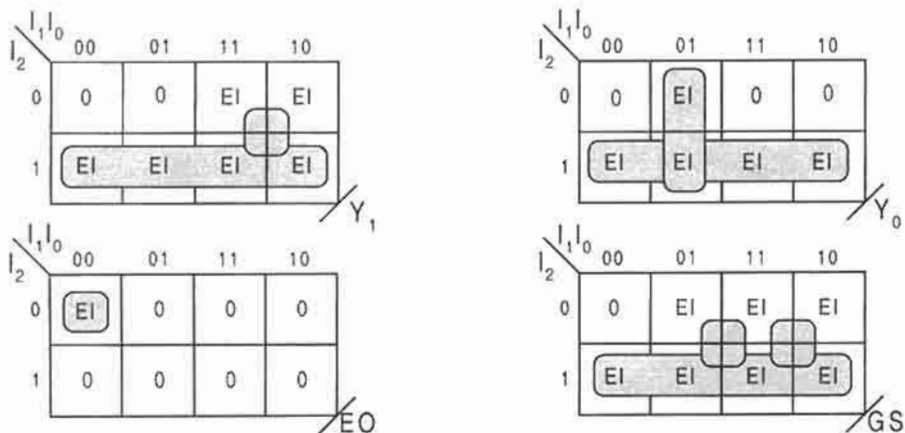


FIGURE 6.15

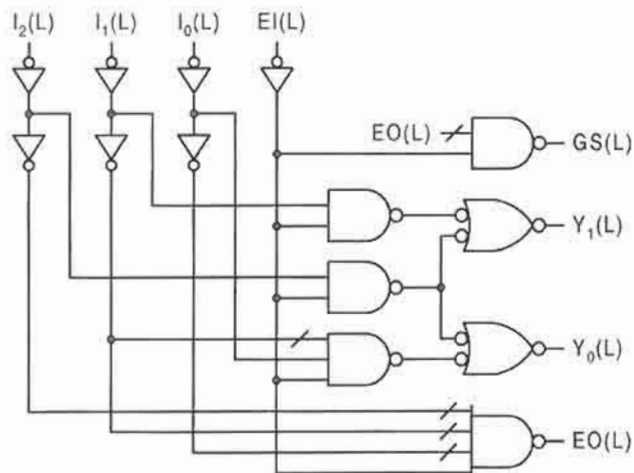
Logic symbol for an  $n$ -to- $m$  priority encoder with cascading capability.

| Priority Schedule                | EI | $I_2$ | $I_1$ | $I_0$ | GS | $Y_1$ | $Y_0$ | EO |
|----------------------------------|----|-------|-------|-------|----|-------|-------|----|
| Null state                       | 0  | X     | X     | X     | 0  | 0     | 0     | 0  |
| $I_2$ (highest) -- encoded as 11 | 1  | 1     | X     | X     | 1  | 1     | 1     | 0  |
| $I_1$ (middle) -- encoded as 10  | 1  | 0     | 1     | X     | 1  | 1     | 0     | 0  |
| $I_0$ (lowest) -- encoded as 01  | 1  | 0     | 0     | 1     | 1  | 0     | 1     | 0  |
| Inactive state -- encoded 00     | 1  | 0     | 0     | 0     | 0  | 0     | 0     | 1  |

(a)



(b)

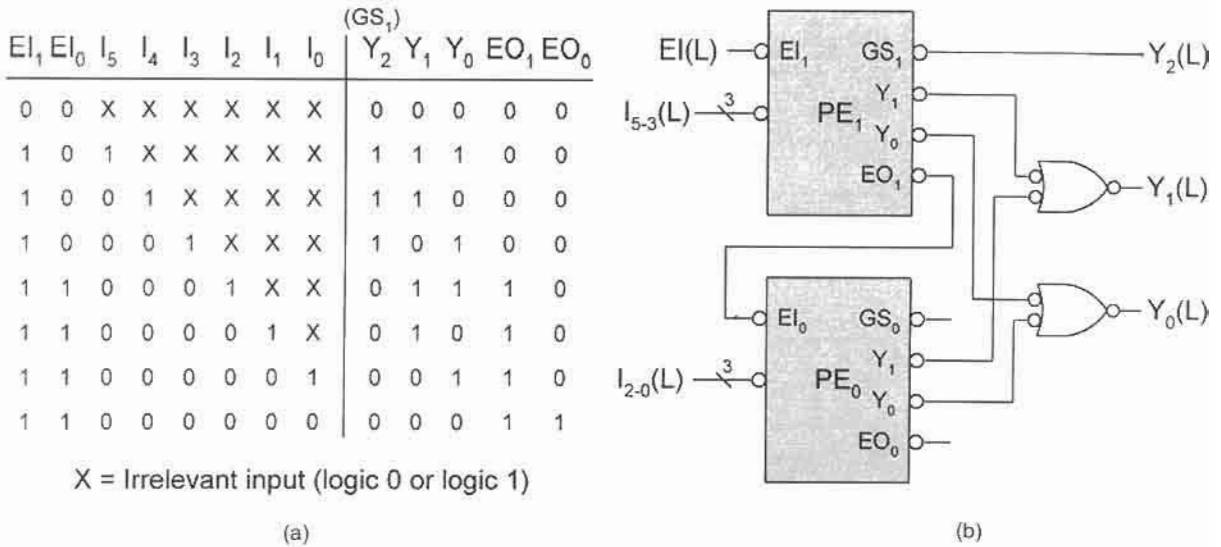


(c)

**FIGURE 6.16**

Design of a three-input priority encoder with cascading capability. (a) Priority schedule and collapsed truth table. (b) EV K-maps. (c) Minimized logic circuit.



**FIGURE 6.17**

Two 3-to-2 PEs in cascade to produce a 6-to-3 PE. (a) Collapsed truth table. (b) Logic circuit.

of the  $GS$  output to go active for any valid encoding condition for that stage, meaning  $EI$  active concurrently with a single active address input, and to provide an additional output bit (MSB) for each stage added in cascade fashion.

Two 3-to-2 PEs of the type in Fig. 6.16 are shown stacked in Fig. 6.17 to form a 6-to-3 PE. The truth table for the two PE combination is given in Fig. 6.17a and includes the cascading bits  $EI$ ,  $EO$ , and  $GS$  for  $PE_1$  and  $PE_0$ . From Eqs. (6.12) and Fig. 6.17, the cascading bits for the system are expressed as

$$\begin{aligned}
 GS_1 &= \overline{EO}_1 EI_1 \\
 EO_1 &= \bar{I}_5 \bar{I}_4 \bar{I}_3 EI_1 = EI_0 \\
 EO_0 &= \bar{I}_2 \bar{I}_1 \bar{I}_0 EI_0.
 \end{aligned} \tag{6.13}$$

Notice that  $GS_1 = Y_2$  indicates a valid encoding condition only for active inputs  $I_5$ ,  $I_4$ , and  $I_3$ , while  $GS_0$  (not shown) indicates a valid encoding condition only for active inputs  $I_2$ ,  $I_1$ , and  $I_0$ . Output state 100 cannot occur according to Eqs. (6.12) and (6.13) and Fig. 6.17a.

Priority encoders of the type represented in Fig. 6.15 are commercially available as IC chips. Typically, they are produced in the 8-to-3 line size, such as the 74x148, which can be stacked to produce 16-to-4 line and 32-to-5 line PEs. Their applications include code conversion, code generation, and  $n$ -bit encoding in digital systems having a hierarchy of subsystems that must be prioritized.

Those PEs that do not have  $EO$  and  $GS$  outputs, and hence cannot be stacked, are also available commercially as ICs. Their design closely follows the design of PEs of the type in Fig. 6.16. They are commonly produced in 9-to-4 line size for use as BCD priority encoding, keyboard encoding, and range selection.

## 6.5 CODE CONVERTERS

Various codes are discussed at length in Section 2.10. On occasion it is necessary to convert one code to another. Devices that are designed to execute a code conversion are called *code converters*. Considering the many codes that are currently in use, it follows that there are a very large number of converters possible. Not taking into account any particular area of specialty, a few of the more common code conversions are as follows:

Binary-to-Gray conversion and vice versa  
BCD-to-XS3 conversion and vice versa  
Binary-to-BCD conversion and vice versa

### 6.5.1 Procedure for Code Converter Design

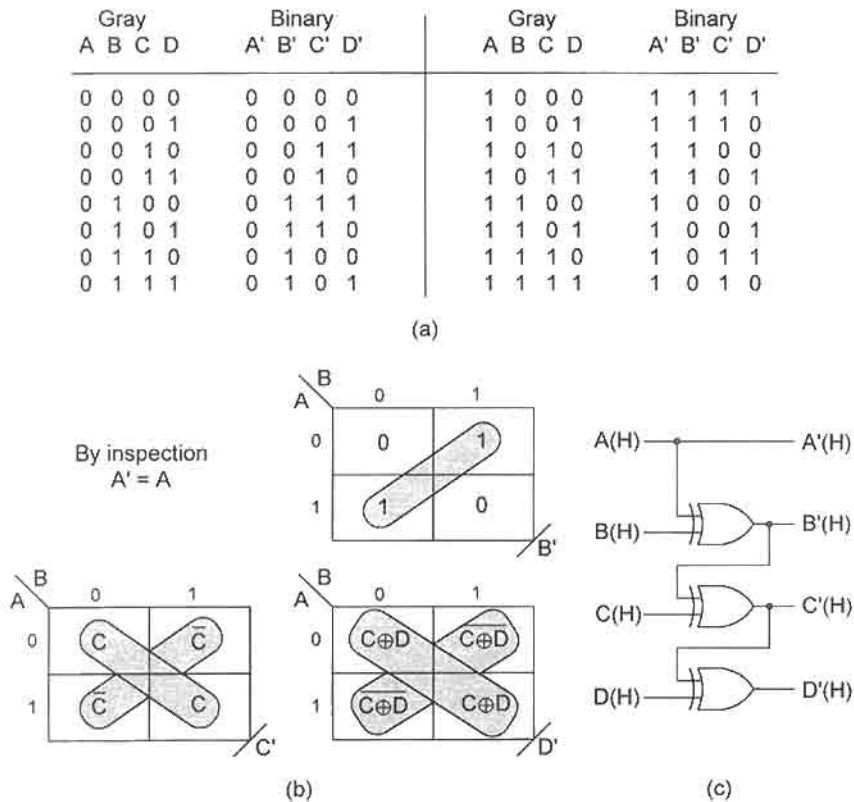
The following is a simple three-step procedure that will be followed in this text, often without reference to this subsection:

1. Generally, follow the design procedure in Subsection 6.1.5.
2. If conversion involves any of the decimal code input (e.g., BCD), only 10 states can be used. The six unused input states are called *false data* inputs. For these six states the outputs must be represented either by don't cares ( $\phi$ 's) or by some unused output state, for example all 1's. That is, if the requirement is for *false data rejection* (FDR), then the output states must correspond to at least one unused output state; if not,  $\phi$ 's are entered for the output states. Thus, FDR means that the outputs must never correspond to a used output state when any one of the six unused states arrive at the input terminals. If false data is not rejected, then the outputs corresponding to the six unused states can take on any logic values, including those of used output states.
3. If the input code is any other than binary or BCD and if EV K-maps are to be used in minimizing the logic, it is recommended that the input code be arranged in the order of ascending binary, taking care to match each output state with its corresponding input state.

### 6.5.2 Examples of Code Converter Design

To illustrate the code converter design process, four examples are presented. These examples are quite adequate since the conversion procedure varies only slightly from one conversion to another. The four examples are Gray-to-binary conversion, BCD-to-XS3 conversion, BCD-to-binary conversion, and BCD-to-seven-segment-display conversion. Of these, the last two are by far the most complex and perhaps the most important considering that binary, BCD, and the seven-segment display are commonly used in digital design and computer technology.

**Gray-to-Binary Conversion** The Gray-to-binary conversion table for 4-bit codes is given in Fig. 6.18a. Here, for convenience of plotting EV K-maps, the input Gray code and the corresponding output binary code have been rearranged such that the Gray code is given in ascending minterm code (compare Tables 2.1 and 2.12). The second-order EV

**FIGURE 6.18**

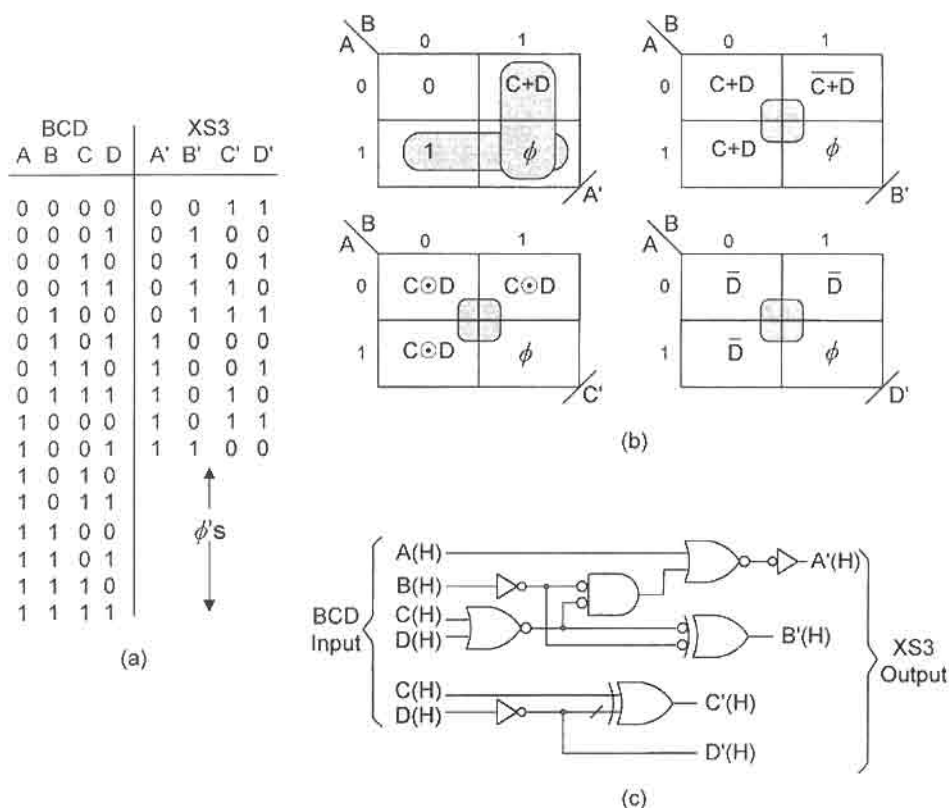
Design of a 4-bit Gray-to-binary converter. (a) I/O truth table. (b) Output EV K-maps plotted from (a) showing minimum cover. (c) Resulting logic circuit according to Eqs. (6.14).

K-maps, shown in Fig. 6.18b, are plotted directly from the truth table and yield the minimum cover.

$$\begin{aligned}
 A' &= A \\
 B' &= A \oplus B \\
 C' &= A \oplus B \oplus C \\
 D' &= A \oplus B \oplus C \oplus D,
 \end{aligned}
 \tag{6.14}$$

from which the logic circuit of Fig. 6.18c results. Noticing the trend in Eqs. (6.14), it is clear that an XOR gate is added in series fashion with each additional bit of the Gray-to-binary conversion. With this trend in mind any size Gray-to-binary converter can be implemented without the need to repeat the steps indicated in Fig. 6.18.

**BCD-to-XS3 Conversion** As a second example, consider the conversion between two decimal codes, BCD and XS3. Shown in Fig. 6.19a is the truth table for the BCD-to-XS3 conversion where, for this design, false data is *not* rejected. Thus,  $\phi$ 's become the output

**FIGURE 6.19**

Design of the BCD-to-XS3 converter without FDR. (a) Truth table. (b) EV K-maps showing minimum cover. (c) Logic circuit according to Eqs. (6.15).

XS3 code patterns corresponding to the six unused input BCD states. The resulting EV K-maps for the output functions are given in Fig. 6.19b, from which the gate-minimum cover is extracted as

$$\left\{ \begin{array}{l} A' = B(C + D) + A \\ B' = B \oplus (C + D) \\ C' = C \odot D \\ D' = \bar{D} \end{array} \right\}, \quad (6.15)$$

representing a gate/input tally of 5/10, excluding inverters. The subfunctions for cell 2 in Fig. 6.19b result from an appropriate use of the  $\phi$ 's. Shown in Fig. 6.19c is the three-level logic circuit for this converter implemented with NOR/XOR/INV logic assuming that the inputs and outputs are all active high. The subject of mixed logic inputs to an XOR gate is discussed at length in Subsection 3.9.4.

Had FDR been a design objective for the BCD-to-XS3 converter, the  $\phi$ 's in Fig. 6.19a would have to be replaced by an unused (FDR) output state. If the FDR state is taken to be 1111, the K-maps of Fig. 6.19b are altered accordingly, and the resulting gate-minimum

output expressions become

$$\begin{aligned}
 A' &= B(C + D) + A \\
 B' &= B \oplus (C + D) + AB \\
 C' &= C \odot D + AC + AB \\
 &= C \oplus \bar{D} + AC + AB \\
 D' &= \bar{D} + AC + AB,
 \end{aligned} \tag{6.16}$$

now representing a gate/input tally of 10/22, excluding an inverter but including three shared PIs. Clearly, the FDR feature comes at a significant price in terms of hardware. FDR states other than the 1111 can be used, as for example the 0000 state, but all increase the cost in hardware even further.

**Conversion between BCD and Binary** The BCD and binary codes are two of the most widely used codes in digital design, so it is fitting that conversion between them be considered. The simplest approach is to establish workable algorithms to produce an expandable modular design. Even so, such tasks rank among the more difficult conversion problems. Consider, for example, that a two-digit BCD word converts to an 8-bit binary number, whereas an 8-bit binary word converts to a three-digit BCD representation.

Two algorithms will now be considered that make conversion between these two codes tractable and that lead to modular designs. In these algorithms shifting right by one bit is equivalent to dividing by 2 ( $sr1 = \div 2$ ) and shifting left by one bit is equivalent to multiplying by 2 ( $sl1 = \times 2$ ). Also, LSD and MSD refer to the least significant digit and most significant digit, respectively, for the BCD number. A detailed treatment of the binary and BCD number systems is provided in Sections 2.3 and 2.4, and the pencil-and-paper conversion method between the binary and BCD number systems is discussed in Subsection 2.4.1.

The first of the two algorithms, the BCD-to-binary conversion, requires that the BCD number first be placed in imaginary storage cells. For example, a two-decade BCD number will occupy eight imaginary storage cells. After this has been done, then the algorithm proceeds as follows:

**Algorithm 6.1 BCD-to-Binary Conversion by the  $(\div 2)/(-3)$  Process**

- (1) Shift the BCD number to the right by one bit into the new LSD position, but keeping account of the bits shifted out of the LSD.
- (2) Subtract 0011 from the LSD (or add 1101 to the LSD if in 2's complement) iff the new LSD number is greater than 7 (0111). After subtracting 3, shift right immediately even if the new LSD is greater than 7.
- (3) Repeat steps (1) and (2) until the final LSD number can no longer be greater than decimal 7. The answer is now in binary.

Algorithm 6.1 is sometimes referred to as the shift-right/subtract 3 [or  $(\div 2)/(-3)$ ] algorithm. The algorithm for binary-to-BCD conversion can be thought of as the mathematical dual of Algorithm 6.1. In this case the process could be called the shift-left/add 3 [or  $(\times 2)/(+3)$ ] algorithm. Begin by placing the binary number outside and to the right of the LSD positions, then proceed as follows:

**Algorithm 6.2 Binary-to-BCD Conversion by the  $(\times 2)/(+3)$  Process**

- (1) Shift the binary number to the left by one bit into the new LSD position.
- (2) Add 0011 to the new LSD iff the new LSD number is greater than 4 (0100). After adding 3, shift left immediately.
- (3) Repeat steps (1) and (2). When all binary bits have been shifted into digit positions, the process ceases and the answer is in BCD.

To design converters by either Algorithm 6.1 or 6.2, it is not necessary to resort to arithmetic means as implied by the algorithms. Rather, a modular approach is easily established by constructing a relatively simple truth table followed by appropriate minimization methods. The process is now illustrated by designing an 8-bit BCD-to-binary converter.

**BCD-to-Binary Conversion** A truth table matching BCD with the corresponding binary from 0 to 19 is given in Fig. 6.20a. Inherent in this truth table is the shift-right/subtract-3 Algorithm 6.1. The decimal range 0 to 19 is chosen to illustrate the process but is easily extended to 39, 79, or 99, etc. This truth table will be used to design a BCD-to-binary module that can be cascaded to produce any size converter. It is possible to use a decimal range of 0 to 99 for this purpose, but the size of the module is considered too large to be of value for this example.

Shown in Fig. 6.20b are the K-maps and minimum cover for four of the five output functions of the 2-digit BCD-to-binary module. The two-level minimum expressions for the BCD-to-binary module, as read from the K-maps and truth table, are

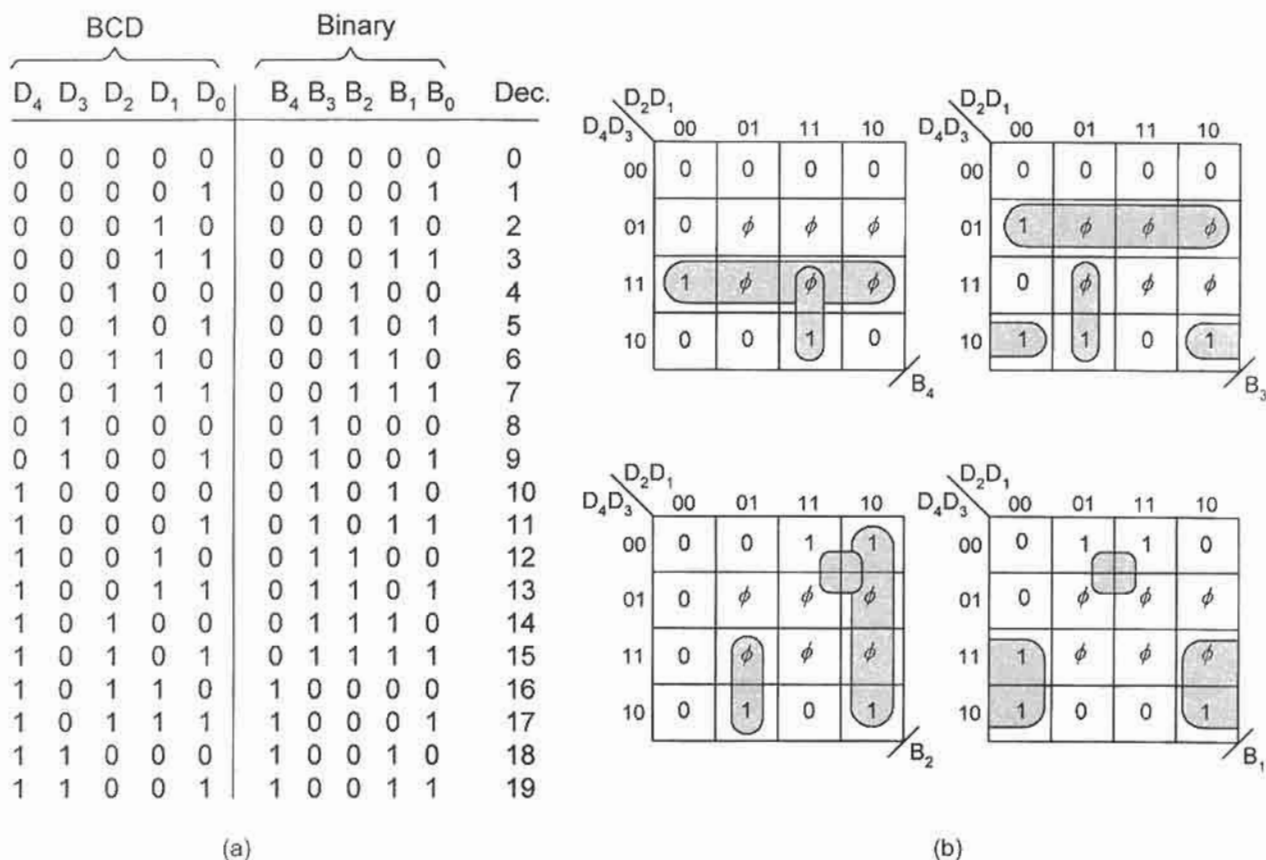
$$\begin{aligned}
 B_4 &= D_4 D_2 D_1 + D_4 D_3 \\
 B_3 &= D_4 \bar{D}_3 \bar{D}_1 + D_4 \bar{D}_2 D_1 + \bar{D}_4 D_3 \\
 B_2 &= D_4 \bar{D}_2 D_1 + \bar{D}_4 D_2 + D_2 \bar{D}_1 \\
 B_1 &= D_4 \bar{D}_1 + \bar{D}_4 D_1 = D_4 \oplus D_1 \\
 B_0 &= D_0 \quad \text{by inspection,}
 \end{aligned} \tag{6.17}$$

which represent a gate/input tally of 11/27 excluding inverters but including one shared PI. The logic circuit for this module is given in Fig. 6.21a and is cascaded in Fig. 6.21b to produce the 8-bit BCD-to-binary converter. Notice that the four modules are cascaded in such a way as to satisfy the right-shift requirement of Algorithm 6.1. If expansion beyond 8 bits is needed, each additional bit requires that an additional module be added in cascade fashion.

**BCD-to-Seven-Segment Display** No discussion of code conversion is complete without including the BCD-to-seven-segment decoder (converter). Light-emitting diodes (LEDs) and liquid crystal displays (LCDs) are used extensively to produce the familiar Arabic numerals. The use of the BCD-to-seven-segment decoder is an important means of accomplishing this.

Shown in Fig. 6.22a is the seven-segment display format and the ten decimal digits that are produced by the decoder/display. The truth table for the converter is given in Fig. 6.22b and features a blanking input  $BI$ , but lacks FDR since  $\phi$ 's are assigned to the six unused input states.

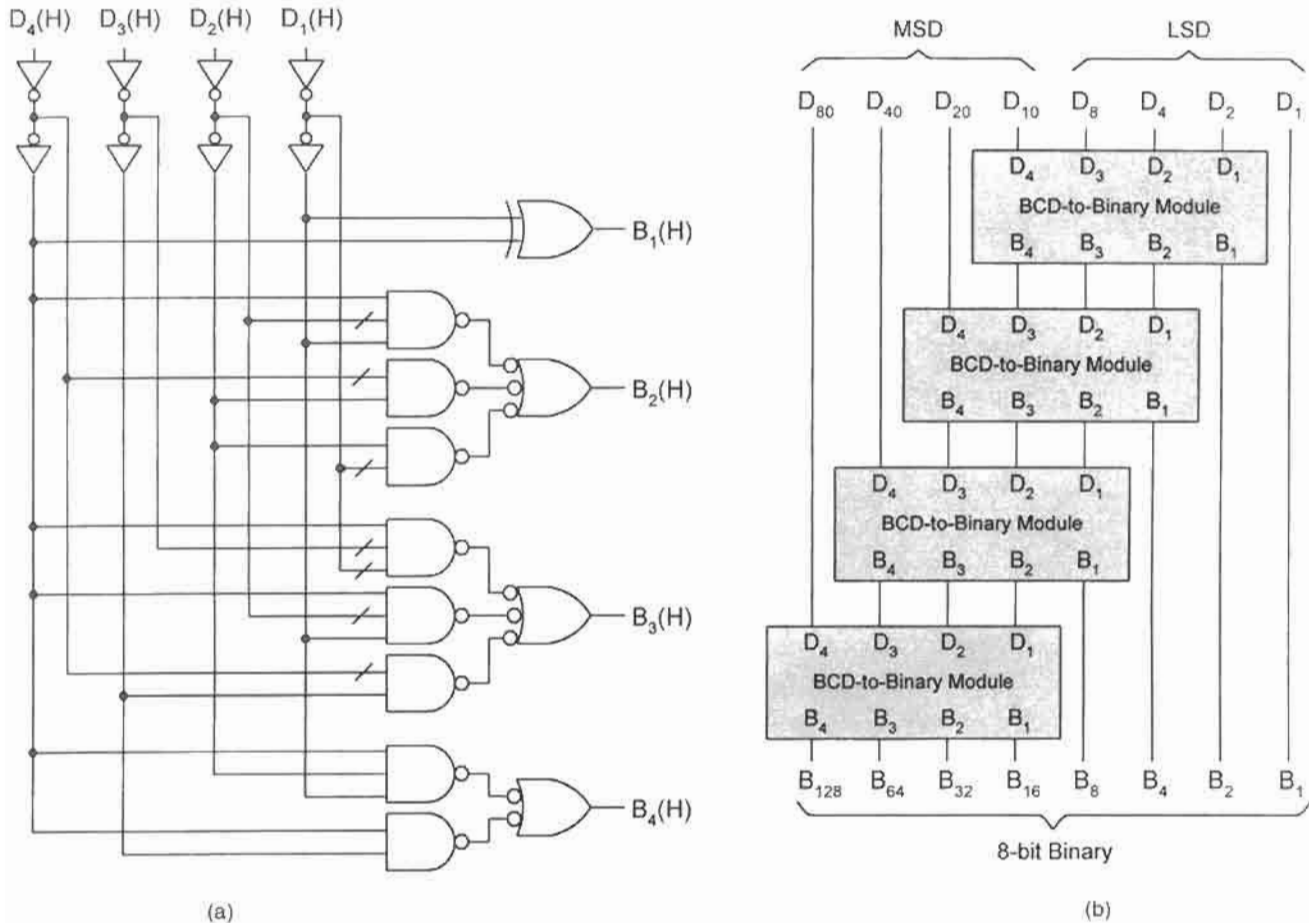
The EV K-maps for the seven segment outputs are shown in Fig. 6.22c, where a near-minimum POS cover for each has been looped out. This results in the following

**FIGURE 6.20**

Design of an 8-bit BCD-to-binary converter. (a) Truth table for a 2-digit BCD-to-binary module. (b) K-maps plotted directly from the truth table showing minimum cover.

expressions

$$\begin{aligned}
 a &= (A + B + C + \bar{D})(\bar{B} + D) \\
 b &= \bar{B} + C \odot D = \bar{B} + C \oplus \bar{D} \\
 &= (\bar{B} + C + \bar{D})(\bar{B} + \bar{C} + D) \\
 c &= (B + \bar{C} + D) \\
 d &= (B + C + \bar{D})(\bar{B} + C \oplus D) \\
 &= (B + C + \bar{D})(\bar{B} + C + D)(\bar{B} + \bar{C} + \bar{D}) \\
 e &= \bar{D}(\bar{B} + C) \\
 f &= (\bar{C} + \bar{D})(A + B + \bar{D})(B + \bar{C}) \\
 g &= (A + B + C)(\bar{B} + \bar{C} + \bar{D}),
 \end{aligned} \tag{6.18}$$

**FIGURE 6.21**

Implementation of the BCD-to-binary module and the 8-bit BCD-to-binary converter. (a) Logic circuit for the 2-digit module according to Eqs. (6.17). (b) Cascaded modules to produce the 8-bit converter.

which represent a POS gate/input tally of 19/50, excluding inverters. Notice that two of these seven expressions are also presented in hybrid (POS/XOR) form and that one,  $d$ , is a three-level expression if the XOR term is considered one level as in Fig. 3.26a.

A BCD-to-seven-segment converter uses either a *common cathode* or *common anode* LED display as shown in Fig. 6.23, but where the individual LEDs are arranged as in Fig. 6.22a. The common anode configuration requires that the outputs in Eqs. (6.18) be active low while the common cathode configuration requires that they be active high. For this example the common cathode configuration of LEDs is chosen, requiring that the expressions in Eqs. (6.18) be complemented and active low. When this is done, the resulting two-level SOP (active low) expressions become Eqs. (6.19),



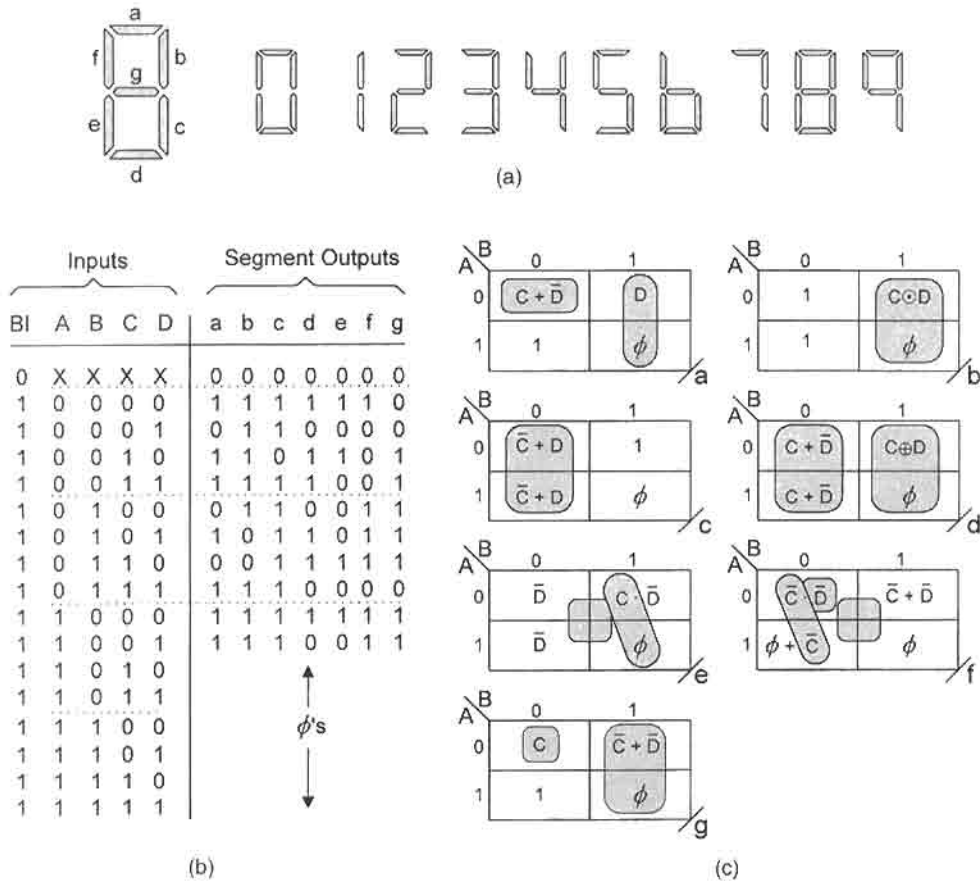
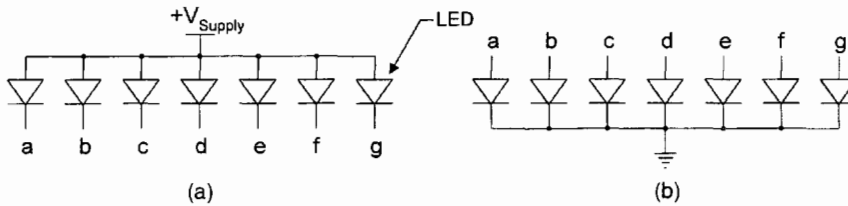


FIGURE 6.22

Near-minimum design of a BCD-to-seven-segment converter. (a) Display format and the ten decimal digits. (b) Truth table with a blanking input and without FDR. (c) EV K-maps for the seven-segment outputs showing near-minimum POS cover as given by Eqs. (6.18).

$$\begin{aligned}
 a(L) &= [\bar{A}\bar{B}\bar{C}D + B\bar{D}](L) \\
 b(L) &= [B\bar{C}D + BC\bar{D}](L) \\
 c(L) &= [\bar{B}\bar{C}\bar{D}](L) \\
 d(L) &= [\bar{B}\bar{C}D + B\bar{C}\bar{D} + BCD](L) \\
 e(L) &= [D + B\bar{C}](L) \\
 f(L) &= [CD + \bar{A}\bar{B}D + \bar{B}C](L) \\
 g(L) &= [\bar{A}\bar{B}\bar{C} + BCD](L)
 \end{aligned} \tag{6.19}$$

and are implemented in Fig. 6.24, where the active low blanking input  $BI(L)$  is realized by using inverting tri-state drivers with active low controls as in Fig. 3.8d. Thus,  $BI$  serves to

**FIGURE 6.23**

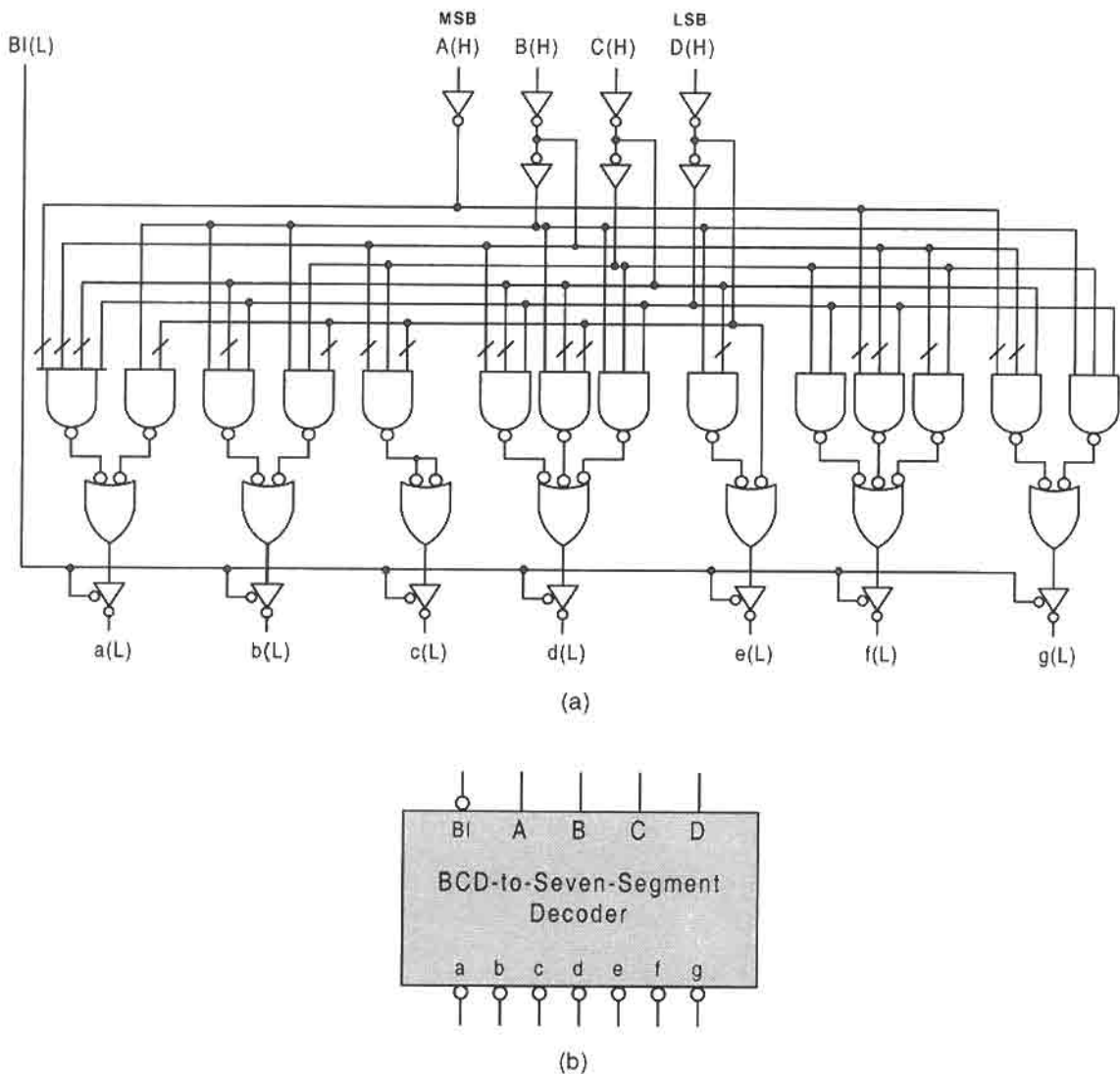
LED configurations for the BCD-to-seven-segment display converter. (a) Common anode. (b) Common cathode.

enable the decoder if  $BI(L) = 1(L)$ , or disable it if  $BI(L) = 0(L)$ . Notice that the common cathode configuration of LEDs in Fig. 6.23b requires the inputs to be  $1(H) = HV$  to force the diode into forward bias (conducting mode). Thus, a  $0(L)$  for any output in Eqs. (6.19) is a  $1(H)$ , which is the same as a 1 in the positive logic truth table of Fig. 6.22b. Coupling the decoder of Fig. 6.24 with the common anode configuration requires the use of noninverting tri-state drivers with active low controls as in Fig. 3.8b. In this case, each output in Eqs. (6.19) would be issued active high such that any  $0(H)$  output (to the LED) forces a diode in Fig. 6.23a into forward bias. A commercial IC chip with logic suitable for coupling with the common anode LED configuration of Fig. 6.23a is the 74x49. Its logic differs somewhat from that of Fig. 6.24 because it generates the blanking condition in a different way—it uses a form of FDR—and it reverses the input lettering from MSB (D) to LSB (A).

The blanking feature shown in Fig. 6.25 is useful in removing leading zeros in integer displays and trailing zeros in fixed-point decimal displays. When the blanking feature is used in this way it is called *zero-blanking*. For example, 036.70 would appear if no zeros are blanked but would be 36.7 after zero-blanking. To accomplish the zero-blanking capability requires that additional logic be connected to the  $BI$  input. The idea here is that when the inputs to an MSD stage are zero, the zero-blanking logic must deactivate  $BI$  [ $BI(L) = 0(L)$ ] but must not do so for intermediate zeros as, for example, in 40.7. ICs with this capability are designed with a *zero-blanking input* (ZBI) and a *zero-blanking output* (ZBO) so that when the decade stages are connected together, ZBO-to-ZBI, zero blanking can *ripple* in the direction of the radix point terminal. This is easily accomplished as illustrated in Fig. 6.25 for an integer display, where external logic is connected to the  $BI$  inputs of the BCD-to-seven-segment decoders of Fig. 6.24 such that *only* leading zeros are blanked in ripple fashion from MSD-to-LSD.

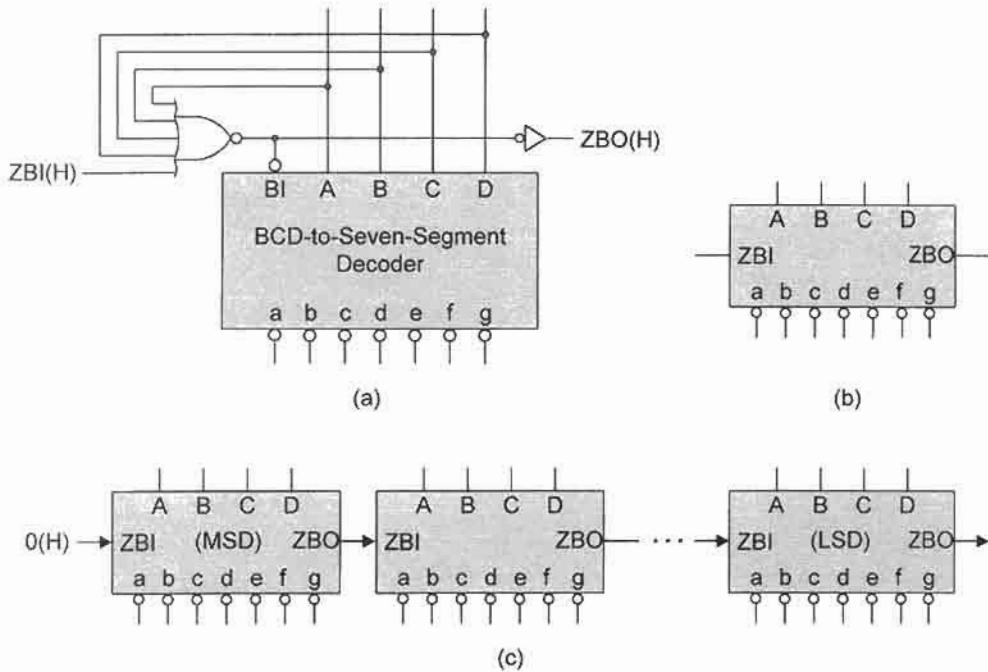
## 6.6 MAGNITUDE COMPARATORS

A device that determines which of two binary numbers is larger or if they are equal is called a *magnitude comparator* or simply *comparator*. A vending machine, for example, must engage a comparator each time a coin is inserted into the coin slot so that the desired item can be dispensed when the correct change has been inserted. The block diagram symbol for an  $n$ -bit comparator with cascading capability is given in Fig. 6.26. Here, it is to be understood that  $gt$  and  $(A > B)$  represent  $A$  greater than  $B$ ;  $eq$  and  $(A = B)$  represent  $A$  equal to  $B$ ;  $lt$  and

**FIGURE 6.24**

Logic circuit (a) and circuit symbol (b) for the BCD-to-seven-segment decoder according to Eqs. (6.19) and Fig. 6.22, featuring an active low blanking input  $BI$  implemented by using inverting three-state drivers with active low controls.

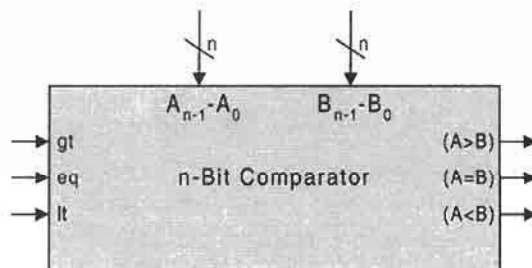
$(A < B)$  represent  $A$  less than  $B$ . For cascading purposes, the inputs  $gt$ ,  $eq$ , and  $lt$  to the  $k$ th stage are the outputs ( $A > B$ ,  $A = B$ , and  $A < B$ ) from the next least significant  $(k - 1)$ th stage, while the corresponding outputs of the  $k$ th stage are the inputs ( $gt$ ,  $eq$ , and  $lt$ ) to the next most significant  $(k + 1)$ th stage. Thus, the magnitudes of the  $k$ th stage are more significant than those of the  $(k - 1)$ th stage, as the magnitudes of the  $(k + 1)$ th stage are more significant

**FIGURE 6.25**

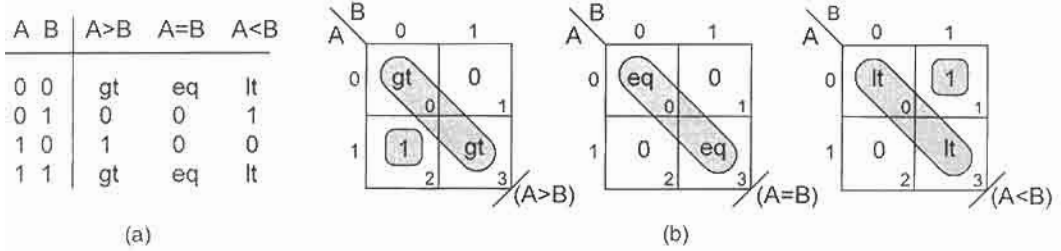
BCD-to-seven-segment decoding with zero-blanking capability. (a) External logic required for zero blanking. (b) Circuit symbol for zero-blanking decoder module. (c) Cascaded modules for integer representation showing rippled zero blanking of leading zeros from MSD-to-LSD stages.

than those of the  $k$ th stage, etc. Though seemingly trivial, these facts are important in establishing the proper truth table entries for comparator design, as the following examples will illustrate.

The design of a useful cascadable comparator begins with the 1-bit design. Shown in Fig. 6.27 are the EV truth table and EV K-maps for a cascadable 1-bit comparator. The

**FIGURE 6.26**

Circuit symbol for an  $n$ -bit comparator with cascading capability.

**FIGURE 6.27**

Design of a cascadable 1-bit comparator. (a) EV truth table. (b) Output EV K-maps showing minimum cover with XOR-type functions.

gate-minimum cover for each of the three outputs, as extracted from the K-maps, is

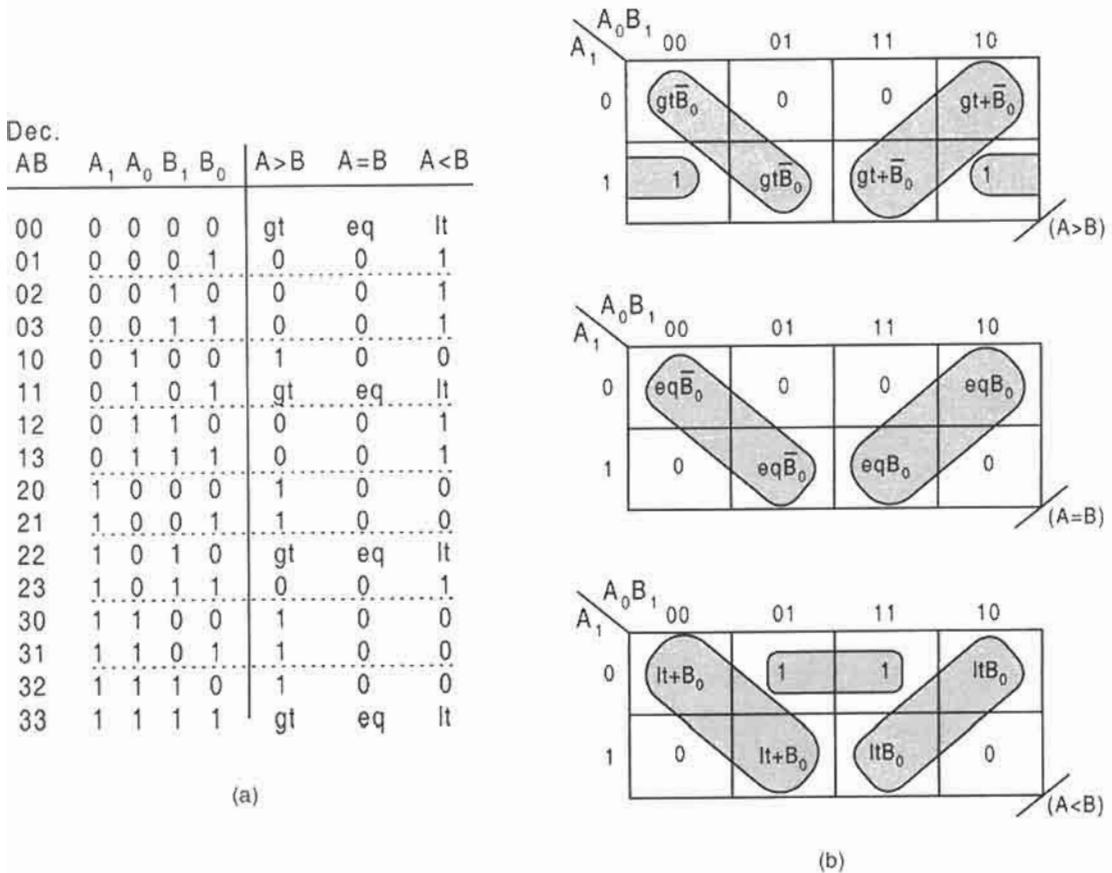
$$\left\{ \begin{array}{l} (A > B) = gt(A \odot B) + A\bar{B} \\ (A = B) = eq(A \odot B) \\ (A < B) = lt(A \odot B) + \bar{A}B \end{array} \right\}, \quad (6.20)$$

as given in three-level form. These represent a gate/input tally of 8/16, excluding inverters. Notice that in the truth table of Fig. 6.27a all three inputs, *gt*, *eq*, and *lt*, appear only when *A* and *B* are of equal magnitude and that logic 1 appears when one, *A* or *B*, dominates in magnitude.

In order to establish an important trend in the output expressions, one that can be used to establish the logic for any size comparator, it is helpful to construct the truth table and EV K-maps for a cascadable 2-bit comparator. These are provided in Fig. 6.28. As in the 1-bit comparator design, the EVs (*gt*, *eq*, and *lt*) in the truth table of Fig. 6.28a are the outputs from the next least significant stage, which explains why they appear only when *A* and *B* are of equal magnitude. The gate-minimum output cover given in Fig. 6.28b yields the following expressions in three-level form:

$$\left\{ \begin{array}{l} (A > B) = gt\bar{A}_0\bar{B}_0(A_1 \odot B_1) + A_0(gt + \bar{B}_0)(A_1 \odot B_1) + A_1\bar{B}_1 \\ \quad = gt(A_1 \odot B_1)(A_0 \odot B_0) + A_0\bar{B}_0(A_1 \odot B_1) + A_1\bar{B}_1 \\ (A = B) = eq\bar{A}_0\bar{B}_0(A_1 \odot B_1) + eqA_0B_0(A_1 \odot B_1) \\ \quad = eq(A_1 \odot B_1)(A_0 \odot B_0) \\ (A < B) = \bar{A}_0(lt + B_0)(A_1 \odot B_1) + ltA_0B_0(A_1 \odot B_1) + \bar{A}_1B_1 \\ \quad = lt(A_1 \odot B_1)(A_0 \odot B_0) + A_0B_0(A_1 \odot B_1) + \bar{A}_1B_1 \end{array} \right. \quad (6.21)$$

These represent a gate/input tally of 11/29 (excluding inverters) with a maximum fan-in of 4. Notice that in arriving at the second equation for each of the outputs, use is made of the absorptive law, Eqs. (3.13), in the form  $gt + \bar{B}_0 = gtB_0 + \bar{B}_0$  and  $lt + B_0 = lt\bar{B}_0 + B_0$ . In comparison, a two-level optimization of this comparator gives a gate/input tally of 21/60,

**FIGURE 6.28**

Design of a cascadable 2-bit comparator. (a) Compressed truth table. (b) EV K-maps plotted from (a) showing minimum cover involving XOR-type functions.

excluding inverters, and has a maximum fan-in of 7. Thus, the advantage of the multilevel gate-minimum form over that of the two-level implementation is evident. This is especially true if CMOS is used in the design of the comparator. Recall that the CMOS EQV gate in Fig. 3.27a has about the same gate path delay as a two-input NAND gate.

If  $A = B$  in Eqs. (6.20) and (6.21), there results  $(A > B) = gt$  and  $(A < B) = lt$ , each of which must be logic 0 for the  $A = B$  condition from the next least significant stage. Therefore, it follows generally that

$$(A = B) = (\overline{A > B}) \cdot (\overline{A < B}), \quad (6.22)$$

which yields  $(A = B) = 0$  for any condition other than  $A = B$ .

The trend that emerges in the output expressions is easily deduced from an inspection of Eqs. (6.20) and (6.21), thereby permitting the three output expressions to be written for any

number of bits. As an example, the output expressions for a cascadable 4-bit comparator become

$$\left\{ \begin{array}{l} (A > B) = gt \cdot \prod_{i=0}^3 (A_i \odot B_i) + A_0 \bar{B}_0 \prod_{i=1}^3 (A_i \odot B_i) + A_1 \bar{B}_1 \prod_{i=2}^3 (A_i \odot B_i) \\ \quad + A_2 \bar{B}_2 (A_i \odot B_i) + A_3 \bar{B}_3 \\ (A = B) = eq \cdot \prod_{i=0}^3 (A_i \odot B_i) = \overline{(A > B)} \cdot \overline{(A < B)} \\ (A < B) = lt \cdot \prod_{i=0}^3 (A_i \odot B_i) + \bar{A}_0 B_0 \prod_{i=1}^3 (A_i \odot B_i) + \bar{A}_1 B_1 \prod_{i=2}^3 (A_i \odot B_i) \\ \quad + \bar{A}_2 B_2 (A_i \odot B_i) + \bar{A}_3 B_3 \end{array} \right\}, \quad (6.23)$$

where

$$\prod_{i=0}^3 (A_i \odot B_i) = (A_0 \odot B_0)(A_1 \odot B_1)(A_2 \odot B_2)(A_3 \odot B_3)$$

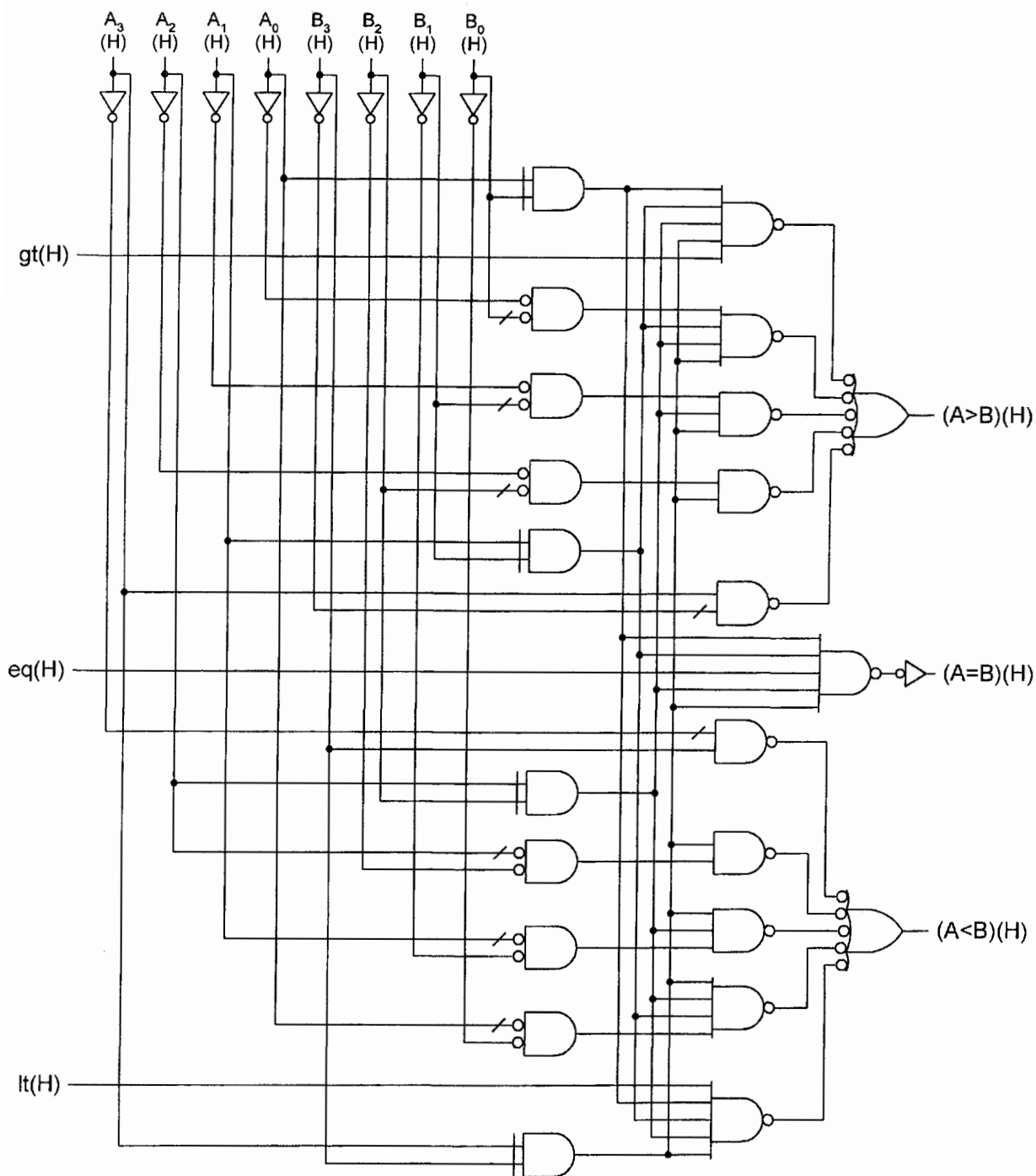
and

$$\prod_{i=2}^3 (A_i \odot B_i) = (A_2 \odot B_2)(A_3 \odot B_3), \text{ etc.}$$

Implementation of Eqs. (6.23) is given in Fig. 6.29, using three-level NAND/NOR/EQV logic and limiting fan-in to 5. Here, the gate/input tally is 23/67, excluding inverters.

The commercial chip that is equivalent to the cascadable 4-bit comparator of Fig. 6.29 is the 74xx85. Though the two differ somewhat in the logic circuit makeup, they function in exactly the same way. Either can be cascaded to form a comparator of any number of bits in multiples of four bits. Shown in Fig. 6.30 is an 8-bit comparator formed by cascading two 4-bit comparators in series. Notice that the inputs to the *least significant stage* are required to be at the fixed logic levels shown.

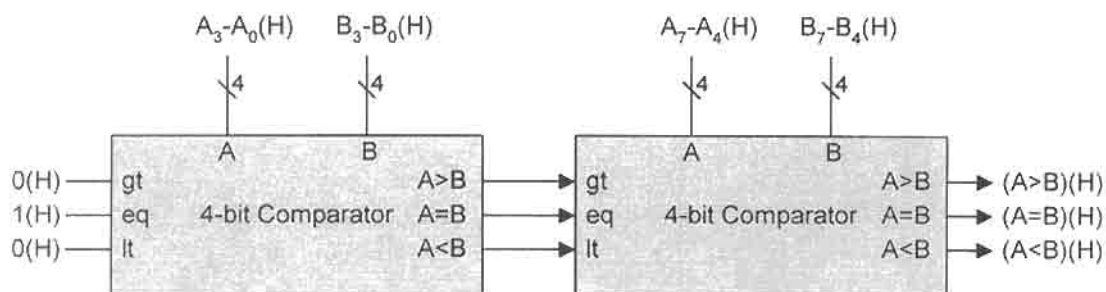
Combining three or more comparators in series suffers significantly in propagation delay. Much better is the series/parallel arrangement shown in Fig. 6.31. Here, six 4-bit comparators are combined to form a 24-bit comparator where no more than two comparators are placed in series and all inputs and outputs are active high. In this case, a dominant  $A$  or  $B$  word is picked up by one of the five stages in ascending order of significance and is issued as either  $(A > B)$ ,  $(A = B)$ , or  $(A < B)$  from the MSB stage of the output comparator. Words  $A$  and  $B$  of equal magnitude are picked up by the least significant stage and issued by the output comparator as  $(A = B)$ . The series/parallel comparator arrangement in Fig. 6.31 is justifiable on the basis of path delay arguments when the alternative is considered — namely, the series configuration of six 4-bit comparators. Notice that not all of the 24 bits of the comparator need be used. Any size  $A$ ,  $B$  words up to 24 bits can be compared by using the comparator of Fig. 6.31. The only requirement is that words  $A$  and  $B$  be of equal length and that MSB inputs not in use be held at 0(H).



**FIGURE 6.29**

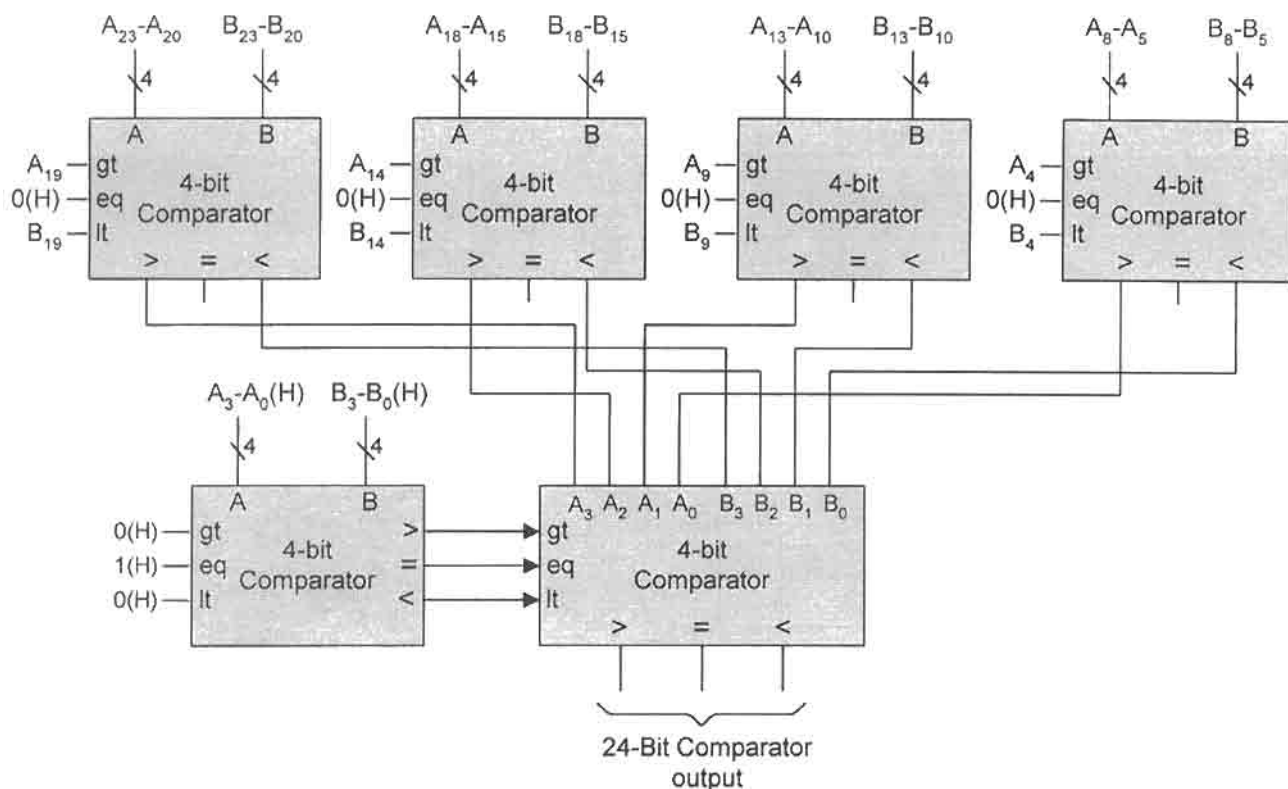
Three-level logic circuit for the cascadable 4-bit comparator by using NAND/NOR/EQV logic with a fan-in limit of 5.





**FIGURE 6.30**

A cascaded 8-bit comparator created by combining two 4-bit comparators.



**FIGURE 6.31**

Speed-optimized series/parallel arrangement for a 24-bit comparator composed of six 4-bit comparators.

## 6.7 PARITY GENERATORS AND ERROR CHECKING SYSTEMS

A parity bit can be appended to any word of  $n$  bits to generate an even or an odd number of 1's (or 0's). A combinational logic device that generates even or odd parity is called a *parity generator*. A device that is used to detect even or odd parity is called a *parity detector*. To understand the concept of parity generation and detection, consider the following 8-bit words to which a ninth parity bit is appended as the LSB shown in brackets:

1 1 0 1 0 1 0 1 [1]: Even parity generation = Odd parity detection

1 1 0 1 0 1 0 1 [0]: Odd parity generation = Even parity detection

or

1 1 0 1 0 0 0 1 [1]: Odd parity generation = Even parity detection

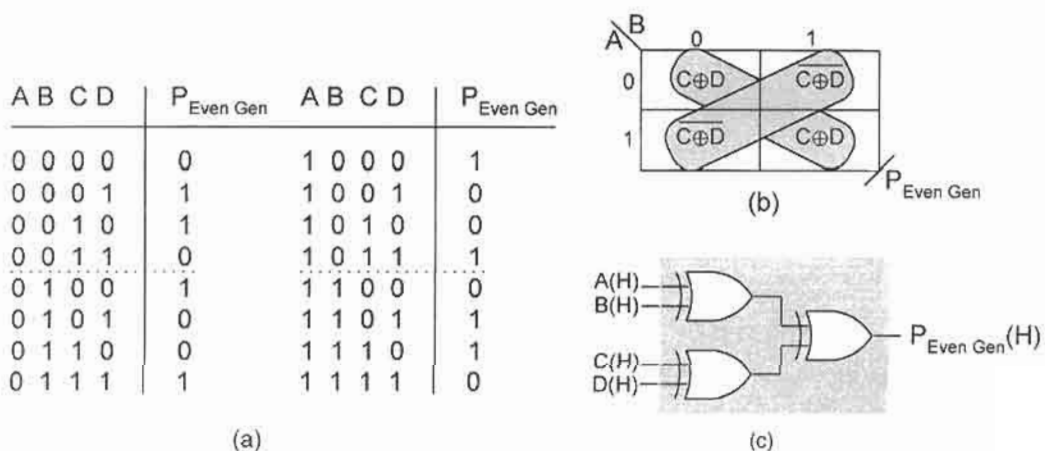
1 1 0 1 0 0 0 1 [0]: Even parity generation = Odd parity detection.

Thus, parity generation includes the parity bit in the count of 1's, whereas parity detection excludes the parity bit in the count but uses the parity bit to identify the parity status (even or odd) of the word. The parity bit may be appended either at the LSB position, as in the examples just given, or at the MSB position.

XOR gates can be combined to produce  $n$ -bit parity circuits. As an example, consider the design of a 4-bit *even-parity generator module* shown in Fig. 6.32. The second-order EV K-maps in Fig. 6.32b follow directly from the truth table in Fig. 6.32a and yield the output expression

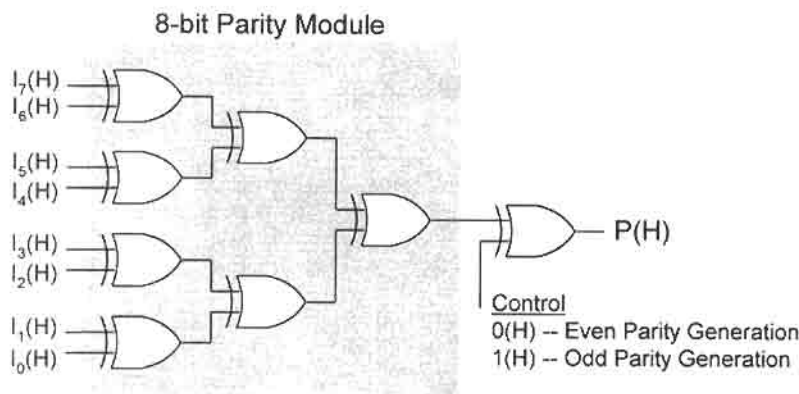
$$P_{\text{Even Gen}} = A \oplus B \oplus C \oplus D, \quad (6.24)$$

which is implemented in Fig. 6.32c.



**FIGURE 6.32**

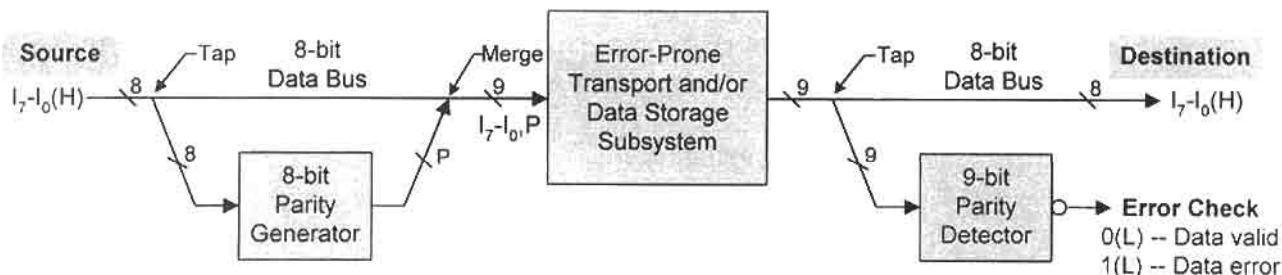
Design of the 4-bit even parity generator module. (a) Truth table. (b) EV K-map. (c) Logic circuit according to Eq. (6.24).

**FIGURE 6.33**

An 8-bit even/odd parity generator circuit with a control input.

This 4-bit parity generator module of Fig. 6.32 can be combined with another to form the 8-bit parity generator module shown in Fig. 6.33. Here, an additional XOR gate is added as a control to generate either an even or odd parity generate output. Notice that two 8-bit parity generator modules can be cascaded to produce a 16-bit module, and two 16-bit modules can be cascaded to produce a 32-bit module, etc., each with a control XOR gate to produce either an even or odd parity generation output. Note also that any  $2^n$ -bit parity module is an  $n$ -level XOR circuit, excluding any control XOR gate. Thus, a 32-bit parity module represents a 5-level XOR circuit, a 64-bit module is a 6-level circuit, etc.

**Error Checking Systems** Errors occur in digital systems for a variety of reasons. These reasons include logic noise, power supply surges, electromagnetic interference, and crosstalk due to physically close signals. Single-error detection in a digital system usually amounts to a parity checking system where the parities at the source and destination sites are compared. A typical single-error checking system for an 8-bit data bus is illustrated in Fig. 6.34. In this case an 8-bit parity generator of the type in Fig. 6.33 is used at the source to generate either an even or odd parity bit. The 9-bit parity detector at the destination is the 8-bit parity

**FIGURE 6.34**

Error checking system for an 8-bit data bus by using an 8-bit parity generator at the source and a 9-bit parity detector at the destination site.

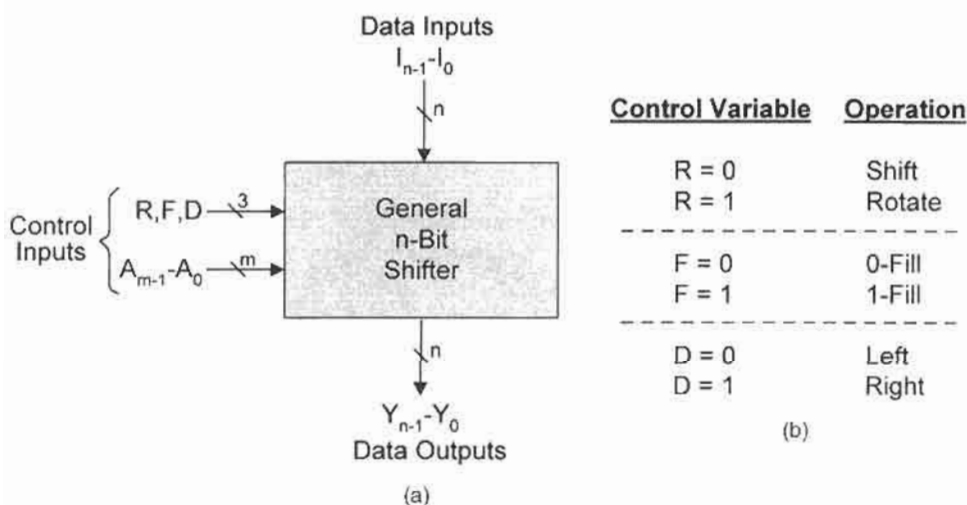
generator module of Fig. 6.33 with an XOR gate and inverter as the output stage, hence an EQV gate, as implied in Fig. 6.34.

The parity checking scheme illustrated in Fig. 6.34 is valid for the detection of a single error in the 8-bit word. Actually, it is valid for any odd number of errors, but the probability that three or more errors will occur in a given word is near zero. What a single-bit parity checking system cannot do is detect an even number of errors (e.g., two errors). It is also true that the error checking system of Fig. 6.34 cannot correct any single error it detects. To do so would require detecting its location, which is no trivial task. To identify the location of an error bit requires multiple parity detection units on the submodular level down on the bit level, a significant cost in hardware. However, when this is done, an erroneous bit can be corrected. Memory systems in modern computers have such single-error correction capability.

## 6.8 COMBINATIONAL SHIFTERS

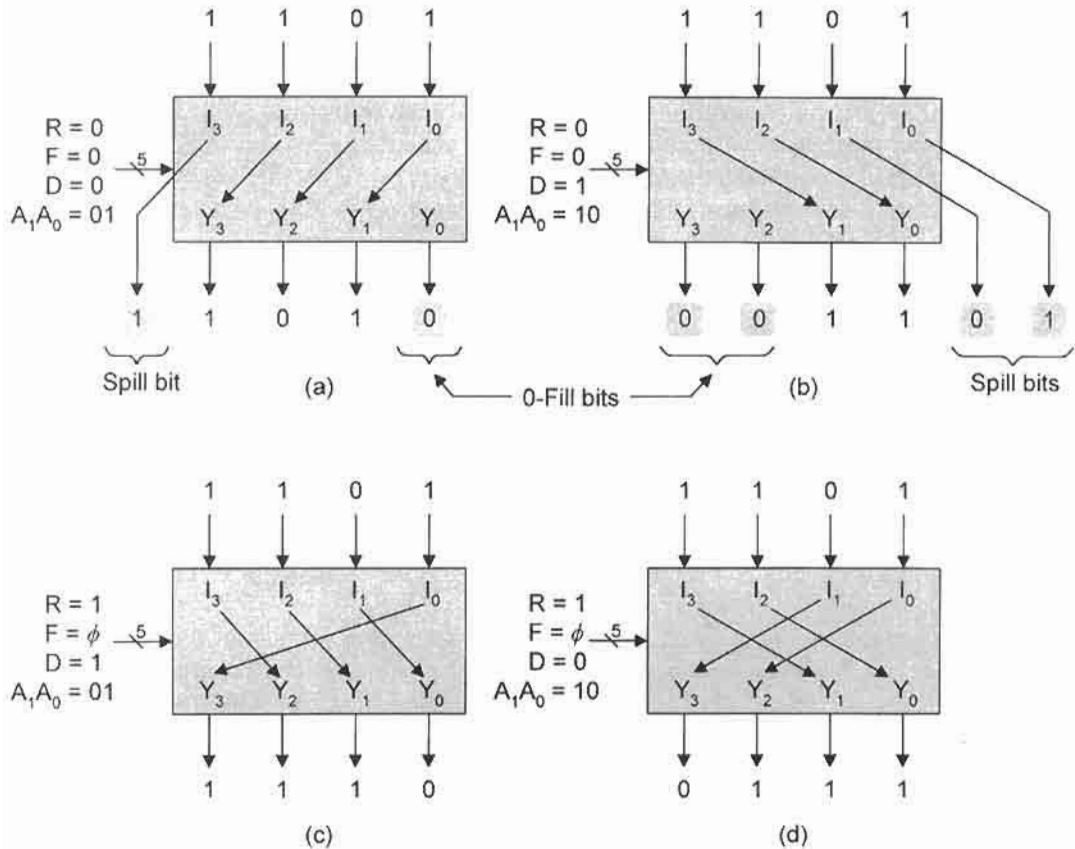
Shifting or rotating of word bits to the right or left can be accomplished by using combinational logic. Devices that can accomplish this are called *combinational shifters*, or *barrel shifters* if their function is only to rotate word bits. Shifters are used for bit extraction operations, transport, editing, data modification, and arithmetic manipulation, among other applications.

A general  $n$ -bit shifter is an  $(n + m + 3)$ -input/ $n$ -output device represented by the logic symbol in Fig. 6.35a, with the interpretations of the control inputs given in Fig. 6.35b. A shifter of this type accepts  $n$  data input bits ( $I_{n-1} - I_0$ ) and either passes these values straight through to the data outputs ( $Y_{n-1} - Y_0$ ), or shifts or rotates them by one or more bit positions to the right or left with 0 or 1 fill on command of the  $m + 3$  control inputs. The control inputs consist of a rotate control ( $R$ ), fill control ( $F$ ), direction control ( $D$ ), and  $m$  inputs ( $A_{m-1} - A_0$ ) to control the number of bit positions to be shifted or rotated—usually binary encoded to 0, 1, 2, 3, ...,  $p$  bit positions. For the shifter of Fig. 6.35, the control



**FIGURE 6.35**

A general  $n$ -bit shifter. (a) Block diagram symbol. (b) Interpretation of the control inputs  $R$ ,  $F$ , and  $D$ .

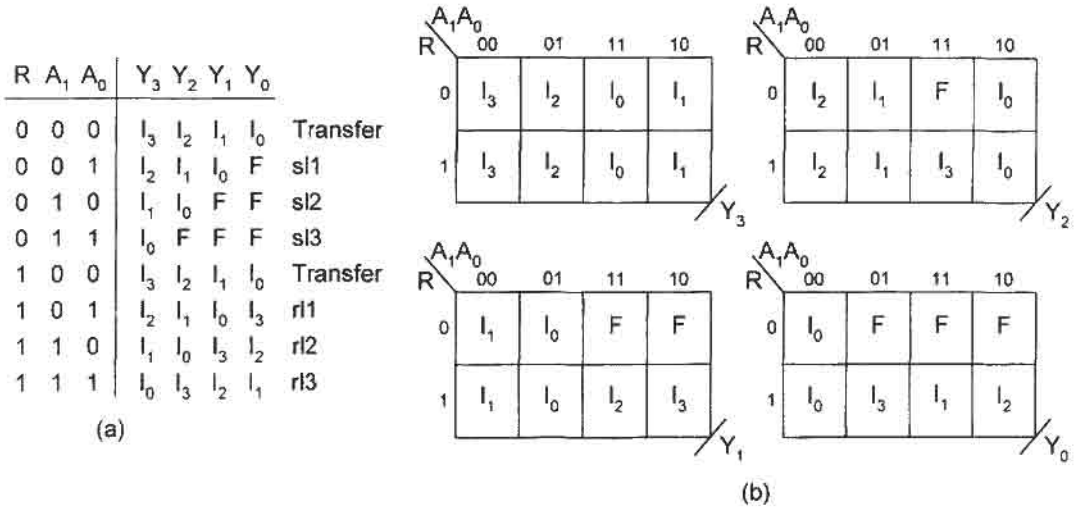
**FIGURE 6.36**

Examples of shift and rotation operations by using a general 4-bit shifter. (a) Shift left one bit (sl1) with 0-fill. (b) Shift right two bits (sr2) with 0-Fill. (c) Rotate right one bit (rr1). (d) Rotate left two bits (rl2).

input word is given the positional weights  $RFDA_{m-1} \cdots A_0$  and is sometimes represented as  $S_{k-1} \cdots S_1 S_0$ , where  $k = m + 3$ .

The control variable ( $R = 0$ ) causes the shifter in Fig. 6.35 to shift the input word (right or left) by  $p$  bits while, at the same time, causes Spill and 0- or 1-Fill by  $p$  bits. The control variable ( $R = 1$ ) rotates the input word (right or left) by  $p$  bits with no Spill or Fill. These operations are illustrated in Fig. 6.36 for a 4-bit shifter with input word 1101. In Figs. 6.36a and 6.36b, the shifter is set for shifting operations, first shift left by one bit (sl1) and then shift right by two bits (sr2) with spill and 0-fill bits equal to the number of bits shifted. Note that the spill bits are always lost. In Figs. 6.36c and 6.36d, the shifter is set for rotation with settings for 1-bit right rotation (rr1) and 2-bits left rotation (rl2), respectively. Notice that rotation requires that end bits, normally spilled (discarded) in a shift operation, be rotated around to the opposite end of the shifter. The rotation mode could be called the barrel shifter mode, the word barrel implying "shift around."

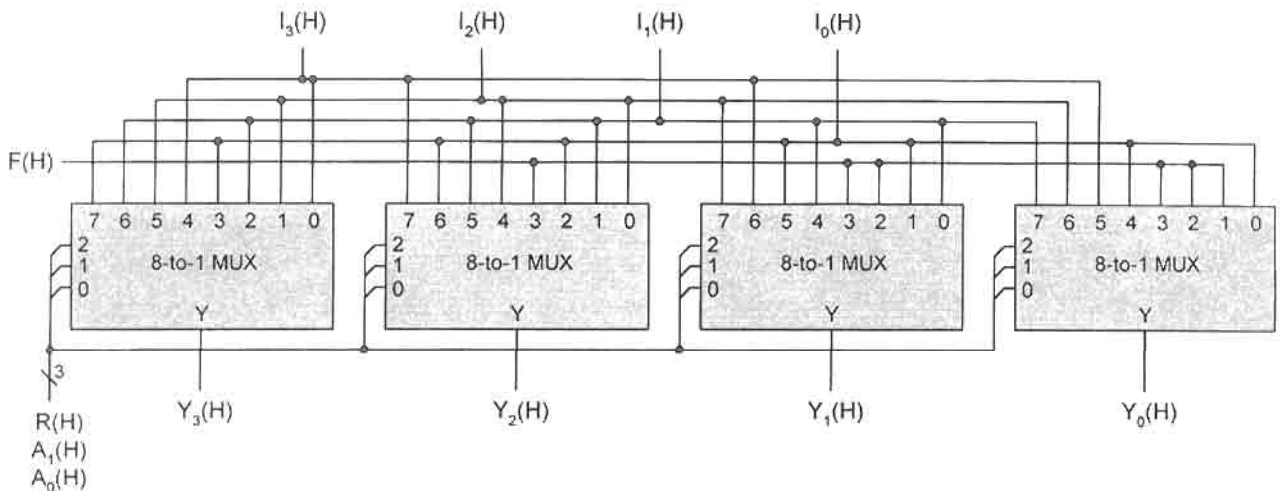
The design of a 4-bit shifter capable of left shifting or rotation by up to three bits is shown in Fig. 6.37. To reduce the size of the table, the fill bit  $F$  is included as an entered variable in

**FIGURE 6.37**

Design of a 4-bit general shifter that can shift or rotate left up to three bits with  $F$  fill. (a) EV Truth table. (b) Output K-maps suitable for 8-to-1 MUX implementation.

the truth table of Fig. 6.37a, and the absence of the direction variable  $D$  implies that  $D = 0$  for left shift/rotate. Thus, there are four control variables  $R$ ,  $F$ ,  $A_1$ , and  $A_0$ , as indicated in Fig. 6.37a. The EV K-maps in Fig. 6.37b are plotted directly from the truth table.

The four outputs represented by the K-maps in Fig. 6.37b can be implemented in a variety of ways, including discrete logic. However, their form suggests an 8-to-1 MUX approach, which is the approach of choice for this example. Shown in Fig. 6.38 is the MUX implementation of the general 4-bit left shifter/rotator represented by Fig. 6.37.

**FIGURE 6.38**

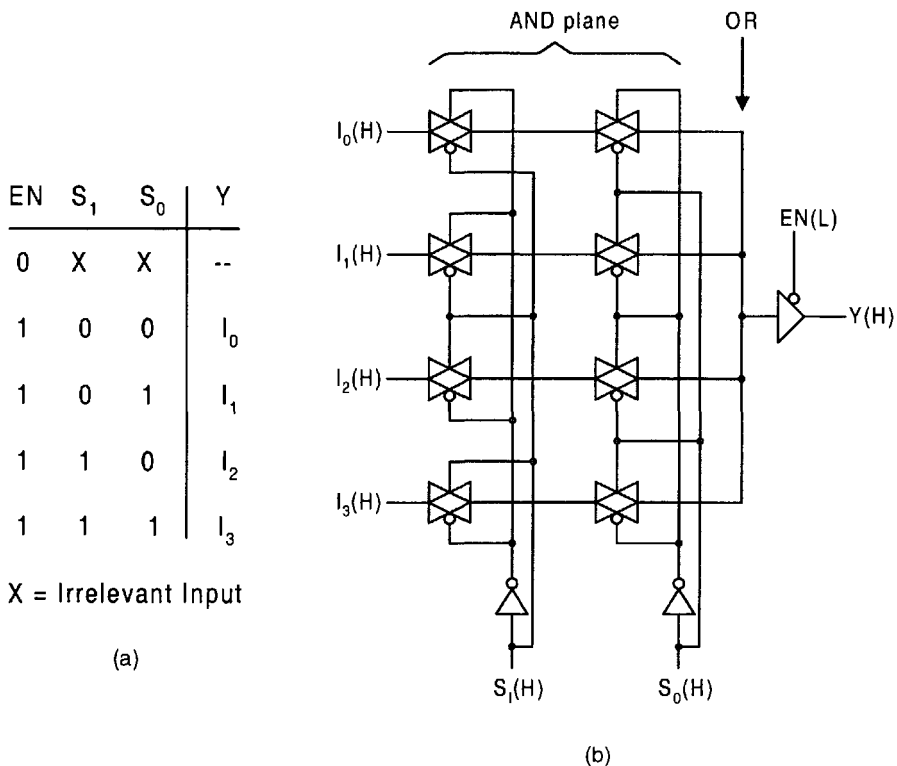
MUX implementation of the general 4-bit left shifter/rotator with  $F$  (0 or 1) fill. The MUX select inputs are weighted  $RA_1A_0$  for  $S_2S_1S_0$ , respectively.

Notice that in Fig. 6.38 the control variable  $D$  is missing, meaning that  $D = 0$  for left shifting. Had the variable  $D$  been included for left or right shifting/rotating, fourth-order EV K-maps and 16-to-1 MUXs would have to be used if the architecture of Fig. 6.38 were to be retained. Also note that this shifter can be cascaded by connecting  $Y_3(H)$  of one stage to  $F(H)$  of the next most significant stage, etc.

## 6.9 STEERING LOGIC AND TRI-STATE GATE APPLICATIONS

Any of the combinational logic devices discussed to this point can be designed by using transmission gates together with inverters. When this is done the devices are classified as *steering logic*. A transmission gate lacks logic function capability by itself, but can be configured with other transmission gates to steer the logic signals in a manner that carries out a logic function. Inverters are necessarily included in most steering logic designs because transmission gates are passive and noninverting, as explained in Section 3.5.

As a simple example, consider a 4-to-1 MUX defined by the truth table in Fig. 6.39a. This device can be implemented easily with CMOS transmission gates and inverters and at



**FIGURE 6.39**

Transmission gate implementation of the 4-to-1 MUX. (a) Truth table. (b) Logic circuit with a tri-state buffered/enabled output.

a considerable savings in transistor count and design area, as indicated in Fig. 6.39b. Here, the MUX is enabled and buffered by using a tri-state driver with an active low control input. Thus, the disable condition [ $EN(L) = 0(L)$ ] is actually a disconnect state as indicated in Fig. 3.8b and is represented by the dash in the truth table of Fig. 6.39a. Notice that the AND plane, equivalent to the four four-input NAND gates in Fig. 6.4d, is constructed with only eight transmission gates, and that the OR operation is “wired” to the tri-state driver since only one line can be active at any given time. Consequently, the transmission gate design represents a significant savings in hardware cost and will be faster (shorter throughput) compared to the NAND gate design in Fig. 6.4d.

An important aspect of steering logic designs is that the transmission gates are non-restoring devices and must be buffered to prevent degradation of the signal. The following is offered as a “rule of thumb” in dealing with transmission gate designs:

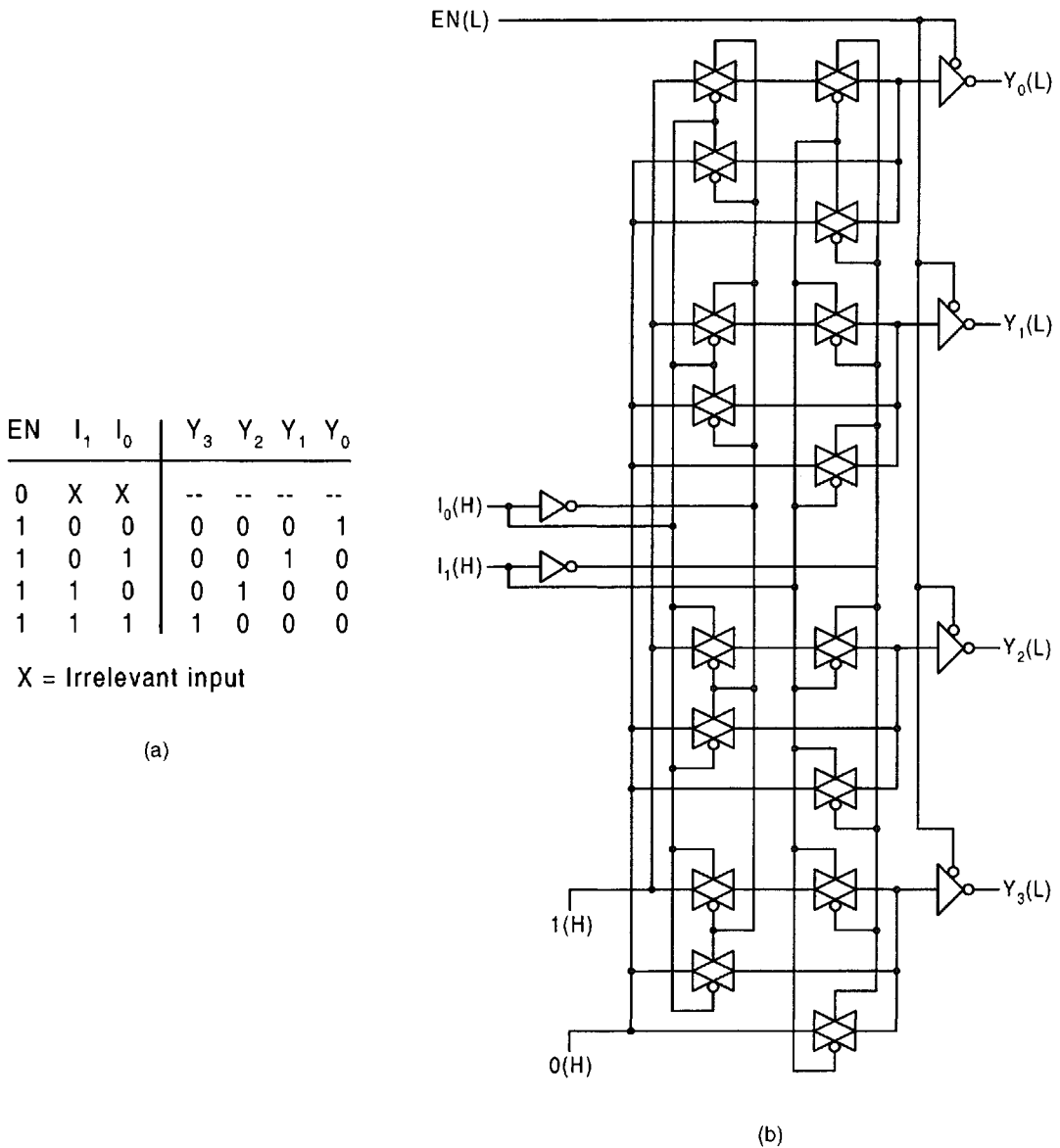
For signal-restoring purposes in logic designs that use transmission gates, plan to buffer each signal for every *four* transmission gates through which the signal must pass. CMOS transmission gates should be counted as two pass transistors.

The design of a 2-to-4 decoder is used here as another simple example of a CMOS transmission gate (TG) implementation of a combinational logic device. Shown in Fig. 6.40a is the truth table for this decoder and in Fig. 6.40b is the TG implementation of the decoder taken directly from the truth table. The outputs are shown enabled and buffered with inverting tri-state drivers having an active low control input as in Fig. 3.8d. If active high outputs are required, noninverting tri-state drivers of the type shown in Fig. 3.8b can be used. Notice that each “1” CMOS TG leading to a  $Y$  output must have a “0” TG associated with it and that all TGs have complementary EN inputs connected to them from an inverter. Thus, since there is a series of two “1” TGs per  $Y$  output, there are two “0” TGs for each output, making a total of 16 TGs or a total of 60 transistors, including inverters and tri-state buffer/enables. In comparison, the gatewise CMOS NAND implementation of the same decoder yields a transistor count of only 34. Though the transistor count for the TG design is greater than that for the gatewise implementation, the speed (throughput) should be comparable and perhaps a little faster for the TG design.

## 6.10 INTRODUCTION TO VHDL DESCRIPTION OF COMBINATIONAL PRIMITIVES

With ever-increasing complexity of digital systems, there comes a greater need for simulation, modeling, testing, automated design, and documentation of these systems. The challenge here is to make the English language readable by a computer for computer-aided design (CAD) purposes. Hardware description languages (HDLs) satisfy these requirements. VHSIC (for very high speed integrated circuit) is such an HDL. It was funded by the Department of Defense in the late 1970s and early 1980s to deal with the complex circuits of the time. However, the capabilities of the VHSIC language soon reached their limit, giving way to more advanced HDLs that could meet the challenges of the future. One important language that has emerged is known as VHSIC Hardware Description Language or simply VHDL. VHDL was first proposed in 1981 as a standard for dealing with very complex



**FIGURE 6.40**

Transmission gate implementation of the 2-to-4 decoder. (a) Truth table. (b) Logic circuit with active low outputs produced by tri-state buffer/enables.

circuits. It has since gone through several revisions and in 1987 and 1993 was adopted as the IEEE 1076-1987 and 1076-1993 standards, respectively. The examples given in this section will use these standards.

VHDL is a powerful tool capable of top-down design descriptions covering various levels of abstraction ranging from the *behavior level*, to the *dataflow level*, to the *structural*

*level*. Examples of the behavior level of representation are truth tables and algorithmic descriptions. The structural level of representation typically includes various *primitives* together with the interconnections required to make a circuit. The primitives covered in this chapter include discrete gates, decoders, encoders, MUXs, comparators, parity generators, and combinational shifters. Other primitives, including those associated with arithmetic circuits, PLDs, and sequential circuits, will be considered in subsequent chapters.

Before illustrating the use of VHDL for some well known combinational primitives, it will be helpful to review some assignment statements relevant to behavioral modeling. For example,

```
a <= b;
```

is read as “*a* is assigned the value of *b*.” As a second example,

```
Y <= I2 after 10 ns;
```

is interpreted as follows: “*Y* is assigned the value of *I2* after 10 nanoseconds have elapsed.” In these two examples, “<=” is an assignment operator that assigns a value to a signal. Another assignment operator is the “:=” operator. It is used to assign a value to a variable:

```
result := X; or delay := 4 ns;
```

Here, *result* and *delay* are variables and are assigned the values of *X* and 4 ns, respectively.

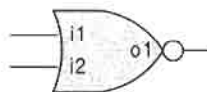
To illustrate, consider the VHDL behavioral description of the two-input NOR gate primitive given in Fig. 6.41a. In this case the **entity** is called *nor2* with inputs *i1* and *i2*, and output, *o1*, as indicated in Fig. 6.41b. The architecture for *nor2* is arbitrarily called *nor2\_behavior* and provides that “*o1* is assigned the value *i1* **nor** *i2* [or **not**(*i1* **or** *i2*)] after 5 nanoseconds.” Thus, 5 ns is the propagation delay of the gate. Notice that all VHDL keywords, such as **entity**, **port**, and **end**, and the VHDL logical “**or**” are highlighted in bold for visual effect. Also, note that the entries are case insensitive. The keyword **port** is used to identify the input and output ports of the entity.

As a second example consider the VHDL behavioral description for a 4-to-1 MUX primitive given in Fig. 6.42. For this example an arbitrary delay is assigned (to the data selection process) the keywords **generic** (**del**:time), meaning that time delay (**del**) is determined by

```
entity nor2 is
    port (i1, i2: in bit; o1: out bit);
end nor2;

architecture nor2_behavior of nor2 is
begin
    o1 <= not (i1 or i2) after 5 ns;
end nor2_behavior;
```

(a)



(b)

**FIGURE 6.41**

Behavioral model for a 2-input NOR gate. (a) VHDL entity declaration and behavioral description. (b) NOR gate circuit symbol with inputs, *i1* and *i2*, and output, *o1*.

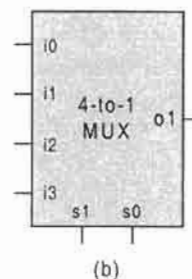
```

entity mux4 is
  generic (del: time);
  port (i0, i1, i2, i3: in bit_vector (0 to 3);
        o1: out bit_vector (0 to 3));
  group sel is (s1,s0);
end mux4;

architecture mux4_behavior of mux4 is
  begin
    o1 <= i0 after del when s1 = '0' and s0 = '0' else
          i1 after del when s1 = '0' and s0 = '1' else
          i2 after del when s1 = '1' and s0 = '0' else
          i3 after del when s1 = '1' and s0 = '1';
  end mux4_behavior;

```

(a)

**FIGURE 6.42**

Behavioral model for a 4-to-1 MUX. (a) Entity declaration and behavioral description. (b) Logic symbol.

the logic and the environment and, therefore, is not given a specific value for any of the behavioral events. Again, as in Fig. 6.41, the VHDL keywords are highlighted as a visual effect for the reader. The logic symbol for the 4-to-1 MUX is given in Fig. 6.42b and indicates four data inputs (i3, i2, i1, i0), two data select inputs (s1 and s0), and a single output, o1, all active high.

Notice that the VHDL language grammar is to some extent intuitive. For example, **group sel is** (s1, s0) identifies a collection of named entities (s1, s0) as belonging to the group name “sel.” Or, the third line under **architecture / begin** has the following meaning: Output o1 is assigned the value i2 after an arbitrary delay when the select inputs, s1 and s0, are 1 and 0, respectively, or else. . . . The behavioral model in Fig. 6.42 is but one of several VHDL description formats that could be used. The reader should experiment with others to gain experience in behavioral modeling.

The complete VHDL gate-level description of the 1-bit comparator is next offered as the final example in this chapter. In Figs. 6.43a and 6.43b are given the truth table and logic circuit symbol for the 1-bit comparator. The K-maps and gate-minimum cover for the bit comparator were given previously in Fig. 6.27b, resulting in the output expressions given by Eqs. (6.20). By using the factorization method presented in Subsection 4.9.1, Eqs. (6.20) are converted to two-level minimum form as follows:

$$\begin{aligned}
 a\_gt\_b &= gt(a \odot b) + a\bar{b} = gta + gt\bar{b} + a\bar{b} \\
 a\_eq\_b &= eq(a \odot b) = eq\bar{a}\bar{b} + eqab \\
 a\_lt\_b &= lt(a \odot b) + \bar{a}b = lt\bar{a} + lt\bar{b} + \bar{a}b.
 \end{aligned}
 \tag{6.25}$$

Here, *gt*, *eq*, and *lt* have the same meaning as is used in Section 6.6.

The logic circuit representing Eqs. (6.25) is shown in Fig. 6.43c, where the gate numbers and intermediate output functions, *im1*, *im2*, *im3*, . . . are specified for each inverter and NAND gate. This is done for tracking purposes during the VHDL description that follows.

The VHDL gate-level description of the bit-comparator is divided into three parts: entity declaration, *behavioral* description, and the *structural* description. The average primitive

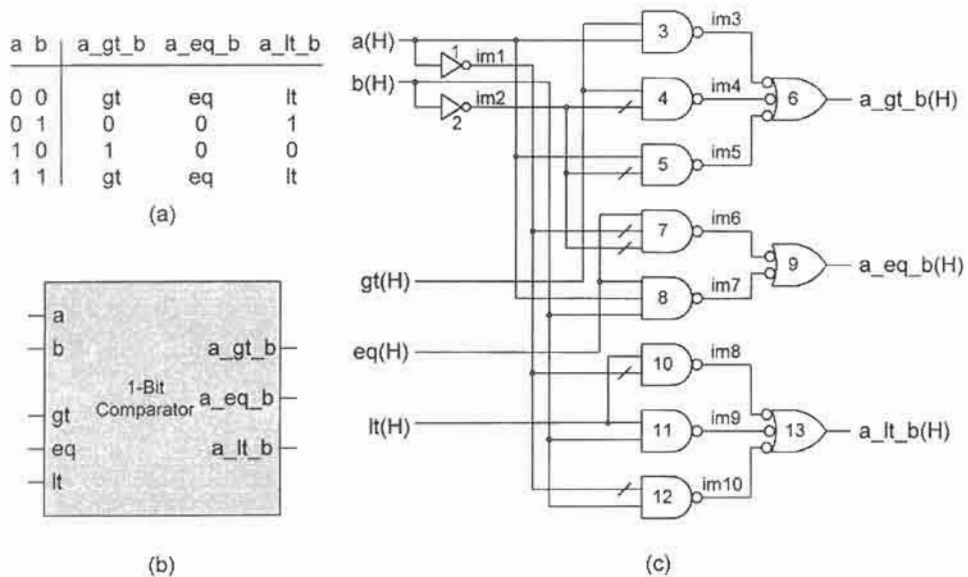


FIGURE 6.43

Design of the cascadable bit-comparator. (a) Truth table. (b) Logic circuit symbol. (c) Circuit diagram in NAND/INV logic according to Eqs. (6.25).

propagation delay, denoted as *avg\_delay*, is computed in each *gate model* at the end of the VHDL description by using  $(t_{plh} + t_{phl})/2$ , which is Eq. (6.1) discussed in Subsection 6.1.3. The propagation delays are defined in Fig. 6.2. As in the two previous examples, the VHDL keywords and logic operators are indicated in bold for visual effect. Two new keywords have been added: **generic map** associates constants within a portion of the VHDL description to constants defined outside that portion; **port map** associates port signals within a portion of the VHDL description to ports outside of that portion.

The following is the complete gate-level VHDL description of the 1-bit comparator given the name *bit\_compare*:

```
entity bit_compare is
    generic (tplh1, tphl1, tplh2, tphl2, tplh3, tphl3: time);
    port (a, b, gt, eq, lt: in bit; a_gt_b, a_eq_b, a_lt_b: out bit);
end bit_compare;

architecture behave_comp of bit_compare is
begin
    a_gt_b <= '1' when a > b else '0';
    a_lt_b <= '1' when a < b else '0';
    a_eq_b <= gt if gt <= '1' else
               eq if eq <= '1' else
               lt if lt <= '1';
end behave_comp;
```

**architecture** structure\_comp of bit\_compare is

**component** inv

**generic** (tplh, tphl: time); **port** (i1: in bit; o1: out bit);

**end component**;

**component** nand2

**generic** (tplh, tphl: time); **port** (i1, i2: in bit; o1: out bit);

**end component**;

**component** nand3

**generic** (tplh, tphl: time); **port** (i1, i2, i3: in bit; o1: out bit);

**end component**;

**for all:** inv **use entity** avg\_delay\_inv;

**for all:** nand2 **use entity** avg\_delay\_nand2;

**for all:** nand3 **use entity** avg\_delay\_nand3;

-- Intermediate signals must be declared:

**signal** im1, im2, im3, im4, im5, im6, im7, im8, im9, im10: bit;

**begin**

a\_gt\_b output

gate1: inv **generic map** (tplh, tphl) **port map** (a, im1);

gate2: inv **generic map** (tplh, tphl) **port map** (b, im2);

gate3: nand2 **generic map** (tplh, tphl) **port map** (gt, a, im3);

gate4: nand2 **generic map** (tplh, tphl) **port map** (gt, im2, im4);

gate5: nand2 **generic map** (tplh, tphl) **port map** (a, im2, im5);

gate6: nand3 **generic map** (tplh, tphl) **port map** (im3, im4, im5, a\_gt\_b);

a\_eq\_b output

gate7: nand3 **generic map** (tplh, tphl) **port map** (eq, im1, im2, im6);

gate8: nand3 **generic map** (tplh, tphl) **port map** (eq, a, b, im7);

gate9: nand2 **generic map** (tplh, tphl) **port map** (im6, im7, a\_eq\_b);

a\_lt\_b output

gate10: nand2 **generic map** (tplh, tphl) **port map** (lt, im1, im8);

gate11: nand2 **generic map** (tplh, tphl) **port map** (lt, b, im9);

gate12: nand2 **generic map** (tplh, tphl) **port map** (im1, b, im10);

gate13: nand3 **generic map** (tplh, tphl) **port map** (im8, im9, im10, a\_lt\_b);

**end** structure\_comp;

The following are the gate model descriptions for inv, nand2 and nand3:

**entity** inv is

**generic** (tplh: time := 4ns; tphl: time := 2 ns);

**port** (i1: in bit; o1: out bit);

**end** inv;

```

architecture avg_delay_inv of inv is
begin
    o1 <= not i1 after (tphl + tphl)/2;
end avg_delay1;

entity nand2 is
    generic (tphl: time := 6ns; tphl: time := 4 ns);
    port (i1, i2: in bit; o1: out bit);
end nand2;

architecture avg_delay_nand2 of nand2 is
begin
    o1 <= i1 nand i2 after (tphl + tphl)/2;
end avg_delay2;

entity nand3 is
    generic (tphl: time := 7ns; tphl: time := 5 ns);
    port (i1, i2, i3: in bit; o1: out bit);
end nand3;

architecture avg_delay_nand3 of nand3 is
begin
    o1 <= not (i1 and i2 and i3) after (tphl + tphl)/2;
end avg_delay3;

```

In the preceding examples VHDL syntax has been applied sometimes without comment. There are relatively few syntax rules that need be followed to create proper VHDL descriptions of devices. The following are some of the more important examples of these syntax rules:

- VHDL is *not* case sensitive. Upper- or lowercase characters can be used as desired.
- Identifiers must begin with a letter and subsequent characters must be alphanumeric but may contain the underscore “\_”. For example, in Fig. 6.42 the identifiers are mux4 and mux4\_behavior.
- The semicolon “;” is used to indicate the termination of a statement. For example: “**end** nand3;”.
- Two dashes “--” are used to indicate the beginning of a *comment*. A comment in VHDL is not read by the compiler but serves as a message or reminder to the reader.

An interesting and useful feature of VHDL is that it supports what is called *operator overloading*. This feature permits custom operations to be defined. The following example illustrates how operator overloading can be used to define a new data type:

```

function “and” (l,r: std_logic_1164) return UX01 is

```

**begin**

    return((and\_table)(l,r));

**end** "and";

·  
·  
·

**architecture** example of and\_operation is

**signal** Y,A,B: std\_logic\_1164;

**begin**

    Y <= A and B;

**end** example;

Here, std\_logic\_1164 refers to an IEEE standard logic package within which UX01 is a subtype for 4-valued logic systems. Thus, the operation "and" takes on the new meaning "and\_table" contained in the standard package. Also, l and r (in line 1) are two of a class of *value kind attributes* that return the leftmost element index (l) or rightmost element index (r) of a given type or subtype.

Several operators, or *constructs* as they are sometimes called, have been used in the examples given previously. Although it is true that the VHDL language contains a large number of these constructs, only a few are necessary for most logic synthesis purposes. The most important of these are given in Fig. 6.44. Included are assignment, relational, logical,

|                                 |                     |      |                       |
|---------------------------------|---------------------|------|-----------------------|
| <u>Assignment Operators:</u>    |                     |      |                       |
| <=                              | Signal assignment   | :=   | Variable assignment   |
| <u>Relational Operators:</u>    |                     |      |                       |
| =                               | Equality            | /=   | Inequality            |
| <                               | Less than           | >    | Greater than          |
|                                 |                     | <=   | Less than or equal    |
|                                 |                     | >=   | Greater than or equal |
| <u>Logical Operators:</u>       |                     |      |                       |
| and                             | AND                 | or   | OR                    |
| xor                             | XOR                 | nand | NAND                  |
|                                 |                     | nor  | NOR                   |
|                                 |                     | not  | Logical negation      |
| <u>Arithmetic Operators:</u>    |                     |      |                       |
| +                               | Addition            | -    | Subtraction           |
|                                 |                     | *    | Multiplication        |
|                                 |                     | /    | Division              |
| <u>Shift Operators:</u>         |                     |      |                       |
| sll                             | Shift left logical  | srl  | Shift right logical   |
| rol                             | Rotate left logical | ror  | Rotate right logical  |
| <u>Miscellaneous Operators:</u> |                     |      |                       |
| **                              | Exponentiation      | abs  | Absolute value        |
|                                 |                     | &    | Concatenation         |

**FIGURE 6.44**

Partial listing of important VHDL operators supported by the IEEE 1076-1993 standard.

| <u>Value</u> | <u>Description</u> | <u>Value</u> | <u>Description</u> |
|--------------|--------------------|--------------|--------------------|
| 'U'          | Uninitialized      | 'X'          | Unknown            |
| '0'          | Logic 0 (driven)   | '1'          | Logic 1 (driven)   |
| 'L'          | Logic 0 (read)     | 'H'          | Logic 1 (read)     |
| '.'          | Don't care         | 'Z'          | High impedance     |

**FIGURE 6.45**

Eight logic data types supported by the IEEE 1076-1164 standard.

arithmetic, shift, and miscellaneous operators. Be aware that some of the operators have different meanings depending on the synthesis tool used.

The IEEE standard 1164 supports standard data types that allow multiple I/O values to be represented. As an example, the standard data type having eight values permit the accurate modeling of a digital circuit during simulation and is presented in Fig. 6.45. The word “driven” used in the description of data type characters ‘0’ and ‘1’ indicates that these logic values are assigned (or forced) to a signal (e.g., an output). The word “read” would apply to input logic values that must be read by a device. Note that each data type character must be enclosed in single quotes as, for example, ‘X’.

VHDL is a large and complex language that is easy to learn at the beginning but difficult to master. It is particularly well suited to the design of very large systems, perhaps more so than any other HDL. Libraries of circuit elements can be easily built, used, and reused in a very effective and efficient manner, and this can be done at different levels of abstraction ranging from the block diagram level to the transistor level. In fact, one of VHDL’s strengths is that it offers nearly unlimited use of reusable components and access to standard libraries such as the built-in IEEE 1076-1993 Standard and the IEEE 1076-1164 Standard. Used in the hands of a skilled designer, VHDL can greatly increase productivity as well as facilitate the move into more advanced tools (for example, simulators) and advanced target systems. Further Reading contains essential references for continued development in VHDL.

## 6.11 FURTHER READING

Most textbooks on digital design cover one or more of the performance characteristics related to digital design. Typical among these are the texts by Comer, McCluskey, Tinder, and Wakerly. The text by Wakerly covers these subjects in considerable detail and considers various logic families. The performance characteristics covering several of the more common logic families can be found in the *Electrical Engineering Handbook* (R. C. Dorf, Editor-in-Chief).

- [1] D. J. Comer, *Digital Logic and State Machine Design*, 3rd ed. Sanders College Publishing, Fort Worth, TX, 1995.
- [2] R. C. Dorf, Editor-in-Chief, *Electrical Engineering Handbook*, 2nd ed., CRC Press, Boca Raton, FL, 1997, pp. 1769–1790.
- [3] E. J. McCluskey, *Logic Design Principles*. Prentice-Hall, Englewood Cliffs, NJ, 1986.
- [4] R. F. Tinder, *Digital Engineering Design: A Modern Approach*. Prentice-Hall, Englewood Cliffs, NJ, 1991.



- [5] J. F. Wakerly, *Digital Design Principles and Practices*, 2nd ed. Prentice-Hall, Englewood Cliffs, NJ, 1994.

The usual combinational logic devices such as MUXs, decoders, code converters, and comparators are covered adequately by most texts, including those just cited, but the texts by Tinder and Wakerly provide what is perhaps the best coverage of the group.

Steering logic seems to be covered adequately in only a few texts, among which are those of Hayes and Katz.

- [6] J. P. Hayes, *Introduction to Digital Logic Design*. Addison Wesley, Reading, MA, 1993.  
 [7] R. H. Katz, *Contemporary Logic Design*. Benjamin/Commings Publishing, Redwood City, CA, 1994.

There are numerous texts and reference books on VHDL. For instructional purposes the texts of Dewey, Navabi, Pellerin and Taylor, Perry, Roth, and Skahill are good choices. The texts by Pellerin and Taylor and by Skahill include CD-ROMs containing fully functional VHDL compilers. The text by Dewey is somewhat unusual in that it nicely combines digital design and analysis with VHDL.

- [8] A. M. Dewey, *Analysis and Design of Digital Systems with VHDL*. PWS Publishint Co., Boston, 1997.  
 [9] Z. Navabi, *VHDL Analysis and Modeling of Digital Systems*. McGraw-Hill, New York, 1993.  
 [10] D. Pellerin and D. Taylor, *VHDL Made Easy*. Prentice Hall PTR, Upper Shaddle River, NJ, 1997.  
 [11] P. J. Ashenden, *The Designer's Guide to VHDL*, Morgan Kaufmann Publishers, San Francisco, CA, 1996.  
 [12] C. H. Roth, Jr., *Digital Systems Design Using VHDL*. PWS Publishing Co., Boston, 1998.  
 [13] K. Skahill, *VHDL for Programmable Logic*. Addison-Wesley, Reading, MA, 1996.

The latest VHDL IEEE standard is the 1076-1993 standard. Standard 1076 has been augmented by standards 1164, 1076.3 and 1076.4. These latest standards are identified as follows:

Standard 1076-1993, *IEEE Standard VHDL Language Reference Manual*, IEEE, 1994.

Standard 1164-1993, *IEEE Standard Multivalue Logic System for VHDL Model Interoperability*, IEEE, 1993.

Standard 1076.3, *VHDL Synthesis Packages*, IEEE, 1995.

Standard 1076.4, *VITAL ASIC Modeling Specification*, IEEE, 1995.

These IEEE documents can be obtained from IEEE at the following address: IEEE Service Center, 445 Hoes Lane, PO Box 1331, Piscataway, NJ 08855-1331 (Phone: 1-800-678-IEEE).

## PROBLEMS

- 6.1** The propagation delays for a state-of-the-art CMOS logic gate are calculated to be  $\tau_{plh} = 0.25$  ns and  $\tau_{phl} = 0.35$  ns with a power dissipation of 0.47 mW. Calculate the power-delay product (PDP) in picojoules predicted for this gate.

- 6.2** The voltage parameters for a high-speed CMOS (HC) gate are measured to be  $V_{IL\max} = 0.23V_{DD}$  and  $V_{IH\min} = 0.59V_{DD}$  with  $V_{OL\max} = 0.08V_{DD}$  and  $V_{OH\min} = 0.90V_{DD}$  for a supply of 2.8 V.
- Calculate the noise margins for this gate.
  - Use the values calculated in part (a) to explain what they mean in relationship to the interpretation of logic 1 and logic 0 for this gate.
- 6.3** Construct a logic circuit that combines two 16-to-1 MUXs to form a 32-to-1 MUX. (Hint: Use an inverter to select the appropriate MUX.)
- 6.4** Use an 8-to-1 MUX to implement each of the following functions, assuming that all inputs and outputs are active high.
- $W(A, B, C) = \sum m(1, 2, 4, 5, 6)$
  - $X(A, B, C) = A\bar{B}C + \bar{A}\bar{B}\bar{C} + AB\bar{C} + ABC + A\bar{B}\bar{C}$
  - $Y(A, B, C) = \prod M(0, 1, 2, 6, 7)$
  - $Z(A, B, C) = (A + B) \odot (\bar{A}C) + A\bar{B}$
- 6.5** Repeat Problem 6.4 but instead use a 4-to-1 MUX to implement each function. To do this use *minimum* external logic and the two most significant inputs as the data select variables.
- 6.6** Repeat Problem 6.5 assuming that only input  $B$  arrives from a negative logic source and that one inverter is permitted to be used on a data select input.
- 6.7** Use a 4-to-1 MUX to implement each of the following functions, assuming that all inputs and outputs are active high. It is required that minimum external logic to the MUX be used in each case, and that the data select inputs be  $A$  and  $B$ .
- $U(A, B, C, D) = \sum m(0, 4, 5, 7, 8, 9, 13, 15)$
  - $V(A, B, C, D) = \prod M(0, 5, 8, 9, 11, 12, 15)$
  - $W(A, B, C, D) = \sum m(4, 5, 7, 12, 14, 15) + \phi(3, 8, 10)$
  - $X(A, B, C, D) = \prod M(0, 1, 2, 5, 7, 9) \cdot \phi(4, 6, 10, 13)$
- 6.8** Implement the following function by using the hardware indicated (nothing else). Assume that the inputs arrive as  $A(H)$ ,  $B(L)$ ,  $C(H)$ , and  $D(L)$ , and that the output is issued active high.

$$F(A, B, C, D) = \sum m(0, 1, 3, 4, 6, 8, 9, 10, 11, 12, 15)$$

**Permitted hardware:** One 4-to-1 MUX; one NAND gate; one XOR gate.

- 6.9** Implement the following function by using the hardware indicated (nothing else). Assume that the inputs arrive as  $A(L)$ ,  $B(H)$ ,  $C(H)$ , and  $D(H)$ , that the output is issued active high. (Hint: First find the absolute minimum expression for  $Z$ .)

$$Z = \bar{A}\bar{C}\bar{D} + \bar{A}\bar{B}\bar{C}D + (\bar{A} + B)C\bar{D} + (A \odot B)CD$$

**Permitted hardware:** One 2-to-1 MUX; one NAND gate; one AND gate.

- 6.10** (a) Configure a 6-to-64 decoder by using only 3-to-8 decoders.  
 (b) Configure a 6-to-64 decoder by using only 4-to-16 and 2-to-4 decoders.

- 6.11** Implement the function in Problem 4.28 by using a 16-to-1 MUX assuming that all inputs and the output are active high.
- 6.12** Repeat Problem 6.11 if only the  $B$  input is active low and *no* additional hardware (e.g., an inverter) is permitted.
- 6.13** Design a bitwise logic function generator that will generate any of the 16 possible logic functions. End with a single expression  $F$  that represents the 16 bitwise logic functions. To do this use a 4-to-1 MUX and nothing else. (Hint: Interchange the names for the data and data-select inputs to the MUX.)
- 6.14** Implement each function in Problem 6.4 by using a 3-to-8 decoder and the necessary external hardware, assuming that all inputs and outputs are active high.
- 6.15** Implement function  $F$  in Problem 6.8 by using a 4-to-16 decoder, one OR gate, and two NAND gates (maximum fan-in of 6), taking the input activation levels as given in Problem 6.8.
- 6.16** Repeat Problem 6.15 by replacing the 4-to-16 decoder with two 3-to-8 decoders and one inverter.
- 6.17** The function below is to have inputs that arrive as  $A(H)$ ,  $B(L)$ , and  $C(H)$ , with an output  $F(L)$ .

$$F(A, B, C) = \sum m(0, 1, 6, 7)$$

- (a) Implement this function by using a 3-to-8 decoder and one NAND gate (nothing else). Assume that the decoder has active low outputs. (Hint: Use the AND form of the two conjugate NAND gate circuit symbols to meet the requirement of an active low output.)
- (b) Repeat part (a) by using two 2-to-4 decoders, a NAND gate, and one inverter (nothing else).
- 6.18** The circuit shown in Fig. P6.1 connects a decoder to a MUX. Analyze this circuit by finding  $Y(H)$  in terms of inputs  $A$ ,  $B$ ,  $C$ , and  $D$ .

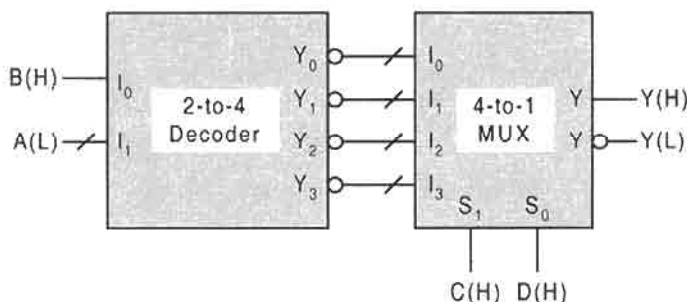


FIGURE P6.1

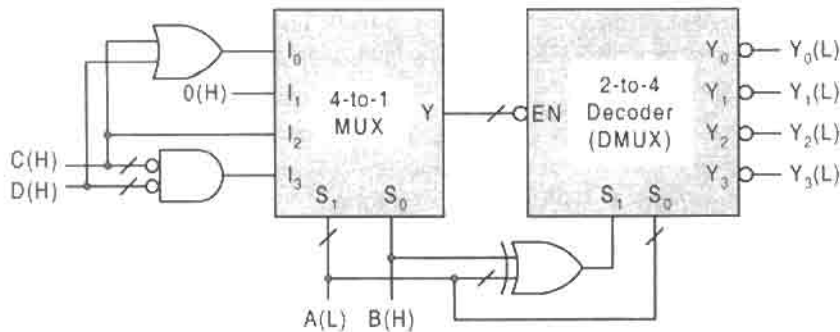


FIGURE P6.2

- 6.19** Analyze the network in Fig. P6.2 by obtaining the output logic expressions for  $Y_0(L)$ ,  $Y_1(L)$ ,  $Y_2(L)$ , and  $Y_3(L)$  in terms of inputs  $A$ ,  $B$ ,  $C$ , and  $D$ . Assume that the DMUX is enabled if  $EN(L) = 1(L)$ .
- 6.20** Design an 8-input ( $I_7$ – $I_0$ ), noncascadable priority encoder (PE) that will issue all  $2^n$  binary states. Thus, include an  $EN$  input but do not include the cascading bits  $EI$ ,  $EO$ , and  $GS$ .
- Construct the collapsed priority schedule table for this PE by using  $X$  as the irrelevant input symbol. Assume that the logic value  $EN = 1$  enables the PE. (Hint: There are three outputs.)
  - Map the output functions into fourth-order EV K-maps with axes  $I_7$ ,  $I_6$ ,  $I_5$ ,  $I_4$  and extract minimum SOP expressions for these functions.
  - Implement the results of part (b) by using NAND/INV logic assuming that all inputs and outputs are active low.
- 6.21** Repeat Problem 6.20 but with cascading bits  $EI$ ,  $EO$ , and  $GS$ , all of which must be active low.
- 6.22** Use NOR/INV logic to design a noncascadable three-input priority encoder that will operate according to the following priority schedule:

Input  $C$  — Highest priority encoded as 10  
 Input  $A$  — Middle priority encoded as 01  
 Input  $B$  — Lowest priority encoded as 00  
 Assign 11 to the inactive state.

Assume that all inputs arrive active high and that the outputs are issued active low.

- 6.23** Design a Gray-to-BCD code converter by using four 4-to-1 MUXs and a gate-minimum external logic. The inputs arrive as  $A(H)$ ,  $B(L)$ ,  $C(H)$ , and  $D(H)$ , and the outputs are all active high.
- 6.24** Use three XOR gates and two OR gates (nothing else) to design a gate-minimum circuit for a 4-bit binary-to-2's complement (or vice versa) converter. Let  $B_3$ ,  $B_2$ ,  $B_1$ ,  $B_0$  and  $T_3$ ,  $T_2$ ,  $T_1$ ,  $T_0$  represent the 4-bit binary and 2's complement words, respectively,

and assume that all inputs and outputs are active high. (Hint: Use second-order EV  $K$ -maps, constructed from the truth table, and XOR patterns. Factor  $B_3$  from the SOP minimum expression and then apply the defining relation for XOR.)

- 6.25** Design a gate-minimum circuit for a BCD-to-creeping code converter assuming that all inputs and outputs are active high and that the inputs are restricted to the ten BCD states. To do this refer to Table 2.10 and plan to use second-order EV  $K$ -maps for convenience.
- 6.26** (a) Design an 8-bit binary-to-BCD converter based on Algorithm 6.2 in Subsection 6.5.2. To do this, first design the minimum NAND/INV logic for the converter module required to convert to BCD the binary numbers equivalent of decimal 0 through 19. Note that the shift-left/add-3 algorithm is inherent in the truth table for the converter module and that the LSB in binary is the same as the LSB in BCD. Next, cascade the modules as in Fig. P6.3 to carry out Algorithm 6.2 for the 8-bit converter. All inputs and outputs are assumed to be active high.
- (b) Use the converter module of part (a) together with Fig. P6.3 to find the BCD for a binary number equivalent to  $159_{10}$ .
- 6.27** Analyze the logic circuit for the BCD-to-seven-segment decoder in Fig. 6.24 by constructing a mixed-logic truth table for active high binary inputs equivalent to decimals  $2_{10}$  and  $9_{10}$ . Thus, the seven outputs must all be active low, suitable to drive the common cathode LED display in Fig. 6.23b.
- 6.28** (a) Following Fig. 6.28a, construct the truth table for a cascadable 3-bit comparator, but do not map it.
- (b) By using only Eqs. (6.23), write the three output expressions for the 3-bit comparator.

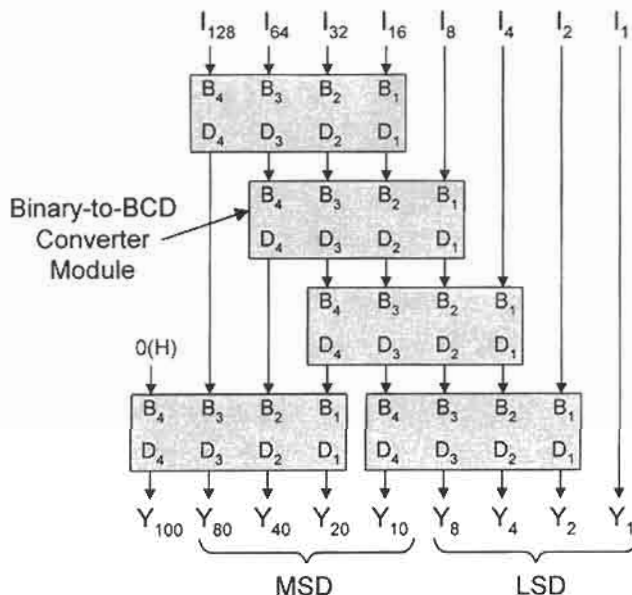


FIGURE P6.3

- (c) Implement the results of part (b) by making use of Eq. (6.22). Is an *eq* input necessary?
- 6.29** Given the block symbol for a 4-bit cascadable comparator in Fig. 6.30 and with the appropriate gate-minimum NAND/EQV/INV external logic, design a 5-bit cascadable comparator. (Hint: Use the 1-bit comparator as the output (MSB) stage.)
- 6.30** Design a 4-bit even-parity detector with respect to logic 1 by using only XOR gates (nothing else). Show how this result can be used to produce a 4-bit odd-parity detector without adding additional hardware.
- 6.31** (a) Design a logic circuit that will detect a majority of seven inputs *A*, *B*, *C*, *D*, *E*, *F*, and *G* that are active at any one time, and this under the following conditions: Circuits 1 and 2 represent *majority functions* that must each detect a majority of its three inputs that are active. Thus, if any four or more of all seven inputs are active, the output *Majority Detect* will be active; otherwise the output will be inactive. To do this, use the “divide-and-conquer” approach. Referring to Fig. P6.4, construct truth tables for *identical* circuits 1 and 2 such that their outputs are active any time two or three of their inputs are active. Inputs *A* and *D* are the MSB inputs for each of the two circuits. Next, map each majority function (actually, one will do) from the truth tables and extract a gate-minimum cover. Finally, introduce the input *G* = *Z* into the logic for circuit 3 such that the output, *Majority Detect*, is active iff the input conditions are met. End with a gate-minimum logic circuit that will contain XOR functions.
- (Hints: Properly done, the output *Majority Detect* can be obtained directly without the use of a truth table. Note that if *G* is inactive, the output of circuit 3 can detect a majority of 4, 5, or 6 active inputs. However, with input *G* active, the output of circuit 3 can detect a majority of only 5 or 7 active inputs. To obtain a gate-minimum circuit for *Majority Detect* (seven gates and one inverter for Circuit 3, 15 total), it will be necessary to plot a fourth-order K-map with input *G* as the entered variable.)
- (b) Repeat part (a) but without circuits 1 and 2. Thus, find the optimum two-level logic expression for the output *Majority Detect* with the seven inputs presented directly to circuit 3. To do this, plot a fourth-order EV K-map with EVs *E*, *F*, and *G* (a third-order compression with a Map Key of 8), then use the logic minimizer (e.g., BOOZER software bundled with this text) to obtain the result. Do not

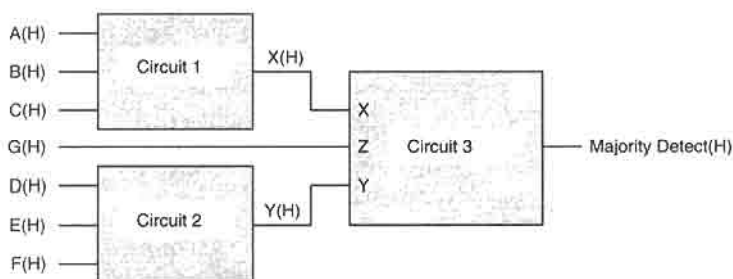


FIGURE P6.4

implement the function. [Hint: The easiest approach is to count the number of inputs ( $A$ ,  $B$ ,  $C$ , and  $D$ ) that are active for each cell of the fourth-order K-map. Then enter the three EVs (in minimum subfunction form with the aid of a third-order K-map) in those cells as needed to bring the count of active inputs to four or more. If this is correctly done, 35 EPIs and no inverters will be required.]

- (c) From the K-map in part (b), extract a gate-minimum expression by using XOR patterns. If this is correctly done, the gate count will be reduced to 18 (a four-level function) with no inverters required.

**6.32** Design a 4-bit general shifter that can shift or rotate, right or left, up to three bits with  $F$  fill. To do this, do the following:

- Construct the truth table for this shifter.
- Use the truth table of part (a) to plot the fourth-order K-map for each of the four outputs.
- From the results of part (b), write the expression for each output in a form suitable for a 4-to-1 MUX implementation by taking the data select inputs as  $A_1$  and  $A_0$ . Do not implement.

**6.33** Find the gate-minimum external logic for a 4-bit shifter that will cause it to operate according to the following table. To do this, make use of XOR patterns. Show the resulting logic circuit required to be connected to the shifter assuming that all inputs and outputs are active high.

| <i>Shifter input</i> | <i>Action</i>                      |
|----------------------|------------------------------------|
| Even 1 parity        | Shift right 1 position with 0 fill |
| Odd 1 parity         | Shift left 2 positions with 1 fill |
| 1111 (Exception)     | Transfer                           |

(Hint: It is only necessary to know the logic for  $F$ ,  $D$ ,  $A_1$ , and  $A_0$ , which are the outputs for the truth table. Properly done, the minimum external circuitry will consist of three XOR gates, two NAND gates, and two inverters. Note that maxterm code must be used to extract  $A_0$  from the K-map).

- 6.34** Combine two 4-to-1 steering logic MUXs, shown in Fig. 6.39b, to produce an 8-to-1 line MUX. To do this you are permitted to use a single additional CMOS inverter and a CMOS OR gate.
- 6.35** Write the VHDL behavioral description for the majority function  $F = AB + BC + AC$ . To do this, use the keywords that are defined in Section 6.10.
- 6.36** Write the VHDL behavioral and structural descriptions for a 2-to-4 decoder with active high outputs. To do this follow the examples in Section 6.10.
- 6.37** At the discretion of the instructor, write the VHDL behavioral description for any device featured in this chapter. Note that more than one correct approach may exist in writing the behavioral description of a given device.
- 6.38** At the discretion of the instructor, write the VHDL structural description of any device for which Problem 6.37 has been assigned. Note that correct approaches often vary in writing structural descriptions of devices.

# Programmable Logic Devices

---

## 7.1 INTRODUCTION

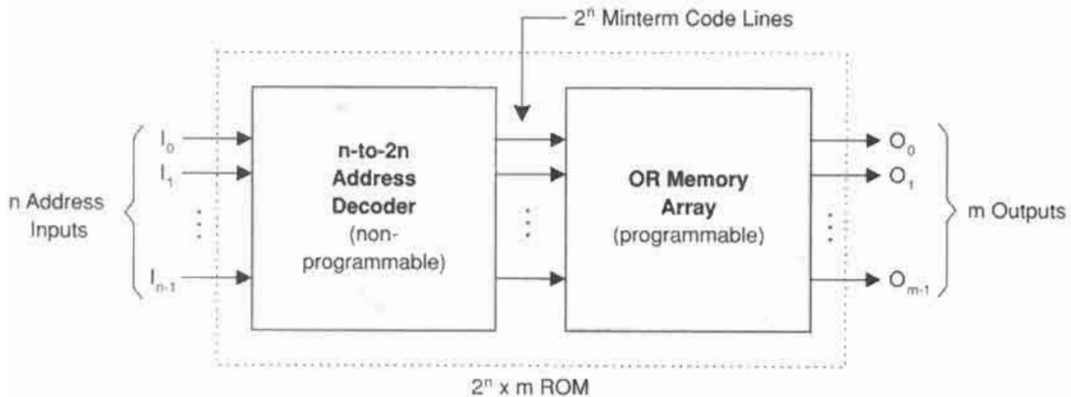
A class of devices called *programmable logic devices* (or PLDs) can be thought of as universal logic implementers in the sense that they can be configured (actually programmed) by the user to perform a variety of specific logic functions. So useful and versatile are these PLDs that one might question why any other means of design would ever be considered. Well, the answer is, of course, that there is a time and place for a variety of approaches to design — that is, no one single approach to design satisfies all possible problem situations. However, the option to use PLDs offers the logic designer a wide range of versatile devices that are commercially available for design purposes.

Some PLDs are made to perform only combinational logic functions; others can perform both combinational and sequential logic functions. This chapter will consider those PLDs capable of performing both combinational and sequential logic functions, but the exemplar applications will be limited to combinational logic design. Four commonly used PLDs considered here are the *read-only memory* (ROM) devices and their subgroups, the *field programmable logic array* (FPLA) devices, the *programmable array logic* (PAL) devices and their subgroups, and *field programmable gate arrays* (FPGAs) and subgroups. Other PLDs include *erasable programmable logic devices* (EPLDs), including *erasable programmable ROMs*, *generic array logic* (GAL) devices, and *field programmable logic sequencers* (FPLSs). Except for FPGAs, most of the PLDs mentioned have some commonality, namely a two-level AND/OR configuration. What is connected to the AND/OR network distinguishes one PLD from another. The development that follows attempts to illustrate the differences between these various PLDs and to provide a few useful examples of their application to combinational logic design.

## 7.2 READ-ONLY MEMORIES

A ROM is an  $n$ -input/ $m$ -output device composed of a nonprogrammable decoder (ANDing) stage and a programmable (OR) stage as illustrated in Fig. 7.1. Bit combinations of the  $n$



**FIGURE 7.1**

Block diagram showing the fixed address decoder (AND) stage and the programmable OR memory stage for a ROM of size  $2^n$  words  $\times$   $m$  bits.

input variables are called *addresses*, and in ROMs there are  $2^n$  minterm code lines, each representing a coded minterm on the output side of the decoder stage. Therefore, since there are  $n$  possible addresses to a ROM, there are  $2^n$  possible *words* that can be stored in the ROM, each word being  $m$  bits in size. Any  $m$ -bit output word programmed into the ROM can be selected by the appropriate input address and is nonvolatile — it is stored permanently in the ROM.

The dimensions and size of an  $n$ -input/ $m$ -output ROM are given by

$$\underbrace{2^n \times m}_{\text{Dimensions}} = \underbrace{(2^n)(m) \text{ bits}}_{\text{Size}},$$

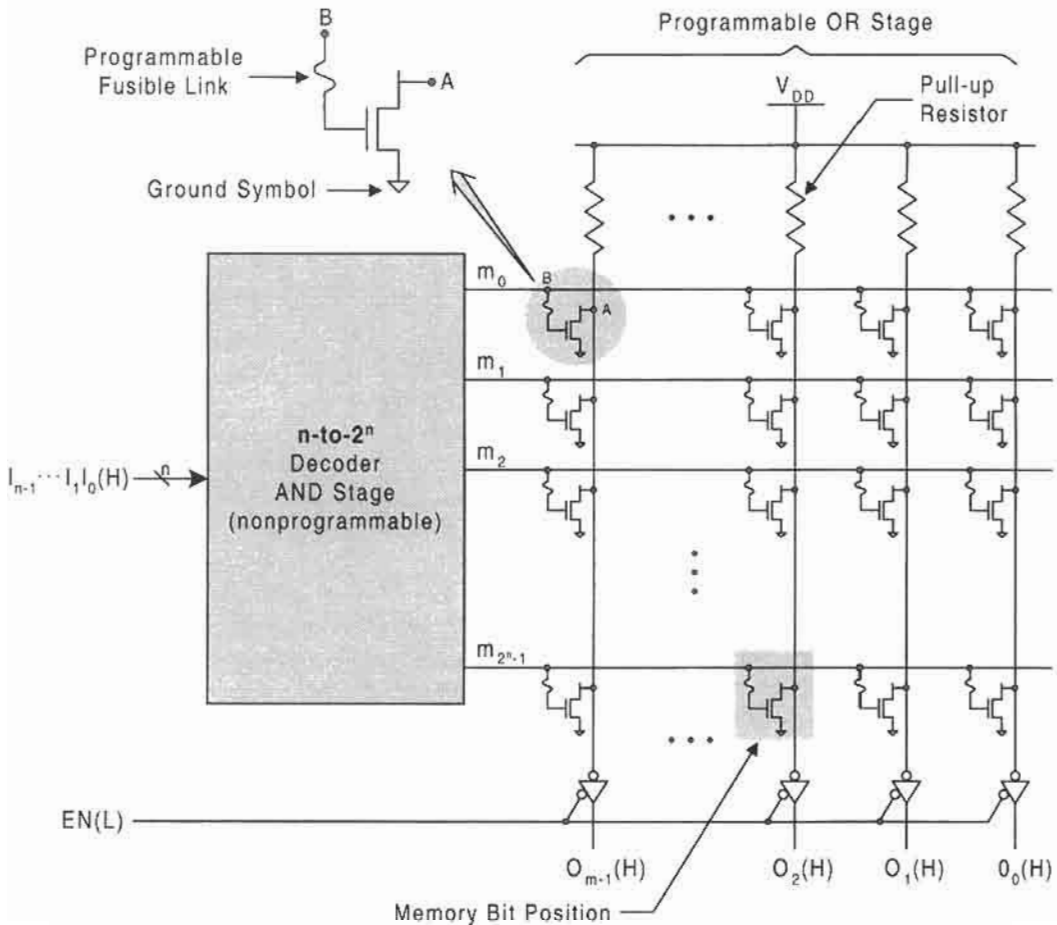
meaning that  $2^n$  words, each of  $m$  bits, produce a total of  $(2^n)(m)$  bits. The size of a ROM may be rounded off to the nearest integer power of 2 in K ( $10^3$ ) bits of ROM. For example, an 8-input/4-output ROM is represented as a  $2^8 \times 4 = 256 \times 4$  bit = 1,024 bit or 1Kbit ROM. The problem with the bit-roundoff form of representation is that knowledge of the dimensions is lost. Thus, a 1Kbit ROM could be any of the following:  $2^7 \times 8 = 2^6 \times 16$ , etc. This can be avoided by giving the ROM size in dimension ( $2^n \times m$ ) form, which clearly specifies the number of inputs and outputs.

ROMs may differ in a variety of ways, but the main differences center about the manner in which they are programmed, and on whether or not they can be erased and reprogrammed and how this is accomplished. Members of the ROM family of PLDs may be divided into three main categories:

- Read-only memories (ROMs) — Mask programmable OR stage only
- Programmable ROMs (PROMs) — User programmable once
- Erasable PROMs (EPROMs) — User erasable many times

Mask-programmable ROMs are programmed during the fabrication process by selectively including or omitting the switching elements (transistors or diodes) that form the memory array stage of the ROM. Because the masking process is expensive, the use of mask-programmable ROMs is economically justifiable only if large numbers are produced to perform the same function.

When one or a few ROM-type devices are needed to perform certain functions, PROMs can be very useful. Most PROMs are fabricated with *fusible links* on all transistors (or diodes) in the OR memory stage, thereby permitting user programming of the ROM — a write-once capability. Shown in Fig. 7.2 is the circuit for an unprogrammed  $2^n \times m$  PROM



**FIGURE 7.2**

Logic circuit for an unprogrammed  $2^n \times m$  PROM showing the nonprogrammable decoder (AND) section and programmable NMOS connections (fusible links) for each normally active bit location in the OR (memory) section. Tri-state drivers provide an enable capability.

consisting of a nonprogrammable decoder (AND) section and NMOS programmable connections (fusible links) on every memory bit location to form the programmable OR section. Also shown are inverting tri-state drivers (see Fig. 3.8d) on the PROM outputs to provide an active low enable control capability. The PROM chip is produced with all transistors “connected” such that each memory bit position is normally active high. Thus, when a decoder output line becomes active, all connected NMOS are turned ON, pulling those bit positions to 0(H), resulting in a 1(H) from each tri-state driver. A bit position is programmed when a fusible link is “blown,” disconnecting that bit position. Disconnected bit positions cannot be pulled low and therefore, must remain at 1(H). If all bit positions on an OR column are disconnected by programming, the output is 0(H) from the tri-state driver. Or, if one or more OR column bit positions are left connected, the output will be a 1(H) if a decoder line to one of those bit positions becomes active — the OR function. The PROM of Fig. 7.2 is programmed (one time only permitted) with a PROM programmer by applying voltage pulses to target fusible links, causing disconnection of these bit positions.

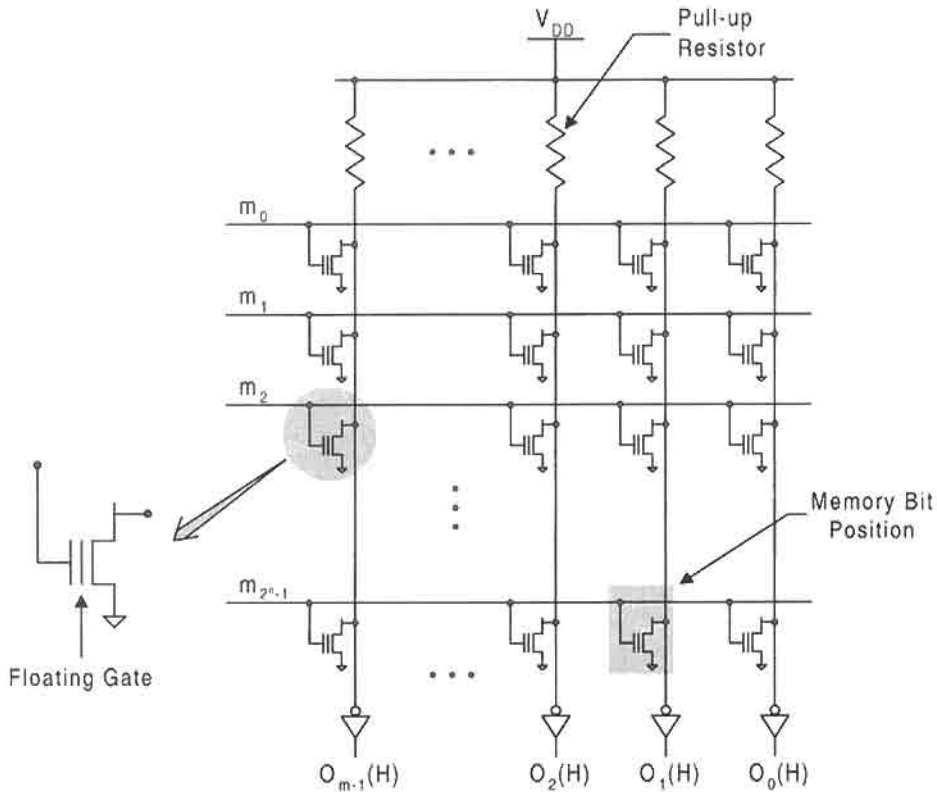
The masking process of a mask-programmable ROM places NMOS connections at pre-determined (programmed) memory bit positions. The positioned NMOS connections would look similar to those in Fig. 7.2, except their fusible links would be missing. Because the masking process is expensive, mask-programmable ROMs are used only for high-volume commercial applications.

Much more useful, generally, are the EPROMs, since they can be programmed, erased, and reprogrammed many times. These devices fall into two main categories: *ultraviolet erasable PROMs* (UVEPROMs) and *electrically erasable PROMs* (EEPROMs). In either case the technology is similar — use is made of *floating-gate NMOS transistors* at each memory bit location, as illustrated by the OR memory stage in Fig. 7.3.

Each transistor in Fig. 7.3 has two gates, a connected outer gate and an inner floating (unconnected) gate that is surrounded by a highly insulating material. Programming occurs when a high positive voltage is applied to the connected gate inducing a negative charge on the floating gate which remains after the high voltage is removed. Then, when a decoder line becomes active (HV), the negative charge prevents the NMOS from being turned ON, thereby maintaining a 1(H) at the memory bit position. This is equivalent to blowing a fusible link in Fig. 7.2. If all floating-gate NMOS in an OR column are so programmed, the output from the inverter is 0(H). But if a decoder line is active to any unprogrammed floating-gate NMOS, that bit position will be pulled to ground 0(H), causing the output to be 1(H) from the inverter — again, the OR function.

Erasure of a programmed floating-gate NMOS occurs by removing the negative charge on its floating gate. This charge can remain on the gate nearly indefinitely, but if the floating gate in a UVEPROM is exposed (through a “window”) to ultraviolet light of a certain frequency, the negative charge on the floating gate is removed and erasure occurs. Similarly, if a voltage of negative potential is applied to the outer connected gate of an EEPROM, removal of the negative charge occurs. The technology for the UVEPROMs and EEPROMs differ somewhat as to the manner in which the floating gate is insulated; otherwise, they share much in common.

Technologies other than those just described are used to manufacture PROMs. For example, in *bipolar* PROMs, diodes with fusible links replace the NMOS in Fig. 7.2 with each diode conducting in the  $A \rightarrow B$  direction (see blowup of the fusible link). Now, however,

**FIGURE 7.3**

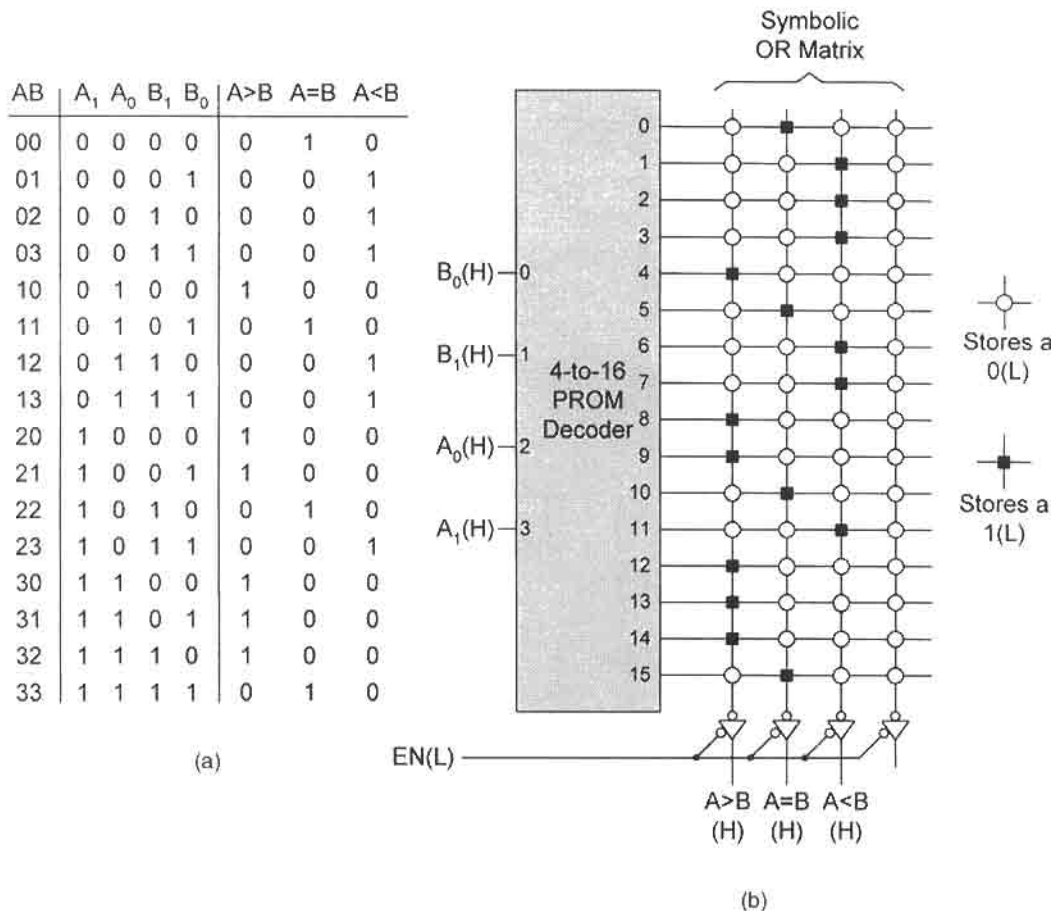
Logic circuit for an unprogrammed OR (memory) section of an EPROM illustrating the floating-gate NMOS transistor technology required for a program/erase/program cycle.

the decoder has active low outputs. Then, when a decoder output becomes active,  $1(L) = 0(H)$ , a bit position for a connected diode is pulled low,  $0(H)$ . A disconnected (programmed) diode maintains a  $1(H)$  at the bit position when selected. If all diode bit positions in an OR column are disconnected, then the output for that column must be  $0(H)$  from the inverting tri-state driver. However, if one (or more) of the bit position diodes in an OR column is left connected and is selected by an active low decoder line, the output will be  $1(H)$  from the tri-state driver. The diode technology, used extensively in the early stages of ROM development, is now in less common use than MOS technology.

### 7.2.1 PROM Applications

The AND section of any ROM device is a nonprogrammable decoder, and since a decoder is a minterm code generator the following requirement must be met for ROM programming:

*To program a ROM, all input and output data must be represented in canonical form.*

**FIGURE 7.4**

PROM implementation of a noncascadable 2-bit comparator. (a) Canonical truth table. (b) Decoder and symbolic representation of fusible bit position patterns in the OR memory section.

This fact is illustrated by considering the noncascadable 2-bit comparator represented by the canonical truth table in Fig. 7.4a. This truth table has been constructed from Fig. 6.28 by setting  $gt = 1$  when  $A > B$ ,  $eq = 1$  when  $A = B$ , and  $lt = 1$  when  $A < B$ , or by setting these inputs to zero otherwise. Shown in Fig. 7.4b is a  $2^4 \times 4$  PROM that has been programmed to function as the 4-input/3-output noncascadable 2-bit comparator. Here, a symbolic notation is used so as to avoid the need to exhibit the details of the logic circuit, including the output inverters. For consistency with Fig. 7.2, tri-state drivers are added to the output with the understanding that a filled square (■) represents the storage of 1(L) and that an open circle (○) represents the storage of 0(L). Notice that one of the four output lines is left unused.

Programming a PROM that functions as the cascadable 2-bit comparator of Fig. 6.28 would require a PROM of dimensions  $2^7 \times 3$ , a significant increase in hardware. In fact, to do so might seem to be hardware overkill, considering that a 7-to-128 line decoder

section would be required. Recall that the three-level logic circuit for the cascadable 2-bit comparator in Fig. 6.28 has a gate/input tally of only 23/67.

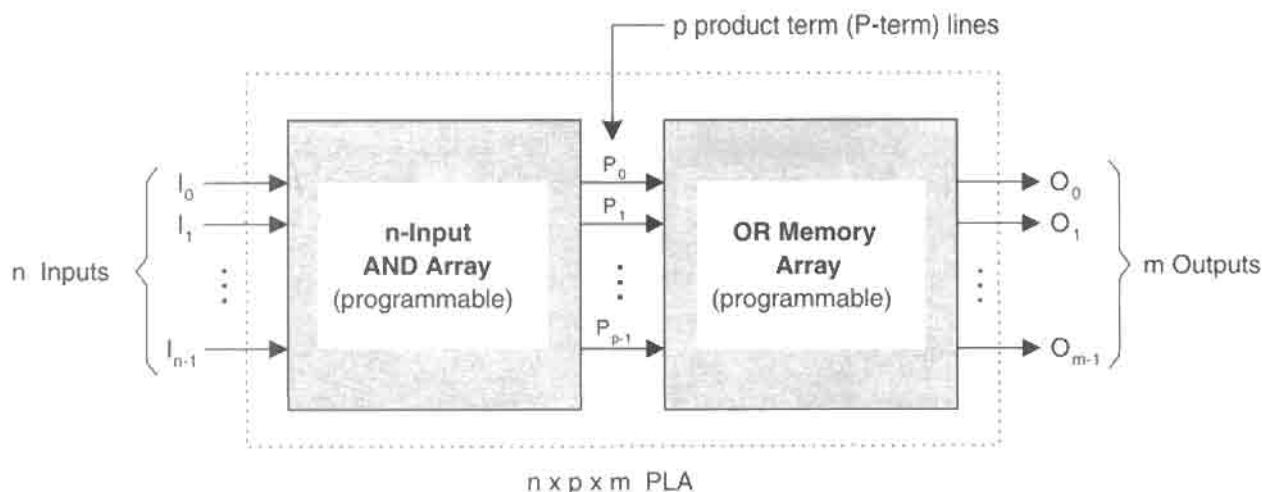
### 7.3 PROGRAMMABLE LOGIC ARRAYS

Like the ROM, the PLA is an  $n$ -input/ $m$ -output device composed of an input ANDing stage and a memory (ORing) output stage. Unlike the ROM, however, both stages of the PLA are programmable, as indicated by the block diagram in Fig. 7.5. The AND matrix (array) generates the product terms (p-terms), while the OR matrix ORs the appropriate product terms together to produce the required SOP functions.

The dimensions of a PLA are specified by using three numbers:

$$\begin{array}{ccccc} n & \times & p & \times & m \\ \uparrow & & \uparrow & & \uparrow \\ \text{No. of} & & \text{No. of} & & \text{No. of} \\ \text{inputs} & & \text{product} & & \text{outputs} \\ & & \text{terms} & & \end{array}$$

The number  $p$  gives the maximum number of product terms (p-terms) permitted by the PLA. The magnitude of  $p$  is set by the PLA manufacturer based on expected user needs and is usually much less than  $2^n$ , the decoder output of a ROM. For example, a PLA specified by dimensions  $16 \times 48 \times 8$  would have 16 possible inputs and could generate 8 different outputs (representing 8 different SOP expressions) composed of up to 48 unique ORed p-terms. A p-term may or may not be a minterm. In contrast, a 16-input ROM could generate the specified number of outputs with up to  $2^{16} = 65,536$  minterms.



**FIGURE 7.5**

Block diagram for an  $n$ -input/ $m$ -output PLA showing the programmable AND and OR array stages and the  $p$  product-term lines.

The comparison here is 48 p-terms and 48 ORing connections for the unprogrammed PLA vs 65,536 minterms and 65,536 ORing connections for the unprogrammed ROM, a vast difference in hardware. Typically, commercial PLAs have inputs ranging in number from 8 to 16, with 20 to 60 addressable p-terms and up to 10 outputs that may or may not have controlled polarity by using XOR gates on the output lines (see Subsection 3.9.6). PLA IC chips of most any  $n \times p \times m$  dimensions can be manufactured to user specifications.

PLAs, like ROMs, are constructed of interconnecting arrays of switching elements that perform the AND and OR operations. Members of the PLA family fall generally into two classes:

- Programmable logic arrays (PLAs) — Mask programmable AND and OR stages
- Field programmable logic arrays (FPLAs) — One-time programmable AND and OR stages

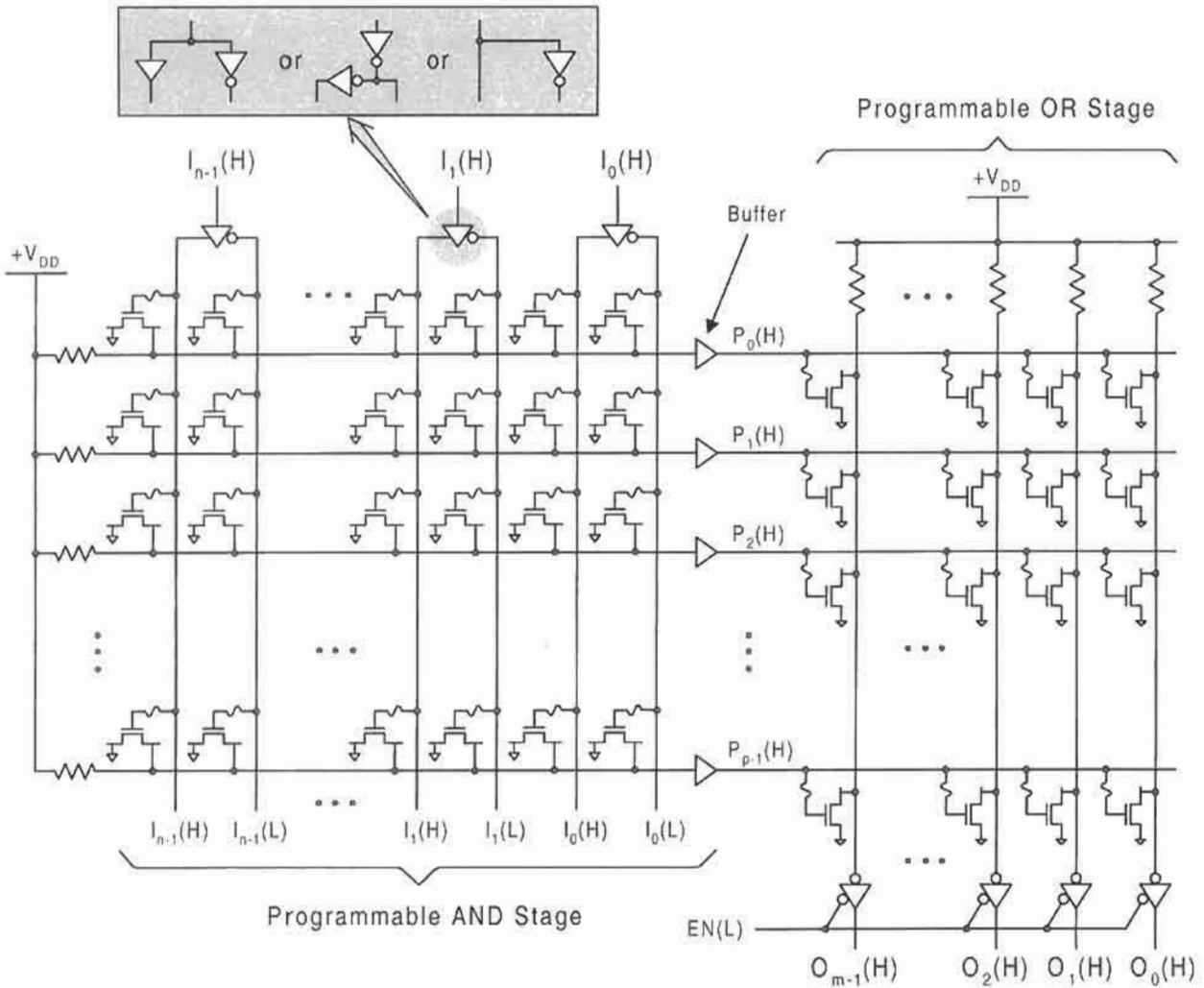
Thus, PLAs are programmed during fabrication in a manner similar to ROMs, while FPLAs are write-once programmed by the user.

Shown in Fig. 7.6 is the MOS version for an unprogrammed  $n \times p \times m$  FPLA illustrating the programmable bit connections in both the AND and OR array sections. A given p-term row can become active,  $P_i(H) = 1(H)$ , iff all of the NMOS switches on the AND side are either disconnected or turned OFF. If any one of the NMOS in a p-term row is turned ON, then that row is pulled low, 0(H). An active p-term line causes a connected OR bit position in its path to be pulled low, resulting in a 1(H) from the output tri-state inverter. The buffer between the AND and OR stages is necessary to boost and sharpen the signal. Such a buffer could consist of two CMOS inverters. Notice that tri-state drivers provide an active low enable control on the OR stage.

The programming of a PLA is best understood by considering the 3-input/2-output FPLA segment in Fig. 7.7. Here, the single p-term line is programmed to generate  $O_1(H) = 1(H)$  and  $O_0(H) = 0(H)$  any time the p-term  $\bar{I}_1 \cdot I_0$  becomes active. Notice that both NMOS bit positions for  $I_2$  are disconnected together with those for the  $I_1(L)$  and  $I_0(H)$  lines. Thus, if  $I_1$  is 0(H) and  $I_0$  is 1(H), the p-term line  $P_i(H)$  is forced active which, in turn, causes output  $O_1(H)$  to become 1(H), but not output  $O_0(H)$ , whose bit position has been disconnected, causing it to become 0(H). In effect, disconnection (blowing) of a fusible link in the AND plane actually “makes the connection” of the p-term input ( $I_j$  or  $\bar{I}_j$ ), and disconnection of a fusible link in the OR plane stores a  $1(H) = 0(L)$ . These facts may make it easier to understand the symbolic representations illustrated in Subsection 7.3.1.

### 7.3.1 PLA Applications

Unlike the programming of ROMs which require canonical data, PLAs require minimum or reduced SOP data. Thus, the most efficient application of an FPLA would be one for which the needed product terms fit within the FPLA's limited p-term capability. A good example is the FPLA implementation of the 4-bit combinational shifter of Fig. 6.37. Shown in Fig. 7.8 is the truth table (reproduced from Fig. 6.37a) and the EV K-maps and minimum cover for this shifter. The minimum output expressions for this shifter are obtained directly

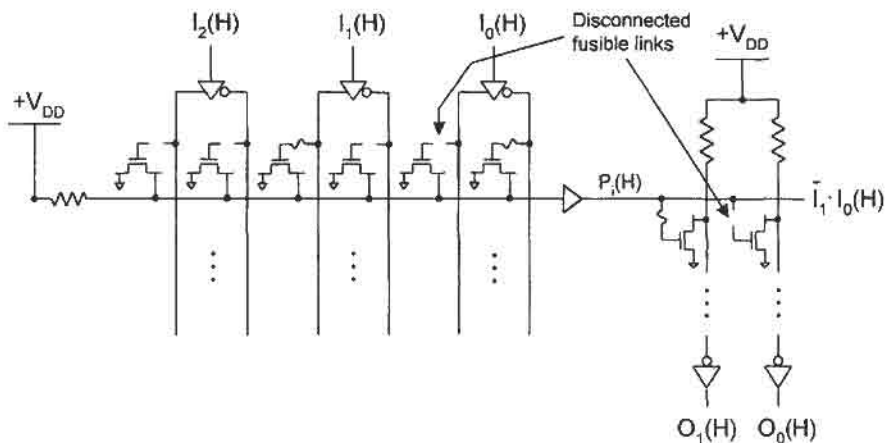
**FIGURE 7.6**

MOS version of an unprogrammed  $n \times p \times m$  FPLA showing the programmable NMOS connections for normally active bit locations in the AND and OR (memory) sections, and showing tri-state outputs with active low enable controls.

from the K-maps and are given as follows:

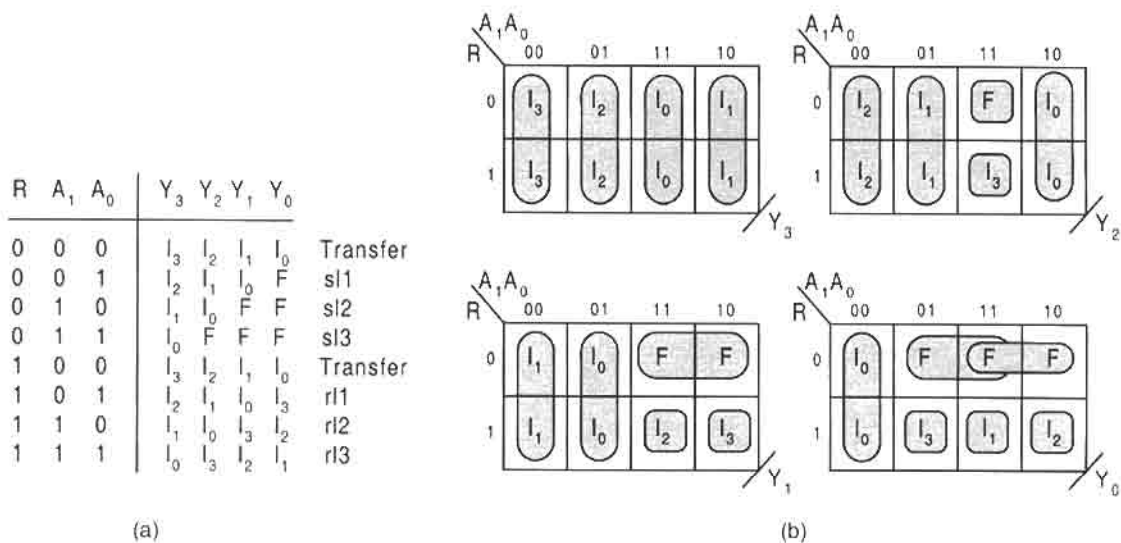
$$\begin{aligned}
 Y_3 &= \bar{A}_1 \bar{A}_0 I_3 + \bar{A}_1 A_0 I_2 + A_1 \bar{A}_0 I_1 + A_1 A_0 I_0 \\
 Y_2 &= \bar{A}_1 \bar{A}_0 I_2 + \bar{A}_1 A_0 I_1 + A_1 \bar{A}_0 I_0 + RA_1 A_0 I_3 + \bar{R}FA_1 A_0 \\
 Y_1 &= \bar{A}_1 \bar{A}_0 I_1 + \bar{A}_1 A_0 I_0 + RA_1 \bar{A}_0 I_3 + RA_1 A_0 I_2 + \bar{R}FA_1 \\
 Y_0 &= \bar{A}_1 \bar{A}_0 I_0 + R\bar{A}_1 A_0 I_3 + RA_1 \bar{A}_0 I_2 + RA_1 A_0 I_1 + \bar{R}FA_1 + \bar{R}FA_0.
 \end{aligned} \tag{7.1}$$



**FIGURE 7.7**

Portion of a 3-input/2-output FPLA programmed for a single  $p$ -term showing connected and disconnected NMOS bit positions in both AND and OR planes.

Having obtained the minimum expressions for the four outputs given by Eqs. (7.1), all that remains is to construct the  $p$ -term table from which the FPLA can be programmed. In this case an FPLA having minimum dimensions of  $8 \times 19 \times 4$  will be required. Note that there are 20  $p$ -terms but one ( $\bar{R}FA_1$ ) is a shared PI. Presented in Fig. 7.9 is the  $p$ -term table for the 4-bit shifter. Notice that uncomplemented input variables are indicated by logic 1

**FIGURE 7.8**

Design of a 4-bit general shifter that can shift or rotate left up to three bits with  $F$  fill. (a) EV truth table. (b) Output EV K-maps showing minimum two-level cover suitable for FPLA implementation.

| p-Term                  | Inputs   |          |                       |                       |                       |                       |                       |                       | Outputs               |                       |                       |                       |
|-------------------------|----------|----------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|
|                         | <i>R</i> | <i>F</i> | <i>A</i> <sub>1</sub> | <i>A</i> <sub>0</sub> | <i>I</i> <sub>3</sub> | <i>I</i> <sub>2</sub> | <i>I</i> <sub>1</sub> | <i>I</i> <sub>0</sub> | <i>Y</i> <sub>3</sub> | <i>Y</i> <sub>2</sub> | <i>Y</i> <sub>1</sub> | <i>Y</i> <sub>0</sub> |
| $\bar{A}_1\bar{A}_0I_3$ | —        | —        | 0                     | 0                     | 1                     | —                     | —                     | —                     | 1                     | 0                     | 0                     | 0                     |
| $\bar{A}_1A_0I_2$       | —        | —        | 0                     | 1                     | —                     | 1                     | —                     | —                     | 1                     | 0                     | 0                     | 0                     |
| $A_1\bar{A}_0I_1$       | —        | —        | 1                     | 0                     | —                     | —                     | 1                     | —                     | 1                     | 0                     | 0                     | 0                     |
| $A_1A_0I_0$             | —        | —        | 1                     | 1                     | —                     | —                     | —                     | 1                     | 1                     | 0                     | 0                     | 0                     |
| $\bar{A}_1\bar{A}_0I_2$ | —        | —        | 0                     | 0                     | —                     | 1                     | —                     | —                     | 0                     | 1                     | 0                     | 0                     |
| $\bar{A}_1A_0I_1$       | —        | —        | 0                     | 1                     | —                     | —                     | 1                     | —                     | 0                     | 1                     | 0                     | 0                     |
| $A_1\bar{A}_0I_0$       | —        | —        | 1                     | 0                     | —                     | —                     | —                     | 1                     | 0                     | 1                     | 0                     | 0                     |
| $RA_1A_0I_3$            | 1        | —        | 1                     | 1                     | 1                     | —                     | —                     | —                     | 0                     | 1                     | 0                     | 0                     |
| $\bar{R}FA_1A_0$        | 0        | 1        | 1                     | 1                     | —                     | —                     | —                     | —                     | 0                     | 1                     | 0                     | 0                     |
| $\bar{A}_1\bar{A}_0I_1$ | —        | —        | 0                     | 0                     | —                     | —                     | 1                     | —                     | 0                     | 0                     | 1                     | 0                     |
| $\bar{A}_1A_0I_0$       | —        | —        | 0                     | 1                     | —                     | —                     | —                     | 1                     | 0                     | 0                     | 1                     | 0                     |
| $RA_1\bar{A}_0I_3$      | 1        | —        | 1                     | 0                     | 1                     | —                     | —                     | —                     | 0                     | 0                     | 1                     | 0                     |
| $RA_1A_0I_2$            | 1        | —        | 1                     | 1                     | —                     | 1                     | —                     | —                     | 0                     | 0                     | 1                     | 0                     |
| $\bar{R}FA_1$           | 0        | 1        | 1                     | —                     | —                     | —                     | —                     | —                     | 0                     | 0                     | 1                     | 1                     |
| $\bar{A}_1\bar{A}_0I_0$ | —        | —        | 0                     | 0                     | —                     | —                     | —                     | 1                     | 0                     | 0                     | 0                     | 1                     |
| $RA_1\bar{A}_0I_3$      | 1        | —        | 0                     | 1                     | 1                     | —                     | —                     | —                     | 0                     | 0                     | 0                     | 1                     |
| $RA_1\bar{A}_0I_2$      | 1        | —        | 1                     | 0                     | —                     | 1                     | —                     | —                     | 0                     | 0                     | 0                     | 1                     |
| $RA_1A_0I_1$            | 1        | —        | 1                     | 1                     | —                     | —                     | 1                     | —                     | 0                     | 0                     | 0                     | 1                     |
| $\bar{R}FA_0$           | 0        | 1        | —                     | 1                     | —                     | —                     | —                     | —                     | 0                     | 0                     | 0                     | 1                     |

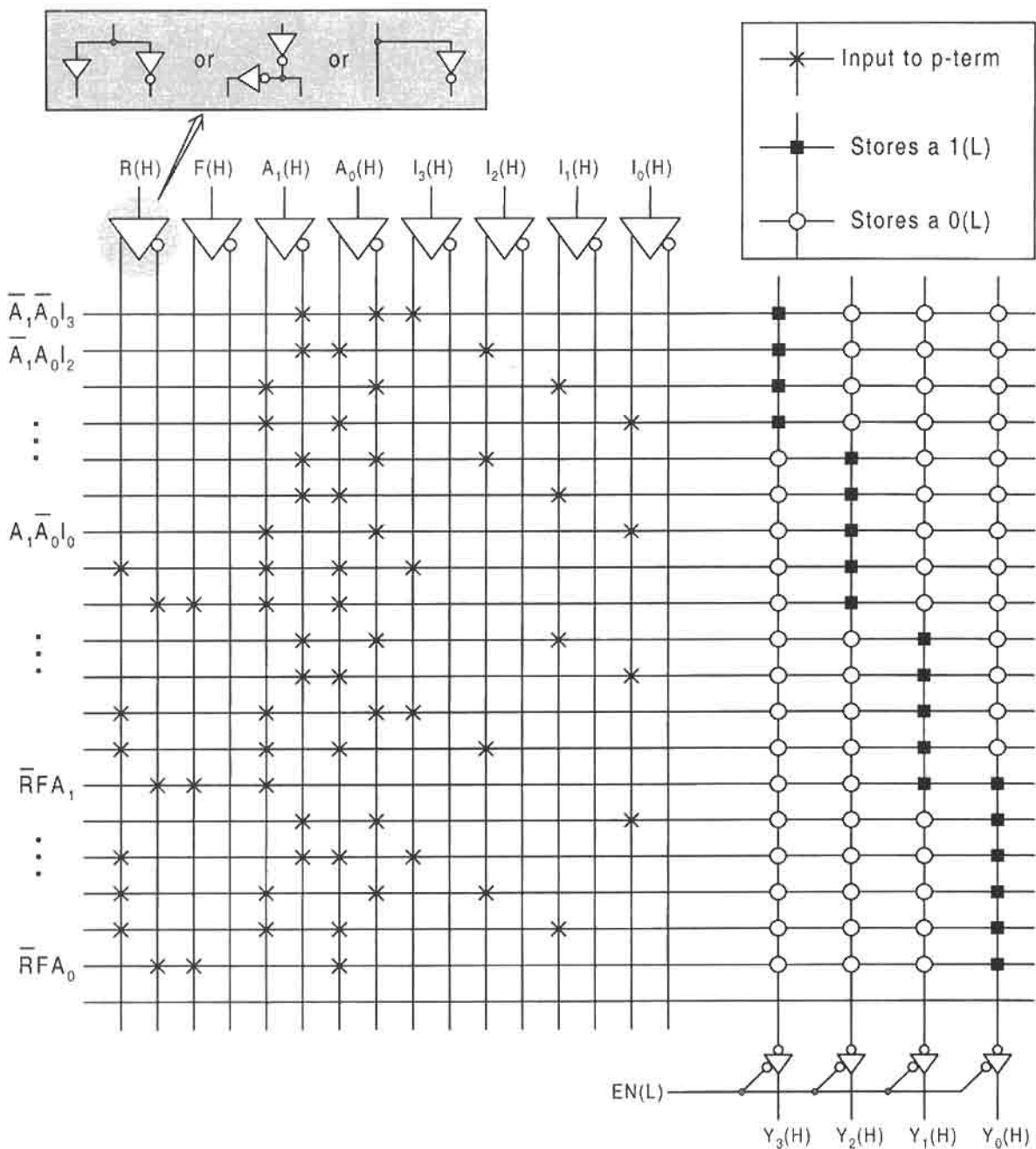
FIGURE 7.9

P-term table for the 4-bit shifter represented in Fig. 7.8 and by Eq. (7.1).

and complemented input variables by logic 0, and the absence of an input to a p-term is indicated by a dash.

Presented in Fig. 7.10 is the symbolic representation of an  $8 \times 20 \times 4$  FPLA programmed according to the p-term table in Fig. 7.9. The symbolism that is used avoids the need to provide details of the specific technology used in the FPLA. However, reference to Figs. 7.6 and 7.7 permits one to associate the symbolism of Fig. 7.10 with FPLA MOS technology. Notice the existence of the shared PI,  $\bar{R}FA_1$ , in the  $Y_1$  and  $Y_0$  output columns.

The symbolism of Fig. 7.10 is meant to be easily understood. The  $\times$  in the AND plane signifies an input to a p-term and represents a disconnected fusible NMOS link in the sense of Fig. 7.7. The filled square ( $\blacksquare$ ) in the OR plane represents the programmed storage of 1(L) created by a connected fusible NMOS link in the sense of Fig. 7.7, and the open circle ( $\circ$ ) in the OR plane indicates the programmed storage of 0(L) created by a disconnected fusible NMOS link. To assist the reader in deciphering this notation, representative p-terms are provided in Fig. 7.10 at the left and adjacent to their respective p-term lines.



**FIGURE 7.10**

Symbolic representation of an  $8 \times 20 \times 4$  FPLA programmed according to the p-term table in Fig. 7.9 and showing the fusible bit position patterns in both the AND and OR planes. Representative p-terms are given to the left of their respective p-term lines.

## 7.4 PROGRAMMABLE ARRAY LOGIC DEVICES

Perhaps the most widely used PLD is the *programmable array logic* (PAL) device. The PAL device can be programmed only in the AND plane. For reference purposes the main differences between ROMs, PLAs and PAL devices may be stated as follows:

- ROM devices** — Programmable in the OR plane only
- PLA devices** — Programmable in both AND and OR planes
- PAL devices** — Programmable in the AND plane only

Thus, a PLA device may be thought of as a combination of ROM and PAL device programming characteristics. PAL devices, like PLAs, commonly include a variety of devices external to their OR planes, including XOR gates, AND–OR–invert logic, and registered outputs.

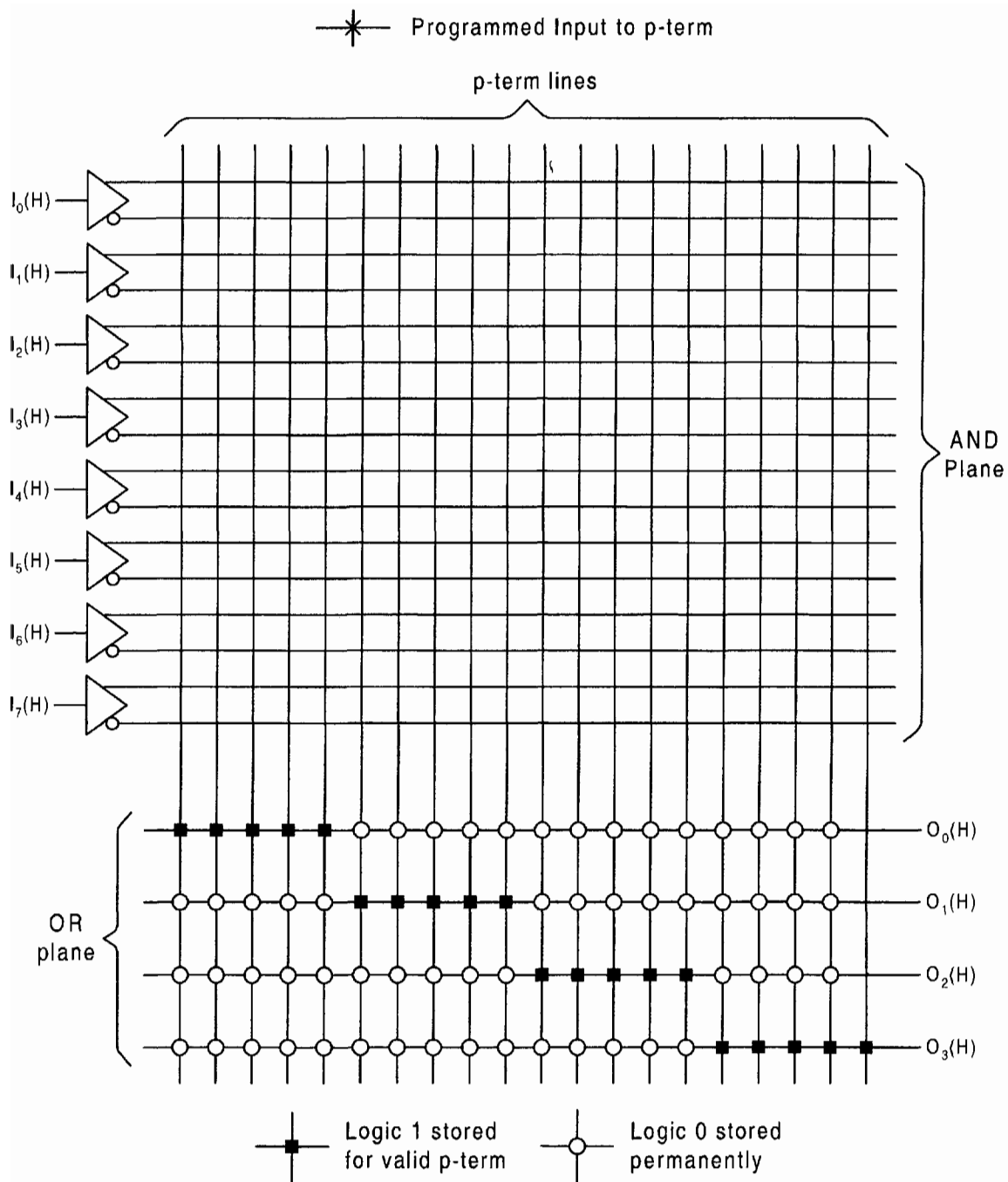
The acronym PAL is a registered trademark of Advanced Micro Devices, Inc. Therefore, hereafter it will be understood that use of the name PAL will acknowledge AMD's right of trademark for all devices that carry the name PAL.

PAL devices characteristically provide a fixed number of p-terms per output and cannot take advantage of shared PIs. This is illustrated by the unprogrammed  $8 \times 20 \times 4$  PAL device shown in Fig. 7.11, which allows up to five p-terms per output. If the number of p-terms for a given output exceeds the number provided by the PAL device, the remaining p-terms can be given to another column output line, and the two ORed external to the OR plane. Thus, in the case of the 4-bit shifter, the  $Y_0(H)$  output, requiring six p-terms, would have to have one or more of its p-terms given to a fifth output line (not shown) and the two lines ORed external to the OR plane. The MOS version of a basic PAL device would look like that for the PLA in Fig. 7.6, except that the NMOS bit positions in the OR stage would be permanently connected — no fusible links.

The basic PAL device simply employs an AND and OR section either in the form of AND/OR or AND/NOR as in Fig. 7.12a. However, the output logic of most PAL devices goes beyond the basic PAL. Shown in Fig. 7.12b is a segment of a PAL device that supports an *L-type* (logic-type) macrocell consisting of a controlled inverter (XOR gate), an AND-controlled output enable, and a feedback path. The feedback path is useful for cascading combinational logic functions, or for the design of asynchronous (self-timed) sequential machines covered in Chapter 14. In either case, one output function is fed back to become the input in the generation of another output function.

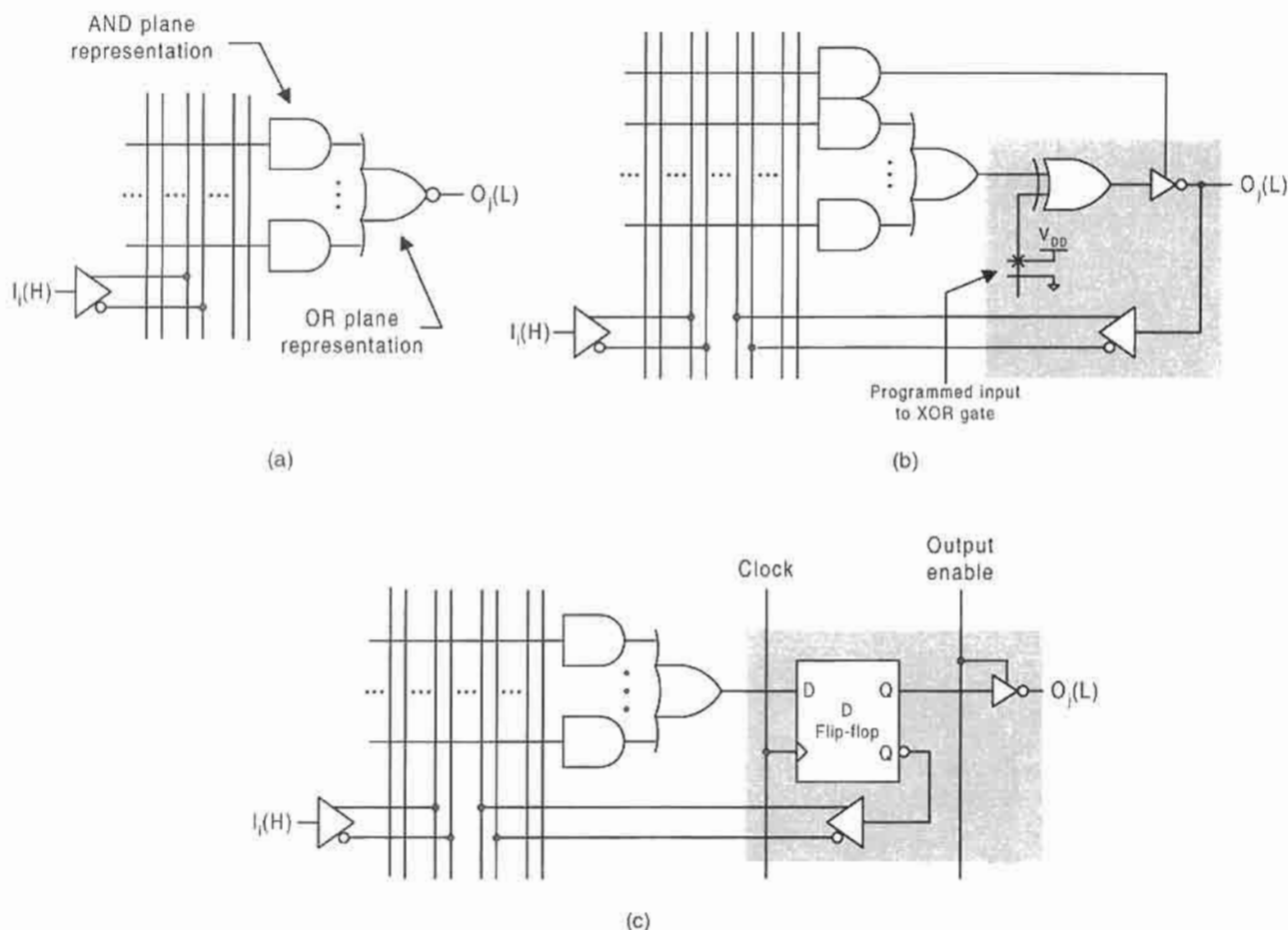
Macrocells containing on-chip flip-flops are also found in PAL devices, as illustrated in Fig. 7.12c. These are called *R-type* (registered-type) macrocells; they support feedback from a flip-flop output and a controlled tri-state driver/enable. Both the clock signal and the output enable signal can be supplied externally or can be generated from within the PAL device. PAL devices with R-type macrocells are useful in the design of synchronous (clock-driven) sequential machines, which are discussed at length in Chapters 10 through 13. The description of flip-flops and the details of their design and operation are presented in Chapter 10.

The versatility of PAL devices is improved significantly by the use of *V-type* (variable-type) macrocells such as that illustrated in Fig. 7.13. Now, the output signal generated from a 4-to-1 MUX can be combinational or registered depending on the data select inputs  $S_1$  and  $S_0$  whose logic values are set by programming fusible links. Thus, data select inputs



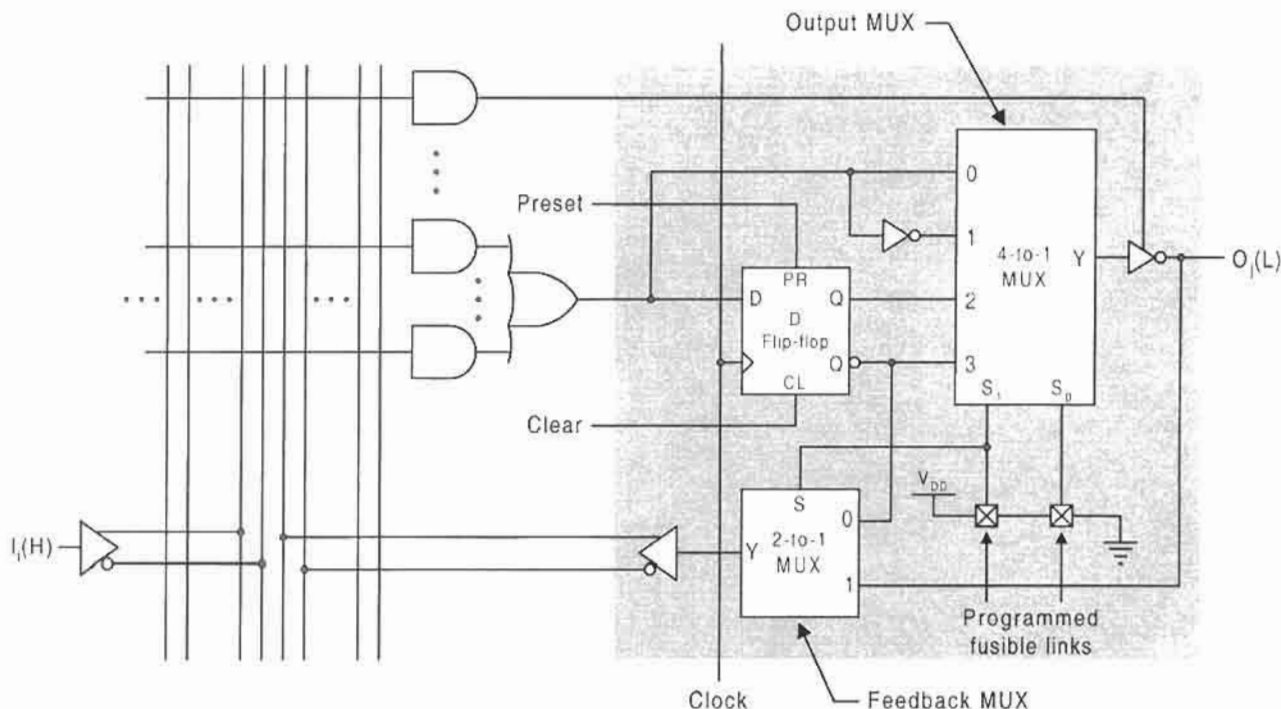
**FIGURE 7.11**

Symbolic representation of a basic unprogrammed  $8 \times 20 \times 4$  PAL with five p-term capability (unalterable) for each of the four outputs.

**FIGURE 7.12**

Logic equivalent segments of real PAL devices showing macrocell logic. (a) Basic I/O PAL. (b) L-type macrocell for a programmable I/O PAL device with controlled inverter output, feedback capability, and AND-controlled enable. (c) Macrocell for a registered (R-type) I/O PAL device with feedback from a flip-flop and with a controlled enable.

of  $S_1, S_0 = 00$  or  $S_1, S_0 = 01$  generate an active low or active high output (from the tri-state driver), respectively, directly from the AND/OR sections of the PAL device — hence, combinational. For data select inputs of  $S_1, S_0 = 10$  or  $S_1, S_0 = 11$ , registered (flip-flop) outputs are generated active low or active high, respectively, from the inverting tri-state driver. Thus, the V-type macrocell combines the capabilities of the L-type and R-type macrocells of Fig. 7.12. But the V-type macrocell goes well beyond these capabilities and offers even more flexibility. A 2-to-1 MUX permits the active high and active low feedback signals to be generated either by the 4-to-1 MUX with its four options or directly by the active low output of the flip-flop. The data select input  $S$  to the 2-to-1 MUX is taken from the programmed data select input  $S_1$  to the 4-to-1 MUX, as shown in Fig. 7.13. Because of the flexibility they offer, PAL devices with V-type macrocells are a popular choice for designers of both combinational logic or sequential machine design.

**FIGURE 7.13**

Output macrocell for V-type PAL devices showing I/O connections and cell logic. Source: *PAL Device Data Book*, Advanced Micro Devices, Inc., Sunnyvale, CA, 1992.

## 7.5 MIXED-LOGIC INPUTS TO AND OUTPUTS FROM ROMs, PLAs, AND PAL DEVICES

The subject of mixed-logic inputs to decoders was discussed in Subsection 6.3.2. There, two options were given for dealing with inputs that arrive both from positive and negative logic sources. These input rules are necessary since the decoder is normally an IC chip with no user access to its internal structure. Because ROMs, PLAs, and PALs also exist as IC chips, the same input rules also apply to them. For reference purposes, these input rules are stated as follows:

### For Mixed-Logic Inputs to ROMs, PLAs, and PALs

(1) In the program table for a ROM, PLA or PAL device, complement each bit in the column of an active low input

or

(2) Use an inverter on the input line of an active low input to one of these PLDs and do not complement the bits in that column of the program table.

Clearly, these mixed-logic input rules are basically the same as those stated in Subsection 6.3.2 for decoders.

The mixed-logic output rules for ROMs, PLAs, and PALs differ somewhat depending on the PLD in question. The mixed-logic output rules for ROMs are as follows:

**For Mixed-Logic Outputs from ROMs**

- (1) In the ROM program table complement each bit in the column of any output from a ROM that is required to be active low,  
or  
(2) Use the inverter capability of the ROM, or add one externally, on that output line to generate the active low output and *do not* complement the bits in the output column.

Since ROMs accept only canonical (minterm) data, it is permissible, as an option, to complement the output column for an active low output. This is equivalent to complementing the conventional (1's and 0's) K-map for the active low output to yield

$$Y_{SOP}(L) = \bar{Y}_{SOP}(H), \quad (7.2)$$

which follows from the definition of complementation given by Eq. (3.2). But this is only possible for canonical data as in the case of a ROM. For example, suppose it is required that the output ( $A = B$ ) in Fig. 7.4 be generated active low. To do this, the ( $A = B$ ) output column can be complemented, interchanging the (○) and (■) symbols, or an inverter can be connected to the ( $A = B$ ) output line from the PROM.

The situation for PLA and PAL devices is much different from that for ROMs relative to active low outputs. Now, reduced or minimum forms constitute the output functions, and complementation of the output columns of active low outputs would result in radically different functions. Equation (7.2) does not apply to output column complementation for these PLDs. The rule for dealing with active low outputs from PLA and PAL devices is stated in the following way:

**For Active Low Outputs from PLA and PAL Devices**

If a given output line from a PLA or PAL device must be generated with an activation level different from that provided internally by the PLD, use *must* be made of an inverter added externally to that line. Complementation of an output column in the p-term table is *not* permitted.

As an example, consider the FPLA in Fig. 7.10, which has been programmed to function as a 4-bit shifter with  $F$  fill. Suppose that  $R$  arrives active low and that all outputs must be delivered to the next stage active low. To achieve this objective with minimum external logic, the  $R$  column is complemented and, if mixed logic outputs are not provided internal to the FPLA, inverters are placed on the output lines for  $Y_3$ ,  $Y_2$ ,  $Y_1$ , and  $Y_0$ . Complementation of the  $R$  column in the p-term table of Fig. 7.9 requires that the 1's and 0's be interchanged but leaving all dashes unaltered. Thus, the ×'s in the two  $R$  columns in Fig. 7.10 will be moved from the active high column to the active low column and vice versa. The meaning of the ×'s was explained previously in discussing the symbolism of Fig. 7.10.

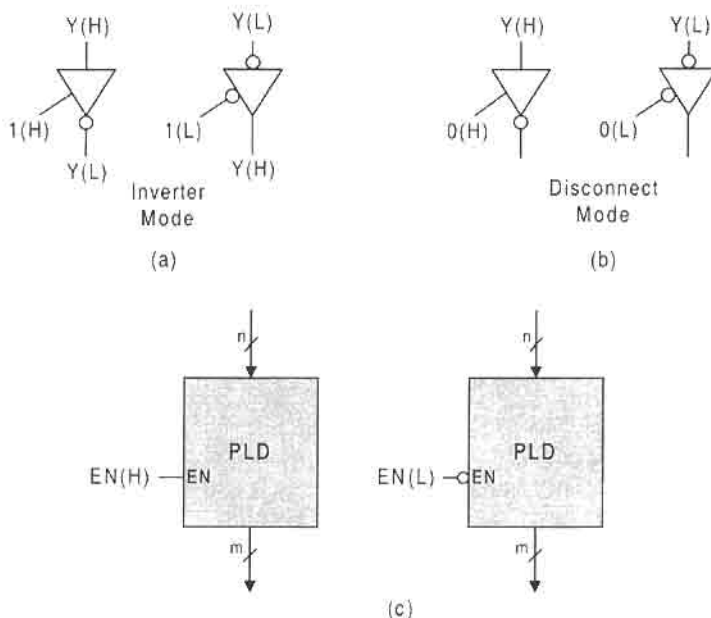


## 7.6 MULTIPLE PLD SCHEMES FOR AUGMENTING INPUT AND OUTPUT CAPABILITY

Occasions arise when the I/O requirements of a design exceed the capabilities of the available PLD. When this happens the designer may have no alternative but to combine PLDs in some suitable fashion to meet the design requirements. To accomplish this requires the combined use of tri-state driver and *wired-OR* technologies, which permit nearly an unlimited number of outputs from different PLDs to be ORed together. The use of OR gates to accomplish this task would suffer the disadvantage of fan-in limitations and speed reduction.

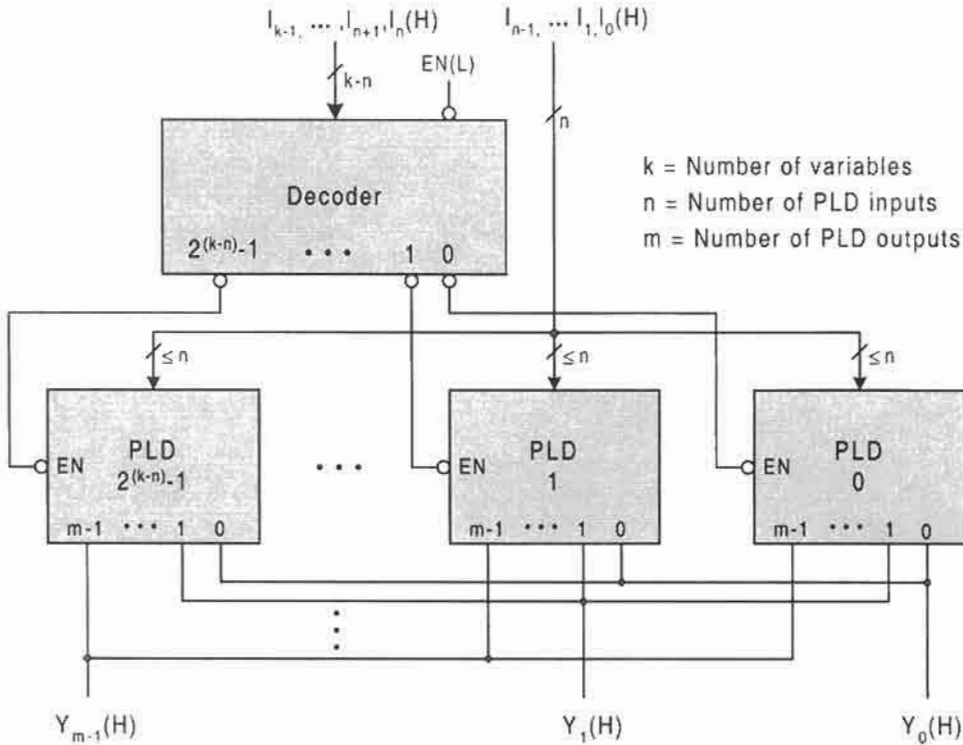
Figures 7.2 and 7.6 illustrate the use of inverting tri-state drivers (see Fig. 3.8) with active low enable control  $EN(L)$  on the output lines from the PROM and FPLA, respectively. These tri-state drivers not only function to satisfy the logic level requirements of the device outputs, but also permit a type of PLD multiplexing based on the transfer and disconnect (high  $Z$ ) modes of the tri-state drivers. The two principal modes of tri-state driver operation are illustrated in Figs. 7.14a and 7.14b, and the block diagrams for PLDs with active high tri-state driver outputs controlled by active high and active low enable controls are shown in Fig. 7.14c.

By using a multiplexed scheme involving a decoder and PLDs with active low tri-state enable controls, it is possible to increase the input capability beyond that of the stand-alone PLDs. Such a scheme is shown in Fig. 7.15, where a  $(k - n)$ -to- $2^{(k-n)}$  line decoder is used to select  $2^{(k-n)}$   $n$ -input PLDs, each with  $m$  outputs. The use of active low tri-state enables,



**FIGURE 7.14**

Tri-state driver/enables used in PLDs. (a) Transfer and (b) disconnect modes of operation for active high and active low tri-state drivers. (c) Block diagrams for a PLD with active high and active low tri-state driver/enables, and active high outputs.

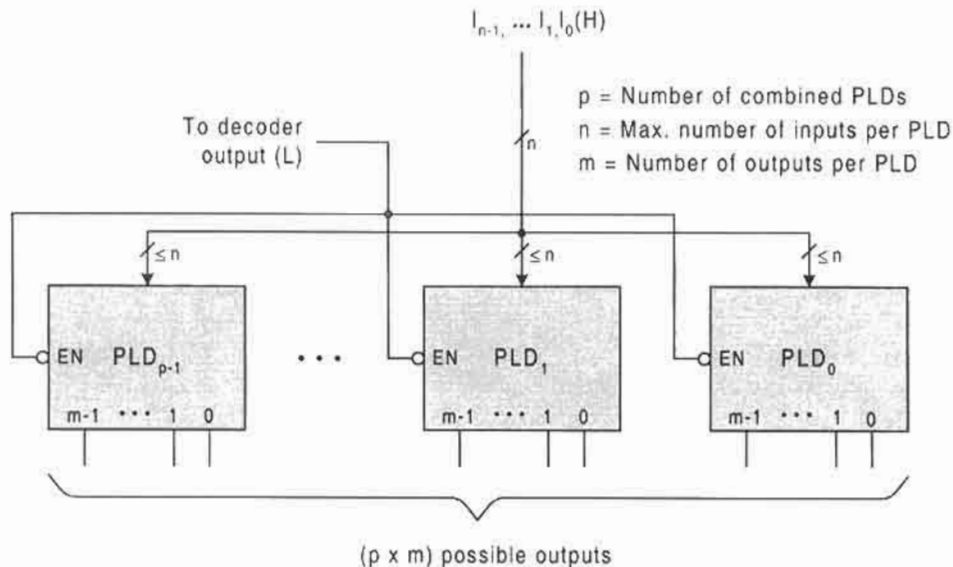
**FIGURE 7.15**

Multiplexed scheme by using a decoder and  $n$ -input PLDs to increase input variable capability from  $n$  to  $k > n$ .

as in Fig. 7.14, makes possible the wire-ORed connection of outputs shown in Fig. 7.15. This is so because the most significant  $I_{k-1}, \dots, I_n$  input bits activate only one of the  $2^{(k-n)}$   $n$ -input PLDs at any given time. As indicated in Fig. 7.15 by the notation  $\leq n$ , not all the available inputs to a given PLD need be used. Also, each PLD output need not be connected (wire-ORed) with the corresponding outputs from other PLDs to form *tri-state bus* lines.

Although Fig. 7.15 satisfies the need to augment the input capability of PLDs, it does not address the problem of limited output capability. When the number of output functions of a design exceeds the output capability of the PLDs in use, a parallel arrangement of the type shown in Fig. 7.16 can be used. This scheme is applicable to any stage of the multiplexed configuration of Fig. 7.15. It indicates that  $p$  PLDs, each of  $m$  outputs, yield a maximum of  $(p \times m)$  possible outputs per stage, thereby increasing the output capability from  $m$  to  $(p \times m)$ . However, it is important to understand that *these outputs must not be wire-ORed together*, since they are from PLDs that are activated by the same decoder output — the PLDs in Fig. 7.16 are not multiplexed. Note that the PLDs need not have the same number of inputs or outputs, but the number of inputs is limited to  $n$  or less.

**EXAMPLE 7.1** Suppose it is required to generate three output functions of 10 variables by using  $2^8 \times 4$  PROMs. Since the number of input variables exceeds the number of PLD

**FIGURE 7.16**

Scheme for increasing the output function capability from  $m$  to  $(p \times m)$  for any stage of the PLD configuration shown in Fig. 7.15.

inputs by 2, a 2-to-4 decoder is required to selectively activate the four PROMs one at a time. Presented in Fig. 7.17 is the ROM program table that satisfies the requirements of this example. Since four PROMs are required ( $2^{(k-n)} = 4$ ), the program table must be partitioned into four parts, each part being a program for a single PROM. Once programmed, the four PROMs are configured with the 2-to-4 decoder as illustrated in Fig. 7.18. The outputs are wire-ORed together as shown to generate the three outputs of 10 variables.

**EXAMPLE 7.2** Augmentation of the input, output, and  $p$ -term capability of the PLDs in use is illustrated by the following example. Consider the implementation of 16 functions of 10 variables by using  $8 \times 20 \times 4$  FPLAs subject to the following  $p$ -term constraints dictated by the function makeup:

Eight of the functions each require that  $20 < p \leq 40$ , while the remaining 8 require that  $p \leq 20$ . Here,  $p$  is the number of  $p$ -terms per function.

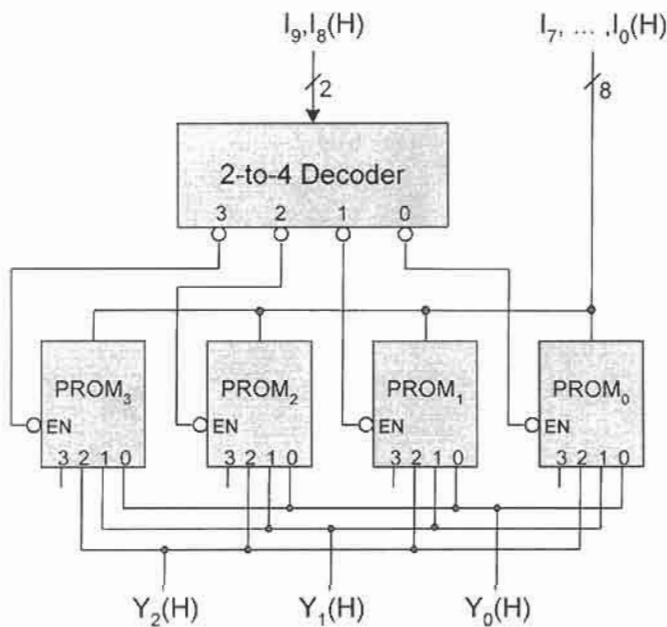
The  $p$ -term program table format for this system is given in Fig. 7.19. The  $p$ -terms are listed on the left side of the table, and the 10 FPLA inputs to the  $p$ -terms are indicated by using the notation given in Fig. 7.19. It is implied by the program table of Fig. 7.19 that two inputs,  $I_9$  and  $I_8$ , to a 2-to-4 decoder are to be used to determine the FPLA assignment. With this in mind, one possible assignment scheme is as follows: The eight functions requiring 20 to 40  $p$ -terms take decoder addresses of 00 or 01, while functions requiring 20 or fewer  $p$ -terms take decoder addresses of 10 or 11.

Implementation of the 16 functions of 10 variables, according to the decoder assignments just given, requires that six  $8 \times 20 \times 4$  FPLAs be selected by the 2-to-4 decoder as shown in Fig. 7.20. Here, four of the functions,  $Y_3$ ,  $Y_2$ ,  $Y_1$ , and  $Y_0$ , are assigned to FPLA<sub>0</sub> and

| PROM Inputs |       |       |       |       |       |       |       |       |       | PROM Outputs      |       |       |
|-------------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------------------|-------|-------|
| $I_9$       | $I_8$ | $I_7$ | $I_6$ | $I_5$ | $I_4$ | $I_3$ | $I_2$ | $I_1$ | $I_0$ | $Y_2$             | $Y_1$ | $Y_0$ |
| 0           | 0     | 0     | 0     | 0     | 0     | 0     | 0     | 0     | 0     | PROM <sub>0</sub> |       |       |
| 0           | 0     | 1     | 1     | 1     | 1     | 1     | 1     | 1     | 1     |                   |       |       |
| 0           | 1     | 0     | 0     | 0     | 0     | 0     | 0     | 0     | 0     | PROM <sub>1</sub> |       |       |
| 0           | 1     | 1     | 1     | 1     | 1     | 1     | 1     | 1     | 1     |                   |       |       |
| 1           | 0     | 0     | 0     | 0     | 0     | 0     | 0     | 0     | 0     | PROM <sub>2</sub> |       |       |
| 1           | 0     | 1     | 1     | 1     | 1     | 1     | 1     | 1     | 1     |                   |       |       |
| 1           | 1     | 0     | 0     | 0     | 0     | 0     | 0     | 0     | 0     | PROM <sub>3</sub> |       |       |
| 1           | 1     | 1     | 1     | 1     | 1     | 1     | 1     | 1     | 1     |                   |       |       |

**FIGURE 7.17**

Partitioned PROM program table required to generate three output functions of 10 variables by using four  $2^8 \times 4$  PROMs.



**FIGURE 7.18**

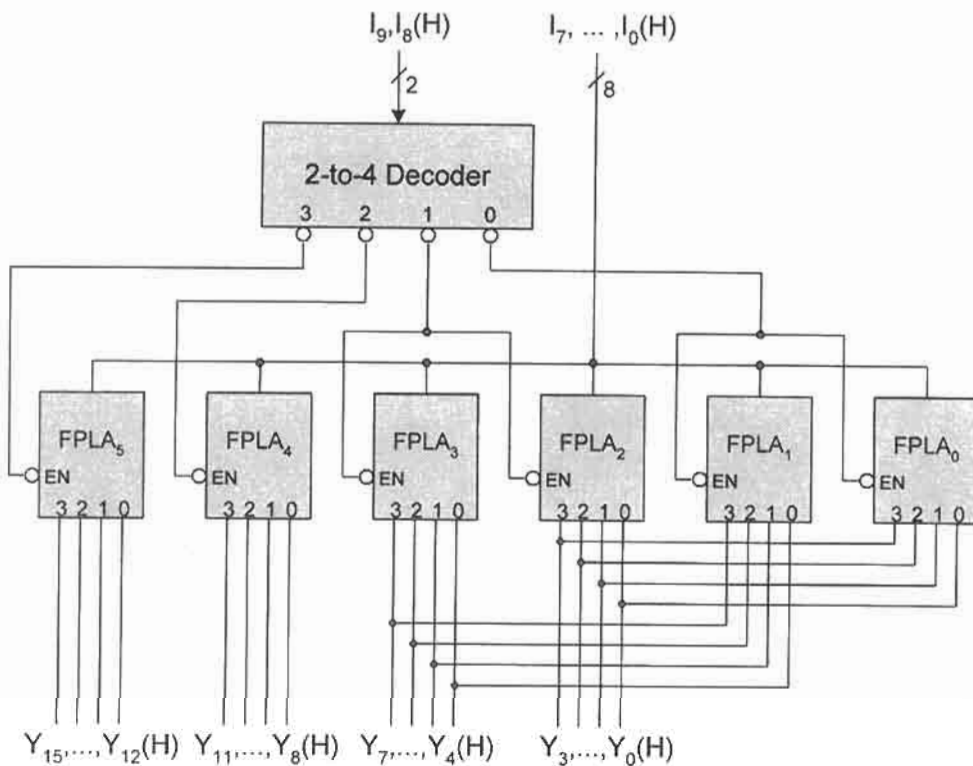
Multiplexed scheme required by the program table of Fig. 7.17 to generate three functions of 10 variables by using four  $2^8 \times 4$  PROMs.

| P-terms | FPLA Inputs |       |       |       |       |       |       |       |       |       | FPLA Outputs |     |       |       |       |       | FPLA Assignment |
|---------|-------------|-------|-------|-------|-------|-------|-------|-------|-------|-------|--------------|-----|-------|-------|-------|-------|-----------------|
|         | $I_9$       | $I_8$ | $I_7$ | $I_6$ | $I_5$ | $I_4$ | $I_3$ | $I_2$ | $I_1$ | $I_0$ | $Y_{15}$     | ... | $Y_3$ | $Y_2$ | $Y_1$ | $Y_0$ |                 |
|         |             |       |       |       |       |       |       |       |       |       |              |     |       |       |       |       |                 |

**FIGURE 7.19**

P-term table format for a multiple FPLA scheme to generate 16 functions of 10 variables by using six  $8 \times 20 \times 4$  FPLAs subject to the p-term conditions stated in Example 7.2.

FPLA<sub>2</sub>; the remaining four,  $Y_7$ ,  $Y_6$ ,  $Y_5$ , and  $Y_4$ , are assigned to FPLA<sub>1</sub> and FPLA<sub>3</sub>. Notice that the active low tri-state enables for FPLA<sub>0</sub> and FPLA<sub>1</sub> are connected together and to a single decoder output, as are those for FPLA<sub>2</sub> and FPLA<sub>3</sub>. By wire-ORing the outputs in this manner, eight of the 16 output functions are each permitted to have up to 40 p-terms with the remaining eight functions limited to 20 p-terms.

**FIGURE 7.20**

Scheme to generate 16 functions of 10 variables by using  $8 \times 20 \times 4$  FPLAs subject to p-term requirements: 8 functions for which  $20 < p \leq 40$ , and 8 functions for which  $p \leq 20$ .

## 7.7 INTRODUCTION TO FPGAs AND OTHER GENERAL-PURPOSE DEVICES

The devices briefly discussed in this section vary in size from SSI to LSI circuit modules and range greatly in complexity from simple gate-level modules to those having a multiplicity of components, including flip-flops. Of the general purpose devices, the simplest are the AND–OR–invert or OR–AND–invert type devices with logic equivalent gate counts numbering typically in the three to five range. These SSI circuits are often used as building blocks in complex logic devices. Then, within the PLD spectrum of devices, the most complex class belongs to the field programmable gate array (FPGA) devices, which may contain a variety of primitive components, including discrete gates, MUXs, and flip-flops. Since this chapter deals only with combinational logic devices, the treatment here will deal mostly with those PLDs and general-purpose devices that are combinational in character.

The discussions to this point in the text have dealt only with combinational logic, consequently, the reader is not expected to grasp the significance of flip-flop operation in the output logic stage to some PLDs. The use of these “registered” PLDs will become evident in later chapters when sequential machines are discussed in detail. Treatment here will begin with simple general-purpose building block devices and will end with an introduction to the complex FPGAs.

### 7.7.1 AND–OR–Invert and OR–AND–Invert Building Blocks

Just as the XOR and EQV functions can be implemented by what amounts to one gate level of MOS transistors as in Figs. 3.26 and 3.27, so also can the AND–OR–invert and OR–AND–invert functions be implemented with just one gate level of transistors. Shown in Fig. 7.21a is the CMOS realization of the AND–OR–invert (AOI) gate. It is called a gate since it is a CMOS SSI circuit and has a propagation delay equivalent to that of a single NAND or NOR gate. There are many useful applications of this basic building block, including integration with much larger PLD logic blocks. The physical truth table and its mixed-logic interpretation are presented in Figs. 7.21b and 7.21c, respectively. The output logic expression can be read directly from the mixed-logic truth table and is

$$F(L) = [AB + CD](L), \quad (7.3)$$

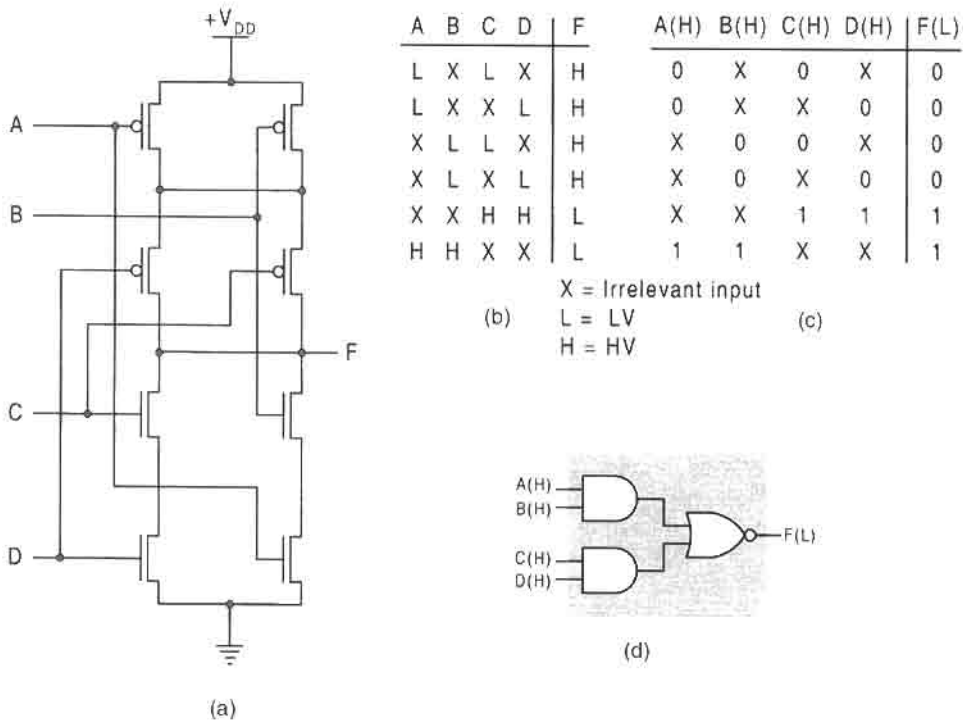
which results in the logic equivalent circuit for the AOI gate shown in Fig. 7.21d.

As a simple example of the use of the AOI gate, consider the two-level active low EQV function given by

$$F(L) = (AB + \bar{A}\bar{B})(L).$$

This function can be implemented in Fig. 7.21d by connecting A to C and B to D via two inverters, a transistor count of 12. In comparison, the EQV gate of Fig. 3.27a requires only six transistors.

The CMOS realization of the OR–AND–invert (OAI) gate is the dual of that for the AND–OR–invert gate and is easily obtained by flipping the latter end-for-end while interchanging all NMOS with PMOS and vice versa. This is done in Fig. 7.22a. The same duality exists between the truth tables of Figs. 7.21 and 7.22, where H’s and L’s are interchange between physical truth tables and 1’s and 0’s are interchanged between mixed-logic truth tables. The output expression for the OAI gate is obtained directly from Fig. 7.22c by reading the 0’s

**FIGURE 7.21**

The AND-OR-invert (AOI) gate. (a) CMOS circuit. (b) Physical truth table. (c) Mixed-logic interpretation of the physical truth table. (d) Logic equivalent circuit for the AOI gate.

in the **F** column and complementing the results to give

$$F(L) = [(A + B)(C + D)](L), \quad (7.4)$$

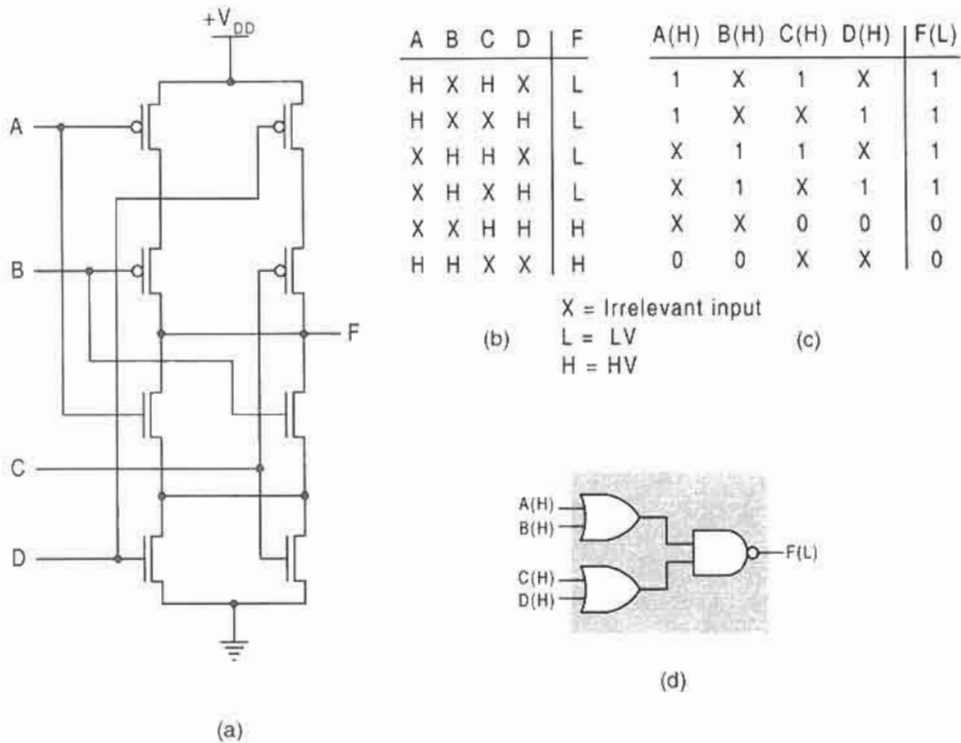
which yields the logic equivalent circuit for the OAI gate shown in Fig. 7.22d.

As a simple example of the use of the OAI gate, consider the two-level active low XOR function given by

$$F(L) = (A + B)(\bar{A} + \bar{B})(L).$$

This function can be implemented in Fig. 7.22d by connecting A to C and B to D via two inverters for a total of 12 transistors, twice the transistor count of the XOR gate of Fig. 3.26a.

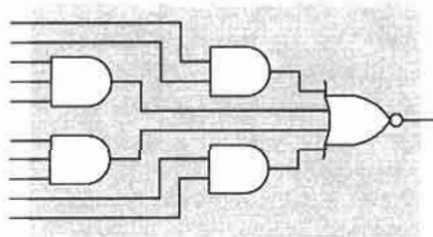
There are several variations on the AOI and OAI gate themes of Figs. 7.21 and 7.22. For example, adding an inverter to the output of each of these gates makes them AND-OR and OR-AND gates. More input stages can also be added to the AOI or OAI gates. Shown in Fig. 7.23 is the logic equivalent circuit for the 74x54, a 10-input AOI circuit. Clearly, the additional input capability adds to the versatility of the device.

**FIGURE 7.22**

The OR-AND-invert (OAI) gate. (a) CMOS circuit. (b) Physical truth table. (c) Mixed-logic interpretation of the physical truth table. (d) Logic equivalent circuit for the OAI gate.

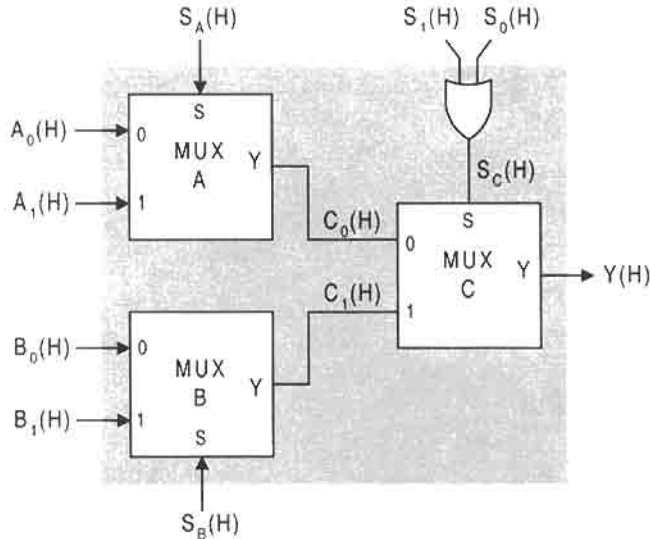
### 7.7.2 Actel Field Programmable Gate Arrays

As indicated previously, the architectures of FPGAs can be very complex and, in fact, are generally regarded as being at the high end in complexity of the programmable gate array (PGA) spectrum. The ACT-1 family of FPGAs (from Actel Corp.) discussed in this

**FIGURE 7.23**

Logic equivalent circuit for the 10-input 74x54 AOI circuit.



**FIGURE 7.24**

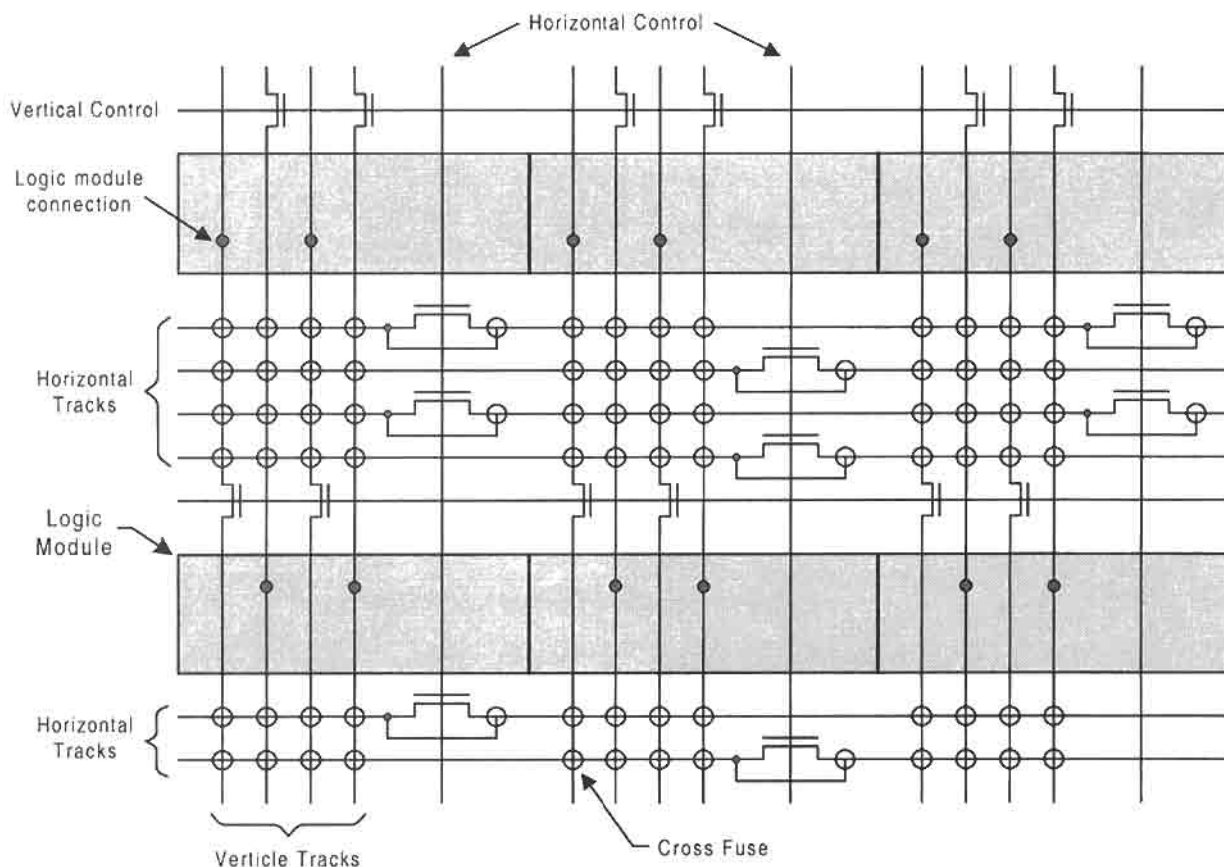
An ACT-1 family logic module. Source: *ACT Family Field Programmable Gate Array Databook*, Actel Corp., Sunnyvale, CA, 1991.

subsection are programmable by the end user. They represent some of the simpler PGAs in the sense that the logic components of these devices consist of relatively simple combinational structures. However, the programmable switching matrices required to interconnect the ACT-1 logic cells are relatively complex and cannot be reprogrammed.

Shown in Fig. 7.24 is an ACT-1 logic module consisting of three 2-to-1 MUXs and a 2-input OR gate for a total of eight inputs and one output. The output logic function is easily deduced from Fig. 7.24 to be

$$\begin{aligned}
 Y &= (\bar{S}_1 + \bar{S}_0)C_0 + (S_1 + S_0)C_1 \\
 &= (\bar{S}_1 + \bar{S}_0)(\bar{S}_A A_0 + S_A A_1) + (S_1 + S_0)(\bar{S}_B B_0 + S_B B_1). \quad (7.5)
 \end{aligned}$$

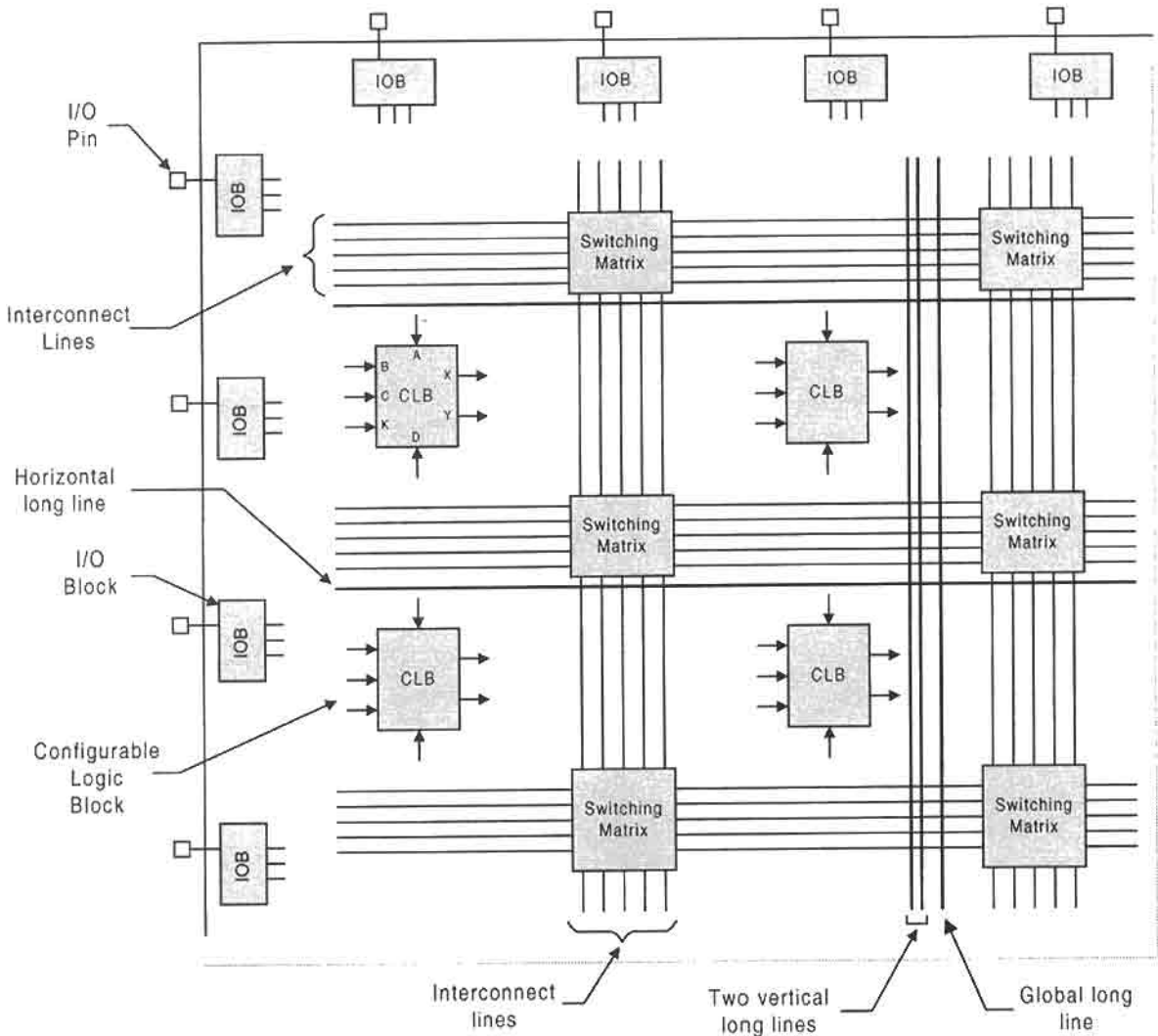
To achieve the status of an FPGA, Actel enmeshes hundreds of these modules in a matrix of programmable interconnections. A segment of the ACT-1 interconnect architecture is illustrated in Fig. 7.25, courtesy of the Actel Corporation. Here, dedicated vertical tracking lines connect with each input to and output from a logic module, while other vertical tracking lines function as feedthrough between channels. Connections between horizontal and vertical tracking lines are made by “blowing” the cross fuses at intersections in accordance with the required fuse pattern program. Since some wire tracks may be blocked by previously allocated tracks, a sequence of jogs from horizontal to vertical tracks and vice versa can be used to circumvent the blockage, thereby permitting connection to an appropriate logic module. Because of the versatility jog programming provides, the ACT-1 family of FPGAs achieves a gate-equivalence capability several times the number of logic modules. For example, the ACT-1 A1010, which has 295 logic modules, purports to have a gate equivalency of 1200.

**FIGURE 7.25**

Interconnect architecture for ACT-1 family of FPGAs. Source: *ACT Family Field Programmable Gate Array Databook*, Actel Corp., Sunnyvale, CA, 1991.

### 7.7.3 Xilinx FPGAs

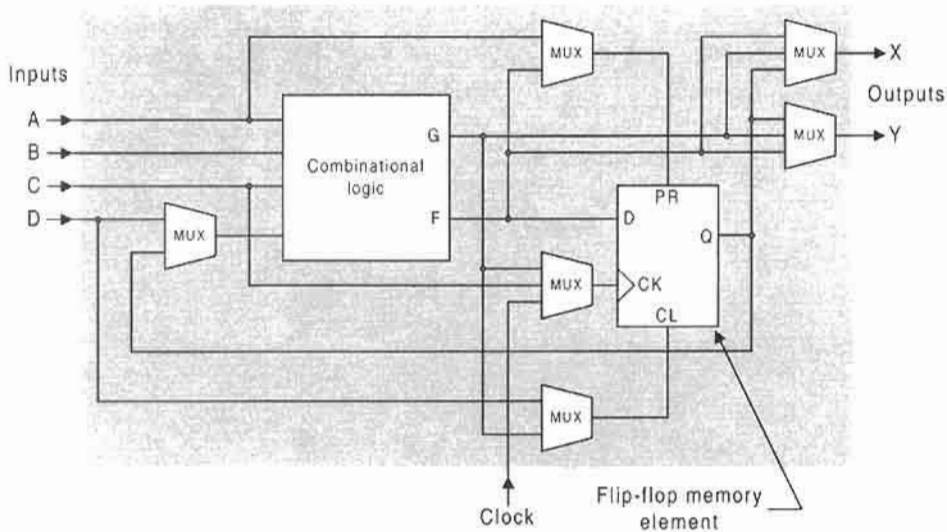
Field programmable gate arrays (FPGAs) offer the most design versatility of the PLDs considered so far, and their architecture differs markedly from those of PALs and Actel PLDs. The Xilinx FPGA family of IC devices consists of an array of configurable logic blocks (CLBs), I/O blocks (IOBs), and a switching interconnect matrix, as illustrated in Fig. 7.26. The low-end FPGAs support three kinds of interconnects: direct, general-purpose, and long-line interconnections. The *direct interconnects* (not shown in Fig. 7.26) connect CLBs to adjacent CLBs for localized applications with minimum propagation delay. The *general-purpose interconnects* connect CLBs to other CLBs via the horizontal and vertical interconnect lines and switching matrices. Finally, the *long-line interconnects* are reserved for signals that must be distributed to numerous CLBs and/or IOBs with minimum time-delay distribution (time skew) problems. The programming of sophisticated devices such as Xilinx FPGAs requires the use of dedicated software such as XACT by Xilinx, Inc. FPGAs

**FIGURE 7.26**

Segment of a Xilinx 2000 series logic cell architecture showing configurable logic blocks (CLBs), I/O blocks (IOBs), switching matrices, and vertical and horizontal interconnect lines. Source: *The Programmable Logic Data Book*, 1994. Xilinx Inc., San Jose, CA 95124.

by other manufacturers, which differ significantly from those of Xilinx, require their own dedicated software.

Though Xilinx no longer produces the series 2000 FPGAs, a general description of these devices is useful to introduce the subject of FPGAs. Shown in Fig. 7.27 is the Xilinx series 2000 FPGA logic cell, which consists of a combinational logic section, six MUXs, and a memory element (flip-flop). (Note: The reader is not expected to have a knowledge of flip-flops, which are covered at length in Chapter 10.) The combinational logic section can generate any single function of four variables,  $F = G$ , or any two functions, each of

**FIGURE 7.27**

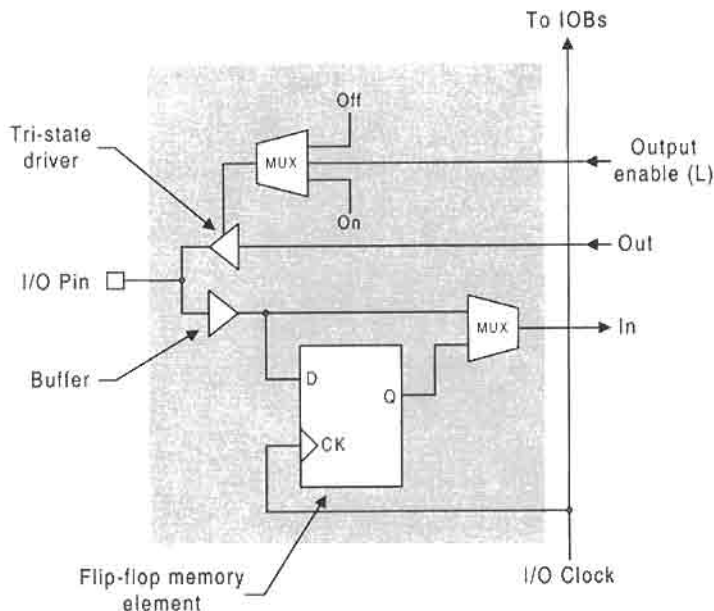
Logic structure for a Xilinx series 2000 logic cell block (LCB) with four data inputs, a clock input, and two outputs. Figure courtesy of Xilinx Inc., San Jose, CA.

three or fewer variables with separate  $F$  and  $G$  outputs. The functions are generated by a 16-bit programmable lookup table based on static RAM technology, thereby making reprogramming possible.

The simplest Xilinx IOB belongs to the 2000 series and is one consisting of two MUXs, a tri-state driver/enable, a noninverting buffer, and a flip-flop, as shown in Fig. 7.28. The I/O pin can function as either a dedicated input or a dedicated output, or it can be switched dynamically between the two. When the tri-state driver is disabled (disconnect mode), the I/O pin performs as a dedicated input. In this mode the 2-to-1 MUX selects either a buffered input from the flip-flop or one directly from the buffered input and delivers that input to the logic cell. When the tri-state driver is enabled (transfer mode), the I/O pin functions as a dedicated output from the logic cell. However, in this mode the 2-to-1 MUX can select the logic cell output via the buffer and return it to the logic cell as feedback, a bi-directional I/O condition.

The size and capability of the Xilinx FPGAs vary dramatically depending on the series and family to which the FPGA belongs. Shown in Table 7.1 are a few specifications for representative FPGAs of three families belonging to the XC4000 series. They range in CLB numbers from 100 for the XC4003E to 8464 for the XC40250XV. The XC40250XV features a gate range up to 500,000 (including RAM), has more than 100,000,000 transistors, and can operate at over 100 MHz. In comparison, the Pentium II microprocessor has 7,500,000 transistors but can operate at higher speeds.

Presented in Fig. 7.29 is the simplified architecture for the Xilinx XC4000 family of CLBs. Each CLB contains three function generators and two independent memory elements (flip-flops) that are triggered on either the rising edge or falling edge of the clock signal, depending on the logic level from the 2-to-1 MUXs. Multiplexers in the CLB map the four control inputs  $C1$ ,  $C2$ ,  $C3$ , and  $C4$  into the internal control signals  $H1$ ,  $Din$ ,  $S/R$ , and  $CK_{EN}$  in any required manner. Combinational logic can be extracted directly from the three

**FIGURE 7.28**

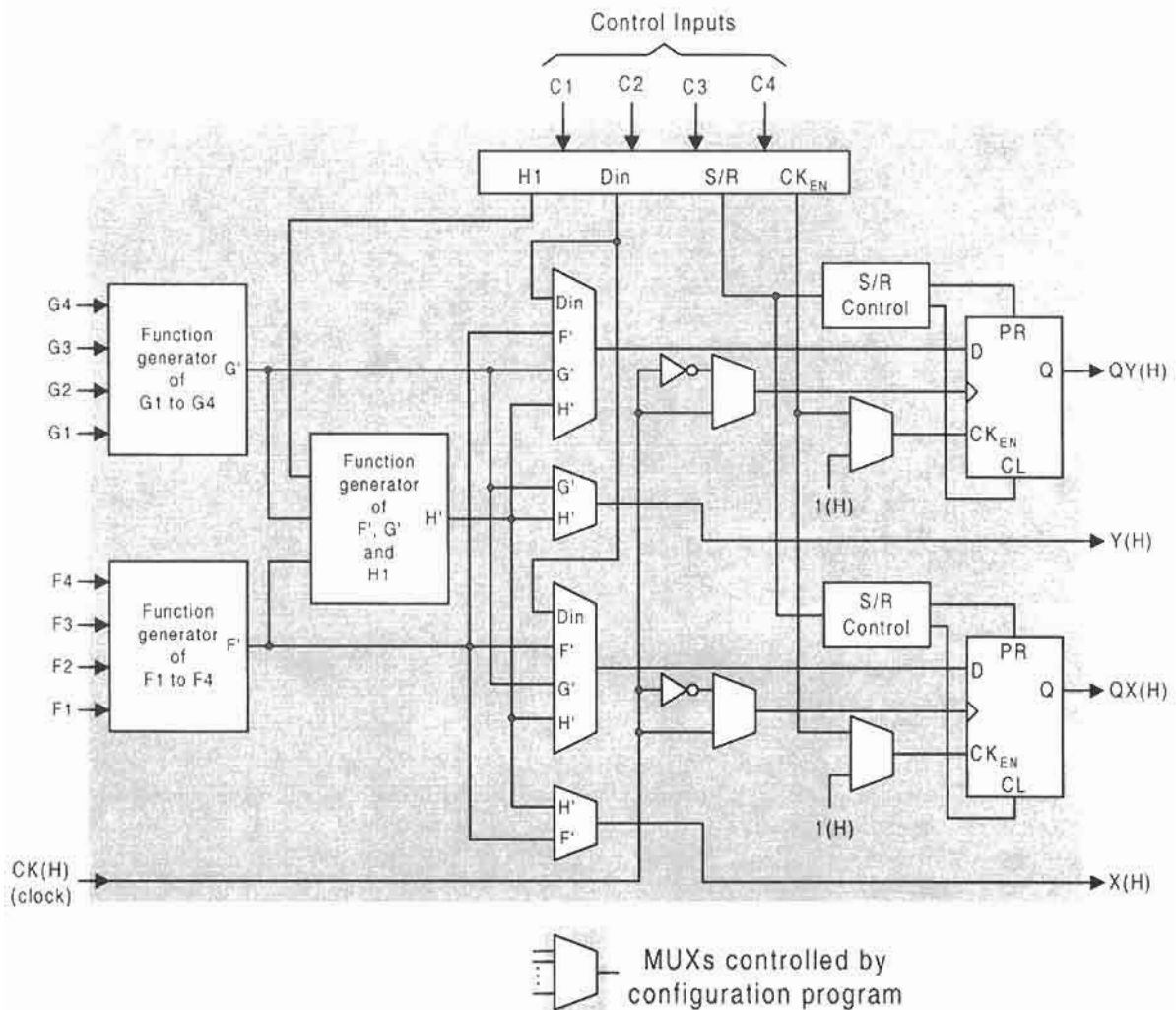
Logic structure for a relatively simple Xilinx I/O block (IOB). Figure courtesy of Xilinx Inc., San Jose, CA.

function generators at the  $X$  and  $Y$  outputs via the  $G'$ ,  $F'$  MUXs. By this means the CLB can implement any two functions of up to four variables, any function of five variables, or even some functions of up to nine variables. As an added advantage of the XC4000 series CLBs, function generators  $F'$  and  $G'$  contain dedicated arithmetic logic to increase the performance of the basic arithmetic and comparator operations.

The flip-flops generate outputs  $QX$  and  $QY$  from programmable data inputs via the 4-to-1 MUXs. The  $S/R$  control logic either presets ( $PR = 1$ ,  $CL = 0$ ) or clears ( $CL = 1$ ,  $PR = 0$ ) the flip-flops, depending on the logic level of the  $S/R$  input. A clock enable  $CK_{EN}$  input to the flip-flops permits the CLB to hold (store) data for an indefinite period of time. Again, it should be understood that the reader need not have a knowledge of flip-flops to obtain useful information from this subsection. The subject of flip-flops is covered at length in Chapter 10.

**Table 7.1 Range of Xilinx FPGAs Belonging to the XC4000 CMOS Series**

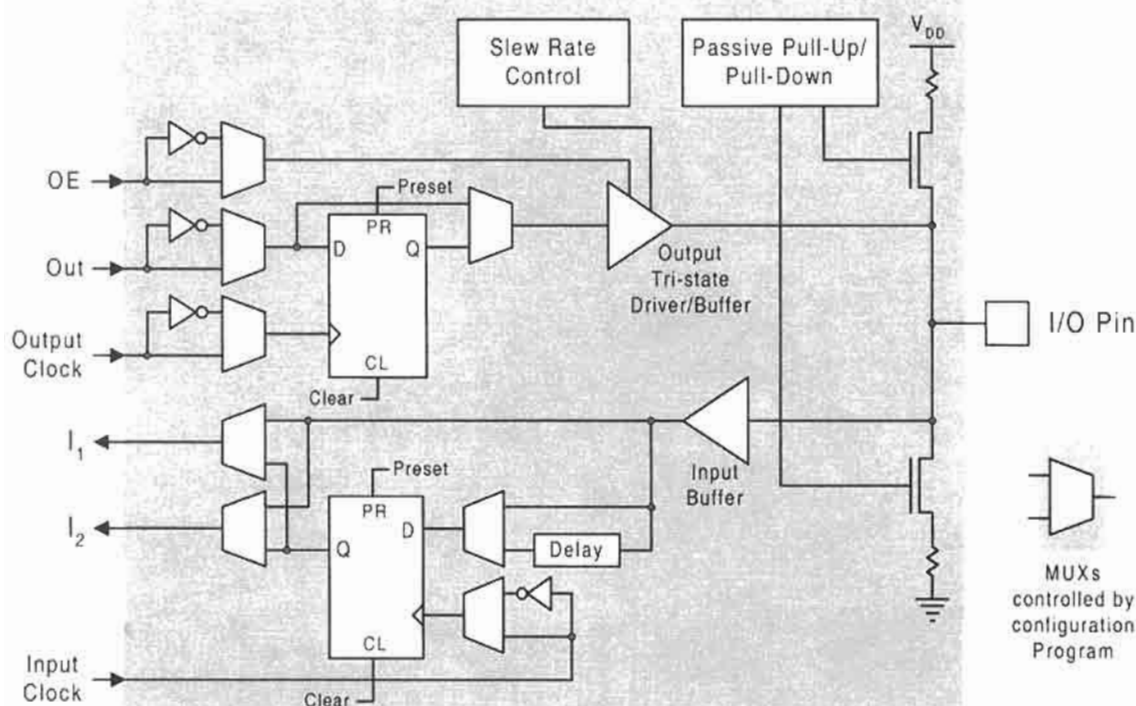
| Technology<br>(supply/scale<br>factor — maximum<br>frequency) | Product<br>name | Maximum<br>logic gates<br>(no RAM) | Maximum<br>RAM bits<br>(no logic) | CLB<br>matrix array    | Number<br>of flip-flops | Maximum<br>user I/O |
|---------------------------------------------------------------|-----------------|------------------------------------|-----------------------------------|------------------------|-------------------------|---------------------|
| 5 V/0.5 $\mu$ –66 MHz                                         | XC4003E         | 3 K                                | 3.2 K                             | $10 \times 10 = 100$   | 360                     | 80                  |
| 3.3 V/0.3 $\mu$ –80 MHz                                       | XC4085XL        | 85 K                               | 100 K                             | $56 \times 56 = 3,136$ | 7,168                   | 448                 |
| 2.5 V/0.25 $\mu$ –100 MHz                                     | XC40250XV       | 250 K                              | 271 K                             | $92 \times 92 = 8,464$ | 18,400                  | 448                 |

**FIGURE 7.29**

Simplified block diagram for the Xilinx XC4000 family of CLBs. Source: *The Programmable Logic Data Book*, Xilinx, Inc., 1994.

As expected, the I/O blocks (IOBs) for the XC4000 families of FPGAs are user-configurable, relatively complex, and versatile, just as are the CLBs. Shown in Fig. 7.30 is the simplified block diagram representing this family of IOBs. The buffered input signals from the I/O pin either are issued directly to the CLB (see inputs labeled  $I_1$  and  $I_2$ ), or are issued to the CLB from the flip-flop output  $Q$  after an optional delay, all determined by 2-to-1 MUXs. The delay can be used to eliminate the need for a data hold-time requirement at the external pin.

The output signals from the CLB can be inverted or not inverted and can either pass directly to the I/O pad or be stored in the memory element (flip-flop). The output enable OE signal acts on the tri-state driver/buffer to produce either a bidirectional I/O capability or a

**FIGURE 7.30**

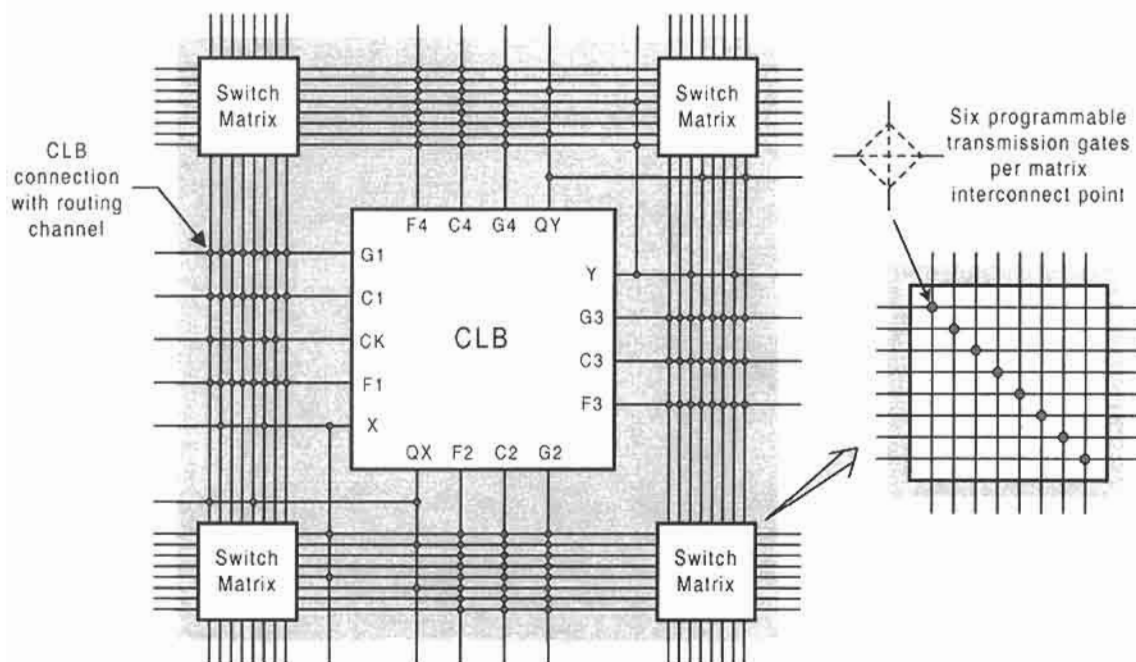
Simplified block diagram for the XC4000 series input/output block (IOB). Source: *The Programmable Logic Data Book*, Xilinx, Inc., 1994.

monodirectional input capability by imposing a high-impedance condition on the tri-state driver. The slew-rate control can also act on the tri-state driver, but does so to minimize the power consumption from bus transients when switching noncritical signals.

There are many other features and programmable options of the IOB. These include programmable pull-up and pull-down resistors that connect unused I/O pins (via NMOS) to either  $V_{DD}$  or ground to minimize power consumption. Separate input and output clock signals can be inverted or not inverted to produce rising- or falling-edge triggering of the flip-flops. Also, the flip-flops can be globally preset or cleared as is the case for the CLBs. Other characteristics of the IOBs, as well as those of the CLBs, are best understood by consulting Xilinx's *Programmable Logic Data Book* (see Further Reading at the end of this chapter).

The matrix of programmable interconnects for the Xilinx XC4000 families of FPGAs are significantly different from and more complex than those of the discontinued XC2000 series. For the XC4000 series there are three main types of interconnects: *single-length lines*, *double-length lines*, and *longlines*. A typical routing scheme for CLB connections to adjacent single-length lines is illustrated in Fig. 7.31. The switch matrix consists of six programmable transmission gates at the intersection of each single-length line as shown in the figure. The transmission gate configuration permits a given line signal to be routed



**FIGURE 7.31**

Interconnect scheme between CLB and routing channels consisting of single-length lines. Also shown are the six programmable transmission gate positions per interconnect point within each switch matrix. Source: *The Programmable Logic Data Book*, Xilinx, Inc., 1994.

in any one of three directions, in one of two transverse directions or on the same line, or along any combination of these. The single-length lines are used primarily for local network branching with fan-out greater than 1. The routing channels are designed to minimize path delay and power consumption, and their number between switch matrices varies with the size of the matrix.

Double-length lines (not shown) are those that interconnect every other switch matrix and are, therefore, twice as long as the single-length lines shown in Fig. 7.31. The double-length lines, which pass by two CLBs before entering a switch matrix, are grouped in pairs and offer the most efficient implementation of point-to-point interconnections of intermediate length. Any CLB input, except CK, can be driven from adjacent double-length lines, and the CLB outputs can be routed to nearby double-length lines in either horizontal and vertical directions.

The longlines (also not shown in Fig. 7.31) run the length of the array in both vertical and horizontal directions. They are designed to distribute signals of various types throughout the array with minimum time delay (skew) problems. Networks with high fan-out and time-critical signals are ideally suited for longline usage. A programmable switch located at the center of each intersecting longline permits a signal to be split into two independent signals, both traveling half the vertical or horizontal length of the array. Inputs to the CLB can be driven by adjacent longlines, but outputs from the CLB can be routed to adjacent



longlines only via tri-state drivers. Programmable interconnect points between longlines and single-length lines are available. However, no interconnect points are provided between double-length lines and others.

Logic cell arrays (LCAs), such as the XC4000 series FPGAs, have the unique property that they can be reconfigured within a system and can even be dynamically altered to perform different functions at different times in a process. This family of devices contain  $16 \times 1$  and  $32 \times 1$  static RAM primitives that are user-configurable via look-up tables. Self-diagnosis, hardware for different environments, or dual-purpose applications — these exemplify the versatility that is inherent in reconfigurable LCAs. Properly used, such devices can minimize design effort and reduce costs. However, all of the above are possible only with the use of CAD help, the topic of Section 7.8. For much more complete information on Xilinx FPGAs, the reader is referred to Further Reading at the end of this chapter.

#### 7.7.4 Other Classes of General-Purpose PLDs

To provide the reader with an overall perspective on the diversity of general-purpose PLDs and for reference purposes, the following additional classes of complex PLDs (CPLDs) are offered as an extension of the devices discussed in the previous two subsections:

**Generic Array Logic (GAL) Devices:** Erasable MSI PLDs that may contain AOIs, XOR gates, and registers in the output stage for sequential machine design. GAL is a registered trademark of Lattice Semiconductor, Hillsboro, OR 97124.

**Erasable programmable logic devices (EPLDs):** Erasable CMOS-based devices whose macrocells typically contain discrete gates, MUXs, and registers (flip-flops) for sequential machine design. Some EPLDs may contain arithmetic logic units (ALUs). Both Altera and Xilinx Corporation offer EPLDs of various complexity. Detailed information on EPLDs are available from the *Altera Data Book*, Altera Corporation, and from *The Programmable Logic Data Book*, Xilinx Corporation, 1994.

**Field programmable logic sequencers (FPLS):** Similar to PAL and GAL devices that have output logic consisting of discrete gates and flip-flops. Detailed information on these devices is available from Phillips, *Programmable Logic Devices (PLD) Handbook*, Phillips Semiconductor, Sunnyvale, CA, 1994; and *Programmable Logic Data Handbook*, Signetics Co., Sunnyvale, CA, 1992.

### 7.8 CAD HELP IN PROGRAMMING PLD DEVICES

The programming of a PAL IC device, like that of a ROM or PLA, can be accomplished by a physical device (a programmer) that applies voltage pulses to target fusible links, causing disconnection of the bit positions as discussed in Section 7.2. The programmer may be a dedicated device or one of universal capability for ROMs, PLAs, and PAL devices, and it may even have erasure capability. In any case, the difficult part of the programming process is providing the instructions required to fit the Boolean expressions into the capability of the specific PLD device, for example, a PAL device, that may support an L-, R-, or V-type

macrocell structure. This difficulty is overcome by proprietary software from manufacturers of PLD devices and PLD device programmers. A commonly used CAD package for this purpose is called ABEL (for Advanced Boolean Expression Language, a product of DATA I/O Corp., Redmond, WA). The ABEL compiler accepts I/O data in tabular form, minimizes it by using algorithms based on Espresso (discussed in Section 4.8), and generates a *fuse map* that can be read in one of several standard formats by the programming device: JEDEC (Joint Electron Device Engineering Council) format, ASCII hex format, Intel Hex format, etc. As the name implies, the fuse map (pattern) targets those fusible links that must be disconnected.

ABEL is not the only PLD programming compiler that is currently in use for PAL devices. A CAD software called PALASM (for PAL assembler) is commonly used to convert SOP Boolean expressions or truth tables into fuse maps for PAL devices. I/O pin specifications on the chip are required. Like ABEL, the PALASM compiler generates a fuse map in JEDEC format that can be read by the PAL programming device. PALASM is available without cost from AMD Inc. (Advanced Micro Devices, Inc.).

For Xilinx FPGA devices, dedicated and sophisticated CAE (computer-aided engineering) software called XACT (for Xilinx automated CAE tools) is available from the Xilinx Corp. XACT uses a three-step interrelated and iterative design process: *design entry*, *design implementation*, and *design verification*. For design entry at the gate level, the designer may begin with a schematic capture and labeling of the circuit to be implemented. To do this the designer can choose Viewlogic's Viewdraw, Mentor Graphics' Design Architect, or OrCAD's SDT, since these are all supported by the XACT development system. Design entry at the behavioral level (for example, Boolean equations or state machine descriptions) is best accomplished by the Xilinx-ABEL and X-BLOX products of Xilinx and other products by CAE vendors. Also, for complex systems, the use of high-level hardware description languages such as VHDL is recommended. Xilinx offers HDL interfaces for synthesis tools from synopsis and Viewlogic Systems. Mentor Graphics, Cadence Design Systems, and Exemplar Logic also offer HDL synthesis tools fashioned for Xilinx FPGAs.

One advantage of the Xilinx design environment is that the designers can combine schematic, text, gate-level, and behavioral-level descriptions at the design entry stage and then reuse such descriptions within the same system or in other systems at some future time. This is called *mix-mode design entry* and can be carried out by using different design entry tools, allowing the designer to choose the most effective and efficient design tool for each portion of the design.

Following the design entry comes the design implementation. Here, the FPGA design entry tools are mapped into the resources of the target device architecture, optimally selecting the routing channels that connect the CLBs and IOBs of the logic cell array. Although this can be accomplished automatically, the designer can and should exert control over the implementation process to minimize potential problems. For this purpose the Xilinx design environment provides an interactive, graphics-based editor that permits user manipulation of the logic and routing schemes for the FPGA device.

The design verification is the last part of the design process and necessarily follows the implementation stage. In-circuit testing, simulation at various levels, and static timing analysis are involved in the verification stage. This is accomplished by use of timing calculators, back-annotation capability, and static timing analyzers, which are available from Xilinx Corp. and various third-party vendors.

## FURTHER READING

Any recent text will have some coverage of the basic PLDs: ROMs, PLAs, and PAL devices. However, there are a few texts that appear to cover these subjects better than most. The text by Nelson, Nagle, Carroll, and Irwin and that by Yarbrough appear to cover the basics rather well and extend their coverage into the industrial level. Other important books that deal strictly with digital design with PLDs are those of Pellerin and Holley, Lala, Bolton, and Carter. The text by Tinder appears to be the only one dealing with mixed logic inputs to and outputs from PLDs. For multiple PLD schemes for augmenting input and output capability, the text by Tinder and that by Ercegovac and Lang are recommended. Data handbooks are often a necessary source for detailed current information, and none is better than that for PAL devices by Advanced Micro Devices.

- [1] M. Bolton, *Digital Systems Design with Programmable Logic*. Addison-Wesley, Reading, MA, 1990.
- [2] J. W. Carter, *Digital Designing with Programmable Logic Devices*. Prentice Hall, Upper Saddle River, NJ, 1997.
- [3] M. D. Ercegovac and T. Lang, *Digital Systems and Hardware/Firmware Algorithms*. John Wiley, New York, 1985.
- [4] P. K. Lala, *Digital System Design Using Programmable Logic Devices*. Prentice Hall, Englewood Cliffs, NJ, 1990.
- [5] V. P. Nelson, H. T. Nagle, B. D. Carroll, and J. D. Irwin, *Digital Logic Circuit Analysis and Design*. Prentice Hall, Englewood Cliffs, NJ, 1995.
- [6] D. Pellerin and M. Holley, *Practical Design Using Programmable Logic*. Prentice Hall, Englewood Cliffs, NJ, 1991.
- [7] *PAL Device Data Book*. Advanced Micro Devices, Inc., Sunnyvale, CA, 1992.
- [8] R. F. Tinder, *Digital Engineering Design: A Modern Approach*. Prentice Hall, Englewood Cliffs, NJ, 1991.
- [9] J. M. Yarbrough, *Digital Logic: Applications and Design*. West Publishing Co., Minneapolis/St. Paul, 1997.

The subject of FPGAs is covered adequately by several recent texts and more extensively by the device manufacturers of these devices. For introductory material on FPGAs, the texts by Katz and Yarbrough (previously cited) are adequate. However, for current detailed information, no sources are better than the recent data books from Xilinx, Actel, and Altera. For the most recent information on the Xilinx CX4000XV family of FPGAs, the world wide web is perhaps the best source. GAL devices are covered by Lattice Semiconductor's data book. For EPLD component specifications and applications, the reader will find Intel's data book useful.

- [10] *ACT Family Field Programmable Gate Array Databook*. Actel Corp., Sunnyvale, CA, 1991.
- [11] *Altera Data Book*. Altera Corp., San Jose, CA, 1995.
- [12] *GAL Data Book*. Lattice Semiconductor, Hillsboro, OR, 1992.
- [13] <http://www.xilinx.com/spot/virtexspot.htm>
- [14] R. H. Katz, *Contemporary Logic Design*. The Benjamin/Commings Publishing Co., Inc., Redwood City, CA, 1994.
- [15] *Programmable Gate Array Data Book*. Xilinx, Inc., San Jose, CA, 1995.
- [16] *Programmable Logic Data Book*. Intel Corp., Santa Clara, CA, 1994.

- [17] *The Programmable Logic Data Book*. Xilinx, Inc., San Jose, CA, 1996.  
 [18] *XACT, Logic Cell Array Macro Library*. Xilinx, Inc., San Jose, CA, 1992.

## PROBLEMS

- 7.1** A  $2^4 \times 4$  ROM is to be used to implement the following system of three functions, assuming that all inputs and outputs are active high.

$$y_1(a, b, c, d) = \sum m(0, 1, 2, 5, 7, 8, 10, 14, 15)$$

$$y_2(a, b, c, d) = \sum m(0, 2, 4, 5, 6, 7, 8, 10, 12)$$

$$y_3(a, b, c, d) = \sum m(0, 1, 2, 3, 4, 6, 8, 9, 10, 11)$$

- (a) Construct the ROM program table for this system of three functions.
  - (b) From the program table of part (a), construct the symbolic representation of fusible links by following the example in Fig. 7.4.
- 7.2** A  $2^4 \times 4$  ROM is to be used to design and implement a BCD-to-binary module that can be cascaded to produce any size of a BCD-to-binary converter (e.g., see Fig. 6.21).
- (a) Construct the ROM program table for this module following Fig. 6.20.
  - (b) From the program table in part (a), construct a symbolic representation of the fusible links by referring to the example in Fig. 7.4.
- 7.3** A  $2^4 \times 4$  PROM is to be used to implement the following system of three functions:

$$F_1(A, B, C) = \prod M(0, 2, 4)$$

$$F_2(A, B, C, D) = \prod M(3-12)$$

$$F_3(A, B, C) = \prod M(2, 3, 5, 7)$$

- (a) Construct the program table for this ROM if the inputs arrive as  $A(H)$ ,  $B(H)$ ,  $C(L)$ , and  $D(H)$ , and the outputs are  $F_1(H)$ ,  $F_2(L)$ , and  $F_3(H)$ . Use of inverters is *not* permitted.
  - (b) From the program table in part (a), construct a symbolic representation of the fusible links following the example in Fig. 7.4.
- 7.4** A multiplexed scheme of  $2^6 \times 4$  EPROMs is to be used to design and implement a circuit that will convert 8-bit one-hot code (see Table 2.11) to 4-bit binary. Assume that all false data are rejected and indicated by binary 1000.
- (a) In place of a ROM program table, give the coded canonical SOP forms for each binary output.
  - (b) Illustrate with a block diagram how the outputs of the multiplexed scheme must be wired-ORed to produce the four binary outputs of the converter. (Refer to Section 7.6 and Figs. 7.14 and 7.15 for a discussion of multiplexed schemes and wired-ORed connections.)

**7.5** Design an XS3-to-Gray code converter by using a  $4 \times 8 \times 4$  FPLA. Assume that all inputs and outputs are active high and that false data are *not* rejected.

- Construct the minimized p-term table for this converter.
- From the p-term table in part (a), construct the symbolic representation for the fusible links following the example in Fig. 7.10.

**7.6** The following three functions are to be implemented by using a  $4 \times 8 \times 4$  FPLA.

$$F_1 = A\bar{B} + \bar{A}B\bar{C} + \bar{B}C + AC$$

$$F_2 = A \oplus (\bar{B}C)$$

$$F_3(A, B, C) = \prod M(1, 3, 6)$$

- Construct the minimized p-term table for the three functions. Assume that the inputs arrive as  $A(H)$ ,  $B(H)$ , and  $C(L)$  and that the outputs are issued as  $F_1(H)$ ,  $F_2(L)$ , and  $F_3(H)$ .
- From the p-term table in part (a), construct the symbolic representation for the fusible links following the example in Fig. 7.10. An inverter is permitted on the active low input.

**7.7** A BCD-to-XS3 code converter is to be designed by using a  $4 \times 12 \times 4$  PAL.

- Construct the minimized p-term table for this converter. Assume that the inputs and outputs are all active low, and that all false data are encoded as 0000. Keep in mind that PALs cannot take advantage of shared PIs as can PLAs.
- From the p-term table of part (a), construct the symbolic representation for the fusible links following the example in Fig. 7.11. Inverters may be used on the inputs.

**7.8** A cascading 2-bit comparator is to be designed by using a PAL.

- Given the compressed truth table and EV K-maps in Fig. 6.28, find the minimum SOP logic expressions for the three outputs and construct the minimum p-term table from these expressions. Assume that all inputs and outputs are active high.
- From the p-term table in part (a), determine the minimum size PAL that can be used and then construct the symbolic representation of the fusible links for this comparator. Include tri-state enables.

**7.9** The three functions in Problem 7.1 are to be designed by using a  $4 \times 16 \times 4$  PAL.

- Construct the minimized p-term table for these three functions keeping in mind that a PAL cannot take advantage of shared PIs as can PLAs. Assume that the inputs arrive as  $a(L)$ ,  $b(L)$ ,  $c(H)$ , and  $d(H)$ , and that the outputs must be issued as  $y_1(H)$ ,  $y_2(L)$ , and  $y_3(H)$ . Note that inverters are *not* permitted on the inputs.
- From the program table of part (a), construct the symbolic representation of fusible links by following a form similar to that of Fig. 7.11, but with tri-state enables on the outputs.

**7.10** The Actel (ACT-1) logic module, shown in Fig. 7.24, is embedded by the hundreds in Actel's FPGAs. This module is remarkably versatile in its ability to implement a large number of simple SOP functions active high, or POS functions active low. As

examples, implement each of the following logic functions by using a single ACT-1 module assuming all inputs are active high: [Hint: Use Fig. 7.24, not Eq. (7.5), and plan to include 0's and 1's as inputs where needed.]

- (a)  $Y(H) = (\bar{A} + \bar{B})(H) = A \cdot B(L)$  A two-input NAND gate
- (b)  $Y(H) = \bar{A}\bar{B}C(H) = (A + B + \bar{C})(L)$  A p-term (or s-term)
- (c)  $Y(H) = (\bar{A}B + A\bar{B})(H) = [(\bar{A} + \bar{B})(A + B)](L)$  Defining relations for XOR
- (d)  $Y(A, B, C)(H) = \Sigma m(2, 3, 5, 7)(H)$  Canonical SOP function

**7.11** The AOI and OAI gates in Figs. 7.21, 7.22, and 7.23 are versatile building blocks that can be used in the implementation of a variety of logic functions. Simple examples are given in Subsection 7.7.1. With a minimum of external logic, apply these AOI and OAI gates in creative ways to implement the following functions:

- (a) Use one AOI gate (nothing else) to implement the expression for  $(A = B)(H)$  given by Eq. (6.22).
- (b) Use a minimum number of AOI gates to implement the three-function system in Problem 7.6 with input and output activation levels as stated. (Hint: For  $F_1$ , use Fig. 7.23)
- (c) Use a minimum number of OAI gates to implement the three-function system in Problem 7.3 with input and output activation levels as stated. (Hint: For  $F_2$ , use the dual of Fig. 7.23.)

*This page intentionally left blank*

# Arithmetic Devices and Arithmetic Logic Units (ALUs)

---

## 8.1 INTRODUCTION

In this chapter digital circuits will be designed with electrical capabilities that can be interpreted as performing the basic arithmetic operations of binary numbers. The basic operations include

- Addition
- Subtraction
- Multiplication
- Division

Now, Boolean equations are uniquely defined so as to perform specific arithmetic operations, and the 1's and 0's, which have previously been used only as logic levels, must take on a numerical significance. The reader must keep in mind that an arithmetic circuit is only the electrical analog of the arithmetic operation it represents. In fact, it is the interpretation of the electrical circuit's behavior that bridges the gap between physical and logic domains.

The treatment of arithmetic circuits presented in this text is not intended to be a treatise on the subject. Rather, the intent is to introduce the subjects at both the beginning and intermediate-to-advanced levels utilizing, where necessary, appropriate algorithms for the basic operations in question. The subjects of arithmetic logic units (ALUs) and the application of dual-rail methods, which are covered in later sections, fall within the intermediate-to-advanced level of coverage and may not be expected to be part of a first-level course in digital design.

## 8.2 BINARY ADDERS

Because of the nature of binary and the requirements for arithmetic manipulation, approaches to basic adder design vary considerably depending on the form the manipulation takes. There are ripple-carry adders, carry-save, carry select, and carry-look-ahead adders,



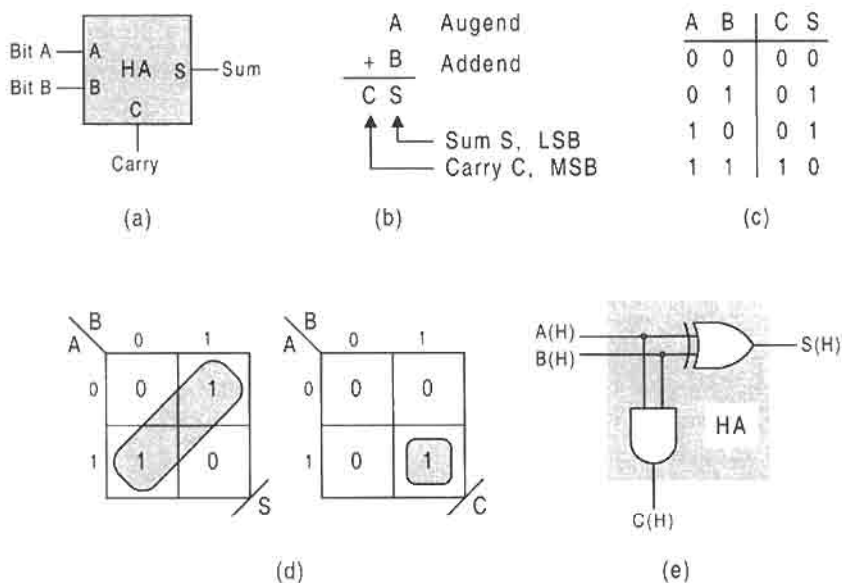
the last three being classified as “high-speed” adders. This list of four does not nearly cover the scope of adder design, nor does it establish the general character of these arithmetic devices. It does introduce the concept of computational speed as it relates to the addition process, however that is characterized.

### 8.2.1 The Half Adder

The *half adder* (HA) is the simplest of all the arithmetic circuits and may be regarded as the smallest building block for modular design of arithmetic circuits. The HA consists of two inputs,  $A$  and  $B$ , and two outputs, sum  $S$  and carry  $C$ , as indicated by the logic circuit symbol in Fig. 8.1a. The operation format and truth table for the HA are given in Fig. 8.1b and 8.1c. Here,  $A$  plus  $B$  yields the result  $CS$ , where carry is the MSB and sum is the LSB. When mapped as in Fig. 8.1d, the results for sum and carry are read directly as

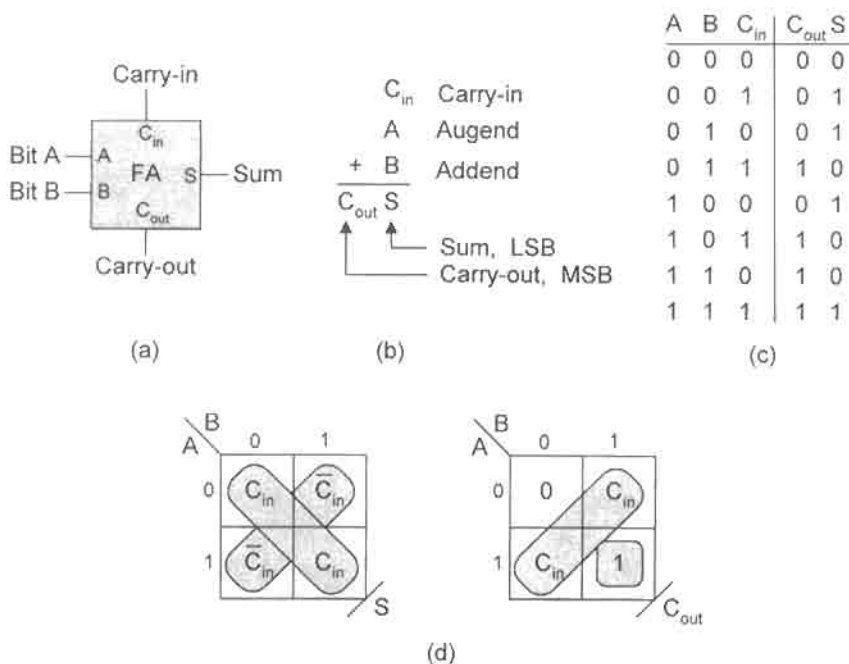
$$\begin{cases} S = A \oplus B \\ C = A \cdot B \end{cases} \quad (8.1)$$

The logic circuit for the HA is implemented by using Eqs. (8.1) and is given in Fig. 8.1e. Here, the choice is made to use the XOR gate in the implementation of the HA. However, use could have been made of the two- or three-level XOR realizations given in Fig. 3.28, or the transmission gate approach by using AOI or OAI gates as in Fig. 7.21 and 7.22. The XOR gate, given in Fig. 3.26, is the simplest CMOS design possible.



**FIGURE 8.1**

Design of the half adder (HA). (a) Block diagram for the HA. (b) Operation format. (c) Truth table for  $A$  plus  $B$ , showing carry  $C$ , and sum  $S$ . (d) K-maps for sum and carry. (e) Logic circuit for the HA.

**FIGURE 8.2**

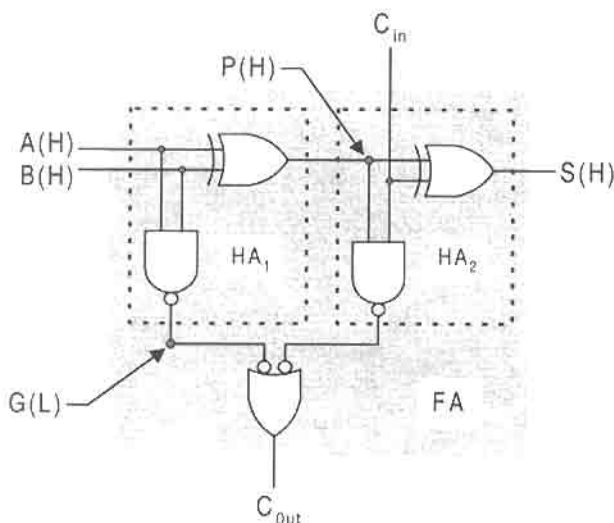
Design of the full adder (FA). (a) Block diagram for the FA. (b) Operation format for the FA. (c) Truth table for  $A$  plus  $B$  plus  $C_{in}$ , showing carry-out  $C_{out}$ , and sum  $S$ . (d) EV K-maps for sum and carry-out.

### 8.2.2 The Full Adder

The half adder (HA) just designed has severe limitations in modular design applications because it cannot accept a carry-in from the previous stage. Thus, the HA cannot be used for multiple bit addition operations. The limitations of the HA are easily overcome by using the *full adder* (FA) presented in Fig. 8.2. The FA features three inputs,  $A$ ,  $B$ , and carry-in  $C_{in}$ , and two outputs, sum  $S$  and carry-out  $C_{out}$ , as indicated by the logic symbol and operation format in Figs. 8.2a and 8.2b. The truth table for  $A$  plus  $B$  plus  $C_{in}$  is given in Fig. 8.2c with outputs  $C_{out}$  and  $S$  indicated in the two columns on the right. As in the case of the HA, the inputs are given in ascending binary order. EV K-maps for sum and carry-out in Fig. 8.2d, showing diagonal XOR patterns, are plotted directly from the truth table and give the results

$$\left\{ \begin{array}{l} S = C_{in}(\bar{A} \oplus \bar{B}) + \bar{C}_{in}(A \oplus B) \\ \quad = A \oplus B \oplus C_{in} \\ C_{out} = C_{in}(A \oplus B) + AB \end{array} \right\}. \quad (8.2)$$

Here, multioutput optimization has been used to the extent that the term  $(A \oplus B)$  is used by both the  $S$  and  $C_{out}$  expressions. Recall that  $A \oplus B \oplus C_{in} = (A \oplus B) \oplus C_{in}$ .

**FIGURE 8.3**

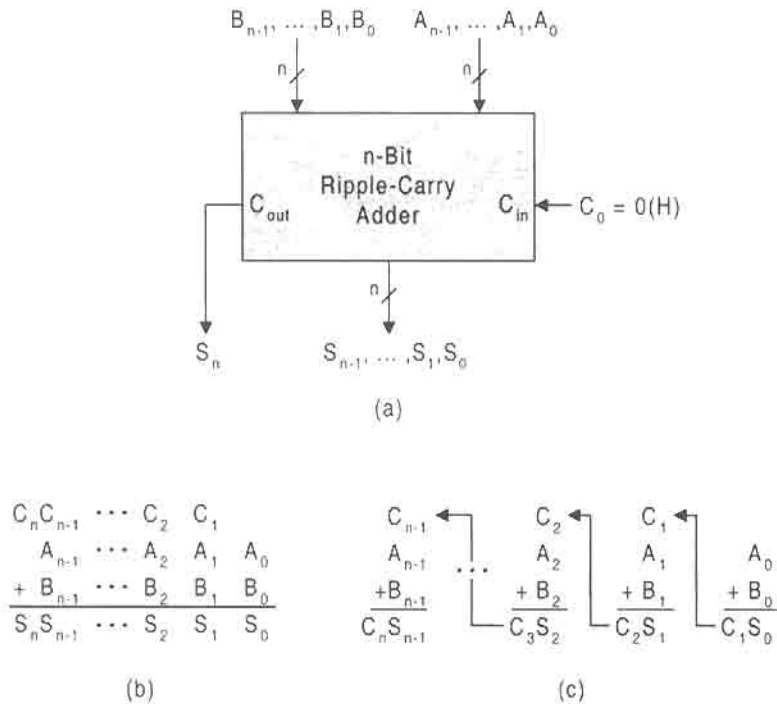
Logic circuit for the full adder (FA) showing it to be composed of two HAs and a NAND gate.

The output expressions in Eqs. (8.2) are used to implement the FA shown in Fig. 8.3. As indicated, the FA is composed of two HAs and a NAND gate. The nodes  $P(H)$  and  $G(L)$  are indicated for later reference in Section 8.4. Notice that the NAND logic in Fig. 8.3 can be replaced by AND/OR logic but, for reasons to be discussed in Section 8.5, are kept as NAND.

### 8.2.3 Ripple-Carry Adders

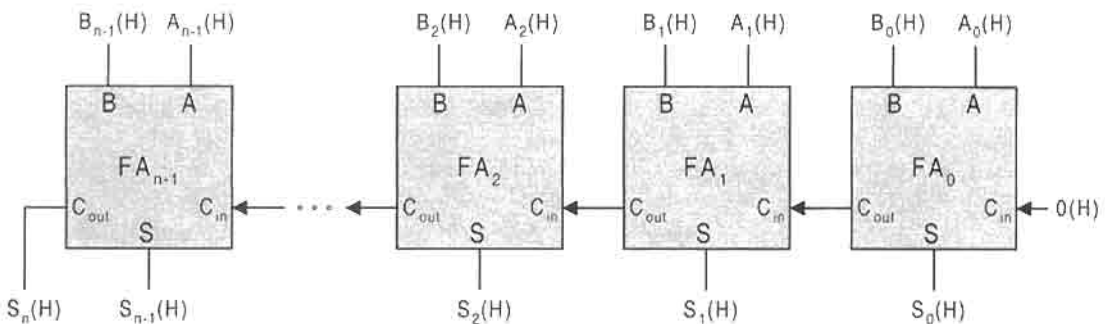
The full adder (FA) can now be used to design a *ripple-carry adder*, sometimes called a *pseudoparallel adder* or simply *parallel adder*. An  $n$ -bit ripple-carry (R-C) adder is a  $(2n + 1)$ -input/ $(n + 1)$ -output combinational logic device that can add two  $n$ -bit binary numbers. The block diagram symbol and general operation format for this adder are presented in Figs. 8.4a and 8.4b, together with an illustration of the ripple-carry effect in Fig. 8.4c. The general operation format represents the familiar addition algorithm used in conventional arithmetic where carry from an addition operation is always to the next most significant stage. Notice that the subscripts are consistent with the powers of 2 to the left of the radix point in polynomial notation as is discussed in Section 2.3. Thus, the bits of each word representing a number are written in ascending order of positional weight from right to left. The addition process follows Algorithm 2.8 given in Section 2.9. Actually, the position of the radix point in the two numbers is arbitrary, since the adder has no means of sensing these positions. The only requirement is that the user of this adder make certain the radix points “line up” just as in conventional arithmetic. Thus, if significant bit positions exist to the right of the radix point for augend  $A$  and addend  $B$ , meaning that these numbers have a fraction component, then there must be an equal number of such positions for the two numbers, each of  $n$  bits total.

The modular design of the  $n$ -bit ripple-carry adder follows directly from Fig. 8.4. All that is required is that a series array of  $n$  FAs designated  $FA_0, FA_1, \dots, FA_{n-1}$ , one for

**FIGURE 8.4**

The  $n$ -bit ripple-carry adder. (a) Block diagram circuit symbol. (b) Operation format. (c) Ripple carry effect.

each bit, be connected such that the carry-out of one stage is the carry-in to the next most significant stage. This connection is shown in Fig. 8.5 where it is assumed that all inputs arrive active high. Notice that the condition of no carry to the initial stage ( $FA_0$ ) is satisfied by  $C_0 = 0(H)$  (ground) or by using a HA for this stage.

**FIGURE 8.5**

An  $n$ -bit ripple-carry adder implemented with  $n$  full adders.

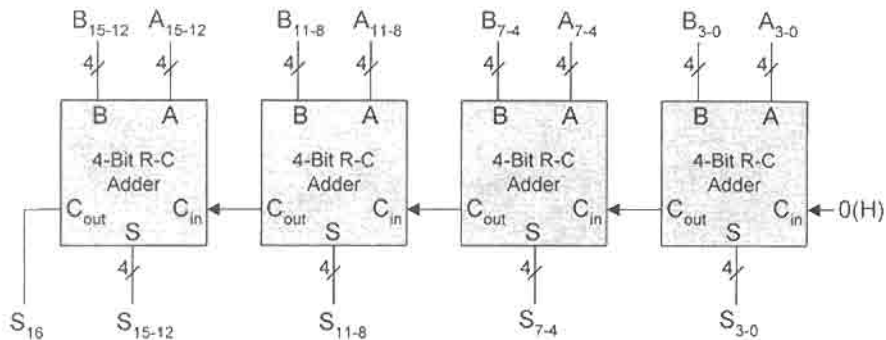


FIGURE 8.6

A 16-bit ripple-carry adder implemented with four 4-bit ripple-carry (R-C) adders.

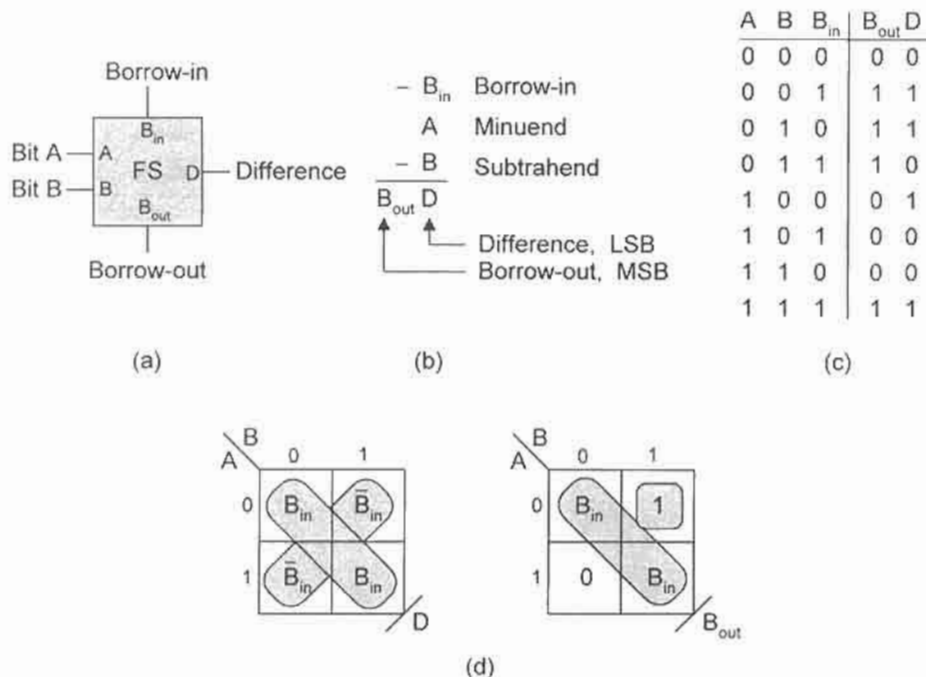
An  $n$ -bit ripple-carry adder is more likely to be designed by using  $n$   $m$ -bit adder modules rather than individual FAs. One example, represented in Fig. 8.6, features four 4-bit ripple-carry (R-C) adders in ripple-carry fashion to produce a 16-bit adder. Other examples include a 16-bit R-C adder composed of two 8-bit R-C adders, or a 32-bit implemented with four 8-bit R-C adders.

There are practical limitations associated with the operation of an R-C adder of the type presented in Figs. 8.5 and 8.6. To operate an R-C adder correctly, *all* input bits, for augend  $A$  and addend  $B$ , must be presented stably to the adders for a time equal to or greater than the time it takes for the carry signal to ripple from stage to stage across the entire adder. Thus, this ripple time determines the maximum frequency of input data presentation (to the adder stages) for a series of addition operations. Remember that an addition operation in an R-C adder is not complete until the carry signal passes through the last (MSB) stage of the adder. Discussions of ways to speed up the addition process will be considered later in Sections 8.4 and 8.5.

### 8.3 BINARY SUBTRACTORS

The electrical equivalent of the binary subtraction operation can be carried out by using full subtractors, just as the electrical equivalent of binary addition can be accomplished by using full adders. Although this is rarely done, the design of the *full subtractor* provides a good introduction to the more useful subject of adder/subtractors (devices that can serve in either capacity) and to the subject of parallel dividers considered in Section 8.7.

The design of the full subtractor (FS) follows in a manner similar to the design of the FA with, of course, some important differences. The inputs now become the minuend  $A$ , subtrahend  $B$ , and borrow-in  $B_{in}$ , and the outputs become the difference  $D$  and borrow-out  $B_{out}$ , which are shown by the block symbol in Fig. 8.7a. The operation format for the FS and truth table for  $A - (B \text{ plus } B_{in})$  are given in Figs. 8.7b and 8.7c. Notice that  $B_{out} = 1$  any time that  $(B \text{ plus } B_{in}) > A$ , and that the difference  $D = 1$  any time the three inputs exhibit odd parity (odd number of 1's) — the same as for sum  $S$  in Fig. 8.2c.

**FIGURE 8.7**

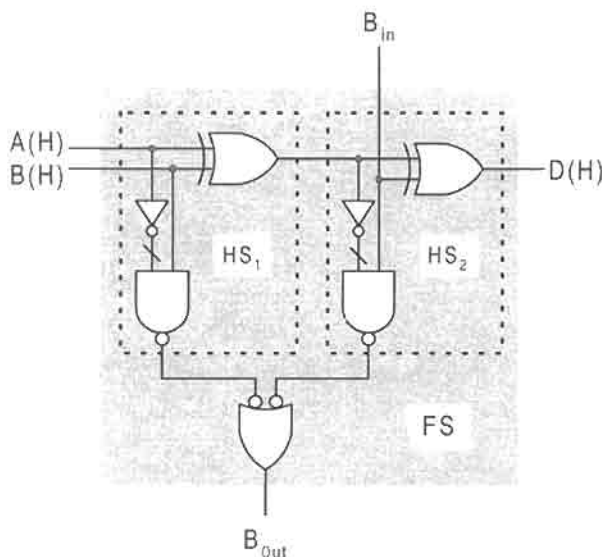
Design of the full subtractor (FS). (a) Block diagram for the FS. (b) Operation format. (c) Truth table for  $A - (B \text{ plus } B_{in})$ , showing outputs borrow-out  $B_{out}$  and difference  $D$ . (d) EV K-maps for difference and borrow-out.

The EV K-maps for difference and borrow-out are plotted directly from the truth table in Fig. 8.7c and are given in Fig. 8.7d, where again XOR diagonal patterns exist. The outputs, as read from the K-maps, are

$$\left\{ \begin{array}{l} D = A \oplus B \oplus B_{in} \\ B_{out} = B_{in}(\bar{A} \oplus \bar{B}) + \bar{A}B \end{array} \right. \quad (8.3)$$

where it is recalled from Eqs. (3.23) that  $A \odot B = \bar{A} \oplus \bar{B}$ . The FS can now be implemented from Eqs. (8.3) with the results shown in Fig. 8.8. Observe that the FS consists of two half subtractors (HSs) and that the only difference between a HS and a HA is the presence of two inverters in the NAND portion of the FS circuit. Use will be made of this fact in the next section dealing with the subject of adder/subtractors, devices that can perform either addition or sign-complement subtraction.

Full subtractors can be cascaded in series such that the borrow-out of one stage is the borrow-in to the next most significant stage. When this is done, a *ripple-borrow subtractor* results, similar to the ripple-carry adders of Figs. 8.5 and 8.6. However, the ripple-borrow subtractor suffers from the same practical limitation as does the ripple-carry adder — namely, that the subtraction process is not complete until the borrow signal completes

**FIGURE 8.8**

Logic circuit for the full subtractor (FS) showing it to be composed of two half subtractors (HSs) and a NAND gate.

its ripple path across the entire subtractor. As with the ripple-carry adder, the ripple-borrow subtractor is limited to applications not requiring high-speed calculations.

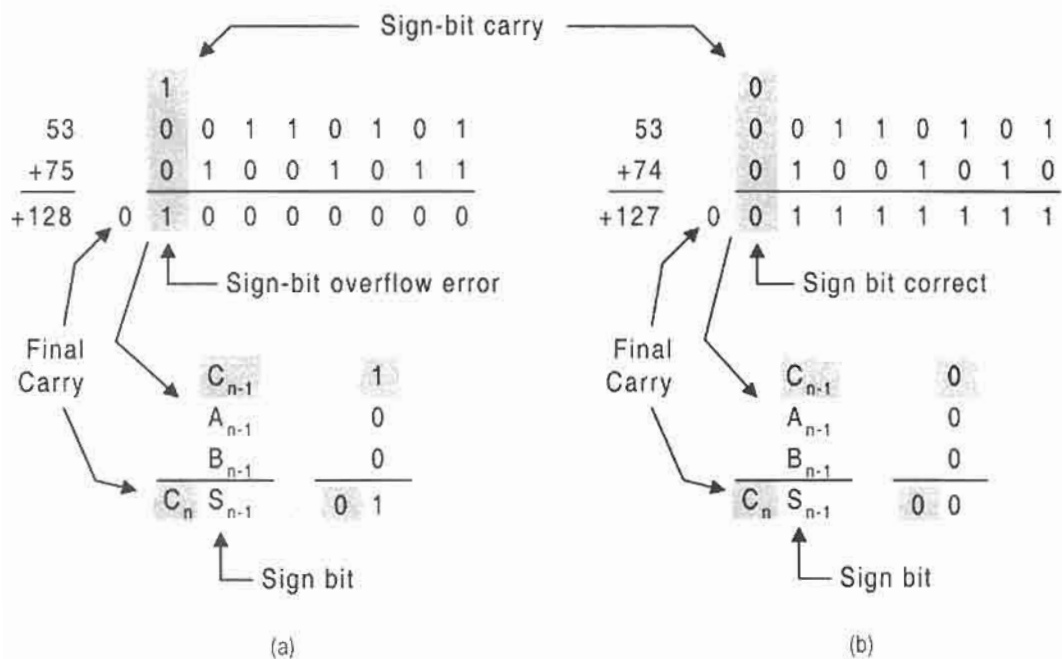
### 8.3.1 Adder/Subtractors

The adder/subtractor approach to arithmetic circuit design permits the use of one set of hardware to accomplish both addition- and subtraction-type operations. One means to accomplish this becomes evident from an inspection of the FA and FS circuits of Figs. 8.3 and 8.8. As was pointed out previously, the FA and FS differ only by two inverters in the AND/OR part of the circuit. If the inverters are replaced by controlled inverters (XOR gates as in Fig. 3.31c), an adder/subtractor results. Thus, when the control input to the XOR gate is 1(H) the XOR gate functions as an inverter, creating an FS, or when the control input is 0(H) the device functions as an FA. When such FA/FS modules are connected in series as in Figs. 8.5 and 8.6, the result is a parallel adder/subtractor.

Another approach to adder/subtractor design makes use of 2's complement arithmetic as discussed in Subsection 2.9.2. The design of the adder/subtractor now follows by using the ripple-carry adder hardware of Fig. 8.5 together with XOR gates used as controlled inverters on the  $B_i(H)$  inputs. The result is the  $n$ -bit adder/subtractor shown in Fig. 8.9. If the mode control input is set  $\bar{A}/S(H) = 1(H)$ , the operation is subtraction [ $A_i$  plus ( $-B_i$ )] in 2's complement and the final carry-out is discarded. However, if the add/subtract control is set  $\bar{A}/S(H) = 0(H)$ , the operation is addition and the final carry-out becomes the most significant sum bit  $S_n$ . For subtraction the final sum bit,  $S_{n-1}$ , is the sign bit, which can be positive (if 0) or negative (if 1) depending on the outcome. Notice that for subtraction a 1(H) is introduced into FA<sub>0</sub> as the initial carry-in  $C_0$ , a requirement of Eq. (2.14). Also, note

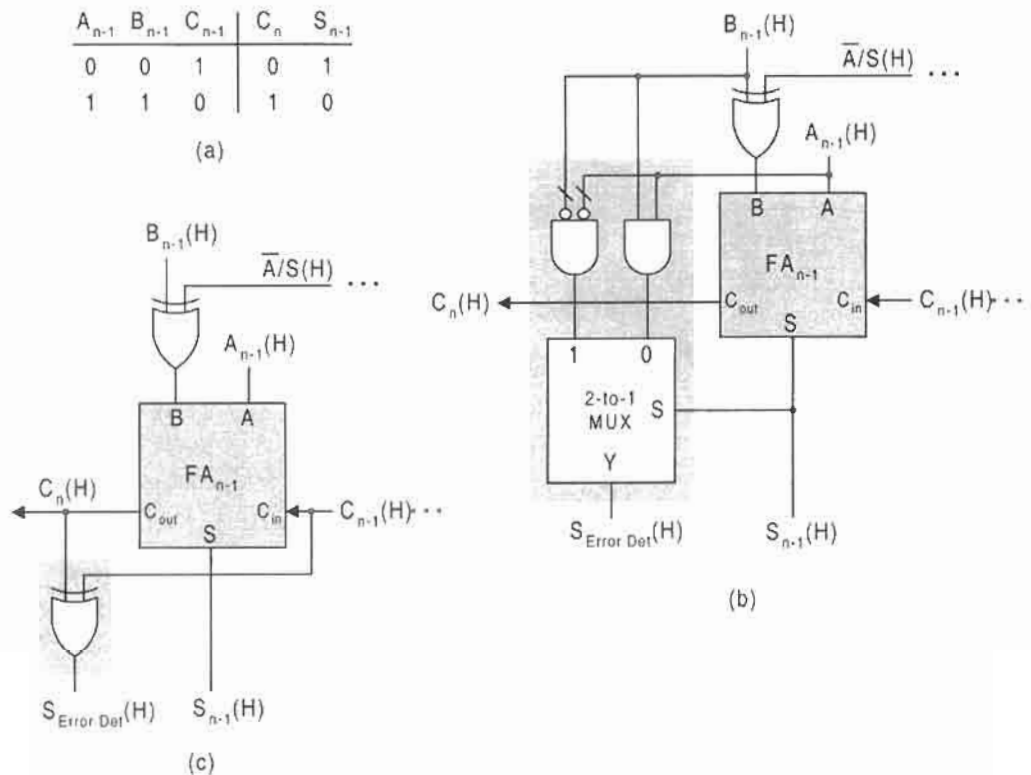






**FIGURE 8.10**

Eight-bit additions showing sign bits and final two carries. (a) Sign-bit overflow error created for sum exceeding decimal 127. (b) Correct sign bit for sum equal to 127.



**FIGURE 8.11**

Overflow error detection circuits for sign-bit stage. (a) Truth table for overflow error in the sign-bit stage. (b) Overflow error detection circuit by using external data inputs and the sum output. (c) Alternative overflow error detection circuit by using input and output carries.

sum of two positive numbers only if  $C_n \neq C_{n-1}$ . Thus, a sign-bit overflow error detector can be implemented by

$$S_{Error\ Det} = C_n \oplus C_{n-1}, \quad (8.4)$$

requiring that the sign-bit carry-in  $C_{n-1}$  be accessible, which it may not be for IC chips. Another approach permits a detector to be designed that depends only on the external inputs to and sum bit from the sign-bit stage. A further inspection of truth table for an FA in Fig. 8.2c indicates that a sign-bit overflow error can occur only if  $A = B$  when  $C_{out} \neq S$  for the  $(n-1)$ th stage. Shown in Fig. 8.11a is the truth table for the sign-bit overflow error conditions based on this fact. From this truth table there results the expression

$$S_{Error\ Det} = \bar{S}_{n-1}(A_{n-1}B_{n-1}) + S_{n-1}(\bar{A}_{n-1}\bar{B}_{n-1}), \quad (8.5)$$

which permits the use of the 2-to-1 MUX implementation shown in Fig. 8.11b. For purposes of comparison, the implementation of Eq. (8.4) is given in Fig. 8.11c.

## 8.4 THE CARRY LOOK-AHEAD ADDER

The ripple-carry (R-C) adder discussed in Section 8.3 is satisfactory for most applications up to 16 bits at moderate speeds. Where larger numbers of bits must be added together at high speeds, fast adder configurations must be used. One clever, if not also intuitive, design makes use of the modular approach while reducing the propagation time of the R-C effect. The approach has become known as the *carry look-ahead (CLA) adder*. In effect, the CLA adder “anticipates” the need for a carry and then generates and propagates it more directly than does a standard R-C adder.

The design of the CLA adder begins with a generalization of Eqs. (8.2). For the  $i$ th stage of the ripple-carry adder, the sum and carry-out expressions are

$$\left\{ \begin{array}{l} S_i = A_i \oplus B_i \oplus C_i \\ \quad = \text{Sum of the } i\text{th stage} \\ C_{i+1} = C_i(A_i \oplus B_i) + A_i B_i \\ \quad = \text{Carry-out of the } i\text{th stage} \end{array} \right\} \quad (8.6)$$

From the expression for  $C_{i+1}$  it is concluded that  $C_{i+1} = 1$  is assured if  $(A_i \oplus B_i) = 1$  and  $C_i = 1$ , or if  $A_i B_i = 1$ .

Next, it is desirable to expand Eqs. (8.6) for each of  $n$  stages, beginning with the 1st (first) stage. To accomplish this, it is convenient to define two quantities for the  $i$ th stage,

$$\left\{ \begin{array}{l} P_i = A_i \oplus B_i = \text{Carry Propagate} \\ G_i = A_i \cdot B_i = \text{Carry Generate} \end{array} \right\}, \quad (8.7)$$

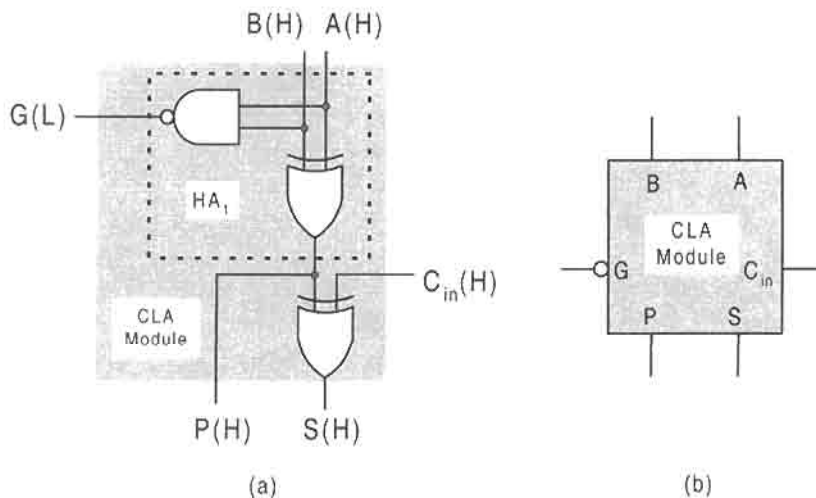
which are shown in Fig. 8.3 to be the intermediate functions  $P(H)$  and  $G(L)$  in the full

adder. Introducing these equation into an expansion of Eqs. (8.6) gives

$$\begin{aligned}
 \text{1st stage} & \left\{ \begin{array}{l} S_0 = P_0 \oplus C_0 \\ C_1 = P_0 C_0 + G_0 \end{array} \right\} \\
 \text{2nd stage} & \left\{ \begin{array}{l} S_1 = P_1 \oplus C_1 \\ C_2 = P_1 C_1 + G_1 \\ \quad = P_1 P_0 C_0 + P_1 G_0 + G_1 \end{array} \right\} \\
 \text{3rd stage} & \left\{ \begin{array}{l} S_2 = P_2 \oplus C_2 \\ C_3 = P_2 C_2 + G_2 \\ \quad = P_2 P_1 P_0 C_0 + P_2 P_1 G_0 + P_2 G_1 + G_2 \end{array} \right\} \\
 \vdots \\
 \text{\textit{n}th stage} & \left\{ \begin{array}{l} S_n = P_n \oplus C_n \\ C_{n+1} = P_n C_n + G_n \\ \quad = P_n P_{n-1} P_{n-2} \cdots P_0 C_0 + P_n P_{n-1} P_{n-2} \cdots \\ \quad \quad P_1 G_0 + \cdots + G_n \end{array} \right\}. \quad (8.8)
 \end{aligned}$$

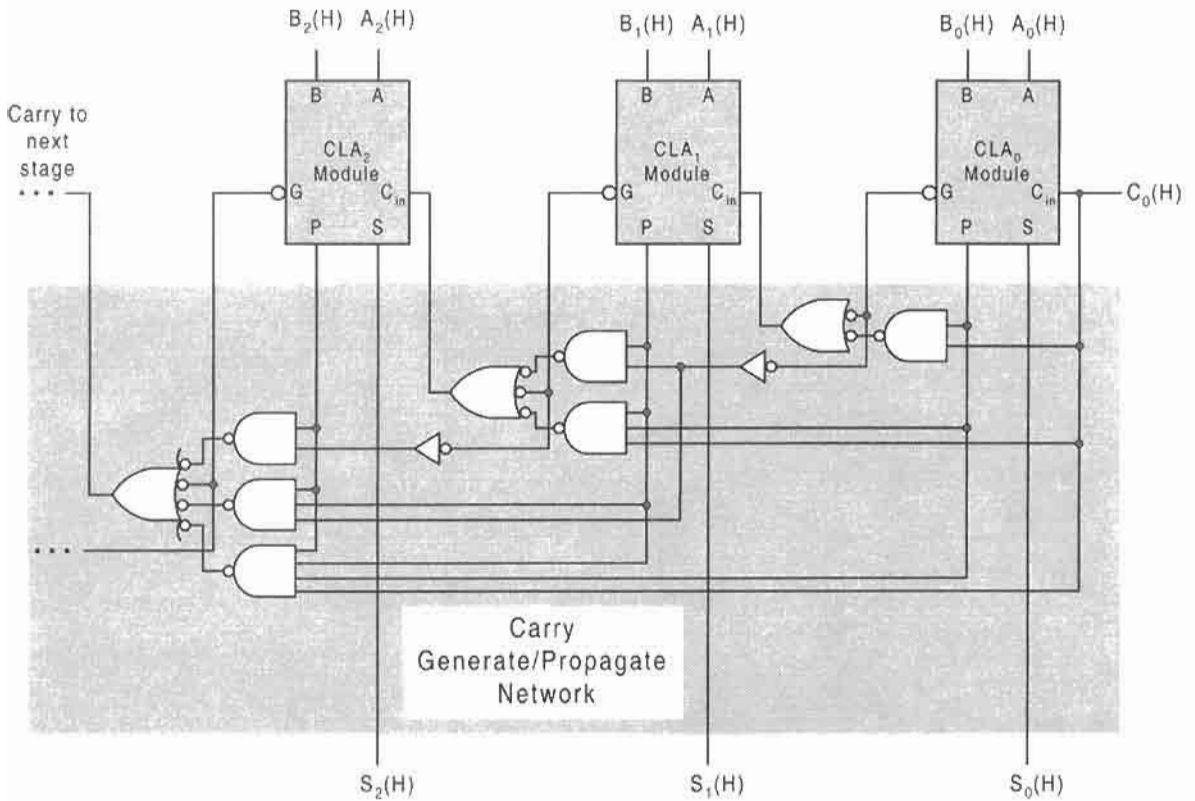
To implement Eqs. (8.8), use is made of the carry look-ahead module shown in Fig. 8.12, which is deduced from Fig. 8.3 and Eqs. (8.7). Notice that the CLA module has but one half adder but has two additional outputs, carry generate  $G$  and carry propagate  $P$ . Whereas the  $S_i$  terms are produced within the each FA, the  $C_{i+1}$  terms must be formed externally by what is called the *carry generate/propagate (CGP) network*. It is the CGP network to which  $G(L)$  and  $P(H)$  must be connected in accordance with Eqs. (8.8).

Construction of an  $n$ -bit CLA adder requires the use of  $n$  CLA modules together with the CGP network according to Eqs. (8.8). This is done in Fig. 8.13 for the three least significant



**FIGURE 8.12**

The carry-look-ahead (CLA) module. (a) Logic circuit deduced from Fig. 8.3 and Eqs. (8.7). (b) Logic circuit symbol.

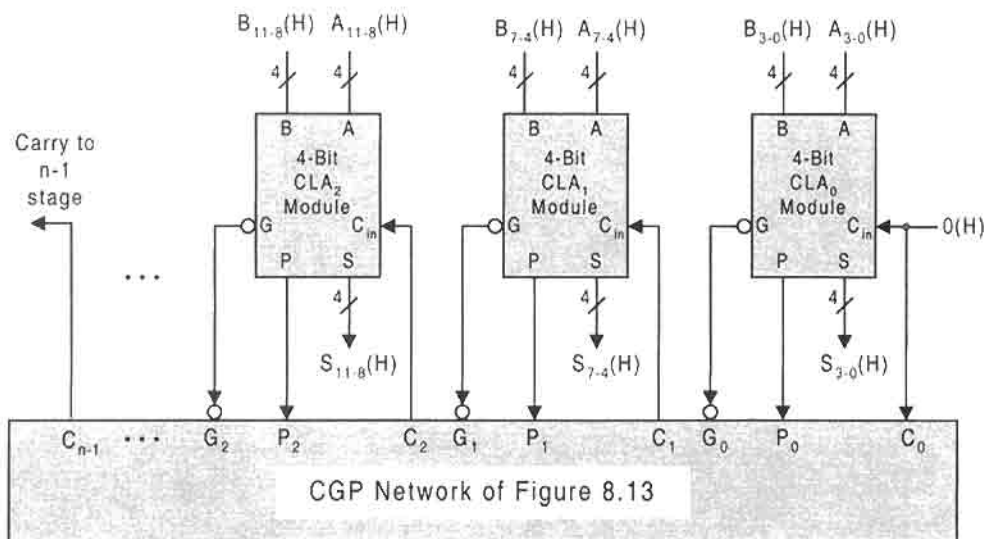
**FIGURE 8.13**

The three least significant stages of an  $n$ -bit carry look-ahead (CLA) adder showing the carry generate/propagate network.

bit stages of an  $n$ -bit CLA adder. Each of the CLA modules are of the type given in Fig. 8.12 and the shaded portion below the CLAs in Fig. 8.13 represents the CGP network as required by Eqs. (8.8). Notice that for  $C_0(H) = 0(H)$ , a requirement for proper operation of the adder, one NAND gate in each stage is disabled. This does not mean that such disabled NAND gates should be eliminated from the CGP network. All NAND gates shown in the CGP network become necessary if  $m$ -bit CLA adder modules are to be cascaded to form  $m$   $n$ -bit CLA adders. It is also interesting to note that the  $CLA_0$  module together with its CGP logic is exactly the same as an FA in Fig. 8.3. Furthermore, if the extra hardware is of no concern, all CLA modules can be replaced by FAs with  $G$  and  $P$  outputs.

An inspection of Fig. 8.13 makes it clear that the hardware requirement for the CGP network increases significantly as the number of stages increases. The gate count for the CLA adder in Fig. 8.13 is 5, 11, and 18 for the three stages 0, 1, and 2 as shown. A fourth stage would have a total gate count of 26 and a fifth stage would be 35, etc. In fact, the total number of gates required for an  $n$ -bit CLA is given by

$$\text{Total gate count} = \underbrace{(n^2 + 3n)/2}_{\text{CGP network}} + \underbrace{3n}_{\text{CLA modules}} = (n^2 + 9n)/2 \quad (8.9)$$

**FIGURE 8.14**

The three least significant stages of a carry look-ahead (CLA) adder showing the carry generate/propagate network between 4-bit modules.

This count is to be compared to  $5n$  for a ripple-carry (R-C) adder of the type shown in Fig. 8.5. For example, an 8-bit CLA adder requires 68 gates, compared to 40 for the R-C adder, but the CLA adder is more than twice as fast. Furthermore, as the number of stages increases, so does the maximum fan-in requirement. For the CLA adder, the maximum fan-in is  $(n + 1)$ , compared to 2 for the R-C adder. The possibility of additional hardware to compensate for fan-in and fan-out limitations of the CLA adder must be weighed against the fact that the maximum propagation delay for the  $n$ -bit CLA adder of Fig. 8.13 is 4 gate delays, compared to  $2n$  for the R-C adder.

The modular approach to CLA adder design can be taken one step further by using the CGP network for  $n$ -bit CLA adder modules to create a larger adder. To do this, one simply cascades  $m$   $n$ -bit CLA modules in series to create an adder of  $k = m \times n$  bits in size. This is demonstrated in Fig. 8.14, where 4-bit CLA adder modules are cascaded by using the CGP network shown in Fig. 8.13. The adder in Fig. 8.14 is called a *group CLA adder*. This group CLA adder configuration saves hardware, but at the expense of increased propagation delay. For example, a 16-bit CLA adder with 1-bit CLA modules requires 200 gates with 4 gate-levels of delay, 2 for generation of  $G$  and  $P$  and 2 for carry generation. In comparison, a 16-bit group CLA adder with 4-bit CLA modules requires 118 gates but has 6 gate-levels of delay (2 extra gate delays for carry). Note that the comparisons just made do not take into account any additional gates required to deal with fan-in and fan-out limitations.

The foregoing discussion suggests that there is a practical limit to the number of CLA adder stages that can be used without creating excessive amounts of hardware with the accompanying fan-in and fan-out restrictions and path delay problems. To overcome this limitation the concept of the group CLA adder configuration can again be used, but in a rather different way. For example, each 4-bit CLA stage in Fig. 8.14 can be replaced by 4-bit R-C FA stages or by 4-bit adder/subtractors stages as in Fig. 8.9 such that the MSB module of each stage is equipped with  $G$  and  $P$  outputs or is a CLA module of the type in

Fig. 8.12. In fact, the nature of the adder stage to be cascaded in CLA form is immaterial as long as the MSB module of each stage has  $G$  and  $P$  outputs to be connected to the CGP network. In any case combining R-C and CLA technologies reduces hardware requirements but, of course, does so at the expense of increasing path delay.

## 8.5 MULTIPLE-NUMBER ADDITION AND THE CARRY-SAVE ADDER

To add more than two binary numbers by the “conventional” methods discussed in Sections 8.3 and 8.4 requires that the addition process be carried out in stages, meaning that  $k$  operands are added in  $k - 1$  two-operand additions, each addition (following the first) being that of the accumulated sum with a operand. This staged addition operation is relatively slow since carries are propagated in each stage. However, addition of binary numbers is not limited to two number addition stages. Actually, many binary numbers can be added together in ways that reduces the total addition time. One way to accomplish this is to use a type of “fast” adder called an *iterative carry-save (ICS) adder*. The algorithm for iterative CS addition of more than two operands is stated as follows:

Algorithm 8.1: Sum  $S = A + B + C + D + E \cdots$  by Carry-Save Method

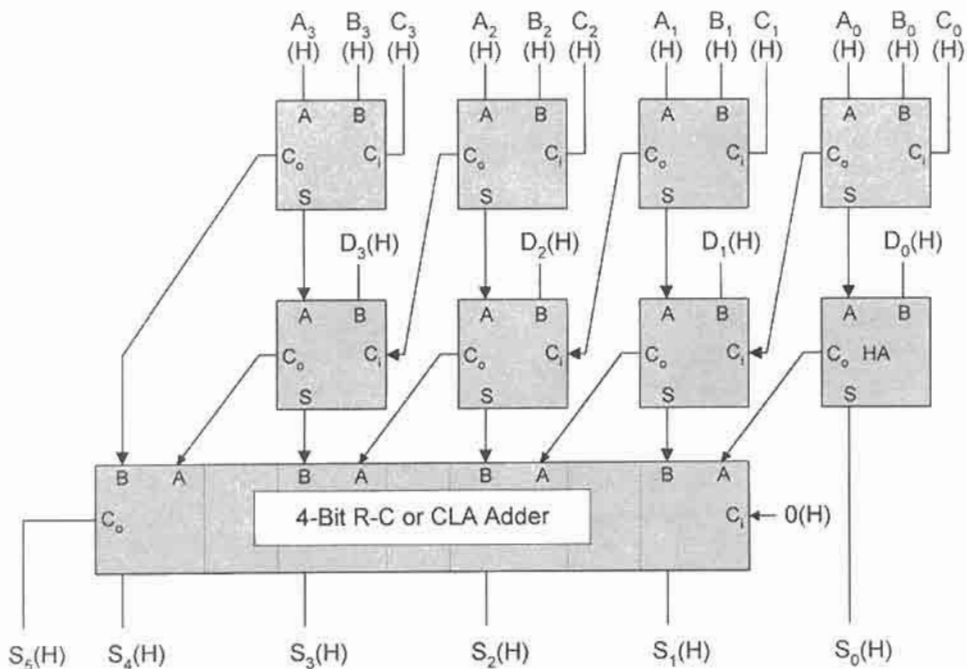
- (1) Set integers  $A = A_{n-1}A_{n-2} \cdots A_1A_0$ ,  $B = B_{n-1}B_{n-2} \cdots B_1B_0$ ,  $C = C_{n-1}C_{n-2} \cdots C_1C_0$ , etc.
- (2) Sum  $S^0 = A + B + C = S_{n-1}^0S_{n-2}^0 \cdots S_1^0S_0^0$  exclusive of all carries-out  $C_o^1$ .
- (3) Sum  $S^1 = S^0 + D + C_o^1 = S_{n-1}^1S_{n-2}^1 \cdots S_1^1S_0^1$  exclusive of all carries-out  $C_o^2$ , but with carries  $C_o^1$  shifted left by one bit.
- (4) Sum  $S^2 = S^1 + E + C_o^2 = S_{n-1}^2S_{n-2}^2 \cdots S_1^2S_0^2$  exclusive of all carries  $C_o^3$  but with carries  $C_o^2$  shifted left by one bit.
- (5) Continue process until last integer has been added and only two resultant operands remain: pseudosum  $S'$  and final CS carries  $C'$ .
- (6) End with final sum  $S = S' + C' = S_nS_{n-1} \cdots S_1S_0$  by using either R-C or CLA addition.

The carry-save process described in Algorithm 8.1 applies to  $m$  integer operands and involves  $m - 2$  CS additions resulting in two operands: a pseudosum  $S'$  and the final CS carries  $C'$ . The final step (step 6 in Algorithm 8.1) adds the two operands  $S'$  and  $C'$  by using either a R-C adder or a CLA adder. The CS process (steps 1 through 5 in Algorithm 8.1) avoids the ripple-carry problem of R-C adders, resulting in a savings of addition time. For R-C adders of the type in Figs. 8.5 and 8.6, a time  $(m - 1)t_{R-C}$  would be required to add  $m$  integer operands. However, for addition of  $m$  integer operands by the CS method, a time  $(m - 2)t_{CS} + t_{R-C}$  is required, which represents a significant savings of time for  $m > 3$ , since  $t_{CS} < t_{R-C}$  for a given number of addition operations. If CLA addition is involved, the time  $t_{R-C}$  can be replaced by  $t_{CLA}$  with less savings in time between the two methods.

The iterative CS process just described in Algorithm 8.1 is illustrated in Fig. 8.15a where four 4-bit integers,  $A$ ,  $B$ ,  $C$ , and  $D$ , are added. The ICS adder required for this process is shown in Fig. 8.15b, where use is made of FAs, a HA, and either a R-C adder or a CLA adder for the final sum operation.

|       |   |   |   |                                                    |                                             |
|-------|---|---|---|----------------------------------------------------|---------------------------------------------|
| 0     | 1 | 1 | 1 | A                                                  | } Three operands for 1 <sup>st</sup> CS sum |
| 1     | 0 | 1 | 0 | B                                                  |                                             |
| 1     | 1 | 1 | 1 | C                                                  |                                             |
| <hr/> |   |   |   | S <sup>0</sup>                                     | } Three operands for 2 <sup>nd</sup> CS sum |
| 1     | 1 | 1 | 1 | C <sub>0</sub> <sup>1</sup>                        |                                             |
| 1     | 0 | 1 | 1 | D                                                  |                                             |
| <hr/> |   |   |   | S <sup>1</sup> = Pseudosum, S'                     | } Two operands for final sum                |
| 1     | 0 | 1 | 0 | C <sub>0</sub> <sup>2</sup> = Final CS carries, C' |                                             |
| 1     | 0 | 1 | 0 | SUM, S                                             |                                             |

(a)



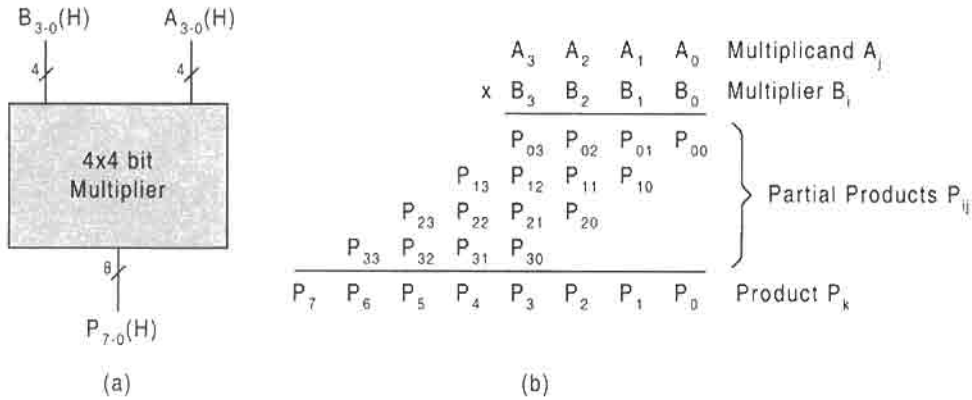
(b)

**FIGURE 8.15**

The carry-save (CS) method of addition. (a) CS addition of four 4-bit numbers. (b) The CS adder designed to add four 4-bit operands by using FAs as a HA and a 4-bit R-C or CLA adder.

## 8.6 MULTIPLIERS

An  $n \times m$  multiplier is an  $(n + m)$ -input/ $(n + m)$ -output device that performs the logic equivalent of an  $n$ -bit  $\times$   $m$ -bit binary multiplication. Shown in Fig. 8.16a is the block circuit symbol, and in Fig. 8.16b the operation format for a 4-bit  $\times$  4-bit multiplier that follows Algorithm 2.10, given in Subsection 2.9.4. As indicated in the operation format the

**FIGURE 8.16**

Characteristics of the  $4 \times 4$  bit multiplier. (a) Block circuit symbol. (b) Operation format showing partial products and final product bits.

multiplication process consists of four multiplicand bits  $A_j$ , four multiplier bits  $B_i$ , partial product bits  $P_{ij}$ , and the final eight product bits  $P_k$ .

The  $4 \times 4$  bit multiplication process, represented in Fig. 8.16b and by Example 2.27 in Subsection 2.9.4, can be implemented at the modular level by using HAs and FAs to perform the summation between the partial products and carries. Note that the partial products are generated by AND operations  $P_{ij} = B_i \cdot A_j$ , and that the product bits in each column of Fig. 8.16 are the sums of these partial products and the carries between stages as indicated by the XOR operations in Fig. 8.17.

Some explanation of Fig. 8.17 may be helpful. Recalling the sum expression for an FA in Eq. (8.2), it follows that  $k$  FAs can accommodate  $2k$  XOR operators or  $2k + 1$  variables in an XOR string such as that of Eq. (5.16). For  $2k - 1$  XOR operators in the string, use can be made of one HA. Note that the  $C_{ij}$  symbols in Fig. 8.17 refer to carries from the  $i$ th to the  $j$ th sum stage.

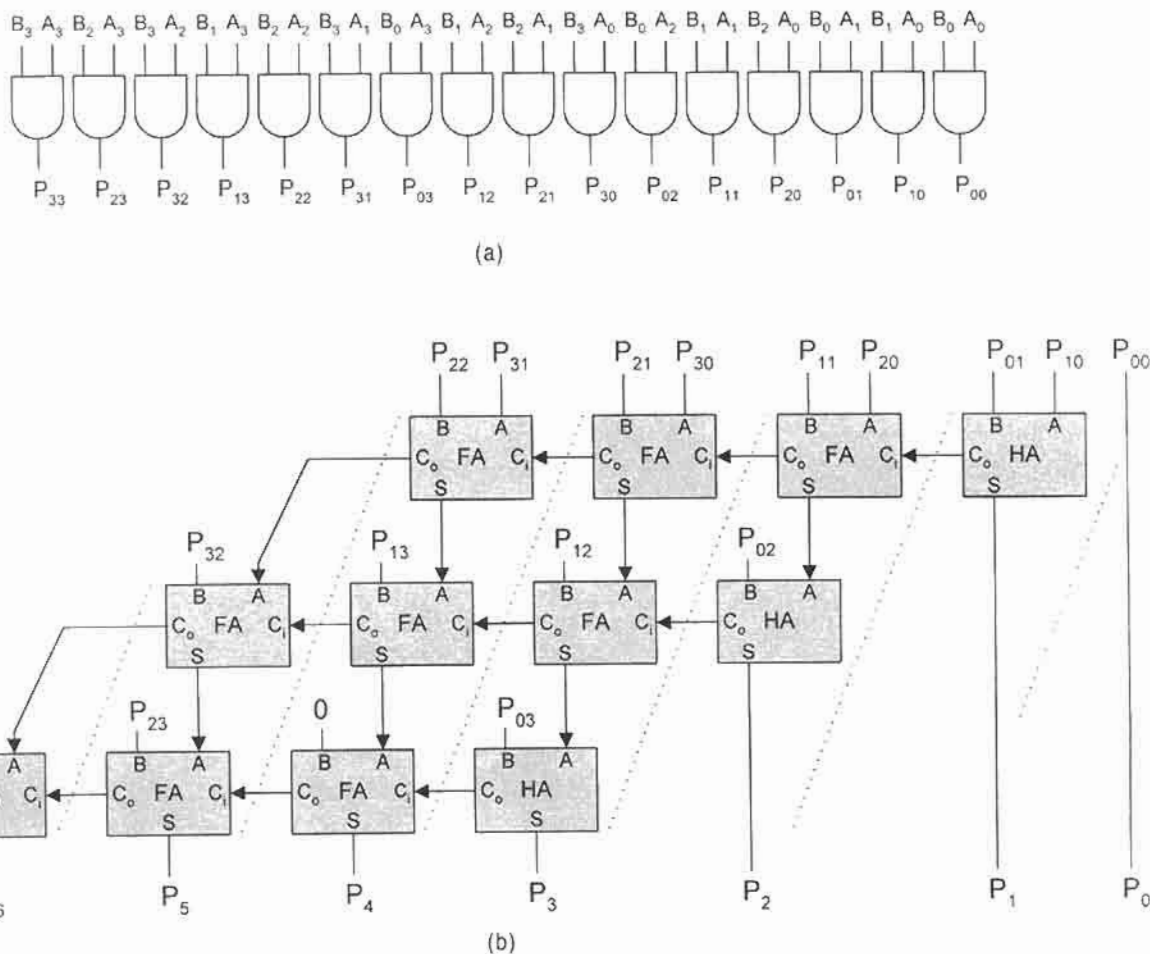
The product bits  $P_i$  indicated in Fig. 8.17 can be computed by first generating the partial products followed by a summation of these partial products with the appropriate carries. This is done in Fig. 8.18, where six of the eight sum stages are shown separated by dotted

$$\begin{aligned}
 P_0 &= P_{00} \\
 P_1 &= P_{01} \oplus P_{10} \\
 P_2 &= P_{02} \oplus P_{11} \oplus P_{20} \oplus C_{12} \\
 P_3 &= P_{03} \oplus P_{12} \oplus P_{21} \oplus P_{30} \oplus [C_{23} \oplus C_{23}'] \\
 P_4 &= P_{13} \oplus P_{22} \oplus P_{31} \oplus [C_{34} \oplus C_{34}' \oplus C_{34}'] \\
 P_5 &= P_{23} \oplus P_{32} \oplus [C_{45} \oplus C_{45}' \oplus C_{45}'] \\
 P_6 &= P_{33} \oplus [C_{56} \oplus C_{56}'] \\
 P_7 &= C_{67}
 \end{aligned}$$

**FIGURE 8.17**

The summations of partial products and carries required to produce the product bits for the multiplication of two 4-bit operands shown in Fig. 8.16b.

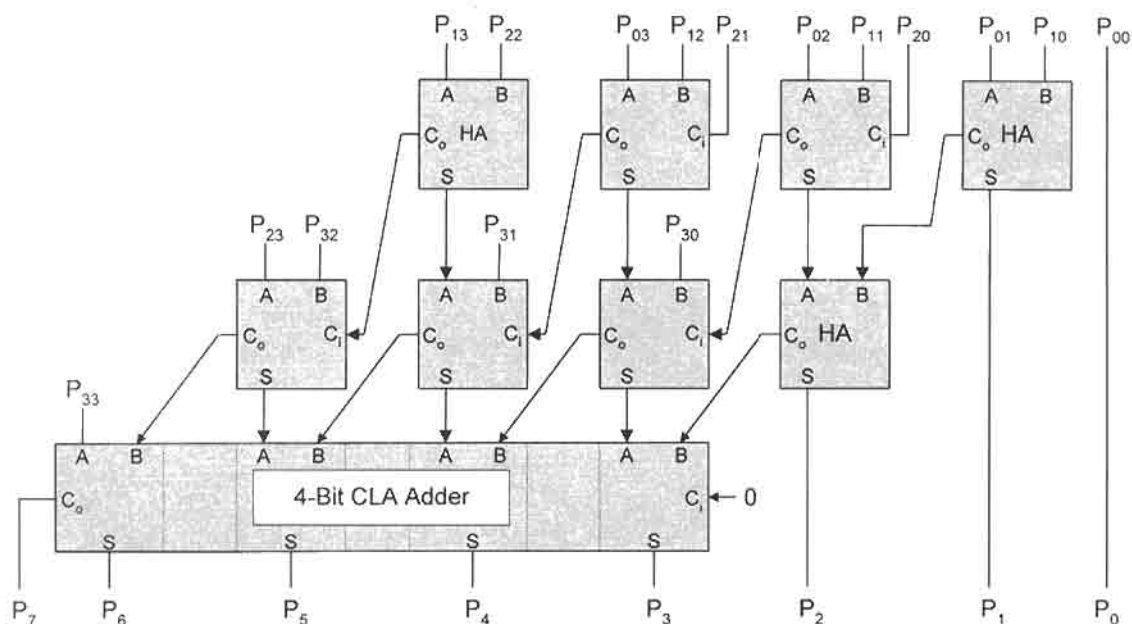


**FIGURE 8.18**

Implementation of the 4×4 bit multiplier of Fig. 8.17. (a) Generation of the partial products. (b) Use of FAs and HAs to sum the partial products and carries.

lines, the final stage being the carry  $C_{67} = P_7$  and the initial stage being  $P_{00} = P_0$ . Although the string of XOR operations in each product expression of Fig. 8.17 can be computed in any arbitrary order, the carry configurations of Fig. 8.18 are so chosen as to make use of standard 4-bit R-C adders with HAs as initial stages. Now it is possible to replace the 4-bit R-C adders with 4-bit CLA adders at a significant savings in calculation time. Note that the number of carries  $C_{ij}$  for each stage in Fig. 8.17 agrees with those in Fig. 8.18.

The multiplication process can also be carried out by using the carry-save (CS) method of summing the partial products and carries in a manner similar to that of Fig. 8.15. However, in this case the operands must be summed by the iterative CS method expressed by Algorithm 8.1. Such a CS scheme, shown in Fig. 8.19, is a type of *array multiplier* called an *iterative CS multiplier*. Here, a 4-bit CLA adder is used for the final sum of the two operands,  $S^1$  and  $C_0^2$ , as in Fig. 8.15.

**FIGURE 8.19**

The iterative carry-save (ICS) method with FAs, HAs, and a 4-bit CLA adder used to multiply two 4-bit operands. Partial products  $P_{ij}$  are generated as in Fig. 8.18.

The iterative CS multiplier of Fig. 8.19 has the advantage of reduced computation time compared to the R-C approach illustrated in Fig. 8.18. For  $n$  rows and  $2n - 1$  columns of partial products, the summation time for an iterative CS (ICS) multiplier is

$$t_{ICS} = (n - 2)t_{FA} + t_{R-C}, \quad (8.10)$$

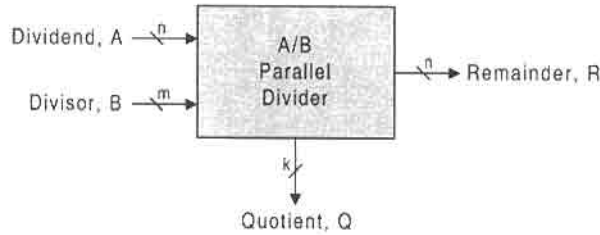
where  $t_{FA}$  is the delay of a FA and  $t_{R-C}$  is the time required for the R-C adder to complete the summation. In comparison, the summation time required for a multiplier that uses only R-C adders (as in Fig. 8.18), is

$$t_{RCA} = 2(n - 1)t_{FA} + t_{R-C} \quad (8.11)$$

Thus for large  $n$ , the iterative CS multiplier is about twice as fast as the one that uses only R-C adders. If CLA adders are used for both types of multipliers, the difference in speed between the two is reduced and may even shrink to near zero for certain values of  $n$ .

## 8.7 PARALLEL DIVIDERS

An  $n \div m$  parallel divider is an  $(n + m)$ -bit/variable-bit output device that performs the electrical equivalent of the binary operation symbolized by  $A \div B = Q$  with remainder  $R$ . As used here,  $A$  and  $B$  are the dividend and divisor operands, respectively, and  $Q$  is

**FIGURE 8.20**

Block diagram symbol for an  $n/m$  parallel divider.

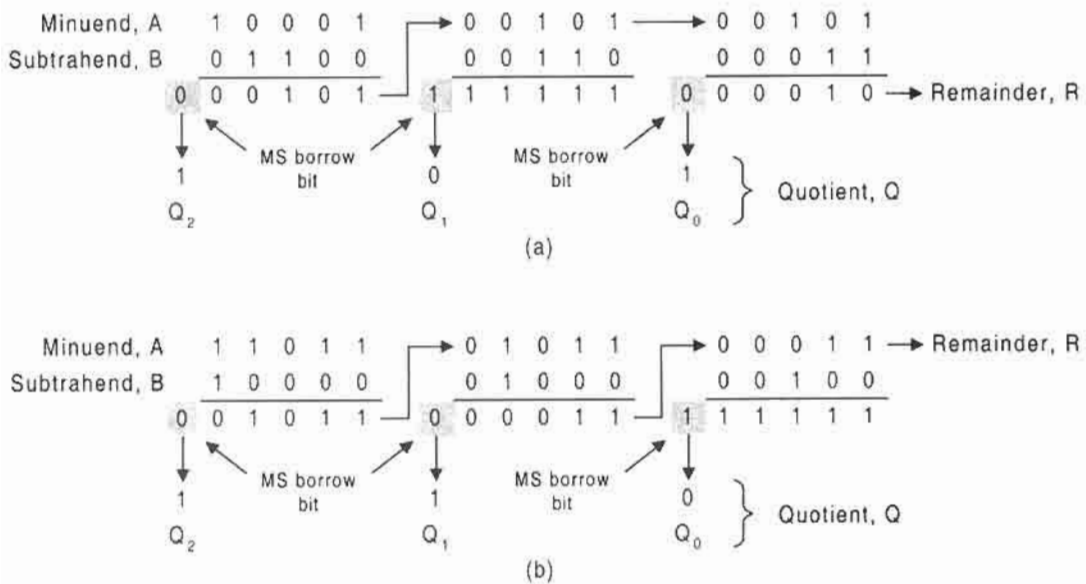
the quotient. The block diagram symbol for this divider is given in Fig. 8.20, where it is understood that  $m \leq n$  for the binary integers of these operands.

The details of the logic circuitry for a divider depend on the algorithm used to execute the division operation. Recall that in Subsection 2.9.5, Algorithm 2.12 presented a division procedure that is close to the familiar *subtract-and-shift* procedure used in decimal long division. It is this same procedure that is used to design a parallel divider, but modified in the following way to accommodate hardware application:

1. Successively subtract the subtrahend  $B$  from the minuend  $A$  by starting from the MSB end of  $A$  and shifting 1 bit toward the LSB after each subtraction stage:
  - (a) When the most significant (MS) borrow bit for the present stage is 0, the minuend for the next stage (remainder from the present stage) is the difference of the present stage.
  - (b) When the MS borrow for the present stage is 1, the minuend for the next stage is the minuend of the present stage.
2. Complement the MS borrow bit for each stage and let it become the quotient bit for that stage.
3. Repeat steps 1 and 2 until the subtrahend  $B$  has been shifted to the LSB end of the minuend  $A$ . The final remainder  $R$  will be determined by the logic level of the MS borrow as in step 1a or 1b.

The procedure just described, a modification of Algorithm 2.12, is illustrated in Fig. 8.21a for a 5-bit dividend,  $A = 10001$  and a 3-bit divisor  $B = 011$ . The result is  $A \div B = Q$  with remainder  $R$ , where the 3-bit quotient is  $Q = 101$  and the 5-bit remainder is  $R = 00010$ . Thus, in decimal  $17 \div 3 = 5$  with remainder 2, which would be written as 17 and  $2/3$  or  $17.66666 \dots$ . Similarly, in Fig. 8.21b,  $A = 11011_2 (27_{10})$ ,  $B = 100_2 (4_{10})$  with the result  $Q = 00110_2 (6_{10})$  and  $R = 00011_2 (3_{10})$ .

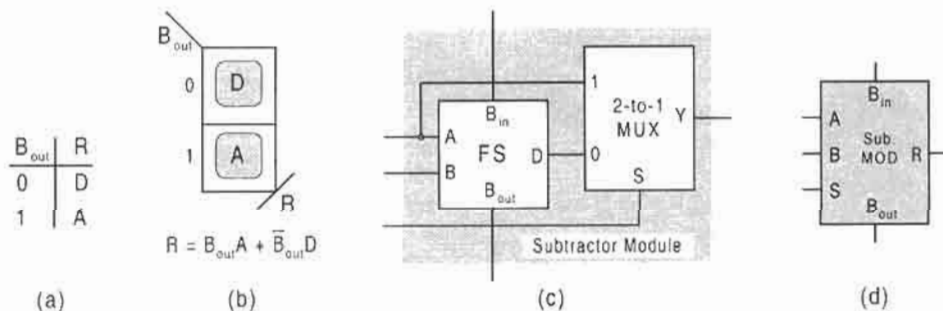
To design a parallel divider, the requirements of the subtract-and-shift process, illustrated in Fig. 8.21, must be met. First, the subtrahend must be successively subtracted from the minuend, and then shifted from the MSB end of the minuend toward its LSB end by one bit after each subtraction stage. This is easily accomplished by shifting the subtrahend presentation to an array of full subtractors (FSs). Second, the remainder  $R$  must be properly gated. Taking  $B_{out}$  to mean the MS borrow for a given stage, the division process requires that  $R = D$  when  $B_{out} = 0$  and  $R = A$  for  $B_{out} = 1$ , where  $D$  and  $A$  are the difference and minuend, respectively, for the present stage. Shown in Fig. 8.22 are the truth table (a), EV K-map (b), and the subtractor module (c), together with its block symbol (d), that meet the

**FIGURE 8.21**

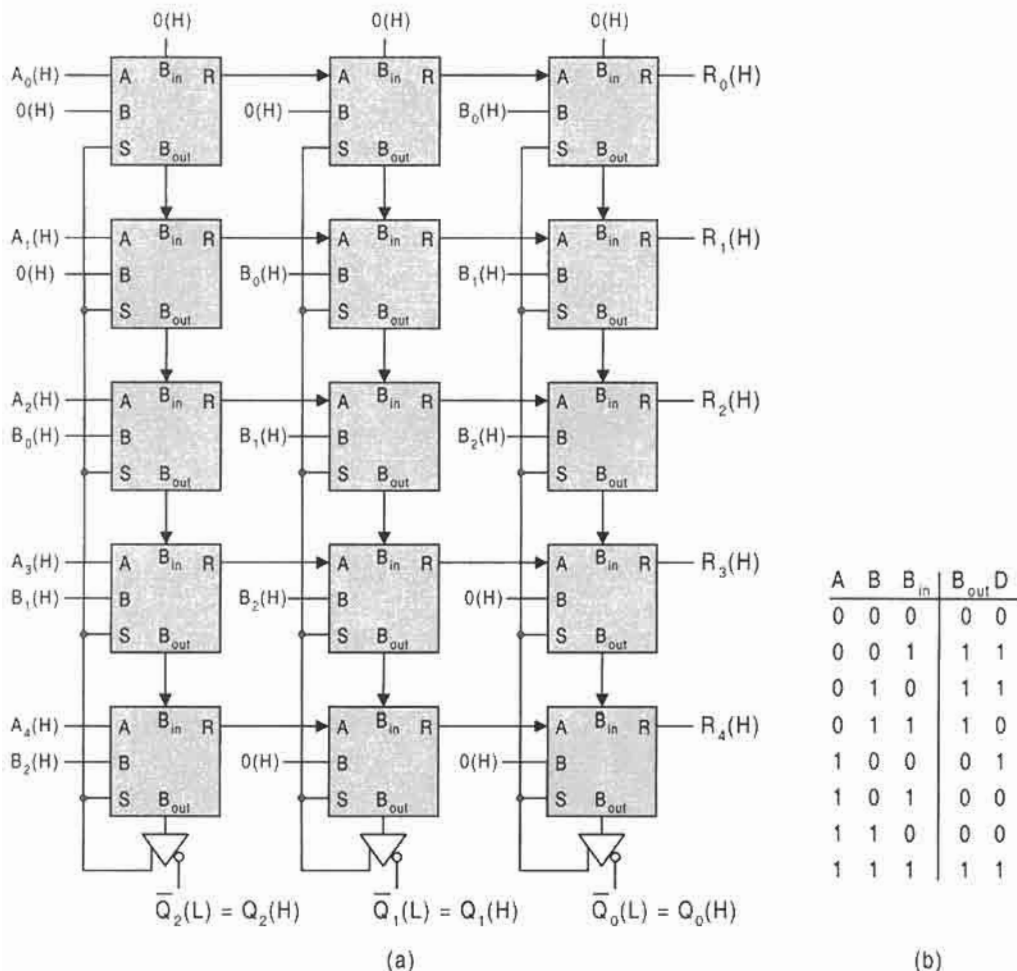
Two illustrations of the subtract-and-shift method of binary division for 5-bit dividends and 3-bit divisors. (a) The result is  $Q = 00101$  ( $5_{10}$ ) with  $R = 00010$  ( $2_{10}$ ) when  $A = 10001$  ( $17_{10}$ ) and  $B = 011$  ( $3_{10}$ ). (b) The result is  $Q = 00110$  ( $6_{10}$ ) with  $R = 00011$  ( $3_{10}$ ) when  $A = 11011$  ( $27_{10}$ ) and  $B = 100$  ( $4_{10}$ ).

remainder requirements just given. Notice that the expression  $R = B_{out}A + \bar{B}_{out}D$  applies to a 2-to-1 MUX with inputs  $A$  and  $D$  and output  $R$  when  $B_{out}$  is the data select input.

All that remains is to construct an array of subtractor modules for the subtract-and-shift process required for the parallel divider. This is done in Fig. 8.23a for a dividend (minuend) of 5 bits and a divisor (subtrahend) of 3 bits. Here, the quotient outputs  $Q_2(H)$ ,  $Q_1(H)$ , and  $Q_0(H)$  are issued complemented from the active low outputs of inverter symbols defined in Fig. 7.6. The truth table for a full subtractor is given in Fig. 8.23b to assist the reader in analyzing this divider circuit.

**FIGURE 8.22**

Design of the subtractor module for use in the design of a parallel divider. (a) Truth table for remainder  $R$  and final borrow-out. (b) EV K-map and gating logic for  $R$ . (c) The subtractor module by using a FS and a 2-to-1 MUX. (d) Circuit block symbol for the subtractor module in (c).

**FIGURE 8.23**

(a) Parallel divider for a 5-bit dividend,  $A$ , 3-bit divisor,  $B$ , a 3-bit quotient,  $Q$ , and a 5-bit remainder,  $R$ , designed with an array of 15 subtractor modules of the type shown in Fig. 8.22. (b) Truth table for a full subtractor.

The divider in Fig. 8.23 can be expanded to accommodate larger dividends and quotients by adding subtractor modules in both the  $Y$ - and  $X$ -directions, respectively. Referring to Fig. 8.20, the relationship between  $n$ ,  $m$ , and  $k$  is given by

$$k = n - m + 1 \quad (8.12)$$

for full usage of an  $n \times k$  array of subtractor modules. For example, a 16-bit dividend ( $n = 16$ ) and an 8-bit divisor ( $m = 8$ ) can be used to generate a 9-bit quotient ( $k = 9$ ) in a  $16 \times 9$  array of subtractor modules. Or a 32-bit dividend and a 24-bit divisor can be used to generate a 9-bit quotient in a  $32 \times 9$  array of subtractor modules. In all such cases the remainder is of  $n$  bits. It is also acceptable to use any lesser portion of a given array of subtractor modules to carry out a divide operation, but Eq. (8.12) must still apply to that

portion of the array that is used. However, to do this requires proper placement of 0's on modules that are not used. For example, if  $m = n$ , only one quotient bit is generated and 0's must be placed on all open inputs to the right of the MSB column of the array.

Dividers of the type shown in Fig. 8.23 can be classified as "fast" dividers. This is because they are asynchronous in nature, meaning that the results are generated as fast as the logic permits. However, it is characteristic of such circuits that with increasing operand size the hardware requirements increase rapidly making them impractical for many applications where space requirements are important. There are other methods of achieving the division operation with less hardware but at the expense of operation time, as expected. These methods require storage elements such as registers and fall outside the treatment of this text.

## 8.8 ARITHMETIC AND LOGIC UNITS

As the name implies, the *arithmetic and logic unit (ALU)* is a universal combinational logic device capable of performing both arithmetic and logic operations. It is this versatility that makes the ALU an attractive building block in the *central processing unit (CPU)* of a computer or microprocessor. It is the object of this section to develop the techniques required to design and cascade ALU devices following three very different approaches: the *dedicated ALU approach*, the *MUX approach*, and the *dual-rail approach* with completion signals.

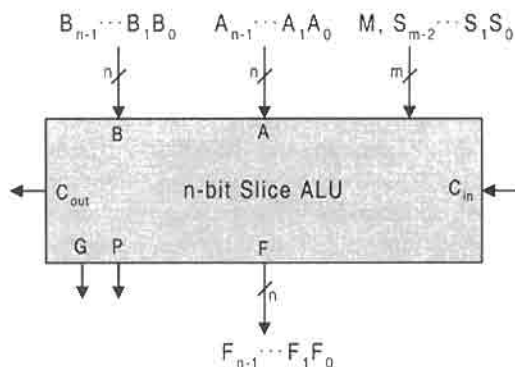
The number and complexity of the operations that a given ALU can perform is a matter of the designer's choice and may vary widely from ALU to ALU, as will be demonstrated in this section. However, the choice of operations is usually drawn from the list in Fig. 8.24. Other possible operations include zero, unity, sign-complement, magnitude comparison, parity generation, multiplication, division, powers, and shifting. Multiplication, division, and related operations such as arithmetic shifting are complex and are found only in the most sophisticated ALU chips. Also, the AND, OR, and XOR operations are often applied to complemented and uncomplemented operands, making possible a wide assortment of such operations.

Presented in Fig. 8.25 is the block diagram symbol for a general  $n$ -bit slice ALU. This ALU accepts two  $n$ -bit input operands,  $B_{n-1} \cdots B_1 B_0$  and  $A_{n-1} \cdots A_1 A_0$ , and a carry-in bit,  $C_{in}$ , and operates with them in some predetermined way to output an  $n$ -bit function,  $F_{n-1} \cdots F_1 F_0$  and a carry-out bit,  $C_{out}$ . Here, the term  *$n$ -bit slice* indicates a partition of identical  $n$ -bit modules of stages that can be cascaded in parallel. Thus, an FA in an  $n$ -bit R-C adder could be called a 1-bit slice for that adder. Also, use of sign-complement arithmetic avoids the need for both carry and borrow parameters.

| Arithmetic Operations | Logic Operations |
|-----------------------|------------------|
| Negation              | Transfer         |
| Increment             | Complementation  |
| Decrement             | AND              |
| Addition              | OR               |
| Subtraction           | XOR (EQV)        |

**FIGURE 8.24**

Arithmetic and logic operations common to ALUs.

**FIGURE 8.25**

Block diagram symbol for a general 1-bit slice ALU with CLA capability.

The choice of operation between the two operands,  $A$  and  $B$ , is determined by the  $m$  mode/select inputs,  $M, S_{m-2} \dots S_1 S_0$ , shown in Fig. 8.25. The mode input  $M$  sets the ALU for either an arithmetic or logic operation, and the function select inputs,  $S_{m-2} \dots S_1 S_0$ , determine which particular operation, within the arithmetic or logic mode, is to be performed. Just as the carry-out bit  $C_{out}$  is required for cascading standard R-C arithmetic units, as in Fig. 8.5, the carry propagate and carry generate bits,  $P$  and  $G$ , are required for cascading carry look-ahead (CLA) units. In this section, ALUs with R-C and CLA capabilities are designed. Commercial ALU chips are available that have both of these features.

### 8.8.1 Dedicated ALU Design Featuring R-C and CLA Capability

The EV operation table in Fig. 8.26 represents a simple 1-bit slice ALU capable of performing four specific arithmetic functions and four specific logic functions, all on command of

|                       | M | S <sub>1</sub> | S <sub>0</sub> | F                                     | Operation*                                                                       | C <sub>out</sub>                                       |
|-----------------------|---|----------------|----------------|---------------------------------------|----------------------------------------------------------------------------------|--------------------------------------------------------|
| Arithmetic Operations | 0 | 0              | 0              | $A \oplus C_{in}$                     | Transfer (LSB $C_{in} = 0$ ) or increment (LSB $C_{in} = 1$ ) of $A$             | $A \cdot C_{in}$                                       |
|                       | 0 | 0              | 1              | $\overline{A} \oplus C_{in}$          | 1's (LSB $C_{in} = 0$ ) or 2's (LSB $C_{in} = 1$ ) complement of $A$             | $\overline{A} \cdot C_{in}$                            |
|                       | 0 | 1              | 0              | $A \oplus B \oplus C_{in}$            | $A$ plus $B$ if LSB $C_{in} = 0$ or $A$ plus $B$ plus 1 if LSB $C_{in} = 1$      | $C_{in}(A \oplus B) + A \cdot B$                       |
|                       | 0 | 1              | 1              | $\overline{A} \oplus B \oplus C_{in}$ | $B$ minus $A$ if LSB $C_{in} = 1$ or $\overline{A}$ plus $B$ if LSB $C_{in} = 0$ | $C_{in}(\overline{A} \oplus B) + \overline{A} \cdot B$ |
| Logic Operations      | 1 | 0              | 0              | $A$                                   | Transfer $A$                                                                     | 0                                                      |
|                       | 1 | 0              | 1              | $\overline{A}$                        | Complement of $A$                                                                | 0                                                      |
|                       | 1 | 1              | 0              | $A + B$                               | $A$ OR $B$                                                                       | 0                                                      |
|                       | 1 | 1              | 1              | $\overline{A} + B$                    | $A$ complement OR $A$                                                            | 0                                                      |

\* Subtraction operations assume 2's complement arithmetic.

**FIGURE 8.26**

Operation table for a simple 1-bit slice ALU showing output functions,  $F$  and  $C_{out}$ , for four arithmetic operations ( $M = 0$ ) and four logic operations ( $M = 1$ ).

the three mode/select inputs  $M$ ,  $S_1$ , and  $S_0$ . On the right side of the table are the function expressions  $F$ , a brief description of each function operation, and carry-out expressions,  $C_{out}$ . The expressions for  $F$  and  $C_{out}$  are deduced from Eqs. (8.2) for the full adder (FA) together with Eqs. (3.23). Note that *false carry rejection* during the logic mode is realized by placing zeros in the  $C_{out}$  column for the four logic operations—the carry-out function has no relevance in a logic operation. Notice further that the two possible logic states for  $C_{in}$  lead to different interpretations for each of the four arithmetic operations. For example,  $A \oplus C_{in}$  is the transfer of  $A$  if  $C_{in} = 0$ , but represents the increment of  $A$  if the LSB  $C_{in} = 1$  ( $B = 0$  is implied). Or,  $\bar{A} \oplus C_{in}$  represents the 1's complement of  $A$  if LSB  $C_{in} = 0$  but is the 2's complement of  $A$  if LSB  $C_{in} = 1$  ( $B = 1$  is implied). Subtraction operations by this ALU are carried out by 2's complement arithmetic as discussed in Subsection 2.9.2.

The dedicated ALU of Fig. 8.26 is now designed by using the EV K-map methods with XOR patterns as discussed in Section 5.2. Shown in Figs. 8.27a and 8.27b are the third-order EV K-maps for  $F$  and  $C_{out}$ , which are plotted directly from the operation table in Fig. 8.26 by using the mode/select inputs as the K-map axes. These third-order EV K-maps represent three orders of map compression because there are three EVs. By compressing the third-order K-maps by one additional order (hence now four orders of K-map compression), there results the second-order EV K-maps shown in Figs. 8.27c and 8.27d. Optimum cover is then extracted from these second-order K-maps by using associative XOR-type patterns, as indicated by the shaded loops. (See Section 5.2 for a discussion of associative patterns). From these K-maps there results

$$\left\{ \begin{array}{l} F = (\bar{M}C_{in}) \oplus (A \oplus S_0) \oplus (S_1 B) + MS_1 B \\ C_{out} = (\bar{M}C_{in})[(A \oplus S_0) \oplus (S_1 B)] + \bar{M}(S_1 B)(A \oplus S_0) \end{array} \right\}, \quad (8.13)$$

which represent four-level logic with a collective gate/input tally of 10/22 excluding any inverters that may be required. Notice that several terms in Eqs. (8.13) are shared between

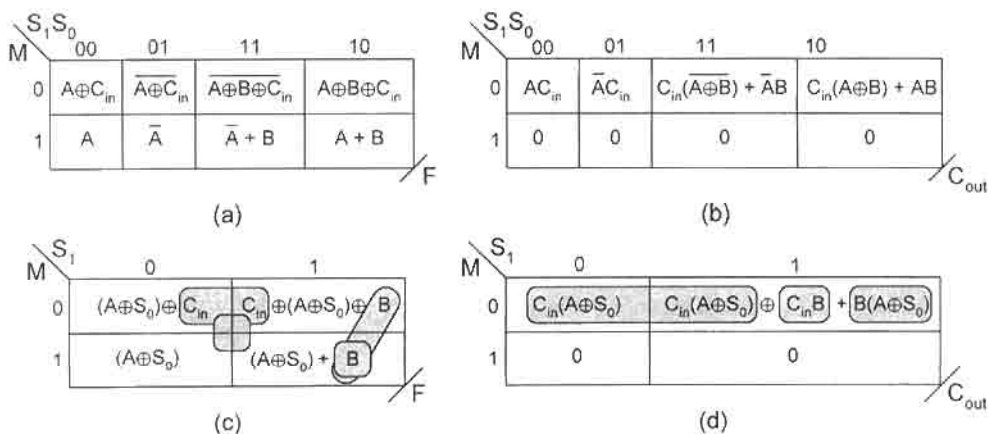
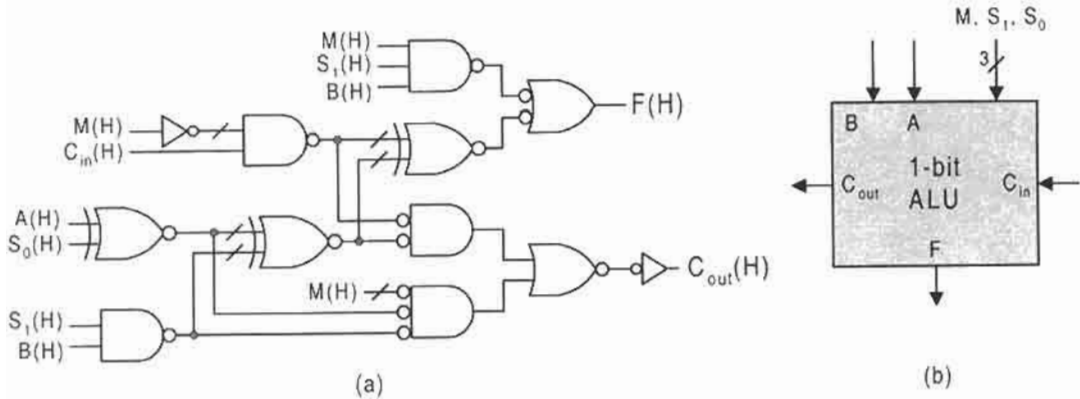


FIGURE 8.27

K-map representations for function  $F$  and carry-out  $C_{out}$  given in the operation table of Fig. 8.26 for a 1-bit slice ALU. (a), (b) Third-order EV K-maps plotted directly from Fig. 8.26 (c), (d) Second-order EV K-maps showing optimum cover for the two-output system.



**FIGURE 8.28**

(a) Optimum gate-level implementation of the 1-bit slice ALU represented by the operation table of Fig. 8.26 and by Eqs. 8.13. (b) Block diagram symbol for the 1-bit ALU in (a).

the two functions, a result of the optimal organization of the operations in the table of Fig. 8.26.

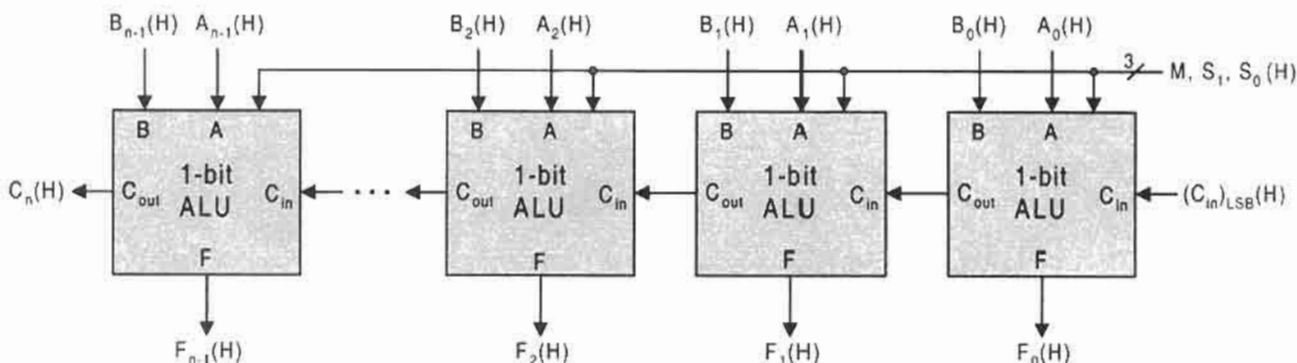
Some explanation of Eqs. (8.13) is necessary. Referring to function  $F$ , the separate “island” loop for operand  $B$  in cell 3 of Fig. 8.27c is necessary to complete the cover for that cell. This is so because after  $(A \oplus S_0)$  and  $B$  have been used to loop out the associative patterns, cover still remains in cell 3, as explained for a similar case in Section 5.4. The residual cover is extracted either by looping out operand  $B$  as an island to give the term  $MS_1B$ , or by looping out  $(A \oplus S_0)$  in the  $M$  domain to give the term  $M(A \oplus S_0)$ . It is the former that is chosen for this design example.

Equations (8.13) can be implemented in a number of ways by using discrete logic. One means of accomplishing this is shown in Fig. 8.28, where use is made of NAND/NOR/EQV logic, assuming that all inputs and outputs are active high. Notice that this is a four-level circuit with a maximum fan-in of 3. Also, the reader is reminded that two active low inputs to an XOR gate or EQV gate retains the function, as discussed in Subsection 3.9.4.

An  $n$ -bit ripple-carry (R-C) ALU can be produced by cascading the 1-bit slice ALU of Fig. 8.28 in a manner similar to that used to produce an  $n$ -bit R-C adder from  $n$  FAs in Fig. 8.5. This is done in Fig. 8.29, but with the added requirement of connecting the three mode/select input to all stages as shown. It is also possible to construct an  $n$ -bit R-C ALU by cascading  $m$ -bit ALU modules in a manner similar to cascading configuration of R-C adders in Fig. 8.6.

The two functions in Eqs. (8.13) are not the only expressions that can be derived from the operation table for  $F$  and  $C_{out}$  in Fig. 8.26. Referring to Fig. 5.9, it can be seen that the functions  $F_1$  and  $F_2$  are exactly those for  $F$  and  $C_{out}$ , respectively, if the appropriate substitutions are made and if the don't cares in the  $F_2$  K-map are each set to logic zero. Thus, from Eqs. (5.71) and (5.72) the two outputs for the 1-bit ALU now become either

$$\left\{ \begin{array}{l} F = \bar{M}[C_{in} \oplus (A \oplus S_0) \oplus (S_1 B)] + M(A \oplus S_0 + S_1 B) \\ C_{out} = \bar{M}C_i[(A \oplus S_0) \oplus (S_1 B)] + \bar{M}(S_1 B)(A \oplus S_0) \end{array} \right\} \quad (8.14)$$

**FIGURE 8.29**

An  $n$ -bit R-C ALU produced by cascading  $n$  1-bit slice ALUs of the type shown in Fig. 8.28.

or

$$\left\{ \begin{array}{l} F = [\overline{M}S_1B(A \oplus S_0)] \oplus (\overline{M}C_i) \oplus (S_1B) \\ C_{out} = \overline{M}C_i[(A \oplus S_0) \oplus (S_1B)] + \overline{M}(S_1B)(A \oplus S_0) \end{array} \right\}, \quad (8.15)$$

depending on whether or not subfunction partitioning is used for a combined CRMT and two-level result for function  $F$ . The two outputs in Eqs. (8.14) represent a five-level system with a combined gate/input tally of 11/24, and those in Eqs. (8.15) represent a four-level system with a total gate/input tally of 11/25, both excluding possible inverters. Thus, the CRMT results in Section 5.11 are comparable but somewhat less optimal than those of Eqs. (8.13) extracted from K-maps.

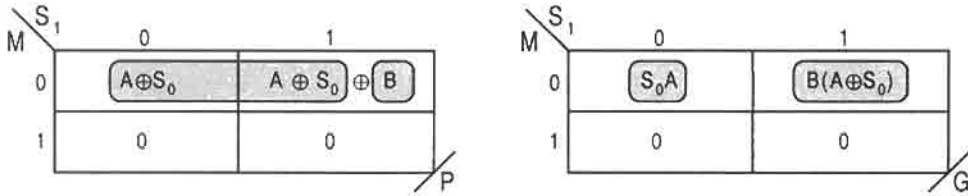
The  $n$ -bit R-C ALU of Fig. 8.29, like the R-C adder, suffers a size limitation due to the ripple-carry effect, as discussed in Subsection 8.2.3. To overcome this limitation the carry look-ahead (CLA) feature can be coupled with the ALU design. (See Section 8.4 for a discussion of the CLA adder.) In Fig. 8.30 is the I/O table for the 1-bit slice ALU with CLA

|                       | M | S <sub>1</sub> | S <sub>0</sub> | F                                     | P              | G                      |
|-----------------------|---|----------------|----------------|---------------------------------------|----------------|------------------------|
| Arithmetic Operations | 0 | 0              | 0              | $A \oplus C_{in}$                     | $\overline{A}$ | 0                      |
|                       | 0 | 0              | 1              | $\overline{A} \oplus C_{in}$          | $\overline{A}$ | A                      |
|                       | 0 | 1              | 0              | $A \oplus B \oplus C_{in}$            | $(A \oplus B)$ | $A \cdot B$            |
|                       | 0 | 1              | 1              | $\overline{A} \oplus B \oplus C_{in}$ | $(A \oplus B)$ | $\overline{A} \cdot B$ |
| Logic Operations      | 1 | 0              | 0              | A                                     | 0              | 0                      |
|                       | 1 | 0              | 1              | $\overline{A}$                        | 0              | 0                      |
|                       | 1 | 1              | 0              | $A + B$                               | 0              | 0                      |
|                       | 1 | 1              | 1              | $\overline{A} + B$                    | 0              | 0                      |

\* Subtraction operations assume 2's complement arithmetic.

**FIGURE 8.30**

Operation table for the simple 1-bit slice ALU of Fig. 8.26 showing CLA output functions  $P$  and  $G$  based on Eqs. (8.7).

**FIGURE 8.31**

EV K-maps and minimum cover for carry propagate ( $P$ ) and carry generate ( $G$ ) parameters given in Fig. 8.30.

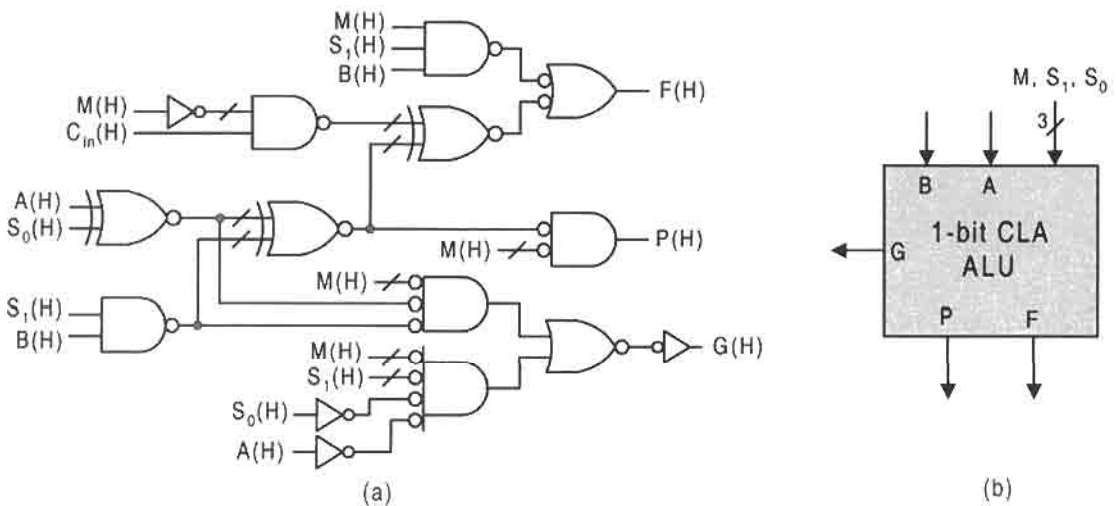
capability. Here, the  $C_{out}$  functions of Fig. 8.26 are replaced with those for carry propagate,  $P$ , and carry generate,  $G$ , which are derived from Eqs. (8.2) and (8.7). Logic 0's are placed in the  $P$  and  $G$  columns for the logic operations to indicate false carry rejection, as was done in the operation table of Fig. 8.26.

The design of the 1-bit slice ALU of Fig. 8.30 continues by plotting the  $P$  and  $G$  outputs in second-order K-maps as was the case for  $C_{out}$  in Fig. 8.27d. This is done in Fig. 8.31, where optimum cover is indicated with shaded loops for each of the outputs, yielding the results

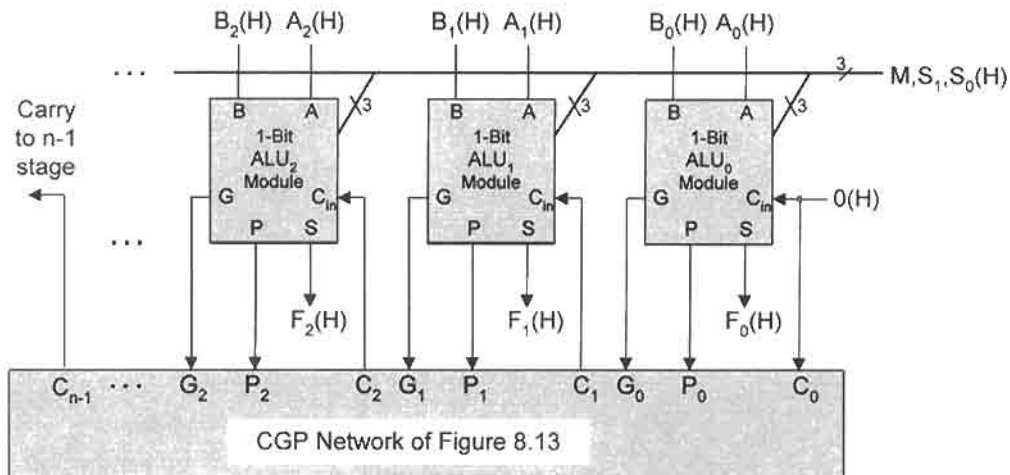
$$\left\{ \begin{array}{l} P = \bar{M}[(A \oplus S_0) \oplus (S_1 B)] \\ G = \bar{M}(S_1 B)(A \oplus S_0) + \bar{M} \bar{S}_1 S_0 A \end{array} \right\}. \quad (8.16)$$

Notice that a single associative pattern is used to extract optimum cover for output,  $P$ .

In completing the design of the 1-bit slice ALU with CLA capability, it must be remembered that it is the carry parameters in Eqs. (8.16) that take the place of the  $C_{out}$  expression in Eqs. (8.13). This is done by combining the carry expressions of Eqs. (8.16) with the expression for function  $F$  in Eqs. (8.13). The result is the logic circuit in Fig. 8.32a and its block diagram symbol in Fig. 8.32b for a 1-bit slice ALU with CLA capability.

**FIGURE 8.32**

(a) Optimum gate-level implementation of the 1-bit slice ALU with CLA capability represented by the operation table of Fig. 8.30 and by Eqs. (8.16). (b) Block diagram symbol for the 1-bit ALU in (a).

**FIGURE 8.33**

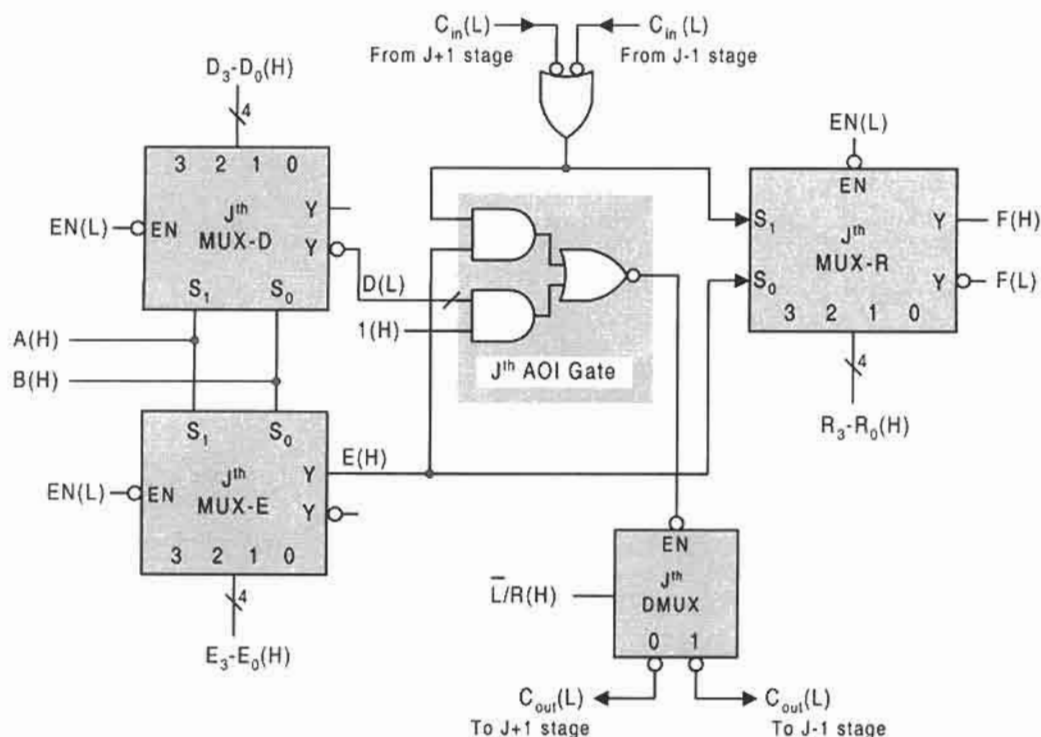
The three least significant stages of an  $n$ -bit carry look-ahead (CLA) ALU showing the block symbol for the carry generate/propagate network used between 1-bit modules.

The 1-bit slice ALU module in Fig. 8.32 can be cascaded by using the CLA carry generate/propagate (CGP) network given in Fig. 8.13. This is accomplished in Fig. 8.33, where the three least significant 1-bit stages are shown connected to the CGP network. Cascadable units of this type can be cascaded as groups to produce even larger units. This is done by connecting the carry-out of one  $n$ -bit stage to the carry-in of another, etc., with the mode/select inputs connected to all stages.

### 8.8.2 The MUX Approach to ALU Design

There are occasions when a designer requires an ALU that is fully programmable, that will perform a wide variety of tasks both arithmetic and logic, and that can be put to a circuit board with off-the-shelf components, or that can be designed for VLSI circuit applications. The MUX approach to ALU design can provide this versatility and computational power, which are difficult to achieve otherwise. In this subsection a programmable ALU (PALU) will be designed by the MUX approach that can perform the following operations on two  $n$ -bit operands:

1. Arithmetic operations
  - (a) Add with carry, or increment
  - (b) Subtract with borrow, or decrement
  - (c) Partial multiply or divide steps
  - (d) One's or 2's complement
2. Comparator operations
  - (a) Equal
3. Bitwise logic operations
  - (a) AND, OR, XOR, and EQV
  - (b) Logic-level conversion
  - (c) Transmit or complement data bit

**FIGURE 8.34**

Implementation of the  $J$ th 1-bit slice PALU with shift left and shift right capability by using three 4-to-1 MUXs, a DMUX, a NAND gate, and an AOI gate.

#### 4. Shift operations

##### (a) Shift or rotate left or right by one bit

It is required that each 1-bit slice of the PALU have a resulting output function  $F$  and that the PALU be cascaded with respect to the carry-in and carry-out parameters,  $C_{in}$  and  $C_{out}$ . Also, it is required that the PALU operate with *false carry rejection*, that is, that  $C_{out}$  be disabled for all nonarithmetic operations.

The logic circuit diagram for the  $J$ th 1-bit slice PALU and its block diagram symbol are presented in Figs. 8.34 and 8.35. As can be seen, the PALU consists of three 4-to-1 MUXs, a 1-to-2 DMUX, a NAND gate, and an AOI gate (see Subsection 7.7.1 for a discussion of AOI gates). For reference purposes the MUXs are named

MUX-D  $\Rightarrow$  Disable Carry MUX — Output  $D$

MUX-E  $\Rightarrow$  Extend Carry MUX — Output  $E$

MUX-R  $\Rightarrow$  Result MUX — Output  $F$

To help understand how the PALU is able to perform such a variety of tasks as listed earlier, it useful to write the Boolean expressions for the outputs from the three MUXs and from

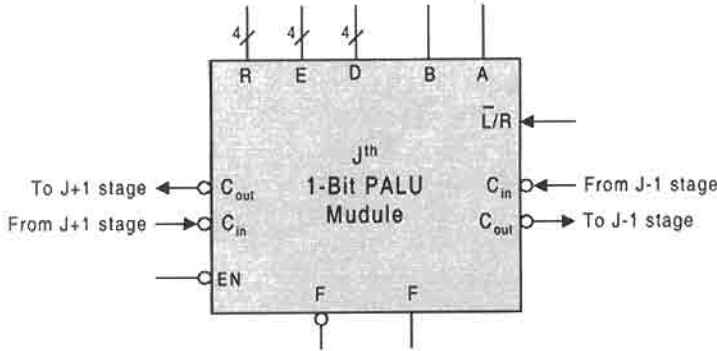


FIGURE 8.35

Block diagram symbol for the  $J$ th 1-bit PALU module of Fig. 8.34.

the AOI gate. Referring to Fig. 8.34, these outputs are

$$\left\{ \begin{array}{l} F = R = \sum_{i=3}^0 = R_i m_i = R_3 C_{in} E + R_2 C_{in} \bar{E} + R_1 \bar{C}_{in} E + R_0 \bar{C}_{in} \bar{E} \\ E = \sum_{i=3}^0 = E_i m_i = E_3 AB + E_2 A \bar{B} + E_1 \bar{A} B + E_0 \bar{A} \bar{B} \\ D = \sum_{i=3}^0 = D_i m_i = D_3 AB + D_2 A \bar{B} + D_1 \bar{A} B + D_0 \bar{A} \bar{B} \end{array} \right\} \quad (8.17)$$

and

$$C_{out} = C_{in} E + \bar{D}, \quad (8.18)$$

where the control address inputs  $R_i$ ,  $E_i$ , and  $D_i$  are the coefficients of  $m_i$  that represent the canonical forms of input sets  $C_{in}$ ,  $E$  or  $A$ ,  $B$ . Thus,  $R_2 m_2 = R_2 C_{in} \bar{E}$  or  $E_0 m_0 = E_0 \bar{A} \bar{B}$ , etc.

Notice that Eqs. (8.17) derive directly from Eq. (6.5), but where now the data select and data inputs are interchanged—that is, there are now four data select inputs (control address inputs) each for  $R_i$ ,  $E_i$  and  $D_i$ , two data inputs,  $A$  and  $B$ . Referring to the K-map in Fig. 6.4c for a 4-to-1 MUX, it is easy to see that 16 possible functions in data variables  $S_1$  and  $S_0$  result by assigning 1's and 0's to the four variables  $I_3$ ,  $I_2$ ,  $I_1$ , and  $I_0$  in the K-map, taking  $EN=1$ . Thus, for programming purposes, the four components for each of the control address inputs (coefficients of  $m_i$ ),  $R_i$ ,  $E_i$ , and  $D_i$ , are encoded in the order of descending positional weight, but are given in decimal equivalent to represent any one of the 16 possible logic functions in variables,  $A$  and  $B$ . As examples,  $D = A + \bar{B}$  is represented as  $D = 1101_2 = 13_{10}$ ,  $E = A \bar{B}$  if  $E = 0100_2 = 4_{10}$ , or  $F = E \odot C_{in}$  is represented as  $R = 1001_2 = 9_{10}$  and  $F = E \oplus C_{in}$  when  $R = 0110_2 = 6_{10}$ , etc.

In Fig. 8.34 it can be seen that the MUX output signals,  $D(L)$  and  $E(H)$ , together with carry-in  $C_{in}$ , drive the AOI gate, but that only the MUX output  $E(H)$  combines with  $C_{in}(H)$  in MUX-R to produce the resultant function  $F$ . The carry-out  $C_{out}(L)$  from the AOI gate, on the other hand, is a function of  $E$ ,  $D$ , and the carry-in input,  $C_{in}$ , as indicated in Eq. (8.18). This is necessary for the arithmetic operations that the PALU must perform. The  $\bar{D}$  in Eq. (8.18) is used to disable  $C_{out}$  for nonarithmetic operations. If  $D$  is set equal to 0 for

a given nonarithmetic operation, it follows that  $\bar{D} = 1$  and  $C_{out}(L) = 1(L) = 0(H)$ , which is interpreted as a carry-out disable for that operation. Thus, the carry-out disable feature of the PALU is equivalent to the false carry rejection used in the ALU operation tables of Figs. 8.26 and 8.30. In a sense, the output of MUX-D performs the same mode control operation as does  $M$  in the ALU of Figs. 8.28 and 8.29.

The operation of the PALU is illustrated by 12 examples in the operation table of Fig. 8.36 with the help of the  $n$ -bit PALU shown in Fig. 8.37. The first five operations in Fig. 8.36 are arithmetic while the last seven are logic. To understand the entries in this table, some explanation is necessary. The control address inputs,  $R$ ,  $E$ , and  $D$ , are the binary coefficients in Eqs. (8.17) represented in decimal. Operations (1) through (4) are arithmetic operations that follow directly from Eqs. (8.2) for the FA and require specific values for the  $C_{in}$  to the LSB stage of the  $n$ -bit PALU in Fig. 8.37. For these arithmetic operations, the carry must propagate from LSB-to-MSB (left), which requires that the direction input be set to  $\bar{L}/R = 0$ . Note that this PALU cannot support CLA capability since not all carry operations are based on Eqs. (8.2).

Operation (1) requires that the operand magnitudes fall within the limits set in Subsection 8.3.1. Operation (2),  $A$  minus  $B$ , adds  $A$  to the 2's complement of  $B$  and requires

|      | Operation *                                  | $F$                              | $C_{out}$                             | $\bar{L}/R$ | $R$ | $E$ | $D$ |
|------|----------------------------------------------|----------------------------------|---------------------------------------|-------------|-----|-----|-----|
| (1)  | $A$ plus $B$ (LSB $C_{in} = 0$ )             | $A \oplus B \oplus C_{in}$       | $C_{in}(A \oplus B) + AB$             | 0           | 6   | 6   | 7   |
| (2)  | $A$ minus $B$ (LSB $C_{in} = 1$ )            | $A \oplus \bar{B} \oplus C_{in}$ | $C_{in}(A \oplus \bar{B}) + A\bar{B}$ | 0           | 6   | 9   | 11  |
| (3)  | Increment $B$ (LSB $C_{in} = 1$ )            | $B \oplus C_{in}$                | $C_{in} \cdot B$                      | 0           | 6   | 10  | 15  |
| (4)  | 2' complement of $A$ (LSB $C_{in} = 1$ )     | $\bar{A} \oplus C_{in}$          | $C_{in} + \bar{A}$                    | 0           | 6   | 3   | 3   |
| (5)  | $A = B$ (LSB $C_{in} = 0$ )                  | $(A \odot B) \bar{C}_{in}$       | $C_{in} + A \oplus B$                 | 0           | 2   | 9   | 9   |
| (6)  | $A \oplus B$                                 | $A \oplus B$                     | 1                                     | $\phi$      | 10  | 6   | 0   |
| (7)  | $A \cdot B$                                  | $A \cdot B$                      | 1                                     | $\phi$      | 10  | 8   | 0   |
| (8)  | $\bar{A} + B$                                | $\bar{A} + B$                    | 1                                     | $\phi$      | 10  | 11  | 0   |
| (9)  | Complement $A$                               | $\bar{A}$                        | 1                                     | $\phi$      | 10  | 3   | 0   |
| (10) | Transfer $B$                                 | $B$                              | 1                                     | $\phi$      | 10  | 10  | 0   |
| (11) | Shift $A$ left 1 bit (Fill = LSB $C_{in}$ )  | $C_{in}$                         | $A$                                   | 0           | 12  | 0   | 3   |
| (12) | Shift $B$ right 1 bit (Fill = MSB $C_{in}$ ) | $C_{in}$                         | $B$                                   | 1           | 12  | 0   | 5   |

\* Subtraction operations assume 2's complement arithmetic with LSB  $C_{in} = 1$ .

**FIGURE 8.36**

Twelve sample operations generated by the 1-bit slice PALU in Fig. 8.34 showing the shift direction input and the decimal values for the data select variables,  $R$ ,  $E$ , and  $D$ .





given in Fig. 8.36.

$$\begin{aligned}
 F &= (A \odot B) \bar{C}_{in} \\
 C_{out} &= C_{in} E + D \\
 &= C_{in} (A \odot B) + A \oplus B \\
 &= C_{in} + A \oplus B
 \end{aligned}$$

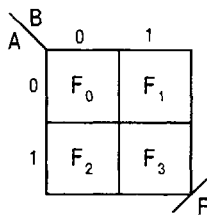
By comparing the result  $F = (A \odot B) \bar{C}_{in}$  with Eqs. (6.20) for a 1-bit comparator, it becomes evident that  $\bar{C}_{in} = eq$  is the  $(A = B)$  output from the next least significant stage.

The remaining two operations, (11) and (12), are shift operations that can be considered both arithmetic and logic in nature. For example, operation (11) shifts word  $A$  by one bit to the left, which can be interpreted as a  $\times 2$  operation, and hence a partial multiplication step. Referring to Section 8.6 and Algorithm 2.10, it is clear that a bit-by-bit left shift is a partial step used to execute the multiplication of two operands. This requires that the  $J$ th function bit  $F_J$  receive the  $C_{in}$  from the  $(J - 1)$ th stage, and that the  $C_{out}$  of the  $J$ th stage be the  $A$  input to that stage, for which  $F = R = C_{in} = 1100_2 = 12_{10}$  and  $D = \bar{A} = 0011_2 = 3_{10}$ .

There are many other operations possible by the PALU that are not listed in the operation table of Fig. 8.36. As examples, operation (9) can be interpreted as the 1's complement of  $A$  according to Algorithm 2.7 as applied to binary, and operation (7) can be considered as a partial product required in the multiplication of two operands. Also, arithmetic operations other than the operations (1), (2), and (3) are possible.

There are a total of 16 bitwise logic operations that can be generated by the PALU, but only five are listed in Fig. 8.36. For reference purposes, the 16 logic functions in two operands,  $A$  and  $B$ , that can be generated by Eqs. (8.17) are summarized by

$$F = \sum_{i=3}^0 F_i m_i = F_3 AB + F_2 A\bar{B} + F_1 \bar{A}B + F_0 \bar{A}\bar{B} \quad (8.19)$$

|                                                                                     |  |       |       |       |       |                         |       |       |       |       |               |
|-------------------------------------------------------------------------------------|--|-------|-------|-------|-------|-------------------------|-------|-------|-------|-------|---------------|
|  |  |       |       |       |       |                         |       |       |       |       |               |
|                                                                                     |  | $F_3$ | $F_2$ | $F_1$ | $F_0$ | $F$                     | $F_3$ | $F_2$ | $F_1$ | $F_0$ | $F$           |
| 0                                                                                   |  | 0     | 0     | 0     | 0     | 0                       | 1     | 0     | 0     | 0     | $A \cdot B$   |
| 0                                                                                   |  | 0     | 0     | 0     | 1     | $\bar{A} \cdot \bar{B}$ | 1     | 0     | 0     | 1     | $A \odot B$   |
| 0                                                                                   |  | 0     | 0     | 1     | 0     | $\bar{A} \cdot B$       | 1     | 0     | 1     | 0     | $B$           |
| 0                                                                                   |  | 0     | 0     | 1     | 1     | $\bar{A}$               | 1     | 0     | 1     | 1     | $\bar{A} + B$ |
| 1                                                                                   |  | 0     | 1     | 0     | 0     | $A \cdot \bar{B}$       | 1     | 1     | 0     | 0     | $A$           |
| 1                                                                                   |  | 0     | 1     | 0     | 1     | $\bar{B}$               | 1     | 1     | 0     | 1     | $A + \bar{B}$ |
| 1                                                                                   |  | 0     | 1     | 1     | 0     | $A \oplus B$            | 1     | 1     | 1     | 0     | $A + B$       |
| 1                                                                                   |  | 0     | 1     | 1     | 1     | $\bar{A} + \bar{B}$     | 1     | 1     | 1     | 1     | 1             |

(a)

(b)

**FIGURE 8.38**

The 16 possible bitwise logic functions for operands  $A$  and  $B$  according to Eq. (8.19) (a) K-map representation. (b) Tabular representation.

and can be represented either by the K-map in Fig. 8.38a or by the table in Fig. 8.38b. The table is generated by assigning 1's and 0's to the four coefficients,  $F_3$ ,  $F_2$ ,  $F_1$ , and  $F_0$ . These functions will again be used by an even more versatile ALU, which is designed in the following section.

## 8.9 DUAL-RAIL SYSTEMS AND ALUs WITH COMPLETION SIGNALS

As implied in Section 8.8, ALUs are important because they make possible the use of the same device to perform many different operations, thereby increasing versatility while minimizing the need to combine different modules for those operations. Because of these advantages, ALUs are commonly found in processors where the specific operations are performed on command from the controller in the processor. Although these ALUs support a variety of arithmetic and logic operations and may include CLA capability, they typically have single rail carries (like those treated in Section 8.8) and cannot communicate to the processor when a given operation has been completed. To remedy this situation, completion signals are issued following worst-case delays that are associated with the various ALU operations.

This section features a programmable ALU (PALU) that will issue a final completion (DONE) signal immediately following the completion of any operation, no matter how complex or simple it is. This is a significant feature for an ALU, since arithmetic operations require more time to complete (because of the carry problem) than do bitwise logic operations. Used in a microprocessor, PALUs with DONE signals avoid the need to assign worst-case delays to the various ALU operations. Thus, whenever an operation (logic or arithmetic) is completed, a DONE signal is sent to the CPU (central processing unit), thereby permitting immediate execution of the next process without unnecessary delay.

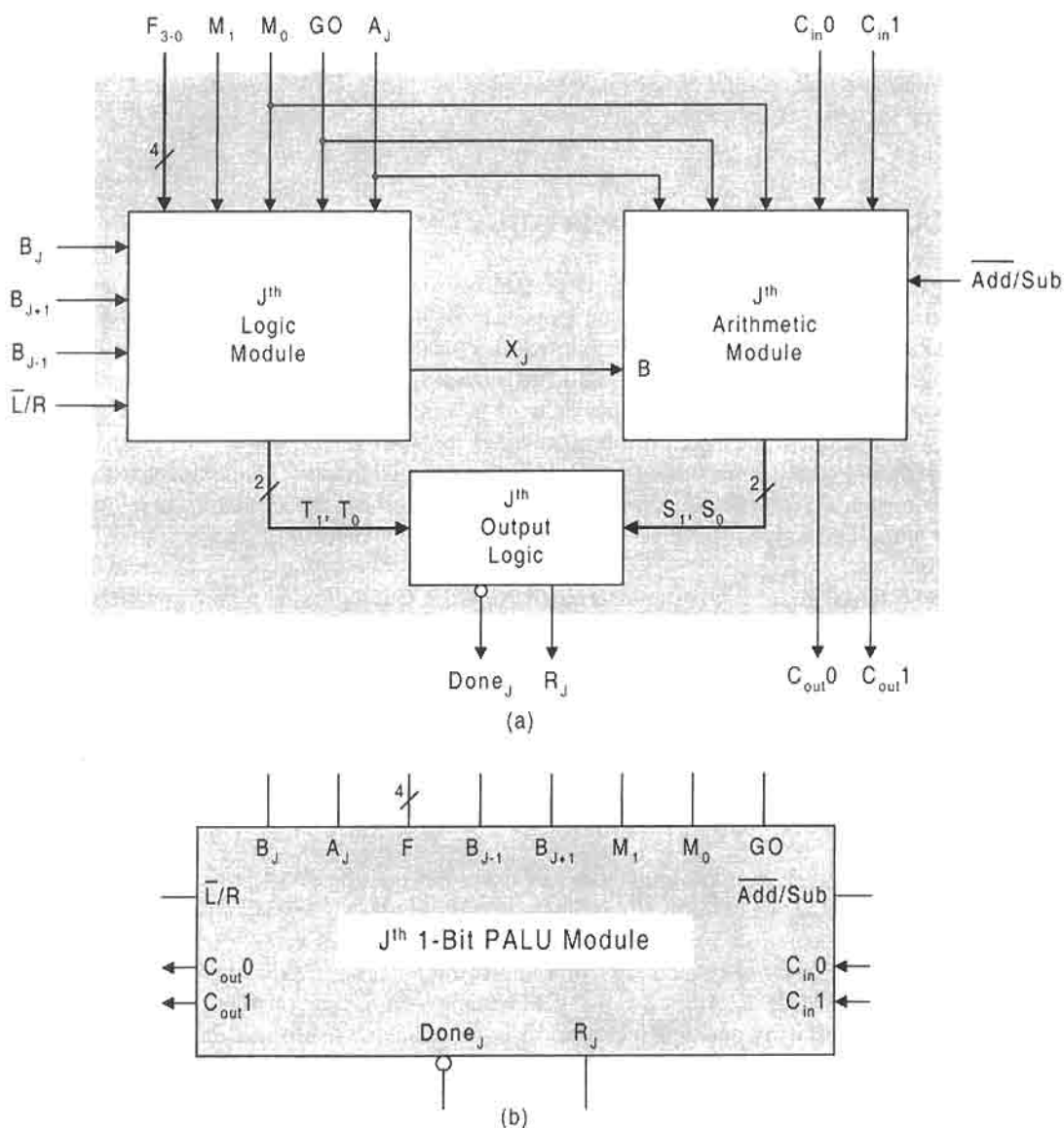
Listed in Fig. 8.39 are the four modes of operation that the PALU can perform. As indicated, the PALU can perform bitwise logic operations ( $M_1 M_0 = 01$ ), or it can perform left or right shift operations ( $M_1 M_0 = 11$ ) on operand  $B$ . But it can also perform arithmetic operations on the result of either a logic or a shift operation, as indicated by mode controls  $M_1 M_0 = 00$  and  $M_1 M_0 = 10$ , respectively. For example, any logic operation in Fig. 8.38 (e.g.,  $A \oplus B$ ) or shift in operand  $B$  can be added to or subtracted from operand  $A$ . With DONE signals issued following the completion of each process, it is clear that this PALU offers a higher degree of versatility than is available from the ALUs in Section 8.8.

An ALU will now be designed that is even more versatile than that of the MUX approach in Subsection 8.8.2. In addition, it is the goal of this section to develop the concepts

| $M_1$ | $M_0$ | MODE                    |
|-------|-------|-------------------------|
| 0     | 0     | Arithmetic on Logic     |
| 0     | 1     | Logic                   |
| 1     | 0     | Arithmetic on B-Shift   |
| 1     | 1     | B-Shift (right or left) |

**FIGURE 8.39**

Modes of PALU operation.

**FIGURE 8.40**

Block diagram structure (a) and block circuit symbol (b) for the  $J^{\text{th}}$  1-bit PALU module capable of performing the modes of operation listed in Fig. 8.39 with completion (*Done*) signals.

of dual-rail systems and completion signals, and apply them to the design of a PALU with operational capability defined by Fig. 8.39. Also, both ripple-carry (R-C) and carry look-ahead (CLA) designs will be considered. The following develops the details of this unusual but versatile PALU.

Shown in Figs. 8.40a and 8.40b are the block diagram and logic circuit symbol for a 1-bit slice PALU that can perform the operations represented in Fig. 8.39 and issue a *Done* signal following the completion of each operation. An inspection of Fig. 8.40 indicates that there

are inputs specific to the  $J$ th module and those that are global (applied to all modules in a cascaded system). Specific to the  $J$ th PALU module are the operands inputs  $A_J$ ,  $B_J$ ,  $B_{J-1}$ , and  $B_{J+1}$ . The  $B_{J-1}$  input arrives from the  $B$  input to the next LSB stage in a cascaded configuration and is necessary for left shifting. Similarly, the  $B_{J+1}$  input arrives from the  $B$  input to the next MSB stage to permit right shifting. The input and output dual-rail carries shown in Fig. 8.40 are also specific to the  $J$ th module and are defined as follows:

$$\left\{ \begin{array}{l} C_{in}0 = \text{carry-in 0 to stage } J \text{ from stage } J-1 \\ C_{in}1 = \text{carry-in 1 to stage } J \text{ from stage } J-1 \\ C_{out}0 = \text{carry-out 0 to stage } J+1 \text{ from stage } J \\ C_{out}1 = \text{carry-out 1 to stage } J+1 \text{ from stage } J \end{array} \right\}. \quad (8.20)$$

The meaning here is that  $C_{in}0 = 1$  when the carry-in to the  $J$ th stage is logic 0, and  $C_{in}1 = 1$  when the carry-in to the  $J$ th is logic 1. Thus, both carry-in parameters cannot be active at the same time. Similarly,  $C_{out}0 = 1$  when carry-out to the  $(J+1)$ th is logic 0, or  $C_{out}1 = 1$  when the carry to the  $(J+1)$ th is logic 1, where again only one carry parameter can be active at any given time.

The global inputs to the PALU include the two mode control inputs,  $M_1$  and  $M_0$ , the function generate signals,  $F_3$ ,  $F_2$ ,  $F_1$ , and  $F_0$ , a shift-direction input  $\bar{L}/R$  (meaning right, “not left” when active), an add/subtract input  $\bar{Add}/Sub$  (meaning subtract, not add when active), and a start signal called  $GO$ . The add/subtract control input operates the same as that use for the adder/subtractor design in Subsection 8.3.1, but only if the mode control  $M_0 = 0$  according to Fig. 8.39. Also, the shift-direction control  $\bar{L}/R$  is operational only if mode control  $M_1 = 1$ , as indicated in Fig. 8.39.

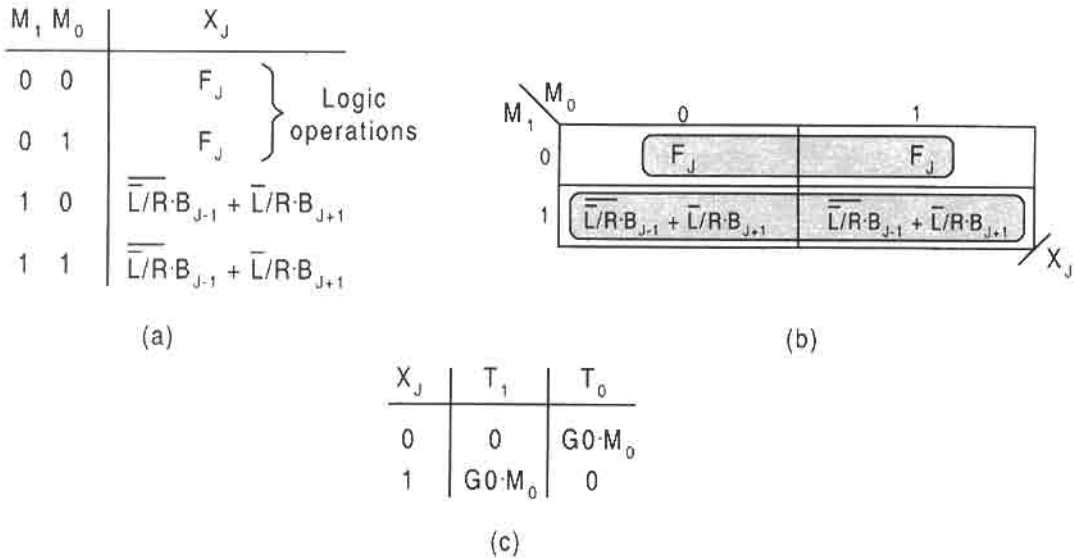
The two outputs,  $Done_J$  and  $R_J$ , are specific to the  $J$ th PALU module. When the  $J$ th stage result of a bitwise logic operation or arithmetic operation is indicated by the output  $R_J$ , a completion signal  $Done_J$  is issued. However, it is the requirement of an  $n$ -bit PALU design that a final (overall) completion signal,  $DONE$ , will not be issued until the  $Done$  signals from all  $n$  stages have become active. Thus, the results from those  $n$  stages must not be read until the final  $DONE$  has emerged.

**Logic Module** The logic module is responsible for carrying out both the 16 bitwise logic operations given by Eq. (8.19) and the shift left or right operation with 0 or 1 fill. (See Section 6.8 for details of a combinational shifter.) Presented in Fig. 8.41 are the output parameters,  $X_J$ ,  $T_1$ , and  $T_0$ , for the  $J$ th PALU logic module. The output function  $X_J$ , representing the mode control settings for logic and shift operations (according to Fig. 8.39), is given by the truth table and EV K-map in Figs. 8.41a and 8.41b. The dual-rail outputs from the logic module,  $T_1$  and  $T_0$ , are defined in the truth table of Fig. 8.41c and represent only logic and shift modes — arithmetic operations are excluded.

The output function  $X_J$  is read directly from the EV K-map in Fig. 8.41b and is

$$X_J = \bar{M}_1 F_J + M_1 (\bar{L}/R \cdot B_{J-1} + \bar{L}/R \cdot B_{J+1}), \quad (8.21)$$

where the quantity  $(\bar{L}/R \cdot B_{J-1} + \bar{L}/R \cdot B_{J+1})$  represents the shift left or right operation but only when mode control  $M_1 = 1$ , as required by the mode control table in Fig. 8.39. Thus, right shift by one bit occurs when  $\bar{L}/R = 1$  and left shift by one bit occurs when  $\bar{L}/R = 0$ . Function  $F_J$  represents the 16 possible bitwise logic operations, as in Eq. (8.19),

**FIGURE 8.41**

Requirements of output parameters for the  $J$ th PALU logic module according to Fig. 8.39. (a) Truth table showing mode settings for logic and shift operations. (b) EV K-map for (a). (c) Dual-rail outputs representing only logic and shift operations from logic module.

and is given here for reference purposes:

$$F_J = F_3 AB + F_2 A\bar{B} + F_1 \bar{A}B + F_0 \bar{A}\bar{B}. \quad (8.22)$$

The dual-rail outputs from the  $J$ th logic module, defined in Fig. 8.41c, are read directly from the truth table and are

$$\left\{ \begin{array}{l} T_1 = GO \cdot M_0 \cdot X \\ T_0 = GO \cdot M_0 \cdot \bar{X} \end{array} \right\}_J. \quad (8.23)$$

The meaning of these dual-rail parameters is as follows: For logic or shift operations, the mode control requirement is  $M_0 = 1$  according to Fig. 8.39. Thus, for  $GO = 1$  (start active),  $T_1 = 1$  and  $T_0 = 0$  if  $X_J = 1$ , or  $T_1 = 0$  and  $T_0 = 1$  if  $X_J = 0$ . The dual-rail outputs are necessary to generate completion signals following logic and shift operations.

Presented in Fig. 8.42 is the logic circuit for the  $J$ th PALU logic module as required by Eqs. (8.21)–(8.23). The 4-to-1 MUX provides the 16 possible bitwise logic functions of the two operands,  $A_J$  and  $B_J$ , as represented by Eq. (8.22), but only if the mode control setting is  $M_1 = 0$ . The shift right/left portion of the circuit is activated when the mode setting is  $M_1 = 1$ . Then when  $\bar{L}/R = 1$ ,  $B_{J+1}$  is received from the next MSB stage producing a 1-bit right shift, or if  $\bar{L}/R = 0$ ,  $B_{J-1}$  is received from the next LSB stage, forcing a 1-bit left shift.

**Arithmetic Module** To design an arithmetic module with completion signal capability, it is necessary to first establish the concept of dual-rail carries. This is accomplished by rewriting the equations for a full adder as they pertain to the  $J$ th 1-bit PALU arithmetic

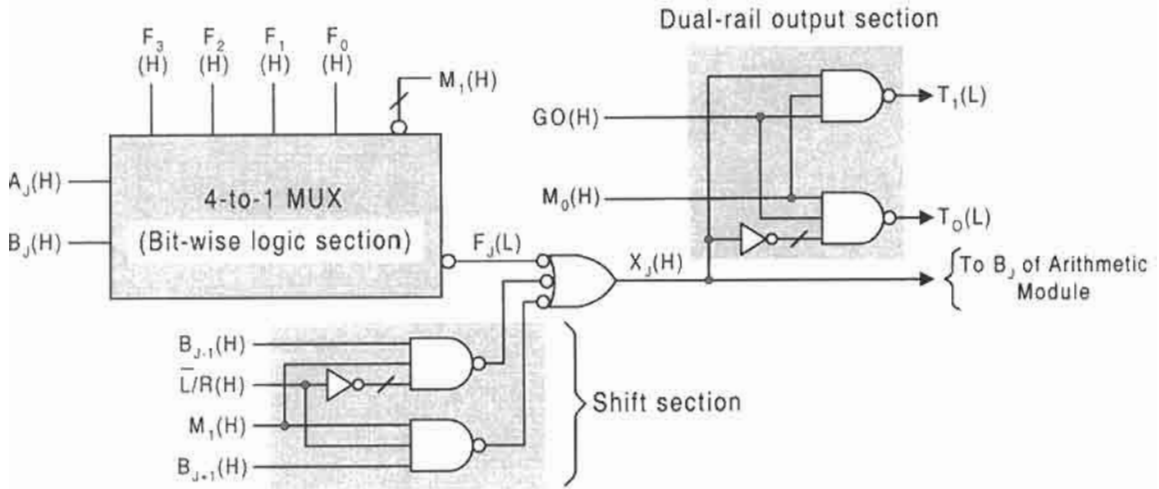


FIGURE 8.42

FIGURE 8.42

Logic circuit for the  $J$ th PALU logic module based on Eqs. (8.21)–(8.23).

module. The sum expression from Eqs. (8.2) is restated here as

$$S = A \oplus B \oplus C_{in}$$

and is partitioned into the following two parts with respect to logic 1 and logic 0:

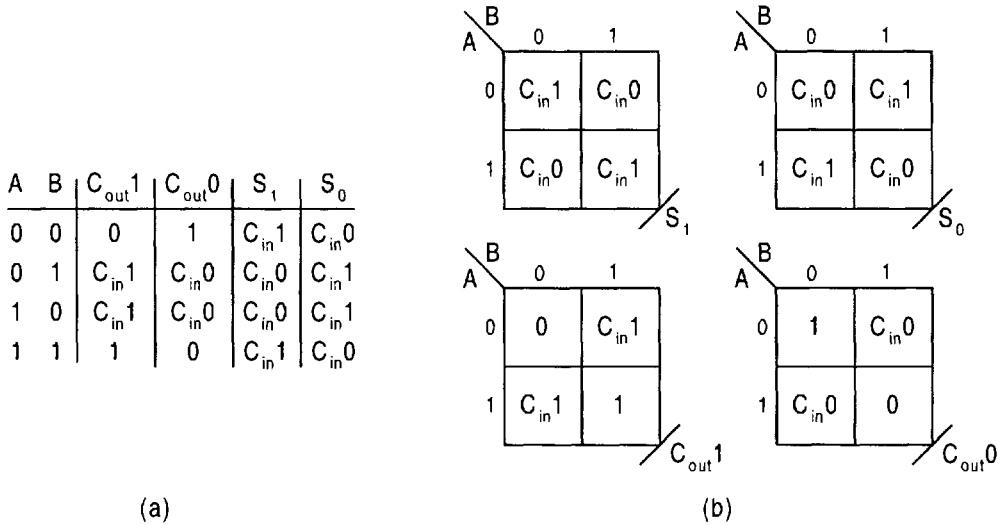
$$\begin{aligned} S_1 &= (A \oplus B)\bar{C}_{in} + (A \odot B)C_{in} \\ &= (A \oplus B)C_{in}0 + (A \odot B)C_{in}1 \end{aligned}$$

and

$$\begin{aligned} S_0 &= (A \oplus B)\bar{C}_{in} + (A \odot B)C_{in} \\ &= (A \oplus B)C_{in}1 + (A \odot B)C_{in}0 \end{aligned} \quad (8.24)$$

Here, use has been made of Eq. (3.4) in the form  $S = x\bar{y} + \bar{x}y$ , where  $x = A \oplus B$  and  $y = C_{in}$ . Thus, for  $S_1$ , carry-in is represented as  $\bar{C}_{in} = C_{in}0$ , and  $C_{in} = C_{in}1$  whereas for  $S_0$  the carry-in is represented as  $\bar{C}_{in} = C_{in}1$  and  $C_{in} = C_{in}0$ . The split-rail sums,  $S_1$  and  $S_0$ , in Eqs. (8.24), are summarized in Fig. 8.43a together with the dual-rail carry-outs,  $C_{out}1$  and  $C_{out}0$ , as they are affected by the operands,  $A$  and  $B$ , and the dual-rail carry-ins,  $C_{in}1$  and  $C_{in}0$ . Here,  $S_1$  is active if the sum is logic 1 otherwise inactive, or  $S_0$  is active if the sum is logic 0 otherwise inactive.

The carries have a similar meaning. An active  $C_{in/out}1$  implies a carry (in or out) of logic 1, and an active  $C_{in/out}0$  implies a carry (in or out) of logic 0. Thus,  $C_{in/out}1$  and  $C_{in/out}0$  cannot both be active (take logic 1) at the same time. Also,  $C_{in/out}1 = C_{in/out}0 = 0$  is an indeterminate state of no carry. In effect, a three-state code representation is used for carry-in  $\{C_{in}1, C_{in}0\}$ , carry-out  $\{C_{out}1, C_{out}0\}$ , and sums  $\{S_1, S_0\}$ . This means that each of these three dual-rail pairs can assume one of the three state values taken from the set  $\{00, 01, 10\}$ . All three pairs are required to be set initially in logic state (00) and then transit to either state (01) or (10) following the activation of the start signal  $GO$  shown in Fig. 8.40. State (11) is not permitted.

**FIGURE 8.43**

Requirements of a split-rail sum and carry system. (a) Truth table for sum and carry partitions. (b) EV K-maps plotted from the truth table in (a).

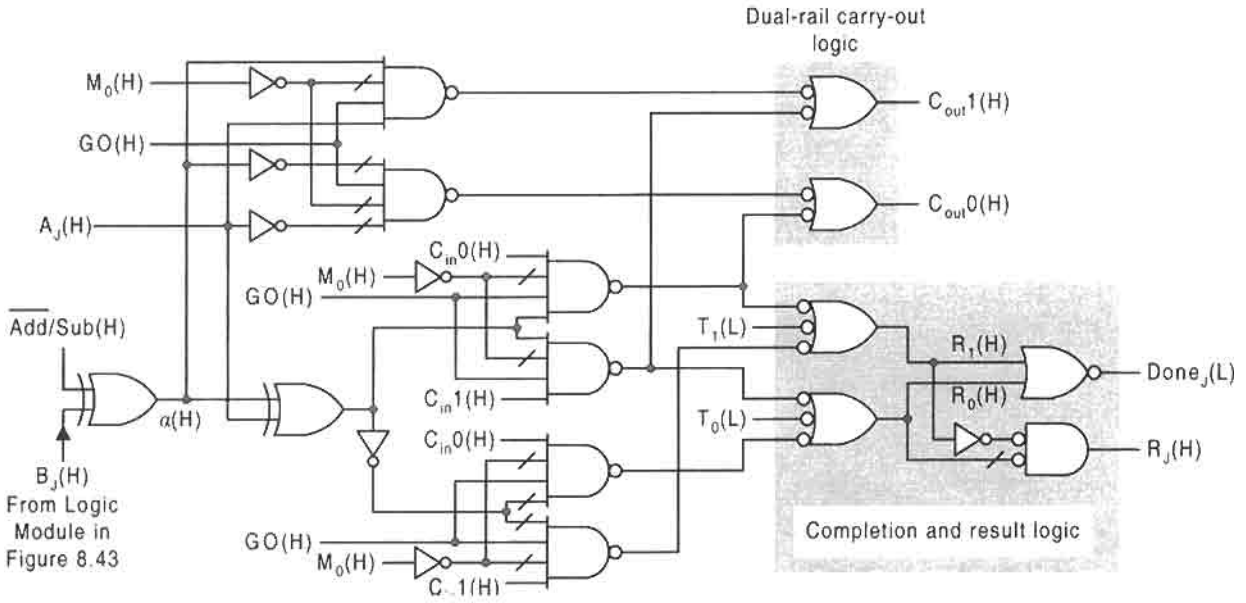
With these definitions in mind and by referring to Subsection 8.2.2 for the full adder, the carry partitions in the truth table of Fig. 8.43 can be understood. The dual-rail sum and carry-outs in Fig. 8.43a are plotted in the EV K-maps of Fig. 8.43b. From these K-maps there results the dual-rail carry-out expressions for the  $J$ th stage,

$$\left\{ \begin{array}{l} C_{out}1 = (A \oplus B)C_{in}1 + AB \\ C_{out}0 = (A \oplus B)C_{in}0 + \bar{A}\bar{B} \end{array} \right\} \quad (8.25)$$

Now, it is necessary to introduce the mode control  $M_0$  (consistent with Fig. 8.39) and start input  $GO$  together with an add/subtract parameter  $\alpha = B \oplus (\text{Add/Sub})$  that replaces operand  $B$  in Eqs. (8.24) and (8.25). When this is done the dual-rail sum and carry-out parameters for  $J$ th the module become

$$\left\{ \begin{array}{l} S_1 = GO \cdot \bar{M}_0[(A \oplus \alpha)C_{in}0 + (A \odot \alpha)C_{in}1] \\ S_0 = GO \cdot \bar{M}_0[(A \oplus \alpha)C_{in}1 + (A \odot \alpha)C_{in}0] \\ C_{out}1 = GO \cdot \bar{M}_0[(A \oplus \alpha)C_{in}1 + A\alpha] \\ C_{out}0 = GO \cdot \bar{M}_0[(A \oplus \alpha)C_{in}0 + \bar{A}\bar{\alpha}] \end{array} \right\} \quad (8.26)$$

Use of the mode control  $M_0$  avoids issuing a false sum from the arithmetic module and acts as a false data rejection feature during nonarithmetic operations. The XOR function  $\alpha = B \oplus (\text{Add/Sub})$  that replaces operand  $B$  permits the subtraction operation by using 2's complement arithmetic as is done for the adder/subtractor in Subsection 8.3.1. For addition,  $\text{Add/Sub} = 0$  passes operand  $B$ , but for subtraction  $\text{Add/Sub} = 1$  complements operand  $B$ , as required in 2's complement arithmetic.

**FIGURE 8.44**

Logic circuit for the  $J$ th arithmetic module represented by Eqs. (8.26) and (8.27), showing the completion ( $Done_J$ ) and result ( $R_J$ ) output logic that includes the dual-rail inputs  $T_1$  and  $T_2$  from the logic module and the dual-rail carry-out logic.

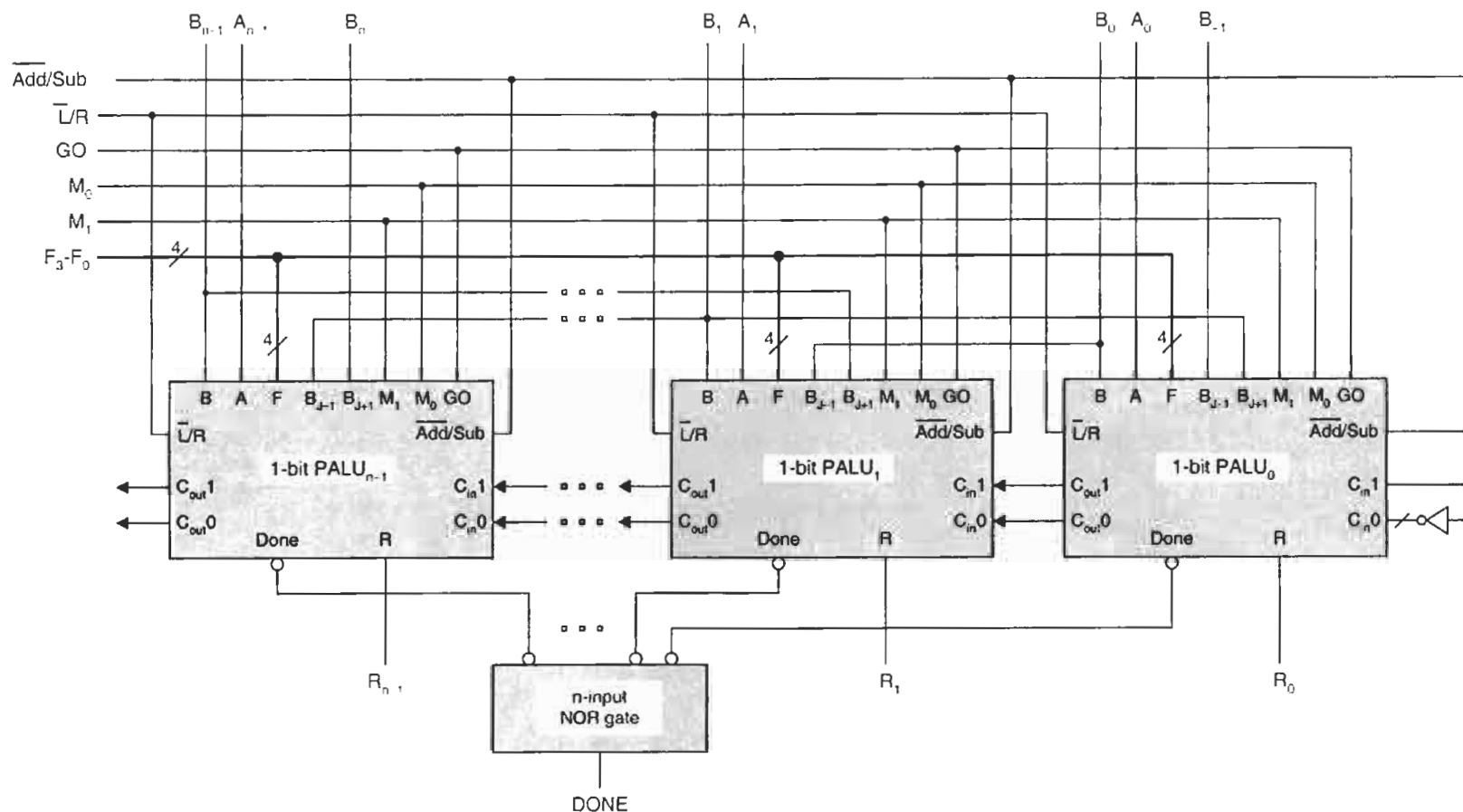
Shown in Fig. 8.44 is the logic circuit for the  $J$ th arithmetic module as represented by Eqs. (8.26). In addition, there is included the completion ( $Done_J$ ) and result ( $R_J$ ) logic section that derives from equations that combine the results from the logic module with those from the arithmetic module. For the  $J$ th PALU module, these equations are given by

$$\left\{ \begin{array}{l} R_1 = T_1 + S_1 \\ R_0 = T_0 + S_0 \\ \text{and} \\ R = R_1 \cdot \bar{R}_0 \\ Done = R_1 + R_0 \end{array} \right\}. \quad (8.27)$$

Notice that the result  $R = R_1 \bar{R}_0$  is a resolved result required to issue a logic 1 when  $R_1 = 1$  or a logic 0 when  $R_0 = 1$  according to the definitions of the dual-rail components of  $R$ . However, a completion signal  $Done = 1$  is issued in either case to indicate that a valid result is present. Thus, except for the initialization state  $GO = 0$ , for which  $R_1 = R_0 = 0$ , one or the other of  $R_1$  or  $R_0$  will be active, indicating a valid result, but they will never both be active simultaneously.

The cascading of 1-bit modules to form an  $n$ -bit PALU is easily accomplished as illustrated in Fig. 8.45. All that is required is to connect each set of dual-rail carry-outs from one stage to the next MSB stage and all global inputs to all stages. The shift inputs,  $B_{j-1}$  and





**FIGURE 8.45**

Logic circuit for an  $n$ -bit PALU with completion signal capability consisting of cascaded 1-bit modules of the type shown in Fig. 8.40. Note that all inputs and outputs are assumed to be active high and that the  $n$ -input NOR gate is of CMOS construction as illustrated in Fig. 8.46.

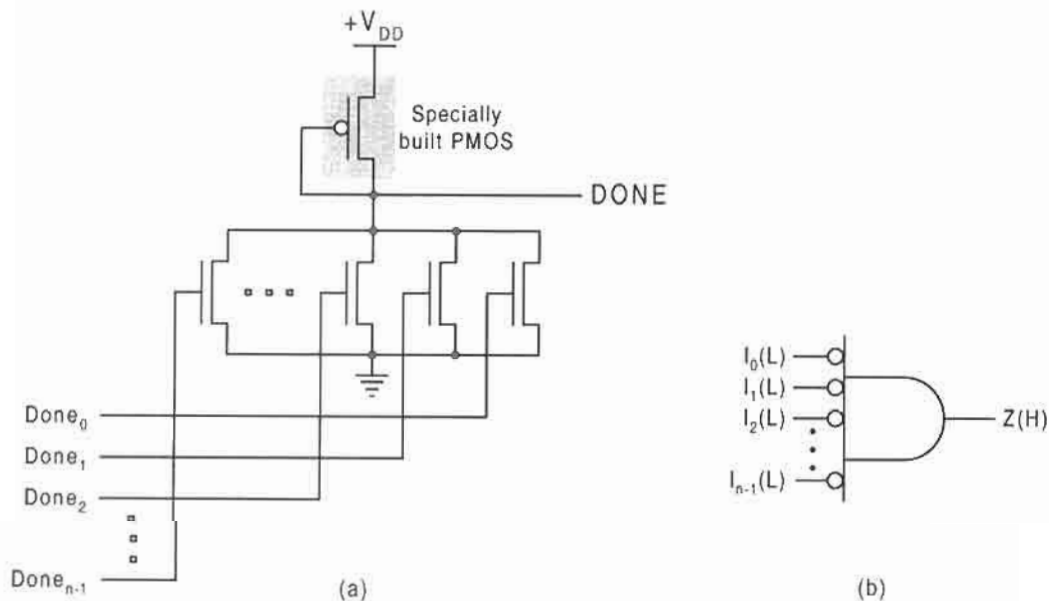
$B_{j+1}$ , must be connected appropriately for left and right shifting as shown. Barrel shifting (rotation) right can be accomplished by connecting the  $R_0$  output to the  $B_n$  input. Similarly, barrel shifting left results if the  $R_{n-1}$  output is connected to the  $B_{-1}$  input. Notice that the carry-in inputs,  $C_{in1}$  and  $C_{in0}$ , to the LSB (PALU<sub>0</sub>) stage are correctly initialized for add or 2's complement arithmetic (see Subsection 8.3.1 regarding adder/subtractors). Thus, if  $\text{Add/Sub} = 1$ , required for subtraction by 2's complement, a logic 1 is carried in to the LSB stage ( $C_{in1} = 1$ ). Conversely, if  $\text{Add/Sub} = 0$  for addition, a logic 0 is carried in ( $C_{in0} = 1$ ).

The  $n$ -input NOR gate in Fig. 8.45 requires special consideration. This gate must AND the individual  $\text{Done(L)}$  signals to produce the final  $\text{DONE(H)}$  represented by the expression

$$\text{DONE} = \prod_{i=0}^{n-1} (\text{Done})_i. \quad (8.28)$$

Thus, the conjugate gate form shown in Fig. 3.13b must be used. With inputs to such a gate numbering more than four, there is the problem of fan-in as discussed in Section 4.10. The larger the fan-in, the greater is the path delay through the gate. In fact, there is a definite limit as to the number of inputs available in commercial NOR gate chips.

The fan-in problem is effectively eliminated by using the CMOS NOR gate construction shown in Fig. 8.46a. Here, the number of permissible inputs up to about eight will have negligible effect on the path delay through the gate, which is essentially that of a two-input NOR gate. All  $\text{Done}$  inputs must go to LV before the output  $\text{DONE}$  can go to HV. So if one



**FIGURE 8.46**

Multiple input NOR gate specifically designed to minimize fan-in-limitations. (a) CMOS circuit required for Fig. 8.45. (b) Generalized NOR gate symbol for this circuit.

or more of the inputs are at HV, the output is pulled low (to LV). Thus, the PMOS must be specially designed so that the Drain-to-Source resistance remains sufficiently high so as to minimize Drain current when one or more of the NMOS are turned ON. Here, the PMOS serves the same function as the depletion-mode NMOS in Fig. A.1 of Appendix A. Both function as pull-up transistors. Hence, the CMOS NOR gate of Fig. 8.46 could be replaced by the NMOS technology of Figure A.1. In any case, the NOR gate logic symbol for this circuit is given in Fig. 8.46b, which is the same as that in Fig. 3.13b.

### 8.9.1 Carry Look-Ahead Configuration

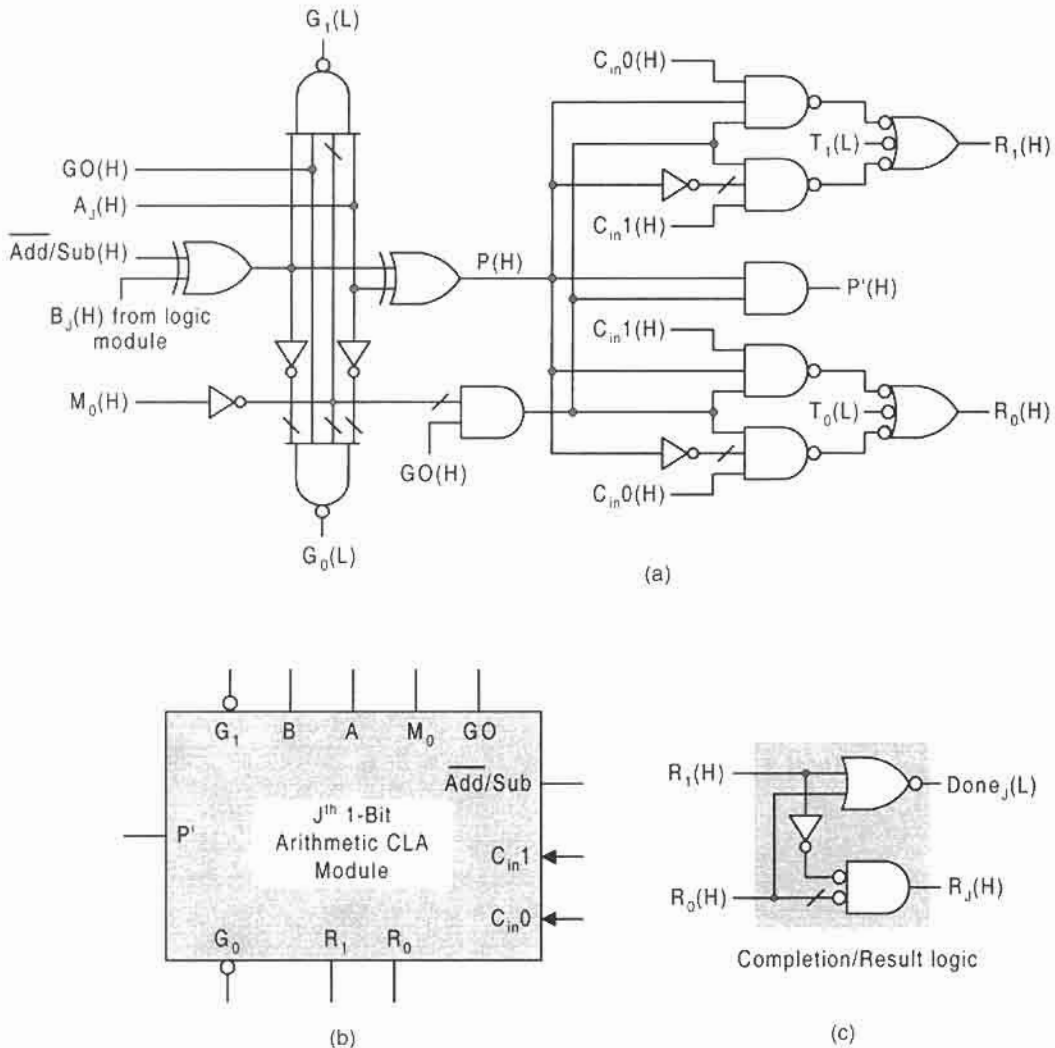
Like the R-C adder of Fig. 8.5, the PALU just described suffers a size limitation due to the ripple-carry effect. The carry propagation time increases with increase in the size of the PALU. To reduce significantly the propagation time, carry look-ahead (CLA) capabilities can be incorporated into a dual-rail PALU design. From Eqs. (8.7) the following definitions can be applied to the  $J$ th 1-bit PALU module:

$$\left\{ \begin{array}{l} P = A \oplus \alpha = \text{Carry propagate} \\ P' = GOM_0 \cdot P = \text{Modified carry propagate} \\ G_1 = GOM_0 \cdot A\alpha = \text{Carry generate w/r to logic 1} \\ G_0 = GOM_0 \cdot \bar{A}\bar{\alpha} = \text{Carry generate w/r to logic 0} \end{array} \right\}. \quad (8.29)$$

Here,  $G_1$  and  $G_0$  are the dual-rail carry generate parameters, and  $\alpha = B \oplus (\overline{Add/Sub})$  is the add/subtract parameter that replaces operand  $B$  in Eqs. (8.7). Introducing Eqs. (8.29) into the sum and carry-out expressions of Eqs. (8.26) yields

$$\left\{ \begin{array}{l} S_1 = (GOM_0)PC_{in0} + (GOM_0)\bar{P}C_{in1} \\ S_0 = (GOM_0)PC_{in1} + (GOM_0)\bar{P}C_{in0} \\ C_{out1} = GOM_0PC_{in1} + GOM_0A\alpha \\ \quad = P'C_{in1} + G_1 \\ C_{out0} = GOM_0PC_{in0} + GOM_0\bar{A}\bar{\alpha} \\ \quad = P'C_{in0} + G_0 \end{array} \right\}, \quad (8.30)$$

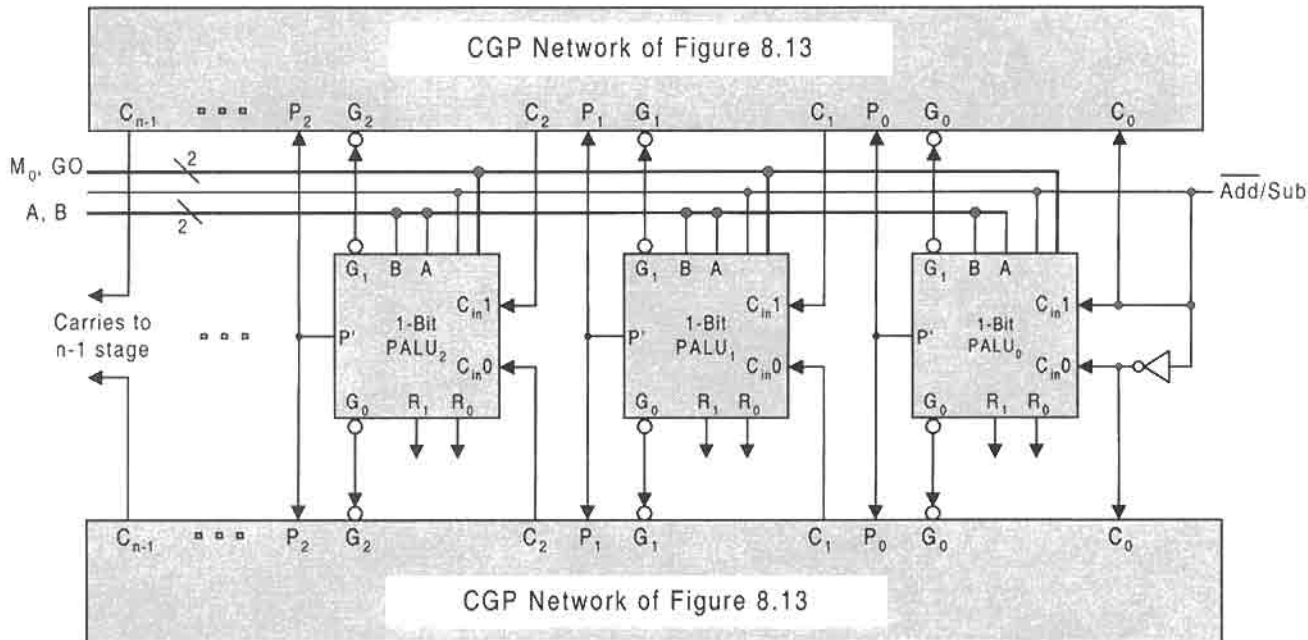
which are applied to the  $J$ th 1-bit PALU module with CLA capability. As in Eqs. (8.26), the appearance of the mode control  $\bar{M}_0$  in Eqs. (8.29) and (8.30) avoids issuing a false sum from the arithmetic module and acts as a false carry rejection feature during nonarithmetic operations. The carry-out expressions  $C_{out1} = P'C_{in1} + G_1$  and  $C_{out0} = P'C_{in0} + G_0$  can be expanded as in Eqs. (8.8) and, therefore, constitute the CGP network similar to that in Fig. 8.13 with  $P'$  replacing  $P$  in that network. Thus, all that remains in the design of the dual-rail PALU with CLA capability is to implement Eqs. (8.29) together with the sum expressions in Eqs. (8.30). Presented in Fig. 8.47a is the logic circuit for the arithmetic module of a 1-bit PALU with completion signal and CLA capability as required by Eqs. (8.27), (8.29), and (8.30). Remember that it is the modified carry propagate parameter  $P'$ , not  $P$ , that combines with the dual-rail carry generate parameters,  $G_1$  and  $G_0$ , to form the CGP network as in Fig. 8.13. The logic circuit symbol for this arithmetic module is given in Fig. 8.47b.

**FIGURE 8.47**

The  $J$ th 1-bit arithmetic module for a PALU with completion signal and CLA capability according to Eqs. (8.29) and (8.30). (a) Logic circuit showing carry propagate, and dual-rail carry generate, carry inputs and result signals. (b) Block circuit symbol for the logic circuit in (a). (c) Completion/result logic circuit for combined logic and arithmetic modules.

The PALU can be completed by combining the logic module in Fig. 8.42 with the arithmetic module of Fig. 8.47. This requires that the completion signals from the logic module,  $T_1$  and  $T_0$ , be combined with the completion/result signals from the arithmetic module,  $S_1$  and  $S_0$ , to yield the *Done* and *R* signals as indicated in Fig. 8.47c. Further modifications and a significant increase in hardware are required to make the PALU symmetric with respect to both operands. In this case either an *A* or *B* shift would be possible with arithmetic operations performed on either.

The 1-bit PALU must now be cascaded in a manner similar to the CLA adder in Fig. 8.13, except that now one CGP network is needed for logic 1 ( $G_1$ ,  $P'$ ) and another is needed for

**FIGURE 8.48**

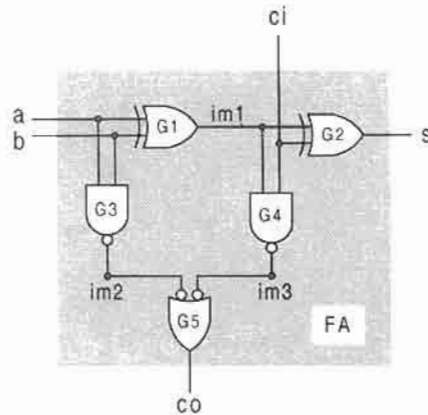
The three least significant bit stages of an  $n$ -bit arithmetic module for a PALU with CLA and completion signal capability showing the carry generate/propagate (CGP) network required for the dual-rail carries and carry generate parameters.

logic 0 ( $G_0, P'$ ), as required for dual-rail carries. This is demonstrated in Fig. 8.48, where for simplicity only the arithmetic module is featured. Notice that the carry-in's for the LSB stage are properly initialized for addition or 2's complement arithmetic. Hence, for subtraction,  $Add/Sub = 1$  introduces a logic 1 into  $C_{in1}$  and a logic 0 into  $C_{in0}$ , as required by Eq. (2.14). But for addition,  $Add/Sub = 0$  introduces a logic 0 into carry-in  $C_{in1}$  and a logic 1 into  $C_{in0}$  (meaning 0 carry-in). To complete the PALU design, it is necessary to include the logic module in Fig. 8.42 with the arithmetic module in Fig. 8.48 and combine the completion and result signals as indicated in Fig. 8.47c. It is also necessary to connect the  $X_j$  output from the logic module to the  $B_j$  input of the arithmetic module for each stage.

Clearly, the hardware commitment for the dual-rail CLA PALU increases considerably as the number of stages increases. For this reason it is recommended that the group CLA method be used on, say, 4-bit stages. For example, each of the 1-bit PALUs in Fig. 8.48 would be replaced by four 1-bit stages of a "conventional" type (i.e., without CLA capability) and then cascaded with the dual-rail CPG networks as in Fig. 8.48. This requires that only the MSB stage of each group of four need be altered, as in Fig. 8.47, to accommodate the CLA feature.

## 8.10 VHDL DESCRIPTION OF ARITHMETIC DEVICES

To recap what was stated or intimated in Section 6.10, the VHDL model of a circuit is called its *entity*, which consists of an *interface description* and an *architectural description*.

**FIGURE 8.49**

Logic circuit for the full adder showing inputs, outputs, and intermediate connection labels required for the VHDL description.

The interface of an entity specifies its circuit ports (inputs and outputs) and the architecture gives its contents (e.g., its interconnections). The architecture of an entity is divided into different levels of abstraction, which include its *structure*, *dataflow*, and *behavior*, the behavior description being the most abstract. The following VHDL description of the full adder, shown in Fig. 8.49, illustrates these three levels of abstraction (refer to Section 6.10 for a description of the key words used):

```
-- Declare entity:
```

```
entity full_adder_example is
```

```
    port (a, b, ci: in bit; s, co: out bit);
```

```
end full_adder_example;
```

```
-- Declare architecture beginning with structure:
```

```
architecture structure of full_adder_example is
```

```
    component XOR2
```

```
        port (x, y: in bit; z: out bit); -- declares XOR gate
```

```
    component NAND2
```

```
        port (x, y: in bit; z: out bit); -- declares NAND gate
```

```
end component;
```

```
signal im1, im2, im3: bit; -- declares intermediate signals
```

```
-- Declare interconnections between component gates:
```

```
begin
```

```
    G1: xor2 port map (a, b, im1);
```

```
    G2: xor2 port map (im1, ci, s);
```

```

    G3: nand2 port map (a, b, im2);
    G4: nand2 port map (im1, ci, im3);
    G5: nand2 port map (im2, im3, co);
end full_adder_example;

-- Declare dataflow:
architecture dataflow of full_adder_example is
begin
    s <= a xor b xor ci after 12 ns;
    co <= (a and b) after 10 ns or (im1 and ci) after 16 ns;
                                                    -- 16 ns is worst case delay
end dataflow;

-- Declare behavior:
architecture behavior of full_adder_example is
begin
    process (a, b, ci)
        variable a, b, ci, s: integer;
        begin
            if a = '0' then a := 0; else a := 1;      -- converts a to integer
        end if
            if b = '0' then b := 0; else b := 1;      -- converts b to integer
        end if
            if ci = '0' then ci := 0; else ci := 1;    -- converts ci to integer
            s := a + b + ci;                            -- computes sum of inputs
        case s is
            when 0 => s <= '0'; co <= '0';
            when 1 => s <= '1'; co <= '0';
            when 2 => s <= '0'; co <= '1';
            when 3 => s <= '1'; co <= '1';
        end case;
    end process;
end full_adder_example;

```

In the full adder example just given, notice that all keywords are presented in bold type and that the symbol “=>” is read as “is the value of.” Also note that the operators that are used are those listed in Fig. 6.44 of Section 6.10 and that the double dash “--” is used to indicate the beginning of a comment. The delay times given for *s* and *co* are based on average gate path delays of 6 ns for the XOR gate and 5 ns for the two-input NAND gate, as expressed in Eq. (6.1).

An important feature of VHDL is its modularity capability, which allows models to be reused in the description of other larger entities. A good example is the VHDL structural

description of a four-bit adder composed of four full adders described in the previous example. If Fig. 8.49 is used with reference to Figs. 8.4 and 8.5, the structure of the four-bit adder is given as follows:

```
entity four_bit_adder is
    port (a0, a1, a2, a3, b0, b1, b2, b3, ci: in bit; s0, s1, s2, s3, co: out bit;
end four_bit_adder;

architecture connect_four of four_bit_adder is
    component full_adder
        port (a, b, ci: in bit; s, co: out bit);
    end component;
for all: full_adder use entity full_adder_example;
signal c1, c2, c3: bit
begin
    FA0: full_adder port map (a0, b0, ci, s0, c1);
    FA1: full_adder port map (a1, b1, c1, s1, c2);
    FA2: full_adder port map (a2, b2, c2, s2, c3);
    FA3: full_adder port map (a3, b3, c3, s3, co);
end connect_four;
end four_bit_adder;
```

Just given is an architectural description for the full-adder primitive, followed by that for a four-bit adder formed by cascading four full-adder primitives. However, within VHDL compilers, encapsulations of such primitives are provided so that they can be easily retrieved and used in the architectural descriptions of larger systems. Thus, for well-known primitives like those just considered, there is no need to construct the detailed architectural descriptions — this has already been accomplished for the convenience of the user. These primitives exist within standard logic packages. The IEEE 1164 standard library is an example, and its contents are made available by making the statements

```
library ieee
use std_logic_1164.all
```

Once a standard library is made available in the design description, use can be made of data types, functions, and operators provided by that standard. Standard data types include bit, bit\_vector, integer, time, and others, and the operators are of the type given in Fig. 6.44. The standard package defined by the IEEE 1076 standard includes declarations for all the standard data types. For detailed information on these and other subjects related to standard libraries and packages the reader is referred to Further Reading.

## FURTHER READING

Most recent texts give a fair account of the basic arithmetic devices, including the full adder, parallel adders, subtractors, adder/subtractors, and carry look-ahead adders. Typical



examples are the texts of Comer, Ercegovac and Lang, Hayes, Katz, Pollard, Sandige, Tinder, Wakerly, and Yarbrough. The subject of multiple operand addition and the carry-save adder appears to be covered adequately only in texts by Ercegovac and Lang and by Tinder. Note that some of the listed devices may or may not be the strength of a text.

- [1] D. J. Comer, *Digital Logic and State Machine Design*, 3rd. ed., Sanders College Publishing, Fort Worth, TX, 1995.
- [2] M. D. Ercegovac and T. Lang, *Digital Systems and Hardware/Firmware Algorithms*. John Wiley & Sons, New York, 1985.
- [3] J. P. Hayes, *Introduction to Digital Logic Design*. Addison Wesley, Reading, MA, 1993.
- [4] R. H. Katz, *Contemporary Logic Design*. Benjamin/Commings Publishing, Redwood City, CA, 1994.
- [5] L. H. Pollard, *Computer Design and Architecture*. Prentice Hall, Englewood Cliffs, NJ, 1990.
- [6] R. S. Sandige, *Modern Digital Design*. McGraw-Hill, New York, 1990.
- [7] R. F. Tinder, *Digital Engineering Design: A Modern Approach*. Prentice-Hall, Englewood Cliffs, NJ, 1991.
- [8] J. F. Wakerly, *Digital Design Principles and Practices*, 2nd ed., Prentice-Hall, Englewood Cliffs, NJ, 1994.
- [9] J. M. Yarbrough, *Digital Logic Applications and Design*. West Publishing Co., Minneapolis/St. Paul, 1997.

A few books adequately cover combinational multipliers. These include the texts by Ercegovac and Lang, Katz, Pollard, and Tinder, all previously cited. Of these, only the text by Tinder appears to cover combinational dividers.

A somewhat older text by Kostopoulos covers a rather broad range of subjects relative to arithmetic methods and circuits, including a good treatment of combinational multipliers and dividers. A recent text by Parhami provides an exhaustive treatment of the subject and is highly recommended.

- [10] G. K. Kostopoulos, *Digital Engineering*. John Wiley & Sons, New York, 1975.
- [11] B. Parhami, *Computer Arithmetic: Algorithms and Hardware Designs*. Oxford University Press, New York, 2000.

The subject of arithmetic and logic units (ALUs) is somewhat esoteric. Nevertheless, it is covered to one extent or another by a few well-known texts. These include those by Ercegovac and Lang, Hayes, Katz, Tinder, and Yarbrough, all previously cited. In addition, the text of Mead and Conway discusses an ALU suitable for processor application that is the starting point for the ALU treated in Subsection 8.8.2 of this text. Apparently, only the text by Tinder develops the dedicated multilevel ALU by using XOR/SOP logic. Coverage of dual-rail arithmetic systems, particularly ALUs, is difficult if not impossible to find in any text. The source on which this text is based is the thesis of Amar cited next. Here, dual-rail ALUs, multipliers, and dividers, all with completion signals, are discussed in detail.

- [12] A. Amar, "ALUs, Multipliers and Dividers with Completion Signals," M.S. Thesis. School of Electrical Engineering and Computer Science, Washington State University, Pullman, WA, 1994.
- [13] C. Mead and L. Conway, *Introduction to VLSI Systems*. Addison-Wesley, Reading, MA, 1980.

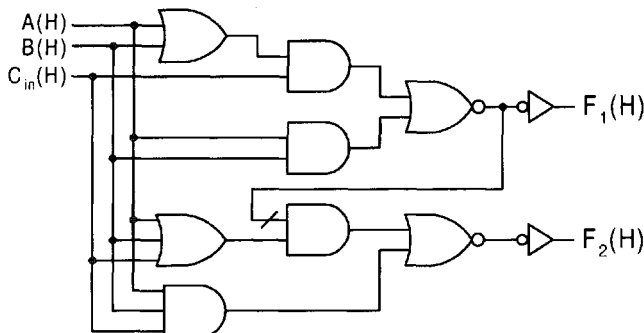


FIGURE P8.1

## PROBLEMS

- 8.1** Use one half adder, one NOR gate, and one inverter (nothing else) to implement a circuit that will indicate the product of a positive 2-bit binary number and 3. Assume all inputs and outputs are active high. (Hint: Construct a truth table with  $AB$  as the inputs and  $Y_3, Y_2, Y_1, Y_0$  as the outputs.)
- 8.2** Use a full adder (nothing else) to implement a circuit that will indicate the binary equivalent of  $(x^2 + x + 1)$ , where  $x = AB$  is a 2-bit binary word (number). Assume that the inputs and outputs are all active high. (Hint: Construct a truth table with  $AB$  as the inputs and  $Y_3, Y_2, Y_1, Y_0$  as the outputs.)
- 8.3** Design a three-input logic circuit that will cause an output  $F$  to go active under the following conditions:

All inputs are logic 1

An odd number of inputs are logic 1

None of the inputs are logic 1

To do this use *one* full adder and *two* NOR gates (nothing else). Assume that the inputs arrive active high and that the output is active low.

- 8.4** Prove that the logic circuit in Fig. P8.1 is that of a full adder. Also, prove that  $C_{out}$  is the majority function  $AB + BC + AC$ . (Hint: Make use of K-maps or truth tables to solve this problem.)
- 8.5** Use the logic circuit in Fig. P8.1 (exactly as it is given) to construct a staged CMOS implementation of the full adder. Refer to Fig. 3.41 for assistance if needed.
- 8.6** Use the symbol for a 3-to-8 decoder and two NAND gates (nothing else) to implement the full adder. Assume that the inputs arrive as  $A(H)$ ,  $B(H)$ , and  $C_{in}(L)$  and that the outputs are issued active high. (Hint: First construct the truth table for the full adder, taking into account the activation levels of the inputs and outputs.)
- 8.7** Without working too hard, design a 4-bit binary-to-2's complement converter by using half adders and inverters (nothing else). Use block symbols for the half adders and assume that the inputs and outputs are all active high.

- 8.8** Without working too hard, use *four* full adders (nothing else) to design a circuit that will convert XS3 to BCD. Assume that all inputs and outputs are active high. (Hint: Use 2's complement arithmetic.)
- 8.9** Analyze the adder/subtractor in Fig. 8.9 in 4 bits by adding or subtracting the binary equivalent of the numbers listed below. To do this, give the sum (or difference) and carry logic values at each stage.
- (a)  $A = 1101$ ;  $B = 0111$  if  $\bar{A}/S(H) = 0(H)$   
 (b)  $A = 1101$ ;  $B = 0111$  if  $\bar{A}/S(H) = 1(H)$   
 (c)  $A = 0110$ ;  $B = 1101$  if  $\bar{A}/S(H) = 1(H)$
- 8.10** Analyze the 3-bit carry look-ahead adder (CLA) in Fig. 8.13 by introducing the number given below. To do this, give the  $G(L)$  and  $P(H)$  logic values in addition to the sum and carry values.
- (a)  $A = 011$ ;  $B = 110$   
 (b)  $A = 111$ ;  $B = 101$
- 8.11** (a) By using Algorithm 2.14 in Subsection 2.9.6 and two 4-bit ripple/carry adders, complete the design of the single-digit BCD adder in Fig. P8.2a, one that can be bit-sliced (cascaded) to produce an  $n$ -digit BCD adder. To do this, first find the minimum logic for the correction parameter  $X$  that will indicate when the sum is

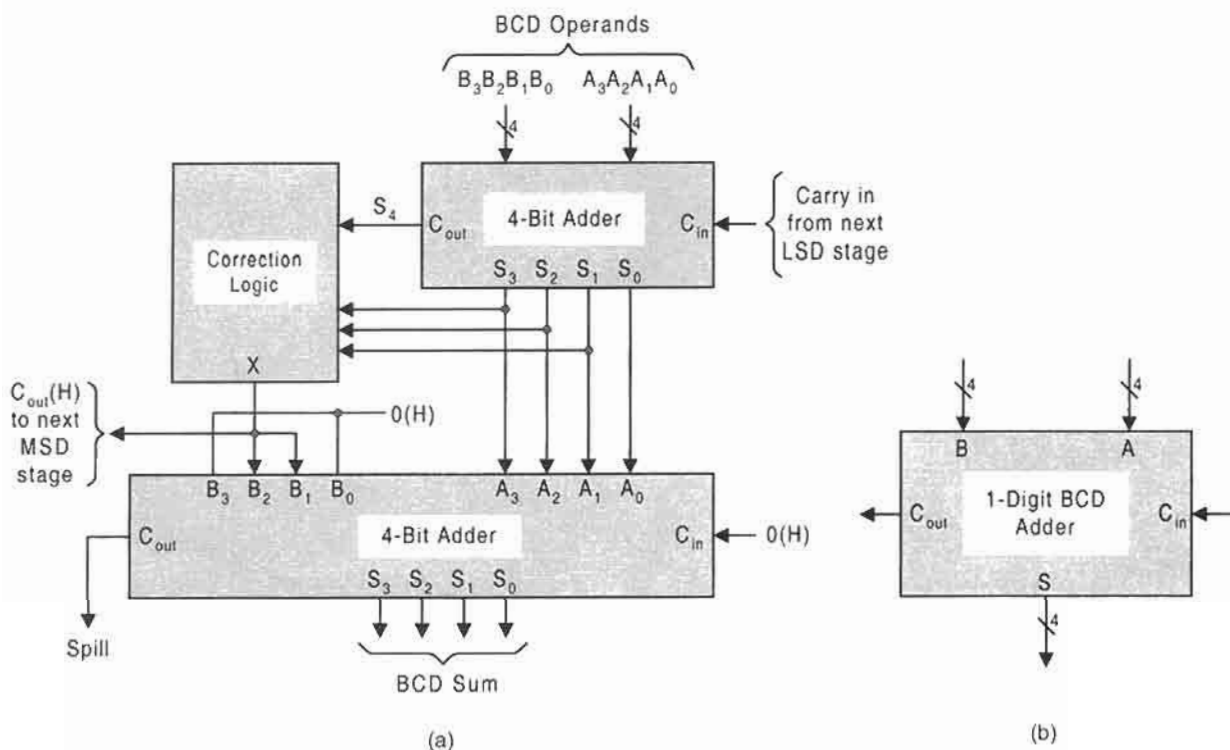


FIGURE P8.2

greater than 9 but less than 19. Then use one of two 4-bit adders (as shown) to add 0110 to the sum conditionally on  $X$ . Note the logic symbol for the 1-digit BCD adder in Fig. P8.2b. What restrictions, if any, are placed on the operands?

- (b) Test the design of a two-digit BCD adder by adding the following two BCD numbers:

Test #1  $A = 9_{10}; B = 7_{10}$

Test #2  $A = 34_{10}; B = 19_{10}$

To do this, indicate the logic values for each sum, carry, and correction parameter  $X$ .

- 8.12** (a) Alter the design of Problem 8.11 with the appropriate external logic so as to create a one-digit BCD adder/subtractor that can be bit-sliced to form an  $n$ -digit BCD adder/subtractor, all with  $B$  as the subtrahend. (Hint: Add an enable  $EN(H) = \bar{A}/S(H)$  mode control to the correction logic of Problem 8.11 and use 10's complement arithmetic to perform BCD subtraction operations. To do this, follow Fig. 8.9 with the appropriate external logic on the input for cascading purposes. Also, remember that a 1 carry cannot be generated in BCD subtraction. It is important to note that, in this case, the negative number is not converted to 10's complement as in Algorithm 2.15, which is an alternative means of BCD subtraction.)
- (b) Test the design of a two-digit BCD adder/subtractor by introducing the following numbers and mode control:

Test #1  $A = 6_{10}, B = 29_{10}, \text{ and } \bar{A}/S(H) = 0(H)$

Test #2  $A = 6_{10}, B = 29_{10}, \text{ and } \bar{A}/S(H) = 1(H)$

To do this, indicate the logic values for each operand, sum, carry, mode control, and correction parameter. Note that if the result of a subtraction is a negative number, it will be represented in 10's complement, and that its true value can be found by negation of each digit separately.

- 8.13** Use two 4-bit binary adders and the necessary correction and external logic to design a one-digit XS3 adder (similar to BCD adder in Fig. P8.2) that can be cascaded to form an  $n$ -digit XS3 adder. To do this, use the following algorithm:

Add XS3 numbers by using binary addition: If there is no 1 carry from the 4-bit sum, correct that sum by subtracting 0011 ( $3_{10}$ ). If a 1 carry is generated from the 4-bit sum, correct that sum by adding 0011. Remember that XS3 numbers less than 3 or greater than 12 are not used and that the sum of two XS3 numbers cannot exceed  $24_{10}$ .

[Hint: First, find the minimum logic for the correction parameter  $X$  that will indicate when the sum is greater than 12 but less than 25, the range over which a 1 carry is generated. Also, controlled inverters (XOR gates) must be used for the addition or subtraction of  $3_{10}$ .] What restrictions, if any, are placed on the operands?

- 8.14** (a) Alter the design of Problem 8.13 so as to create a one-digit XS3 adder/subtractor that can be cascaded to form an  $n$ -digit XS3 adder/subtractor, all with  $B$  as the

subtrahend. Note that a 1 carry cannot be generated in XS3 subtraction. (Hint: An additional four controlled inverters are needed for the add/subtract operations.)

- (b) Test the design of a two-digit XS3 adder/subtractor by introducing the following numbers and mode control:

Test #1  $A = 6_{10}$ ,  $B = 29_{10}$ , and  $\bar{A}/S(H) = 0(H)$

Test #2  $A = 6_{10}$ ,  $B = 29_{10}$ , and  $\bar{A}/S(H) = 1(H)$

To do this indicate the logic values for each operand, sum, carry, mode control, and correction parameter. Note that the decimal value of a negative XS3 number is found by subtracting  $\dots 0011$  from the negated number and reading it as a BCD number.

- 8.15** In Fig. P8.3a is shown a network containing several combinational logic devices including a 4-bit ripple/carry adder.

- (a) Complete the truth table in Fig. P8.3b.  
 (b) Use a decoder and a single OR gate to accomplish the result given in part (a). Assume that the decoder has active high outputs.

- 8.16** (a) Design a 4-bit noncascadable comparator by using two 4-bit subtractors and one NOR gate (nothing else). [Hint: It will be necessary to switch operands on one of the two subtractors. Also, in a subtractor, a final borrow-out of 1 indicates (minuend) < (subtrahend), but a final borrow-out of 0 indicates (minuend)  $\geq$  (subtrahend). Thus, if both borrow-outs are logic 0, then the two numbers are equal. Note that a negative difference is given in 2's complement but taking into account  $B_{in})_{LSB} = 0$ .]

- (b) Test the design in part (a) by using the following operands:

Test #1  $A = 1101$ ;  $B = 0110$

Test #2  $A = 0110$ ;  $B = 1101$

Test #3  $A = 1010$ ;  $B = 1010$

- (c) Show that the difference of the two operands can also be read from the circuit.

- 8.17** (a) By using Eqs. (8.8), complete the carry look-ahead adder (CLA) circuit in Fig. 8.13 for a cascable 4-bit CLA adder unit. Thus, include the carry generate/propagate logic from the fourth stage.

- (b) Test the results by adding the following numbers:

Test #1  $A = 0111$ ;  $B = 0110$

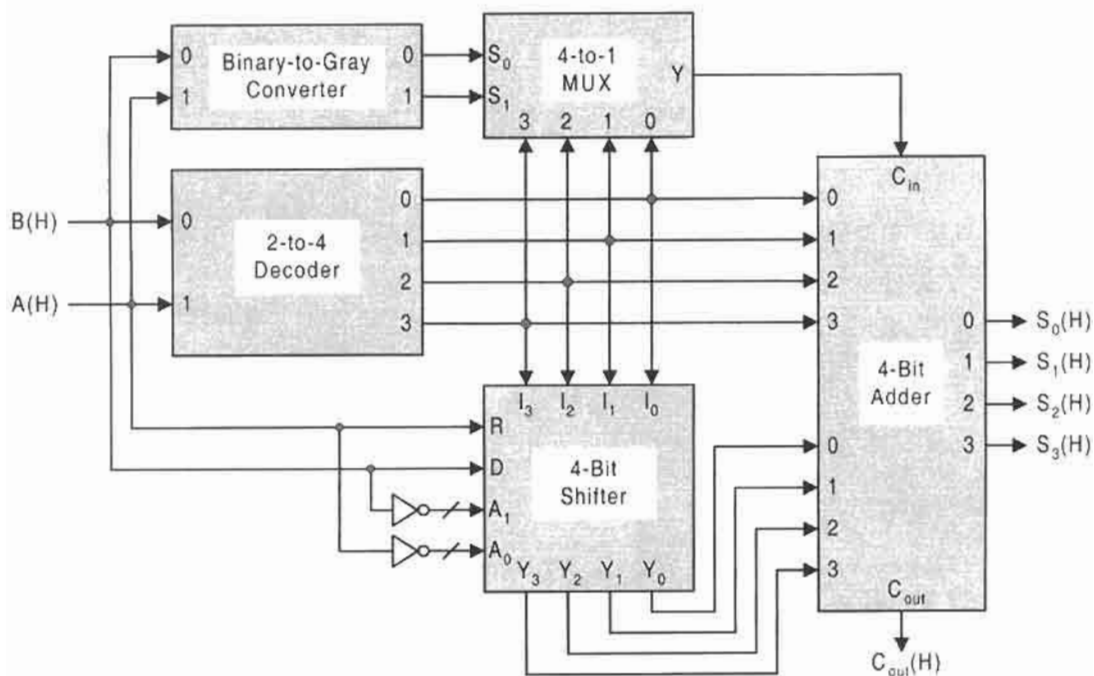
Test #2  $A = 1101$ ;  $B = 1010$

- 8.18** Analyze the carry-save circuit of Fig. 8.15b by introducing the three operands given in Fig. 8.15a into the circuit. To do this, give the logic values for each operand, sum, and carry.

- 8.19** Analyze the  $4 \times 4$  binary multiplier in Fig. 8.18 by introducing the following operands into the circuit:

- (a)  $A = 1101$ ;  $B = 0110$

- (b)  $A = 1001$ ;  $B = 1011$



(a)

| A | B | $C_{out}$ | $S_3$ | $S_2$ | $S_1$ | $S_0$ |
|---|---|-----------|-------|-------|-------|-------|
| 0 | 0 |           |       |       |       |       |
| 0 | 1 |           |       |       |       |       |
| 1 | 0 |           |       |       |       |       |
| 1 | 1 |           |       |       |       |       |

(b)

To do this, list all partial products, indicate the sum and carry values at each stage, and end with the final product values.

- 8.20** (a) Combine the  $4 \times 4$  binary multiplier in Fig. 8.18 with the 8-bit binary-to-BCD converter in Fig. P6.3 to produce a one-digit BCD multiplier that can be bit-sliced to form an  $n$ -digit BCD multiplier. What restrictions must be placed on the operands?

- (b) Test the results of part (a) by introducing the following operands into the resulting circuit of part (a):

Test #1  $A = 0111$ ;  $B = 1000$

Test #2  $A = 1001$ ;  $B = 0011$

|                       | M | S <sub>1</sub> | S <sub>0</sub> | F | C <sub>out</sub>       | Operation*                   |               |
|-----------------------|---|----------------|----------------|---|------------------------|------------------------------|---------------|
| Arithmetic Operations | { | 0              | 0              | 0 | A⊕B⊕C <sub>in</sub>    | C <sub>in</sub> (A⊕B) + A·B  | A plus B      |
|                       |   | 0              | 0              | 1 | A⊕B̄⊕C <sub>in</sub>   | C <sub>in</sub> (A⊙B) + A·B̄ | A minus B*    |
|                       |   | 0              | 1              | 0 | (A+B)⊕C <sub>in</sub>  | C <sub>in</sub> ·(A+B)       | A plus ĀB    |
|                       |   | 0              | 1              | 1 | (ĀB̄)⊕C <sub>in</sub> | C <sub>in</sub> ·B̄ + A      | A plus (A+B̄) |
| Logic Operations      | { | 1              | 0              | 0 | A⊕B                    | ϕ                            | A XOR B       |
|                       |   | 1              | 0              | 1 | A⊙B                    | ϕ                            | A EQV B       |
|                       |   | 1              | 1              | 0 | A + B                  | ϕ                            | A OR B        |
|                       |   | 1              | 1              | 1 | Ā · B̄                | ϕ                            | Ā AND B̄     |

\* Subtraction operations assume 2's complement arithmetic.

FIGURE P8.4

(c) Use a block symbol for the 1-digit BCD multiplier together with the block symbol for the binary-to-BCD converter of Fig. P6.3 to design a  $2 \times 2$  BCD multiplier. To do this, form an array of 1-digit multipliers and connect them properly to a 4-digit BCD adder. Indicate the digit orders of magnitude ( $10^0$ ,  $10^1$ ,  $10^2$ , and  $10^3$ ) at all stages of the multiplier.

**8.21** By using the results shown in Fig. 6.19, alter the design of the BCD multiplier of Problem 8.20 so as to produce a cascadable one-digit XS3 multiplier. (Hint: It will be necessary to design an XS3-to-BCD converter as an output device.)

**8.22** With reference to Fig. 8.22, analyze the parallel divider shown in Fig. 8.23. To do this, introduce the operands  $A = 11010$  and  $B = 110$  and indicate on the logic circuit the logic value for each operand, borrow, remainder, and quotient.

**8.23** Shown in Fig. P8.4 is the operation table for a cascadable one-bit arithmetic and logic unit (ALU) that has three mode/select inputs that control four arithmetic operations and four bitwise logic operations.

(a) Design this ALU by using a gate-minimum logic. Note that this design includes the use of compound XOR-type patterns similar to those used in Fig. 8.27. End with a logic circuit for both function  $F$  and  $C_{out}$ .

(b) Test the design of part (a) by introducing the following operands with  $(C_{in})_{LSB} = \text{Add/Sub. for arithmetic operations}$ :

$$\text{Tests \#1} \quad A = 10; \quad B = 11 \quad 2\text{-Bit ALU}; \quad MS_1S_0 = \begin{Bmatrix} 000 \\ 101 \end{Bmatrix}$$

$$\text{Tests \#2} \quad A = 0100; \quad B = 0111 \quad 4\text{-Bit ALU}; \quad MS_1S_0 = \begin{Bmatrix} 001 \\ 100 \end{Bmatrix}$$

# Propagation Delay and Timing Defects in Combinational Logic

---

## 9.1 INTRODUCTION

To this point in the text, combinational logic circuits have been treated as though they were composed of “ideal” gates in the sense of having no propagation delay. Now it is necessary to take a step into the real world and consider that each gate has associated with it a propagation time delay and that, as a result of this delay, undesirable effects may occur.

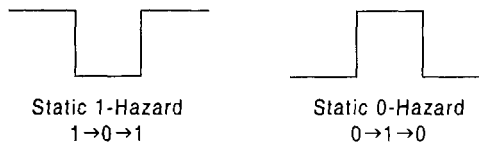
Under certain conditions unwanted transients can occur in otherwise steady-state signals. These transients have become known as *glitches*, a term that derives from the German *glitsche*, meaning a “slip” (hence, the slang, error or mishap). A glitch is a type of *logic noise* that is undesirable because its presence in an output may initiate an unwanted process in a next-stage switching device to which that output is an input. In some circuits glitches can be avoided through good design practices; in other circuits they are unavoidable and must be dealt with accordingly.

There are three kinds of logic noise that occur in combinational logic circuits and that are classified as *hazards*.

### Static hazards:

*Static 1-hazard* (also called *SOP hazard*) — A glitch that occurs in an otherwise steady-state 1 output signal from SOP logic because of a change in an input for which there are two asymmetric paths (delay-wise) to the output.

*Static 0-hazard* (also called *POS hazard*) — A glitch that occurs in an otherwise steady-state 0 output signal from POS logic because of a change in an input for which there are two asymmetric paths (delay-wise) to the output.





**Dynamic hazards:** Multiple glitches that occur in the outputs from multilevel circuits because of a change in an input for which there are three or more asymmetric paths (delay-wise) of that input to the output.



**Function hazards:** A type of logic noise that is produced when two or more inputs to a gate are caused to change in close proximity to each other.

In this chapter the discussion will center on how these hazards occur and how they can be avoided or eliminated. Since the subject of hazards is also of considerable importance to sequential machine design, it will be revisited in subsequent chapters.

## 9.2 STATIC HAZARDS IN TWO-LEVEL COMBINATIONAL LOGIC CIRCUITS

A single glitch that is produced as a result of an asymmetric path delay through an inverter (or gate) is called a *static hazard*. The term “static” is used to indicate that the hazard appears in an otherwise steady-state output signal. Thus, a static hazard is not “stationary” or “motionless,” as implied by the usual usage of the word static, but is quite unstationary and transient.

The best way to introduce static hazard detection and elimination in combinational logic is by means of simple examples. However, before proceeding further it will be helpful to define certain terms that are used in identifying static hazards in SOP or POS combinational logic circuits, and to provide a simple procedure for their elimination:

**Coupled variable:** An input variable that is complemented in one term of an output expression and uncomplemented in another term.

**Coupled term:** One of two terms containing *only one* coupled variable.

**Residue:** That part of a coupled term that remains after removing the coupled variable.

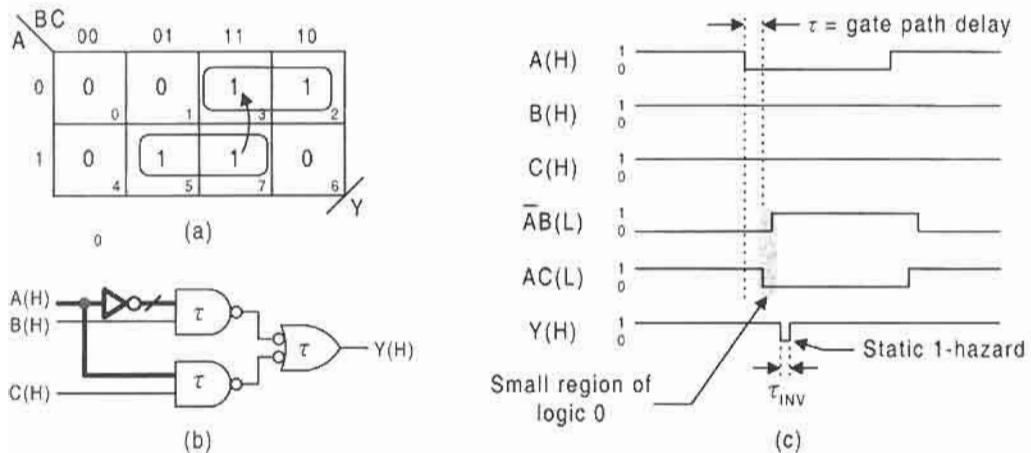
**Hazard cover (or consensus term):** The RPI required to eliminate the static hazard:

AND the residues of coupled p-term to obtain the SOP hazard cover, or

OR the residues of coupled s-terms to obtain the POS hazard cover.

Note that in either case the RPI (redundant prime implicant) is a result of the application of a consensus law given by Eqs. (3.14).

**Static Hazard Detection and Elimination** Static hazard detection involves identifying the coupled terms in an logic expression. Static hazard elimination occurs when the consensus p-term RPI is ORed to the SOP expression containing the static hazard, or when the consensus s-term RPI is ANDed to the POS expression containing the static hazard. Note that if the RPI is contained in a more minimum term, that term should be used.

**FIGURE 9.1**

A static 1-hazard produced by a  $1 \rightarrow 0$  change in input A. (a) K-map for Equation (9.1) showing loops for coupled terms and transition from state 111 to state 011. (b) Logic circuit for Equation (9.1) showing asymmetric paths for A. (c) Timing diagram for the circuit in (b) illustrating the production of the static 1-hazard after two gate path delays ( $2\tau$ ) following the change in A.

Consider the SOP function given by

$$Y = \bar{A}B + AC. \quad (9.1)$$

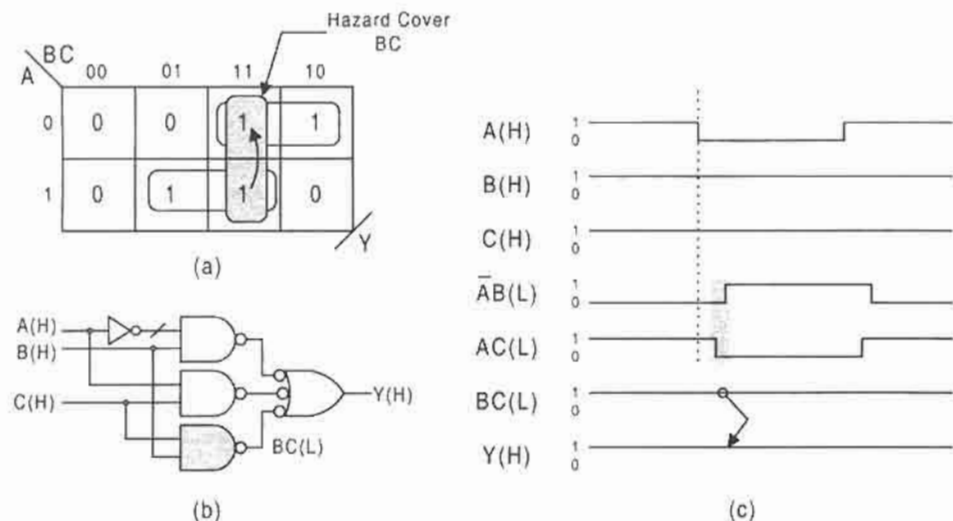
$\begin{array}{c} \uparrow \quad \quad \downarrow \\ 011 \quad 111 \end{array}$

Here, A is the coupled variable,  $\bar{A}B$  and AC are the coupled terms, and the hazardous transition read in minterm code is  $111 \rightarrow 011$  as indicated by the coupled terms in Eq. (9.1) and by the K-map in Fig. 9.1a. The logic circuit for Eq. (9.1) is given in Fig. 9.1b, where the two asymmetric paths of input A to the output are indicated by the heavier lines. With all inputs active high, the transition  $111 \rightarrow 011$  produces a static 1-hazard after a  $1 \rightarrow 0$  change in input A, as illustrated by the logic timing diagram in Fig. 9.1c. Thus, when the coupled terms are ORed, a small region of logic 0 (shaded region) creates the SOP hazard of magnitude equal to that through the inverter. The path delay through a NAND gate is designated by  $\tau$  with input leads assumed to be ideal with zero path delay.

The ANDed residues of Eq. (9.1) is the RPI BC. When this is added to Eq. (9.1) there results

$$Y = \bar{A}B + AC + \underbrace{BC}_{\text{Hazard cover}}, \quad (9.2)$$

which eliminates the hazard. This is demonstrated by the K-map, logic circuit, and timing diagram in Fig. 9.2. Notice that the hazard cover BC in the K-map of Fig. 9.2a is an RPI and that it covers the hazardous transition  $111 \rightarrow 011$  indicated by the arrow. When this RPI is added to the original expression, as in Eq. (9.2), the result is the logic circuit in

**FIGURE 9.2**

Elimination of the static 1-hazard by adding hazard cover. (a) K-map showing RPI that covers the hazardous transition 111 → 011. (b) Logic circuit that includes the shaded hazard cover gate BC(L). (c) Timing diagram showing the elimination of the static hazard in Fig. 9.1 due to presence of the hazard cover term BC(L).

Fig. 9.2b, which completely eliminates the static 1-hazard as illustrated in Fig. 9.2c. In fact, the hazard is eliminated regardless of the magnitude of the delay through the inverter — the hazard cannot form even if, for example, the delay is a ridiculous thousand times that of a NAND gate. It is also true that the results shown in Fig. 9.2 are valid if AND/OR/INV logic replaces the NAND/INV logic shown. In this case the coupled terms and hazard cover RPI would all be active high, but the waveforms would remain the same, the benefit of mixed-logic notation.

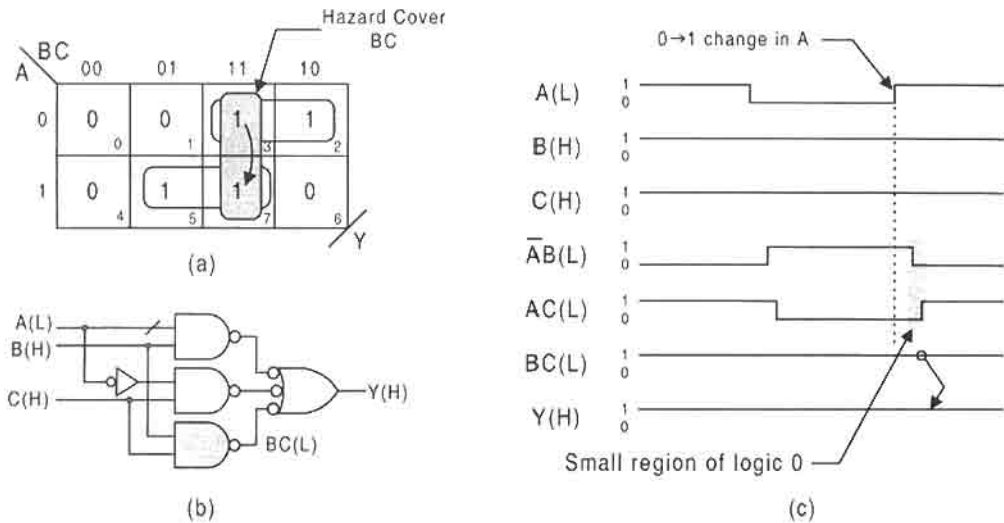
There is the question of how the activation level of the coupled variable  $A$  affects the results illustrated in Figs. 9.1 and 9.2. If  $A$  arrives active low,  $A(L)$ , then the inverter must be placed on the  $A$  line to the  $AC$  gate. The static 1-hazard is still formed, but as a result of a 011 → 111 transition following a 0 → 1 change in input  $A$ . This is illustrated in Fig. 9.3 for purposes of comparison with Figs. 9.1 and 9.2. Nevertheless, the static 1-hazard is eliminated by the hazard cover  $BC$  as shown in Fig. 9.3. Again, replacing the NAND/INV logic by AND/OR/INV logic would not alter the waveforms but would change the activation levels of the coupled terms and hazard cover to active high.

The forgoing discussion dealt with static hazards in SOP logic. The detection and elimination of static 0-hazards in POS combinational logic follows in similar but dual fashion. Consider the function

$$Y = (A + B)(\bar{A} + C), \quad (9.3)$$

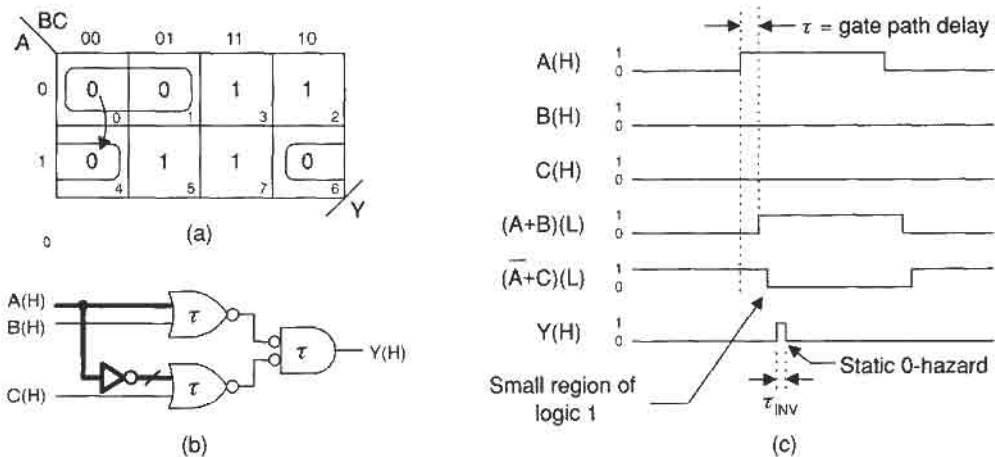
$\begin{array}{ccc} & \text{000} & \text{100} \\ & \downarrow & \uparrow \end{array}$

where  $A$  is again the coupled variable but now  $(A + B)$  and  $(\bar{A} + C)$  are the coupled terms.

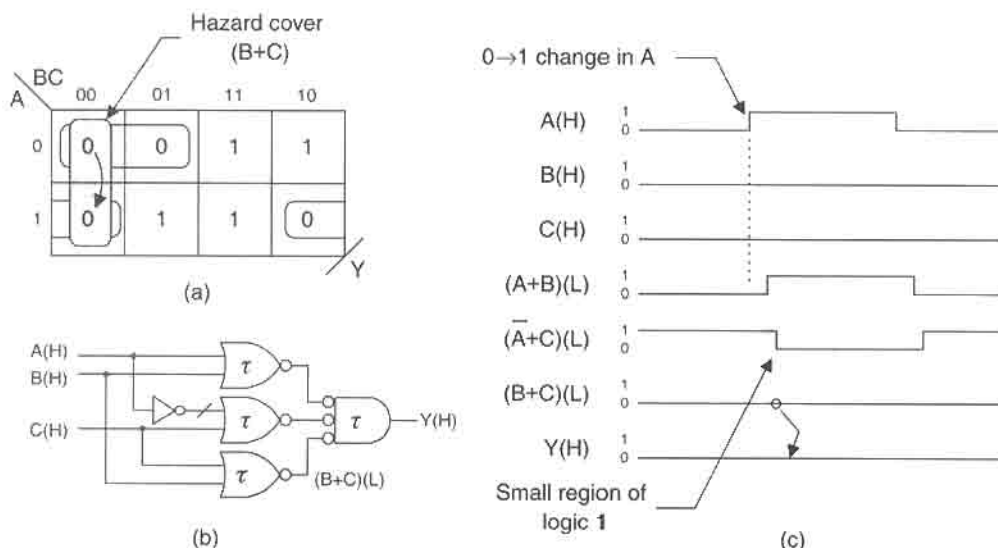
**FIGURE 9.3**

Elimination of the static 1-hazard for the case of an active low coupled variable. (a) K-map showing RPI that covers the hazardous transition  $011 \rightarrow 111$ . (b) Logic circuit that includes the shaded hazard cover gate  $BC(L)$ . (c) Timing diagram showing the elimination of the static 1-hazard due to presence of the hazard cover term  $BC(L)$ .

Read in maxterm code, assuming NOR/INV logic and active high inputs, the hazardous transition is  $000 \rightarrow 100$  following a  $0 \rightarrow 1$  change in  $A$  as shown in Fig. 9.4a. The logic circuit for Eq. (9.3) is given in Fig. 9.4b, where the two asymmetric paths for input  $A$  to the output are indicated by heavy lines. The static 0-hazard is formed as a result of the two

**FIGURE 9.4**

A static 0-hazard produced by a  $0 \rightarrow 1$  change in input  $A$ . (a) K-map for Eq. (9.3) showing loops for coupled terms and transition from state 000 to state 100. (b) Logic circuit for Eq. (9.3) showing asymmetric paths for  $A$ . (c) Timing diagram for the circuit in (b) illustrating the production of the static 0-hazard after two gate path delays  $2\tau$  following the change in  $A$ .

**FIGURE 9.5**

Elimination of the static-0 hazard by adding hazard cover. (a) K-map showing RPI that covers the hazardous transition  $000 \rightarrow 100$ . (b) Logic circuit that includes the shaded hazard cover gate  $(B + C)(L)$ . (c) Timing diagram showing the elimination of the static 0-hazard due to presence of the hazard cover term  $(B + C)(L)$ .

asymmetric paths (delay-wise) such that a small region of logic 1 exists when the waveforms for the coupled terms are ANDed as illustrated in Fig. 9.4c.

The static 0-hazard shown in Fig. 9.4c is eliminated by adding hazard cover to the function of Eq. (9.3). The ORed residues for function  $Y$  in Eq. (9.3) is  $(B + C)$ . When this is added (ANDed) to Eq. (9.3), there results

$$Y = (A + B)(\bar{A} + C) \cdot \underbrace{(B + C)}_{\text{Hazard cover}}, \quad (9.4)$$

which eliminates the static 0-hazard as illustrated in Fig. 9.5. Notice that  $(B + C)$  covers the transition  $000 \rightarrow 100$  as indicated by the arrow in the K-map of Fig. 9.5a, and that it is by definition an RPI. The logic circuit for Eq. (9.4) is given in Fig. 9.5b and is now hazard-free as illustrated in Fig. 9.5c. Here, it is seen that the small region of logic 1 that caused the static 0-hazard in Fig. 9.4 has been rendered ineffectual because of the ANDed RPI  $(B + C)(L)$ , which remains at 0(L) during the hazardous transition. Note that if only input  $A$  arrives active low  $A(L)$ , the static 0-hazard still forms, but as result of a  $100 \rightarrow 000$  transition following a  $1 \rightarrow 0$  change in  $A$ . Changing from NOR/INV logic to OR/AND/INV logic in Figs. 9.4 and 9.5 does not alter any of the conclusions drawn to this point. However, the activation levels of the coupled terms and hazard cover must be changed to active high in Fig. 9.5c, but leaving their waveforms unaltered.

Detection and elimination of static hazards in two-level combinational logic circuits is actually much simpler than would seem evident from the foregoing discussion. Actually, all that is necessary is to follow the simple procedure given next.

### Procedure for Detection and Elimination of Static Hazards in Combinational Logic Circuits

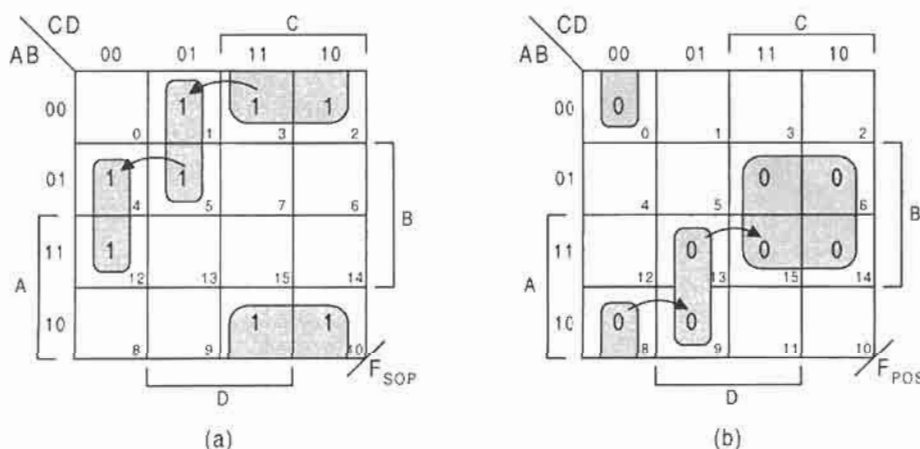
1. Identify the couple terms in an SOP or POS expression.
2. OR their consensus (RPI) P-term to the SOP expression, or AND their consensus s-terms to the POS expression.
3. Reject any set of two terms containing more than one couple variable. Remember: *Only one variable is allowed to change in a static hazard transition. All other variables must be constant.*
4. Read the initial and final states from the coupled terms in a hazardous transition by using minterm code for SOP and maxterm code for POS.

As an example, consider the following minimum SOP function of four variables showing two hazard transitions together with the corresponding hazard cover for each:

$$F_{SOP} = B\bar{C}\bar{D} + \bar{A}\bar{C}D + \bar{B}C + \underbrace{\bar{A}\bar{B}D + \bar{A}\bar{B}\bar{C}}_{\text{Hazard cover}} \quad (9.5)$$

$\begin{array}{ccc} & 0001 & 0011 \\ & \downarrow & \downarrow \\ & 0100 & 0101 \end{array}$

In this expression, NAND/INV logic is assumed with inputs that arrive active high. Here, the hazard cover  $\bar{A}\bar{B}\bar{C}$  is the ANDed residues (consensus p-term) of coupled terms  $B\bar{C}\bar{D}$  and  $\bar{A}\bar{C}D$ , where  $D$  is the coupled variable. And hazard cover  $\bar{A}\bar{B}D$  is the ANDed residues of couple terms  $\bar{A}\bar{C}D$  and  $\bar{B}C$ , where  $C$  is the coupled variable. These static 1-hazard transitions are illustrated in the K-map of Fig. 9.6a by using arrows indicating a  $1 \rightarrow 0$  change in the couple variable. The consensus terms can be seen to cover the hazard transitions (arrows).



**FIGURE 9.6**

K-map representation of static hazards in the function  $F$  represented by Eq. (9.5) and by Eq. (9.6). (a) Static 1 hazard transitions. (b) Static 0 hazard transitions.

Terms  $B\bar{C}\bar{D}$  and  $\bar{B}C$  cannot be coupled terms and cannot produce a static hazard, since they contain more than one couple variable—they have no consensus term. These terms form what is called a *function hazard*, a subject that is discussed in a later section.

The procedure for detection and elimination of static 0-hazards in POS logic is the dual of that for the detection and elimination of static 1 hazards in SOP logic. As an example, consider the minimum POS representation of the function  $F$  together with the hazard transitions and hazard cover given in Eq. (9.6):

$$F_{POS} = (B + C + D)(\bar{A} + C + \bar{D})(\bar{B} + \bar{C}) \cdot \underbrace{(\bar{A} + B + C)(\bar{A} + \bar{B} + \bar{D})}_{\text{Hazard cover}}. \quad (9.6)$$

Here again, inputs  $C$  and  $D$  are the coupled variables where the inputs are assumed to arrive active high but with NOR/INV logic. Notice that the initial and final states are read from the coupled terms by using maxterm code and are exactly those shown by arrows in Fig. 9.6b, indicating a  $0 \rightarrow 1$  change in the coupled variable. The hazard covers are the ORed residues (consensus s-terms) of the coupled terms and are ANDed to the original expression in Eq. (9.6). Note also that the s-terms  $(B + C + D)$  and  $(\bar{B} + \bar{C})$  are not coupled terms and cannot produce a static 0-hazard since they contain two coupled variables—remember, only one variable is allowed to change in the production of a static hazard. Also, if a coupled variable arrives active low with all other inputs active high, then the arrow indicating the hazard transition for that coupled variable must be reversed in Eq. (9.6).

As a final example, consider the function  $Z$  of five variables,

$$Z = \bar{B}\bar{C}D + \bar{C}\bar{D}\bar{E} + \bar{A}\bar{D}\bar{E} + ABE + \bar{A}\bar{B}C + \underbrace{\bar{A}\bar{B}\bar{C}\bar{E} + ABC\bar{D}}_{\text{Hazard cover}}. \quad (9.7)$$

where the coupled variables are easily seen to be  $D$  and  $E$ . Assuming NAND/INV or AND/OR/INV logic and that all inputs arrive active high, the two hazard transitions are indicated by arrows in Eq. (9.7). Thus, read in minterm code, coupled terms  $\bar{B}\bar{C}D$  and  $\bar{A}\bar{D}\bar{E}$  produce a static 1-hazard during the transition  $01010 \rightarrow 01000$  following a  $1 \rightarrow 0$  change in  $D$ , while terms  $ABE$  and  $\bar{C}\bar{D}\bar{E}$  generate a static 1-hazard during the transition  $11101 \rightarrow 11100$  following a  $1 \rightarrow 0$  change in  $E$ . Adding the ANDed residues of the coupled terms for each hazard transition gives the hazard-free expression in Eq. (9.7).

It should be clear to the reader that the detection and elimination of static hazards can be conducted without the aid of K-maps or logic circuits simply by following the four steps given previously. Exceptions to this rule are discussed in the next section, where diagrams of a rather different sort are used to simplify the process of identifying and eliminating static hazards in multilevel XOR-type functions.

### 9.3 DETECTION AND ELIMINATION HAZARDS IN MULTILEVEL XOR-TYPE FUNCTIONS

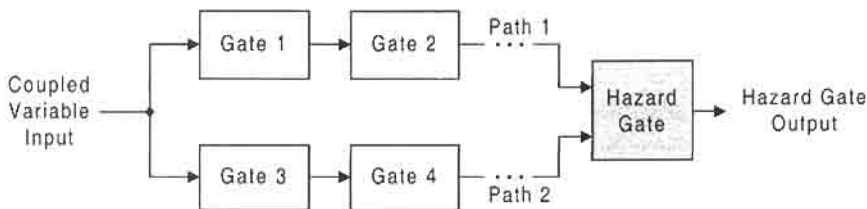
Conventional static hazard analysis used in two-level logic does not address the problem of hazards in multilevel XOR-type functions. This section presents a simple but general procedure for the detection and elimination of static hazards in these functions. It is shown that all static hazards can be eliminated with redundant cover derived by using a method based on lumped path delay diagrams (LPDDs), and that this method is of unrestricted applicability. The problems associated with dynamic hazards, as they relate to static hazard cover, are also considered.

Multilevel XOR-type functions of the type considered in this section find use in arithmetic circuits, such as dedicated arithmetic logic units, and in error detection circuits. These functions can be obtained from logic synthesis algorithms or from K-map extraction as was demonstrated in Chapter 5. If steady, clean outputs from these functions are required, it is necessary to remove any logic (hazard) noise that may be present.

Modern CMOS IC technology has produced XOR and EQV gates whose speed and compactness are close to those of other two-input gates (see, e.g., Figs. 3.26 and 3.27). This has made the use of XOR and EQV gates more practical and has led to the development of various methods of multilevel function implementation that take advantage of these gates. These implementations can produce gate-minimum results not possible with two-level logic. When fan-in restrictions on two-level implementations are considered, multilevel implementations become even more attractive.

The simpler multilevel functions include the XOR/SOP and EQV/POS forms. The XOR/SOP form connects p-terms with OR and XOR operators, while the EQV/POS form connects s-terms with AND and EQV operators. XOP and EOS forms are special cases of XOR/SOP and EQV/POS, respectively, and are considered to be special two-level forms of representation. The XOP form connects p-terms only with XOR operators, and the EOS form connects s-terms only with EQV operators. Multilevel functions more complex than XOR/SOP and EQV/POS are classified simply as compound multilevel forms for lack of a practical classification scheme.

As was stated in Section 9.2, a static hazard is a glitch in an otherwise steady-state output signal and is produced by two asymmetric paths from a single input. Figure 9.7 is a generalization of the condition that allows the static hazard to form in multilevel circuits. The coupled variable must be an input to the initial gate in each path (Gates 1 and 3 in



**FIGURE 9.7**

Alternative paths of the coupled variable to the hazard gate that is necessary for static hazard formation in a multilevel logic circuit.



Fig. 9.7). The signal must propagate through each path until both paths reach a single gate, called the *hazard gate*. If, for example, the signal requires more time to propagate through Path 1 than through Path 2, the signal from Path 2 will reach the hazard gate before the signal from Path 1. This asymmetry in path delay may cause a static hazard, depending on the nature of the hazard gate and the gates in the two paths. Here, an inverter is considered to be a gate and the hazard gate may or may not be the output gate. Also, lead delays are treated as ideal with negligible path delay, and each gate has an inherent delay associated with it that depends on its type, the number of inputs, and the technology used to implement it.

Static hazard analysis in multilevel functions is more complicated than in two-level logic. Nevertheless, the static hazard is formed in agreement with the requirements of Fig. 9.7. Each path may consist of any number of gates, and the gates in Path 1 may differ in several respects from those in Path 2. Thus, if

$$(\Sigma \text{ Path 1 delays}) \neq (\Sigma \text{ Path 2 delays}), \quad (9.8)$$

hazard formation is possible according to Fig. 9.7. Furthermore, in multilevel functions of the type considered in this section, the difference between the Path 1 and Path 2 delays may range from that of an inverter to one or more gate delays. Thus, the size (or strength) of the static hazard glitch in a multilevel logic circuit may be considerable. Whereas the static hazard glitch in two-level logic caused by an inverter may or may not cross the switching threshold, a static hazard glitch in a multilevel XOR-type circuit may be quite large and may easily cross the switching threshold.

### 9.3.1 XOP and EOS Functions

The simplest XOR/SOP or EQV/POS functions that produce static hazards are very similar to conventional two-level functions. If no more than one term in an SOP function can be active at any given time, the terms are mutually disjoint and the OR operators can be replaced with XOR operators as indicated by Corollary I in Subsection 3.1.1.1. The result is an XOP function. Hazards in an XOP function can be detected and eliminated by a method parallel to that described for SOP functions in Section 9.2.

As an example, consider the reduced function  $N$  in SOP and XOP form:

$$N_{SOP} = \bar{A}BC + ABD + \bar{A}\bar{B}CD + \bar{A}\bar{B}\bar{C}\bar{D} \quad (9.9)$$

$$N_{XOP} = \bar{A}BC \oplus ABD \oplus \bar{A}\bar{B}CD \oplus \bar{A}\bar{B}\bar{C}\bar{D}. \quad (9.10)$$

The p-terms are mutually disjoint, so the direct conversion from SOP to XOP is permitted. It follows from the conventional methods discussed in Section 9.2 that two static hazards will occur in  $N_{SOP}$  of Eq. (9.9): between coupled terms  $ABD$  and  $\bar{A}BC$  on a  $1111 \rightarrow 0111$  transition following a  $1 \rightarrow 0$  change in  $A$ , and between coupled terms  $ABD$  and  $\bar{A}\bar{B}CD$  on a  $(1111 \rightarrow 1011)$  transition following a  $1 \rightarrow 0$  change in  $B$ . Each hazard is caused by an inverter through which the coupled variable must pass. This inverter makes the two path delays (Fig. 9.7) unequal, allowing the hazards to form following a change in each coupled variable. Each hazard is eliminated by adding a consensus p-term consisting of the ANDed

residues, giving the result

$$N_{SOP} = \bar{A}BC + ABD + A\bar{B}CD + A\bar{B}\bar{C}\bar{D} + \underbrace{(BCD + ACD)}_{\text{Hazard cover}} \quad (9.11)$$

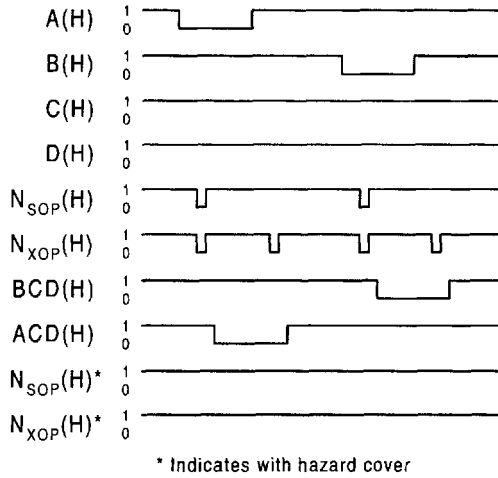
The method of hazard detection and elimination for  $N_{XOP}$  in Eq. (9.10) is quite similar to that of  $N_{SOP}$  in Eq. (9.11). However, a hazard can now occur in either direction between the coupled terms. Thus, for coupled terms  $ABD$  and  $\bar{A}BC$  a static 1-hazard is produced on either a  $1111 \rightarrow 0111$  or  $0111 \rightarrow 1111$  transition following a  $1 \rightarrow 0$  or  $0 \rightarrow 1$  change in couple variable  $A$ . Similarly, for coupled terms  $ABD$  and  $A\bar{B}CD$  a static 1-hazard can occur on either a  $1111 \rightarrow 1011$  or  $1011 \rightarrow 1111$  transition following a  $1 \rightarrow 0$  or  $0 \rightarrow 1$  change in coupled variable  $B$ . The complete details of these events are provided in Eq. (9.12), where the two hazards have been eliminated by the adding hazard cover. Note that each cover term is added to the function with an OR operator and not with an XOR operator. This is important because after hazard cover is added, the terms are no longer mutually disjoint. Therefore, adding the hazard cover by using an XOR operator would fundamentally alter the function.

$$N_{XOP} = \bar{A}BC \oplus ABD \oplus A\bar{B}CD \oplus A\bar{B}\bar{C}\bar{D} + \underbrace{(BCD + ACD)}_{\text{Hazard cover}} \quad (9.12)$$

The bidirectionality of the XOP hazard production in Eq. (9.12) is due to the nature of the XOR gate. Unlike an OR gate or an AND gate, an XOR gate will produce an output change with *any* single input change. Furthermore, if both inputs to an XOR gate change simultaneously, the final output will be the same as the initial output. Therefore, if the two inputs change at different times but in close proximity to one another, a short glitch will occur, regardless of the state transition involved or the order in which the inputs change. However, on a  $01 \rightarrow 10$  change in the two inputs to an OR gate, for example, the order in which the inputs change will determine whether or not a hazard occurs. This difference between XOR gates and OR gates is the reason a static hazard can be caused by a coupled variable change in either direction for an XOP function but in only one direction for an SOP function.

The timing diagram in Fig. 9.8 illustrates the behavior of  $N_{SOP}$  and  $N_{XOP}$  without and with hazard cover. At least one hazard cover signal is active during each hazard to prevent it from propagating to the final output signal. Note that hazards occur in  $N_{SOP}$  only on the falling edge of the coupled variable, but occur in  $N_{XOP}$  on both the rising and falling edge of the coupled variable.

A relationship similar to that between SOP and XOP functions exists between POS and EOS functions. If no more than one term in a POS function can be inactive at any given time, the terms are mutually conjoint and the AND operators can be replaced with EQV operators to form an EOS function. Hazards in the EOS function will be identical to those in POS, except that hazards in EOS will occur in both transitional directions. Hazard cover is formed by ORing the residues of the two coupled terms. An AND operator must be used to connect each hazard cover term to the original function.

**FIGURE 9.8**

Timing diagram for functions  $N_{SOP}$  and  $N_{XOP}$  without and with (\*) hazard cover in accordance with Eqs. (9.11) and (9.12).

As an example, consider the reduced POS and EOS forms of function  $L$ , which are the complements of function  $N$  in Eqs. (9.11) and (9.12), i.e.,  $L = \bar{N}$ . These POS and EOS functions are represented by the following expressions, together with the hazard transitions and hazard cover for each:

$$\begin{aligned}
 L_{POS} = & \underbrace{(A + \bar{B} + \bar{C})}_{0111} (\bar{A} + \bar{B} + \bar{D}) \underbrace{(\bar{A} + B + \bar{C} + \bar{D})}_{1111} (\bar{A} + B + C + D) \cdot \underbrace{(\bar{B} + \bar{C} + \bar{D})(\bar{A} + \bar{C} + \bar{D})}_{\text{Hazard cover}} \quad (9.13)
 \end{aligned}$$

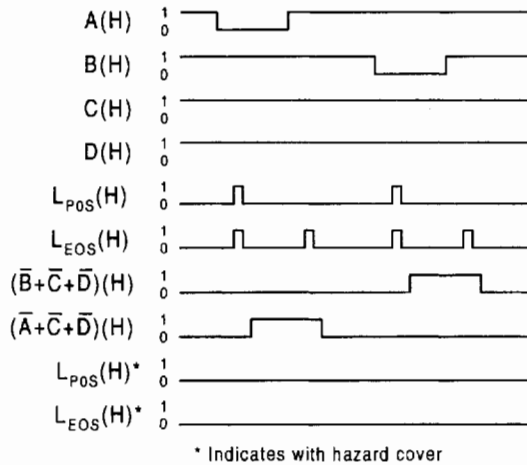
$\xrightarrow{1111} \quad \xleftarrow{1011}$

$$\begin{aligned}
 L_{EOS} = & (A + \bar{B} + \bar{C}) \odot (\bar{A} + \bar{B} + \bar{D}) \odot (\bar{A} + B + \bar{C} + \bar{D}) \odot (\bar{A} + B + C + D) \cdot \underbrace{(\bar{B} + \bar{C} + \bar{D})(\bar{A} + \bar{C} + \bar{D})}_{\text{Hazard cover}} \quad (9.14)
 \end{aligned}$$

$\xrightarrow{0111} \quad \xleftarrow{1111}$

The coupled variables are  $A$  and  $B$ , and the coupled terms and hazard transitions are indicated by arrows. The hazard cover terms for both the POS and EOS forms are  $(\bar{B} + \bar{C} + \bar{D})$  and  $(\bar{A} + \bar{C} + \bar{D})$ , each of which is the ORed residues of the respective coupled terms. Notice that in both cases the hazard cover terms are ANDed to the original expressions.

The timing diagram in Fig. 9.9 illustrates the behavior of  $L_{POS}$  and  $L_{EOS}$  without and with hazard cover. This behavior is similar to that of  $N_{SOP}$  and  $N_{XOP}$  in Fig. 9.8, but static

**FIGURE 9.9**

Timing diagram for functions  $L_{POS}$  and  $L_{EOS}$  without and with (\*) hazard cover in accordance with Eqs. (9.13) and (9.14).

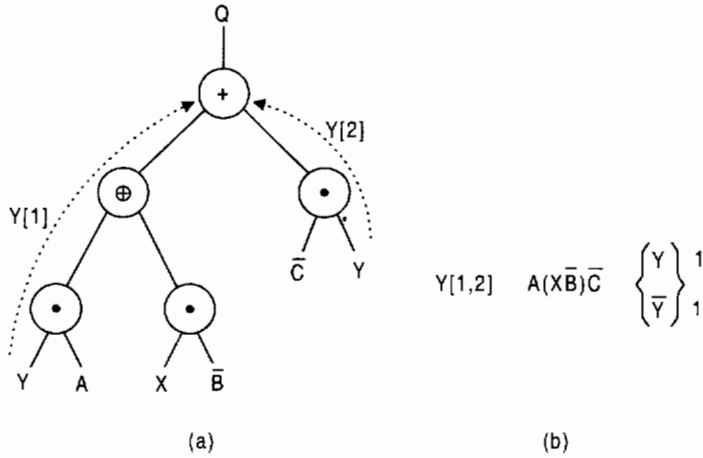
hazards are static 0 hazards in  $L_{POS}$  and  $L_{EOS}$  rather than the static 1 hazards as in  $N_{SOP}$  and  $N_{XOP}$ . Again notice that the static hazards in  $L_{EOS}$ , like those in  $N_{XOR}$ , are formed following both a  $0 \rightarrow 1$  and  $1 \rightarrow 0$  change in the coupled variable. It is this characteristic that distinguishes SOP and POS forms from XOP and EOS forms. The former types generate static hazards on a single change of the coupled variable, whereas the latter types generate static hazards on both  $1 \rightarrow 0$  and  $0 \rightarrow 1$  changes in the coupled variable.

### 9.3.2 Methods for the Detection and Elimination of Static Hazards in Complex Multilevel XOR-type Functions

Function minimization methods involving K-map XOR-type patterns and Reed–Muller transformation forms are considered in detail in Chapter 5. For certain functions these methods lead to gate-minimum forms that cannot be achieved by any other means. An inspection of these forms reveals that they are of the general form

$$F = (\alpha \oplus \beta) + \Gamma + \cdots, \quad (9.15)$$

where  $\alpha$ ,  $\beta$ , and  $\Gamma$  can be composed of SOP, POS, XOP, or EOS terms or some combination of these. The XOP and EOS functions discussed in Subsection 9.3.1 are a special case of Eq. (9.15), and the methods used there for the detection and elimination of static hazards parallel those used for two-level logic discussed in Section 9.2. However, these simple methods cannot be applied to the more complex functions considered in this subsection. Now, use must be made of special graphic methods to assist in the determination of path delay asymmetry according to Fig. 9.1.

**FIGURE 9.10**

Hazard detection and hazard cover for function  $Q$  in Eq. (9.15). (a) LPDD showing two paths for input  $Y$ . (b) Path requirements of input  $Y$  to produce an active  $Q$  output and the hazard cover for the static 1 hazard.

Consider the five-variable function

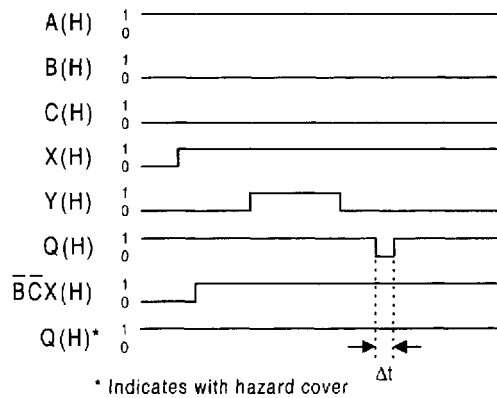
$$Q = (AY) \oplus (X\bar{B}) + \bar{C}Y, \quad (9.16)$$

which is a relatively simple form of Eq. (9.15). This function has a static 1-hazard for which the hazard cover cannot be readily identified by the standard methods used in Subsection 9.3.1. Other means must be used to identify the conditions for hazard formation and the cover that is necessary to eliminate it.

Shown in Fig. 9.10a is the *lumped path delay diagram* (LPDD) that is a graphical equivalent of the logic circuit for function  $Q$  in Eq. (9.16). Use of the LPDD makes possible a simple means of detecting and eliminating the static 1-hazard. However, some explanation of the use of this diagram is needed. The inputs are assumed to be active high, and inputs such as  $\bar{B}$  and  $\bar{C}$  imply the use of an inverter that is not shown. Two paths,  $Y[1]$  and  $Y[2]$ , for variable  $Y$  are shown from input to output  $Q$ . The path requirements for input  $Y$  and  $\bar{Y}$  that cause the output  $Q$  to be active are given in Fig. 9.10b. Thus, for  $Y$  inactive ( $\bar{Y}$ ), path  $Y[1]$  is enabled to cause  $Q = 1$  if both  $A$  and  $X\bar{B}$  are active. And for  $Y$  active, path  $Y[2]$  is enabled to cause  $Q = 1$  if  $\bar{C}$  active ( $C = 0$ ). The hazard cover is found by ANDing both enabling path requirements to give  $(AX\bar{B}\bar{C})$  as indicated in Fig. 9.10b. But for  $Y$  inactive, input  $A$  is irrelevant to path  $Y[1]$ . Thus, the final result for function  $Q$  is given by

$$\begin{aligned} Q &= (AY) \oplus (X\bar{B}) + \bar{C}Y + AX\bar{B}\bar{C} \\ &= (AY) \oplus (X\bar{B}) + \bar{C}Y + X\bar{B}\bar{C}. \end{aligned} \quad (9.17)$$

The timing diagram for function  $Q$  in Fig. 9.11 confirms the results presented in Fig. 9.10 and in Eq. (9.17). Notice that the hazard cover  $\bar{B}\bar{C}X$  removes the static 1-hazard as indicated by  $Q(H)^*$ . The size (strength) of the static hazard, indicated by  $\Delta t$ , is the difference in path

**FIGURE 9.11**

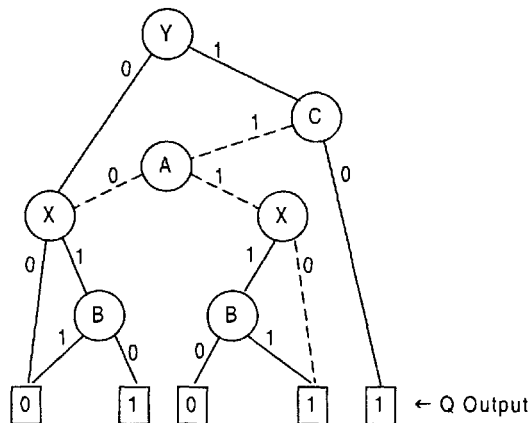
Timing diagram for functions  $Q$  without and with (\*) hazard cover in agreement with Eq. (9.17).

delays  $Y[1]$  and  $Y[2]$  in Fig. 9.10a as it relates to Fig. 9.1. This path delay asymmetry may be expressed as

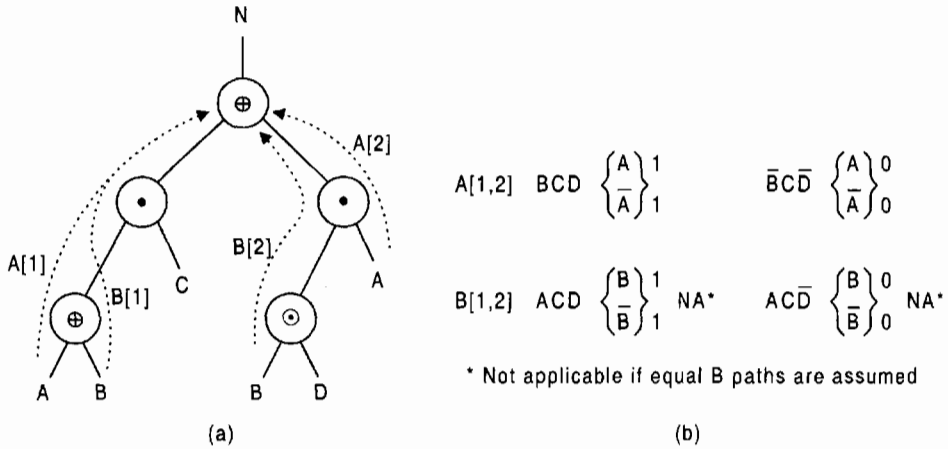
$$\Delta t = (t_{AND} + t_{XOR}) - (t_{AND}) = t_{XOR},$$

which is easily ascertained from an inspection of Fig. 9.10a.

That the hazard cover for Eq. (9.16) is  $\bar{B}\bar{C}X$  and is independent of input  $A$  can be easily demonstrated by the use of a binary decision diagram (BDD) constructed from an inspection of Eq. (9.16). This BDD is given in Fig. 9.12, where the binary decisions begin with variable  $Y$  and end with an output (for  $Q$ ) that is either logic 0 or logic 1. The requirement for static 1-hazard cover is that the decisions must lead to an active output  $Q$ . Thus, for  $Y = 0$  the path to  $Q = 1$  is enabled if  $X = 1$  and  $B = 0$  or  $X\bar{B}$  and is enabled for  $Y = 1$  if  $C = 0$

**FIGURE 9.12**

BDD for function  $Q$  in Eq. (9.16) showing the binary decisions required for an active output independent of input  $A$ .

**FIGURE 9.13**

Hazard detection and hazard cover for function  $N$  in Eq. (9.18). (a) LPDD showing two paths for input  $A$  and two paths for input  $B$ . (b) Path requirements for inputs  $A$  and  $B$  required to produce an active  $N$  output, and the hazard covers necessary to eliminate the static 1 hazards.

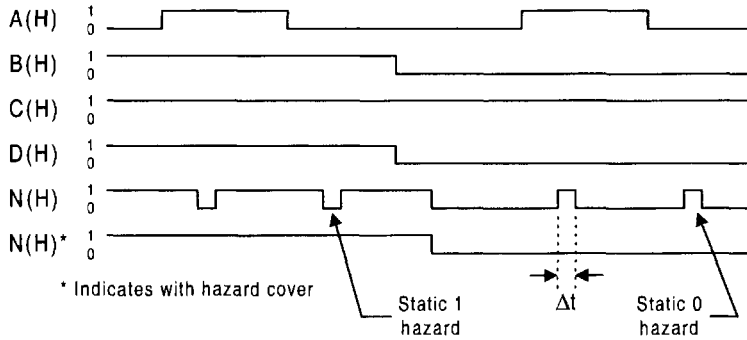
or  $\bar{C}$ , all read in positive logic. ANDing the two enabling conditions gives the hazard cover  $X\bar{B} \cdot \bar{C}$ . Clearly, the binary decisions involving input  $A$  (dashed lines) are irrelevant to the enabling path conditions for an active output  $Q$ .

A BDD is a graphical representation of a set of binary-valued decisions, each of which ends with a result that is either logic 1 or logic 0. Thus, the BDD allows one to easily determine the output for any possible combination of input values. The BDD is used by starting with an input variable (top of the BDD) and proceeding in a downward path to an output logic value that corresponds to the value of the last input in that path. Thus, the final element, usually presented in a rectangle, is the logic value of the function (e.g., output  $Q$  in Fig. 9.12) for the input values used.

As a second and more complex example, consider the four-variable function

$$N = [(A \oplus B)C] \oplus [(B \odot D)A], \quad (9.18)$$

which is a three-level form of Eqs. (9.9) and (9.10) obtained by the CRMT method discussed in Section 5.7 taking  $\{A, C\}$  as the *bond set*. The LPDD, shown in Fig. 9.13, indicates that there is at least one static 1-hazard and one static 0-hazard associated with this function. Input  $A$  has two paths to the output  $Z$ ,  $A[1]$  and  $A[2]$ . Path  $A[1]$  to output  $N$  is enabled if  $C$  is active, with  $B$  active or inactive depending on input  $A$ . Path  $A[2]$  to output  $N$  is enabled if  $B$  and  $D$  are logically equivalent (via the EQV operator). Therefore, for input conditions  $BCD$ , output  $N$  is active for  $A = 1$  via path  $A[2]$  only and is active for  $A = 0$  via path  $A[1]$  only, indicating a static 1-hazard. However, for input conditions  $\bar{B}\bar{C}\bar{D}$  the output  $N$  is inactive for both  $A$  and  $\bar{A}$ : For  $A = 1$  the output  $N = 1 \oplus 1 = 0$ , and for  $A = 0$  the output  $N = 0 \oplus 0 = 0$ . When the output is inactive for both input conditions, a static 0-hazard is indicated. The result of this static hazard analysis is that  $BCD$  becomes the static 1-hazard cover that must be ORed to Eq. (9.18), and that  $\bar{B}\bar{C}\bar{D} = (B + \bar{C} + D)$  is the static 0-hazard cover that must be ANDed to Eq. (9.18). When hazard cover is added, the final expression

**FIGURE 9.14**

Timing diagram for function  $N$  in Eq. (9.18) showing static 1 and static 0 hazards and showing the result of adding hazard cover according to Eq. (9.19).

for  $N$  becomes

$$N = \{[(A \oplus B)C] \oplus [(B \odot D)A] + \underbrace{BCD}_{\text{Hazard cover}}\} \cdot (B + \bar{C} + D), \quad (9.19)$$

which is now a five-level function. Note that the order in which the hazard cover is added is immaterial. Thus, Eq. (9.19) could have been completed by first ANDing  $(B + \bar{C} + D)$  to the original expression followed by ORing  $BCD$  to the result.

The paths  $B[1]$  and  $B[2]$  are not expected to create static hazards, assuming that the XOR and EQV gates have nearly the same path delays. This is a good assumption if CMOS technology is used for their implementation as in Figs. 3.26 and 3.27. However, if the two  $B$  path delays are significantly asymmetric, then both static 1 and static 0 hazards would exist and would be eliminated by adding hazard covers in addition to those for the path  $A$  hazards (see Fig. 9.13b).

The timing diagram in Fig. 9.14 illustrates the results expressed by Eqs. (9.18) and (9.19) and by Fig. 9.13. The static 1-hazard occurs with changes in  $A$  under input conditions  $BCD$ , and static 0 hazards occur with changes in  $A$  under input conditions  $\bar{B}\bar{C}\bar{D}$ . But when  $BCD$  is ORed to the expression in Eq. (9.18), the static 1-hazard disappears. Similarly, when  $\bar{B}\bar{C}\bar{D} = (B + \bar{C} + D)$  is ANDed to function  $N$  in Eq. (9.18), the static 0 hazards disappear. Notice that the strength of either type of static hazard in Fig. 9.14 is the difference in delay between the two paths expressed by

$$\Delta t = (t_{XOR} + t_{AND}) - t_{AND} = t_{XOR},$$

where each hazard is initiated after a delay of  $(t_{XOR} + t_{AND})$  following a change in input  $A$ . This information is easily deduced from an inspection of Fig. 9.13.

The BDD for function  $N$  in Eq. (9.18) is given in Fig. 9.15. Once the coupled variables have been identified by the LPDD, the BDD can be used to obtain the hazard cover for both the static 1-hazard and static 0-hazard. An inspection of the binary decisions required to render  $N = 1$  indicate a path  $BC$  for input condition  $A = 0$  and a path  $BD$  for  $A = 1$ . When these two input paths are ANDed together the result is  $BCD$ , the enabling condition



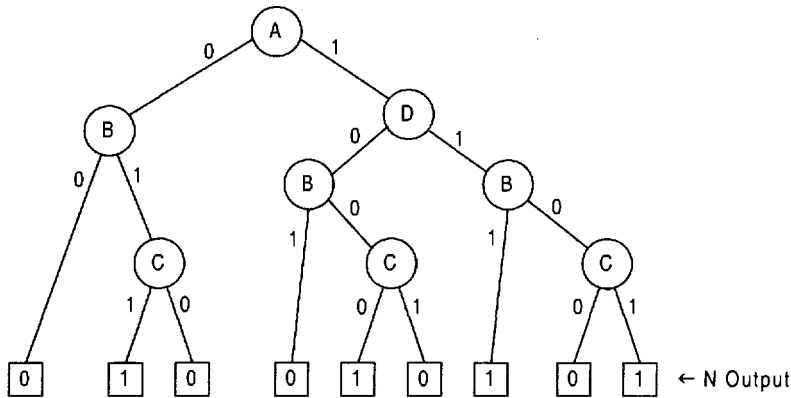


FIGURE 9.15

BDD for function  $N$  in Eq. (9.18) showing binary decisions required for static 1 and static 0 hazard formation.

for the static 1-hazard to form and, hence, also the hazard cover for that hazard. There are no other valid input paths for output  $N = 1$ , since their ANDing (intersection) is logic 0. In a similar fashion, the binary decisions required to produce  $N = 0$  indicate a path  $\bar{B}$  for  $A = 0$  and  $\bar{D}\bar{B}C$  for input condition  $A = 1$ . When these input path conditions are ANDed the result is  $\bar{B}C\bar{D}$ , which when complemented yields  $(B + \bar{C} + D)$ , the hazard cover for the static 0-hazard. All other ANDed input path combinations for  $A = 1$  result in logic 0 and hence are invalid.

BDDs can be very useful in identifying the hazard cover(s) for a given coupled variable, which is best identified by first by using an LPDD. The difficulty is not in the reading of the BDD to obtain the hazard cover, but in its construction. The reader should appreciate the fact that constructing of a BDD from a Boolean expression of the type considered in this section is no trivial task. In contrast, the LPDD, which is essentially a logic circuit, is easily constructed from the Boolean expression. For this reason, LPDDs should be used for most hazard analyses, reserving the use of BDDs for the difficult cases where the hazard cover is not easily revealed by an inspection of the LPDD.

### 9.3.3 General Procedure for the Detection and Elimination of Static Hazards in Complex Multilevel XOR-Type Functions

The static 1 and static 0 hazards in  $N$  were detected and eliminated by following a procedure that is applicable to any function. The procedure consists of the following three steps:

**Step I:** Use an LPDD to identify the two paths for each coupled variable whose path delays to the output differ according to Fig. 9.1. A determination of the path delays is not always a straightforward task, since the technology used for the individual gates may not be known. Worse yet, integrated circuits may make such determination nearly impossible without empirical data.

**Step II:** Find the hazard conditions and hazard cover for each coupled input variable in the LPDD by ANDing the variables that enable the two paths from the coupled variable to the output with those variables required to block (disable) all other paths. The gates

in the two enabling paths must not prevent the propagation of the coupled variable to the output stage. To accomplish this, other noncoupled variable inputs to AND operators must be active (logic 1), and other noncoupled variable inputs to OR operators must be inactive (logic 0). All other paths to the output stage must be blocked by proper selection of input activation levels. The use of a BDD is quite suitable for the purpose of finding hazard cover. Moreover, the BDD can also be used to simplify separately static 1-hazard cover and static 0-hazard cover. Note that static 1 and static 0 covers *must never* be simplified together.

For  $N_{XOP}$ , the LPDD (Fig. 9.13a) shows that  $C$  must be logic 1 so that the AND operator does not block path  $A[1]$ . The output from the EQV operator, which becomes an input to the AND operator in path  $A[2]$ , must also be logic 1. Inputs  $B$  and  $D$  must therefore be logically equivalent so as to enable path  $A[2]$  to the output. There are no other paths to consider. Thus, the hazard cover is  $BCD$  for the static 1-hazard and  $\overline{BCD} = (B + \bar{C} + \bar{D})$ .

**Step III:** Add the hazard cover to the original function by using an OR operator for static 1-hazard cover and by using an AND operator for a static 0-hazard cover. The 1-hazard cover and the 0-hazard cover must be added separately but may be added in either order. In Eq. (9.19) the static 1-hazard cover is added to  $N$  before the static 0-hazard cover. If the 0-hazard cover were added to Eq. (9.19) before the 1-hazard cover, the result would be

$$N = \{[(A \oplus B)C] \oplus [(B \odot D)A] \cdot \underbrace{(B + \bar{C} + \bar{D})}_{\text{Hazard cover}}\} + BCD \quad (9.20)$$

Eqs. (9.19) and (9.20) have gate/input tallies of 9/20, excluding inverters. There is often little advantage of one implementation over the other.

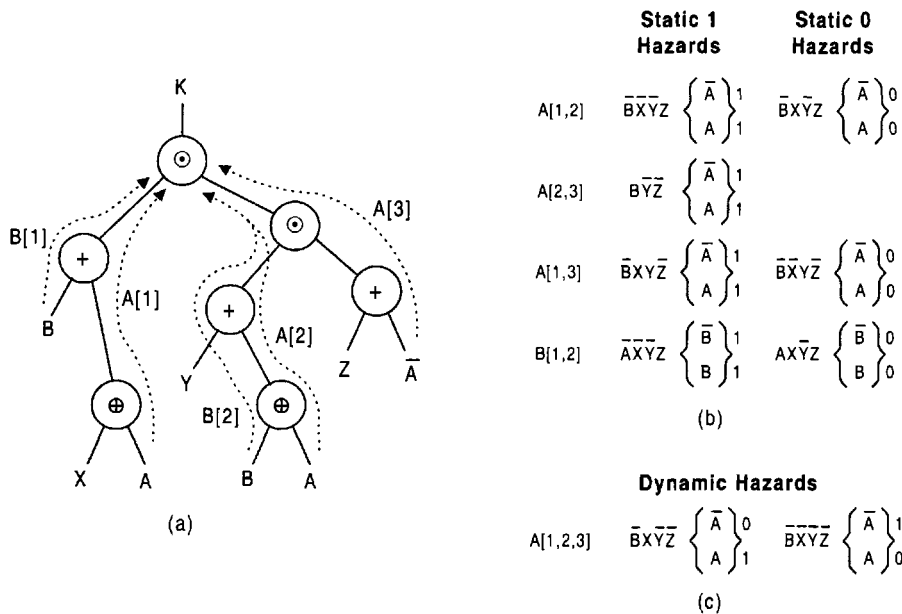
#### 9.3.4 Detection of Dynamic Hazards in Complex Multilevel XOR-Type Functions

Up to this point the discussion has centered around the detection and elimination of static hazards in multilevel XOR-type functions. As it turns out these functions also have a propensity to form dynamic hazards, and there may or may not be a means of eliminating these defects. One example is the five-variable, four-level function given by

$$K = [B + (A \oplus X)] \odot \{[Y + (A \oplus B)] \odot [\bar{A} + Z]\}. \quad (9.21)$$

The LPDD for this function, presented in Fig. 9.16a, reveals both static and dynamic hazards as indicated in Figs. 9.16b and 9.16c. The enabling path conditions for inputs  $A$  and  $B$  required to produce the static 1 and static 0 hazards are found in the same manner as for function  $N$  in Fig. 9.13. Obviously, static hazard analysis is considerably more involved than the analysis for function  $N$ . Furthermore, the additional logic required to eliminate all static hazards in function  $K$  is considerable. To eliminate both static 1 and 0 hazards requires that four p-terms be ORed to function  $K$  in Eq. (9.21) and by the ORing of three s-terms to that result, as required by Fig. 9.16b. The order in which the ORed and ANDed terms are added is immaterial.

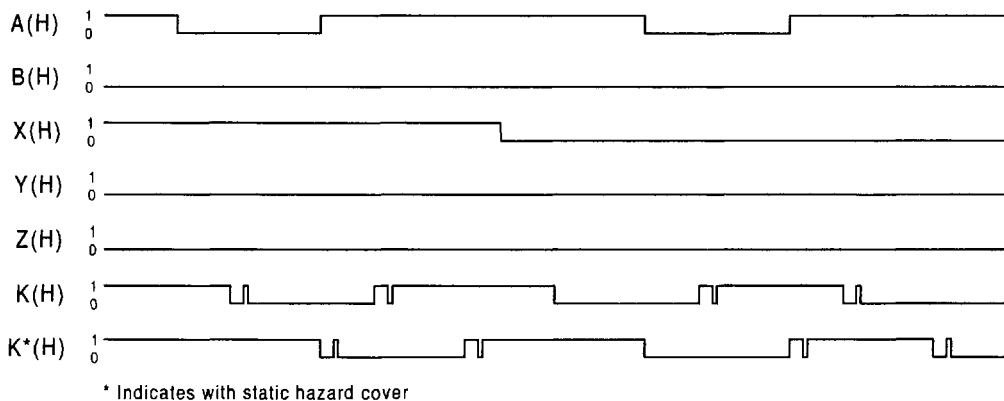
The input conditions required to produce dynamic hazards in function  $K$  are given in Fig. 9.16c. Remember: *Dynamic hazards can exist in a given function only if there are three or more paths of an input to the output.* This condition is satisfied for input  $A$  as indicated. Notice that a dynamic hazard is identified in an LPDD when the enabling paths

**FIGURE 9.16**

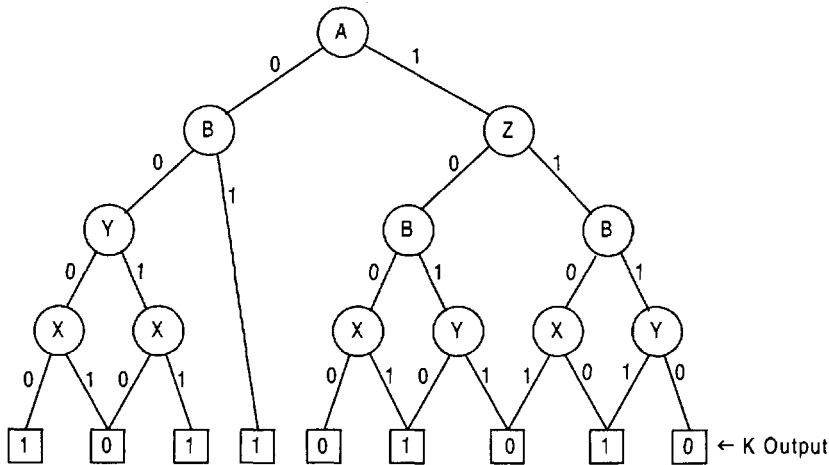
Hazard analysis of function  $K$  in Eq. (9.21). (a) LPDD showing three paths for input  $A$  and two paths for input  $B$ . (b) Path enabling requirements for  $A$  and  $B$  to produce static 1 and static 0 hazards. (c) Path  $A$  enabling requirements for the production of dynamic hazards in function  $K$ .

of the coupled variable to the output yield both a logic 1 and logic 0 as in Fig. 9.16c. This same information can be gleaned from a BDD, but, because of the difficulty in constructing the BDD, the LPDD approach is preferred.

The timing diagram for Eq. (9.21) is shown in Fig. 9.17, where dynamic hazards of the 1-0-1-0 and 0-1-0-1 types occur following  $1 \rightarrow 0$  and  $0 \rightarrow 1$  changes in coupled variable

**FIGURE 9.17**

Timing diagram for function  $K$  in Eq. (9.21), showing dynamic hazards produced without and with static hazard cover under input conditions given in Fig. 9.16c.

**FIGURE 9.18**

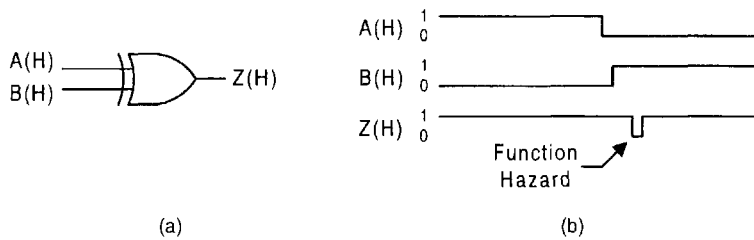
BDD for function  $K$  in Eq. (9.21) that can be used to show enabling paths of input  $A$  to output  $K$  as in Figs. 9.16b and 9.16c.

$A$ , respectively. Notice that the dynamic hazards continue to occur even after input  $X$  is changed from logic 1 to logic 0. This is predictable from Fig. 9.16c, since the enabling paths for input  $A$ ,  $\bar{B}\bar{X}\bar{Y}\bar{Z}$  and  $B\bar{X}\bar{Y}\bar{Z}$ , are satisfied in both cases. Input conditions other than these enabling paths for  $A$  would not allow production of the dynamic hazards. As indicated by the  $K^*$  waveform in Fig. 9.17, static hazard cover cannot be used to eliminate a dynamic hazard.

The enabling paths for input  $A$ , shown in Figs. 9.16b and 9.16c, can also be deduced from the BDD in Fig. 9.18 for function  $K$  in Eq. (9.21). However, somewhat greater effort is needed to obtain this information from the BDD owing to its nature. For example, the enabling paths of  $A$  required to produce the dynamic hazards are seen to be  $\bar{A}\bar{B}\bar{X}\bar{Y}\bar{Z}$  and  $A\bar{B}\bar{X}\bar{Y}\bar{Z}$ , yielding  $K = 0$  and  $K = 1$ , respectively, with active  $X$ . Similarly, for  $\bar{X}$  the enabling paths of  $A$  are observed to be  $\bar{A}\bar{B}\bar{X}\bar{Y}\bar{Z}$  and  $A\bar{B}\bar{X}\bar{Y}\bar{Z}$  for  $K = 1$  and  $K = 0$ , respectively. The static 1 and static 0 hazards due to coupled variable  $A$  are deduced from the BDD in a similar manner.

A few points need to be remembered when using LPDD and/or BDD graphical means to obtain the enabling paths of coupled variables.

- The LPDD should be used to identify the coupled variable and any asymmetry that may exist in the alternative paths.
- The LPDD or a BDD can be used to deduce the enabling paths for that coupled variable.
- A BDD must be constructed for each coupled variable, whereas a single LPDD can be used for all coupled variables.
- Both logic values of the coupled variable must be considered when using either the LPDD or BDD, but only for the LPDD must account be taken of blocked paths.

**FIGURE 9.19**

Demonstration of function hazard formation. (a) An XOR gate. (b) Timing diagram showing production of a function hazard when inputs  $A$  and  $B$  are changed in close proximity to each other.

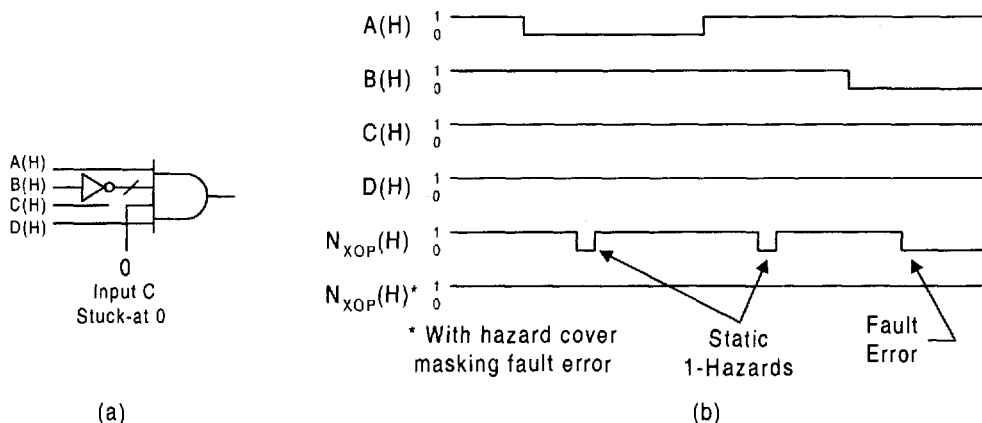
## 9.4 FUNCTION HAZARDS

In the expression for  $Z_{SOP}$  given by Eq. (9.7), it is observed that pairs of terms such as  $B\bar{C}D$  and  $C\bar{D}\bar{E}$  or  $\bar{A}\bar{D}\bar{E}$  and  $ABE$  each contain two couple variables. These pairs of terms are not coupled terms and cannot produce static hazards in the sense of Section 9.2. Also, their ANDed residues are always logic 0—as are the ORed residues logic 1 for pairs of s-terms containing two (or more) coupled variables in a POS expression. But these pairs of terms can produce another type of hazard called a *function hazard*, which is also static in the sense that it occurs in an otherwise steady-state signal. Function hazards result when an attempt is made to change two or more coupled variables in close proximity to each other. Potential hazards of this type are very common. In fact, any two (or more) input gate can produce a function hazard if the two inputs are caused to change in close proximity to each other. As an example, consider a simple XOR gate in Fig. 9.19a. If the two inputs are changed close together as shown in Fig. 9.19b, a function hazard results. In effect, function hazards in most circuits can be avoided if care is taken not to permit the inputs to change too close together in time.

## 9.5 STUCK-AT FAULTS AND THE EFFECT OF HAZARD COVER ON FAULT TESTABILITY

If, by some means, an input to a logic gate becomes permanently stuck at logic 0 or logic 1, a single *stuck-at fault* is said to exist. Inadvertent shorted connections, open connections, or connections to the voltage supply can take place during the manufacture of a given device such as a gate. When this happens the device fails to operate correctly. Models have been created to test specifically for stuck-at faults in various logic devices. One such model has become known as the *single stuck-at fault model* and is regarded as the simplest and most reliable model to use. Here, exactly one line, say to a gate, is assumed to be fixed at a logic 1 or logic 0 and, therefore, cannot respond to an input signal. Testing for such faults in a complex combinational logic circuit is often complicated and may involve the application of elaborate testing procedures, the subject of which is beyond the scope of this text. For the reader wishing more information on fault models, test sets, design testability, and related subject matter, references are given in Further Reading at the end of this chapter.

Because a single input change to an XOR or EQV operator produces an output change, multilevel functions containing these operators can be more easily tested than their

**FIGURE 9.20**

Effect of stuck-at fault on function  $N_{XOP}$  in Eq. (9.12). (a) AND representing the  $\bar{A}\bar{B}CD$  term and showing a stuck-at fault on input C. (b) Timing diagram showing effect of the stuck-at 0 fault and the masking effect of hazard cover.

two-level SOP or POS counterparts. This, of course, is one advantage in the use of XOP, EOS, and CRMT circuits discussed in Chapter 5. However, if static hazards must be eliminated in these circuits prior to fault testing, this advantage may be lessened or eliminated. Static hazard cover must always be redundant cover (i.e., not essential to function representation). Redundant cover can make stuck-at fault testing more difficult and may even mask an existing stuck-at fault. When considering the testability of a circuit, the designer must consider the effect of any static hazard cover needed.

As an example, consider function  $N_{XOP}$  in Eqs. (9.10) and (9.12) before and after the addition of static hazard cover. Suppose there is a stuck-at-0 fault at any input to the term  $\bar{A}\bar{B}CD$ . This fault causes an output error on the input condition 1011. However, the addition of hazard cover  $ACD$  holds the output active and masks the presence of this fault. Thus, after hazard cover is added, one cannot test for this fault by the standard methods of observing the final output. The timing diagram in Fig. 9.20 illustrates the masking effect of hazard cover in the  $N_{XOP}$  function. This timing diagram can be easily understood if it is recalled that an odd number of 1's in an XOR string such as that for function  $N_{XOP}$  in Eqs. (9.10) and (9.12) yields a logic 1 for that function. Consequently, introducing the change 1111  $\rightarrow$  1011 into these equations with and without the hazard cover  $ACD$  and with  $C = 0$  results in the timing diagram shown in Fig. 9.20b. In conclusion it can be stated that *fault detection and location test sets* should be used prior to the addition of hazard cover; if not, some stuck-at faults may not be detected and located.

## FURTHER READING

The subject of static hazards in two-level combinational logic circuits is covered adequately in texts by Breuer and Friedman, Katz, McCluskey, Tinder, Wakerly, and Yarbrough. Dynamic hazards and function hazards are also covered by McCluskey. However, there is no

known published information on static hazards in circuits of the XOR type considered in this chapter.

- [1] M. A. Breuer and A. D. Friedman, *Diagnosis and Reliable Design of Digital Systems*. Computer Science Press, 1976, pp. 10–13.
- [2] R. H. Katz, *Contemporary Logic Design*. Benjamin/Cummings Publishing Co., Redwood City, CA, 1992.
- [3] E. J. McCluskey, *Logic Design Principles*. Prentice-Hall, Englewood Cliffs, NJ, 1986.
- [4] R. F. Tinder, *Digital Engineering Design: A Modern Approach*. Prentice-Hall, Englewood Cliffs, NJ, 1991.
- [5] J. F. Wakerly, *Digital Design Principles and Practice*. Prentice-Hall, Englewood Cliffs, NJ, 1986.
- [6] J. M. Yarbrough, *Digital Logic*. West Publishing Co., Minneapolis/St. Paul, 1997.

A discussion of the construction and application of binary decision diagrams (BDDs) is limited to a few texts, among which are those of De Micheli and Sasao (Ed.). However, more extensive information is available from published journal articles. Typical of these are articles by Akers and Bryant.

- [7] S. Akers, "Binary Decision Diagrams," *IEEE Trans. on Computers*, **C-27**, 509–516 (1978).
- [8] R. Bryant, "Graph-based Algorithms for Boolean Function Manipulation," *IEEE Trans. on Computers* **C-35**(8), 677–691 (1986).
- [9] G. De Micheli, *Synthesis and Optimization of Digital Circuits*. McGraw-Hill, New York, 1994.
- [10] T. Sasao (Ed.), *Logic Synthesis and Optimization*. Kluwer Academic Publishers, Boston, 1993.

The subjects of fault detection and fault models are well covered by a number of texts and articles. For the beginning reader the text of Hayes does a commendable job. The text by McCluskey (previously cited) and that by Nelson, Nagle, Carroll, and Irwin are also recommended. For the advanced reader the texts by De Mecheli (previously cited) and Lala can be useful.

- [11] J. P. Hayes, *Introduction to Digital Design*. Addison-Wesley, Reading, MA, 1993.
- [12] P. K. Lala, *Fault Tolerant and Fault Testable Hardware Design*. Prentice-Hall, Englewood Cliffs, NJ, 1985.
- [13] V. P. Nelson, H. T. Nagle, B. D. Carroll, and J. D. Irwin, *Digital Logic Circuit Analysis and Design*. Prentice-Hall, Englewood Cliffs, NJ, 1995.

The following articles are noteworthy for their coverage of fault detection and testing, and of fault-tolerant systems:

- [14] A. Chatterjee and M. A. d'Abreu, "The Design of Fault-Tolerant Linear Digital State Variable Systems: Theory and Techniques," *IEEE Trans. on Computers* **42**(7), 794–808 (1993).
- [15] T. Lin and K. G. Shin, "An Optimal Retry Policy Based on Fault Classification," *IEEE Trans. on Computers* **43**(9), 1014–1025 (1994).
- [16] B. Vinnakota and N. K. Jha, "Diagnosability and Diagnosis of Algorithm-Based Fault-Tolerant Systems," *IEEE Trans. on Computers* **42**(8), 924–937 (1993).

## PROBLEMS

- 9.1** A function  $Y$  is represented in the K-map of Fig. P9.1. Refer to the examples in Section 9.2 and do the following:
- Loop out a minimum SOP and POS cover and then use arrows in separate K-maps to indicate the direction of the hazardous SOP and POS transitions that are present in this function.
  - Find the hazard covers and combine them with the minimum SOP and POS expressions. Also, show these covers on the appropriate K-maps.

| AB \ CD | CD |    |    |    |
|---------|----|----|----|----|
|         | 00 | 01 | 11 | 10 |
| 00      | 0  | 1  | 1  | 0  |
| 01      | 1  | 1  | 1  | 1  |
| 11      | 0  | 0  | 0  | 1  |
| 10      | 0  | 1  | 1  | 1  |

FIGURE P9.1

- 9.2** The following minimum SOP function contains both static 1-hazards and static 0-hazards:

$$F = \bar{A}\bar{B}\bar{C} + BD + AC\bar{D} + \bar{B}\bar{C}\bar{D}.$$

- Map and loop out this function in a fourth-order K-map and indicate the direction of each SOP hazardous transitions on the map by using an arrow. Follow the example in Fig. 9.6a.
  - Find the hazard covers and add them to the original expression above.
  - By following the example in Eq. (9.5), confirm that the same information can be obtained directly from the minimum SOP expression just given.
  - Repeat parts (a), (b), and (c) for the POS hazards (static 0-hazards) in the minimum POS expression for this function.
  - Use the gate/input tally (exclusive of possible inverters) to compare the SOP and POS expressions with hazard cover included. Which result is simpler?
- 9.3** The following function contains a single static 1-hazard:

$$F_{SOP} = \bar{A}\bar{B}\bar{C} + AC + CD.$$

- From this expression (without using a K-map), determine the coupled terms, the initial and final states of the hazardous transition, and the hazard cover to be added to the expression. To do this, follow the example in Eq. (9.5).



- (b) Use a timing diagram to show the development of this hazard, similar to the example in Fig. 9.1c. Then, by adding the hazard cover, show that the hazard is eliminated following the example in Fig. 9.2c. Assume that the inputs and output are all active high.
- (c) Construct the logic circuit for  $F_{SOP}$  and include the hazard cover.
- 9.4** Map the expression in Problem 9.3 and extract a minimum expression for  $F_{POS}$ . This function contains a single static 0-hazard.
- (a) From this expression (without using a K-map), determine the coupled variable, coupled terms, the initial and final states of the hazardous transition, and the hazard cover to be added to the expression. To do this, follow the example in Eq. (9.6).
- (b) Use a timing diagram to show the development of this hazard, similar to the example in Fig. 9.4c. Then by adding the hazard cover, show that the hazard is eliminated following the example in Fig. 9.5c. Assume that the inputs and output are all active high.
- (c) Construct the logic circuit for  $F_{POS}$  and include the hazard cover.
- 9.5** Each of the following minimum or reduced functions contains one or more static hazards. For each function (without using a K-map), determine the coupled variable, coupled terms, the initial and final states of the hazardous transition, and the hazard cover to be added.
- (a)  $W = \bar{A}BC\bar{D} + \bar{B}\bar{C}\bar{D} + A\bar{C}D$
- (b)  $R = (U + W + \bar{X})(\bar{U} + \bar{V} + \bar{W})(\bar{V} + X)$
- (c)  $G = W\bar{X}\bar{Y} + XYZ + WY\bar{Z} + \bar{W}\bar{Y}$
- (d)  $T = (\bar{A} + \bar{B} + \bar{C})(\bar{A} + B + C)(A + D)(B + \bar{D})$
- (e)  $Y = \bar{w}y\bar{z} + xz + \bar{x}\bar{y}$
- 9.6** A five-variable function  $Z$  is represented in the second-order K-map of Fig. P9.2. It contains a single SOP hazard (static 1-hazard).
- (a) Extract minimum SOP cover for this function and determine the coupled variable, coupled terms, the initial and final states of the hazardous transition, and the hazard cover to be added to the expression. To do this, follow the example in Eq. (9.5). (Hint: There are two possible coupled terms depending on how minimum cover is extracted.)
- (b) Use a timing diagram to show the development of this hazard, similar to the example in Fig. 9.1c. Assume that all inputs are active high. Then by adding the hazard cover, show that the hazard is eliminated following the example in Fig. 9.2c.

|   |   | B           |                |
|---|---|-------------|----------------|
|   |   | 0           | 1              |
| A | 0 | $CD\bar{E}$ | $E$            |
|   | 1 | $\bar{E}$   | $\bar{C}D + E$ |

FIGURE P9.2

- 9.7** Find the minimum POS expression for the function in Fig. P9.2. How many static 0-hazards does this function have? What do you conclude as to the relative complexity of the SOP and POS expressions for this function when account is taken of hazard cover?
- 9.8** The following six-variable function has several static 1-hazards. Construct a table listing the coupled variable, coupled terms, initial and final states, and the hazard cover for each of the hazards.

$$F = \bar{A}\bar{B}C\bar{D}\bar{F} + \bar{A}BCE + ABCF + \bar{A}\bar{B}C\bar{E} + \bar{D}\bar{E}F + C\bar{D}\bar{E}$$

- 9.9** The following multilevel functions have one or more static hazards. For each expression (without using a K-map), determine the coupled variable, coupled terms, the initial and final states of the hazardous transition (read in alphabetical order), and the hazard cover to be added to the expression. To do this, follow the examples in Eq. (9.12), (9.13), and (9.14), whichever is relevant to the particular function.
- (a)  $G = W\bar{X}\bar{Y} \oplus XYZ \oplus WY\bar{Z} \oplus \bar{W}\bar{Y}$
- (b)  $T = (\bar{A} + \bar{B} + \bar{C}) \odot (\bar{A} + B + C) \odot (A + D) \odot (\bar{B} + \bar{D})$
- 9.10** The following three-level XOR-type function has two static 1-hazards:

$$f = (Y \oplus W) \oplus (\bar{X}Z) + \bar{W}Y.$$

- (a) Construct the lumped path delay diagram (LPDD) for this function. From the LPDD determine the hazard cover and initial and final states for each of the static hazards. Follow the example in Fig. 9.10. (Hint: Keep in mind the bidirectionality of the static hazards in XOR-type functions and read the states for function  $f$  in the order of  $WXYZ$ ).
- (b) By using two binary decision diagrams (BDDs), show the binary decisions required for each static 1-hazard formation.
- 9.11** The following three-level function has both a static 1-hazard and a static 0-hazard:

$$F = [X \oplus \bar{W}Y] \oplus (WZ).$$

- (a) Construct the LPDD for this function (exactly as written). Then, determine the hazard cover and initial and final states for each of the static hazards. Read the states in alphabetical order. Follow the example in Fig. 9.13 and Eq. (9.19).
- (b) Use a timing diagram to show the development of the two hazards, similar to the example in Fig. 9.14. Then by adding the hazard cover, show that the hazards are eliminated. Assume that all inputs and the output are active high.
- (c) By using a binary decision diagram (BDD), show the binary decisions required for the static 1-hazard formation.
- 9.12** The following four-level function has three static 1-hazards, one static 0-hazard, and one dynamic hazard:

$$Y = [B \oplus (\bar{A}D)] \oplus [A\bar{B} \oplus AC\bar{D} \oplus B\bar{C}D]$$

- (a) Construct an LPDD for this function (exactly as written) and find the hazard cover for each of the static hazards and the conditions required to produce the dynamic hazard. Indicate the initial and final states for each of the static hazards. Follow the example in Fig. 9.16. (Hint: No dynamic hazard exists due to a change in either input  $A$  or  $D$ , and the one that does exist is conditional on  $A$ . Also, look for the possibility that a static hazard may change from a static 1-hazard to a static 0-hazard depending on the order of change of one or two variables.)
  - (b) Use a BDD to show the enabling paths of the variable whose change is responsible for the dynamic hazard formation. Follow the example in Fig. 9.18.
  - (c) Use a timing diagram (simulation) to show the development of the dynamic hazard. Demonstrate that the dynamic hazard cannot be eliminated by adding static hazard cover.
  - (d) Demonstrate with both an LPDD and a timing diagram that the dynamic hazard can be eliminated by making the  $B$  paths to the output less asymmetric.
- 9.13** At the discretion of the instructor, use the EXL-Sim2002 simulator included on the CD-ROM bundled with this text to simulate any of the following problems: 9.3b, 9.4b, 9.6b, 9.11b, 9.12c, 9.12d. For example, an assignment 9.13/9.6(b) requires the use of EXL-Sim2002 to verify the existence of the static 1-hazard and its hazard cover by simulating the five variable function  $Z$  in Fig. P9.2.

# Introduction to Synchronous State Machine Design and Analysis

---

## 10.1 INTRODUCTION

Up to this point only combinational logic machines have been considered, those whose outputs depend solely on the present state of the inputs. Adders, decoders, MUXs, PLAs, ALUs, and many other combinational logic machines are remarkable and very necessary machines in their own right to the field of logic design. However, they all suffer the same limitation. They cannot perform operations sequentially. A ROM, for example, cannot make use of its present input instructions to carry out a next-stage set of functions, and an adder cannot count sequentially without changing the inputs after each addition. In short, combinational logic devices lack *true memory*, and so lack the ability to perform sequential operations. Yet their presence in a sequential machine may be indispensable.

We deal with sequential devices all the time. In fact, our experience with such devices is so commonplace that we often take them for granted. For example, at one time or another we have all had the experience of being delayed by a modern four-way traffic control light system that is vehicle actuated with pedestrian overrides and the like. Once at the light we must wait for a certain sequence of events to take place before we are allowed to proceed. The controller for such a traffic light system is a fairly complex digital sequential machine.

Then there is the familiar elevator system for a multistory building. We may push the button to go down only to find that upward-bound stops have priority over our command. But once in the elevator and downward bound, we are likely to find the elevator stopping at floors preceding ours in sequence, again demonstrating a sequential priority. Added to these features are the usual safety and emergency overrides, and a motor control system that allows for the carrier to be accelerated or decelerated at some reasonable rate. Obviously, modern elevator systems are controlled by rather sophisticated sequential machines.

The list of sequential machines that touch our daily lives is vast and continuously growing. As examples, the cars we drive, the homes we live in, and our places of employment all use sequential machines of one type or another. Automobiles use digital sequential machines to control starting, braking, fuel injection, cruise control, and safety features. Most homes have automatic washing machines, microwave ovens, sophisticated audio and video devices of various types, and, of course, computers. Some homes have complex security, energy,

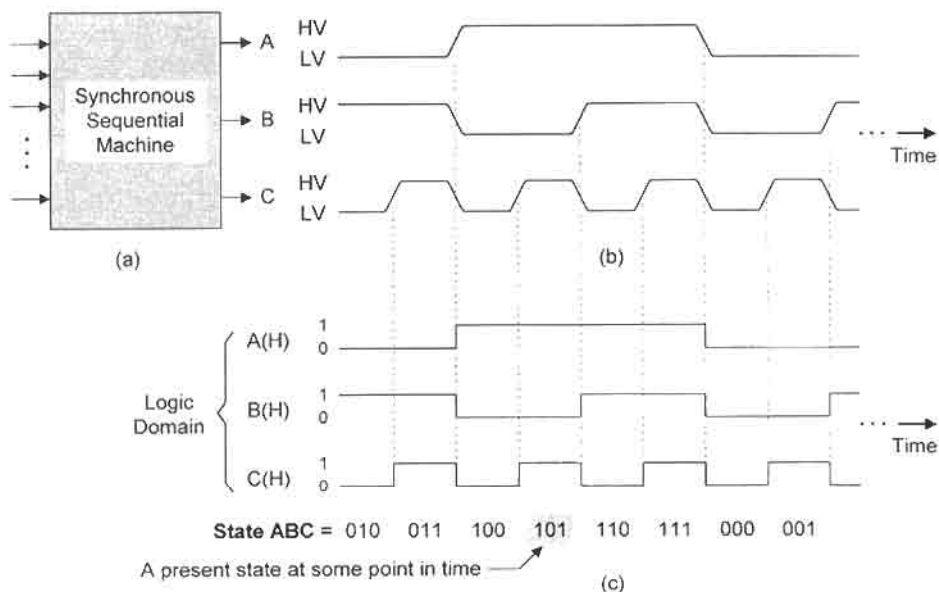
and climate control systems. All of these remarkable and now commonplace gifts of modern technology are made possible through the use of digital sequential machines.

The machines just mentioned are called *sequential machines*, or simply *state machines*, because they possess true memory and can issue time-dependent sequences of logic signals controlled by present and past input information. These sequential machines may also be *synchronous* because the data path is controlled by a *system clock*. In synchronous sequential machines, input data are introduced into the machine and are processed sequentially according to some algorithm, and outputs are generated — all regulated by a system clock. Sequential machines whose operation is clock independent (i.e., self-timed) are called *asynchronous sequential machines*, the subject of Chapters 14, 15, and 16.

Synchronous sequential machines and their design, analysis, and operation are the subjects covered in this chapter. Treatment begins with a discussion of the models used for these machines. This is followed by a discussion of an important type of graphic that is used to represent the sequential behavior of sequential machines and by a detailed development of the devices used for their memory. The chapter ends with the design and analysis of relatively simple state machines. The intricacies of design are numerous and require detailed consideration. For this reason they are discussed later, in Chapter 11.

### 10.1.1 A Sequence of Logic States

Consider that a synchronous sequential machine has been built by some means and that it is represented by the block symbol in Fig. 10.1a. Then suppose the voltage waveforms from its three outputs are detected (say with a waveform analyzer) and displayed as in



**FIGURE 10.1**

A sequence of logic events from a synchronous state machine. (a) Block diagram symbol and (b) output voltage waveforms. (c) Timing diagram representing the positive logic interpretation of the voltage waveforms and showing a sequence of logic states.

Fig. 10.1b. From these physical waveforms the positive logic waveforms are constructed and a sequence of logic states is read in the order *ABC* as shown in Fig. 10.1c. A group of logic waveforms such as these is commonly known as a *timing diagram*, and the sequence of logic states derived from these waveforms is seen to be a binary count sequence. Here, *A*, *B*, and *C* are called *state variables* because their values collectively define the *present state* of the machine at some point in time. Knowing the present state in a sequence of states also reveals the *next state*. Thus, in Fig. 10.1c, if state 101 is the present state, then 110 is the next state. This short discussion evokes the following definition:

A *logic state* is a unique set of binary values that characterize the logic status of a sequential machine at some point in time.

A sequential machine always has a finite number of states and is therefore called a *finite state machine (FSM)* or simply *state machine*. Thus, if there are  $N$  state variables, there can be no more than  $2^N$  states in the FSM and no fewer than 2. That is, for any FSM,

$$2 \leq (\text{number of states}) \leq 2^N$$

For example, a two-state FSM requires one state variable, a three- or four-state FSM requires two state variables, five- to eight-state FSMs require three state variables, etc. More state variables can be used for an FSM than are needed to satisfy the  $2^N$  requirement, but this is done only rarely to overcome certain design problems or limitations. The abbreviation FSM will be used frequently throughout the remainder of this text.

To help understand the meaning of the various models used in the description and design of FSMs, four binary sequences of states are given in Fig. 10.2, each presenting a different feature of the sequence. The simple ascending binary sequence (a) is the same as that in Fig. 10.1. This sequence and the remaining three will be described as they relate to the various models that are presented in the following section.

## 10.2 MODELS FOR SEQUENTIAL MACHINES

Models are important in the design of sequential machines because they permit the design process to be organized and standardized. The use of models also provides a means of communicating design information from one person to another. References can be made to specific parts of a design by using standard model nomenclature. In this section the general model for sequential machines will be developed beginning with the most elemental forms.

Notice that each state in the sequence of Fig. 10.2a becomes the *present state (PS)* at some point in time and has associated with it a *next state (NS)* that is predictable given the PS. Now the question is: What logic elements of an FSM are required to do what is required in Fig. 10.2a? To answer this question, consider the thinking process we use to carry out a sequence of events each day. Whether the sequence is the daily routine of getting up in the morning, eating breakfast, and going to work, or simply giving our telephone number to someone, we must be able to remember our present position in the sequence to know what the next step must be. It is no different for a sequential machine. There must be a memory section, as in Fig. 10.3a, which generates the present state. And there must be a *next state logic* section, as in Fig. 10.3b, which has been programmed to know what the next state must be, given the present state from the memory. Thus, an FSM conforming to the model